

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Inhaltsverzeichnis

1	Einführung	1-1
1.1	Zur Web Fundamentals Vorlesung	1-1
1.1.1	Organisatorisches	1-1
1.1.2	Inhalte der Vorlesung	1-1
1.2	Begriffsdefinitionen	1-2
1.2.1	Hypermedia	1-2
1.2.2	Hypertext	1-2
1.2.3	World Wide Web	1-2
1.2.4	HTML	1-2
1.2.5	URL und URI	1-3
1.2.6	HTTP	1-3
1.2.7	Website vs. Webpage vs. Homepage	1-3
1.3	Funktionsweise des World Wide Web	1-3
1.4	Webbrowser	1-4
1.4.1	Google Chrome	1-5
1.4.2	Microsoft Edge	1-5
1.4.3	Apple Safari	1-6
1.4.4	Mozilla Firefox	1-6
1.4.5	Opera	1-6
1.5	HTML – Erste Schritte	1-7
1.5.1	Was ist HTML?	1-7
1.5.2	HTML-Tags und Elemente	1-7
1.5.3	HTML-Attribute	1-10
1.5.4	Kommentare	1-10
1.5.5	Entitäten	1-11
1.5.6	HTML-Grundgerüst	1-11
1.5.7	Der DOCTYPE	1-12
1.6	Die Geschichte von HTML	1-13
1.7	Standardisierung im Web	1-15
1.7.1	Organisationen	1-15
1.7.2	Überprüfen von Standards	1-16
1.8	Weiterführende Literatur	1-16
2	HTML von A bis Z	2-1
2.1	Kopfdaten	2-1

2.1.1	Seitentitel	2-1
2.1.2	Basis-URL	2-1
2.1.3	Externe Verlinkungen und Style-Angaben	2-2
2.1.4	Meta-Angaben	2-2
2.2	Elemente zur Seitenstrukturierung	2-4
2.2.1	Inhaltsbereich	2-4
2.2.2	Kopfbereich	2-4
2.2.3	Fußbereich	2-5
2.2.4	Navigation	2-5
2.2.5	Abschnitte	2-5
2.2.6	Artikel	2-6
2.2.7	Nebenabschnitte	2-6
2.2.8	Überschriften	2-7
2.2.9	Überschriftengruppen	2-8
2.3	Elemente zur Textstrukturierung	2-8
2.3.1	Absätze	2-8
2.3.2	Adressen	2-8
2.3.3	Hauptinhalt	2-9
2.3.4	Suchbereich	2-9
2.3.5	Allgemeiner Bereich	2-9
2.3.6	Listen	2-10
2.3.7	Vorformatierter Text	2-11
2.3.8	Horizontale Linie	2-12
2.3.9	Zitate	2-12
2.4	Links und Elemente auf Textebene	2-13
2.4.1	Verweise	2-13
2.4.2	Weitere Elemente auf Textebene	2-15
2.5	Bilder	2-17
2.5.1	Bilder einfügen	2-17
2.5.2	Pixelweite und Auflösung berücksichtigen	2-18
2.5.3	Art Direction mit <code><picture></code>	2-19
2.5.4	Abbildungen	2-20
2.5.5	Lazy Loading von Bildern	2-22
2.6	Formulare	2-22
2.6.1	Ein Formular erstellen	2-22
2.6.2	Formularfelder beschriften	2-23
2.6.3	Eingabefelder	2-23
2.6.4	Buttons	2-26
2.6.5	Mehrzeilige Eingabefelder	2-27
2.6.6	Radio Buttons und Checkboxes	2-28
2.6.7	Auswahllisten	2-29
2.6.8	Zusatzangaben bei Formularelementen	2-31
2.6.9	Formulare strukturieren	2-32
2.6.10	Vollständiges HTML-Grundgerüst mit Formular	2-33
2.7	Weiterführende Literatur	2-34

3	CSS-Grundlagen	3-1
3.1	Einführung	3-1
3.1.1	Definition	3-1
3.1.2	Wozu CSS?	3-1
3.1.3	Versionen	3-2
3.2	Grundlegende Syntax	3-3
3.2.1	Beispiele	3-3
3.2.2	Kommentare	3-4
3.2.3	Gruppierung von Regeln	3-4
3.3	Arten der Anwendung (Einbindung)	3-5
3.3.1	Direkt beim Element	3-5
3.3.2	Direkt im Kopfbereich	3-5
3.3.3	Als eigenes Dokument	3-6
3.3.4	Einbinden aus weiteren Stylesheets	3-6
3.4	Arten von Selektoren	3-7
3.4.1	Element-/Typselektoren	3-7
3.4.2	Universalselektor	3-8
3.4.3	Klassenselektoren	3-8
3.4.4	ID-Selektoren	3-9
3.4.5	Attributselektoren	3-10
3.4.6	Nachfahr-Kombinatoren	3-12
3.4.7	Kind-Kombinatoren	3-13
3.4.8	Nächste Geschwister-Kombinatoren	3-13
3.4.9	Nachfolgende Geschwister-Kombinatoren	3-14
3.4.10	Pseudoelemente	3-14
3.4.11	Pseudoklassen	3-16
3.5	Vererbung von Eigenschaften	3-19
3.6	Die Kaskade	3-20
3.6.1	Filterung	3-20
3.6.2	Herkunft und Wichtigkeit	3-20
3.6.3	Kontext	3-22
3.6.4	Element-Attached-Styles (Inline-Styles)	3-22
3.6.5	Layers	3-22
3.6.6	Specificity (Gewichtung)	3-23
3.6.7	Reihenfolge	3-25
3.7	Werte an CSS-Eigenschaften übergeben	3-25
3.7.1	Numerische Angaben	3-25
3.7.2	Zeichenketten und Schlüsselwörter	3-26
3.7.3	Farben	3-27
3.7.4	Winkelmaße und Zeitangaben	3-29
3.8	Erste CSS-Formate	3-29
3.8.1	Schriften-Formatierung	3-30
3.8.2	Text-Formatierung	3-33
3.8.3	Abstände und Ausrichtung	3-34
3.8.4	Hintergründe	3-36
3.8.5	Listen	3-39

3.9	Weiterführende Literatur	3-40
4	CSS Box-Modell & Responsive Web	4-1
4.1	Das CSS Box-Modell	4-1
4.1.1	Überblick	4-1
4.1.2	Dimensionsangaben für den Inhalt	4-2
4.1.3	Innenabstand	4-5
4.1.4	Rahmen	4-6
4.1.5	Außenabstand	4-8
4.1.6	Besonderheiten und Wechsel der Box-Eigenschaften	4-9
4.2	Positionierung von Elementen	4-12
4.2.1	Statische Positionierung	4-12
4.2.2	Relative Positionierung	4-13
4.2.3	Absolute Positionierung	4-14
4.2.4	Fixe Positionierung	4-15
4.2.5	Sticky Positionierung	4-16
4.2.6	Floats	4-17
4.3	Einfache Anordnungen und Layouts mit <code>float</code>	4-18
4.3.1	Horizontales Anordnen von Elementen	4-18
4.3.2	Einfache Layouts	4-19
4.4	Media Queries	4-22
4.4.1	Medienspezifische Stylesheets	4-22
4.4.2	Medientypen	4-23
4.4.3	Aufbau von Media Queries	4-23
4.4.4	Medienmerkmale	4-24
4.4.5	Verknüpfung von Medienmerkmalen	4-25
4.4.6	Viewport angeben	4-26
4.4.7	Media Queries einsetzen	4-26
4.5	Weiterführende Literatur	4-31
5	CSS Layout-Frameworks	5-1
5.1	CSS Flexible Box Layout	5-1
5.1.1	Einen flexiblen Container definieren	5-1
5.1.2	Flexibilität und Reihenfolge der Boxen festlegen	5-3
5.1.3	Ausrichtung der Boxen bestimmen	5-5
5.2	CSS Grid Layout	5-7
5.2.1	Grid Terminologie	5-8
5.2.2	Einen Grid-Container definieren	5-8
5.2.3	Elemente im Raster anordnen	5-13
5.3	CSS-Frameworks: Bootstrap	5-15
5.3.1	Bootstrap allgemein	5-15
5.3.2	Bootstrap einbinden	5-16
5.3.3	Das Bootstrap Grid verwenden	5-17
5.3.4	Zusätzliche Bootstrap Funktionalitäten	5-21
5.3.5	Weitere CSS-Frameworks	5-21
5.4	CSS Custom Properties	5-22

5.4.1	Anlegen von Custom Properties	5-23
5.4.2	Custom Properties anwenden	5-23
5.4.3	Die Kaskade und Vererbung ausnützen	5-23
5.5	CSS Resets & Normalisierung	5-24
5.5.1	Alles zurücksetzen: CSS Reset	5-25
5.5.2	Sinnvoll zurücksetzen: CSS Normalisierung	5-26
5.6	CSS Methodiken	5-27
5.6.1	Historische Architekturansätze: OOCSS, SMACSS und ITCSS	5-27
5.6.2	Block Element Modifier (BEM)	5-28
5.6.3	Style Tools for UI Components (SUIT CSS)	5-29
5.6.4	Cascade Layers (@layer)	5-31
5.6.5	Weitere Methodiken und moderne Konzepte	5-32
5.7	Weiterführende Literatur	5-33
6	Web Accessibility	6-1
6.1	Allgemeines	6-1
6.2	Die Web Accessibility Initiative	6-2
6.2.1	Ziele	6-2
6.2.2	Arbeits- und Communitygruppen	6-2
6.2.3	Ergebnisse und Guidelines	6-4
6.3	Web Content Accessibility Guidelines (WCAG)	6-6
6.3.1	Allgemeines	6-6
6.3.2	WCAG 2.2	6-7
6.3.3	Evaluierung mit den WCAG 2.2	6-11
6.4	Accessible Rich Internet Applications (WAI-ARIA)	6-14
6.4.1	Allgemeines	6-14
6.4.2	WAI-ARIA 1.2	6-14
6.4.3	WAI-ARIA einsetzen	6-20
6.5	Unterstützende Technologien	6-21
6.5.1	Testen mit unterstützenden Technologien	6-22
6.6	Weiterführende Literatur	6-22
7	Datenformate	7-1
7.1	XML	7-1
7.1.1	Einführung	7-1
7.1.2	Ein XML-Beispieldokument	7-2
7.1.3	XML-Grammatik	7-4
7.1.4	Well-formed und Valid	7-7
7.1.5	Validierung durch Schemas	7-8
7.1.6	XML-Applikationen	7-16
7.2	JSON	7-16
7.2.1	Allgemein	7-17
7.2.2	Ein JSON-Beispieldokument	7-17
7.2.3	JSON-Syntax	7-19
7.2.4	Validierung durch JSON-Schema	7-21
7.2.5	Beispiel-Schema	7-23

7.2.6	JSON-Schema verwenden	7-25
7.3	YAML	7-25
7.3.1	Einführung	7-25
7.3.2	Ein YAML-Beispieldokument	7-25
7.3.3	YAML-Syntax	7-27
7.3.4	YAML validieren	7-30
7.4	Weiterführende Literatur	7-30
8	JavaScript-Grundlagen	8-1
8.1	Allgemeines zu JavaScript	8-1
8.1.1	Was ist JavaScript?	8-1
8.1.2	Terminologie	8-1
8.1.3	Versionen, Geschichte und Browserunterstützung	8-2
8.2	Lexikalische Struktur	8-3
8.2.1	Zeichensatz	8-3
8.2.2	Groß- und Kleinschreibung	8-3
8.2.3	Strichpunkte	8-4
8.2.4	Kommentare	8-4
8.3	Datentypen	8-4
8.3.1	Zahlen	8-4
8.3.2	Zeichenketten	8-6
8.3.3	Boolesche Werte	8-7
8.3.4	Triviale Datentypen	8-7
8.3.5	Arrays	8-8
8.4	Variablen	8-8
8.4.1	Variablennamen	8-9
8.4.2	Deklaration	8-9
8.4.3	Loses Typkonzept	8-10
8.4.4	Konstanten	8-10
8.4.5	Strikter Modus	8-11
8.5	Operatoren	8-12
8.5.1	Zuweisungsoperatoren	8-12
8.5.2	Vergleichsoperatoren	8-13
8.5.3	Arithmetische Operatoren	8-13
8.5.4	Logische Operatoren	8-14
8.5.5	Bitweise Operatoren	8-15
8.5.6	Sonstige Operatoren	8-15
8.6	Kontrollstrukturen	8-15
8.6.1	Verzweigungen	8-15
8.6.2	Schleifen	8-17
8.7	Funktionen	8-18
8.7.1	Definieren von Funktionen	8-18
8.7.2	Aufrufen von Funktionen	8-19
8.7.3	Funktionen und der Gültigkeitsbereich von Variablen	8-22
8.7.4	Function Scope vs. Block Scope – var vs. let	8-24
8.7.5	Funktionen: Ein Beispiel	8-25

8.7.6	Anonyme Funktionen und Arrow Functions	8-26
8.7.7	Vordefinierte Funktionen	8-27
8.8	Weiterführende Literatur	8-27
9	JavaScript im Browser	9-1
9.1	Objekte	9-1
9.1.1	Was ist ein Objekt?	9-1
9.1.2	Zugriff auf Eigenschaften und Methoden	9-3
9.1.3	Das eigene Objekt mit <code>this</code> ansprechen	9-3
9.1.4	Definieren von Objekten	9-4
9.1.5	Vordefinierte Objekte	9-7
9.1.6	Alle Eigenschaften eines Objekts ausgeben: <code>for...in</code> Schleife . . .	9-11
9.1.7	JavaScript-Objekte und JSON	9-12
9.2	Client-Side JavaScript	9-13
9.2.1	Allgemeines	9-13
9.2.2	Einbindung in HTML	9-14
9.2.3	<code>window</code> -Objekt	9-17
9.2.4	<code>document</code> -Objekt	9-19
9.3	Weiterführende Literatur	9-20
10	JavaScript DOM – Document Object Model	10-1
10.1	Allgemeines	10-1
10.1.1	Versionen und Geschichte	10-1
10.2	W3C DOM und DOM Living Standard	10-2
10.2.1	Ebenen des W3C DOM	10-2
10.2.2	DOM und HTML Living Standards	10-2
10.3	Repräsentation der Dokumentstruktur	10-3
10.4	Manipulation der Dokumentstruktur	10-5
10.5	Events	10-13
10.5.1	Arten von Events	10-13
10.5.2	Verarbeiten von Events	10-14
10.5.3	Standardverhalten von Events unterbinden	10-17
10.5.4	Zugriff auf das Listener-Element	10-18
10.5.5	Event-Listener Beispiele	10-18
10.6	Weiterführende Literatur	10-22
11	JavaScript Workflow und Tools	11-1
11.1	Allgemeines zu Workflows	11-1
11.2	Node.js zur Workflow-Unterstützung	11-2
11.2.1	Node.js im Überblick	11-2
11.2.2	Node.js installieren	11-3
11.2.3	Node.js Packages	11-4
11.3	Stylesheet Präprozessoren am Beispiel von Sass	11-6
11.3.1	Sass Einführung	11-6
11.3.2	Variablen	11-7
11.3.3	Verschachtelung	11-8
11.3.4	Partials und Module	11-9

11.3.5	Mixins	11-9
11.3.6	Vererbung	11-10
11.3.7	Weitere Funktionen	11-11
11.4	Tasks automatisieren mit Gulp	11-11
11.4.1	Gulp im Überblick	11-11
11.4.2	Gulp vorbereiten	11-12
11.4.3	Tasks definieren	11-13
11.4.4	JavaScript-Minifizierung mit gulp-terser	11-14
11.4.5	Präprozessor für Styles mit Minifizierung und Sourcemaps	11-16
11.4.6	Codequalität durch Linting	11-18
11.4.7	Testserver mit Live-Reload	11-20
11.4.8	Automatische Erkennung von Änderungen	11-21
11.4.9	Abläufe im Default Task kombinieren	11-22
11.4.10	Dependencies ersetzen	11-23
11.5	Weiterführende Literatur	11-23
12	HTML JavaScript APIs	12-1
12.1	Allgemein	12-1
12.1.1	HTML-APIs und Feature Detection	12-1
12.2	Canvas	12-2
12.2.1	Rechtecke	12-3
12.2.2	Pfade	12-4
12.2.3	Text	12-6
12.2.4	Bilder	12-7
12.2.5	Beispiele	12-8
12.3	Video	12-9
12.3.1	Container und Codecs	12-9
12.3.2	Video Markup	12-11
12.3.3	Kontrollelemente	12-11
12.4	Audio	12-13
12.4.1	Container und Codecs	12-13
12.4.2	Audio Markup	12-14
12.4.3	Kontrollelemente	12-14
12.5	Geolocation	12-15
12.5.1	Abfragen der aktuellen Position	12-15
12.5.2	Anzeigen einer Karte	12-16
12.6	Local Storage	12-17
12.7	Drag & Drop	12-19
12.7.1	Elemente ziehbar machen	12-20
12.7.2	Mit dem Ziehen von Elementen starten	12-20
12.7.3	Elemente zum Fallenlassen auswählen	12-21
12.7.4	Operationen beim Fallenlassen ausführen	12-21
12.7.5	Weitere Events und Anmerkungen	12-21
12.8	Weitere Technologien	12-22
12.8.1	Web Sockets	12-22
12.8.2	Indexed Database	12-22

Inhaltsverzeichnis	9
--------------------	---

12.8.3 Web Workers	12-23
12.8.4 Service Workers	12-23
12.9 Weiterführende Literatur	12-23

Quellenverzeichnis	12-24
---------------------------	--------------

Vorlesung 1

Einführung

1.1 Zur Web Fundamentals Vorlesung

Die Vorlesung Web Fundamentals – kurz WEF1VO – findet im ersten Semester des Bachelorstudiums Medientechnik und -design statt. Im zweiten Semester wird sie durch Web Technologies (WET2VO), im dritten durch Web Architectures (WAR3VO) fortgesetzt. Weiters ist diese Vorlesung inhaltlich mit Web Design (WED1IL) abgestimmt, welches die konzeptionellen Grundlagen für die hier vermittelten technischen Inhalte liefert. Im Folgenden sind die für dieses Semester relevanten Informationen kurz zusammengefasst.

1.1.1 Organisatorisches

Die Vorlesung zu Web Fundamentals behandelt die Grundlagen der Webentwicklung und ist auf *zwölf Termine zu je zwei Einheiten* aufgeteilt. Die Beurteilung erfolgt im Rahmen einer *elektronischen Klausur*, die am Ende des Semesters stattfindet. Die Klausur dauert 90 Minuten und findet ohne Zuhilfenahme der Unterlagen statt.

Die dazugehörigen Übungen sind auf *zwölf Termine zu zwei Einheiten* aufgeteilt. In jeder Übung werden die Inhalte der Vorlesung praktisch angewandt. Die Beurteilung der Übungen erfolgt durch fünf Übungsabgaben, die zu ausgewählten Übungen einzureichen sind. Die restlichen Übungen finden als code-along Einheiten statt und dienen der gemeinsamen Erarbeitung von Inhalten.

Die Unterlagen zum Kurs befinden sich auf der eLearning-Plattform der FH Oberösterreich in den zugehörigen Kursen für Vorlesung¹ und Übung².

1.1.2 Inhalte der Vorlesung

Im Verlauf des Semesters werden folgende Themengebiete behandelt:

- Hypertext Markup Language (HTML),
- Cascading Style Sheets (CSS),
- Box-Modell, Responsive Web und Layout-Frameworks,

¹<https://elearning.fh-ooe.at/course/view.php?id=38355>

²<https://elearning.fh-ooe.at/course/view.php?id=38353>

- Web Accessibility,
- Datenformate (XML, YAML, JSON),
- JavaScript/ECMAScript,
- JavaScript DOM,
- JavaScript Workflow & Tools,
- HTML JavaScript APIs.

1.2 Begriffsdefinitionen

Bevor mit der Vermittlung von Inhalten begonnen werden kann, ist es notwendig, die wichtigsten Begriffe aus dem Fachbereich zu definieren bzw. abzuklären.

1.2.1 Hypermedia

Hypermedia ist die Erweiterung von Hypertext durch die Einbindung weiterer Medien wie z. B. Bild, Ton oder Bewegtbild. Das größte, derzeit existierende Hypermedia-System ist das World Wide Web.

1.2.2 Hypertext

Als *Hypertext* wird eine Menge von Dokumenten (Knoten) bezeichnet, die Querverweise (Links, Kanten) enthalten, welche es den Leser*innen erlauben, mithilfe einer interaktiven Software (Browser) einfach von einem Dokument zum anderen zu gelangen. Der Begriff wurde vom US-amerikanischen Soziologen Ted Nelson 1963 zum ersten Mal verwendet. In der 1987 veröffentlichten Ausgabe von [41] definiert er den Begriff folgendermaßen:

By Hypertext I simply mean non-sequential writing; a body of written or pictorial material interconnected in such a complex way that it could not be presented or represented on paper. Hypertext is the generic term for any text, which cannot be printed.

1.2.3 World Wide Web

Das *World Wide Web* (auch WWW oder einfach Web) ist ein globales, interaktives Informationssystem, das auf Hypertext bzw. Hypermedia basiert. Entgegen der Verwendung im allgemeinen Sprachgebrauch ist das World Wide Web nicht mit dem Internet gleichzusetzen. Das World Wide Web ist lediglich einer von vielen Diensten im Internet. Andere sind z. B. E-Mail, Instant Messaging, Streamingdienste oder Cloud-Services. Das Internet ist also das Computernetz, über welches das World Wide Web verfügbar gemacht wird.

1.2.4 HTML

Die *Hypertext Markup Language* (HTML) ist das Dokumentenformat des WWW. HTML ist eine textbasierte Auszeichnungssprache (engl. „markup language“). Das Markup verleiht dem Text eine Struktur, die von Software (z. B. Webbrowsern) gelesen werden kann.

1.2.5 URL und URI

Ein URL – kurz für *Uniform Resource Locator* – ist ein Identifikationsschema, mit dem eine Ressource (z. B. eine Website) im Internet eindeutig zugeordnet werden kann. Ein URL ist (vereinfacht dargestellt) aus der Zugriffsmethode (meist ein Netzwerkprotokoll wie HTTP) und einem für diese Zugriffsmethode spezifischen Teil (meist Domain-Name und gesuchte Datei) aufgebaut: `http://www.domain.com/file.html`. URLs sind in RFC 1738 [7] – einem Standard der Internet Engineering Taskforce (IETF)³ – definiert.

Ein URL ist genau genommen eine Unterform eines URIs, des *Uniform Resource Identifiers*. Diese Art der Bezeichnung von Ressourcen ist noch etwas allgemeiner und umfassender und wurde im RFC 3986 [6] formuliert.

Um URLs zu standardisieren, existiert auch ein URL Living Standard [55] der WHATWG (siehe Abschnitt 1.7), der versucht, die IETF-Standards zu vereinheitlichen und mit den Gegebenheiten moderner Browser abzugleichen.

1.2.6 HTTP

Das *Hypertext Transfer Protokoll* (kurz HTTP) ist das Übertragungsprotokoll für Daten über ein Netzwerk, im Speziellen für Websites. Details zu HTTP werden im zweiten Semester in Web Technologies (WET2VO) behandelt.

Wichtig: Im Akronym HTTP ist das Wort „Protokoll“ bereits enthalten. Von einem „HTTP-Protokoll“ zu sprechen ist daher eine Tautologie und sollte vermieden werden.

1.2.7 Website vs. Webpage vs. Homepage

Im allgemeinen Sprachgebrauch werden die Begriffe Website, Webpage und Homepage oft gleichgesetzt bzw. schlichtweg falsch verwendet. Jedoch handelt es sich um unterschiedliche Dinge: Eine *Website* ist ein Verbund mehrerer, einzelner *Webpages*. Eine dieser Webpages ist wiederum die *Homepage*, die als Start- oder Einstiegsseite in die Website fungiert. Abbildung 1.1 stellt dies schematisch dar.

1.3 Funktionsweise des World Wide Web

Beim Aufruf einer Website findet ein Kommunikationsprozess statt, in den zwei Parteien involviert sind. Der *Client* fordert eine Website an, diese wird von einem *Server* auf der Gegenstelle bereitgestellt und zurück geliefert. Die Website selbst liegt im Dateisystem des Servers. Die Kommunikation erfolgt dabei über das Internet mit Hilfe von HTTP (was dabei genau passiert, in WET2VO erläutert). Dieser Ablauf ist in Abbildung 1.2 dargestellt.

Im Falle von einfachen, statischen HTML-Seiten kann der Server auch entfallen und diese können direkt lokal vom eigenen Rechner geöffnet werden. Erst bei serverseitigen Technologien (z. B. PHP) ist ein Webserver zwingend nötig. Notwendig ist dieser natürlich auch dann, wenn HTML-Seiten der Öffentlichkeit über das WWW zugänglich gemacht werden sollen (im Gegensatz zu lokalem Entwickeln, wie es im Rahmen der WEF-Übungen geschieht).

³<https://www.ietf.org/>



Abbildung 1.1: Eine Website enthält mehrere einzelne Webpages. Eine dieser Webpages ist die Homepage, also die Startseite der Website.

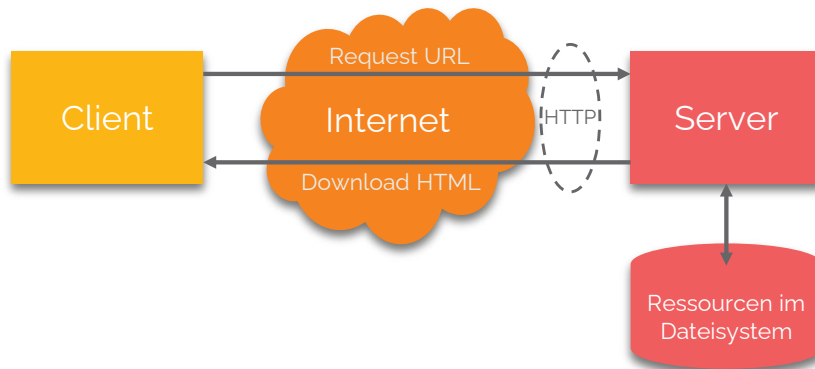


Abbildung 1.2: Client-Server-Kommunikation über das Internet.

Der typische Vertreter eines Clients ist der *Webbrowser*, aber auch eine *Suchmaschine* ist eine Art von Client. Auch sie ruft Websites ab, nur werden deren Inhalte indiziert und nicht angezeigt, wie es beim Webbrowser üblicherweise der Fall ist.

1.4 Webbrowser

Der *Webbrowser*, kurz *Browser* oder auch *Web-Client* genannt, ist die Software, die im Kommunikationsmodell die Anfragen an den Server stellt. Er ist auch gleichzeitig die Visualisierungsoberfläche für HTML-Markup, stellt also dieses Markup grafisch dar. Dieser Vorgang wird auch das Rendern des HTML-Markups genannt. Verantwortlich für diesen Vorgang ist die Rendering-Engine, die quasi unter der Haube des Browsers steckt. Sie ist auch dafür verantwortlich, wie gut ein Browser diverse Webstandards erfüllt, wie dieser Scriptsprachen wie JavaScript interpretiert und in welcher Form neue Technologien unterstützt werden. Es gibt derzeit fünf Browser, die im Bereich der Webentwicklung am Desktop eine Rolle spielen.

1.4.1 Google Chrome

*Chrome*⁴ wird von Google entwickelt und ist seit Ende 2008 als freier Browser verfügbar. Die aktuelle Version ist 141.0. Dies belegt den sehr kurzen Releasezyklus. Chrome ist als Open-Source Projekt unter dem Namen Chromium verfügbar, welches vor allem bei Linux-Distributionen zum Einsatz kommt.

Neu war zum Einführungszeitpunkt vor allem, dass in Chrome alle Tabs als eigene Prozesse realisiert (d. h. eigene kleine Programmteile) sind. Bei einem Absturz eines Tabs wird nicht der gesamte Browser in Mitleidenschaft gezogen. Auch hat dieses „abgeschlossene“ Tabbed Browsing Sicherheitsvorteile (Schadcode bleibt im einzelnen Tab und kann nicht übergreifen).

Chrome verwendete bis Version 27.0 die *WebKit2*-Engine. Diese basiert auf der unter KDE (Linux) entwickelten *KHTML*-Engine und wird nach wie vor aktiv von Apple weiter entwickelt. Die WebKit-Engine gehört auch zu den performantesten Renderern, sowohl für HTML als auch bei der Interpretierung von JavaScript. Seit Version 28.0 kommt in Chrome eine Abspaltung (engl. „fork“) von WebKit namens *Blink* zum Einsatz. Als Grund für den Umstieg wird eine größere Flexibilität bei der Entwicklung von Chrome-spezifischen Features begründet. Während früher WebKit als führend bei der Implementierung von Standards und der Integration neuer Technologien angesehen wurde, kommt diese Rolle nun eher Blink zu. Laut [45] liegt der Marktanteil für Oktober 2025 im Desktop-Bereich bei 79,53 %, was ihn zum meistbenutzten Browser macht.

1.4.2 Microsoft Edge

*Microsoft Edge*⁵ ist der aktuelle Browser von Microsoft. Er wurde erstmals mit Windows 10 veröffentlicht und legt den Fokus stark auf die Unterstützung von aktuellen und zukünftigen Web-Technologien, alte und proprietäre Technologien (wie etwa ActiveX) werden nicht mehr unterstützt. Als Engine kam mit der Veröffentlichung 2015 EdgeHTML (eine Abspaltung der Internet Explorer Engine Trident) zum Einsatz. 2018 kündigte Microsoft jedoch an, die Entwicklung der eigenen Engine einzustellen, und Edge fortan auf Blink zu basieren. Seit Anfang 2020 ersetzen diese Versionen die vorigen (nun Edge Legacy genannten) Versionen vollständig. Die Versionsnummern sind denen der Blink-Engine angepasst, die aktuelle ist daher wie bei Google Chrome 141.0. Der Marktanteil von Edge liegt laut [45] im Oktober 2025 bei 8,01 %, was ihn zur Nummer zwei am Desktop-Browsermarkt macht.

Der Vorgänger von Edge war der allseits bekannte *Internet Explorer*, der von 1995 bis 2015 entwickelt wurde. Die letzte Version war 11.0. Internet Explorer war lange Zeit der marktbeherrschende Browser bzw. kämpfte mit dem Netscape Navigator im sogenannten Browserkrieg⁶ (1995–1998) um die Vorherrschaft. In diese Zeit fällt die Entwicklung immer neuer Funktionalitäten, die auf Kosten der Webstandards gingen und für eine Vielzahl an noch immer vorherrschenden Kompatibilitätsproblemen im Web verantwortlich sind. Internet Explorer verwendete seit Version 4.0 die *Trident*-Engine. Die Engine stand lange in der Kritik, Webstandards nur unzureichend zu unterstützen bzw. falsch

⁴<https://www.google.com/chrome/>

⁵<https://www.microsoft.com/edge/>

⁶<https://de.wikipedia.org/wiki/Browserkrieg>

umzusetzen, wenngleich die letzte Version zufriedenstellender war. Allerdings wird diese nicht mehr weiterentwickelt, weshalb neue Technologien nicht mehr unterstützt werden. Laut [45] belegt der Internet Explorer mit einem Anteil von 0,18 % im Oktober 2025 mittlerweile keine Rolle mehr unter den Desktop-Browsern.

1.4.3 Apple Safari

*Safari*⁷ ist ein Browser, der in Apples Betriebssystem macOS integriert ist. Die aktuelle Version ist 26.0. Im September wurde die Versionierung der von iOS und iPadOS angepasst und ist somit das Kalenderjahr, welches der initialen Veröffentlichung folgt. Kurzzeitig existierten auch Versionen für Microsoft Windows, jedoch wurde Safari ab Version 6 nicht mehr für Windows veröffentlicht.

Wie früher Chrome verwendet auch Safari die *WebKit2*-Engine, es gelten zum Großteil die selben Dinge, die bereits in Abschnitt 1.4.1 erwähnt wurden, wenngleich Apple eine eigene Abspaltung der WebKit-Engine verwendet. Der Marktanteil im Desktop-Bereich laut [45] im Oktober 2025 beträgt 4,35 %. Damit ist der Browser die aktuelle Nummer drei am Desktop-Browsermarkt.

1.4.4 Mozilla Firefox

*Firefox*⁸ ist ein freier Webbrowser der Mozilla Foundation und seit 2002 in Entwicklung. Er gilt als Nachfolger des Netscape Navigators, einer der ersten populären Browser (neben dem Microsoft Internet Explorer). Die aktuelle Version ist 144.0. Der Releasezyklus neuer Versionen beträgt ca. 6 Wochen.

Mozilla Firefox verwendet intern die hauseigene *Gecko*-Engine. Diese zielt darauf ab, möglichst viele Webstandards umzusetzen. Bei der Umsetzung kommender Standards bzw. neuer Entwicklungen ist Gecko gut vertreten, jedoch nicht immer an vorderster Front. Aus diesem Grund wurde 2013 begonnen, eine neue Engine namens *Servo* von Grund auf zu entwickeln. Da diese jedoch in nicht absehbarer Zeit einsatzbereit war, wurde das Quantum Projekt gestartet, das fertige Servo-Komponenten in Gecko integrieren soll. Seit Version 57 hieß Firefox daher *Firefox Quantum*, da die ersten Servo-Komponenten bereits integriert wurden. Im Oktober 2019 wurde er jedoch wieder auf „Firefox Browser“ umbenannt. Firefox hatte im Oktober 2025 laut [45] einen globalen Marktanteil im Desktop-Bereich von 3,64 %, was ihn nur mehr zum viertmeistgenutzten Browser im Desktopbereich macht.

1.4.5 Opera

*Opera*⁹ wird vom norwegischen Unternehmen Opera Software entwickelt, die erste Version erschien 1996. Bis 2005 war Opera kostenpflichtig bzw. mit Werbung versehen. Seit Version 8.50 ist der Browser vollständig kostenlos.

Bis Opera 12 wurde die eigens entwickelte Rendering-Engine *Presto* verwendet. Diese war bei der Umsetzung von Webstandards im Vordergrund und bot zusätzlich einen

⁷<https://www.apple.com/safari/>

⁸<https://www.firefox.com/>

⁹<https://www.opera.com/>

Interpretationsmodus für nicht standardkonformes (vor allem auf alte Internet Explorer optimiertes) HTML. Im Juli 2013 wurde ein Versionssprung auf Opera 15 vollzogen. Seit damals kommt – wie bei neuen Google Chrome und Microsoft Edge Versionen – die Blink Rendering-Engine zum Einsatz. Im April 2023 wurde Version 100 mit einem neuen User-Interface veröffentlicht. Die aktuelle Version ist 122.0. Opera weist laut [45] im Oktober einen Desktop-Marktanteil von 1,91 % auf.

1.5 HTML – Erste Schritte

Essentiell für einen erfolgreichen Start in die Webentwicklung ist das Verständnis ihrer zentralen Sprache – *HTML*. Beginnend mit diesem Abschnitt werden die wichtigsten Grundlagen dazu erläutert. Ziel ist es, ein Verständnis für den Aufbau und die Funktionsweise zu vermitteln, weniger jedes einzelne Element im Detail zu erklären. Für dies sei auf Dokumentationen im Web verwiesen – allen voran das Mozilla Developer Network¹⁰, welches eine gute Einführung inklusive aktueller Element-Referenz enthält.

1.5.1 Was ist HTML?

HTML – kurz für *Hypertext Markup Language* – ist die Beschreibungssprache für Web-Dokumente.

Hypertext bezeichnet, wie in Abschnitt 1.2.2 erwähnt, eine bestimmte Art von Text, die Informationen durch Hyperlinks mit anderen Hypertext-Knoten verknüpft.

Markup bedeutet zu Deutsch „auszeichnen“, also Textstellen zu kennzeichnen und ihnen somit eine spezielle Bedeutung zu geben. Diese Auszeichnung geschieht mit Hilfe von *HTML-Tags* (siehe dazu auch Abschnitt 1.5.2). Die ausgezeichneten Textpassagen werden vom Browser dann interpretiert und dargestellt (gerendert). Moderne Versionen von HTML zielen dabei auf eine semantische korrekte, d. h. logische Auszeichnung ab, die unmissverständlich bezeichnet, worum es sich bei einer Textstelle handelt (z. B. „Dieser Text ist eine Überschrift“). Details zu dieser Art von Auszeichnung finden sich in Vorlesung 2.

1.5.2 HTML-Tags und Elemente

HTML-Tags stehen immer in spitzen Klammern: `< >` – z. B. `<h1>`, `<p>`, `<a>` etc.

Der Großteil der HTML-Tags ist zweigeteilt, d. h. besitzt einen öffnenden und ein schließenden Tag. Der öffnende zeigt den Beginn der Auszeichnung an, es wird dazu der Name des Tags in spitze Klammern gestellt (z. B. `<h1>`). Der schließende Tag kennzeichnet das Ende der aktuellen Auszeichnung, es wird der Name des Tags hinter einen Schrägstrich (Slash) in die spitzen Klammern gestellt (z. B. `</h1>`). Um den Text „Das ist eine Überschrift“ als solche zu kennzeichnen, ist folgendes Markup notwendig:

```
1 <h1>Das ist eine Überschrift</h1>
```

Dieses Konstrukt aus Tags mitsamt Inhalt wird dann als *HTML-Element* bezeichnet.

¹⁰<https://developer.mozilla.org/en-US/docs/Web/HTML>

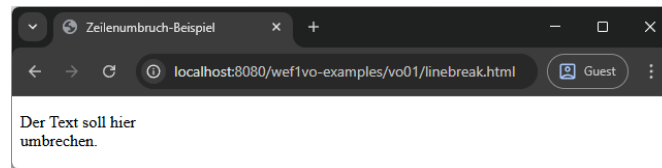


Abbildung 1.3: Der `<br>`-Tag bewirkt einen Zeilenumbruch nach dem Wort „hier“.

Case Sensitivity

Die Groß- und Kleinschreibung von HTML-Elementen kann prinzipiell variiert werden, d. h. es ist sowohl `<h1>` als auch `<H1>` für eine Überschrift erster Ordnung möglich. Es hat sich jedoch die klein geschriebene Variante in modernen Webseiten durchgesetzt.

Standalone-Tags

Während die meisten Elemente jeweils einen öffnenden und schließenden Teil besitzen, gibt es auch einige Tags, die nur aus einem Element (dem öffnenden Teil) bestehen. Sie werden *Standalone-Tags* (alleinstehende Tags) genannt und wirken in der Regel punktuell, d. h. sie kennzeichnen nicht einen Bereich wie andere HTML-Elemente. Der bekannteste Vertreter ist der Zeilenumbruch (engl. „line break“) `<br>`, der den Text innerhalb eines Absatzes an einer definierten Stelle umbricht. Um den Text „Der Text soll hier umbrechen.“ nach dem Wort „hier“ umzubrechen, d. h. das Wort „umbrechen“ in die nächste Zeile zu befördern, ist folgendes Markup notwendig:

```
1 <p>Der Text soll hier<br>umbrechen.</p>
```

Das Ergebnis ist in Abbildung 1.3 ersichtlich. Der Text selbst steht hier in einem Absatz (engl. „paragraph“), der durch das `<p>`-Element gekennzeichnet wird.

Wichtige HTML-Elemente für den Anfang

Tabelle 1.1 enthält zunächst eine Reihe von wichtigen Elementen für den Anfang. Mehr folgen in Vorlesung 2.

Verschachtelung

Viele (jedoch nicht alle) HTML-Elemente können wiederum andere HTML-Elemente enthalten. Man spricht in diesem Fall von *Verschachtelung*. Wichtig ist hierbei die korrekte Art und Weise, wie Tags ineinander verschachtelt werden. Überkreuzungen müssen unter allen Umständen vermieden werden. Als Regel gilt:

Werden mehrere HTML-Elemente ineinander verschachtelt, so sind sie genau in der umgekehrten Reihenfolge zu schließen, in der sie geöffnet wurden.

Korrekt ist also folgendes Markup:

```
1 <p><em>Dieses Markup ist korrekt verschachtelt.</em></p>
```

Inkorrekt wäre folgendes Beispiel:

```
1 <p><em>Achtung: Überkreuzung!</p></em>
```

Tabelle 1.1: Eine Liste der grundlegendsten HTML-Elemente.

Element	Bedeutung
<code><html></code>	Das sogenannte Root-Element. Jedes HTML-Dokument beginnt und endet mit ihm. Alle anderen Elemente sind in ihm enthalten.
<code><head></code>	In diesem Element sind alle Meta-Informationen zur aktuellen Seite enthalten. Zum Beispiel der Titel, Verweise auf Styles und Skripte oder der verwendete Zeichensatz.
<code><title></code>	Der Seitentitel. Er erscheint in der Regel in der Kopfzeile/im Tab des Browsers.
<code><body></code>	Innerhalb dieses Elements müssen alle Elemente vorkommen, die im eigentlichen Seiteninhalt aufscheinen sollen.
<code><h1></code> , <code><h2></code> , <code><h3></code> , <code><h4></code> , <code><h5></code> , <code><h6></code>	Sechs Ebenen von Überschriften. Sie sollten prinzipiell immer sequentiell verwendet werden (d. h. immer mit <code><h1></code> beginnen). Der Browser rendert sie von 1–6 in immer kleiner werdenden Größen, dies ist jedoch anpassbar (mehr dazu in Vorlesung 3).
<code><p></code>	Ein Absatz. Vor und nach ihm zeigt der Browser standardmäßig einen vertikalen Abstand an.
<code><div></code>	Ein Bereich bzw. Container. Dieses Element hat keine spezifische Bedeutung und wird oft für Layouts oder Gruppierung verwendet.
<code><a></code>	Ein Anker (engl. „anchor“) im Hypertext. Dieses Element ist der klassische Hyperlink.
<code>
</code>	Dieses Standalone-Element bewirkt einen Zeilenumbruch an der jeweiligen Stelle.
<code></code>	Repräsentiert eine betonte Textstelle (engl. „emphasize“). Browser stellen diese Betonung standardmäßig kursiv dar.
<code></code>	Bindet ein Bild bzw. eine Grafik ein.
<code></code>	Ein generischer Bereich ähnlich <code><div></code> , jedoch auf der Textebene. Er kennzeichnet also nur einzelne Wörter.

Whitespaces im Quellcode

Gestaltlose Zeichen wie Leerzeichen, Tabulatoren oder auch Zeilenumbrüche im Quellcode (erzeugt durch Drücken der Enter-Taste, nicht durch Einfügen eines `<br>`-Tags) werden vom Browser prinzipiell ignoriert. Dies hat zur Folge, dass es keinen Unterschied für das Erscheinungsbild einer Webpage macht, ob der gesamte Code in einer oder jeder Zeile in seiner eigenen Zeile mit Einrückungen geschrieben wird.

So führt sowohl dieses Markup-Fragment aus drei aufeinanderfolgenden Absätzen

```
1 <p>Erster Absatz.</p>
2   <p>Zweiter Absatz.</p>
3 <p>Dritter Absatz.</p>
```

als auch dieses

```
1 <p>Erster Absatz.</p><p>Zweiter Absatz.</p><p>Dritter Absatz.</p>
```

zur Darstellung in Abbildung 1.4. Man beachte die Zeilenumbrüche, Einrückung und zusätzlichen Leerräume im ersten Code-Fragment. Diese fehlen im zweiten Fragment komplett und trotzdem ist das Ergebnis dasselbe, da der Browser diese gestaltlosen Zeichen ohnehin ignoriert. Verantwortlich für die Tatsache, dass im Browser die drei

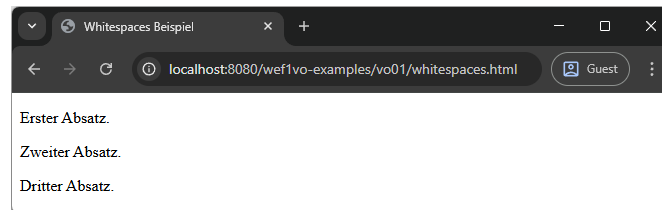


Abbildung 1.4: Egal ob Zeilenumbrüche, Einrückungen oder zusätzliche Leerräume verwendet werden – das Ergebnis im Browser sieht immer gleich aus: die drei Absätze stehen uneingerrückt untereinander.

Zeilen untereinander stehen, ist das `<p>`-Element. Es erzeugt jeweils einen Absatz, der durch einen vertikalen Abstand vor und nach dem Text repräsentiert wird.

1.5.3 HTML-Attribute

Jeder HTML-Tag hat eine bestimmte Bedeutung, in vielen Fällen ist aber noch zusätzliche Information notwendig, um das Element etwa eindeutig zu kennzeichnen oder bei einem Hyperlink (repräsentiert durch den `<a>`-Tag) anzugeben, wohin der Link führt. Diese Zusatzinformationen können über *Attribute* angegeben werden. Ein Element kann ein oder mehrere Attribute besitzen, diese werden in der Form `name="wert"` in den öffnende HTML-Tag geschrieben. Um etwa einem Absatz die eindeutige Bezeichnung (ID) `content` zu geben, ist folgendes Markup nötig:

```
1 <p id="content">Etwas Inhalt</p>
```

Jeder Tag hat eine unterschiedliche Menge an möglichen Attributen, diese sind in der Dokumentation des HTML Living Standard [54] definiert. Es gibt jedoch eine Reihe von sogenannten globalen Attributen, die bei jedem Element angefügt werden können. `id` ist ein derartiges globales Attribut. Eine vollständige Auflistung für die aktuellste HTML-Version findet sich in der Living Standard-Spezifikation der WHATWG¹¹ oder beim Mozilla Developer Network¹².

1.5.4 Kommentare

Kommentare im Code dienen dazu, Zusatzinformationen anzumerken, die jedoch nicht in der Webseite selbst ersichtlich sein sollen. So können etwa am Beginn des Dokuments Autor*innenname, Erstellungsdatum oder Versionsnummer angegeben werden. Im Code können bei komplexeren Stellen Anmerkungen gemacht werden. Der Browser ignoriert diese Teile vollständig und zeigt den Inhalt nicht an. Da aber Kommentare Teil des Quelltextes sind, sind sie von jeder Person, die sich den Quellcode der Webseite im Browser ansieht, lesbar.

Kommentare können weiters dazu dienen, ganze Blöcke im HTML-Markup auszukommentieren, z. B. um Fehler zu suchen, oder nicht benötigte Teile temporär zu entfernen.

¹¹<https://html.spec.whatwg.org/multipage/dom.html#global-attributes>

¹²https://developer.mozilla.org/en-US/docs/Web/HTML/Global_attributes

Ein HTML-Kommentar beginnt mit der öffnenden spitzen Klammer, gefolgt von einem Ausrufezeichen und zwei Bindestrichen. Um das Kommentar zu schließen sind zwei Bindestriche gefolgt von der schließenden spitzen Klammer nötig. Ein einzeliges HTML-Kommentar sieht folgendermaßen aus:

```
1 <!-- <p>Dieser Absatz wird vom Browser nicht angezeigt.</p> -->
```

Kommentare können sich auch über mehrere Zeilen erstrecken bzw. auch mitten im Markup vorkommen:

```
1 <!--
2 <p>Ein Absatz.</p>
3 <p>Noch ein Absatz. Beide werden auskommentiert.</p>
4 -->
5 <p>Dieser Teil ist <!-- nicht sichtbar --></p>
```

1.5.5 Entitäten

Einige Zeichen sind in HTML für interne Funktionalitäten in Verwendung. So werden die spitzen Klammern `<` und `>` verwendet, um einen HTML-Tag anzuzeigen. Was nun, wenn im Text eines dieser Zeichen benötigt wird? Würde es einfach so verwendet werden, ginge der Browser davon aus, dass ein HTML-Tag beginnt und könnte evtl. unlesbaren Output erzeugen.

Um dies zu verhindern, können die spitzen Klammern speziell kodiert werden. So wird dem Browser mitgeteilt, dass explizit das Zeichen angezeigt werden soll und nicht ein HTML-Tag beginnt. Diese Kodierungen werden *Entitäten* (engl. „entities“) genannt. Sie beginnen immer mit einem kaufmännischen Und (`&`) und enden mit einem Strichpunkt (`;`). Dazwischen kommt der jeweilige Bezeichner.

Um nun die öffnende spitze Klammer (auch „Kleiner-Zeichen“) zu erhalten, lautet die Entität `<` (lt – lower than). Die schließende spitze Klammer wird durch `>` (gt – greater than) repräsentiert. Auch für deutsche Umlaute und viele Sonderzeichen gibt es Entitäten. Diese mussten früher verwendet werden, als hauptsächlich noch regionale Zeichenkodierungen im Einsatz waren. Seit der starken Verbreitung von *Unicode* (und den damit verbundenen Kodierungen) sind diese nicht mehr zwingend nötig (mehr zu Zeichenkodierungen in Vorlesung 2). Jedoch ist es – gerade bei Sonderzeichen – oft schneller, den Entitätsnamen zu verwenden, als das Sonderzeichen über Copy & Paste einzufügen (z. B. bei `©`, um ein ©-Zeichen zu erzeugen).

Tabelle 1.2 enthält eine kleine Übersicht über einige Entitäten. Eine vollständige Liste findet sich in im HTML Living Standard¹³.

1.5.6 HTML-Grundgerüst

Mit dem Wissen über Elemente und korrekte Verschachtelung kann nun ein erstes, einfaches HTML-Grundgerüst aufstellt werden. Es gibt der Seite einen Titel und zeigt einen einzelnen Absatz mit etwas Inhalt an. Zur besseren Übersicht wurden die Elemente entsprechend ihrer Verschachtelungstiefe eingerückt. Die Ausgabe des Grundgerüsts ist in Abbildung 1.5 ersichtlich.

¹³<https://html.spec.whatwg.org/multipage/named-characters.html#named-character-references>

Tabelle 1.2: Eine Liste mit einigen wichtigen HTML-Entitäten. ` ` steht dabei für ein geschütztes Leerzeichen (engl. „non-breaking space“).

Zeichen	Entität	Zeichen	Entität
<	<code>&lt;</code>	ä	<code>&auml;</code>
>	<code>&gt;</code>	ö	<code>&ouml;</code>
&	<code>&amp;</code>	ü	<code>&uuml;</code>
"	<code>&quot;</code>	Ä	<code>&Auml;</code>
'	<code>&apos;</code>	Ö	<code>&Ouml;</code>
©	<code>&copy;</code>	Ü	<code>&Uuml;</code>
□	<code>&nbsp;</code>	ß	<code>&szlig;</code>

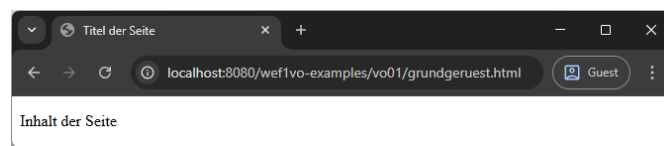


Abbildung 1.5: Das einfachste HTML-Grundgerüst im Browser dargestellt.

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <title>Titel der Seite</title>
5   </head>
6   <body>
7     <p>Inhalt der Seite</p>
8   </body>
9 </html>

```

1.5.7 Der DOCTYPE

Der *DOCTYPE* spezifiziert die verwendete HTML-Version den am Beginn eines HTML-Dokuments. Er garantiert, dass die Seite vom Browser nach den Regeln dieser Version interpretiert wird. DOCTYPEs wurden zum ersten Mal um die Jahrtausendwende mit der Einführung von HTML 4.01 bzw. des Browsers Internet Explorer 5 für macOS notwendig (die ersten DOCTYPEs kamen aber bereits zwei Jahre vorher mit der Veröffentlichung von HTML 4.0 auf). Dieser Browser hatte eine komplett neue Rendering-Engine (genannt *Tasman*), welche sich – verglichen zur parallel entwickelten und verwendeten Trident Engine – viel mehr an Standards (wie das neue HTML 4.01) hielt. Dies hatte zur Folge, dass alte Websites nicht mehr korrekt angezeigt wurden. Mark Pilgrim dazu in [43]:

IE5/Mac was so advanced, it actually broke the Web.

Microsofts Lösung war, den sogenannten DOCTYPE, als erste Zeile eines Dokuments, noch vor dem `<html>`-Element auszulesen. War ein solcher vorhanden, wurde das Dokument standardkonform gerendert, wenn nicht, dann wurde die alte (bisher verwendete)

Methode verwendet. Dieser Modus trägt inzwischen den Namen *Quirks-Modus*, sein Gegenstück ist (wie erwartet) der *Standard-Modus*.

Diese Lösung ermöglicht, dass Browser Markup, das nach einer bestimmten HTML-Version geschrieben wurde, korrekt ausgegeben können. Auch dann noch, wenn sich der Standard inzwischen stark geändert haben sollte.

Für HTML 4.01 gab es drei DOCTYPEs. Der DOCTYPE für den strikten HTML 4.01 Standard (HTML 4.01 Strict), der für den etwas lockeren HTML 4.01 Transitional Standard und ein Standard für Websites mit Frames (HTML 4.01 Frameset). Diese DOCTYPEs sind nach einer eigenen Grammatik aufgebaut, hinter dem URL steckt die Definition, welche Elemente genau wie verwendet werden dürfen:

```
1 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"  
2   "http://www.w3.org/TR/html4/strict.dtd">
```

```
1 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"  
2   "http://www.w3.org/TR/html4/loose.dtd">
```

```
1 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN"  
2   "http://www.w3.org/TR/html4/frameset.dtd">
```

Für XHTML 1.0, den Nachfolger von HTML 4.01, gab es ebenfalls eigene DOCTYPES für dieselben drei Varianten:

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"  
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"  
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Frameset//EN"  
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">
```

Während diese Zeilen ihren Zweck erfüllen, sind sie für Entwickler*innen in der Regel nicht auswendig merkbar. Mit HTML5 (nun mehr der HTML Living Standard) wurde auf diese Tatsache reagiert, der DOCTYPE für diese Version sieht (wie bereits im Grundgerüst verwendet) wie folgt aus. Unterschiedliche Substandards wie Strict oder Transitional gibt es nicht mehr. Auch neue Entwicklungen, wie sie laufend im Living Standard geschehen, werden diese Syntax beibehalten.

```
1 <!DOCTYPE html>
```

1.6 Die Geschichte von HTML

Die Geburtsstunde von *HTML* war 1989 im Schweizer Forschungsinstitut CERN. Tim Berners-Lee entwickelte dort den Prototypen eines Datenformats, das es erlaubte, Dokumente untereinander zu referenzieren. Während auch Firmen wie Apple an Hypertext-Systemen arbeiteten, diese aber nur auf lokale Arbeit an einem Computer ausgelegt waren, sah die Idee von Berners-Lee vor, ein globales System zu schaffen, das über das Internet funktionieren sollte.

Dazu wurde eine einfache Art benötigt, diese Daten zu übertragen und Berners-Lee entwickelte hierfür das HTTP. Das Format für Dokumente wurde HTML, hier

griff Berners-Lee jedoch auf die damals bereits bekannte und auch akzeptierte Markup-Sprach SGML (Standard Generalized Markup Language) zurück und basierte HTML zu großen Teilen darauf. Dies erwies sich als kluger Schachzug und ebnete den Weg für den Siegeszug von HTML.

Im Folgenden sind einige der wichtigsten Schritte in der weiteren Geschichte von HTML aufgelistet:

- **1992:** Die Urversion von *HTML* wird in einem Dokument spezifiziert. Sie kann zunächst nur mit Text arbeiten. Später kommen noch einige einfache Formatierungen und das Einbinden von Bildern hinzu.
- **1993:** Mit *HTML+* werden Erweiterungen wie der Textumfluss von Bildern vorgeschlagen, jedoch nie wirklich verabschiedet.
- **1995:** *HTML 2.0* ist die erste, über einen sog. Request for Comments (RFC) definierte Version. Neu sind etwa Formulare.
- **1997:** Anfang des Jahres bringt *HTML 3.2* Tabellen und nun auch den Textfluss um Bilder. Java Applets kommen auch zum ersten Mal auf. Die zuvor geplante Version 3.0 wird gleich übersprungen.
- **1997:** Ende des Jahres erscheint *HTML 4.0*. Zu den Neuigkeiten zählen Stylesheets, Frames, Scripts und viele anderen Dinge. Zum ersten Mal wird von Strict, Transitional und Frameset Varianten gesprochen.
- **1999:** Viele kleine Fehler in HTML 4.0 führen zur Einführung der *Version 4.01*. Sie bleibt die letzte offizielle HTML-Version bis HTML5.
- **2000:** *XHTML 1.0* wird als Neuformulierung von HTML 4.01 in der Markupsprache XML vorgestellt. Die Version wird 2002 noch einmal überarbeitet.
- **2001:** *XHTML 1.1* setzt den Kurs seines Vorgängers fort. Die Sprache wird in Module aufgeteilt. Nur die strikte Variante bleibt übrig. XHTML 1.1 findet jedoch weniger Verbreitung als sein Vorgänger.
- **2006:** Mit *XHTML 2.0* sollte eine radikale Neuformulierung der Sprache realisiert werden. Aufgrund der strikteren Syntax zog die lediglich als Entwurf vorliegende Spezifikation 2009 den kürzeren gegenüber dem parallel entwickelten HTML5 und die Entwicklung wurde eingestellt.
- **2009:** Der erste Entwurf der neuen *HTML5* Spezifikation wird vorgelegt. Er wird ständig weiter entwickelt und stellt derzeit den Stand der Technik dar. Aufbauend auf den Vorgänger (HTML 4.01) werden neues Vokabular eingeführt und Altlasten entsorgt.
- **2012:** Noch bevor HTML5 als Recommendation verabschiedet ist, wird Ende des Jahres der erste Entwurf von *HTML 5.1* veröffentlicht. Er soll neue Tags und Funktionen einführen, die in Version 5 keinen Platz mehr finden.
- **2014:** *HTML5* wird am 28. Oktober 2014 zum Standard (W3C Recommendation) erhoben.
- **2016:** Am 1. November wird *HTML 5.1* als Recommendation verabschiedet. Die Entwicklungszeit betrug knappe 4 Jahre. Weiters wird ein erster Entwurf für HTML 5.2 im August 2016 veröffentlicht. Er umfasst vor allem Korrekturen für Version 5.1.

- **2017:** *HTML 5.2* wird am 14. Dezember zur Recommendation erhoben und stellt somit den letztgültigen Standard da. Zum selben Zeitpunkt wird mit der Arbeit an HTML 5.3 begonnen.
- **2018:** HTML 5.3 steht seit Oktober 2018 im Status eines Working Draft, also noch am Anfang des Standardisierungsprozesses. Der angepeilte, jährliche Veröffentlichungszyklus wurde nicht mehr eingehalten, parallele Entwicklungen der WHATWG mit dem Living Standard führten zu Verzögerungen.
- **2019:** Am 28. Mai beschließen W3C und WHATWG (siehe Abschnitt 1.7) die Entwicklung von HTML auf den WHATWG *HTML Living Standard* zu verlegen. Dieser stellt somit immer den letztgültigen Entwicklungsstand dar. Die Versionierungen von HTML5 gehören auch der Vergangenheit an. Entsprechende (veraltete) Standards leiten mittlerweile auf den Living Standard weiter.

1.7 Standardisierung im Web

Damit die eingesetzten Technologien im Web nicht einem Wildwuchs durch im Wettbewerb liegende Browserhersteller ausgesetzt sind (wie es schon in den 90ern zwischen Microsoft und Netscape der Fall war) gibt es *Standards*, an die sich Browserhersteller halten können (jedoch nicht müssen). Diese Standards sollen sicherstellen, dass das Erlebnis einer Website unabhängig vom Client dasselbe ist bzw. Benutzer*innen nicht gezwungen sind, für eine bestimmte Seite einen bestimmten Browser zu verwenden.

1.7.1 Organisationen

Zwei wichtige Organisationen sind führend bei der Entwicklung von Standards im Web.

Das W3C

Das Unternehmen, das diese Standardisierungen vornimmt, ist das *World Wide Web Consortium*¹⁴ (kurz W3C). Es wurde 1994 von Tim Berners-Lee gegründet. Es überwacht und entwickelt eine Vielzahl an Webtechnologien und -funktionen, allen voran CSS, aber auch XML, Barrierefrei usw.

Jede neue Technologie-Spezifikation für das Web wird jedoch nicht sofort zum Standard erklärt. Sie durchläuft eine jahrelange Phase permanenter Überarbeitung und Reevaluierung bis sie als Standard verabschiedet wird.

Die Stufen dieses Prozesses sind folgende:

1. Working Draft (WD) – Arbeitsentwurf,
2. Last Call (LC) – Letzter Aufruf für Kommentare zum Entwurf,
3. Candidate Recommendation (CR) – Veröffentlichungsvorschlag,
4. Proposed Recommendation (PR) – Geplante Veröffentlichung,
5. W3C Candidate Recommendation (REC) – Standard.

Da das W3C kein klassisches Standardisierungsunternehmen (wie etwa die ISO) ist, spricht man genau genommen auch nicht von einem Standard, sondern von einer Empfehlung. De facto handelt es sich aber um einen anerkannten Standard.

¹⁴<https://www.w3.org/>

Die WHATWG

Während das W3C ein unabhängiges Konsortium ist, dem vornehmlich Einzelpersonen angehören, hat sich parallel dazu eine Arbeitsgruppe, bestehend aus Personen, die sehr unternehmensnah agierten, gebildet. Zur *Web Hypertext Application Technology Working Group*¹⁵ (WHATWG) gehören etwa Personen der Mozilla Foundation, Opera Software AS, Google oder Apple. Sie wurde gegründet, da das W3C was die Entwicklung von Standards angeht, von vielen (vor allem den Browser Herstellern) sehr lange als zu träge angesehen wurde. Während viele Jahre so etwas wie direkte Konkurrenz herrschte, die in den Parallelentwicklungen XHTML 2.0 (W3C) und HTML5 (WHATWG) gipfelte, gibt es in den letzten Jahren wieder aktive Zusammenarbeit.

Die WHATWG strebt keine Konkurrenz zum W3C im Standardisierungsprozess an. Vielmehr werden WHATWG-Entwürfe, die sogenannten *Living Standards*, vom W3C mitentwickelt. Das bekannteste Beispiel ist der HTML Living Standard [54].

1.7.2 Überprüfen von Standards

Das W3C stellt Entwickler*innen Tools zur Verfügung, die es ermöglichen, die eigene Website auf Standardkonformität zu prüfen. Im Falle von HTML ist dies der *HTML-Validator*¹⁶. Der Validator analysiert das Markup der eigenen Seite und kann aufgrund des verwendeten DOCTYPEs feststellen, ob der angegebene Standard eingehalten wurde. Mögliche Fehler werden mit Erklärungen ausgegeben.

Der Validator funktioniert dem aktuellen HTML Living Standard, aber auch ältere Versionen wie HTML 4.01 oder XHTML 1.0 können überprüft werden.

1.8 Weiterführende Literatur

Wie bereits erwähnt, befindet sich HTML in stetiger Weiterentwicklung. Mit dem Kauf von Literatur passiert es daher unausweichlich, dass diese unter Umständen nach kurzer Zeit bereits nicht mehr aktuell ist. Davon sollte man sich jedoch nicht abschrecken lassen, sind die Änderungen, die laufend im Living Standard geschehen, selten so groß, dass Bücher völlig unbrauchbar werden. Dennoch sollte man bei Literatur darauf achten, dass sie nicht älter als ein paar Jahre ist.

Wer HTML zusammen mit CSS abdecken möchte, kann auf [58] zurückgreifen. Das Buch ist seit Jahren mit kontinuierlichen Neuauflagen das aktuellste und orientiert sich am Living Standard. Zusammen mit den Themen CSS und JavaScript werden viele Dinge der Hypermedia-Vorlesung auf über 1.200 Seiten abgedeckt.

Drei weitere, zwischen 400 und 500 Seiten starke, Einführungswerke sind [40], [57] und [18]. Alle konzentrieren sich auf die Themen HTML und CSS (also kein JavaScript) und legen den Aufbau einer ersten, einfachen Webseite ins Zentrum.

¹⁵<https://whatwg.org/>

¹⁶<https://validator.w3.org/>

Vorlesung 2

HTML von A bis Z

In diesem Kapitel werden alle wichtigen Grundlagen erwähnt, die zur Erstellung eines HTML-Dokuments wichtig sind. Dabei geht es nicht darum, jedes einzelne Element in seinem Detail aufzuzählen, sondern vielmehr den Aufbau und die Denkweise zu vermitteln.

2.1 Kopfdaten

Vor dem eigentlichen, sichtbaren Dokumenteninhalt (der innerhalb des Elements `<body>` platziert wird), gibt es auch noch Informationen und Daten eines Dokuments, die bei jeder Seite mit angegeben werden und etwa von Suchmaschinen oder Webbrowsern ausgewertet werden. Diese sind innerhalb des `<head>`-Elements untergebracht und werden mit dem Elementen `<title>` (Seitentitel), `<base>` (Basis-URL der Seite), `<link>` (Verlinkung zu externen Ressourcen), `<style>` (direkte Angabe von Style-Informationen) und schließlich `<meta>` (Meta-Informationen) angegeben.

2.1.1 Seitentitel

Wie bereits in Vorlesung 1 kurz erwähnt, repräsentiert das `<title>`-Element den *Titel* einer Seite. Dieser wird im Tab bzw. der Kopfzeile des Browsers angezeigt und darf nicht weggelassen werden. HTML-Dokumente ohne Titel sind nicht valide, d. h. der HTML-Validator¹ zeigt einen Fehler an. Die Angabe des Titels ist aber ohnehin immer im Sinne der Autor*innen, denn er wird von Suchmaschinen und in Browser-Lesezeichen (Bookmarks) verwendet. Der Titel sollte daher auch aussagekräftig gewählt werden.

```
1 <title>John Doe's Blog | Gedanken eines Namenlosen</title>
```

2.1.2 Basis-URL

Das `<base>`-Element in Kombination mit dem `href`-Attribut erlaubt es, den *Basis-URL* für eine Seite zu setzen. Somit wird allen relativen Verlinkungen (siehe dazu Abschnitt 2.4.1) dieser URL vorangestellt. Durch die Definition von

```
1 <base href="https://www.johndoe.com/">
```

¹<https://validator.w3.org/>

wird dieser URL jeder relativ angegebenen Quelle vorangestellt. Somit wird ein Bild, das durch `` (Details zu Bildern siehe Abschnitt 2.5.1) angegeben wurde, vom Webbrowser mit dem vollen URL, also `https://www.johndoe.com/image.png` eingebunden. Dies legt jedoch fest, dass die Seite unter der angegebenen Domain verfügbar ist, ein lokales Testen ist nicht mehr möglich.

Praktischer ist oft eine relative Angabe als Basis-URL. Liegen etwa alle Bilder im Verzeichnis `images`, so kann `/images/` als alleiniger Wert des `href`-Attributs verwendet werden, somit wird das Unterverzeichnis automatisch allen Bildangaben vorangestellt. Vorsicht jedoch, dies betrifft dann auch etwa Hyperlinks in der Seite, was nicht immer gewünscht sein mag.

2.1.3 Externe Verlinkungen und Style-Angaben

Über das Element `<link>` können *externe Dokumente* eingebunden werden. Die Syntax dazu wird in Vorlesung 3 beim Einbinden von Stylesheets erläutert.

Ebenso ist es möglich, *CSS-Formatierungen* im Kopfbereich über das `<style>`-Tag anzugeben. Dies wird ebenfalls in Vorlesung 3 thematisiert.

2.1.4 Meta-Angaben

Meta-Angaben erlauben es Autor*innen, zusätzliche Informationen über ein HTML-Dokument bereitzustellen. Sie lassen sich in drei Kategorien unterteilen: *Dokumentdaten*, *Angaben für den Webserver* und die verwendete *Zeichenkodierung*. Alle drei werden durch das `<meta>`-Element spezifiziert, jedoch jeweils durch ein anderes Attribut repräsentiert.

Dokumentdaten

Dokumentdaten sind Angaben, die den Dokumentinhalt genauer spezifizieren. Dies sind etwa Beschreibung, der Autor*innenname, Schlüsselwörter oder auch die verwendete Software (wenn die Seite automatisch erzeugt, d. h. nicht von Hand geschrieben wurde). Diese Kategorien werden beim Element `<meta>` mit dem `name`-Attribut angegeben. Der dazugehörige Wert steht in einem weiteren Attribut namens `content`. Mehrere Meta-Angaben für Dokumentendaten sehen wie folgt aus:

```
1 <meta name="description" content="Der Blog von John Doe">
2 <meta name="author" content="John Doe">
3 <meta name="keywords" content="Blog,John,Doe,News,Neuigkeiten">
4 <meta name="generator" content="JetBrains PhpStorm 2025.2.4">
```

Dokumentendaten haben heutzutage nur noch geringe Bedeutung. Früher wurden die Schlüsselwörter (`name="keywords"`) von Suchmaschinen zur Indizierung verwendet, der häufige Missbrauch hat jedoch dazu geführt, dass diese nicht mehr beachtet werden. Die Beschreibung (`name="description"`) und einige andere Metainformationen werden jedoch nach wie vor berücksichtigt [35].

Angaben für den Webserver

Wird eine Webseite vom Webserver zurückgegeben (Webserver Response), werden bestimmte, so genannte Header-Informationen mitgesendet. Sie enthalten z. B. die ver-

wendete Zeichenkodierung oder andere ähnliche Angaben. Diese Werte können in der HTML-Seite mittels *Webserver Angaben*, auch Pragma Direktiven genannt, überschrieben werden. Beispiele sind neben der verwendeten Zeichenkodierung ein Standard-Stylesheet oder die Weiterleitung auf einen neuen URL nach einer gewissen Zeit.

Um derartige Angaben zu machen, wird das `<meta>`-Element in Kombination mit dem `http-equiv`-Attribut verwendet, das die Art der Anweisung enthält. Wiederum wird ein zweites Attribut `content` verwendet, in dem die Werte für die Anweisungen enthalten sind. Die soeben genannten Anweisungen werden wie folgt umgesetzt:

```
1 <meta http-equiv="content-type" content="text/html; charset=utf-8">
2 <meta http-equiv="default-style" content="styles.css">
3 <meta http-equiv="refresh" content="20; URL=page2.html">
```

Zeile 1 teilt dem Browser mit, dass es sich um ein HTML-Dokument mit der UTF-8 Zeichenkodierung handelt. Zeile 2 gibt an, dass das bevorzugte Stylesheet die Datei `styles.css` ist (diese Angabe wird selten verwendet). Die Angaben in Zeile 3 bewirken, dass nach 20 Sekunden auf die Seite `page2.html` weitergeleitet wird.

Zeichenkodierung

Die *Angabe der Zeichenkodierung* ist essentiell, damit der Webbrowser weiß, wie er die im Dokument enthaltenen Zeichen anzuzeigen hat. Während es beim normalen Alphabet selten zu Problemen kommt, treten diese vor allem bei Sonderzeichen wie Umlauten auf. Die Angabe der korrekten Zeichenkodierung mit der soeben gezeigten Webserver Angabe verhindert dies, ist jedoch umständlich und schwer zu merken. Daher wurde mit HTML5 das Attribut `charset` (kurz für das englische „character set“) eingeführt, welches diese Angabe drastisch vereinfacht. Um die Zeichenkodierung nun anzugeben, kann (und sollte) folgendes Markup verwendet werden:

```
1 <meta charset="utf-8">
```

Zeichensätze und -kodierungen

Als *Zeichensatz* wird die Menge an Zeichen zur Darstellung von Schrift, Zahlen und Sonderzeichen am Computer bezeichnet. Im westlichen Raum dienen das lateinische Alphabet mit Umlauten und Sonderzeichen oder *Unicode* als Zeichensätze. Aufbauend auf diesen gibt es die verschiedensten *Zeichenkodierungen*. Sie geben an, durch wie viele Bytes ein Zeichen repräsentiert wird und welchen Wert es genau hat. Die im Webbereich wichtigste Zeichenkodierung ist UTF-8. Erwähnenswert sind aus historischen Gründen noch ASCII und ISO 8859.

Bei *ASCII* handelt es sich um einen 1963 eingeführten und zuletzt 1986 aktualisierten Standard [3] mit dem 95 Zeichen darstellbar sind (128 gibt es insgesamt, man spricht daher von einer 7-Bit Zeichenkodierung, da $2^7 = 128$). Enthalten sind das lateinische Alphabet in Groß- und Kleinschreibung, die zehn arabischen Ziffern sowie die wichtigsten Satzzeichen.

Aufbauend auf ASCII ist die Zeichenkodierung *ISO 8859-1* [28]. Es handelt sich um eine 8-Bit (= 1 Byte) Kodierung, somit ist Platz für 256 Zeichen ($2^8 = 256$). Die ersten 128 Zeichen entsprechen genau ASCII, die zweiten 128 sind mit deutschen Umlauten und diversen Sonderzeichen befüllt.

Neben ISO 8859-1 gibt es noch ISO 8859-2 bis ISO 8859-16. Diese Zeichenkodierungen unterscheiden sich jeweils durch die verwendeten Sonderzeichen. ISO 8859-15 entspricht ISO 8859-1, enthält aber bereits das Euro-Zeichen (€), ISO 8859-7 enthält griechische, ISO 8859-5 kyrillische Zeichen.

Schließlich gibt es mit *UTF-8* [113] die modernste Zeichencodierung. Sie basiert auf dem Unicode-Zeichensatz, der in Version 17.0 knapp 160.000 Zeichen der allermeisten bekannten Schriftsysteme enthält. UTF-8 bildet dies als 8-Bit Kodierung mit variabler Länge ab. Maximal möglich sind 32 Bit, also 4 Byte, die jedoch bei weitem nicht ausgenutzt werden. Wiederum sind die ersten 127 Zeichen mit ASCII identisch und daher abwärtskompatibel. Die deutschen Sonderzeichen haben jedoch andere Byte-Codes und stimmen nicht mit ISO 8859-1 überein. Somit ergeben sich falsche Darstellungen falls ein in ISO 8859-1 verfasstes Dokument vom Browser als UTF-8 dargestellt wird und umgekehrt.

Um Probleme zu vermeiden, sollte der Zeichensatz am Beginn angegeben werden und das Dokument selbst (also die Datei) im entsprechenden Format abgespeichert werden. Moderne Entwicklungsumgebungen und Editoren erlauben es, die Zeichenkodierung festzulegen.

2.2 Elemente zur Seitenstrukturierung

Die ab hier folgenden Elemente gehören alle zum Inhalt eines HTML-Dokuments, finden sich also innerhalb des `<body>`-Tags wieder.

Strukturelemente dienen zur Strukturierung eines HTML-Dokuments. Sie haben alle bestimmte Bedeutungen, mit denen einzelne Teile des Dokuments gekennzeichnet werden können. Ihnen allen gemeinsam ist die Tatsache, dass sie keine visuelle Änderung des Dargestellten bewirken, sondern lediglich zur logischen Gliederung dienen.

2.2.1 Inhaltsbereich

Das `<body>`-Element ist das oberste Gliederungselement. Es repräsentiert den Inhaltsbereich – alles was auf der Seite sichtbar angezeigt werden soll, muss sich innerhalb dieses Elements befinden.

2.2.2 Kopfbereich

Fast alle Websites haben einen *Kopfbereich* (engl. „header“). In der Regel befindet sich dort die Überschrift und/oder das Logo, aber auch eine Navigation kann zum Kopfbereich der Seite gehören. Um diesen Bereich zu kennzeichnen, wurde das `<header>`-Element (Abschluss mit `</header>`) eingeführt. Mit diesem Tag sollen alle Elemente einer Seite umschlossen werden, die von der Struktur her zum Kopfbereich gehören. Im einfachsten Fall ist das eine Überschrift mit zusätzlicher Tagline, wie folgendes Beispiel zeigt. Visuell ändert sich standardmäßig nichts, wie Abbildung 2.1 zeigt. Als Unterüberschrift (Tagline) sollte übrigens keine `<h1>`–`<h6>` Überschriften verwendet werden, da dies eine neue (ungewollte) Gliederungsebene erzeugt, sondern ein Absatz (`<p>`). Dieser sollte zusammen mit der Überschrift mit dem Element `<hgroup>` (siehe Abschnitt 2.2.9) gruppiert werden.

```
1 <header>
2   <hgroup>
3     <h1>John Doe's Blog</h1>
4     <p>Gedanken eines Namenlosen</p>
5   </hgroup>
6 </header>
```

Das `<header>`-Element kann aber auch in einem Teilabschnitt einer Webseite (begrenzt durch z. B. `<section>` oder `<article>`) die einleitenden Inhalte wie Abschnittsüberschriften, Inhaltsverzeichnisse oder Logos beinhalten.

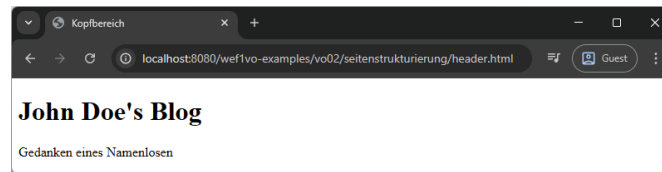


Abbildung 2.1: Überschrift und Unterüberschrift in einem `<header>`-Element verpackt. Der Browser stellt den Kopfbereich nicht explizit dar. Der Tag dient nur zur strukturellen Auszeichnung.

2.2.3 Fußbereich

Der *Fußbereich*, engl. „footer“, ist der Gegenpart zum Kopfbereich und enthält meist Informationen zu Autor*in, weiterführende Links oder Copyright-Informationen. Meist findet sich der Footer ganz am Ende eines Dokuments, jeder Abschnitt kann aber genauso seinen eigenen Fußbereich haben, wenn es nötig ist. Das HTML-Element hierfür lautet `<footer>` bzw. `</footer>`.

Das Beispiel demonstriert einen kurzen Fußbereich am Ende eines stark vereinfachten Dokuments:

```
1 <body>
2   <h1>John Doe's Blog</h1>
3   ...
4   <footer>
5     <p>&copy; 2025 by John Doe</p>
6   </footer>
7 </body>
```

2.2.4 Navigation

Auch für *Navigationen* ist in der HTML-Definition ein Element vorgesehen: `<nav>` (abgeschlossen durch `</nav>`). Es soll dazu verwendet werden, um die Hauptnavigation(en) auf der Seite zu kennzeichnen. Nicht jeder einzelne Link soll und muss in einem `<nav>`-Block stehen.

Navigationen werden üblicherweise mit Aufzählungslisten (siehe Abschnitt 2.3.6) realisiert und später entsprechend visuell angepasst. Das Markup für eine Navigation kann etwa so aussehen (mehr zu den verwendeten Hyperlinks in Abschnitt 2.4.1).

```
1 <nav>
2   <ul>
3     <li><a href="index.html">Home</a></li>
4     <li><a href="about.html">About</a></li>
5     <li><a href="projects.html">Projects</a></li>
6   </ul>
7 </nav>
```

2.2.5 Abschnitte

Eine Webpage besteht selten aus einem langen Themenblock, meist sind es mehrere thematisch ähnliche Dinge, die auf einer Seite zu finden sind. Um diesem Umstand

Rechnung zu tragen, wurde mit HTML5 das `<section>`-Element (Abschluss durch `</section>`) eingeführt. Es kennzeichnet einen inhaltlich zusammengehörigen Bereich innerhalb einer Seite. Typischerweise enthält ein *Abschnitt* eine Überschrift und dazugehörigen Text:

```
1 <section>
2   <h2>Aktuelles</h2>
3   <p>In einem vor kurzem veröffentlichten Bericht&nbsp;...</p>
4 </section>
5 <section>
6   <h2>Projekte</h2>
7   <p>Es folgt eine Liste der aktuellsten Projekte.</p>
8 </section>
```

2.2.6 Artikel

Artikel funktionieren ähnlich wie Abschnitte, sollen jedoch als eigenständiger logischer Teil funktionieren (könnten also losgelöst ohne den umgebenden Inhalt stehen). Typischerweise finden sich Artikel auf Blogs oder Websites von Zeitungen in denen sie, wie der Name schon vermuten lässt, einzelne Artikel repräsentieren. Ein Artikel wird durch `<article>` und `</article>` gekennzeichnet.

```
1 <article>
2   <header>
3     <hgroup>
4       <h2>Die Hagenberg Story</h2>
5       <p>Alles über den Studienort im Mühlviertel</p>
6     </hgroup>
7   </header>
8   <p>Es begann alles im Jahr 1993&nbsp;...</p>
9 </article>
```

2.2.7 Nebenabschnitte

In Zeitungen existieren bei Artikeln nebenbei oft Boxen mit zusätzlichem Inhalt, der nur lose im Zusammenhang mit dem eigentlichen Text steht. Dies kann z. B. bei einem Artikel über Hagenberg eine Box mit demografischen Daten über die Gemeinde sein.

Für diese Zwecke wurde in HTML5 der Tag `<aside>` (Abschluss mit `</aside>`) eingeführt. Er kann des Weiteren für Zitate aus dem Text, Werbung oder auch kleine Navigationsboxen (mit darin enthaltenem `<nav>`-Element) verwendet werden. `<aside>` dient auch für Sidebars in Layouts (mehr dazu in Vorlesung 5) oder zur Anzeige von Werbung.

Ein Beispiel für eine korrekte Verwendung, aufbauend auf dem vorigen Markup:

```
1 <article>
2   <header>
3     <hgroup>
4       <h2>Die Hagenberg Story</h2>
5       <p>Alles über den Studienort im Mühlviertel</p>
6     </hgroup>
7   </header>
8   <p>Es begann alles im Jahr 1993&nbsp;...</p>
9   <aside>
```

```

10    <p>Die Gemeinde Hagenberg hat 3.163 Einwohner*innen&nbsp;...</p>
11    </aside>
12 </article>

```

2.2.8 Überschriften

Wie bereits in Vorlesung 1 kurz erwähnt, gibt es sechs Ebenen von Überschriften, die durch die Elemente `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>` und `<h6>` (mit den jeweils abschließenden Elementen) repräsentiert werden. Diese Überschriften erzeugen inhaltliche Strukturen (die so genannte Dokumenten-Outline), wie sie auch aus Textdokumente (z. B. dieses Skriptum) bekannt und ersichtlich sind. Folgende Abfolge von Überschriften

```

1 <h1>Überschrift 1</h1>
2 <h2>Überschrift 1.1</h2>
3 <h3>Überschrift 1.1.1</h3>
4 <h2>Überschrift 1.2</h2>
5 <h2>Überschrift 1.3</h2>
6 <h1>Überschrift 2</h1>

```

erzeugt die folgende Struktur:

1. Überschrift 1
 - 1.1. Überschrift 1.1
 - 1.1.1. Überschrift 1.1.1
 - 1.2. Überschrift 1.2
 - 1.3. Überschrift 1.3
2. Überschrift 2

Strukturelemente wie `<section>`, `<article>` und dergleichen, haben keinen Einfluss auf die Outline. Die Überschriften werden in aufsteigender Reihenfolge weiterverwendet.

```

1 <h1>John Doe's Blog</h1>
2 <section>
3   <h2>Artikel zu Katzen</h2>
4   <article>
5     <h3>Eine Katze namens Nobody</h3>
6     <p>Es war einmal&nbsp;...</p>
7   </article>
8 </section>

```

Es ergibt sich dadurch folgende (erwartete) Gliederung:

1. John Doe's Blog
 - 1.1. Artikel zu Katzen
 - 1.1.1. Eine Katze namens Nobody

Aus Gründen der Accessibility sollte jede Seite mindestens eine `<h1>`-Überschrift besitzen. Es sollten in der Hierarchie keine Überschriftenebenen übersprungen werden, d. h. auf `<h1>` sollte nie direkt `<h3>` folgen.

2.2.9 Überschriftengruppen

Soll eine Überschrift um eine Unterüberschrift oder Tagline ergänzt werden, so ist es wichtig, für dieses Element *keinen* weiteren Überschrift-Tag zu verwenden. Dies würde automatisch eine Ebene in der Outline erzeugen und dies ist nicht gewünscht. Vielmehr soll ein einfacher Absatz (<p>) zum Einsatz kommen. Damit dieser jedoch semantisch zur Überschrift zuordenbar ist, werden die Überschrift und die Tagline(s) mit einem <hgroup>-Element gruppiert.

Dies war etwa bei der Überschrift des Artikels in Abschnitt 2.2.6 der Fall.

```
1 <article>
2   <h1>John Doe's Blog</h1>
3   <header>
4     <hgroup>
5       <h2>Die Hagenberg Story</h2>
6       <p>Alles über den Studienort im Mühlviertel</p>
7     </hgroup>
8   </header>
9   <p>Es begann alles im Jahr 1993&nbsp;...</p>
10 </article>
```

2.3 Elemente zur Textstrukturierung

Während die Elemente in Abschnitt 2.2 zur Gliederung der gesamten HTML-Seite verwendet wurden, kommen die folgenden Elemente zur Strukturierung von Textinhalten zum Einsatz.

2.3.1 Absätze

Das wohl am weitesten verbreitete Element zur Textstrukturierung ist der Textabsatz. Er wird mit <p> bzw. </p> erzeugt und kann neben Text wiederum eine Vielzahl anderer Elemente enthalten. Ein einfacher Absatz sieht wie folgt aus:

```
1 <p>Etwas Text in einem Absatz.</p>
```

2.3.2 Adressen

Das <address>-Element (Vorsicht, englische Schreibweise mit zwei „d“, Abschluss mittels </address>) wird dazu verwendet, *Kontaktinformationen* auf einer Seite anzugeben. Diese Kontaktinformationen können weiterführende Links, E-Mail- oder in Einzelfällen auch Postadressen sein. Es wird in [54, Abschnitt 4.3.10] ausdrücklich darauf hingewiesen, das Element ausschließlich für Kontaktadressen zu verwenden. Andere Adressen (z. B. Postadressen für eine Wegbeschreibung) sollen regulär in einen Absatz (<p>) gestellt werden. Ein Beispiel für die korrekte Verwendung mit einem Hyperlink auf eine E-Mail-Adresse als Kontaktmöglichkeit (siehe Abschnitt 2.4.1):

```
1 <address>
2   This blog is run by <a href="mailto:john.doe@johndoe.com">John Doe</a>
3 </address>
```

Eine Postadresse wäre nur dann im <address>-Element erlaubt, wenn sie die (einzig) relevante Kontaktmöglichkeit darstellt.

2.3.3 Hauptinhalt

Das `<main>`-Element (abgeschlossen durch `</main>`) kennzeichnet den *Hauptinhalt* in einem Dokument, also den Teil, in dem die wichtigsten Inhalte vorkommen. Im Gegensatz zu `<header>`, `<footer>`, `<nav>` und `<aside>` darf es nur ein einziges, sichtbares `<main>`-Element auf der Seite geben, dieses darf auch nicht innerhalb der soeben genannten Elemente vorkommen, sondern ist in der Regel direkt innerhalb des `<body>`-Elements platziert. Das Element hat auch keinen Einfluss auf die Dokumenten-Outline.

Oft gestaltet sich die Gliederung zusammen mit den anderen Strukturelementen wie folgt:

```
1 <header>...</header>
2 <nav>...</nav>
3 <main>
4   <h1>Die Geschichte von Hagenberg</h1>
5   <p>Es begann alles im Jahr 1993&nbsp;&nbsp;&nbsp;&...</p>
6 </main>
7 <footer>...</footer>
```

2.3.4 Suchbereich

Mit dem Element `<search>` (abgeschlossen durch `</search>`) wird der Bereich einer Seite gekennzeichnet, in dem eine Suche oder eine Filterung durchgeführt werden kann. Meist enthält er ein Suchfeld vom Typ `search` (mehr zu Formularen in Abschnitt 2.6). Die Suchergebnisse selbst sollten *nicht* in diesem Bereich sondern im Hauptbereich der Seite (z. B. in einem `<output>`-Element) dargestellt werden. Das Element ändert nichts am Aussehen der Seite, aber hilft Tools zur Barrierefreiheit, Suchbereiche eindeutig zu identifizieren.

Ein Suchfeld in einem Suchbereich kann wie folgt aussehen:

```
1 <search>
2   <form action="..." method="get">
3     <label for="suche">Suche:</label>
4     <input type="search" id="suche">
5     <button type="submit">Suchen</button>
6   </form>
7 </search>
```

2.3.5 Allgemeiner Bereich

Die allgemeinste Form eines Bereich-Elements ist das `<div>`-Element. Außer eine neue Zeile zu beginnen, hat es keine spezifische Bedeutung. Eine solche erlangt es in der Regel erst durch eine Formatierung mittels CSS. Dieses Element kann wiederum viele andere Elemente beinhalten und sollte immer dann verwendet werden, wenn sonstige gruppierende Elemente nicht passen bzw. auch die Dokumenten-Outline nicht beeinflusst werden soll. Das folgende Markup zeigt ein `<div>`-Element mit einer CSS-Klassenangabe und etwas Inhalt. Mehr zu CSS in Vorlesung 3.

```
1 <div class="catchphrase">
2   <p>John Doe: you won't even know I'm here</p>
3 </div>
```

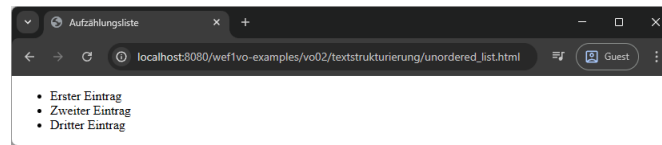


Abbildung 2.2: Eine Aufzählungsliste mit drei Einträgen.

2.3.6 Listen

Listen dienen zur Gruppierung bzw. Aufzählung von Text. Es existieren drei unterschiedliche Arten.

Aufzählungslisten

Aufzählungslisten, im Englischen „unordered lists“ oder „bullet lists“ dienen zur strukturierten Darstellung mehrerer Punkte. Jeder Eintrag wird mit einem Aufzählungszeichen (Bullet) versehen. Eine Aufzählungsliste wird mit dem `<ul>`-Element geöffnet und mit `</ul>` beendet.

Jeder einzelne Eintrag in der Aufzählungsliste (engl. „list item“) wird zwischen `<li>` und `</li>` gestellt.

Das Markup für eine Aufzählungsliste sieht somit wie folgt aus, das Ergebnis ist in Abbildung 2.2 dargestellt.

```
1 <ul>
2   <li>Erster Eintrag</li>
3   <li>Zweiter Eintrag</li>
4   <li>Dritter Eintrag</li>
5 </ul>
```

Eine semantische Alternative zu `<ul>` ist `<menu>`, welches verwendet werden kann, wenn etwa eine Toolbar mit klickbaren Buttons realisiert wird.

Nummerierte Listen

Nummerierte Listen (engl. „ordered lists“) funktionieren ähnlich wie Aufzählungslisten. Anstatt der Bullet Points sind die Einträge nummeriert, es wird das `<ol>`-Element als Beginn der Liste und das `</ol>` zum Abschluss verwendet. Einträge werden wiederum mit dem `<li>`-Element gekennzeichnet. Über die Attribute **reversed** und **start** können die Reihenfolge umgedreht bzw. der Startwert festgelegt werden.

Das Markup für eine nummerierte Liste und das Ergebnis in Abbildung 2.3:

```
1 <ol>
2   <li>Erster Eintrag</li>
3   <li>Zweiter Eintrag</li>
4   <li>Dritter Eintrag</li>
5 </ol>
```

Definitionslisten

Definitions- oder Beschreibungslisten dienen zur Beschreibung von Begriffen. Die Liste wird mit `<dl>` begonnen und durch `</dl>` abgeschlossen. Innerhalb der Liste gibt es

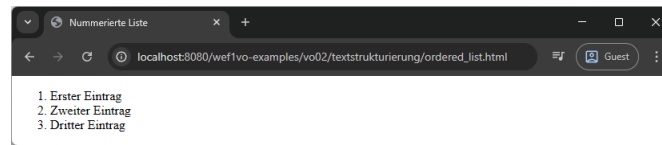


Abbildung 2.3: Eine nummerierte Liste mit drei Einträgen.

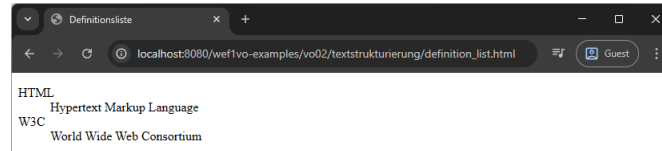


Abbildung 2.4: Eine Definitionsliste mit zwei Einträgen.

zwei Tags, die verwendet werden müssen. Der zu beschreibende Begriff (engl. „definition term“) wird mit `<dt>` `</dt>` ausgezeichnet, die Beschreibung dazu (engl. „definition description“) durch `<dd>` `</dd>`. Die Reihenfolge (`<dt>` vor `<dd>`) muss eingehalten werden.

Das Markup für eine Definitionsliste und das Ergebnis in Abbildung 2.4:

```

1 <dl>
2   <dt>HTML</dt>
3   <dd>Hypertext Markup Language</dd>
4   <dt>W3C</dt>
5   <dd>World Wide Web Consortium</dd>
6 </dl>
```

2.3.7 Vorformatierter Text

Mit dem Element `<pre>` (Abschluss mit `</pre>`) wird ein Block aus *vorformatiertem Text* angegeben. Der Text wird mit einer Festbreitenschrift dargestellt, Struktur wird durch typografische Konventionen anstatt durch Elemente definiert. So werden Leerzeichen und Tabulatoren nicht entfernt und Zeilenumbrüche durch tatsächliche Zeilenumbrüche innerhalb des Blocks erzielt.

Abbildung 2.5 stellt das folgende Beispielmarkup dar:

```

1 <pre>
2   Dieser Text   erscheint
3 wirklich
4   so   seltsam.
5 </pre>
```

Vorsicht: Wird der `<pre>`-Block etwa aus Gründen der Lesbarkeit im HTML-Quellcode mittels Tabulatoren oder Leerzeichen eingerückt, so wird dies auch bei der Ausgabe sichtbar. `<pre>` sollte daher immer uneingerückt am Anfang der Zeile stehen.

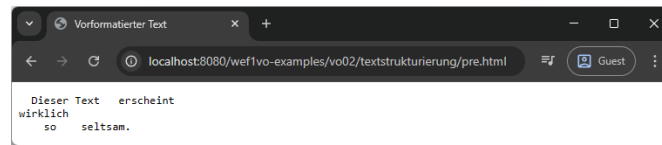


Abbildung 2.5: Vorformatierter Text. Leerräume und Zeilenumbrüche bleiben genauso erhalten, wie sie im Markup eingegeben wurden.

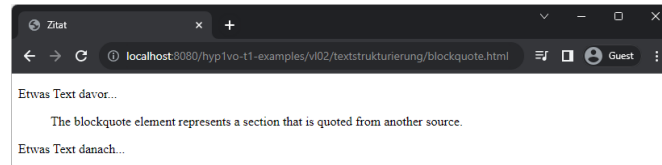


Abbildung 2.6: Das Zitat wird vom Browser standardmäßig eingerückt.

2.3.8 Horizontale Linie

Um einen thematischen Wechsel innerhalb eines Abschnittes (<section>) zu kennzeichnen, kann eine *horizontale Trennlinie* verwendet werden. Sie ist ein Standalone-Element und wird mit <hr> erzeugt. Oft wird sie zwischen zwei aufeinanderfolgenden Absätzen (<p>) platziert, es sind aber auch andere Strukturelemente wie Listen oder Bereiche (<div>) möglich. Zwischen <section>-Elementen wird in der Regel keine horizontale Linie platziert, da diese ohnehin einen thematischen Wechsel kennzeichnen.

```
1 <section>
2   <h1>Über Hagenberg</h1>
3   <p>Die Gemeinde Hagenberg hat nach wie vor eine starke ländliche Ausprägung.</p>
4   <hr>
5   <p>Die 1993 eröffnete Fachhochschule repräsentiert den modernen Teil der Gemeinde.
6   </p>
7 </section>
```

2.3.9 Zitate

Um Text aus verschiedenen Quellen, wie etwa Büchern oder Websites zu zitieren, existiert das `<blockquote>`-Element (Abschluss `</blockquote>`). Durch die Angabe des `cite`-Attributes beim Element kann die Quelle angegeben werden. Im folgenden Beispiel wird die Beschreibung des Elements aus [54, Abschnitt 4.4.4] zitiert, das Ergebnis ist in Abbildung 2.6 ersichtlich.

```
1 <p>Etwas Text davor&nbsp;&nbsp;&...</p>  
2 <blockquote cite="https://html.spec.whatwg.org/multipage/grouping-content.html#the-  
   blockquote-element">  
3   <p>The blockquote element represents a section that is quoted from another source.  
   </p>  
4 </blockquote>  
5 <p>Etwas Text danach&nbsp;&nbsp;&...</p>
```

2.4 Links und Elemente auf Textebene

Während Elemente zur Textstrukturierung einem Text eine bestimmte Bedeutung geben, arbeiten Elemente auf der Textebene direkt auf bzw. in dem Text und erfüllen dort diverse Aufgaben. Sehr wichtig sind dabei Hyperlinks, auf der Textebene arbeiten jedoch viele unterschiedliche Elemente.

2.4.1 Verweise

Verweise oder *Hyperlinks* sind das Kernstück von Hypertext – die Möglichkeit zu anderen Dokumenten zu verlinken. Zuständig in HTML ist hierfür das `<a>`-Element, welches für Anker, im Englischen „anchor“ steht. Um das Verweisziel zu spezifizieren ist immer das `href`-Attribut (für „hypertext reference“) notwendig. Das Ziel kann dabei auf verschiedene Arten angegeben werden.

Relative Links

Relative Links gehen immer vom Speicherort der HTML-Datei aus, in welcher der Link platziert ist. Bei relativen Links sind nur die Zieldatei und ggf. darüber oder darunter liegende Verzeichnisse angegeben. Innerhalb von Websites sollte prinzipiell relativ verlinkt werden, da relative Links unabhängig vom genauen Speicherort immer funktionieren.

Der folgende Link verweist auf eine Datei, die im gleichen Verzeichnis wie die aktuelle Datei liegt.

```
1 <a href="datei1.html">Ein Link</a>
```

Die folgenden zwei Beispiele verweisen auf Dateien, die einmal im Unterverzeichnis `docs` bzw. zwei Verzeichnisebenen höher liegen.

```
1 <a href="docs/datei2.html">Ein Link</a>
2 <a href="../../datei3.html">Ein Link</a>
```

Absolute Links

Absolute Links verweisen in der Regel auf eine externe Ressource. Im Gegensatz zu relativen Links sind hier das Protokoll (z.B. `http` oder `https`) und die volle Domain-Angabe vorhanden. Auch wenn auf Unterseiten der eigenen Website absolut verlinkt werden kann, empfiehlt sich dies nicht. Sollte z.B. die Domain gewechselt werden, so müssen alle Links korrigiert werden.

Die folgenden Beispiele zeigen externe HTTP- (bzw. HTTPS-) Links und einen unter Verwendung von FTP (File Transfer Protocol). Anmerkung: moderne Browser haben die FTP-Unterstützung aus Sicherheitsgründen entfernt.

```
1 <a href="http://speedtest.tele2.net/">HTTP-Link</a>
2 <a href="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/a">
  HTTPS-Link</a>
3 <a href="ftp://ftp.cert.org/">FTP-Link</a>
```

Relative Links mit führendem Slash: root-relative Links

Wird einem relativen Link ein Schrägstrich vorangestellt, wird dieser zu einem root-relativen Link, der sich auf die unterste (Root-) Ebene der Verzeichnisstruktur bezieht. Angenommen in einer Datei `https://johndoe.com/blog/index.html` befinden sich die folgenden Links:

- `<a href="media/projekte.html">Projekte</a>` verweist auf den URL `https://johndoe.com/blog/media/projekte.html`, denn es ist ein relativer Link, der ausgehend vom Verzeichnis `blog` noch eine Ebene tiefer ins Verzeichnis `media` geht und dort auf `projekte.html` verweist.
- `<a href="/media/projekte.html">Projekte</a>` verweist hingegen auf `https://johndoe.com/media/projekte.html`, denn der führende Slash geht aus vom Root der Seite (`https://johndoe.com/`) aus und von dort weiter ins Unterverzeichnis `media` mit der darin enthaltenen Datei `projekte.html`.

E-Mail-Adressen

Um einen Link auf eine *E-Mail-Adresse* zu setzen, wird diese unter Voranstellung des `mailto:-`Pseudoprotokolls in das `href`-Attribut eines Ankers gesetzt. Ein einfacher E-Mail-Link kann daher folgendermaßen erzeugt werden:

```
1 <a href="mailto:john.doe@johndoe.com">Send an e-mail!</a>
```

Ein Klick auf den Link öffnet in der Regel das im Betriebssystem definierte E-Mail-Programm und trägt die Adresse in das „An:“ Feld ein.

Zusätzliche Angaben bei E-Mail-Links

Zusätzlich können auch noch weitere Felder vorausgefüllt werden. Um weitere Daten mitzugeben, muss hinter die Adresse ein `?` gestellt werden. Dies zeigt an, dass Parameter im Format `name=wert` folgen. Auch mehrere solcher Parameter sind möglich, diese müssen mit `&` voneinander getrennt werden. Da dieses Zeichen in HTML aber reserviert ist, muss die entsprechende Entität, also `&`; stattdessen verwendet werden.

Folgende Parameter können mit dieser Syntax mitgegeben werden:

- **to:** Der*die primäre Adressat*in. Diesen Parameter anzugeben ist nicht nötig, er wird ohnehin mit der E-Mail-Adresse befüllt.
- **cc:** Ein*e Adressat*in im CC-Feld.
- **bcc:** Ein*e Adressat*in im BCC-Feld (wird von Empfänger*innen nicht gesehen).
- **subject:** Die Betreffzeile des E-Mails.
- **body:** Der Textbereich (Inhalt) des E-Mails.

Leerzeichen müssen ersetzt („escaped“) werden. Dies geschieht durch die Verwendung von `%20`.

Mit folgendem Markup wird ein E-Mail mit vorgegebenem Text und Betreff an eine Person und an eine weitere im CC geschickt:

```
1 <a href="mailto:john.doe@johndoe.com?cc=survey@johndoe.com&amp;subject=Feedback%20to%20your%20site&amp;body=Dear%20John...">Send Feedback!</a>
```

Lokale Dateien

Es ist auch möglich, direkt auf *Dateien im Dateisystem* zu verlinken. Dazu wird der Pfad im Dateisystem angegeben, vorangestellt wird das `file:///` Protokoll. Die einzelnen Ordnerhierarchien sind mit Schrägstrichen (Slashes), nicht Backslashes zu trennen.

Ein Beispiel für eine Verlinkung ins Dateisystem:

```
1 <a href="file:///C:/Dokumente/HTML/index.html">Ein File-Link</a>
```

Vorsicht: Derartige Links funktionieren in der Regel nur lokal auf ein und demselben Rechner. Wird die HTML-Datei, in der der Link enthalten ist, auf einem Webserver gehostet oder auf einem anderen Computer geöffnet, verweist sie auf einen Pfad, der so auf diesem Rechner wahrscheinlich nicht existieren wird. Daher sollten solche Links nur in Ausnahmefällen verwendet werden.

Interne Links

Auch innerhalb einer HTML-Datei kann mittels Hyperlinks navigiert werden. Zunächst muss an einer Stelle ein Anker definiert werden. Dies geschieht wiederum mit dem `<a>`-Element und mit Hilfe des `id`-Attributs. Dieses ist ein globales Attribut, das einem Element eine eindeutige Bezeichnung gibt. Der Wert des Attributs kann beliebig gewählt werden.

Um diesen Anker „anzuspringen“, wird wiederum das `<a>`-Element, diesmal in Kombination mit dem bekannten `href`-Attribut verwendet. Der Wert ist der Name des zuvor erstellten Ankers mit vorangestelltem `#`-Zeichen.

Ein Beispiel für einen *internen Link*, der auf einen Anker zeigt:

```
1 <h1><a id="anfang">John Doe's Blog</a></h1>
2 ...
3 <p><a href="#anfang">Zum Anfang springen</a></p>
```

Um einen internen Anker zu definieren, muss jedoch nicht zwingend ein `<a>`-Element mit einem `id`-Attribut zum Einsatz kommen. Vielmehr kann letzteres auch direkt beim anzuspringenden Element hinzugefügt werden. Das obige Beispiel kann (und sollte der Einfachheit halber) auch wie folgt realisiert werden:

```
1 <h1 id="anfang">John Doe's Blog</h1>
2 ...
3 <p><a href="#anfang">Zum Anfang springen</a></p>
```

2.4.2 Weitere Elemente auf Textebene

Zur Formatierung von Text gibt es eine Reihe von Elementen, von denen im Folgenden der Großteil kurz aufgelistet und erklärt werden. Eine vollständige Erklärung findet sich im HTML Living Standard².

: Dieses Element wird verwendet, um Textstellen zu betonen (engl. „emphasize“). Browser stellen dies standardmäßig kursiv dar.

: Mit diesem Element wird die hohe Wichtigkeit einer Textstelle gekennzeichnet. Im Normalfall wird diese durch Fettschrift dargestellt.

<small>: Wird verwendet, um Textstellen als „Kleingedrucktes“ zu markieren. Beispiele sind etwa Disclaimer oder Copyright-Richtlinien. Dieses Element sollte nicht dazu verwendet werden um Text kleiner zu schreiben (auch wenn es Browser standardmäßig kleiner rendern), es ist auch nicht das Gegenteil von ****.

²<https://html.spec.whatwg.org/multipage/text-level-semantic.html>

<s>: Dieses Element kennzeichnet Textpassagen, deren Inhalte nicht mehr zeitgemäß oder relevant sind. Browser rendern dies standardmäßig durchgestrichen.

<cite>: Kann verwendet werden, um Titel von Werken wie Büchern oder wissenschaftlichen Papers zu kennzeichnen. Es wird oft gemeinsam mit dem **<blockquote>**-Element verwendet.

<q>: Dieses Element kennzeichnet ein kurzes Zitat oder Teile eines Zitats im Textfluss. Wie auch bei **<blockquote>** kann das **cite**-Attribut zur Quellenangabe verwendet werden.

<dfn>: Dieses Element kennzeichnet einen Ausdruck, der näher beschrieben werden soll (z.B. eine Abkürzung). Die Erklärung dazu kann u.a. mit dem **title**-Attribut angegeben werden.

<abbr>: Mit diesem Element können Abkürzungen (engl. „abbreviations“) oder Akronyme erläutert werden. Die Erklärung des markierten Ausdrucks soll im **title**-Attribut festgehalten werden.

<data>: Dieses Element erlaubt es, zu Inhalten noch maschinenlesbare Informationen zu speichern, die für Benutzer*innen selbst nicht relevant sind. Diese werden mit dem **value**-Attribut angegeben. So etwa für einen Benutzer*innennamen und dazugehöriger ID: `<data value="jdoe">John Doe</data>`.

<time>: Wird verwendet, um Datum oder Uhrzeit anzugeben. Der Inhalt kann ein beliebiges, menschenlesbares Format sein, mit dem Attribut **datetime** kann eine maschinenlesbare Version hinterlegt werden.

<code>: Dieses Element kann verwendet werden, um Code-Fragmente auszuzeichnen. Oft wird es zusätzlich von einem **<pre>**-Element umschlossen.

<var>: Mit diesem Element können mathematische Variablen im Textfluss gekennzeichnet werden.

<samp>: Wird verwendet, um die Ausgabe (engl. „sample output“) von Computerprogrammen zu kennzeichnen.

<kdb>: Kennzeichnet Input von Benutzer*innen über das Keyboard oder andere Eingabegeräte.

<sub> und <sup>: Werden für Tief- und Hochstellung von Text verwendet. Dabei ist es wichtig, die Elemente nur dann zu verwenden, wenn Sub- oder Superscript Teil einer typographischen Konvention ist und dies dadurch zum Ausdruck gebracht werden muss (z. B. um „M^{me}“, die Kurzschreibweise für das französische Wort „Madame“ korrekt zu bezeichnen, oder bei mathematischen Formeln).

<i>: Wird benützt, um Text in einer anderen Stimmung oder anderen Sprache zu kennzeichnen. Bei letzterem sollte das **lang**-Attribut verwendet werden, um die Sprache anzugeben. Browser stellen dies für gewöhnlich kursiv dar.

: Hebt Text hervor, ohne ihn dabei extra zu betonen. Dies kann verwendet werden, um bestimmte Schlüsselwörter einfach hervorzuheben, ohne dabei ihre Bedeutung zu sehr zu steigern. Dieses Element wird nur selten benötigt, meist ist **** die bessere Alternative. Dieses Element wird von Browsern in der Regel fett gerendert.

<u>: Dieses Element soll dazu verwendet werden, um eine Anmerkung zu einem Textteil auszudrücken, die jedoch nicht ausdrücklich erwähnt wurde. Ein Beispiel wäre die Kennzeichnung eines falsch geschriebenen Wortes – dieses würde man auf Papier etwa unterstreichen. Auch Browser stellen das **<u>**-Element unterstrichen dar. Aufgrund der leichten Verwechslung mit Hyperlinks sollte es sparsam bis gar nicht eingesetzt werden.

<mark>: Dieses Element kann benützt werden, um Teile im Text hervorzuheben, d. h. zu markieren, um die Aufmerksamkeit der Leser*innen auf diese Stelle zu richten. Es handelt sich quasi um einen mit dem Textmarker gekennzeichneten Bereich – Browser stellen sie analog dazu gelb hinterlegt dar.

: Dieses Element ist ein allgemeines Markup-Element ohne spezielle Bedeutung. Es verhält sich ähnlich wie **<div>** wird aber im Fließtext verwendet und nicht als eigener Bereich.

2.5 Bilder

Bilder sind zentraler Bestandteil einer Webseite. Mit den modernen Mitteln der neuesten HTML-Versionen ist deren Handhabung im Browser sehr flexibel geworden.

2.5.1 Bilder einfügen

Um *Bilder* in ein HTML-Dokument einzubinden, wird das ****-Element verwendet. Es ist ein Standalone-Element und besitzt mehrere wichtige Attribute, von denen **src** und **alt** verpflichtend verwendet werden müssen. **src** spezifiziert den Speicherort des anzuzeigenden Bildes (Quelle), mit **alt** wird ein Alternativtext spezifiziert, der angezeigt wird, wenn das Bild nicht geladen werden kann bzw. ausgegeben wird, wenn Personen mit visueller Beeinträchtigung die Website von einem Screenreader vorgelesen bekommen.

Das einfachste Markup, um ein Bild einzufügen, sieht also folgendermaßen aus. Das Ergebnis ist in Abbildung 2.7 dargestellt.

```
1 
```

Es empfiehlt sich, beim Einbinden des Bildes auch dessen Größe mitanzugeben. Dies ist mithilfe der Attribute **width** und **height** möglich. Diese Attribute sind zwar nicht zwingend notwendig, denn der Browser kann auch aus den Bilddaten die Dimensionen auslesen. Bekommt er diese Information aber bereits vorab, kann er beim Aufbau/Laden

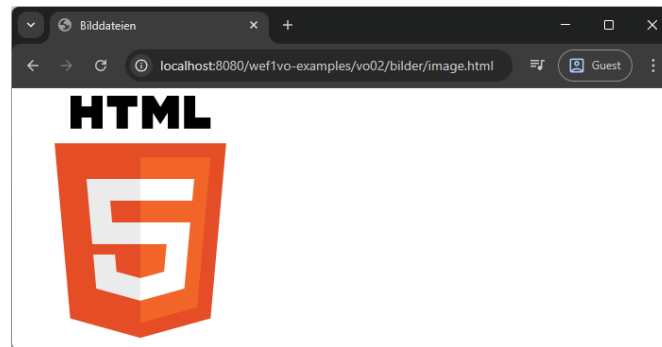


Abbildung 2.7: Die im `src`-Attribut angegebene Bilddatei `html5logo-small.png` wird angezeigt. Der Inhalt des `alt`-Attributs ist nicht sichtbar.

der Seite bereits den exakten Platz für das Bild reservieren. Die beiden Attribute sollten nicht zur Skalierung von Bildern verwendet werden sondern immer die tatsächliche Größe angeben.

Das Markup mit entsprechenden Längen- und Breitenangaben in Pixel (eine Angabe von Einheiten oder Prozent ist nicht möglich). Das Ergebnis sieht nach wie vor wie in Abbildung 2.7 aus.

```
1 
```

2.5.2 Pixeldichte und Auflösung berücksichtigen

Es besteht die Möglichkeit, im `<img>`-Element nicht nur ein einziges Quellbild anzugeben, sondern Alternativen, die dem Browser zur Auswahl zur Verfügung stehen – je nachdem auf welchem Display der Inhalt angezeigt wird oder wie groß der Anzeigebereich (Viewport) ist. Verantwortlich dafür ist das `srcset`-Attribut, das für diese Zwecke verschieden aussehen kann.

Angaben für Pixeldichte

Um etwa für hochauflösende Displays (wie das Retina-Display von Apple) entsprechend besser aufgelöste Grafiken zur Verfügung zu stellen, können im `srcset`-Attribut mehrere Grafikdateien durch Komma getrennt angegeben werden. Nach dem Namen der Dateien erfolgt die Angabe eines *Pixeldichte*-Werts. *1x* steht für reguläre Displays, *2x* für hochauflösende Displays. Diese Angaben können auch noch weiter erhöht werden. Die Angabe von `src` bleibt weiterhin und dient als Fallback, falls der Browser das neue Attribut noch nicht kennt.

Ein Bild in zwei Größen kann dann wie folgt eingebunden werden, der Browser wählt automatisch das passendere Bild:

```
1 
```


Angaben für Auflösungen

Die Angabe der Pixeldichte allein ist jedoch in vielen Fällen unflexibel. Oft möchte man je nach aktueller Größe des Browserfensters (oder auf mobilen Geräten des Viewports) das passende Bild anzeigen, d. h. mehrere Versionen in verschiedenen *Auflösungen* hinterlegen. Hierbei wird im **srcset**-Attribut wieder eine kommagetrennte Liste mit Grafikdateien angegeben, danach folgt aber jeweils die Breite des Bildes in der *w*-Syntax. Ist dieses z. B. 512px Breit, wird 512*w* geschrieben etc.

Für das folgende Bild liegen vier Größen vor, der Browser wählt die passende aus. Wie genau dies passiert, obliegt dem Browser. Google Chrome wählt etwa bei lokal geöffneten Seiten immer das größte Bild.

```
1 
```

Angabe der Größe je nach Auflösung

Die soeben gezeigte Methode hat jedoch den Effekt, dass das jeweils ausgewählte Bild (ohne weitere Angaben) auf die maximal mögliche Breite skaliert wird. Um eine tatsächliche *Größe* anzugeben, kann zusätzlich mit dem **sizes**-Attribut gearbeitet werden. Dieses kann, je nach Viewport-Breite verschiedene Größen setzen. Zunächst kann eine Angabe eine Breitenangabe in Klammern gesetzt werden, danach wird die Größe spezifiziert. Weitere, durch Komma getrennte Angaben sind möglich.

Im folgenden Beispiel wird bis zu eine Viewport-Breite von 600 Pixeln das Bild auf 100 % des Viewports skaliert (100vw – mehr dazu in Vorlesung 3), bis zu einer Breite von 1.200 Pixeln auf 75 % des Viewports und ab dieser Breite wird eine fixe Größe von 1024 Pixeln eingestellt.

```
1 
```

2.5.3 Art Direction mit <picture>

Während das **srcset**-Attribut beim ****-Element die Funktion hat, verschiedene Versionen desselben Bildes je nach Auflösung oder Display anzubieten, gibt es auch noch einen weiteren Anwendungsfall, der damit nur schlecht berücksichtigt werden kann: *Art Direction*. Dieser Begriff bezeichnet den Umstand, dass bei verschiedenen Auflösungen nicht nur unterschiedliche Größen von Bildern zum Einsatz kommen, sondern diese sich auch inhaltlich unterscheiden können. So können bei breiten Anzeigen entsprechend breite Bilder verwendet werden, bei schmalen Viewports (z. B. am Handy) kann eine umgestaltete, schmale Version zum Einsatz kommen.

Dies kann mit dem **<picture>**-Element erreicht werden. Dieses ist ein Container für verschiedene Versionen, diese werden mit mehreren **<source>** Elementen angegeben. Über das Attribut **media** kann das Kriterium angegeben werden, wann dieses Element zum Einsatz kommt, **srcset** spezifiziert die zu verwendende Datei. Die Optionen im Bezug auf Auflösung und Pixeldichte können ebenso wieder angegeben werden.

Im folgenden Beispiel wird ab einer Breite von 990 Pixeln ein HTML5 Sticker angezeigt. Für alle schmälere Viewports kommt das reguläre HTML5 Logo zum Einsatz.

```
1 <picture>
2   <source media="(min-width: 990px)" srcset="html5sticker.png 1800w">
3   
4 </picture>
```

Einsatz verschiedener Grafikformate

Ebenso kann `<picture>` verwendet werden, um den Browser aus mehreren *Grafikformaten* wählen zu lassen. So kann mit dem `type`-Attribut etwa eine Vektorgrafik und eine Pixelgrafik angegeben werden. Unterstützt der Browser erstere, wird sie angezeigt, ansonsten auf die Pixelgrafik ausgewichen. Das folgende Markup stellt dies dar:

```
1 <picture>
2   <source srcset="html5logo.svg" type="image/svg+xml">
3   
4 </picture>
```

2.5.4 Abbildungen

In traditionellen Printpublikationen werden Bilder in der Regel mit Beschreibungen (engl. „captions“) versehen. Ein Beispiel sind die Abbildungen in diesem Skriptum. Seit HTML5 ist dies auch für Webseiten möglich. Die dafür verantwortlichen Elemente sind `<figure>` (mit dem abschließenden `</figure>`) und `<figcaption>` (mit `</figcaption>` als Abschluss).

`<figure>` wird verwendet, um eine Abbildung einzuleiten. Alles innerhalb dieses Elements gehört zu dieser Abbildung. In der Regel wird hier ein `<img>`-Element stehen, aber auch Text wie z. B. Quellcode kann in einer Abbildung platziert werden. Um der dieser eine Beschreibung zu geben, wird ebenfalls noch innerhalb das `<figcaption>`-Element platziert. Wichtig: Diese Beschreibung ersetzt keinesfalls das `alt`-Attribut beim Bild selbst.

Es folgt das Beispiel von vorher, jedoch mit einer Beschreibung in einer Abbildung platziert. Das Ergebnis ist in Abbildung 2.8 ersichtlich.

```
1 <figure>
2   
3   <figcaption>Das offizielle HTML5-Logo. Quelle: <a href="https://www.w3.org/html/
   logo/">W3C</a></figcaption>
4 </figure>
```

Bildformate im Web

Im Web haben sich mittlerweile fünf Bildformate für Pixelbilder etabliert, die von allen modernen Browsern unterstützt werden: GIF, PNG, JPEG, WebP und AVIF. Letzteres funktioniert seit Ende Jänner 2024 allen wichtigen Browsern. Neben Pixelbildern unterstützen moderne Browser auch Vektorbilder. Im Web hat sich dabei SVG als Format durchgesetzt.

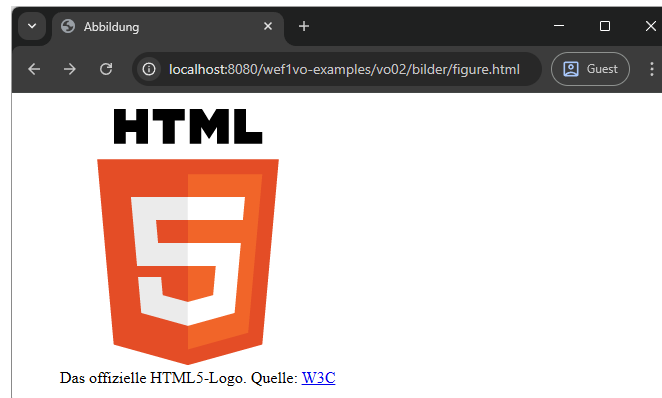


Abbildung 2.8: Die Bilddatei, verpackt in einer Abbildung mit beschreibendem Text.

GIF: Das *Graphics Interchange Format* mit der Dateiendung `.gif` ist ein verlustfrei komprimiertes Bildformat, das 256 Farben darstellen kann. Es unterstützt Transparenzen, Animationen und Interlacing (schichtweisen Aufbau). GIF eignet sich gut für kleine Grafiken wie Buttons oder simple Farbelemente ohne Verläufe.

PNG: Das *Portable Network Graphics* Format mit der Endung `.png` wurde speziell für den Einsatz im Web konzipiert. Es unterstützt 16,7 Millionen Farben und komprimiert ebenfalls verlustfrei. Darüber hinaus sind Transparenz und Interlacing möglich. PNG ist der moderne Nachfolger von GIF, der für die selben Einsatzzwecke gedacht und mit der höheren Farbtiefe auch flexibler ist.

JPEG: JPEG steht für *Joint Photographic Expert Group* und bezeichnet somit das Konsortium, das das Format entwickelt hat. JPEG (Dateiendungen `.jpg` oder auch `.jpeg`) verwendet ein verlustbehaftetes Kompressionsverfahren und unterstützt 16,7 Millionen Farben. JPEG ist sehr gut für Fotografien geeignet, mit dem Kompressionsfaktor lässt sich die Qualität des Bildes in Abhängigkeit von der Dateigröße steuern. Transparenzen werden nicht unterstützt, Interlacing liegt im so genannten Progressive-Verfahren vor.

WebP: WebP [21] – ausgesprochen als *weppy* – ist ein von Google aus dem VP8-Videocodex entwickeltes Format. Es unterstützt sowohl verlustbehaftete als auch verlustfreie Kompression. Bei erster werden laut eigenen Angaben bessere Bildqualitäten bei gleicher Dateigröße als bei JPEG erzielt. WebP wird in allen aktuellen Browsern unterstützt, auch Adobe Photoshop unterstützt das Format nativ. Die Dateiendung lautet `.webp`. Da WebP den anderen drei Formaten in quasi allen Facetten überlegen ist, sollte es mittlerweile als das Standard-Format für Pixelbilder im Web angesehen werden.

AVIF: Das AV1 Image File Format (AVIF) [2] – Dateiendung `.avif` – ist ein offenes Grafikformat für Rastergrafiken, basierend auf dem AV1 Videocodex. Es kann bis zu 25 % kleinere Dateien als WebP in der verlustbehafteten Kompressionsmethode produzieren. Weiters existiert auch eine verlustfreie Version. Während es kleinere Dateien als WebP produziert, benötigen diese oft etwas länger zum Dekodieren, was die Ladezeiten nicht immer dramatisch erhöht.

SVG: *Scalable Vector Graphics* [100] (mit der Endung `.svg`) ist ein zweidimensionales Vektorformat, das auf der Beschreibungssprache XML (siehe Vorlesung 7) basiert. Die Unterstützung für SVG-Bilder im Browser ist seit Jahren sehr gut. Alle wichtigen Browser ermöglichen das Einbinden von SVG-Bildern mittels `<img>`-Tag, auch als CSS-Hintergründe funktionieren SVG-Bilder bereits zuverlässig.

2.5.5 Lazy Loading von Bildern

Wird ein Bild mit dem `<img>`-Tag eingebunden, so fordert der Browser diese externe Ressource (das Bild) sofort an, d. h. er lädt es sofort herunter und zeigt es alsbald an. Dieses Standardverhalten nennt man auch *eager Loading* und ist für Elemente, die sofort auf der Seite sichtbar sein sollen, gewünscht, um Verzögerungen zu vermeiden.

Oft gibt es jedoch Bilder, die erst später, d. h. weiter unten auf einer Seite, benötigt werden. Diese werden erst angezeigt, wenn der*die User*in auch in diesen Bereich scrollt. Da nicht sicher ist, ob dies überhaupt passiert, ist eager Loading hier unter Umständen kontraproduktiv, da Bilder geladen und übertragen werden, die nie für den*die User*in sichtbar werden.

Aus diesem Grund kann das Attribut `loading` mit dem Wert `lazy` beim `<img>`-Tag verwendet werden. In diesem Fall beginnt der Browser erst mit dem Laden des Bildes, wenn der*die User*in in dessen Nähe scrollt.

Das Markup dazu sieht wie folgt aus:

```
1 
```

Achtung: Lazy Loading sollte nicht für Bilder verwendet werden, die sofort im Viewport sichtbar, d. h. am Beginn der Seite, sind. Ebenso muss JavaScript aktiviert sein, damit lazy Loading funktioniert.

2.6 Formulare

Formulare sind interaktive Komponenten einer Webpage, die es Benutzer*innen ermöglichen, Daten aktiv einzugeben. Ein Formular besteht aus verschiedenen Formularelementen (engl. „form controls“) wie z. B. Textfelder, Buttons, Checkboxes oder Slider. Die Erstellung eines Formulars umfasst zunächst die Erstellung des User-Interfaces, in dem eben jene Elemente platziert werden und in weiterer Folge die Weiterverarbeitung der eingegebenen Daten. Diese Weiterverarbeitung geschieht meist serverseitig, Formulardaten können jedoch auch ganz einfach in ein E-Mail verpackt werden.

2.6.1 Ein Formular erstellen

Jedes Formular beginnt mit dem `<form>`-Element und wird mit `</form>` abgeschlossen. Alle Formularelemente werden innerhalb davon platziert.

`<form>` besitzt eine Reihe von Attributen, die bestimmen, auf welche Art die eingegebenen Daten weiterverarbeitet werden. Die wichtigsten sind im Folgenden aufgelistet, die vollständige Liste findet sich in [54, Abschnitt 4.10.3].

- **action:** Gibt das Ziel des Formulars an. Kann als Wert einen URL beinhalten oder auch die Angabe einer E-Mail-Adresse mit `mailto:.` Beim Absenden des Formulars wird dieses Ziel dann aufgerufen.
- **method:** Gibt die Übertragungsmethode an. Zur Auswahl stehen die Werte `get` und `post`. Diese Unterscheidung ist derzeit noch nebensächlich, als Methode kann einfach `post` verwendet werden. Details dazu folgen im zweiten Semester, wenn es um serverseitige Weiterverarbeitung von Formularen geht.
- **enctype:** Gibt die Kodierung der eingegebenen Daten zur Übertragung in Form eines so genannten MIME-Types an. Es sind drei verschiedene Werte vorgesehen:

`application/x-www-form-urlencoded`, `multipart/form-data` und `text/plain`. Ersteres ist der Standardwert, der die Daten in einer zusammengehängten URL-Form weitergibt. Der zweite Typ ist beim Verarbeiten von im Formular hochgeladenen Dateien wichtig. Der letzte Typ gibt Daten als Klartext lesbar aus. Diese Option sollte beim E-Mail-Versand gewählt werden.

- **name**: Der Name des Formularfelds. Dieser kann optional angegeben werden und ist meist nur dann sinnvoll, wenn auf das Formular mittels einer Skriptsprache wie JavaScript oder PHP zugegriffen werden soll.
- **autocomplete**: Dieses Attribut bestimmt, ob die browserinterne Autovervollständigung aller Eingabefelder im Formular (Liste mit vorgeschlagenen Wörtern, die bereits einmal eingegeben wurden) aktiviert ist, oder nicht. Es kann die Werte `on` oder `off` haben, wobei ersteres nicht angegeben werden muss, da die Autovervollständigung standardmäßig aktiviert ist.
- **novalidate**: Dieses Standalone-Attribut (ohne Wertangabe in Anführungszeichen) gibt an, dass das Formular vom Browser nicht auf Gültigkeit überprüft werden soll. Mehr dazu in Abschnitt 2.6.8.

Das Grundgerüst für ein Formular, das per E-Mail übertragen werden soll, sieht wie folgt aus. Es alleine besitzt noch keine visuelle Repräsentation sondern markiert nur einen Bereich als Formular.

```
1 <form action="mailto:john.doe@johndoe.com" method="post" enctype="text/plain">
2   ...
3 </form>
```

2.6.2 Formularfelder beschriften

Bevor die verschiedenen Elemente zur Dateneingabe – die eigentlichen Formularfelder – besprochen werden, sei erwähnt, dass jedes solche Feld auch eine Beschriftung benötigt. Während ein neben ein Eingabefeld platzierter Absatz natürlich visuell ausreichen würde, gibt es ein eigens dafür bestimmtes Element, das auch eine semantische Zuordnung zwischen *Beschriftung* und Eingabefeld ermöglicht: `<label>`.

Um die Zuordnung eindeutig zu ermöglichen, besitzt das Element ein Attribut `for`, dessen Wert mit dem `id`-Attribut des Eingabefeldes übereinstimmen muss. `id` ist ein Universalattribut, das einem Element einen eindeutigen Namen gibt – ideal für die eindeutige Zuordnung einer Beschriftung. Die meisten Browser setzen beim Klick auf die Beschriftung den Cursor in das zugeordnete Eingabefeld.

In den nachfolgenden Beispielen zu den verschiedenen Formularfeldern ist jeweils bereits eine Beschriftung beigelegt.

2.6.3 Eingabefelder

Eingabefelder (engl. „input fields“) sind die zentralen Elemente zur Dateneingabe in Formularen. Es handelt sich dabei um ein einzeiliges, typisiertes Datenfeld, d.h. dasselbe HTML-Element, nämlich `<input>` wird für eine Vielzahl an verschiedenen Anwendungsfällen verwendet und durch das `type`-Attribut näher bestimmt. Es handelt sich dabei um ein Standalone-Element. Eingabefelder müssen innerhalb des `<form>`-Elements platziert werden.



Abbildung 2.9: Zwei identische Textfelder. Die Angabe von `type="text"` ist in diesem Fall also optional. Eingabefelder und Label sind Phrasing Content und erzeugen keine neue Zeile pro Element, der Zeilenumbruch wurde mittels `<br>`-Tag herbeigeführt. In das erste Feld wurde zur Veranschaulichung Text eingegeben, die Beschriftung erfolgte in beiden Fällen mittels `<label>`-Element.

Es gibt eine Reihe von Attributen, die bei den meisten Feldern anwendbar sind. Einige wichtige sind im Folgenden aufgelistet, weitere werden im Verlauf des Abschnitts noch diskutiert, eine komplette Übersicht bietet der HTML Living Standard [54, Abschnitt 4.10.5].

- **value:** Erlaubt es, den Wert eines Eingabefeldes bereits vorab zu definieren (vor auszufüllen). Bei Checkboxes und Radiobuttons kann mit **value** angegeben werden, welcher Wert übertragen werden soll, wenn die Checkbox ausgewählt wurde.
- **size:** Bestimmt die Länge eines Eingabefeldes, also wie lange es optisch aussieht. Die Zahl entspricht der Zeichenanzahl, die im Formularfeld Platz findet, bevor der Text horizontal innerhalb des Feldes zu scrollen beginnt.
- **maxlength:** Der Wert dieses Feldes bestimmt, wie viel Zeichen tatsächlich in das Eingabefeld geschrieben werden können. Diese Angabe muss nicht mit **size** übereinstimmen.
- **readonly:** Dieses Standalone-Element verhindert, dass mit einem Formularelement interagiert werden kann. Readonly-Elemente werden von Browsern in der Regel ausgegraut angezeigt.

Im Folgenden werden exemplarisch drei Varianten (**text**, **password** und **email**) des einzeiligen Eingabefeldes besprochen. Weitere Typen sind **hidden**, **search**, **tel**, **url**, **date**, **month**, **week**, **time**, **datetime-local**, **number**, **range**, **color** und **file**. Details zu diesen Typen finden sich in [54, Abschnitt 4.10.5].

Text

Textfelder werden mit dem Wert **text** für das **type**-Attribut des `<input>`-Elements erzeugt. In diesem Fall kann das Attribut aber auch ganz entfallen, da **text** der Standardwert für Eingabefelder ist, wenn nichts angegeben wird.

Um ein Textfeld zu erzeugen, führen somit sowohl das folgende Markup in Zeile 2 als auch das in Zeile 4 zum (selben) Ziel. Das Ergebnis ist in Abbildung 2.9 zu sehen.

```
1 <label for="with">Input mit Attribut:</label>
2 <input type="text" id="with"><br>
3 <label for="without">Input ohne Attribut:</label>
4 <input id="without">
```

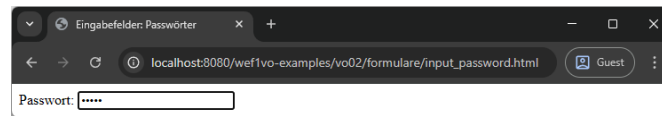


Abbildung 2.10: Ein Passwort-Eingabefeld. Eingebener Text wird vom Browser maskiert.

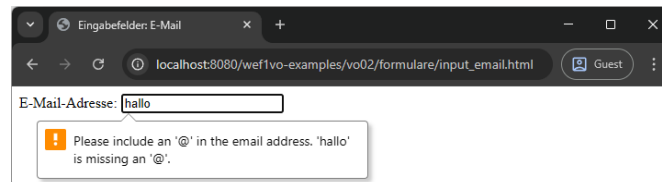


Abbildung 2.11: Ein Eingabefeld vom Typ E-Mail in Verwendung. Im Browser wird bei Eingabe falscher Daten (keine E-Mail-Adresse) das Absenden des Formulars verhindert.

Passwörter

Passwortfelder haben den Typ `password`. Sie funktionieren wie Textfelder, die eingegebenen Zeichen werden jedoch vom Browser nicht angezeigt, sondern durch Platzhalter ersetzt.

Das Markup für ein Passwortfeld sieht wie folgt aus, das Ergebnis (mit einem eingetippten Wort) ist in Abbildung 2.10 zu sehen.

```
1 <label for="pass">Passwort:</label>
2 <input type="password" id="pass">
```

E-Mail-Adressen

Der Formulartyp für *E-Mail-Adressen* unterscheidet sich auf einem Desktop-Browser meist visuell nicht von einem Textfeld, bringt jedoch andere Vorteile mit. So führen alle modernen Browser beim Absenden des Formulars eine inhaltliche Überprüfung für Felder vom Typ E-Mail durch. Wird ein Wort eingegeben, dessen Format keiner E-Mail-Adresse entspricht, so verhindert der Browser das Abschicken des Formulars und gibt eine Fehlermeldung aus (Abbildung 2.11). Auf Geräten mit virtueller Tastatur, wie etwa Smartphones, bewirkt das Auswählen eines E-Mail-Feldes eine Umstellung der Tastatur. So werden die für E-Mail-Adressen relevanten Zeichen „@“ und „.“ auf dem Primärkeyboard eingeblendet).

Wird das Attribut `multiple` angegeben, erlaubt das Feld die Angabe von mehreren E-Mail Adressen, die mit Komma getrennt sind. Dies kann nützlich sein, wenn etwa eine Benutzeroberfläche für ein Webmail Programm erstellt wird. Die „To“, „CC“ oder „BCC“ Felder könnten dann mehrere Adressen erlauben.

Das Markup für ein einfaches E-Mail Feld sieht wie folgt aus:

```
1 <label for="e-mail">E-Mail-Adresse:</label>
2 <input type="email" id="e-mail">
```

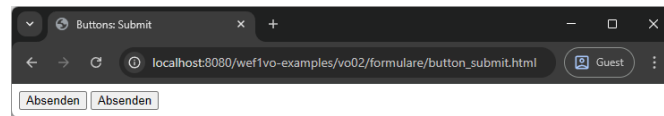


Abbildung 2.12: Zwei Submit-Buttons mit derselben Funktionalität und demselben Aussehen – einmal mittels `<button>`, das andere Mal mittels `<input>`-Element.

Autofill bei E-Mail-Feldern: Durch verschiedene Werte beim `autocomplete`-Attribut (anstelle von `on` und `off`) ist für den Browser eine Unterscheidung möglich, welche Inhalte er bei einem Formularfeld vorschlägt. So schlägt etwa `autocomplete="work email"` die berufliche E-Mail-Adresse vor, `autocomplete="home email"` eine private. Der HTML-Standard definiert hier eine Reihe von Werten, mit denen dem Browser mitgeteilt werden kann, welche Inhalte erwartet werden. Diese können in [54, Abschnitt 4.10.19.7.1] eingesehen werden.

Somit kann ein Eingabefeld für eine E-Mail-Adresse wie folgt gestaltet sein:

```
1 <input type="email" autocomplete="home email">
```

2.6.4 Buttons

Buttons haben die Hauptaufgabe, Formulare abzusenden. Während bei einzeiligen Eingabefeldern ein Absenden durch Drücken der Enter-Taste möglich ist, sollte auf Buttons nie verzichtet werden. Zusätzlich zum Absenden (Submit) existiert ein Button zum Zurücksetzen der Formulardaten (Reset), ein allgemeiner Button ohne spezielle Funktion, sowie ein grafischer Button zum Absenden.

HTML bietet zwei unterschiedliche Möglichkeiten, Buttons zu realisieren: Über das `<input>`-Element oder das sprechendere `<button>`-Element. Beide erfüllen denselben Zweck, `<input>` ist jedoch ein Standalone-Element, während `<button>` mit `</button>` abgeschlossen wird und daher Inhalt haben kann. Dieser Inhalt können Text oder auch andere HTML-Elemente sein, was das `<button>`-Element etwas flexibler macht. Beim Element `<input>` kann zur Beschriftung des Buttons lediglich Text mit Hilfe des `value`-Attributs angegeben werden.

Submit-Button

Der *Submit-Button* hat die Aufgabe, ein Formularfeld abzuschicken. Im Folgenden wird das Markup für beide Möglichkeiten angegeben – ein Submit-Button pro `<form>`-Element genügt aber selbstverständlich. Wie schon zuvor bei den einzeiligen Eingabefeldern spezifiziert das `type`-Attribut (in beiden Fällen) die Art des Buttons. Der Wert `submit` steht für den Submit-Button. Hier kann die Angabe entfallen, da `submit` der Standardwert ist. Abbildung 2.12 zeigt die Darstellung beider Buttons.

```
1 <button type="submit">Absenden</button>
2 <input type="submit" value="Absenden">
```


Reset-Button

Der *Reset-Button* setzt die Inhalte eines Formulars auf Standardwerte zurück. In der Regel werden alle Inhalte dadurch gelöscht (es sei denn, es sind Werte über das `value`-Attribut vorgegeben, dann werden diese wiederhergestellt). Der Reset-Button kann wiederum mittels `<input>`- oder `<button>`-Element erzeugt werden, der Wert des `type`-Attributs muss `reset` sein.

Das folgende Markup-Fragment stellt beide Arten dar, ein Reset-Button genügt aber normalerweise. Auf eine Abbildung wird verzichtet, der Button hat standardmäßig dasselbe Aussehen wie ein Submit-Button (nur die Beschriftung wäre anders).

```
1 <button type="reset">Zurücksetzen</button>
2 <input type="reset" value="Zurücksetzen">
```

Allgemeiner Button

Neben Submit und Reset existiert noch ein weiterer Button, der keine besondere Funktion besitzt. Wird er angeklickt, so passiert einfach nichts. Dieser Button findet dann Verwendung, wenn nicht mit Formulardaten gearbeitet wird, sondern z. B. über JavaScript bestimmte Funktionalitäten per Klick angestoßen werden sollen. Wiederum existieren zwei Arten, bei beiden muss das `type`-Attribut den Wert `button` haben.

Das folgende Markup zeigt wieder beide Versionen, auf die Abbildung wurde aus zuvor erwähnten Gründen wieder verzichtet.

```
1 <button type="button">Klick mich!</button>
2 <input type="button" value="Klick mich!">
```

Zusätzlich existiert noch ein Button des Typs `image`, der Grafiken als Buttons zulässt.

2.6.5 Mehrzeilige Eingabefelder

Während einzeilige Eingabefelder durch das `<input>`-Element repräsentiert werden, werden *mehrzeilige Felder* mit dem `<textarea>`-Element ausgezeichnet. Es handelt sich dabei um ein mehrzeiliges Textfeld, andere Typen sind nicht vorhanden. Auch gibt es ein schließendes Element `</textarea>`. Somit wird ein eventuell vorbelegter Text nicht mit dem `value`-Attribut notiert, sondern einfach zwischen öffnendes und schließendes Element gesetzt. In mehrzeiligen Textfeldern bewirkt das Drücken der Enter-Taste einen Zeilenumbruch und nicht das Absenden des Formulars.

Um die Größe des Feldes festzulegen, gibt es die Attribute `rows` (Anzahl der Zeilen) und `cols` (Anzahl der Spalten bzw. Zeichen). Mittels `maxlength` kann die maximal erlaubte Zeichenlänge festgelegt werden. Das Attribut `readonly` setzt das Feld auf nur lesbar. Das Attribut `wrap` kann die Werte `soft` und `hard` annehmen. Der Standardzustand `soft` bewirkt, dass nur diese Zeilenumbrüche übertragen werden, die explizit durch Drücken von Enter gesetzt wurden. Wurde `hard` angegeben, so werden jene Zeilenumbrüche übertragen, wie sie durch die Größe des Eingabefeldes zustande kamen. Diese muss dann aber zwingend über das `cols`-Attribut spezifiziert werden.

Abbildung 2.13 zeigt ein Formularfeld, das mit dem folgenden HTML-Markup erstellt wurde:

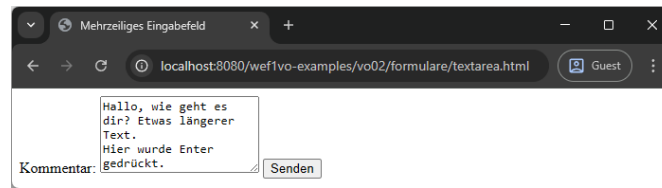


Abbildung 2.13: Ein mehrzeiliges Eingabefeld, dessen Höhe auf fünf Zeilen gesetzt wurde und das mit längerem Fließtext befüllt wurde.

```
1 <label for="comment">Kommentar:</label>
2 <textarea name="text" rows="5" id="comment"></textarea>
3 <button type="submit">Senden</button>
```

In diesem Beispiel wurde das `wrap`-Attribut nicht gesetzt, also wird `soft` als Wert angenommen. Die Ausgabe in einem E-Mail sieht wie folgt aus (`text=` rührt vom `name`-Attribut her, das mit dem Wert „text“ gesetzt wurde):

```
text=Hallo, wie geht es dir? Etwas längerer Text.
Hier wurde Enter gedrückt.
```

Wird nun die Attributliste um `wrap="hard"` (und die Breitenangabe `cols="20"`) ergänzt, sieht die Ausgabe wie folgt aus. Die Umbrüche sind genau so, wie sie durch die Breite des Textfeldes entstanden sind. Dies deckt sich mit Abbildung 2.13.

```
text=Hallo, wie geht es
dir? Etwas längerer
Text.
Hier wurde Enter
gedrückt.
```

2.6.6 Radio Buttons und Checkboxes

Radio Buttons und Checkboxes erlauben es, Auswahlen in einem Formular zu treffen. Radio Buttons bieten dabei n Optionen und erlauben genau 1 Auswahl daraus, Checkboxes bieten ebenfalls n Optionen und erlauben m Auswahlen, d.h. es können alle Kombinationen angehakt werden, oder auch gar nichts.

Radio Buttons

Radio Buttons werden mit dem `<input>`-Element erstellt, das `type`-Attribut muss dabei den Wert `radio` haben. Um eine Gruppe von Radio Buttons zu erzeugen, müssen mehrere solche Elemente im selben Formular erzeugt werden, deren `name`-Attribut sich gleicht. Dadurch wird Zusammengehörigkeit erlangt. Um einen Radio Button aus dieser Gruppe als vorausgewählt zu kennzeichnen, kann diesem das Attribut `checked` gesetzt werden. Soll beim Absenden des Formulars ein bestimmter Wert übergeben werden, kann dieser im `value`-Attribut angegeben werden.

Das folgende Markup zeigt eine Auswahl aus drei Elementen, wobei das erste vorausgewählt ist. Die Abbildung 2.14 zeigt die Darstellung im Browser.

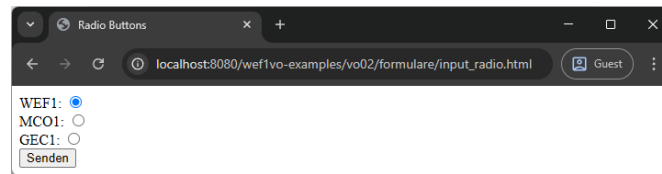


Abbildung 2.14: Drei Radio Buttons mit Beschriftung – der erste ist vorausgewählt.

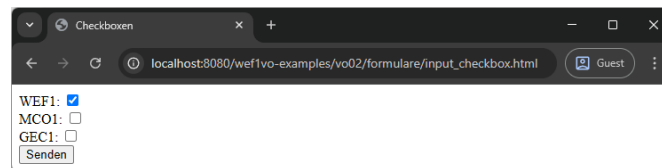


Abbildung 2.15: Drei Checkboxes mit Beschriftung – die erste ist vorausgewählt.

```

1 <label for="wef">WEF1:</label>
2 <input type="radio" name="favorit" value="WEF1" id="wef" checked><br>
3 <label for="mco">MCO1:</label>
4 <input type="radio" name="favorit" value="MCO1" id="mco"><br>
5 <label for="gec">GEC1:</label>
6 <input type="radio" name="favorit" value="GEC1" id="gec"><br>
7 <button type="submit">Senden</button>

```

Checkboxen

Checkboxen werden ebenfalls mit dem `<input>`-Element erstellt, das `type`-Attribut muss dabei den Wert `checkbox` haben. Checkboxes müssen nicht als zusammengehörig markiert werden, d. h. das `name`-Attribut muss nicht angeführt werden bzw. übereinstimmen. Die Attribute `value` und `checked` funktionieren gleich wie bei Radio Buttons.

Das folgende Markup zeigt das obige Beispiel mit Checkboxes realisiert, Abbildung 2.15 zeigt das Ergebnis im Browser.

```

1 <label for="wef">WEF1:</label>
2 <input type="checkbox" value="WEF1" id="wef" checked><br>
3 <label for="mco">MCO1:</label>
4 <input type="checkbox" value="MCO1" id="mco"><br>
5 <label for="gec">GEC1:</label>
6 <input type="checkbox" value="GEC1" id="gec"><br>
7 <button type="submit">Senden</button>

```

2.6.7 Auswahllisten

Auswahllisten, auch Dropdown-Listen genannt, ermöglichen die Auswahl aus einer Liste von Optionen. Um eine solche Liste zu erstellen, wird das Element `<select>` (geschlossen durch `</select>`) verwendet. Mit dem Attribut `name` kann, wie bei Eingabefeldern, ein Name für die Liste vergeben werden. `size` gibt an, wie viele Optionen auf einmal gezeigt werden sollen (Standardwert ist 1). Über die Angabe von `multiple` können durch Drücken von `Strg` (Windows) oder `⌘` (Mac) mehrere Einträge ausgewählt werden.

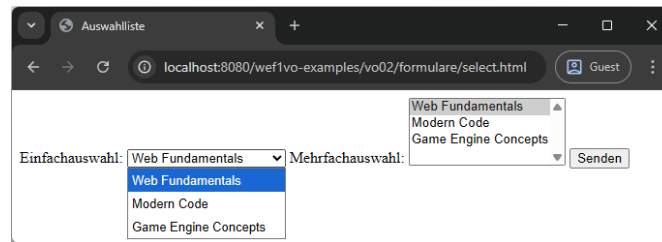


Abbildung 2.16: Zwei Mal dieselbe Auswahlliste, links mit Einfachauswahl, rechts mit Mehrfachauswahl.

Die Einträge werden durch `<option>`-Elemente (abgeschlossen durch `</option>`) innerhalb des `<select>`-Elements erstellt. Über das Attribut `value` kann der Wert zur Übertragung bestimmt werden (wenn dieser vom angezeigten abweichen soll), `selected` spezifiziert den Eintrag, der vorausgewählt sein soll. Um Einträge zu gruppieren, können mehrere `<option>`-Elemente in `<optgroup>`-Tags verpackt werden.

Das folgende Markup stellt zwei Auswahllisten mit drei Elementen dar, einmal mit Einfach-, dann mit Mehrfachauswahl. Abbildung 2.16 zeigt die Darstellung im Browser.

```
1 <label for="einfach">Einfachauswahl:</label>
2 <select name="faecher1" id="einfach">
3   <option value="WEF1" selected>Web Fundamentals</option>
4   <option value="MCO1">Modern Code</option>
5   <option value="GEC1">Game Engine Concepts</option>
6 </select>
7 <label for="mehrfach">Mehrfachauswahl:</label>
8 <select name="faecher2" id="mehrfach" multiple>
9   <option value="WEF1" selected>Web Fundamentals</option>
10  <option value="MCO1">Modern Code</option>
11  <option value="GEC1">Game Engine Concepts</option>
12 </select>
13 <button type="submit">Senden</button>
```

Auswahllisten stylen

Seit Jänner 2025 gibt es das neue Element `<selectedcontent>`. Dieses wird innerhalb eines Buttons noch vor den `<option>`-Elementen eingefügt und enthält immer die aktuell ausgewählte Option. Wird das Element gesetzt und die Auswahlliste mit dem folgenden CSS-Code formatiert, so wird die Standardformatierung des Browsers bzw. des Betriebssystems deaktiviert und die Auswahlliste kann komplett angepasst werden. So ist es etwa auch möglich, HTML innerhalb der `<option>`-Elemente zu verwenden, Bilder einzufügen und ähnliches.

```
1 <label for="customized">Einfachauswahl angepasst:</label>
2 <select name="faecher3" id="customized">
3   <button><selectedcontent></selectedcontent></button>
4   <option value="WEF1" selected>Web Fundamentals <strong>WEF1</strong></option>
5   <option value="MCO1">Modern Code <strong>MCO1</strong></option>
6   <option value="GEC1">Game Engine Concepts <strong>GEC1</strong></option>
7 </select>
8 <button type="submit">Senden</button>
```

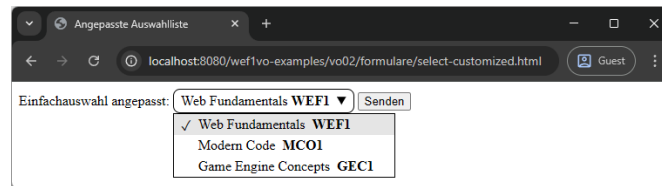


Abbildung 2.17: Eine angepasste Auswahlliste. Die Abkürzung der Lehrveranstaltung erscheint fett, weil sie mit `<strong>` formatiert ist.

```
1 select,
2 ::picker(select) {
3   appearance: base-select;
4 }
```

Das Feature ist bereits teil des HTML Living Standards, wird aber zum aktuellen Zeitpunkt nur von den Blink-basierten Browsern (Chrome, Edge, Opera) unterstützt. Abbildung 2.17 zeigt die Einfachauswahlliste mit einem durch `<strong>` fett gesetzten Wort.

2.6.8 Zusatzangaben bei Formularelementen

Bei den meisten Formularfeldern gibt es drei interessante Zusatzangaben, die hinzugefügt werden können, um kleine, aber feine Verbesserungen in der Benutzbarkeit zu erlangen: Platzhaltertext, Autofokus und Pflichtfelder.

Platzhaltertext

Auch wenn über die Beschriftung mit `<label>` angegeben werden kann, was in ein Formularfeld eingegeben werden soll (z. B. Name oder E-Mail-Adresse), so ist es oft praktisch, zusätzliche Informationen angeben zu können, wie etwa ein Beispieleintrag aussehen könnte. Genau diese Sache kann über *Platzhaltertext* gut gelöst werden. Während das Formularfeld unausgefüllt ist, wird der Platzhaltertext angezeigt, sobald das Feld ausgewählt bzw. mit Inhalt befüllt wurde, verschwindet dieser. Verantwortlich dafür ist das Attribut `placeholder`. Sein Wert wird im Formularfeld angezeigt. Dieses Attribut darf nicht mit `value` verwechselt werden. Denn während `value` wirklich den Wert vorausfüllt (d. h. beim Absenden würde der Wert des `value`-Attributs auch übermittelt werden), wird der Platzhaltertext nur visuell angezeigt und beim Absenden gilt das Formular als leer.

Das folgende Markup zeigt ein E-Mail Feld mit Platzhaltertext, Abbildung 2.18 stellt das Ergebnis im Browser dar.

```
1 <label for="contact">E-Mail:</label>
2 <input type="email" id="contact" placeholder="user@example.com">
3 <button type="submit">Absenden</button>
```

Autofokus

Websites, deren Haupteinsatzgebiet die Suche nach Information ist (z. B. Suchmaschine, Enzyklopädie, Wörterbuch etc.), sollen den Benutzer*innen einen möglichst effektiven

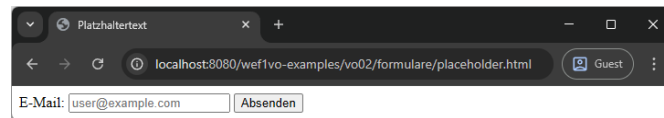


Abbildung 2.18: Der Platzhaltertext wird im Eingabefeld angezeigt. Klickt der*die Benutzer*in hinein, ist das Feld leer.

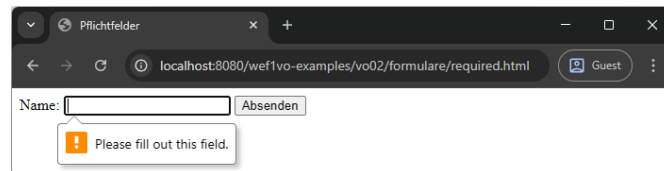


Abbildung 2.19: Wird das Feld nicht ausgefüllt, zeigt der Browser eine Fehlermeldung und verhindert das Absenden.

Ablauf bieten. Meist wird die Seite aufgerufen, der Suchbegriff eingegeben und die Suche abgeschickt. Um diesen Ablauf zu optimieren, bietet es sich an, dem zentralen Suchfeld das Attribut `autofocus` anzufügen. Somit steht der Cursor nach dem Aufruf der Seite sofort in diesem Feld und Benutzer*innen müssen es nicht zuerst mit Mouse, Keyboard oder durch Antippen anwählen.

Das folgende Markup statet ein Suchfeld mit der *Autofokus*-Eigenschaft aus. Auf eine Abbildung wird verzichtet, es wird empfohlen dieses Verhalten mit einer Beispieldatei auszuprobieren.

```
1 <label for="search-box">Suche:</label>
2 <input type="search" id="search-box" autofocus>
3 <button type="submit">Absenden</button>
```

Pflichtfelder

Wie bereits beim Eingabefeld für E-Mail-Adressen (Abschnitt 2.6.3) erwähnt, sieht der HTML-Standard Validierung von Eingaben durch den Browser vor (wo es, aufgrund eindeutiger Muster möglich ist). Eine weitere Möglichkeit besteht darin, Eingabefelder als *Pflichtfelder* zu markieren. Werden sie nicht ausgefüllt, verhindert der Browser das Absenden des Formulars. Dies ist durch Angabe des Attributs `required` möglich.

Nachfolgend das Markup für ein verpflichtendes Textfeld und die Warnung des Browsers in Abbildung 2.19.

```
1 <label for="name">Name:</label>
2 <input type="text" id="name" required>
```

2.6.9 Formulare strukturieren

Bei größeren Formularen ist oft eine logische Gruppierung erwünscht, um mehr visuelle Klarheit zu schaffen. So könnten in einem Bestellformular etwa persönliche Daten gruppiert und dann die Angaben zur Bestellung wiederum separat gegliedert werden. Diese

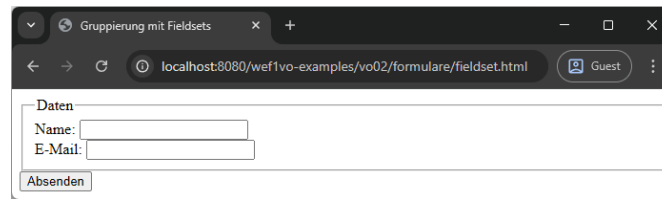


Abbildung 2.20: Die beiden Eingabefelder sind logisch durch ein Fieldset gruppiert. Dieses ist zusätzlich beschriftet.

Gruppierung lässt sich über so genannte *Fieldsets* lösen, das Element hierfür lautet daher `<fieldset>` (mit abschließendem `</fieldset>`) – es enthält alle Formularelemente, die zu einer Gruppe zusammengefasst werden sollen. Visuell wird durch die Angabe eines Fieldsets ein Rahmen um diese Elemente gezogen. Um dem Fieldset eine Beschriftung zu geben, kann das Element `<legend>` (mit `</legend>` als schließendes Element) gleich als erster Tag innerhalb der Gruppierung angegeben werden.

Das folgende Markupbeispiel enthält zwei Eingabefelder, die mit einem beschrifteten Fieldset gruppiert wurden. Abbildung 2.20 zeigt die Darstellung im Browser.

```

1 <fieldset>
2   <legend>Daten</legend>
3   <label for="name">Name:</label>
4   <input type="text" id="name"><br>
5   <label for="contact">E-Mail:</label>
6   <input type="email" id="contact">
7 </fieldset>
8 <button type="submit">Absenden</button>

```

2.6.10 Vollständiges HTML-Grundgerüst mit Formular

Mit dem Wissen über Zeichenkodierungen aus Abschnitt 2.1 lässt sich das ursprünglich in Abschnitt 1.5.6 vorgestellte Grundgerüst nun um die Angabe der Zeichenkodierung ergänzen. Als solche kommt in modernen Websites nur noch UTF-8 in Frage, da sie Probleme mit Sonderzeichen aus verschiedenen Regionen endgültig eliminiert.

Nochmals explizit erwähnt sei die Angabe des `lang`-Attributs beim `<html>`-Element. Dieses Attribut gehört zu den globalen Attributen, kann also bei jedem Element angefügt werden. Durch die Angabe beim Root-Element wird die Sprache des Dokuments bekannt gegeben. Dies ist hilfreich für Suchmaschinen oder andere Tools wie Screenreader für Personen mit visuellen Einschränkungen.

Als Werte können Angaben im Format von Standard-Sprachtags wie z. B. `de` für Deutsch oder `en` für Englisch gemacht werden. Zusätzlich kann optional eine Region mit Bindestrich angehängt und somit noch eine genauere Spezifizierung vorgenommen werden. `de-DE` steht etwa für Deutsch (Deutschland), `de-AT` für Deutsch (Österreich), `en-US` für Englisch (USA) und `en-GB` für Englisch (Großbritannien). Eine vollständige, wenngleich unübersichtliche Liste bietet die IANA³. Richard Ishida stellt ein praktisches Interface zur Suche⁴ zur Verfügung.

³<https://www.iana.org/assignments/language-subtag-registry/language-subtag-registry>

⁴<https://r12a.github.io/app-subtags/>

Ebenfalls enthalten ist ein Formular mit verschiedenen Formularelementen und Attributen. Das Markup sieht wie folgt aus, das finale Ergebnis ist in Abbildung 2.21 zu sehen.

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <title>Vollständiges HTML-Gründgerüst mit Formular</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <h1>Feedback</h1>
9     <form action="mailto:john.doe@johndoe.com" method="post" enctype="text/plain">
10      <fieldset>
11        <legend>Meine Daten</legend>
12        <label for="autor">Name:</label>
13        <input type="text" name="autor" id="autor" size="20" placeholder="John Doe"
14        required><br>
15        <label for="email">E-Mail:</label>
16        <input type="email" name="email" id="email" size="20" placeholder="
17        user@example.com"><br>
18      </fieldset>
19      <fieldset>
20        <legend>Kommentar und Stimmung</legend>
21        <label for="kommentare">Kommentare:</label>
22        <textarea name="kommentare" id="kommentare" rows="8" cols="20" placeholder="
23        Super Beitrag!" required></textarea><br>
24        <p>Ich bin...</p>
25        <label for="happy">happy</label>
26        <input type="radio" name="stimmung" value="happy" id="happy">
27        <label for="neutral">neutral</label>
28        <input type="radio" name="stimmung" value="neutral" id="neutral">
29        <label for="traurig">traurig</label>
30        <input type="radio" name="stimmung" value="traurig" id="traurig">
31      </fieldset>
32      <button type="submit">Kommentar posten</button>
33      <button type="reset">Zurücksetzen</button>
34    </form>
35  </body>
36</html>

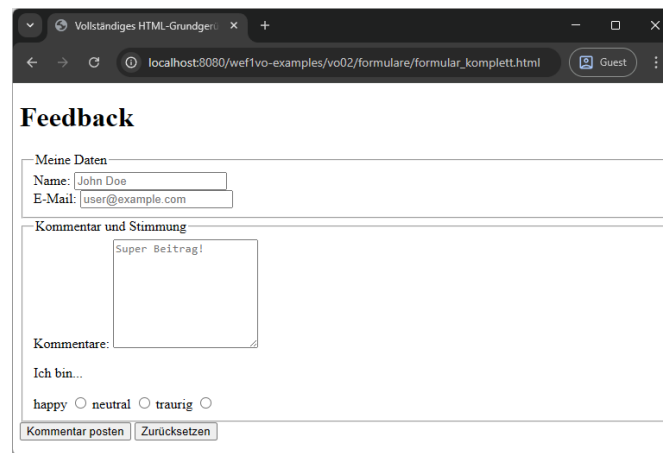
```

2.7 Weiterführende Literatur

Als Nachschlagewerk für HTML-Elemente empfiehlt sich derzeit vor allem der HTML Living Standard der WHATWG [54]. Hier sind selbstverständlich alle Elemente mit allen Details und Regeln aufgelistet.

Eine weitere gute Ressource stellt das Mozilla Developer Network [39] dar. Dieses Wiki kann frei editiert werden und enthält eine sehr kompakte und aktuelle Übersicht zu den Themen HTML, CSS, JavaScript sowie einigen anderen Technologien. Es empfiehlt sich, die englischsprachige Wiki zu verwenden, da die deutsche nicht immer vollständig ist.

Als Informationsquellen können auch [18, 40, 57, 58] dienen, wenngleich die Bücher nicht primäre als Nachschlage-Referenzen gedacht sind und die Elemente ausführlicher



The screenshot shows a web browser window with the title 'Vollständiges HTML-Grundgerüst'. The address bar displays 'localhost:8080/wef1vo-examples/vo02/formulare/formular_komplett.html'. The page content is a feedback form with the following sections:

- Feedback**
- Meine Daten**
 - Name:
 - E-Mail:
- Kommentar und Stimmung**
 -
 - Kommentare:
 - Ich bin...
 - ☒ happy
 - ☐ neutral
 - ☐ traurig
-

Abbildung 2.21: Ein komplettes HTML-Grundgerüst mit Formular.

erklären.

Vorlesung 3

CSS-Grundlagen

Dieses Kapitel erläutert die Grundlagen von Cascading Style Sheets. Von einer Einführung über die grundsätzliche Funktionsweise bis hin zu einigen exemplarischen Formaten. Letztere werden nicht im Detail besprochen, da sie einfach nachgeschlagen werden können.

3.1 Einführung

Der folgende Abschnitt stellt eine grundlegende Einführung in *Cascading Style Sheets* (kurz CSS) dar.

3.1.1 Definition

Cascading Style Sheets sind eine deklarative Sprache zur Definition von Formateigenschaften einer Website. Sie stellen eine unmittelbare Ergänzung zu HTML dar, können aber auch mit anderen Markup-Sprachen wie z. B. XML verwendet werden.

3.1.2 Wozu CSS?

In Vorlesung 2 wurde bereits auf die Wichtigkeit der semantisch korrekten Verwendung von HTML-Elementen hingewiesen. So spezifiziert eine Auszeichnung mit dem `<em>`-Element etwa, dass eine Textstelle betont ist. Um dies visuell darzustellen, stellen Webbrowser die markierte Stelle standardmäßig kursiv dar. Auch alle anderen Elemente stellen Browser in einer bestimmten Art und Weise dar. HTML bietet jedoch keine Möglichkeit (mehr¹), diese vom Browser vorgegebene Darstellung anzupassen. Dies ist auch gut so, denn die Aufgabe von HTML ist es lediglich, Inhalte mit einer Bedeutung zu versehen. Die Darstellung spielt per Definition keine Rolle.

Diese Aufgabe übernehmen die Cascading Style Sheets. Mit ihrer Hilfe kann das Aussehen eines jeden einzelnen HTML-Elements bestimmt bzw. verändert werden, ohne dabei die Semantik zu beeinflussen.

Zusammengefasst gilt:

¹Bis HTML 4.01 gab es noch Elemente und Attribute, die das Aussehen des Dokuments beeinflussen konnten, etwa das Attribut `color`.

In HTML sollen ausschließlich Inhalt und Struktur definiert werden. Cascading Style Sheets dienen zur Definition der Darstellung.

3.1.3 Versionen

Die Entwicklung von Stylesheets begann Anfang der 1990er Jahre, 1995 wurde diese Entwicklung vom W3C berücksichtigt, das damit begann, an der Standardisierung zu arbeiten. Im Dezember 1996 wurde die *CSS Level 1 Recommendation*, also CSS 1.0 [64] veröffentlicht.

CSS Level 2, also CSS2, wurde Mitte 1998 veröffentlicht. Die Umsetzung in den Browsern erfolgte jedoch nur schleppend und auch heute implementiert kaum ein Browser diese Version vollständig. Um auf die Probleme, die mit CSS2 auftraten, zu reagieren, wurde 2002 mit der Entwicklung von CSS Level 2 Revision 1 (kurz CSS 2.1 [62]) begonnen. Diese Version ist die in den letzten Jahren meist benutzte und wurde meist als Standard angesehen. Tatsächlich wurde sie vom W3C erst am 7. Juni 2011 als Recommendation verabschiedet. Die Implementierung ist in allen Browsern sehr gut. Im April 2016 wurde die Arbeit an CSS 2.2 [63] gestartet, welches zum Ziel hat, Fehler aus CSS 2.1 zu beseitigen und auch Features zu entfernen, die nie implementiert oder durch CSS3-Module ersetzt wurden.

Die Entwicklung von *CSS Level 3* (kurz CSS3) startete bereits im Jahr 2000 mit dem Ziel, neue Features zu entwickeln (CSS 2.1 war nur eine Fehlerkorrektur von CSS2 und bot daher nichts Neues). Die Spezifikation ist keine Neuentwicklung von Beginn an, sondern baut im Kern auf CSS 2.1 auf. Neue Entwicklungen werden nicht mehr monolithisch in einem Dokument zusammengefasst, sondern in Form von einzelnen Modulen spezifiziert. Diese werden einzeln entwickelt und befinden sich daher in unterschiedlichen Standardisierungsstufen. CSS3 Module sind in einigen Fällen Ergänzungen in anderen Fällen aber auch Ersatz von CSS 2.1 Eigenschaften. Das W3C veröffentlicht in regelmäßigen Abständen den so genannten *CSS Snapshot* [80], der den aktuellen Status von CSS zusammenfasst. Darin u. a. folgende CSS-Module (aus Ebene 3 bis 5) aufgelistet:

- **CSS 2.1** [62]: Kern der CSS-Spezifikation, befindet sich im Status einer Recommendation (ist also abgeschlossen).
- **CSS Syntax Level 3** [81]: Ergänzungen zur Syntax gegenüber CSS 2.1. Seit 2021 im Status eines Candidate Recommendation Drafts.
- **Media Queries Level 3** [96]: Anpassungen von Layouts, je nach verwendetem Ausgabegerät, seit 5. April 2022 im Status einer Recommendation.
- **Selectors Level 3** [101]: Erweitert die in CSS 2.1 definierten Selektoren um viele neue. Seit November 2018 eine Recommendation.
- **CSS Box Model Level 3** [67]: Ergänzungen und Überarbeitungen zum CSS Box-Modell (siehe Vorlesung 4). Seit April 2024 eine Recommendation.
- **CSS Color Module Level 4** [70]: Ermöglicht neue Arten, Farben in CSS zu spezifizieren. Mit April 2025 ein W3C Candidate Recommendation Draft.
- **CSS Backgrounds and Borders Module Level 3** [66]: Neue Optionen für Hintergründe und Rahmen (wie etwa abgerundete Ecken). Mit März 2024 ein Candidate Recommendation Draft.

- **CSS Images Module Level 3** [76]: Zuständig für die Einbindung von Bildern und Farbverläufen, im Dezember 2023 ein Candidate Recommendation Draft.
- **CSS Fonts Module Level 3** [72]: Ermöglicht es, herunterladbare Schriften zu verwenden. Seit 20. September 2018 eine Recommendation.
- **CSS Multi-column Layout Module** [77]: Mehrspaltiges Anordnen von Text. Seit Mai 2024 ein Candidate Recommendation Snapshot.
- **CSS Cascading and Inheritance Level 5** [69]: Ergänzt die ursprünglich in [62] definierte Kaskade um neue CSS-Funktionen. Der aktuelle Candidate Recommendation Snapshot stammt vom Jänner 2022.
- **CSS Text Module Level 3** [82]: Definiert erweiterte Eigenschaften zur Textmanipulation. Seit September 2024 ein Candidate Recommendation Draft.
- **CSS Transitions Level 1** [84]: Ermöglicht sanfte Übergänge bei Zustandsänderungen, etwa bei Mouse-Over-Effekten. Seit 2018 ein Working Draft.
- **CSS Animations Level 1** [65]: Keyframe-Animationen – im Jahr 2023 ein Working Draft.

Darüber hinaus gibt es noch eine Vielzahl weiterer Module, deren Aufzählung den Rahmen dieses Skriptums sprengen würde. Eine komplette Liste mit Status und Prioritäten kann unter <https://www.w3.org/Style/CSS/current-work> abgerufen werden.

3.2 Grundlegende Syntax

Eine *CSS-Regel* ist wie folgt aufgebaut. Optionale Angaben stehen in eckigen Klammern:

```
1 Selektor [, Selektor_2, ...] {  
2   Eigenschaft_1: Wert_1 [!important];  
3   Eigenschaft_2: Wert_2 [!important];  
4   ...  
5   Eigenschaft_n: Wert_n [!important];  
6 }
```

Am Anfang jeder Angabe steht ein *Selektor*. Er gibt an, worauf sich die Regel beziehen soll (z. B. auf ein `<h1>`-Element). Es ist möglich, noch weitere Selektoren mit Komma getrennt anzugeben.

Danach folgt eine öffnende geschweifte Klammer. Sie zeigt den Beginn der Liste an Eigenschaften an, also der Angaben, die aussagen, wie das selektierte Element aussehen soll. Diese Angaben bestehen aus dem Namen der CSS-Eigenschaft und einem dazugehörigen Wert, getrennt durch einen Doppelpunkt, abgeschlossen durch ein Semikolon. Wird zusätzlich noch das Schlüsselwort `!important` angegeben, erhält die Regel zusätzliche Gewichtung. Die schließende geschweifte Klammer beendet die CSS-Regel.

3.2.1 Beispiele

In den folgenden Beispielen wird die Schriftfarbe mit der Eigenschaft `color` verändert. `font-family` wird verwendet, um die Schriftart anzupassen.

Folgender Code setzt die Schriftfarbe aller `<h1>`-Überschriften auf grün.

```
1 h1 {  
2   color: green;  
3 }
```

Im folgenden Code werden Schriftart (Arial) und Schriftfarbe (rot) für alle `<h2>`-Überschriften mit einer Regel gesetzt. Die `!important`-Deklaration bei der zweiten Eigenschaft gibt dieser Angabe besonderes Gewicht.

```
1 h2 {  
2   color: #FF0000;  
3   font-family: Arial !important;  
4 }
```

Der folgende Code setzt die Schriftfarbe blau; sowohl für `<h3>`-Elemente als auch für alle Absätze (`<p>`).

```
1 h3, p {  
2   color: #0000FF;  
3 }
```

3.2.2 Kommentare

CSS-Kommentare werden mit einem Schrägstrich, gefolgt von einem Stern-Symbol geöffnet und in der umgekehrten Reihenfolge wieder geschlossen. Ein CSS-Kommentar sieht also wie folgt aus:

```
1 /* Dies ist ein CSS-Kommentar */
```

3.2.3 Gruppierung von Regeln

Wie im zweiten Beispiel ersichtlich, können zwei Angaben, die auf dasselbe Element wirken sollen, in einer Regel zusammengefasst werden. Dies ist aufgrund der Übersichtlichkeit und Lesbarkeit eine übliche Vorgehensweise. Es ist jedoch genauso möglich, zwei getrennte Regeln mit je einer Angabe zu schreiben.

Folgender Code ist also genauso möglich ...

```
1 h2 {  
2   color: #FF0000;  
3   font-family: Arial !important;  
4 }
```

...wie jener:

```
1 h2 {  
2   color: #FF0000;  
3 }  
4 h2 {  
5   font-family: Arial !important;  
6 }
```

Im dritten Beispiel wurde eine Eigenschaft zwei unterschiedlichen Elementen in einer Regel zugewiesen. Auch diese Regel kann auf zwei einzelne aufgeteilt werden, ohne das Ergebnis zu beeinflussen.

So ist also sowohl dies möglich ...

```
1 h3, p {  
2   color: #0000FF;  
3 }
```

...wie auch dies:

```
1 h3 {  
2   color: #0000FF;  
3 }  
4 p {  
5   color: #0000FF;  
6 }
```

3.3 Arten der Anwendung (Einbindung)

CSS-Regeln können auf vier Arten angewandt werden:

- als Attribut direkt beim Element (**style**-Attribut),
- direkt im **<head>**-Bereich des HTML-Dokuments (**<style>**-Element),
- verlinkt im **<head>**-Bereich des HTML-Dokuments (**<link>**-Element),
- eingebunden aus weiteren Stylesheets mittels **@import**.

3.3.1 Direkt beim Element

Es besteht die Möglichkeit, CSS-Eigenschaften direkt im HTML-Code zu spezifizieren. Möglich ist dies mit dem **style**-Attribut, welches zu den Universalattributen zählt und somit mit jedem HTML-Element verwendet werden kann. Die Eigenschaften werden als Wert des **style**-Attributs angegeben. Jedes Eigenschaft-Wert-Paar sollte mit Semikolon abgeschlossen werden, um Probleme zu vermeiden.

Um die Farbe einer **<h1>**-Überschrift auf grün zu setzen, kann das folgende Markup verwendet werden.

```
1 <h1 style="color: green;">
```

Auch wenn es sich dabei um eine erlaubte und auch valide Methode handelt, so sollte sie doch sehr sparsam eingesetzt werden. Denn die Angabe von Formatierungen direkt im HTML-Markup widerspricht dem rein bedeutungsgebenden Charakter von HTML. Es ist zudem nur möglich, Eigenschaften einem bestimmten HTML-Element zuzuweisen. Generelle Zuweisungen (z. B. Formatierungen aller **<h1>**-Elemente) sind nicht möglich. Daher ergibt sich auch hier eine sehr starke Änderungsfeindlichkeit.

3.3.2 Direkt im Kopfbereich

Um Formatierungen für eine gesamte HTML-Datei anzugeben, können diese direkt in der jeweiligen Datei notiert werden. Dazu muss im **<head>**-Bereich das Element **<style>** platziert werden (Abschluss mit **</style>**). Innerhalb dieses Elements können beliebig viele CSS-Regeln platziert werden. Früher wurde beim **<style>**-Element das Attribut **type** mit dem Wert **text/css** angegeben, um zu spezifizieren, dass es sich um CSS-Angaben handelt. Seit HTML5 ist dies der Standardwert, daher kann (und sollte) das Attribut weggelassen werden.

Folgender Code setzt die Farbe grün für alle **<h1>**-Überschriften im Dokument.

```
1 <head>  
2   ...  
3   <style>  
4     h1 {
```

```
5     color: green;
6   }
7 </style>
8 </head>
```

Auch diese Variante sollte kritisch betrachtet werden. Während sie zwar mehr Flexibilität bietet, als die direkte Angabe beim Element, so bedeutet sie doch einen großen Eingriff in die HTML-Datei. Diese wird durch die Style-Regeln zusätzlich vergrößert und muss jedes Mal geändert werden, wenn Regeln verändert werden. Diese Variante der Einbindung eignet sich meist dann, wenn nur eine einzelne HTML-Datei vorliegt und auch die Anzahl der CSS-Regeln überschaubar ist.

3.3.3 Als eigenes Dokument

CSS-Angaben werden am Besten in eine eigene Datei ausgelagert. Diese Datei hat die Endung `.css` und ist eine reine Textdatei. Sie enthält nur CSS-Regeln, kein HTML-Markup (also kein `<style>`-Element oder andere Elemente).

Die Einbindung dieser Datei erfolgt wiederum im `<head>` des HTML-Dokuments über das `<link>`-Element. Dieses wird generell dazu verwendet, um externe Ressourcen in das aktuelle Dokument einzubinden. Das Element benötigt zwingend die Attribute `rel` und `href`. `rel` spezifiziert die Beziehung, in der das verlinkte Dokument zum aktuellen steht. Im Falle einer CSS-Datei muss der Wert `stylesheet` lauten. Das `href`-Attribut gibt die Quelle des einzubindenden Dokuments an und sollte idealerweise ein relativer Link auf die CSS-Datei sein. Die Angabe des `type`-Attributs (mit dem Wert `text/css`) kann wie bei der direkten Einbindung entfallen.

Das HTML-Markup zur Einbindung einer externen CSS-Datei sieht wie folgt aus:

```
1 <head>
2   ...
3   <link rel="stylesheet" href="styles.css">
4 </head>
```

Diese Methode ist die flexibelste und sollte, wann immer möglich, verwendet werden. Es besteht lediglich eine (notwendige) Verbindung zwischen HTML-Dokument und CSS-Datei, die nur dann angepasst werden muss, falls sich der Name der CSS-Datei ändert oder eine andere eingebunden werden soll. Ansonsten muss das HTML-Dokument nicht verändert werden, wenn neue Formatierungen hinzugefügt oder bestehende geändert werden sollen.

3.3.4 Einbinden aus weiteren Stylesheets

Mit einer so genannten `@import`-Regel lassen sich weitere, externe Stylesheets mit einer Angabe importieren. Nach dem Schlüsselwort folgt die Spezifizierung des Stylesheets mittels `url`-Angabe. Die Anweisung könnte direkt im `<style>`-Element verwendet werden um dort ein Stylesheet zu laden:

```
1 <head>
2   ...
3   <style>
4     @import url("style.css");
5   </style>
6 </head>
```

Die weitaus gebräuchlichere Variante ist jedoch, in einem externen Stylesheet weitere Stylesheets zu importieren. So muss mit über `<link>` nur ein zentrales Stylesheet inkludiert werden, welches wiederum alle weiteren mittels `@import` lädt:

```
1 /* main.css */
2 @import url("base.css");
3 @import url("layout.css");
4 @import url("navigation.css");
```

Sollen die Angaben eines Stylesheets gleich einem Kaskaden-Layer zugewiesen werden (siehe dazu Abschnitt 3.6.5), kann dieser mit angegeben werden:

```
1 @import url("layout.css") layer(components);
```

Generell ist bei `@import` anzumerken, dass jede `@import`-Angabe einen zusätzlichen Request erzeugt und es in der Praxis daher eher sparsam eingesetzt werden sollte.

3.4 Arten von Selektoren

Der Selektor einer CSS-Regel bestimmt, worauf genau die Eigenschaften der Regel angewandt werden. Dabei gibt es eine Reihe an unterschiedlichen Möglichkeiten, welche im Nachfolgenden erläutert werden.

3.4.1 Element-/Typselektoren

Elementselektoren, auch *Typselektoren* genannt, formatieren immer ein ganz bestimmtes HTML-Element (z. B. `<body>`, `<h1>`, `<em>`, `<span>`, `<div>` etc.). Dabei werden alle Vorkommen des Elements auf der Seite formatiert.

Folgendes Beispieldokument soll formatiert werden:

```
1 <body>
2   <h1>Dies ist die Überschrift</h1>
3   <p>Dies ist ein Absatz und <em>das hier</em> ist hervorgehoben.</p>
4 </body>
```

Folgender CSS-Code soll zur Anwendung kommen. Zusätzlich zu den bereits erwähnten Eigenschaften `color` und `font-family` kommen die Angaben `background-color` zur Angabe der Hintergrundfarbe sowie `font-size` zur Angabe der Schriftgröße zum Einsatz. Das Ergebnis ist in Abbildung 3.1 ersichtlich.

```
1 body {
2   background-color: #00FFFF;
3 }
4 h1 {
5   color: #0000FF;
6 }
7 p {
8   color: #FFFFFF;
9   background-color: #808080;
10 }
11 em {
12   font-family: "Courier New", Courier, monospace;
13   font-size: 24px;
14 }
```




Abbildung 3.1: Alle Formatierungen sind mit Typselektoren durchgeführt. Das gesamte Dokument (`<body>`) erhält eine Hintergrundfarbe, Überschriften (`<h1>`) werden blau eingefärbt, Absätze (`<p>`) erhalten eine graue Hintergrundfarbe und weiße Schrift und betonte Stellen (`<em>`) bekommen eine andere Schriftart und -größe zugewiesen.

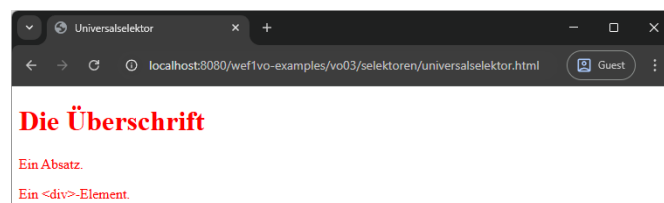


Abbildung 3.2: Der Universalselektor wählt jedes Element im Dokument aus. Daher sind alle rot dargestellt.

3.4.2 Universalselektor

Der *Universalselektor* funktioniert wie ein Typselektor, nur werden alle Elemente eines HTML-Dokuments ausgewählt. Er wird durch das Stern-Symbol (*) repräsentiert. In Kombination mit einem Klassen-, ID- oder Attributselektor kann der Stern auch entfallen, d. h. `*.crazy` entspricht `.crazy` (Details siehe folgende Abschnitte).

Folgendes HTML-Markup enthält drei unterschiedliche Elemente (eine Überschrift, einen Absatz und einen generischen Abschnitt).

```
1 <body>
2   <h1>Die Überschrift</h1>
3   <p>Ein Absatz.</p>
4   <div>Ein &lt;div>-Element.</div>
5 </body>
```

Mit Hilfe des Universalselektors im folgenden CSS-Code wird dieselbe Formatierung für alle Elemente im Dokument, also `<h1>`, `<p>`, `<div>` und auch `<body>` angewandt. Das Ergebnis ist in Abbildung 3.2 ersichtlich.

```
1 * {
2   color: #FF0000;
3 }
```

3.4.3 Klassenselektoren

Klassenselektoren ermöglichen eine Definition von Formatierungen unabhängig vom Element. Sie beginnen in CSS mit einem Punkt (z. B. `.crazy`) und wirken auf alle Elemente, die ein Attribut `class` besitzen, dessen Wert der Name des Klassenselektors ist (z. B.



Abbildung 3.3: Die Klasse `.big` wird sowohl auf die `<h1>`-Überschrift als auch auf den mit `<span>` markierten Bereich angewandt. Die Klasse `.crazy` ist der Anwendung auf Absätze (`<p>`) vorbehalten, da sie in CSS als `p.crazy` angegeben wurde.

`class="crazy"`). Klassen mit demselben Namen dürfen im HTML-Markup bei beliebig vielen Elementen vorkommen, somit sind Klassenselektoren universell einsetzbar. Soll die Verwendung dennoch auf einen bestimmten Elementtyp eingeschränkt werden, kann dieser Typ dem Klassenselektor vorangestellt werden (z. B. `p.crazy`).

Folgendes Markup demonstriert den Einsatz von Klassenselektoren. Alle Elemente mit einem `class`-Attribut werden formatiert:

```
1 <body>
2   <p class="big">Absatz 1 schaut so aus.</p>
3   <p class="crazy">Absatz 2 etwas anders.</p>
4   <p>Aber die Stelle <span class="big">hier</span> wieder wie bei 1.</p>
5 </body>
```

Der nachfolgende CSS-Code kommt dabei zum Einsatz. Das Ergebnis ist in Abbildung 3.3 ersichtlich.

```
1 .big {
2   font-size: xx-large;
3   font-family: Arial, Helvetica, sans-serif;
4 }
5 p.crazy {
6   color: #FF00FF;
7   background-color: #00FF00;
8 }
```

3.4.4 ID-Selektoren

ID-Selektoren arbeiten ähnlich wie Klassenselektoren. Auch sie dienen zur vom Element losgelösten Formatierung. Anstatt mit einem Punkt beginnen sie in CSS mit einem Raute-Zeichen (z. B. `#primary`) und wirken auf das Element mit dem Attribut `id`, dessen Wert mit dem Namen des ID-Selektors übereinstimmt (z. B. `id="primary"`). Hier offenbart sich auch der zentrale Unterschied: `id` ist zwar wie `class` ein Universalattribut, darf also bei jedem Element verwendet werden, der Wert eines jeden `id`-Attributs muss aber einzigartig sein. Somit kann ein ID-Selektor auch nur auf ein einzelnes Element im Markup angewandt werden. Es kann wie bei Klassen eine Einschränkung auf einen Elementtyp vorgenommen werden (z. B. `p#primary`), diese ist jedoch nicht wirklich nötig.

Folgendes HTML-Markup enthält eine ID, welche formatiert werden soll.



Abbildung 3.4: Der zweite Absatz besitzt das `id`-Attribut, welches über CSS formatiert wird. Es darf kein zweites Element mit derselben ID geben.

```
1 <body>
2   <p>Das ist der erste Absatz.</p>
3   <p id="primary">Aber der zweite ist viel wichtiger.</p>
4   <p>Der dritte wieder nicht.</p>
5 </body>
```

Die ID ist in folgendem CSS-Code definiert. Abbildung 3.4 zeigt das Ergebnis.

```
1 #primary {
2   font-size: xx-large;
3   color: red;
4 }
```

3.4.5 Attributselektoren

Attributselektoren erlauben die Formatierung von Elementen auf Basis von Attributen (und deren Werten) im HTML-Markup. Attributselektoren werden in eckigen Klammern angegeben, vor den Klammern steht meist noch ein Element, sodass nur Attributangaben bei einem bestimmten Typ zum Zug kommen. Die folgenden Selektoren stehen zur Verfügung:

- `element[att]`: Selektiert alle Elemente, bei denen das Attribut `att` gesetzt ist (egal, welchen Wert es hat). Zum Beispiel: `p[title]` selektiert alle Absätze, bei denen das `title`-Attribut gesetzt ist.
- `element[att="wert"]`: Selektiert alle Elemente, die ein Attribut `att` besitzen, dessen Wert genau `wert` lautet. Zum Beispiel: Der Selektor `input[type="text"]` liefert alle Eingabefelder, deren `type`-Attribut den Wert `text` hat.
- `element[att~="wert"]`: Wählt alle Elemente aus, die ein Attribut besitzen, das eine durch Leerzeichen getrennte Liste von Werten enthält, von denen einer genau `wert` entspricht. `p[class~="crazy"]` liefert nur Absätze, bei denen mehrere Klassen angegeben sind und eine davon `crazy` lautet, also z.B. Folgenden: `<p class="big crazy primary">`.
- `element[att|="wert"]`: Selektiert Elemente, die ein Attribut besitzen, dessen Wert mit `wert` beginnt und von einem Bindestrich gefolgt wird. Dies ist vor allem bei Sprachattributen interessant, z.B. selektiert `p[lang|="en"]` alle Absätze, deren `lang`-Attributwert mit `en-` beginnt, also etwa `<p lang="en-US">` oder `<p lang="en-UK">`.

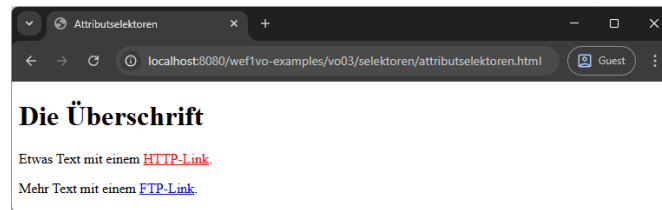


Abbildung 3.5: Der erste Link beginnt mit `http://` und wird daher rot dargestellt. Der zweite ist ein FTP-Link und wird laut Standardformatierung aus dem Browser-Stylesheet blau dargestellt.

- `element[att^="wert"]`: Liefert alle Elemente, deren Attribut `att` mit dem Wert `wert` beginnt. So liefert `a[href^="http://"]` all jene absoluten Links, die mit `http://` beginnen (also keine FTP-, HTTPS- oder andere Links).
- `element[att$="wert"]`: Selektiert alle Elemente, die ein Attribut `att` besitzen, dessen Wert mit `wert` endet. `a[href$=".html"]` erlaubt es, jene Links zu formatieren, die explizit auf HTML-Dateien verweisen (und nicht auf PHP, DOCX, MP3 oder Ähnliches).
- `element[att*="wert"]`: Gibt Elemente zurück, in deren Attribut `att` `wert` enthalten ist. Dies könnte verwendet werden um Links zu selektieren, in denen ein Benutzer*innenname vorkommt. So kann mit `a[href*="TheHoff"]` das Element `<a href="https://www.last.fm/user/TheHoff/charts">` selektiert werden.

Folgendes HTML-Markup beinhaltet zwei Links mit unterschiedlichen Protokollen (HTTP und FTP).

```
1 <body>
2   <h1>Die Überschrift</h1>
3   <p>Etwas Text mit einem <a href="http://speedtest.tele2.net/">HTTP-Link</a>.</p>
4   <p>Mehr Text mit einem <a href="ftp://speedtest.tele2.net/">FTP-Link</a>.</p>
5 </body>
```

Im CSS-Code werden alle Links formatiert, die mit `http://` beginnen. Das Ergebnis ist in Abbildung 3.5 dargestellt.

```
1 a[href^="http://"] {
2   color: #FF0000;
3 }
```

Die Groß- und Kleinschreibung beim Vergleich von Attributen und Attributwerten macht in den meisten Fällen einen Unterschied. Der Selektor `a[href^="http://"]` ergibt *keine* Übereinstimmung, wenn das Attribut im HTML-Markup `HREF` geschrieben ist oder der Wert mit `HTTP://` beginnt. Möchte man unabhängig von Groß- und Kleinschreibung selektieren (case-insensitive), kann ein `i` mit Leerzeichen getrennt in jeden Selektor gesetzt werden. Umgekehrt kann auch Case-Sensitivity erzwungen werden, indem ein `s` mit Leerzeichen getrennt in den Selektor platziert wird:

```
1 a[href^="http://" i] { ... } /* case-insensitive */
2 a[href^="http://" s] { ... } /* case-sensitive */
```

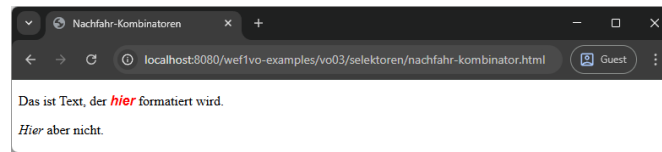


Abbildung 3.6: Die Regel selektiert nur jenes `<em>`-Element, das innerhalb des `<div>`-Elements liegt. Auch wenn dort noch `<p>` und `<strong>` in der Hierarchie dazwischen liegen, greift der Nachfahr-Kombinator.

3.4.6 Nachfahr-Kombinatoren

Nachfahr-Kombinatoren (engl. „descendant combinators“) erlauben es, Elemente zur Formatierung zu selektieren, wenn sie ein Nachfahre eines anderen Elements sind. Von einem Nachfahren wird gesprochen, wenn das Element in einem anderen verschachtelt bzw. enthalten ist – die Verschachtelungstiefe ist dabei egal. Damit ist es etwa möglich, nur all jene `<em>`-Elemente zu selektieren (und somit zu formatieren), die innerhalb eines `<div>`-Elements liegen. `<em>`-Elemente in Absätzen (`<p>`) werden nicht formatiert. Ob aber nun das `<em>`-Element direkt innerhalb von `<div>`-liegt, oder das `<em>`-Element von einem `<strong>`-Element und dieses dann im `<div>`-Element liegt, spielt keine Rolle. `<em>` ist zwar tiefer verschachtelt, aber die Verschachtelungstiefe ist, wie schon erwähnt, nebensächlich.

Um einen derartigen Nachfahr-Kombinator zu erstellen, werden die ineinander verschachtelten Elemente mit Leerzeichen getrennt angegeben, dabei besteht das beinhaltende Element immer links vom enthaltenen. Um also alle `<em>`-Elemente zu selektieren, die in einem `<div>`-Element enthalten sind, muss der Kombinator `div em` lauten. Als Eselsbrücke kann ein Lesen von rechts nach links dienen: „Selektiere alle `em`-Elemente, die in einem `div` liegen“.

Auf diese Art und Weise können beliebig viele Hierarchien erstellt werden. Selbstverständlich können auch andere Selektortypen verwendet werden: `#content .crazy p` selektiert etwa alle Absätze (`<p>`), die in einem Element mit der Klasse `crazy` liegen, welches wiederum in einem Element mit der ID `content` enthalten ist.

Ein Beispiel – folgendes HTML-Markup soll formatiert werden:

```
1 <body>
2   <div>
3     <p>Das ist Text, der <strong><em>hier</em></strong> formatiert wird.</p>
4   </div>
5   <p><em>Hier</em> aber nicht.</p>
6 </body>
```

Die folgende CSS-Regel kommt zur Anwendung, das Ergebnis ist in Abbildung 3.6 dargestellt.

```
1 div em {
2   color: #FF0000;
3   font-family: Arial, Helvetica, sans-serif;
4 }
```

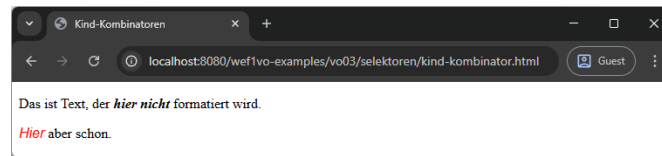


Abbildung 3.7: Die CSS-Regel hat nur Auswirkungen auf den zweiten Absatz, da hier das `<em>`-Element ein Kindelement des Absatzes ist. Im ersten Absatz liegt das `<strong>`-Element in der Hierarchie noch dazwischen.

3.4.7 Kind-Kombinatoren

Kind-Kombinatoren (engl. „child combinators“) funktionieren ähnlich wie Nachfahr-Kombinatoren, selektieren jedoch nur jene Elemente, die genau eine Ebene unterhalb eines anderen liegen, also Kinder (nicht einfach nur Nachfahren) sind. Der Kombinator besteht aus zwei oder mehreren Elementen, die durch das „größer“ Zeichen (`>`) getrennt sind. `p > em` selektiert also nur jene `<em>` Elemente, die direkt innerhalb eines Absatzes liegen.

Es wird dasselbe Markup wie zuvor (mit leicht verändertem Text) verwendet.

```
1 <body>
2   <div>
3     <p>Das ist Text, der <strong><em>hier nicht</em></strong> formatiert wird.</p>
4   </div>
5   <p><em>Hier</em> aber schon.</p>
6 </body>
```

Nun kommt folgende CSS-Regel zur Anwendung, das Ergebnis ist in Abbildung 3.7 dargestellt.

```
1 p > em {
2   color: #FF0000;
3   font-family: Arial, Helvetica, sans-serif;
4 }
```

3.4.8 Nächste Geschwister-Kombinatoren

Nächste Geschwister-Kombinatoren (engl. „next-sibling combinators“) sind die dritte Art und Weise, Formatierungen im Bezug auf die HTML-Struktur vorzunehmen. Hiermit können Elemente selektiert werden, die direkt nach einem anderen auf derselben Ebene vorkommen, also Geschwister sind. Ein Beispiel wäre, alle Absätze zu selektieren, die direkt nach einer `<h1>`-Überschrift kommen.

Der Kombinator besteht aus zwei oder mehr Elementen, die durch das Plus-Zeichen (`+`) getrennt sind. `h1 + p` selektiert eben genau jenen Absatz, der direkt nach der Überschrift platziert ist.

Auch das folgende HTML-Markup stellt dieses Beispiel dar.

```
1 <body>
2   <h1>Hier die Überschrift</h1>
3   <p>Hier der erste, formatierte Absatz.</p>
4   <p>Hier der zweite, unformatierte.</p>
5 </body>
```

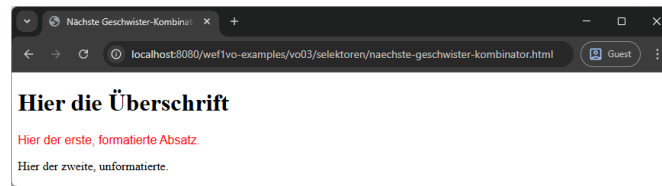


Abbildung 3.8: Nur der erste Absatz wird formatiert. Obwohl beide Absätze *Geschwister* der Überschrift sind (da sie nicht verschachtelt sind sondern auf derselben Ebene liegen), wird nur der direkt auf die Überschrift folgende berücksichtigt.

Die CSS-Regel formatiert nun den ersten Absatz, das Ergebnis ist in Abbildung 3.8 dargestellt.

```
1 h1 + p {
2   color: #FF0000;
3   font-family: Arial, Helvetica, sans-serif;
4 }
```

3.4.9 Nachfolgende Geschwister-Kombinatoren

Der *nachfolgende Geschwister-Kombinator* (engl. „subsequent-sibling combinator“) selektiert alle Elemente, die auf ein anderes, aber nicht unbedingt direkt danach folgen. Der Kombinator besteht aus zwei oder mehreren Elementen, die durch das Tilde-Zeichen (~) getrennt sind. `div ~ p` selektiert alle Absätze, die nach einem `<div>`-Element kommen und dasselbe Elternelement haben.

Das folgende Markup enthält drei Absätze, einer innerhalb eines `<div>`-Elements (Kindelement), zwei andere danach (Geschwisterelemente).

```
1 <body>
2   <div>
3     <p>Erster Absatz.</p>
4   </div>
5   <p>Zweiter Absatz.</p>
6   <p>Dritter Absatz.</p>
7 </body>
```

Die CSS-Regel formatiert nur den zweiten und dritten Absatz, das Ergebnis ist in Abbildung 3.9 dargestellt.

```
1 div ~ p {
2   color: #FF0000;
3   font-family: Arial, Helvetica, sans-serif;
4 }
```

3.4.10 Pseudoelemente

Bis jetzt wurden alle Formatierungen auf Elemente, Klassen, oder IDs angewandt – also auf Dinge, die im HTML-Markup ersichtlich waren (entweder als Element oder als Attribut). Es gibt jedoch auch Szenarien, in denen Eigenschaften formatiert werden sollen, die nicht im Markup so einfach repräsentiert sind. Zu diesem Zweck wurde das

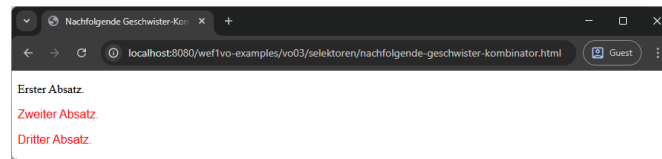


Abbildung 3.9: Alle Absätze, die Geschwister des `<div>`-Elements sind (also auf derselben Ebene liegen), werden formatiert.

Konzept von Pseudoelementen und auch der im nächsten Abschnitt beschriebenen Pseudoklassen eingeführt.

Pseudoelemente sind eine Abstraktion, auf Teile eines bestehenden Elements zu verweisen, für die es keine Repräsentation gibt (z. B. die erste Zeile eines Absatzes). Ein Pseudoelement-Selektor besteht aus dem Element, gefolgt von zwei Doppelpunkten und dem eigentlichen Pseudoelement, also z. B. `p::first-line` um die erste Zeile eines Absatzes zu selektieren.

Eine Auswahl der wichtigsten Pseudoelemente:

- `::first-line`: Selektiert die erste Zeile eines Blockelements (ein Element, das eine neue Zeile beginnt, wie etwa ein Absatz), entweder definiert durch einen automatischen Umbruch oder durch ein `<br>`-Element (z. B. `p::first-line`).
- `::first-letter`: Selektiert den ersten Buchstaben eines beliebigen Blockelements (z. B. `p::first-letter`).
- `::selection`: Erlaubt es, Styles auf den (etwa mit der Maus) selektierten Text auszuführen.
- `::before`: Ermöglicht es, vor dem selektierten Element Text einzufügen (z. B. `p::before {content: "Text:"}` fügt vor allen Absätzen auf der Seite das Wort „Text:“ ein). Da sie Inhalt *erzeugen*, brauchen Sie die Eigenschaft `content`.
- `::after`: Fügt Text nach dem selektierten Element ein.
- `::marker`: Erlaubt es, Listenzeichen von `<li>`-Elementen zu formatieren. Dieses kann mit `content` gesetzt werden.
- `::placeholder`: Formatiert die Platzhalter-Angabe in einem Eingabfeld.

Das HTML-Markup besteht nur aus einem Absatz:

```
1 <body>
2   <p>Dieser Absatz hat so viel Text, dass er bei schmalem Browserfenster umbricht
   und somit weitere Zeilen entstehen. Beim Markieren wird der Bereich gelb
   hinterlegt.</p>
3 </body>
```

Der CSS-Code enthält drei der erwähnten Pseudoelemente, die auf Absätze wirken. Das Ergebnis ist in Abbildung 3.10 dargestellt.

```
1 p::first-line {
2   color: fuchsia;
3 }
4 p::first-letter {
5   font-size: xx-large;
6 }
7 p::selection {
```

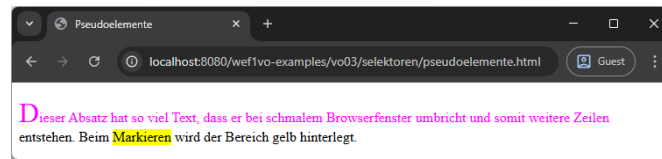



Abbildung 3.10: Die ersten Zeile des Absatzes bekommen über das Pseudoelement die Farbe „Fuchsia“, der erste Buchstabe wird größer dargestellt. Wird ein Bereich selektiert, wird er gelb hinterlegt.

```
8 background-color: yellow;
9 }
```

3.4.11 Pseudoklassen

Pseudoklassen erlauben es, ein Element mit bestimmten Eigenschaften bzw. in einem bestimmten Zustand zu selektieren. Der Selektor besteht aus dem Element, einem Doppelpunkt und der Pseudoklasse, also z. B. `a:link` für einen unbesuchten oder `a:visited` für einen besuchten Link.

Einige wichtige Pseudoklassen:

- `:first-child`: Selektiert jenes Element, welches das erste Kindelement eines anderen Elements ist. Enthält ein `<div>`-Element etwa drei Absätze (`<p>`), selektiert `div p:first-child` nur den ersten der drei.
- `:link`: Selektiert nur jene Links, die noch nicht besucht wurden (z. B. `a:link`).
- `:visited`: Selektiert nur jene Links, die bereits besucht wurden (z. B. `a:visited`).
- `:hover`: Selektiert jenen Link, über den Benutzer*innen gerade die Maus bewegen (z. B. `a:hover`).
- `:active`: Selektiert jenen Link, der gerade angeklickt wird (zwischen Mausklick und Loslassen der Taste – `a:active`).
- `:focus`: Selektiert jenes Element, das zur Zeit den Fokus hat (`a:focus`, wenn der Link z. B. mit Tab angewählt wurde).
- `:nth-child()`, `:nth-last-child()`: Selektiert das n -te Element basierend auf ihrer Position. Erlaubt als Argument einfache Skalare (z. B. `li:nth-child(3)`) oder eine Angabe im Format $a \cdot n + b$. a und b sind beliebige skalare Werte, n entspricht einer Zählervariable (wie etwa i in for-Schleifen), die von 0 weg hochgezählt wird. `li:nth-child(3n+5)` selektiert also das 5., 8., 11. usw. `<li>`-Element. `:nth-last-child()` beginnt von hinten zu zählen.
- `:nth-of-type()`, `:nth-last-of-type()`: Hat dieselbe Syntax wie `:nth-child()`, selektiert jedoch das n -te Element einer Art. `section p:nth-child(3)` wählt etwa den dritten Absatz innerhalb eines Abschnitts aus, selbst wenn darin auch `<div>`-Elemente vorkommen. `:nth-last-of-type()` zählt von hinten.
- `:only-child`: Selektiert Elemente, die keine Geschwister haben.
- `:only-of-type`: Selektiert Elemente, die keine Geschwister desselben Typs haben.
- `:empty`: Selektiert leere Elemente.
- `:root`: Wählt das Wurzelement – in HTML immer `<html>` – aus.

- **:target**: Formatiert ein Verweisziel. Wird etwa ein `<section>` Element mit einem internen Link angesprungen, kann mit `section:target` dieser Abschnitt z.B. visuell hervorgehoben werden.
- **:enabled**, **:disabled**, **:checked**: Damit können Formularelemente ausgewählt werden, die aktiv, inaktiv oder ausgewählt sind (letzteres nur für Checkboxes und Radio Buttons).

In folgendem HTML-Snippet kommen ein Link in einem Absatz, eine Liste und ein Abschnitt mit Absätzen und einem allgemeinen Bereich vor.

```

1 <body>
2   <p>Die <a href="https://www.fh-ooe.at/">FH OÖ</a> im Web.</p>
3   <ol>
4     <li>Ein Listenelement.</li>
5     <li>Noch eins.</li>
6     <li>Und noch eins.</li>
7     <li>Und ein viertes.</li>
8   </ol>
9   <section>
10    <p>Ein Absatz.</p>
11    <div>Ein generischer Bereich.</div>
12    <p>Noch ein Absatz.</p>
13  </section>
14 </body>

```

Der CSS-Code enthält eine Pseudoklasse, die den Link einfärbt, eine weitere selektiert alle geraden Listenelemente und die dritte formatiert lediglich den zweiten Absatz innerhalb eines Abschnitts. Das Ergebnis ist in Abbildung 3.11 dargestellt.

```

1 a:link {
2   color: #FF0000;
3   font-size: large;
4 }
5 li:nth-child(2n) {
6   color: green;
7 }
8 section p:nth-of-type(2) {
9   background-color: lightgray;
10 }

```

Funktionale Pseudoklassen

Neben den ursprünglichen Pseudoklassen, die Zustände eines Elements beschreiben, existieren auch sogenannte *funktionale Pseudoklassen*. Sie enthalten andere Selektoren in Klammern und dienen dazu, komplexe Regeln zu vereinfachen oder zu kombinieren.

:is(): Erlaubt es, mehrere Selektoren zusammenzufassen. Jedes Element, das durch einen der darin enthalten Selektoren angesprochen wird, wird auch formatiert (in diesem Fall Überschriften erster und zweiter Ordnung). Diese Pseudoklasse ist vor allem bei komplizierteren Selektoren (meist Kombinatoren) nützlich.

```

1 :is(h1, h2) {
2   color: purple;
3 }

```

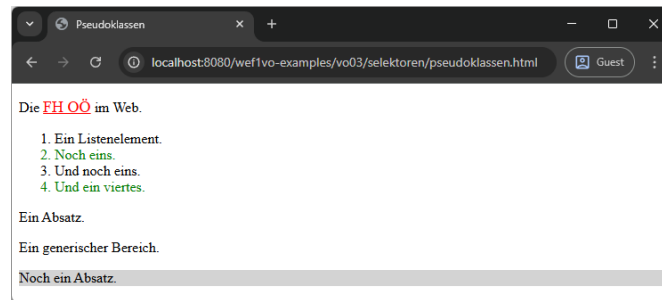


Abbildung 3.11: Die Pseudoklasse `a:link` färbt den Link nur dann rot, wenn er noch nicht besucht wurde, jedes zweite (gerade) Listenelement wird grün dargestellt und nur der zweite Absatz in einem Abschnitt erhält einen hellgrauen Hintergrund.

Neben `:is()` existiert auch noch `:where()`, das quasi identisch funktioniert. Unterschiede existieren nur in der Berechnung der Specificity. Bei `:is()` erhält der Selektor den Wert des spezifischsten Elements, bei `:where()` ist die Specificity immer 0. Mehr dazu in Abschnitt 3.6.6. Hier werden alle Überschriften von der ersten bis zur dritten Ordnung selektiert:

`:not()`: Schließt Elemente aus, die einem bestimmten Selektor entsprechen. So selektiert etwa die folgende Angabe alle Absätze *bis* auf den ersten.

```
1 p:not(:first-child) {
2   color: blue;
3 }
```

`:has()`: Diese Pseudoklasse wählt Elemente aus, die bestimmte Nachfahren enthalten oder Geschwister haben. Im folgenden Beispiel werden etwa nur Artikel formatiert, die auch einen Absatz enthalten:

```
1 article:has(p) {
2   background-color: lightgray;
3 }
```

Abbildung 3.12 zeigt ein Beispiel mit folgendem Markup, in dem die obigen Selektoren zur Anwendung kommen.

```
1 <h1>Überschrift</h1>
2
3 <article>
4   <h2>Artikelüberschrift</h2>
5 </article>
6
7 <article>
8   <h2>Artikelüberschrift</h2>
9   <section>
10    <p>Etwas Text davor</p>
11    <h3>Abschnittsüberschrift</h3>
12    <p>Etwas Text danach</p>
13  </section>
14 </article>
```

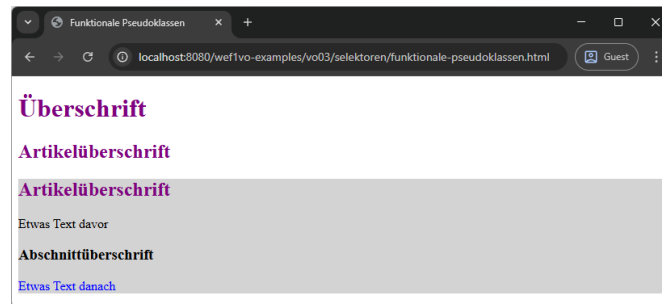


Abbildung 3.12: Alle `<h1>`- und `<h2>`-Überschriften sind violett, nur der Artikel mit den Absätzen ist hellgrau hinterlegt und der erste Absatz im Artikel wird *nicht* formatiert.

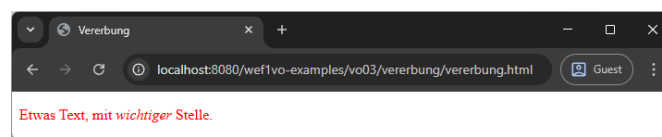


Abbildung 3.13: Vererbung bewirkt, dass auch der Inhalt des `<em>`-Elements rot eingefärbt wird.

Unter <https://flukeout.github.io/> können die verschiedenen CSS-Selektoren spielerisch erlernt und ausprobiert werden.

3.5 Vererbung von Eigenschaften

Selektoren bestimmen, auf welches Element eine Formatierung angewandt wird. Doch oft ist es nicht nur dieses Element, sondern auch alle Nachfahren (d. h. im Element verschachtelten Elemente), die diese Formatierung erhalten. Die Ursache ist der *Vererbungsmechanismus* (engl. „inheritance“), der in CSS zum Einsatz kommt.

Dieses Verhalten wird in der Regel kaum bewusst wahrgenommen, da es sehr natürlich implementiert ist. Gegeben sei folgendes Beispiel:

```
1 <p>Etwas Text, mit <em>wichtiger</em> Stelle.</p>
```

Nun soll mit der folgenden Regel der Absatz rot eingefärbt werden:

```
1 p {
2   color: #FF0000;
3 }
```

Die Vererbung bewirkt, dass der gesamte Absatz rot dargestellt wird (Abbildung 3.13). Gäbe es diesen Mechanismus nicht, wäre der betonte Teil weiterhin schwarz dargestellt und müsste separat formatiert werden. Soll der Inhalt von `<em>` tatsächlich schwarz dargestellt werden, müsste er noch extra formatiert werden.

Ob eine Eigenschaft vererbt wird, ist in der CSS-Spezifikation enthalten. Der Großteil aller Eigenschaften, wie Schriftart und -größe, Textfarben und -betonungen etc. wird vererbt.

Nicht vererbt werden Hintergründe (Hintergrundfarben und -bilder) sowie Rahmen und Abstände. Im Beispiel von Rahmen wäre dies absolut nicht wünschenswert. Nimmt

man das Beispiel von oben und setzt um den Absatz einen Rahmen, so würde das Wort im `<em>`-Element bei Vererbung einen eigenen kleinen Rahmen bekommen. Da dies in den seltensten Fällen gewünscht ist, ist hier die Vererbung außer Kraft gesetzt.

Soll eine Vererbung trotzdem erzwungen werden, kann dies mit dem Schlüsselwort `inherit` geschehen. Es greift auf die Werte von übergeordneten Elementen zu und verwendet diese für das aktuelle Element.

Folgender CSS-Code würde bewirken, dass alle Elemente auf der Seite rote Schrift und gelben Hintergrund verwenden. Die Farben sind für `<body>` definiert, alle anderen Elemente werden angewiesen, diese Werte zu übernehmen. Die `!important` Deklaration verleiht den Anweisungen zusätzliche Wichtigkeit und verhindert, dass sie gegebenenfalls überschrieben werden.

```
1 body {
2   color: red !important;
3   background-color: yellow !important;
4 }
5 * {
6   color: inherit !important;
7   background-color: inherit !important;
8 }
```

Um eine Eigenschaft wieder auf den Standardwert zu setzen, kann das Schlüsselwort `initial` verwendet werden. Mit `unset` wird eine Eigenschaft zurückgesetzt. Dies entspricht `inherit`, wenn sie vererbt war, sonst `initial`. `revert` setzt eine Eigenschaft ebenfalls zurück: auf `inherit` wenn sie vererbt war, oder sonst auf den Wert aus dem Browser-Stylesheet.

3.6 Die Kaskade

Das „C“ in CSS steht für „Cascading“, zu deutsch „kaskadierend“ (oder kaskadenförmig) und weist auf den Mechanismus hin, der bei der Bestimmung der zutreffenden Formatierung zur Anwendung kommt. Bevor ein Element sein Aussehen bekommt, wird mit diesem Regelwerk (definiert in [69, Abschnitt 6]) bestimmt, welches Aussehen dies ist, denn oftmals gibt es mehrere Definitionen, die zum Zug kommen könnten.

3.6.1 Filterung

Bevor die eigentliche Kaskade abgearbeitet wird, findet eine **Filterung** statt [69, Abschnitt 5]). D. h., es werden alle Deklarationen (CSS-Regeln) für das zu formatierende Element identifiziert. Dazu werden alle eingebundenen Stylesheets nach gültigen und für dieses Element zutreffenden Deklarationen durchgesucht und diese in einer Liste für die Kaskade zur Verfügung gestellt. Diese arbeitet sie dann nach folgenden Kriterien ab.

3.6.2 Herkunft und Wichtigkeit

Die Herkunft einer Deklaration basiert darauf, woher sie stammt, und ihre Wichtigkeit hängt davon ab, ob sie mit `!important` (siehe Abschnitt 3.2) deklariert ist oder nicht.

Die Rangfolge der verschiedenen Ursprünge und Wichtigkeiten lautet in aufsteigender Reihenfolge, also von der unwichtigsten bis zur wichtigsten Herkunft:

1. **Deklaration durch das *Browser-Stylesheet*:** Browser geben für einige Elemente ein Standard-Aussehen vor, etwa die Größe der einzelnen Überschriften etc.
2. **Deklaration durch das *User-Stylesheet*:** Benutzer*innen können beim Browser ein eigenes Stylesheet hinterlegen, das z. B. auf allen Seiten die Schrift prinzipiell größer darstellt.
3. **Deklaration durch das *Author-Stylesheet*:** Das Stylesheet, das zusammen mit der Website erstellt wird.
4. **Deklaration aus *CSS-Animationen*:** Animierte Werte werden berechnet und müssen eine höhere Priorität als normale Regeln haben, damit sie immer korrekt funktionieren.
5. ***Important-Deklaration im Author-Stylesheet*:** Alle Style-Angaben mit dem Schlüsselwort `!important` werden als wichtiger betrachtet.
6. ***Important-Deklaration im User-Stylesheet*:** Noch mehr Gewicht, um für sich persönlich im Browser Angaben endgültig überschreiben zu können.
7. ***Important-Deklaration im Browser-Stylesheet*:** In Browser-Stylesheets werden mit `!important`-Regeln Dinge gesetzt, die nicht überschrieben werden können, um böses Verhalten zu verhindern. So ist es Webseiten-Autor*innen etwa nicht möglich, die Maskierung bei Passwortfeldern nicht entfernen.
8. **Deklaration aus *CSS-Transitions*:** Übergänge zwischen CSS-Werten. Diese werden zuletzt berechnet, da sie auf Ergebnissen der Kaskade beruhen.

Arten von Stylesheets

Wie also ersichtlich, gibt es drei Arten von Stylesheets: Das *Browser-Stylesheet*, das *User-Stylesheet* und das *Author-Stylesheet*. Wichtig für die Lehrveranstaltung ist das *Author-Stylesheet*, welches im weiteren Verlauf dieses Skriptums nur mehr schlicht als „Stylesheet“ bezeichnet wird. Ist vom Browser- oder User-Stylesheet die Rede, so werden diese explizit so benannt.

Browser-Stylesheet: Werden in einem HTML-Dokument keine CSS-Angaben verwendet, gibt der Browser das Aussehen in einigen Punkten durch ein so genanntes Browser-Stylesheet (auch Default-Stylesheet oder User Agent Stylesheet) vor. Dieses wird im HTML Living Standard [54, Abschnitt 15] spezifiziert. Browserhersteller sind dazu angehalten, dies genauso zu implementieren². Daher sehen unformatierte Seiten in fast jedem Browser gleich aus.

Da das Browser-Stylesheet in der Herkunftsreihenfolge der Kaskade die niedrigste Priorität erhält, sind alle anderen Style-Angaben wichtiger und überschreiben daher diejenigen im Browser-Stylesheet. Das Browser-Stylesheet ist jedoch meist die Ursache für Formatierungen, die nicht explizit gesetzt wurden (z. B. vertikaler Abstand vor

²Hier ist etwa das Stylesheet, das in Chrome/Chromium zum Einsatz kommt: <https://chromium.googlesource.com/chromium/blink/+refs/heads/main/Source/core/css/html.css>

und nach den diversen Überschriften). Dies sollte immer im Hinterkopf behalten werden. Mit **!important** markierte Regeln aus dem Browser-Stylesheet haben jedoch (mit Ausnahme von Transitions) das höchste Gewicht. Dies nützen die Browser um einige sicherheitskritische Formatierungen zu definieren, die nicht überschrieben werden sollen.

User-Stylesheet: User-Stylesheets ermöglichen Benutzer*innen, eigene Formatierungen vorzunehmen, die sich auf alle im Browser betrachteten Webseiten beziehen. In der Regel wird diese Möglichkeit weniger genutzt, Personen mit schlechter Sehkraft können aber etwa in einem User-Stylesheet die Schriftgröße anpassen. Ein anderes Szenario wäre, mit Hilfe des User-Stylesheets Schrift- und Hintergrundfarbe neutral zu gestalten.

In Mozilla Firefox lässt sich dies durch Änderungen in der Datei `userContent.css` bewerkstelligen, welche sich im Profil-Ordner befindet, wenn dies zuvor in den erweiterten Einstellungen aktiviert wurde. Andere Browser, wie etwa Google Chrome, haben die Unterstützung für User-Stylesheets bereits seit längerem eingestellt³, es gibt jedoch Extensions wie Stylus⁴, die dafür verwendet werden können.

3.6.3 Kontext

Dieser Schritt ist wichtig für fortgeschrittenere CSS-Techniken wie Webkomponenten. CSS lässt mehrere Kontexte zu, so kann eine Seite ihre normalen CSS-Regeln beinhalten (äußerer Kontext) und dann eine externe Komponente (z. B. ein Kalender-Widget) inkludieren, die ihr eigenes CSS mitbringt, mit dem sie formatiert wird (innerer Kontext). Um zu ermöglichen, dass dieses Kalender-Widget im Stile der Seite angezeigt wird, haben die Regeln der Seite, also der äußere Kontext, Vorrang gegenüber dem inneren (den Regeln des Widgets). Bei **!important**-Regeln gewinnen jedoch die Regeln des inneren Kontexts, damit dieser Style-Angaben setzen kann, die von außen nicht überschrieben werden können.

3.6.4 Element-Attached-Styles (Inline-Styles)

Eine CSS-Regel, die direkt bei einem HTML-Element (mit dem `style`-Attribut, siehe Abschnitt 3.3.1) steht, schlägt immer eine CSS-Regel derselben Wichtigkeit aus einem Stylesheet.

3.6.5 Layers

Mit der Regel `@layer` können CSS-Regeln in eigene Schichten eingeteilt werden, was innerhalb einer Herkunft nochmals eine feinere Unterteilung ermöglicht.

Werden etwa mit

```
1 @layer reset, base, components, utilities;
```

vier Schichten definiert, so gewinnen die CSS-Regeln des weiter hinter deklarierten Layers. Regeln im `utilities`-Layer haben also mehr Gewicht als die in `components` usw.

³<https://issues.chromium.org/issues/41091633>

⁴<https://chromewebstore.google.com/detail/stylus/clngdbkpkpeebahjckkfobafhncgmne>

Für Regeln mit `!important` ist es genau umgekehrt. Hier schlagen die Angaben in `reset` die aus `base` usw.

Layers sind ein sehr neues und mächtiges Konzept, das in Vorlesung 5 etwas detaillierter vorgestellt wird.

3.6.6 Specificity (Gewichtung)

Die *Spezifität* (engl. „specificity“) besagt, dass im Falle von zwei im selben Dokument für dasselbe Element spezifizierten Regeln die spezifischere zum Zug kommt. Ein Beispiel wären die folgenden beiden Regeln:

```
1 body h1 {  
2   color: green;  
3 }  
4 h1 {  
5   color: red;  
6 }
```

Beide formatieren `<h1>`-Elemente (einmal rot, einmal grün). Wie lässt sich nun sagen, welche Regel spezifischer ist? CSS sieht dafür einen einfachen, numerischen Mechanismus vor, mit dem sich die Specificity ausrechnen lässt.

Jeder Selektor besitzt ein Gewicht, das in einer dreiteiligen Form beschrieben wird: 0,0,0. Für jeden Teil des Selektors werden nun bestimmte Werte (Gewicht) hinzugezählt:

- ID-Selektoren zählen 1,0,0,
- Klassenselektoren, Pseudoklassen und Attributselektoren zählen 0,1,0,
- Elemente und Pseudoelemente zählen 0,0,1,
- der Universalselektor hat keine Auswirkung, zählt also 0,0,0.

Die funktionalen Pseudoklassen `:is()`, `:has()` und `:not()` selbst tragen nicht zum Gewicht des Selektors bei, sondern ihre Inhalte. Dabei wird zählt nur die Angabe mit der höchsten Gewichtung. `:is(h1, .red, #content)` etwa zählt 1,0,0 (aufgrund des ID-Selektors). Die Ausnahme ist die funktionale Pseudoklasse `:where()`. Sie zählt wie der Universalselektor immer 0,0,0 (unabhängig von ihrem Inhalt).

Anmerkung: Angaben mit dem `style`-Attribut im HTML-Markup sind nicht (mehr) Teil der Specificity-Berechnung, sondern in einer eigenen Ebene der Kaskade (Element-Attached-Styles, Abschnitt 3.6.4) und haben immer höheres Gewicht. Historisch wird dies manchmal noch mit einer vierten Gewichtsangabe dargestellt: 1,0,0,0.

CSS-Regeln mit dem `!important`-Schlüsselwort unterliegen nicht der Specificity-Berechnung mit „normalen“ Regeln, da auch hier im Punkt „Reihenfolge“ in der Kaskade ein anderer Abschnitt zum Zug kommt. Gibt es aber mehrere Regeln mit `!important`-Angaben, die dasselbe Element betreffen, so wird zwischen diesen ebenso die Specificity berechnet.

Für die obigen Angaben lässt sich damit folgendermaßen rechnen:

- `body h1 { color: green; }`: $2 \times$ Elementselektor ergibt 0,0,2.
- `h1 { color: red; }`: $1 \times$ Elementselektor ergibt 0,0,1.

Somit ist die erste Regel spezifischer (da der Wert höher ist) und kommt zum Zug.

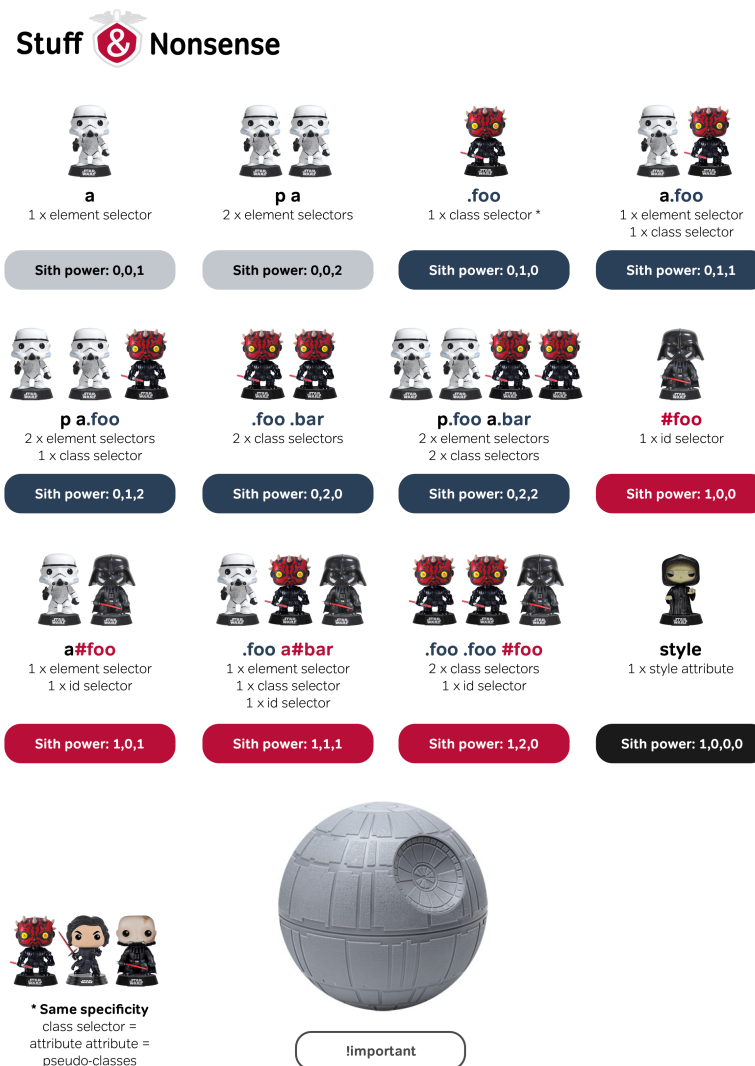


Abbildung 3.14: Je spezifischer ein Selektor, desto höher die Sith-Power: Storm Trooper (0,0,1), Darth Maul (0,1,0), Darth Vader (1,0,0), Emperor (alte Syntax für Inline-Styles 1,0,0,0). Der Todesstern (!important) zerstört alles, da dadurch eine andere Ebene in der Kaskade erreicht wird. Bildquelle: [9].

Vorsicht: Die vier Stellen der Specificity verhalten sich nicht wie eine natürlich Zahl im 10er-System. Würde etwa eine Specificity von 0,0,9 (durch etwa 9 Typselektoren in einer Angabe) bestehen und würde in die CSS-Regel ein weiterer Elementselektor hinzugefügt werden, so ergäbe sich eine Specificity von 0,0,10 und nicht 0,1,0. Somit ist eine CSS-Klasse immer höherwertiger als 10 Elementselektoren.

Abbildung 3.14 stellt einige weitere Beispiele grafisch dar.

Unter <https://specificity.keegan.st/> gibt es einen Specificity-Rechner, der bei Angabe des CSS-Selektors die Specificity errechnet.

3.6.7 Reihenfolge

Sind zwei Angaben nach diesen Regeln immer noch gleichwertig (gleiche Herkunft, gleicher Kontext, gleicher Layer, gleiche Specificity), so „gewinnt“ die später definierte (also im Stylesheet „weiter unten“ angegebene).

3.7 Werte an CSS-Eigenschaften übergeben

CSS-Eigenschaften akzeptieren unterschiedliche Arten von Werten. Dies können numerische Angaben, Zeichenketten, Schlüsselwörter, Farben oder auch Winkelmaße sein.

3.7.1 Numerische Angaben

Eigenschaften, die numerische Angaben benötigen (wie die Schriftgröße oder die Breite eines Elements) akzeptieren eine Reihe an unterschiedlichen Einheiten. Unterschieden wird zwischen absoluten und relativen Werten.

Absolute Werte

Folgende *absolute Werte* sind in CSS zulässig:

- **mm**: Millimeter,
- **cm**: Zentimeter (1 cm = 100 mm),
- **in**: Zoll (Inch, 1 in = 2,54 cm),
- **pt**: Punkt (Point, 1 pt = 1/72 in),
- **pc**: Pica (1 pc = 12 pt),
- **px**: Pixel (1 px = 0,75 pt).

Gebräuchlich sind **px** bei Breitenangaben von Elementen (da der Bezug die Bildschirmauflösung ist und so nicht umgerechnet werden muss) und Schriftgrößen im Bezug auf die Bildschirmdarstellung. Bei Stylesheets für den Druck sind in der Regel die anderen Maßeinheiten angebracht. Pixel sind genau genommen auch eine relative Einheit, da Pixel sehr stark vom Ausgabegerät und dessen Pixeldichte und Auflösung abhängen.

Relative Werte

Zu den *relativen Angaben* zählen:

- **em**: Bezieht sich auf die Schriftgröße (**font-size**), die für das aktuelle Element eingestellt ist. Diese kann explizit gesetzt oder vererbt sein. Hat etwa ein Absatz die Schriftgröße **12px** vererbt bekommen (weil sie z. B. im für `<body>` gesetzt wurde) und wird dann die Regel `p { font-size: 3em }` gesetzt, so hat der Absatz eine Größe von **36px**.
- **rem**: Funktioniert wie **em**, bezieht sich jedoch immer auf die Schriftgröße des Wurzelements (**rem** bedeutet „root em“) und erlaubt daher eine konstante Verwendung von **em**-Werten im Stylesheet.
- **ex**: Diese Einheit ist durch die erste für das Element verfügbare Schriftart definiert und bezieht sich auf deren Höhe des Buchstabens „x“. Diese Größe ist in der Regel bei der Schriftart selbst hinterlegt.

- **ch**: Diese Einheit definiert sich über die Breite des Zeichens „0“.
- **lh**: Die Zeilenhöhe des Elements.
- **rlh**: Die Zeilenhöhe des Wurzelements (analog zu **rem**).
- **vw**: Entspricht 1 % der Anzeigenbreite (viewport width), also der Breite des Browserfensters.
- **vh**: Entspricht 1 % der Anzeighöhe (viewport height), also der Höhe des Browserfensters.
- **vmin**: Entspricht 1 % der kleineren Seite der Anzeige (viewport min).
- **vmax**: Entspricht 1 % der größeren Seite der Anzeige (viewport max).
- **vb** und **vi**: Viewport Block und Viewport Inline Eineinheiten. Entsprechen je 1 % der Breite bzw. Höhe, je nach eingestellter Schreibrichtung. Bei horizontaler Richtung ist 1 **vb** gleich 1 **vh** und 1 **vi** gleich 1 **vw**.
- **svw** und **svh**: Wie **vw** und **vh** nur für den „small viewport“, also der kleinstmögliche Anzeigebereich bei dem der Platz für die User-Interface-Elemente des Browsers schon abgezogen wurde.
- **lvw** und **lvh**: Wie **vw** und **vh** nur für den „large viewport“, also der größtmögliche Anzeigebereich. Hier könnten Inhalte aber durch UI-Elemente verdeckt werden.
- **dvw** und **dvh**: Wie **vw** und **vh** nur für den „dynamic viewport“. Hier können sich die Größen ändern, wenn UI-Elemente verschwinden bzw. erscheinen.
- **%**: Prozentangaben funktionieren ähnlich wie die **em**-Angaben, nur ist der Bezugspunkt nicht nur die Schriftgröße, es können z. B. auch Längenangaben eines übergeordneten Elements sein.

Gebräuchlich sind hier vor allem die Angaben in **em**, **rem** und **%**, auch die Viewport-bezogenen Angaben nehmen stark an Bedeutung zu. Relative Angaben sind vor allem dann vorteilhaft (oder überhaupt notwendig), wenn sich das Stylesheet auf unterschiedlichen Ausgabegeräten (Stichwort: responsive Web) funktionieren soll.

3.7.2 Zeichenketten und Schlüsselwörter

Zeichenketten, oder vereinfacht ausgedrückt Text, muss in CSS in einfache oder doppelte Anführungszeichen gesetzt werden. Gebraucht wird dies etwa, um Text mit der **:before** Pseudoklasse einzufügen:

```
1 p:before {  
2   content: "Text der davor eingefügt wird";  
3 }
```

Viele CSS-Eigenschaften hingegen verlangen Schlüsselwörter. Dies sind vordefiniert und dürfen *nicht* unter Anführungszeichen gesetzt werden. Beispiele sind etwa Farban-gaben oder das Schriftgewicht.

```
1 p {  
2   color: blue;  
3   font-weight: bold;  
4 }
```

3.7.3 Farben

Farben können in CSS auf mehrere Arten definiert werden. Einige davon waren bereits in den vorangehenden Abschnitten im Einsatz. Diese sind:

- benannte Farben (Schlüsselwörter),
- Hexadezimalschreibweise,
- RGB-Angabe,
- HSL-Angabe,
- Lab- und Oklab-Angaben,
- LCh- und Oklch-Angaben,
- HWB-Angabe.

Benannte Farben

Der CSS 2.1 Standard [62] definiert 17 verschiedene Schlüsselwörter für Farben, nämlich **black**, **maroon**, **green**, **olive**, **navy**, **purple**, **teal**, **silver**, **gray**, **red**, **orange**, **lime**, **yellow**, **blue**, **fuchsia**, **aqua** und **white**. In der Color Module Level 4 Spezifikation [70] werden noch weitere 133 Schlüsselwörter definiert, sodass nun insgesamt 150 zulässige Schlüsselwörter für Farben existieren. Eine Übersicht findet sich bei MDN⁵.

```
1 body {  
2   color: rebeccapurple;  
3 }
```

Hexadezimalschreibweise

Seit der ersten CSS-Version können Farben in Hexadezimalschreibweise notiert werden. Hinter dem Rautezeichen (#) folgen drei Hexadezimalwerte zwischen 00 (für 0) und FF (für 255). Diese spezifizieren den jeweiligen RGB (rot, grün, blau) Anteil.

Im folgenden Beispiel wird einmal die Farbe weiß für die Schriftfarbe und die Farbe blau für den Hintergrund definiert.

```
1 section {  
2   color: #FFFFFF;  
3   background-color: #0000FF;  
4 }
```

Sind die erste und zweite Ziffer einer Farbangabe identisch, kann eine verkürzte Schreibweise verwendet werden. So steht #FFF für #FFFFFF oder #00F für #0000FF.

RGB-Angabe

RGB-Farbangaben können auch mit einer Dezimal- oder Prozentschreibweise erreicht werden. Dafür steht die CSS-Funktion **rgb()** zur Verfügung. Eingetragen werden Zahlen von 0 bis 255. So wird das Beispiel mit blauem Hintergrund und weißer Schriftfarbe wie folgt geschrieben. Der Hintergrund ist dabei zu 50 % Transparent – der Alphawert wird zwischen 0.0 (völlig transparent) und 1.0 (völlig opak) angegeben und durch einen Schrägstrich getrennt.

⁵<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/named-color>

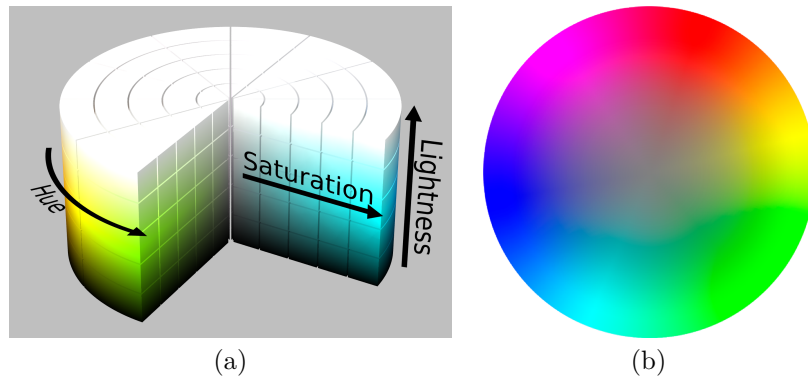


Abbildung 3.15: Der HSL-Zylinder (a) und der zugehörige Farbkreis (b). Hue ist der Winkel am Farbkreis, Saturation ist der Radius, Lightness ist die Höhe des Zylinders. Rot liegt bei 0°, Gelb bei 60°, Grün bei 120°, Cyan bei 180°, Blau bei 240° und Magenta bei 300°. Quellen: [25, 49]

```
1 section {
2   color: rgb(255 255 255);
3   background-color: rgb(0 0 255 / 0.5);
4 }
```

Alternativ können die Farbangaben auch in Prozentangaben erfolgen. Weiß wäre somit `rgb(100% 100% 100%)`.

HSL-Angabe

HSL steht für *Hue*, *Saturation*, *Lightness*, das Farbsystem wird durch einen Zylinder repräsentiert (Abbildung 3.15 (a)). Hue repräsentiert die Grundfarbe aus dem Farbkreis (Abbildung 3.15 (b)), Saturation gibt die Sättigung über den Radius an, je höher der Wert, desto größer ist der Radius. Schließlich definiert Lightness die Helligkeit. 50 % entspricht dabei der normalen Helligkeit, 100 % entspricht immer Weiß, 0 % immer Schwarz. Eine HSL-Farbdefinition ist mit dem Schlüsselwort `hsl()` möglich, eine optionale Angabe der Transparenz erfolgt mittels Schrägstrich.

Anbei die Farbangaben für weiße Schrift und blauen, halbtransparenten Hintgrund:

```
1 section {
2   color: hsl(0 0% 100%);
3   background-color: hsl(240 100% 50% / 0.5);
4 }
```

Lab-, Oklab, LCh-, Oklch und LWB-Angaben

In der CSS Color Module Level 4 Spezifikation [70] wurden neuen Möglichkeiten zur Farbspezifikation eingeführt.

- Mit `lab()` können CIELAB-Farben im Lab-Farbraum (Helligkeit, rot-grün-Wert, gelb-blau-Wert) definiert werden.
- Mit `oklab()` wird selbiges für den etwas moderneren Oklab-Farbraum definiert.

- `lch()` erlaubt es CIELAB-Farben mittels Lightness, Chroma und Hue-Werten anzugeben.
- `oklch` ermöglicht selbiges für den Oklab-Farbraum.
- Mit `hwb()` werden Farben durch Hue, Whiteness und Blackness definiert.

3.7.4 Winkelmaße und Zeitangaben

Für CSS-Transformationen (z. B. Rotation) werden Winkelmaße benötigt, für Zeitangaben stehen ebenfalls zwei Einheiten zur Verfügung.

Winkelmaße

Winkelangaben können in vier verschiedenen Einheiten angegeben werden:

- Grad: Mit der Einheit `deg` werden Grad angegeben. Ein Kreis entspricht 360 Grad (`360deg`).
- Gon: Mit `grad` werden Gon angegeben, ein Kreis entspricht 400 Gon (`400grad`).
- Radian: Das Bogenmaß wird mittels `rad` spezifiziert. Ein Kreis entspricht $2 \cdot \pi$, also gerundet 6.28 (`6.28rad`).
- Vollwinkel: Die Einheit `turn` steht für eine Kreisumrundung, ein Kreis ist also `1turn`.

Mit dem folgenden Code wird ein `<div>`-Element um 90 Grad rotiert (Details zu Transformationen siehe [83]).

```
1 div {  
2   rotate: 90deg;  
3 }
```

Zeitangaben

In zeitbasierten Formatierungen (wie für Animationen oder Übergänge) werden Angaben für die Zeitdauer benötigt. Zur Verfügung stehen Sekunden (`s`) oder Millisekunden (`ms`).

Im folgenden Code wird ein Übergang 2 Sekunden verzögert.

```
1 a {  
2   transition-delay: 2s;  
3 }
```

3.8 Erste CSS-Formate

Nachdem in den vorigen Abschnitten aus Gründen der praktischen Erklärung schon einige Eigenschaften vorgestellt wurden, folgen hier nun exemplarisch einige ausführlicher erklärte CSS-Formate. Weitere können selbst in diversen Dokumentationen [39] nachgelesen werden.

Aufgrund der Menge an Selektoren wird nicht bei jedem ein Browser-Screenshot abgebildet.

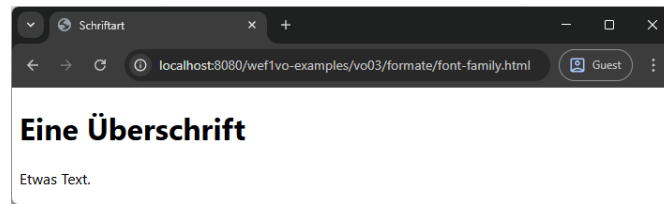


Abbildung 3.16: Per `font-family`-Regel wurde die Schriftart für das ganze Dokument geändert.

3.8.1 Schriften-Formatierung

Unter *Schriften-Formatierung* fallen alle Dinge, die mit Schriftarten, -größe und -stilen zu tun haben. Die dazu gehörigen CSS-Eigenschaften beginnen alle mit **font**.

Schriftart

Die *Schriftart* (engl. „font-family“) gibt die verwendete Schriftfamilie an. Die CSS-Eigenschaft lautet **font-family**. Dabei können nur Schriften verwendet werden, die auch lokal im System der jeweiligen Benutzer*innen installiert sind (z. B. Helvetica Neue oder Arial). Aus diesem Grund ist es ratsam, gleich mehrere Schriften durch Komma getrennt anzugeben. Findet der Browser die erste angegebene Schrift nicht, wird auf die nächste gewechselt usw. Als letzte Option sollte immer eine vordefinierte Schrift verwendet werden, die auf jeden Fall erkannt wird (diese lauten **serif**, **sans-serif**, **monospace**, **cursive**, **fantasy**, **system-ui**, **ui-serif**, **ui-sans-serif**, **ui-monospace**, **ui-rounded**, **math** und **fangsong**). Findet der Browser keine Schrift aus der angegebenen Liste, so wählt er selbst eine Schrift aus. Schriftarten, deren Namen Leer- oder andere Spezialzeichen enthalten, sollten unter Anführungszeichen gestellt werden. Gerade die **ui**-Varianten werden mittlerweile gerne vorab verwendet, um gezielt die Systemschriften zu verwenden.

Folgende Regel definiert für das gesamte Dokument (`<body>`) drei verschiedene System-Schriften, sowie eine Fallback-Option. Das Ergebnis ist in Abbildung 3.16 zu sehen:

```
1 body {  
2   font-family: system-ui, "Segoe UI", Roboto, Arial, sans-serif;  
3 }
```

Schriftgröße

Die *Schriftgröße* (engl. „font-size“) wird mit der Eigenschaft **font-size** definiert. Möglich sind numerische Angaben (sowohl absolut als auch relativ) sowie die vordefinierten Werte **xx-small**, **x-small**, **small**, **medium**, **large**, **x-large**, **xx-large**, **smaller** und **larger**.

Anbei eine Regel, die die Schriftgröße des gesamten Dokuments auf 16 Pixel setzt.

```
1 body {  
2   font-size: 16px;  
3 }
```

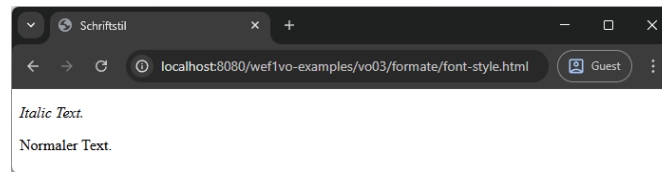


Abbildung 3.17: Der erste der beiden Absätze wird kursiv dargestellt.

Schriftgewicht

Das *Schriftgewicht* (engl. „font-weight“) bestimmt die Schriftstärke, also ob eine Schrift leicht, normal oder fett dargestellt wird. Es haben sich dabei 9 Stufen eingebürgert: Ultraleicht, extraleicht, leicht, halbleicht, normal, halbfett, fett, extrafett und ultrafett. Mit Hilfe der CSS-Eigenschaft **font-weight** lassen sich diese Stufen angeben. Verwendet werden dazu die numerischen Angaben von 100 bis 900 (in 100er Schritten), wobei 100 ultraleicht und 900 ultrafett entspricht. Darüber hinaus gibt es noch die Angaben **normal** (entspricht 500), **bold** (entspricht 700), **bolder** (entspricht 800) und **lighter** (entspricht 200). Der Browser entscheidet, wie er die Angaben den verfügbaren Schriftschnitten zuweist.

Folgende CSS-Regel stellt einen Absatz fett dar:

```
1 p {  
2   font-weight: bold;  
3 }
```

Schriftstil

Der *Schriftstil* (engl. „font-style“) bestimmt die Schriftlage, also ob der normale, kursive oder schräggestellte Schriftschnitt verwendet werden soll. Möglich ist dies mit der CSS-Eigenschaft **font-style**, verfügbare Werte sind **normal**, **italic** oder **oblique**. Bei **italic** wird, wenn verfügbar, der kursive Schriftschnitt einer Schriftart verwendet. **oblique** stellt lediglich den normalen Schriftschnitt schräg und sollte daher nur in begründeten Fällen verwendet werden.

Folgende CSS-Regel erstellt eine CSS-Klasse, die kursive Schrift erzeugt, in Abbildung 3.17 wird das Ergebnis dargestellt.

```
1 .important {  
2   font-style: italic;  
3 }
```

Schrift-Variante

Mit Hilfe der *Schrift-Variante* (engl. „font-variant“) kann Kapitälchen-Text erzeugt werden, also Text bei dem für die Kleinbuchstaben kleinere Großbuchstaben verwendet werden (wie z. B. bei den Wörtern WEB FUNDAMENTALS). Verantwortlich ist die Eigenschaft **font-variant** mit dem Wert **small-caps**. Der zweite mögliche Wert ist **normal**.

Durch die folgende CSS-Regel wird Text in `<em>`-Elementen als Kapitälchen dargestellt.


```

1 em {
2   font-variant: small-caps;
3 }

```

Shorthand-Syntax

Für die gesamte Formatierung von Schriften existiert eine so genannte *Shorthand-Syntax*, also eine Möglichkeit, alle Angaben in eine einzige CSS-Eigenschaft zu verpacken. Diese lautet **font**. Bei der Verwendung müssen Schriftgröße und -art zwingend am Ende der Regel vorkommen. Schriftgewicht, -variante und -stil sind optional und können in beliebiger Reihenfolge am Beginn der Regel stehen.

Folgender Code definiert alle obigen Eigenschaften in einer Regel:

```

1 body {
2   font: bold italic small-caps 16pt system-ui, "Segoe UI", Roboto, Arial, sans-serif;
3 }

```

Web Fonts

Eine der größten Schwachstellen im Web war seit langem das Kapitel Typographie. Website-Autor*innen waren meist auf einige wenige Systemschriftarten limitiert, die auf dem Client-Rechner installiert sein mussten, um korrekt angezeigt zu werden. Um kein Risiko einzugehen, wurde dabei auf ein kleines Set an Schriften zurückgegriffen, die überall vorhanden, aber keineswegs immer passend waren. Das Modul *CSS Fonts Module Level 3* [72] beinhaltet Möglichkeiten, herunterladbare Schriften zu verwenden.

Zentrales Element der Web Fonts ist die **@font-face**-Regel. Sie gibt zusätzliche Schriften an und definiert, wo diese zu finden sind. Folgende Regel definiert die Schriftart *Noto Sans* und gibt den URL zum Download an, sodass sie vom Webbrowser herunter geladen werden kann.

```

1 @font-face {
2   font-family: "Noto Sans";
3   src: url("noto-sans-v42-latin-regular.woff2") format("woff2");
4 }

```

In dem obigen Beispiel wird die (freie) Schriftart Noto Sans (<https://fonts.google.com/noto>) mit Hilfe der **font-family**-Angabe spezifiziert. Der Name kann prinzipiell frei gewählt werden, sollte aber idealerweise den genauen Schriftartnamen wiedergeben. **src** definiert den Pfad, an dem die Schrift verfügbar ist. In diesem Fall ist es ein relativer Pfad im selben Verzeichnis, es können aber auch absolute Pfade angegeben werden. Die **format**-Angabe am Ende spezifiziert den Typ der Schrift näher, kann aber auch entfallen.

Soll nun ein zweiter Schriftschnitt (z. B. kursiv) angegeben werden, muss eine zweite **@font-face**-Regel angelegt werden:

```

1 @font-face {
2   font-family: "Noto Sans Italic";
3   src: url("noto-sans-v42-latin-italic.woff2") format("woff2");
4 }

```

Beide Schriften sollen nun im folgenden HTML-Fragment zur Anwendung kommen:

```

1 <body>
2   <h1>Überschrift</h1>
3   <p>Erster Absatz.</p>
4   <p>Zweiter Absatz.</p>
5 </body>

```

Der CSS-Code dazu enthält zunächst die `@font-face`-Regeln und wendet die neu definierten Schriften dann wie gewohnt an.

```

1 @font-face {
2   font-family: "Noto Sans";
3   src: url("noto-sans-v42-latin-regular.woff2") format("woff2");
4 }
5 @font-face {
6   font-family: "Noto Sans Italic";
7   src: url("noto-sans-v42-latin-italic.woff2") format("woff2");
8 }
9 h1 {
10  font-family: "Noto Sans", Arial, sans-serif;
11 }
12 p {
13  font-family: "Noto Sans Italic", Arial, sans-serif;
14 }
```

Schriftformate: Zur Verwendung als Web Fonts stehen zwei mögliche *Typen* zur Verfügung:

- **WOFF:** Web Open Font Format – entwickelt von Mozilla, dann aber vom W3C übernommen und mittlerweile der alleinige Standard [110]. Wird von allen Browsern unterstützt und hat sich als der gemeinsame Nenner für Schriften im Web etabliert. Die Dateien sind kleiner und enthalten Zusatzinformationen wie die Lizenz. Die aktuelle Version 2.0 (Erweiterung `.woff2`) hat vor allem den Bandbreitenbedarf bei der Übermittlung optimiert.
- **TrueType/OpenType:** Die bekannten `.ttf`- und `.otf`-Schrifttypen werden in allen Browsern unterstützt, haben aber das Problem, dass keinerlei Lizenzinformationen mitgegeben werden. Es sollten nur jene Schriften verwendet werden, an denen Rechte verfügbar sind und bei denen Weiterverbreitung auf diese Art auch rechtlich gesichert ist.

Font-Dienste: In vielen Fällen ist es nicht nötig, selbst Schriften zu definieren, diese bereit zu stellen und einzubinden, denn es gibt einige *Dienste*, die dies auf eine einfache Art ermöglichen. Viele davon bieten darüber hinaus noch freie Schriften an, die völlig ohne lizenzrechtliche Bedenken verwendet werden können. Der bekannteste ist *Google Fonts* (<https://fonts.google.com/>), es existieren jedoch auch andere Anbieter. Google Fonts stellt die Schriften über `@font-face`-Regeln in eigenen Stylesheets zur Verfügung, die auf der eigenen Seite eingebunden werden können. Weiters wird auch der Download der dort verfügbaren Schriften ermöglicht, die WOFF2-Dateien sind dabei jedoch nicht enthalten. Mit Tools wie dem *google-webfonts-helper* (<https://gwfh.mranftl.com/fonts>) können diese jedoch auch einfach heruntergeladen und selbst gehostet werden.

3.8.2 Text-Formatierung

In den Bereich der Text-Formatierung fallen Dinge wie unterstrichener oder durchgestrichener Text oder auch die Schriftfarbe.

Text-Dekoration

Mit Hilfe der *Text-Dekoration* (engl. „text-decoration“) können Streichungen aller Art erzeugt werden. Verantwortlich dafür ist die CSS-Eigenschaft `text-decoration`. Sie kann die Werte `underline` (unterstrichen), `overline` (überstrichen), `line-through` (durchgestrichen), `blink` (blinkend) und aber auch `none` (keine Eigenschaft) annehmen.

Unterstreichungen sind im Fließtext auch in den Printmedien kaum üblich, sie sollten daher sparsam eingesetzt werden. Diese Eigenschaft kann aber auch dazu verwendet



Abbildung 3.18: Im HTML-Markup ist der Text mit normaler Großschreibung geschrieben, durch die CSS-Regel wird Versalschrift erzeugt.

werden, die standardmäßige Unterstreichung von Hyperlinks zu entfernen.

Im folgenden Beispiel wird eine Regel zum Durchstreichen eines `<span>`-Elements mit einer bestimmten Klasse erstellt:

```
1 span.strikeout {
2   text-decoration: line-through;
3 }
```

Text-Transformation

Mit Hilfe der *Text-Transformation* (engl. „text-transform“) ist es möglich, die Groß- und Kleinschreibung eines Texts festzulegen, unabhängig davon wie dieser im HTML-Markup steht. Dies geschieht mit Hilfe der Eigenschaft `text-transform`. Sie kann die Werte `uppercase` (alles in Großbuchstaben), `lowercase` (alles in Kleinbuchstaben), `capitalize` (nur Wortanfänge groß) oder `none` (keine Transformation) annehmen.

Die folgende CSS-Regel stellt jeglichen Text in allen Absätzen komplett in Großbuchstaben dar, das Ergebnis ist in Abbildung 3.18 zu sehen.

```
1 p {
2   text-transform: uppercase;
3 }
```

Text-Farbe

Die Zuweisung von Farbwerten wurde bereits in Abschnitt 3.7.3 besprochen. Die folgende Regel legt die Standardschriftfarbe für das gesamte Dokument (d. h. für das `<body>`-Element) als Maroon fest.

```
1 body {
2   color: maroon;
3 }
```

3.8.3 Abstände und Ausrichtung

Angaben zu *Abständen* und *Ausrichtung* nehmen Einfluss auf die Art und Weise, wie Absätze im Text angezeigt werden.

Einrückung

Ein in Büchern und Zeitschriften weit verbreitetes Gestaltungselement ist das Einrücken der ersten Zeile in einem mehrzeiligen Text (auch in diesem Skriptum zu sehen). Dies

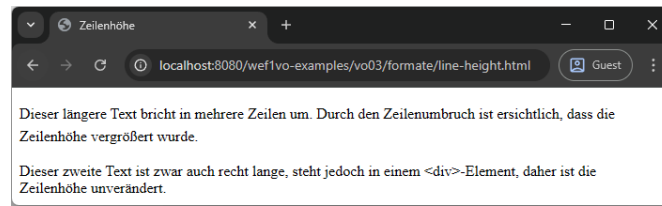


Abbildung 3.19: Der Absatz zeichnet sich durch die größere Zeilenhöhe aus, das `<div>`-Element hat die reguläre Höhe.

kann mit der Eigenschaft `text-indent` erreicht werden. Die Angabe erlaubt wiederum die verschiedensten numerischen Angaben. Negative Werte bewirken eine Ausrückung.

Hier wird die erste Zeile eines Absatzes um die aktuelle Schriftgröße eingerückt.

```
1 p {
2   text-indent: 1em;
3 }
```

Zeilenhöhe

Die Angabe der *Zeilenhöhe* ist vor allem im Zusammenhang mit langem Fließtext interessant, da sie den Abstand zwischen den einzelnen Zeilen bestimmt. Oft kann ein leicht erhöhter Zeilenabstand zur besseren Lesbarkeit beitragen. Mit der Angabe `line-height` kann der Zeilenabstand manipuliert werden. Da diese Angabe nicht zuletzt auch von der Schriftgröße abhängen sollte, sind beide oft gemeinsam zu finden oder wird für die Zeilenhöhe ein relativer Wert mittels `em` Angabe verwendet, der sich direkt auf die Schriftgröße bezieht.

Im folgenden Beispiel wird eine Zeilenhöhe in der 1,5fachen Schriftgröße für Absätze definiert, Abbildung 3.19 zeigt die Darstellung im Browser.

```
1 p {
2   line-height: 1.5em;
3 }
```

Vertikale Ausrichtung

Wird einem Blockelement eine fixe Höhe zugewiesen (mehr dazu in Vorlesung 4) so ist oftmals sehr viel horizontaler Platz vorhanden. Um dann zu entscheiden, ob Text in diesem Element oben, unten oder in der Mitte platziert wird, gibt es die Eigenschaft `vertical-align`. Sie kann die Werte `top` (obenbündig), `middle` (mittig), `bottom` (untenbündig), `baseline` (an der Grundlinie ausrichten), `sub` (tiefer gestellt, ohne die Schriftgröße zu reduzieren), `super` (hochgestellt, ohne die Schriftgröße zu reduzieren), `text-top` (am oberen Schriftrand ausrichten) oder `text-bottom` (am unteren Schriftrand ausrichten) annehmen.

Folgende CSS-Klasse richtet die Schrift vertikal zentriert aus:

```
1 .vcenter {
2   vertical-align: middle;
3 }
```

Horizontale Ausrichtung

Etwas eingängiger als die vertikale Ausrichtung ist die horizontale. Sie bestimmt, wie Fließtext innerhalb eines Absatzes angeordnet wird. Die Eigenschaft **text-align** kann dabei die Werte **left** (linksbündig), **center** (zentriert), **right** (rechtsbündig) oder **justify** (Blocksatz) annehmen.

Im folgenden Beispiel wird der Text in allen Absätzen als Blocksatz angeordnet.

```
1 p {  
2   text-align: justify;  
3 }
```

Textumbruch

Umbrüche im Fließtext werden normal vom Browser selbst aufgrund der vorgegebenen Breite des Browserfensters durchgeführt. Soll dies beeinflusst werden, kann mit der Eigenschaft **white-space** ein anderes Verhalten herbei geführt werden. Folgende Werte sind möglich: **normal** (automatischer Zeilenumbruch), **pre** (Zeilenumbruch wie im Quellcode angegeben – wie beim `<pre>`-Element), **pre-wrap** (wie **pre**, nur erfolgt ein Umbruch, wenn die Breite der Box zur Anzeige nicht mehr ausreicht), **pre-line** (mehrere Whitespaces werden wie bei **normal** zusammengefasst, es erfolgt aber ein Zeilenumbruch wie bei **pre-wrap**) und **nowrap** (kein automatischer Umbruch, nur manuell durch `<br>`).

Die folgende CSS-Klasse deaktiviert alle automatischen Zeilenumbrüche.

```
1 .nowrap {  
2   white-space: nowrap;  
3 }
```

3.8.4 Hintergründe

Hintergründe erlauben es, das Standard-Weiß durch eine andere Farbe oder ein Bild zu ersetzen. Sinnvoll ist die Angabe meist nur bei Blockelementen, Inline-Elemente (Phrasing content) können jedoch genauso einen Hintergrund zugewiesen bekommen.

Hintergrundfarbe

Die *Hintergrundfarbe* wird über die Eigenschaft **background-color** gesetzt. Als Werte sind wiederum die in Abschnitt 3.7.3 angegebenen Farben möglich. Der Standardwert ist **transparent**, der auch explizit gesetzt werden kann, um eine Hintergrundfarbe wieder zu entfernen.

Im Sinne der Accessibility (mehr dazu in Vorlesung 6) sollte darauf geachtet werden, dass stets ein ausreichender Kontrast zwischen Schriftfarbe und Hintergrundfarbe besteht.

Folgende Regel setzt für das gesamte Dokument die Hintergrundfarbe Aqua:

```
1 body {  
2   background-color: aqua;  
3 }
```

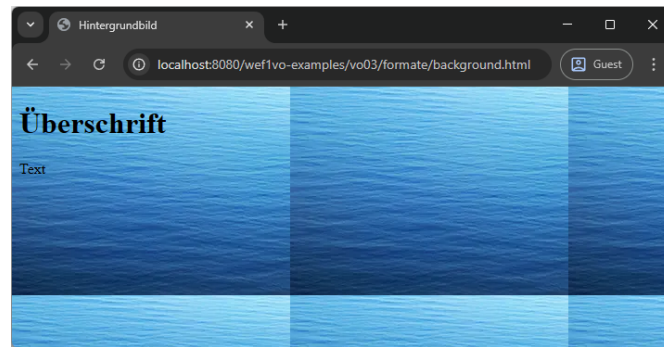


Abbildung 3.20: Da das Hintergrundbild kleiner als die Dimensionen des Browserfensters ist, wird es wiederholt.

Hintergrundbilder

Hintergrundbilder können mit Hilfe der Eigenschaft **background-image** angegeben werden. Als Wert muss ein URL über **url("URL")** übergeben werden. Das eingebundene Hintergrundbild wird im Wallpaperereffekt wiederholt, sofern es nicht die volle Größe ausfüllt. Dieses Verhalten kann mit Hilfe von **background-repeat** angepasst werden (s.u.).

Folgende CSS-Regel legt ein Hintergrundbild auf das gesamte Dokument. Das Hintergrundbild liegt im gleichen Verzeichnis wie die CSS-Datei, Abbildung 3.20 zeigt das Ergebnis.

```
1 body {  
2   background-image: url("background.webp");  
3 }
```

Es ist auch möglich, mehr als nur ein Hintergrundbild zu spezifizieren. Dazu müssen mehrere **url()**-Angaben mit Beistrich getrennt angegeben werden. Kommt es dabei zu Überlagerungen, so wird das erste angegebene Bild ganz oben, das letzte auf der untersten Ebene angezeigt.

Wiederholungseffekt

Mit der Eigenschaft **background-repeat** kann gesteuert werden, ob und wie ein Hintergrundbild wiederholt wird, wenn seine Größe nicht ausreicht, den gesamten Container zu befüllen. Zur Verfügung stehen die Werte **repeat** (Standardeinstellung, Wiederholung in *x*- und *y*-Richtung), **repeat-x** (nur in *x*-Richtung, also horizontal wiederholen), **repeat-y** (nur in *y*-Richtung, also vertikal wiederholen) und **no-repeat** (kein Wiederholen).

Im folgenden Codeausschnitt wird ein zuvor definiertes Hintergrundbild für einen **<div>**-Bereich mit einer ID nur horizontal wiederholt.

```
1 div#container {  
2   background-repeat: repeat-x;  
3 }
```

Wasserzeichen-Effekt

Bei längeren Elementen (z. B. ganzen Seiten, deren vertikaler Inhalt größer ist als das Browserfenster und die daher scrollen müssen) kann entschieden werden, ob der Hintergrund mitscrollen oder statisch verweilen soll. Diese Einstellung wird durch die Eigenschaft **background-attachment** ermöglicht und kann die Werte **scroll** und **fixed** (Wasserzeichen-Effekt) annehmen.

Hier wird das dokumentweite Hintergrundbild beim Scrollen fixiert:

```
1 body {  
2   background-attachment: fixed;  
3 }
```

Position des Hintergrunds

Wird mittels **background-repeat** festgelegt, dass ein Hintergrundbild nicht wiederholt, sondern nur ein Mal angezeigt wird, kann mit der Angabe von **background-position** definiert werden, wie der Hintergrund positioniert werden soll. Die Eigenschaft erwartet zwei durch ein Leerzeichen getrennte Angaben für die horizontale und vertikale Position. Erlaubt sind numerische Werte und die Angaben **top** (vertikal obenbündig), **bottom** (vertikal untenbündig), **center** (horizontal oder vertikal zentriert), **left** (horizontal linksbündig) und **right** (horizontal rechtsbündig).

Im folgenden Beispiel wird eine Hintergrundgrafik horizontal zentriert und vertikal 50 Pixel vom oberen Ende positioniert.

```
1 body {  
2   background-position: center 50px;  
3 }
```

Hintergründe formatieren

Mit mehreren Eigenschaften lässt sich weiter am Erscheinungsbild des Hintergrundbildes feilen.

background-clip kann die Werte **border-box**, **padding-box** oder **content-box** annehmen. Je nachdem erstreckt sich das Hintergrundbild hinter den Rahmen, bis zum Rahmen oder nur im Inhaltsbereich. Dies ist vor allem Beim Einsatz von Rahmen mit Transparenzen interessant.

Mit **background-origin** lässt sich steuern, wo ein Hintergrundbild *beginnt*. Mit dem Wert **border-box** beginnt es bereits hinter dem Rahmen, bei **padding-box** innerhalb des Rahmens und bei **content-box** nur im Inhaltsbereich. Im Unterschied zu **background-clip** beginnt das Hintergrundbild aber nur dort auf der linken Seite, es kann auf der rechten Seite trotzdem bis hinter den Rahmen laufen. Um das zu verhindern, wird **background-clip** benötigt.

Mit der Eigenschaft **background-size** lassen sich Hintergrundbilder skalieren. Die Angaben sind zwei Werte in Pixeln oder Prozent, also etwa **background-size: 20px 40px**, um ein Bild auf 20 Pixel Breite und 40 Pixel Höhe zu skalieren. Alternativ existieren die Werte **contain** und **cover**. Ersteres skaliert das Bild unter Beibehaltung des Seitenverhältnisses so, dass sowohl Höhe als auch Breite in den aktuellen Anzeigebereich

passen. Bei zweiterem wird so skaliert, dass das Hintergrundbild den Anzeigebereich komplett abdeckt (ggf. werden jedoch Teile abgeschnitten).

Shorthand-Syntax

Auch für die Arbeit mit Hintergründen gibt es eine *Shorthand-Syntax*, die es ermöglicht, alle Angaben mit einer Eigenschaft zusammenzufassen. Diese lautet **background**, die Reihenfolge der Einzelangaben ist bei diesem Attribut egal.

Folgende Angabe fasst die oben gezeigten in einer zusammen:

```
1 body {  
2   background: aqua url("background.webp") repeat-x fixed center 50px;  
3 }
```

3.8.5 Listen

Mit Hilfe von CSS lassen sich Listen (meist Aufzählungslisten und nummerierte Listen) formatieren. Vor allem die Aufzählungszeichen stehen hierbei im Mittelpunkt.

Listentyp

Der *Listentyp* ist das Zeichen, das vor jedem einzelnen Listeneintrag angezeigt wird. Bei Aufzählungslisten handelt es sich in der Regel um einen Bullet-Point, bei nummerierten Listen sind es arabische Ziffern. Mit der Eigenschaft **list-style-type** kann dies angepasst werden.

Die folgenden Werte stehen für Aufzählungszeichen zur Verfügung: **disc** (gefüllter Kreis, Bullet-Point), **circle** (leerer Kreis) und **square** (rechteckiger Kasten).

Für Nummerierungen sind folgende Werte möglich: **decimal** (arabische Zahlen), **decimal-leading-zero** (arabische Zahlen mit führender Null), **lower-roman** (römische Zahlen, dargestellt als Kleinbuchstaben), **upper-roman** (römische Zahlen, dargestellt als Großbuchstaben), **georgian** (georgische Nummerierung), **armenian** (armenische Nummerierung), **lower-latin** (lateinische Kleinbuchstaben), **upperlatin** (lateinische Großbuchstaben) und **lower-greek** (griechische Kleinbuchstaben).

Darüber hinaus gibt es mit der Option **none** die Möglichkeit, Aufzählungszeichen jeder Art komplett zu entfernen. Dies ist vor allem dann interessant, wenn eine Liste zur Gliederung einer Navigation verwendet wird, denn dabei werden nur selten Aufzählungszeichen benötigt.

Der folgende Code stellt das Aufzählungszeichen einer Liste auf rechteckige Boxen. Es muss das Listenelement (**** oder ****) selektiert werden, nicht der Listeneintrag (****).

```
1 ul {  
2   list-style-type: square;  
3 }
```

Listenposition

Die *Position des Aufzählungszeichens* kann mittels **list-style-position** bestimmt werden. Möglich sind die Werte **inside** und **outside**, wobei letzterer der Standardwert

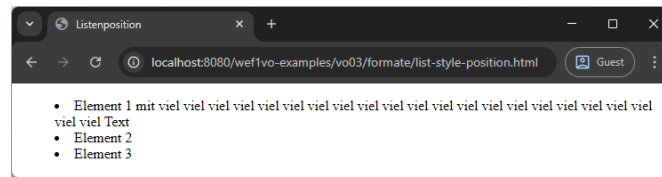


Abbildung 3.21: Beim ersten Aufzählungspunkt mit viel Text ist ersichtlich, dass der Bullet-Point mit dem Text in der Zeile steht.

ist. Wird **inside** gewählt, so beginnt das Aufzählungszeichen mit dem Text, ansonsten steht es davor.

Im folgenden Code wird die Listenposition umgestellt, Abbildung 3.21 zeigt die Darstellung im Browser:

```
1 ul {
2   list-style-position: inside;
3 }
```

Listen-Bilder

Neben der Möglichkeit, andere Aufzählungszeichen zu wählen, können auch Bilder verwendet werden. Dies kann mit der Eigenschaft `list-style-image` bewerkstelligt werden. Sie akzeptiert, ähnlich wie Hintergrundbilder, ebenfalls einen URL auf ein Bild. Dieses wird dann anstatt des regulären Zeichens angezeigt. Die Bildgröße sollte schon im Vorfeld angepasst werden. Obwohl der Browser automatisch versucht, die Größe passend zu wählen, funktioniert dies nicht immer zuverlässig.

Folgende Regel verwendet ein Bild als Aufzählungszeichen für eine Aufzählungsliste.

```
1 ul {
2   list-style-image: url("bullet.webp");
3 }
```

Shorthand-Syntax

Mit der *Shorthand-Syntax* für Listen können ebenfalls wieder die drei einzelnen Angaben zu einer vereint werden. Möglich ist diese mit der Eigenschaft `list-style`. Werden aber die Angaben für `list-style-type` und `list-style-image` gemeinsam angegeben, bekommen die Listen-Bilder Vorrang und werden daher angezeigt.

Folgende Angaben fassen die drei obigen in einer zusammen:

```
1 ul {
2     list-style: square url("bullet.webp") inside;
3 }
```

3.9 Weiterführende Literatur

Das Standardwerk zum Thema Cascading Style Sheets ist [37]. Es bietet eine komplette Einführung mit sehr detailliert erklärten Hintergründen. Die 5. Auflage aus dem Jahr 2023 deckt alle aktuellen Entwicklungen gut ab.

Auf Deutsch gibt es kein alleinstehendes CSS-Werk, das einigermaßen aktuell ist. Hier sollte auf die HTML/CSS-Werke zurückgegriffen werden. In [18, 40, 57, 58] kommen die Themen CSS zusammen mit HTML vor und werden ausreichend erörtert. Ein eigenes CSS-Buch ist daher meist eigentlich nicht nötig, zumal diese Bücher in regelmäßigen Abständen neu aufgelegt werden.

Vorlesung 4

CSS Box-Modell & Responsive Web

4.1 Das CSS Box-Modell

Das *CSS Box-Modell* [62, Kapitel 8; 67] definiert die Berechnung der gesamten Breite und Höhe von HTML-Elementen.

4.1.1 Überblick

In HTML gibt es eine Reihe von Elementen, die standardmäßig einen neuen Absatz erzeugen und deren Darstellung somit als Block, d. h. als Rechteck geschieht. Vor HTML5 wurden diese als Blockelemente bezeichnet, nun gehören sie zu den Kategorien Flow Content, Sectioning Content und Heading Content. Typische Beispiele sind etwa `<p>`, `<div>` oder auch Überschriften und Gliederungselemente. Mit Hilfe des CSS Box-Modells wird bestimmt, welche Dimension ein solches Blockelement hat. Die Gesamtgröße ist dabei eine Addition von vier Werten:

- Breite (**width**) oder Höhe (**height**) des Elements (je nachdem, ob die horizontale oder vertikale Größe berechnet werden soll). Sie definieren die Größe des Inhaltsbereichs.
- Innenabstand des Elements zu seinem Rand bzw. zu dessen Rahmen (**padding**).
- Rahmenstärke (**border-width**).
- Außenabstand vom Rahmen zum nächsten Element (**margin**).

Abbildung 4.1 zeigt ein Beispiel. Ein Blockelement (z. B. `<div>`) erhält eine Breitenangabe von 90px. Darüber hinaus einen Innenabstand von 10px, einen 5px dicken Rahmen, sowie einen Außenabstand von 20px. Obwohl das Element mit 90px Breite definiert wurde, beträgt sein gesamter horizontaler Platzbedarf nun 160px, denn zu den 90px Breite kommen zwei Mal (links und rechts) 10px für den Innenabstand, zwei Mal 5px für die Rahmenstärke und zwei Mal 20px für den Außenabstand zu angrenzenden Elementen hinzu. Für die Höhe wurden 40px definiert. Zusammen mit denselben Werten für Innenabstand, Rahmen und Außenabstand ergibt sich eine Gesamthöhe von 110px.

In den folgenden Abschnitten wird auf die einzelnen CSS-Eigenschaften eingegangen, die bei der Berechnung des Box-Modells eine Rolle spielen.

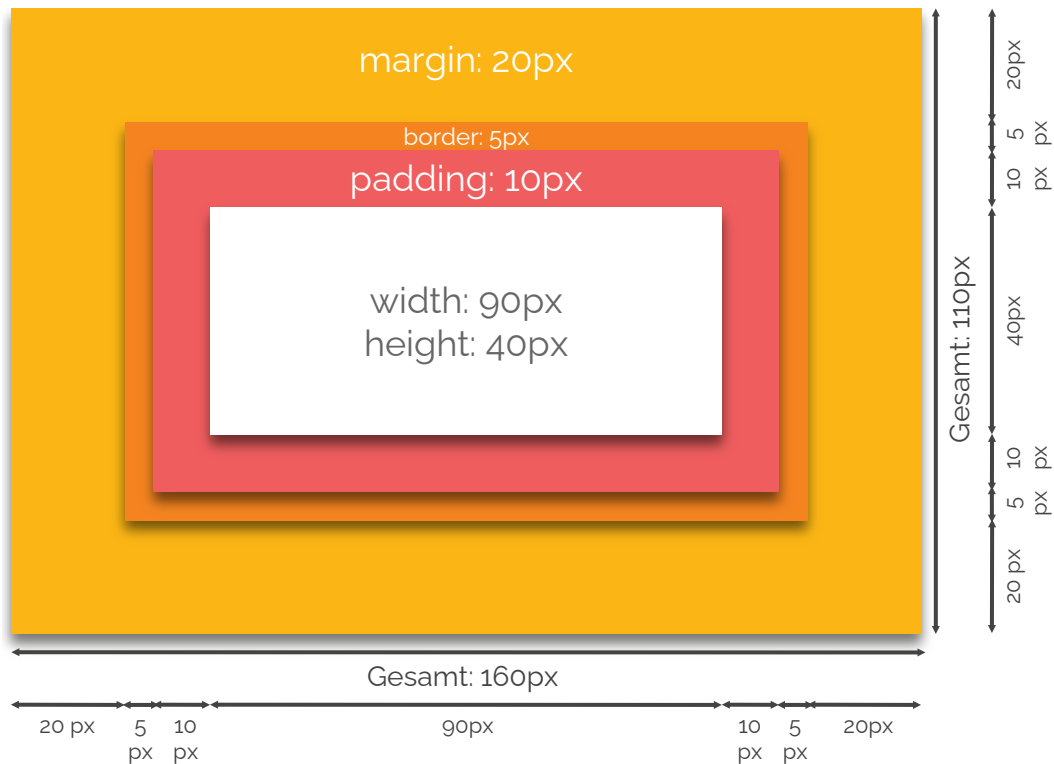


Abbildung 4.1: Die Gesamtbreite eines Objekts mit 90 Pixel Breite, einem Innenabstand von 10 Pixeln, einem Rahmen mit Stärke 5 Pixel und einem Außenabstand von 20 Pixeln ergibt 160 Pixel. Die Gesamthöhe bei 40 Pixel Breite und denselben Angaben für Innen- und Außenabstand sowie Rahmen beträgt 110 Pixel.

4.1.2 Dimensionsangaben für den Inhalt

Um die Dimensionen von Elementen, d. h. ihren Inhaltsbereich anzugeben, sind sowohl fixe als auch Mindest- bzw. Maximalangaben möglich.

Fixe Angaben

Um die *Breite* eines Elements anzugeben, wird die Eigenschaft `width` verwendet. Sie akzeptiert numerische Werte mit den üblichen Angaben. Wird eine Prozentangabe verwendet, bezieht sich diese auf die Größe des einschließenden Elements (im äußersten Fall das Browserfenster).

Um die *Höhe* eines Elements festzusetzen, wird `height` verwendet. Diese Eigenschaft funktioniert analog zur Breitenangabe.

Mit den folgenden CSS-Angaben werden zwei `<div>`-Elemente über ID-Selektoren mit jeweils unterschiedlicher Breite und Höhe definiert, das Ergebnis ist in Abbildung 4.2 ersichtlich.

```
1 #one {
2   width: 150px;
3   height: 50px;
4   background-color: orange;
```

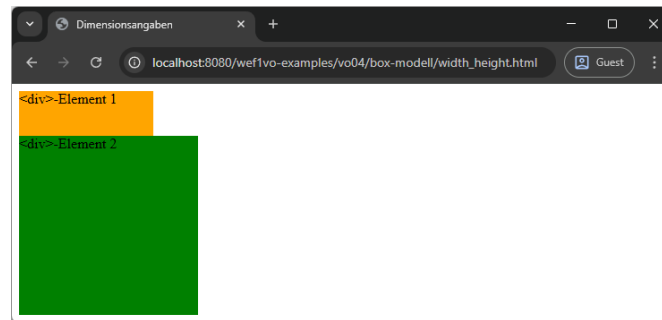


Abbildung 4.2: Die beiden `<div>`-Elemente nehmen genau die Größe ein, mit der sie definiert wurden. Ohne Breitenangabe etwa, würden die Elemente die gesamte Browserfensterbreite beanspruchen.

```
5 }  
6 #two {  
7   width: 200px;  
8   height: 200px;  
9   background-color: green;  
10 }
```

Werden die Angaben zu Breite und Höhe nicht angegeben (dies entspricht auch dem Setzen des Werts `auto`) nimmt das Element so viel horizontalen Platz ein, wie möglich. Ist es von keinem Element umgeben, beansprucht es die gesamte Browserfensterbreite, ansonsten die gesamte Breite des umgebenden Elements.

Wird die Höhe nicht angegeben (oder auf den Wert `auto` gesetzt), so nimmt das Element jedoch die *minimale* Höhe ein. D. h. es wird so weit „zusammengestaucht“, wie möglich.

Egal ob Höhe und Breite angegeben wurden oder nicht, ein Blockelement beansprucht immer eine eigene „Zeile“, d. h. es stehen (ohne zusätzliche Angaben) nie zwei derartige Elemente nebeneinander, auch wenn Platz wäre.

Im folgenden, leicht modifizierten Code, wurden die Dimensionsangaben für das erste `<div>`-Element auf `auto` gesetzt, für das zweite komplett weggelassen. Das Ergebnis ist dasselbe und in Abbildung 4.3 zu sehen.

```
1 #one {  
2   width: auto;  
3   height: auto;  
4   background-color: orange;  
5 }  
6 #two {  
7   background-color: green;  
8 }
```

Mindest- und Maximalangaben

Mit den Eigenschaften `min-width` und `max-width` bzw. `min-height` und `max-height` lässt sich definieren, wie breit bzw. hoch ein Element *mindestens* oder *maximal* sein darf. In Abbildung 4.3 ist ersichtlich, dass ein Element nur so viel Höhe einnimmt,

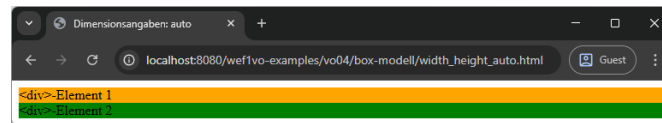


Abbildung 4.3: Ohne Breiten- und Höhenangaben (bzw. mit der Angabe `auto`), nehmen die Elemente so viel horizontalen und so wenig vertikalen Platz ein, wie möglich.

wie der Inhalt dies verlangt. Das kann jedoch zu unerwünschten Effekten führen, vor allem, wenn zum Erstellungszeitpunkt der Seite nicht bekannt ist, wie viel Inhalt ein Bereich aufweisen wird. Soll in diesem Fall verhindert werden, dass ein Element zu niedrig wird, kann die Eigenschaft `min-height` verwendet und das Element damit auf z. B. mindestens 50px gesetzt werden. Somit ist es egal, ob kein Inhalt oder auch nur eine Zeile im Element enthalten sind – dieses wird immer mindestens 50px Höhe aufweisen. Ist jedoch mehr Inhalt vorhanden (der mehr als 50px in der Vertikalen benötigt), so wird das Element normal auf seinen Platzbedarf erweitert.

Um nun auch zu bewirken, dass ein Element nicht höher als z. B. 200px werden darf, kann zusätzlich noch die Eigenschaft `max-height` angegeben werden und somit die Höhe auf 200px begrenzt werden – egal wie viel Inhalt sich im Element befindet. Das Element skaliert nun variabel zwischen 50px und 200px.

Die Eigenschaften `min-width` und `max-width` ermöglichen dasselbe Verhalten auf horizontaler Ebene.

Überlauf bei übergroßem Inhalt

Wird einem Element eine fixe oder auch maximale Höhe und/oder Breite definiert, kann es vorkommen, dass der Inhalt (Text oder auch eine Grafik) zu groß für den definierten Bereich ist. In diesem Fall kann mit Hilfe der Eigenschaft `overflow` bestimmt werden, wie der Browser mit diesem Problem umgehen soll. Zur Auswahl stehen die folgenden Werte:

- **visible:** Der Inhalt wird auf jeden Fall angezeigt und ragt über das Element hinaus.
- **hidden:** Überlaufender Inhalt wird an den Elementgrenzen abgeschnitten.
- **clip:** Überlaufender Inhalt wird wie bei **hidden** abgeschnitten, allerdings lässt sich diese Grenze mit der Eigenschaft `overflow-clip-margin` selbst festlegen.
- **scroll:** Der Browser zeigt Scroll-Leisten am Element an, um den Rest des Inhalts navigierbar zu machen.
- **auto:** Der Browser entscheidet selbst, wie er vorgeht.

Der Standardwert ist **visible**.

Abbildung 4.4 zeigt die fünf Werte in genau dieser Reihenfolge. Verwendet wurde folgender CSS-Code:

```
1 div {
2   width: 100px;
3   height: 100px;
4   float: left; /* nebeneinander platzieren */
5 }
```

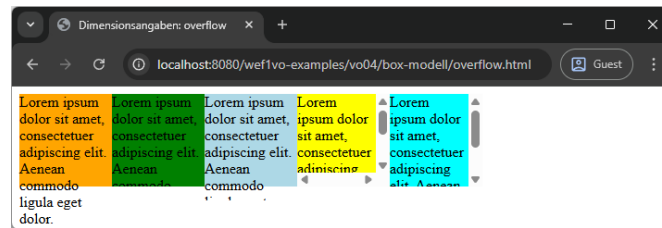


Abbildung 4.4: Bei `visible` ragt der Text aus der orangen Box heraus. In der grünen Box wird der Text durch `hidden` nach dem letzten Wort, das Platz findet, abgeschnitten. `clip` schneidet den Überlauf in der hellblauen Box ab, durch `overflow-clip-margin` geschieht dies aber erst nach 15px. `scroll` erzeugt in der gelben Box permanente Scroll-Leisten (auch horizontal, wo sie eigentlich nicht nötig wären). `auto` agiert am intelligentesten und platziert nur eine vertikale Scroll-Leiste.

```

6 #one {
7   overflow: visible;
8   background-color: orange;
9 }
10 #two {
11   overflow: hidden;
12   background-color: green;
13 }
14 #three {
15   overflow: clip;
16   overflow-clip-margin: 15px;
17   background-color: lightblue;
18 }
19 #four {
20   overflow: scroll;
21   background-color: yellow;
22 }
23 #five {
24   overflow: auto;
25   background-color: cyan;
26 }

```

4.1.3 Innenabstand

Mit dem *Innenabstand* wird der Leerraum zwischen dem Inhalt eines Elements und seinem Rand (oder Rahmen, falls vorhanden) definiert. Abbildung 4.5 zeigt schematisch die Position des Innenabstands in einem Element. Die Angabe ist für alle vier Seiten eines Elements einzeln, oder über eine zusammengefasste Shorthand-Syntax möglich.

Einzelne Angaben

Folgende Einzelangaben für den Innenabstand sind möglich:

- `padding-top`: Innenabstand nach oben.
- `padding-bottom`: Innenabstand nach unten.
- `padding-left`: Innenabstand nach links.

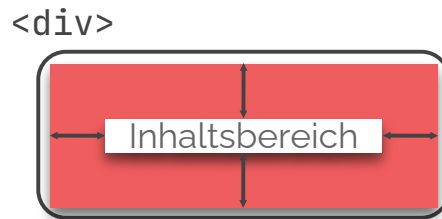


Abbildung 4.5: Der Innenabstand ist der Teil vom Inhalt zum Rand des Elements (z. B. <div>). Das Element wird dadurch größer.

- **padding-right:** Innenabstand nach rechts.

Im folgenden Code wird der Innenabstand eines <div>-Elements oben und unten explizit gesetzt.

```
1 div {  
2   padding-top: 20px;  
3   padding-bottom: 50px;  
4 }
```

Shorthand-Syntax

Die Eigenschaft **padding** erlaubt die Angabe des *Innenabstands* für mehrere Seiten. Möglich sind eine bis vier numerische Angaben:

- Eine Angabe: Alle vier Seiten erhalten den gleichen Innenabstand.
- Zwei Angaben: Oben und unten (1. Wert) sowie links und rechts (2. Wert) erhalten den gleichen Innenabstand.
- Drei Angaben: Der Innenabstand wird für oben (1. Wert), rechts und links (2. Wert) und unten (3. Wert) gesetzt.
- Vier Angaben: Der Innenabstand wird für oben (1. Wert), rechts (2. Wert), unten (3. Wert) und links (4. Wert) gesetzt.

Im folgenden Beispiel werden alle vier Seiten mit einer Anweisung gesetzt.

```
1 div {  
2   padding: 20px 15px 25px 10px;  
3 }
```

4.1.4 Rahmen

Ein *Rahmen* wird immer am Rand eines Elements platziert. Abbildung 4.6 zeigt schematisch die Position. Die Angaben zum Rahmen (Rahmenstärke, Rahmenstil und Rahmenfarbe) können einzeln oder mit einer Shorthand-Syntax gemacht werden.

Einzelne Angaben

Selbst bei den Einzelangaben sind wieder mehrere Möglichkeiten vorhanden. So können die Angaben für Rahmenstärke, -stil und -farbe für alle vier Seiten gemeinsam oder auch einzeln gemacht werden.



Abbildung 4.6: Der Rahmen liegt am Rand des Elements und vergrößert es dadurch um seine Breite.

Rahmenstärke: Die *Stärke* (auch Dicke) des Rahmens wird mittels `border-width` angegeben. Es folgt eine numerische Angabe als Wert.

Folgendes Codestück setzt die Rahmenstärke eines `<div>`-Elements auf 2px:

```
1 div {  
2   border-width: 2px;  
3 }
```

Sollen die einzelnen Seiten unterschiedlich gestaltet werden, kann auf die vier verschiedenen Einzelangaben `border-top-width` (für oben), `border-right-width` (für rechts), `border-bottom-width` (für unten) und `border-left-width` (für links) zurückgegriffen werden.

Rahmenstil: Die *Art* des angezeigten Rahmens wird durch `border-style` definiert. Als Werte stehen eine Reihe von Angaben zur Verfügung: `none` (kein Rahmen), `hidden` (ebenfalls kein Rahmen, nur bei Tabellen mit Eigenschaft `border-collapse: collapse` sinnvoll), `dotted` (gepunktet), `dashed` (gestrichelt), `solid` (durchgezogen) sowie `double` (doppelt durchgezogen). Die Angaben `groove`, `ridge`, `inset` und `outset` sind ebenfalls möglich und erzeugen verschiedene 3D-Rahmeneffekte.

Folgender Code definiert einen durchgezogenen Rahmen für alle vier Seiten:

```
1 div {  
2   border-style: solid;  
3 }
```

Um unterschiedlichen Seiten eines Elements unterschiedliche Rahmenstile zuzuweisen, existieren wiederum verschiedene Einzelangaben: `border-top-style` (für oben), `border-right-style` (für rechts), `border-bottom-style` (für unten) und schließlich `border-left-style` (für links).

Rahmenfarbe: Mit der Eigenschaft `border-color` kann die *Farbe* des Rahmens spezifiziert werden. Möglich sind wie immer die in Abschnitt 3.7.3 erläuterten Farbangaben.

Folgender Code gibt erzeugt einen roten Rahmen:

```
1 div {  
2   border-color: red;  
3 }
```

Auch hierbei ist es möglich, allen vier Seiten unterschiedliche Farben zu geben. Die Eigenschaften dazu lauten: `border-top-color` (für oben), `border-right-color` (für rechts), `border-bottom-color` (für unten) und `border-left-color` (für links).



Abbildung 4.7: Der Außenabstand wird vom Rand des Elements weg berechnet und gehört nicht mehr zum eigentlichen Element.

Shorthand-Syntax

Mittels der Eigenschaft **border** ist es möglich, die Angaben für Rahmenstärke, -stil und -farbe gemeinsam anzugeben. Die Werte werden dazu in dieser Reihenfolge durch Leerzeichen getrennt angegeben.

Folgender Code erzeugt einen 1px starken, durchgehenden schwarzen Rahmen:

```
1 div {
2   border: 1px solid black;
3 }
```

Wiederum ist es möglich, die Shorthand-Syntax nur für einzelne Seiten des Rahmens anzuwenden. Dazu können die Eigenschaften **border-top** (für oben), **border-right** (für rechts), **border-bottom** (für unten) sowie **border-left** (für links) verwendet werden. Die Angabe der Werte erfolgt in der gleichen Reihenfolge.

4.1.5 Außenabstand

Der *Außenabstand* ist der Abstand eines Elements von seinem Eltern- oder Nachbar-element. Im Gegensatz zu Innenabstand und Rahmen liegt der Außenabstand bereits außerhalb des Elements – Abbildung 4.7 stellt dies schematisch dar. Der Außenabstand kann auch negative Werte annehmen. In diesem Fall kommt es zur Überlappung von Elementen.

Wie beim Innenabstand ist die Angabe für alle vier Seiten eines Elements einzeln, oder über eine zusammengefasste Shorthand-Syntax möglich.

Einzelne Angaben

Folgende Einzelangaben für den Außenabstand sind möglich:

- **margin-top**: Außenabstand nach oben.
- **margin-bottom**: Außenabstand nach unten.
- **margin-left**: Außenabstand nach links.
- **margin-right**: Außenabstand nach rechts.

Im folgenden Code wird der Außenabstand eines `<div>`-Elements links und unten explizit gesetzt:

```
1 div {
2   margin-left: 10px;
3   margin-bottom: 20px;
4 }
```

Shorthand-Syntax

Die Eigenschaft `margin` erlaubt die Angabe des Außenabstands für mehrere Seiten (analog zu `padding`). Möglich sind eine bis vier numerische Angaben:

- Eine Angabe: Alle vier Seiten erhalten den gleichen Außenabstand.
- Zwei Angaben: Oben und unten (1. Wert) sowie links und rechts (2. Wert) erhalten den gleichen Außenabstand.
- Drei Angaben: Der Außenabstand wird für oben (1. Wert), rechts und links (2. Wert) und unten (3. Wert) gesetzt.
- Vier Angaben: Der Außenabstand wird für oben (1. Wert), rechts (2. Wert), unten (3. Wert) und links (4. Wert) gesetzt.

Im folgenden Beispiel werden alle vier Seiten mit einer Anweisung gesetzt.

```
1 div {  
2   margin: 10px 15px 30px 20px;  
3 }
```

4.1.6 Besonderheiten und Wechsel der Box-Eigenschaften

Im Umgang mit dem Box-Modell gibt es zwei Besonderheiten, die im Folgenden erklärt werden sollen: Margin Collapse und das alternative (IE) Box-Modell. Darüber hinaus kann mit der CSS-Eigenschaft `display` bestimmt werden, wie sich eine Box verhält.

Margin Collapse

Liegen zwei Blockelemente mit oberen und unteren Außenabständen übereinander, so *kollabieren* die angrenzenden Außenabstände und es bleibt lediglich der größere von beiden übrig. Dies ist der Grund, warum etwa bei aufeinanderfolgenden Absätzen nicht überdurchschnittlich viel Platz entsteht. Elemente die nebeneinander liegen (mit linken und rechten Außenabständen) sind von dieser Besonderheit nicht betroffen.

Ein Beispiel: Absatz 1 hat einen Außenabstand von 16px oben und unten. Absatz 2 liegt unterhalb und hat einen Außenabstand von 50px oben und unten. Der Abstand zwischen beiden Elementen ist nun nicht (wie vielleicht erwartet) 66px, sondern 50px. Die beiden Abstände kollabieren und der größere kommt zum Zug. Abbildung 4.8 zeigt dies.

Das alternative (IE) Box-Modell

Eine weitere Besonderheit ist das *alternative Box-Modell*, auch bekannt als *IE Box-Modell*. Internet Explorer bis Version 5.5 und auch nach wie vor im Quirks-Modus (Anzeigen einer Website, die keinen DOCTYPE besitzt) berechnet das Box-Modell leicht anders. Anstatt, wie vom W3C definiert, zur Breite des Elements Innenabstand und Rahmen dazu zu zählen, werden diese von der Breite subtrahiert.

Abbildung 4.9 stellt das Beispiel vom Anfang des Kapitels im alternativen Box-Modell dar. Wird die Breite des Elements mit 90px angegeben, der Innenabstand mit zwei Mal 10px und der Rahmen mit zwei Mal 5px, so werden die 30px aus Innenabstand und Rahmen nun von den 90px abgezogen, womit effektiv nur mehr 60px für den Inhalt

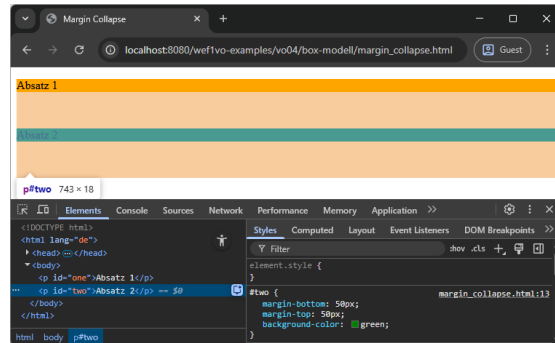


Abbildung 4.8: Der zweite Absatz hat einen oberen und unteren Außenabstand von 50px (ocker dargestellt). Der obere Abstand grenzt direkt an den ersten Absatz an – dies zeigt, dass der untere Abstand des ersten Absatzes (16px) „verschluckt“ wurde.

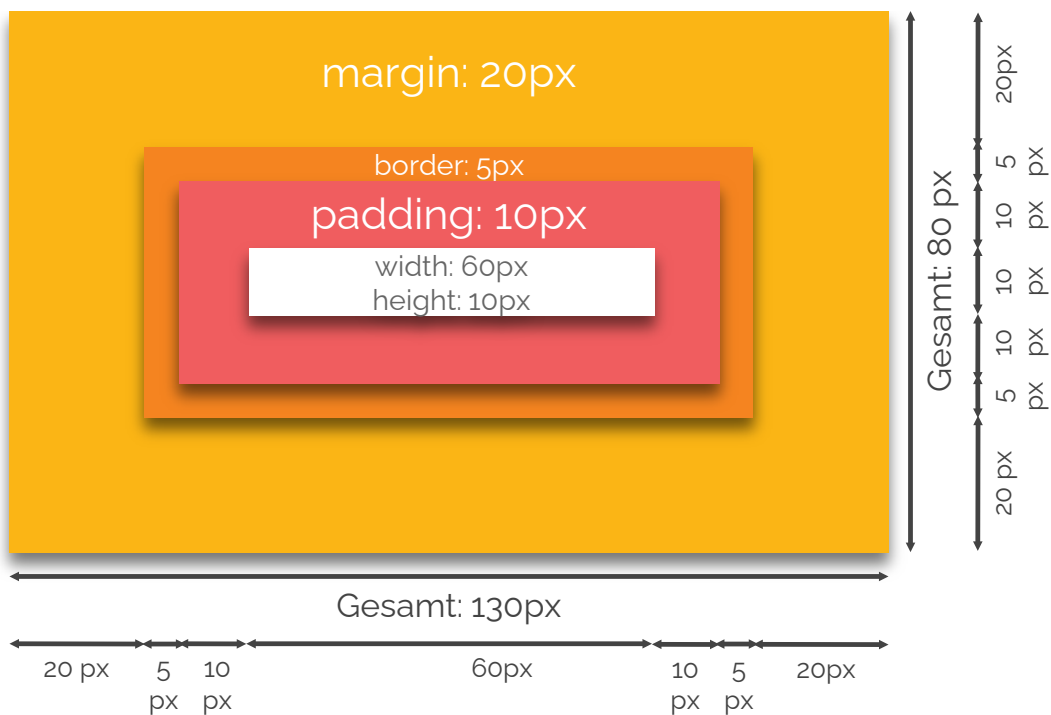


Abbildung 4.9: Im alternativen (IE) Box-Modell sind in der Breite von 90px Innenabstand und Rahmen bereits mit eingerechnet, daher ist die Gesamtbreite nur 130px. Die Gesamthöhe beträgt ebenso nur 80px.

übrig bleiben. Danach wird noch der Rahmen mit zwei Mal 20px addiert, was eine Gesamtbreite von 130px (anstatt der 160px im Standard Box-Modell) ergibt. Ebenso bleiben aus diesem Grund bei einer Höhe von 40px effektiv nur noch 10px übrig, die Gesamthöhe beträgt 80px (statt 110px).

Diese Art der Breitenberechnung erscheint – obwohl sie ursprünglich als „IE Box Model Bug“ bekannt war – nicht völlig abwegig. Innenabstand und Rahmen liegen

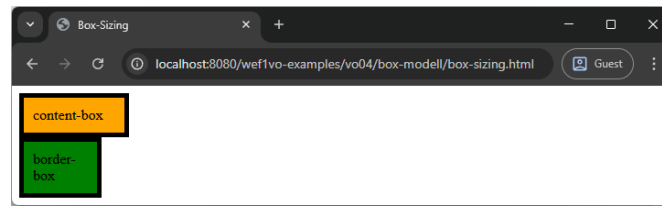


Abbildung 4.10: `div`-Element 1 wird mit dem Wert `content-box`, also nach dem Standard W3C Box-Modell gerendert. `div`-Element 2 wird mit `border-box` dargestellt, also mit dem alternativen (IE) Box-Modell.

schließlich noch im Element, daher ist es nicht unverständlich, dass `width` nicht nur die Breite des Inhalts (wie beim Standard-Box-Modell) sondern die Breite des gesamten Elements (also Inhalt + Innenabstand + Rahmen) angibt.

Diesen Umstand hat auch das W3C erkannt und im CSS Box Sizing Module Level 3 [68] die Eigenschaft `box-sizing` definiert. Sie kann die Werte `content-box` oder `border-box` annehmen. Der erste Wert ist die Standardeinstellung und zieht zur Berechnung das normale W3C Box-Modell heran, der zweite den im Internet Explorer ursprünglich implementieren Modus, der die Breite bis einschließlich des Rahmens berechnet. Obwohl die Spezifikation erst ein Working Draft ist, wird sie von allen Browsern schon lange unterstützt (da sie zuvor in einer anderen Spezifikation definiert wurde).

Folgender Code stellt zwei `<div>`-Elemente mit 90px Breite, 10px Innenabstand und 5px breitem Rahmen in den zwei Box-Modell Varianten dar. Abbildung 4.10 zeigt die Auswirkungen.

```

1 div {
2   width: 90px;
3   padding: 10px;
4   border: 5px solid black;
5 }
6 #one {
7   box-sizing: content-box;
8   background-color: orange;
9 }
10 #two {
11   box-sizing: border-box;
12   background-color: green;
13 }
```

Wechsel der Box-Eigenschaften

Das Browser-Stylesheet gibt vor, welche Elemente einen eigenen Block erzeugen (z. B. `<p>` oder `<div>`) und welche auf Absatzebene (inline) wirken (z. B. `<em>` oder `<img>`). Letztere nehmen nur so viel Platz wie nötig ein, die Eigenschaften `width` und `height` greifen dabei nicht, ebenso sind nur horizontale `margin`-Angaben möglich.

Die CSS-Eigenschaft `display` erlaubt es jedoch, den Charakter eines jeden Elements zu verändern. Sie kann die folgenden Werte annehmen:

- **inline:** Stellt ein Element als Inline-Element, d. h. als Phrasing content dar.

- **block**: Macht ein Element zum Blockelement, alle beschriebenen Dimensionsangaben sind wirksam.
- **inline-block**: Erstellt eine Box, die zwar innerhalb einer Textzeile existiert, aber wie ein Blockelement formatierbar ist.
- **list-item**: Erzeugt zwei Boxen, eine Hauptbox die sich wie ein Blockelement verhält und eine zweite, die das Aufzählungszeichen der Liste enthält.
- **none**: Erzeugt keine Box, d. h. das Element wird nicht mehr dargestellt.

Darüber hinaus gibt es mittels **table**, **table-cell**, **tablecolumn**, **table-row**, sowie **table-column-group**, **table-row-group**, **tableheader-group** und schließlich noch **table-footer-group** noch die Möglichkeit, Elementen das Verhalten von Tabellen bzw. deren Elementen zu geben. Auch für verschiedene Layoutmöglichkeiten, die in CSS3-Modulen ergänzt wurden, wird die Eigenschaft **display** verwendet. Details dazu können MDN¹ entnommen werden bzw. folgen in Vorlesung 5.

4.2 Positionierung von Elementen

Die Anordnung von Elementen erfolgt im Browser im Normalfall im so genannten *Document Flow* (auch Normal Flow) genannt. Dieser ordnet Elemente in der Reihenfolge an, wie sie im Quellcode erscheinen. Blockelemente werden als Boxen, Inline-Elemente in der Zeile mit anderen Elementen dargestellt.

Soll diese automatische Positionierung von Elementen geändert werden, kann dies primär mit der Eigenschaft **position** [79] geschehen. Sie bietet fünf mögliche Positionierungsarten an, die im den folgenden Abschnitten erläutert werden. Zusätzlich dazu kann mit der Eigenschaft **float** ebenfalls eine Neupositionierung erreicht werden. Diese Methode wird in Abschnitt 4.2.6 beschrieben.

4.2.1 Statische Positionierung

Statische Positionierung wird durch das Setzen von **position: static** erreicht. Diese Angabe ist jedoch im Normalfall nicht nötig, da die statische Positionierung dem Document Flow entspricht. Dies wird nur benötigt, um anders positionierte Elemente wieder in den Document Flow zu bringen.

Folgender Code gibt explizit für drei `<div>`-Elemente eine statische Positionierung an. Das Ergebnis ist in Abbildung 4.11 ersichtlich.

```
1 div {  
2   border: 1px solid black;  
3   width: 250px;  
4   height: 100px;  
5   margin: 10px;  
6   position: static;  
7 }  
8 #one {  
9   background-color: orange;  
10 }  
11 #two {  
12   background-color: green;
```

¹<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/display>



Abbildung 4.11: Die drei `<div>`-Elemente sind normal statisch positioniert, genauso als ob die Angabe von `position: static` nicht vorhanden wäre.

```

13 }
14 #three {
15   background-color: yellow;
16 }

```

4.2.2 Relative Positionierung

Relative Positionierung erfolgt durch die Angabe von `position: relative`. Das Element wird dadurch relativ zu seinem normalen Auftreten im Document Flow positioniert, es wird aber nicht aus dem Document Flow entfernt. Dies ist daran ersichtlich, dass die nachfolgenden Elemente nicht nachrücken, also immer noch so platziert sind, als ob das Element an seiner ursprünglichen Stelle wäre.

Um aber eine Neupositionierung zu erreichen, reicht die Angabe von `position: relative` alleine nicht aus. Zusätzlich muss eine Positionsangabe gemacht werden. Diese kann mit Hilfe der Eigenschaften `top` (von oben), `right` (von rechts), `bottom` (von unten) oder `left` (von links) geschehen. Die Elemente arbeiten mit regulären numerischen Werten. Der Bezugspunkt ist die ursprüngliche Position im Document Flow.

Sollen eventuell entstehende Überlappungen behandelt werden, kann bei den betroffenen Elementen die Eigenschaft `z-index` angegeben werden. Sie akzeptiert numerische Werte von 0 bis n . Je höher der Wert, desto weiter vorne wird das Element angeordnet.

Im folgenden Code ist das zweite der drei `<div>`-Elemente relativ positioniert. Abbildung 4.12 zeigt das Ergebnis im Browser.

```

1 div {
2   border: 1px solid black;
3   width: 250px;
4   height: 100px;
5   margin: 10px;
6 }
7 #one {
8   background-color: orange;
9 }
10 #two {
11   position: relative;

```



Abbildung 4.12: Das zweite `<div>`-Element ist relativ zu seiner alten Position um 80px nach rechts (von links ausgehend) und 30px nach unten (von oben ausgehend) verschoben.

```
12 left: 80px;
13 top: 30px;
14 background-color: green;
15 }
16 #three {
17 background-color: yellow;
18 }
```

4.2.3 Absolute Positionierung

Absolute Positionierung wird durch Verwendung von `position: absolute` erreicht. Diese Angabe nimmt das Element aus dem Document Flow heraus, d.h. andere Elemente unterhalb rücken einfach nach. Wiederum geschieht die Positionierung durch Angaben von `top`, `right`, `bottom` oder `left`.

Der Bezugspunkt für eine Neupositionierung ist das nächste Elternelement, das *nicht* mit `position: static` positioniert wurde. Gibt es kein solches, dann ist das Browserfenster der Bezugspunkt. Mögliche Überlappungen können wiederum mit der Eigenschaft `z-index` verwaltet werden.

Im folgenden Beispiel ist das zweite Element absolut positioniert. Das dritte Element rückt direkt an das erste, da das zweite nicht mehr im Document Flow enthalten ist. Abbildung 4.13 zeigt dieses Verhalten.

```
1 div {
2   border: 1px solid black;
3   width: 250px;
4   height: 100px;
5   margin: 10px;
6 }
7 #one {
8   background-color: orange;
9 }
10 #two {
11   position: absolute;
12   left: 80px;
13   top: 30px;
14   background-color: green;
```

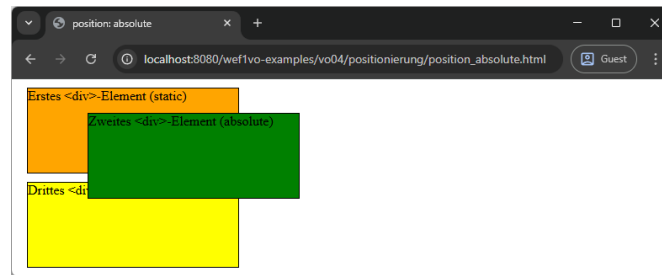



Abbildung 4.13: Das zweite `<div>`-Element ist absolut positioniert. Da kein anderes nicht-statisch platziertes Element als Elternelement vorhanden ist, wird es 80px vom linken und 30px vom oberen Browserfensterrand positioniert.

```

15 }
16 #three {
17   background-color: yellow;
18 }

```

4.2.4 Fixe Positionierung

Auch bei *fixer Positionierung* wird das Element aus dem Document Flow entfernt. Verantwortlich ist die Angabe von `position: fixed`. Im Unterschied zu absoluter Positionierung wird das Element im Bezug zum Ausgabemedium positioniert, bleibt also immer an derselben Stelle am Bildschirm oder am Papier, falls die Seite gedruckt wird. Dies kann etwa für eine mitscrollende Navigation verwendet werden, sollte aber überlegt eingesetzt werden, da der Effekt mitunter störend sein kann.

Auch hier erfolgt die Positionierung mit Hilfe von `top`, `right`, `bottom` oder `left`. Der Bezugspunkt ist der Viewport, also das Browserfenster. Überlappungen können mit `z-index` verwaltet werden.

Folgender Code positioniert das zweite Element fix. Muss gescrollt werden, so bleibt es immer an derselben Stelle. Abbildung 4.14 stellt dies annähernd dar – selbst ausprobieren wird ans Herz gelegt.

```

1 div {
2   border: 1px solid black;
3   width: 250px;
4   height: 100px;
5   margin: 10px;
6 }
7 #one {
8   background-color: orange;
9 }
10 #two {
11   position: fixed;
12   left: 80px;
13   top: 30px;
14   background-color: green;
15 }
16 #three {
17   background-color: yellow;
18 }

```

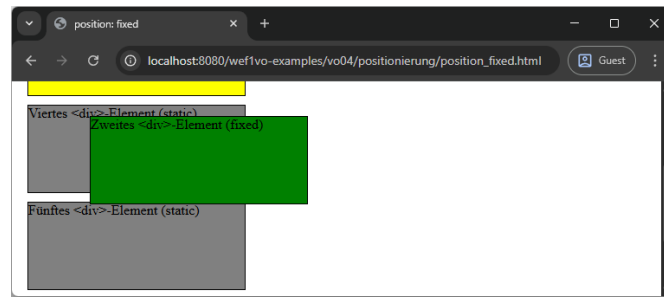


Abbildung 4.14: Das zweite `<div>`-Element ist fix positioniert. Obwohl im Browserfenster schon nach unten gescrollt wurde, befindet sich das Element nach wie vor 80px von links und 30px vom oberen Rand des Browserfensters entfernt.

```
19 #four, #five, #six {
20   background-color: grey;
21 }
```

4.2.5 Sticky Positionierung

Die *sticky Positionierung* stellt eine Hybridform aus relativer und fixer Positionierung dar und wird über `position: sticky` erreicht. Die Angaben `top`, `right`, `bottom` oder `left` geben dabei den Minimalabstand an, den ein Element von einer Seite des Viewports haben muss. Solange der Wert größer ist, verhält sich das Element wie ein relativ positioniertes, wird er unterschritten (etwa durch Scrollen), wird das Element fix bei diesem Abstand positioniert.

Im folgenden Beispiel wird ein Element sticky positioniert. Zunächst bleibt es regulär im Document Flow und ist um 80 Pixel eingerückt. Wird abwärts gescrollt und der obere Abstand des Elements zum Browserfensterrand unterschreitet 30 Pixel, bleibt das Element an dieser Stelle fix stehen. Dies entspricht Abbildung 4.14.

```
1 div {
2   border: 1px solid black;
3   width: 250px;
4   height: 100px;
5   margin: 10px;
6 }
7
8 #one {
9   background-color: orange;
10 }
11
12 #two {
13   position: sticky;
14   left: 80px;
15   top: 30px;
16   background-color: green;
17 }
18
19 #three {
20   background-color: yellow;
```

```
21 }
22
23 #four, #five, #six {
24   background-color: grey;
25 }
```

4.2.6 Floats

Die CSS-Eigenschaft `float` nimmt ein Element aus dem Document Flow und verschiebt es ganz nach links oder rechts, bis es beim nächsten übergeordneten Element ansteht. Eine Verschiebung nach links wird mit `float: left`, eine Verschiebung nach rechts mit `float: right` erwirkt. Nachfolgende Elemente rutschen an den freigewordenen Platz, der Text in diesen Elementen *umfließt* jedoch (im Unterschied etwa zur absoluten Positionierung) das gefloatete Element.

Ursprünglich entworfen wurde die Eigenschaft `float`, um ein Bild innerhalb eines Absatzes einfach von Text umfließen lassen zu können. Die Eigenschaft kann jedoch auch zum schnellen horizontalen Anordnen von Elementen oder sogar für komplette Layouts (siehe Abschnitt 4.3) verwendet werden.

Soll ein Nachrücken der nachfolgenden Elemente verhindert werden, so kann die Eigenschaft `clear` mit den Werten `left`, `right` oder `both` (je nachdem, in welche Richtung zuvor gefloatet wurde) beim nachfolgenden Element angegeben werden.

Abbildung 4.15 zeigt vier verschiedene Anwendungsfälle von `float` und `clear`. Der Code zum Beispiel in Abbildung 4.15 (d) lautet wie folgt:

```
1 div {
2   border: 1px solid black;
3   margin: 10px;
4 }
5 #one {
6   background-color: orange;
7   float: right;
8   width: 250px;
9   height: 100px;
10 }
11 #two {
12   background-color: green;
13   float: left;
14   width: 300px;
15   height: 200px;
16 }
17 #three {
18   background-color: yellow;
19   clear: left;
20   width: 400px;
21   height: 100px;
22 }
```

Floats haben noch einige weitere Besonderheiten, die oftmals bei der Arbeit auftreten. Dazu gehören etwas das Kollabieren übergeordneter Elemente, wenn die darin enthaltenen alle gefloatet sind oder auch die Thematik des Float-Clearings, also Techniken, wie effektiv die `clear`-Eigenschaft angewandt werden kann. Diese Dinge sind ausführlich und gut illustriert in [10] beschrieben.



Abbildung 4.15: Mehrere Anwendungsfälle von `float` und `clear`. Das erste Element ist nach rechts gefloatet, das zweite und dritte rücken auf (a), das erste Element ist nach rechts, das zweite nach links gefloatet, das dritte rückt auf und der Text umfließt das Element (b), die ersten beiden Elemente sind rechts bzw. links gefloatet, das dritte erhält eine `clear: right` Anweisung und wird unter dem rechten positioniert (c), die ersten beiden Elemente sind links und rechts gefloatet, das dritte wird durch `clear: left` unter dem linken positioniert (d).

4.3 Einfache Anordnungen und Layouts mit float

Floats werden (neben ihrer ursprünglichen Anwendung, Bilder von Text umfließen zu lassen) meist für zwei Dinge eingesetzt: das horizontale Anordnen von Elementen in kleinen Bereichen oder für größere Layouts. Angemerkt sei, dass diese Techniken zwar immer noch gültig sind, mittlerweile aber mit Hilfe von moderneren Tools wie Flexbox oder dem Grid Layout umgesetzt werden sollten. Mehr dazu in Vorlesung 5.

4.3.1 Horizontales Anordnen von Elementen

Um Elemente nebeneinander anordnen zu können, reicht eine `float: left` (bzw. eine `float: right`) Anweisung auf einem der Elemente. Dieses wird dann ganz nach links (bzw. rechts) verschoben, das zweite Element rutscht daneben – immer vorausgesetzt, dass genug Platz ist. Ändert sich die Größe des gefloateten Elements, reagiert der daneben platzierte Inhalt darauf und rückt entsprechend nach.

Ein Beispiel ist eine so genannte Staff-Card, die sich auf vielen Firmenwebseiten findet. Hierbei wird neben das Foto einer Person deren Name und Position in der Firma platziert. Das Foto floatet links, die Daten werden entsprechend daneben platziert. Zunächst das HTML-Markup:

```
1 <section class="staff-card">
2   <div class="staff-photo">
```

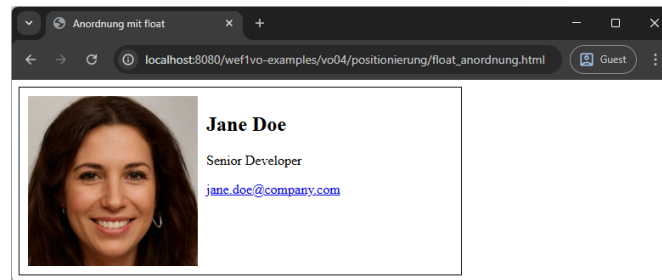


Abbildung 4.16: Das `<div>`-Element mit dem Bild floatet links, der Text wird rechts daneben angeordnet.

```

3    <!-- Image generated by https://thispersondoesnotexist.com/ -->
4    
5  </div>
6  <div class="staff-details">
7    <h2>Jane Doe</h2>
8    <p>Senior Developer</p>
9    <p><a href="mailto:jane.doe@company.com">jane.doe@company.com</a></p>
10 </div>
11 </section>

```

Dieses wird mit folgendem CSS-Code formatiert. Das Ergebnis ist in Abbildung 4.16 ersichtlich.

```

1  .staff-card {
2    width: 500px;
3    height: 200px;
4    padding: 10px;
5    border: 1px solid black;
6  }
7
8  .staff-photo {
9    height: 100%;
10   float: left;
11   margin-right: 10px;
12 }
13
14 .staff-photo img {
15   height: 100%;
16 }

```

4.3.2 Einfache Layouts

Nach demselben Prinzip können Floats verwendet werden, um einfache Layouts zu erstellen. Im folgenden Beispiel wird ein dreispaltiges Layout erstellt, das nicht die gesamte Breite der Seite in Anspruch nimmt, also sich in einem zentrierten Container befindet. Das Layout besteht aus Kopf- und Fußzeile. Dazwischen befinden sich die Navigation, der Bereich für den Hauptinhalt und rechts eine Sidebar. Abbildung 4.17 stellt die grundsätzliche Aufteilung dar.

Das HTML-Gerüst für das Layout bedient sich der bereits bekannten Strukturelemente. Der Kopfbereich verwendet erwartungsgemäß das `<header>`-Element, für die

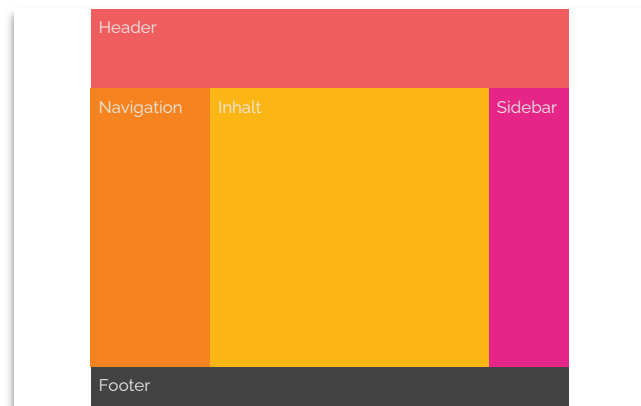


Abbildung 4.17: Die Seite wird in Header, Navigation, Inhalt, Sidebar und Footer aufgeteilt. Navigation, Inhalt und Sidebar sollen dabei nebeneinander positioniert sein.

Navigation kommt `<nav>` zum Einsatz. Der Inhaltsbereich wird mit `<main>` ausgezeichnet, darin wird eine `<section>` verwendet, um den eigentlichen Inhalt zu platzieren und `<aside>` für die Sidebar. Für den Fußbereich kommt `<footer>` zum Einsatz. Um alle Elemente herum wird mit einem `<div>`-Element ein unsichtbarer Container platziert, der zentriert wird und eine maximale Breite bekommt, damit der Inhalt nicht die komplette Browserbreite einnimmt. Dies ist vor allem bei hochauflösenden oder ultrabreiten Displays von Vorteil.

```
1 <body>
2   <div id="container">
3     <header>
4       <h1>Überschrift</h1>
5     </header>
6     <nav>
7       <ul>
8         <li>Navigation 1</li>
9         <li>Navigation 2</li>
10      </ul>
11    </nav>
12    <main>
13      <section id="content">
14        <p>Lorem ipsum dolor sit amet, [...]</p>
15      </section>
16      <aside>
17        <p>Lorem ipsum dolor sit amet, [...]</p>
18      </aside>
19    </main>
20    <footer>
21      <p>&copy; 2025 Der*Die Autor*in</p>
22    </footer>
23  </div>
24 </body>
```

Die Anordnung erfolgt nun komplett über CSS. Zunächst werden mit dem Universal-selektor alle Außenabstände auf 0 gesetzt und das Box-Modell auf `border-box` gesetzt.

Damit können später gefahrlos `padding`- und `border`-Angaben gesetzt werden, ohne dass die nebeneinanderliegenden Spalten die berechnete Gesamtbreite übersteigen, und somit nicht mehr nebeneinander Platz hätten. Der äußere Container wird auf eine fixe Breite gesetzt und wird mit Hilfe einer Außenrandangabe, bei der die vertikalen Werte auf 0, die horizontalen auf `auto` gesetzt werden, zentriert. Der Container bekommt eine Hintergrundfarbe, ebenso der Header.

Danach werden Navigation und Hauptbereich gefloatet – links und rechts, die Summe ergibt genau 100 % der Containerbreite. Die Navigation, meist kürzer als der Inhalt, erhält keine Hintergrundfarbe. Hier „scheint“ die Farbe des Containers durch. Der Hauptbereich bekommt eine Hintergrundfarbe, die für die Sidebar verwendet wird. Diese wird rechts gefloatet, der Abschnitt für den Inhalt links. Die Gesamtbreite entspricht der Breite des Hauptbereichs. Der Abschnitt für den Inhalt wiederum benötigt zur Unterscheidung eine eigene Hintergrundfarbe.

Schließlich soll der Footer unter die beiden Spalten gesetzt werden. Dafür wird `clear: both` verwendet. Der Footer bekommt ebenfalls eine Hintergrundfarbe. Alle fünf Bereichselemente bekommen schließlich noch etwas `padding` gesetzt, damit der Text nicht direkt am Rand ihrer Elemente „klebt“.

Das fertige Layout ist in Abbildung 4.18 ersichtlich.

```
1 * {  
2   margin: 0;  
3   box-sizing: border-box;  
4 }  
5 body {  
6   color: rgb(236 233 230);  
7   font-family: Raleway, Arial, sans-serif;  
8   line-height: 2em;  
9 }  
10 div#container {  
11   width: 960px;  
12   margin: 0 auto;  
13   background-color: rgb(255 132 0);  
14 }  
15 header {  
16   background-color: rgb(250 94 94);  
17 }  
18 nav {  
19   width: 240px;  
20   float: left;  
21 }  
22 main {  
23   width: 720px;  
24   float: right;  
25   background-color: rgb(228 42 135);  
26 }  
27 section#content {  
28   width: 600px;  
29   float: left;  
30   background-color: rgb(255 182 0);  
31   color: rgb(67 67 67);  
32 }  
33 aside {  
34   width: 120px;
```

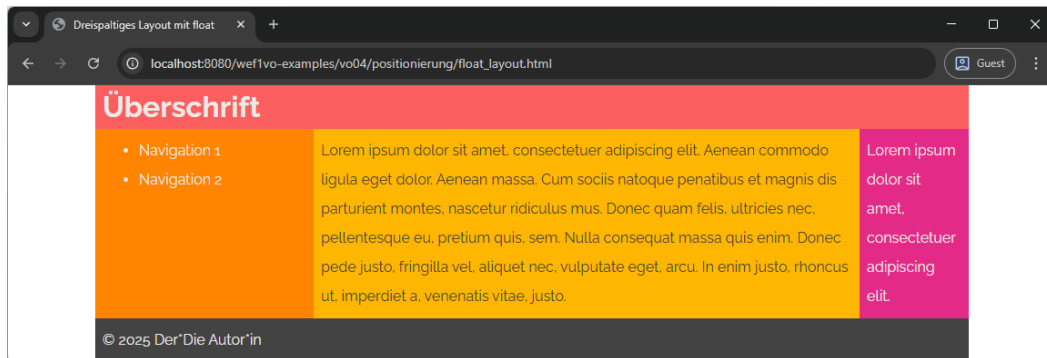


Abbildung 4.18: Das fertige Layout. Die nebeneinander angeordneten Bereiche werden durch `float`-Anweisungen erreicht.

```

35 float: right;
36 }
37 footer {
38   clear: both;
39   background-color: rgb(67 67 67);
40 }
41 header, nav, footer, section, aside {
42   padding: 0.5em;
43 }

```

4.4 Media Queries

In Zeiten, in denen mobile Endgeräte immer verstärkter zur Darstellung von Websites verwendet werden, auf der anderen Seite aber Desktop-Computer immer größere Auflösungen erreichen, wird es mit herkömmlichen Mitteln schwierig, Layouts möglichst gut dem Endgerät anzupassen. *Media Queries* [96–98] versuchen, dieses Problem auf effektive Art und Weise zu lösen.

4.4.1 Medienspezifische Stylesheets

Medienabhängige Stylesheets (ein Stylesheet für den Bildschirm und eines für den Druck etwa) gibt es schon in CSS 2.1 bzw. HTML 4.01. So kann bei der Einbindung mittels `<link>`-Tag das Attribut `media` angegeben werden, für welches Medium ein Stylesheet verwendet werden soll. In folgendem HTML-Fragment aus dem `<head>` eines Dokuments, wird für die Ausgabe am Bildschirm die Datei `style.css` und beim Druck die Datei `print.css` eingebunden:

```

1 <link rel="stylesheet" href="style.css" media="screen">
2 <link rel="stylesheet" href="print.css" media="print">

```

Auch bei der Einbindung mittels `@import` kann der Medientyp angegeben werden:

```

1 @import url("style.css") screen;
2 @import url("print.css") print;

```


Tabelle 4.1: Die derzeit verfügbaren (und sinnvoll interpretierten) Medientypen.

Medientyp	Ausgabegerät
all	Alle Ausgabegeräte.
print	Drucker und Geräte, die einen Drucker simulieren (z. B. die Druckvorschau im Browser).
screen	Alle bildschirmorientierten Geräte, die nicht unter print fallen. Dazu zählen auch Screenreader.

Medienspezifische Angaben können aber auch innerhalb eines Stylesheets gemacht werden. Dabei werden die entsprechenden Angaben mit der `@media`-Anweisung gruppiert. Im Folgenden etwa wird nur beim Ausdruck die Navigation ausgeblendet, andere Regeln im Stylesheet bleiben davon unbeeinflusst und sind auch beim Druck wirksam:

```

1 @media print {
2   nav {
3     display: none;
4   }
5 }
```

4.4.2 Medientypen

Es gibt eine Reihe von verschiedenen Schlüsselwörtern, die im Zusammenhang mit `@media`-Angaben verwendet werden können. Diese haben sich über die Zeit jedoch als oft nicht mehr passend herausgestellt, sodass in der letzten Überarbeitung der Media Query Spezifikation [98, Abschnitt 2.3] nur noch drei Typen empfohlen werden, die von Browsern erkannt und unterschiedlich gehandhabt werden sollen. Tabelle 4.1 stellt diese dar.

Als veraltet angesehen werden die Typen **tty** (textbasierte Computerterminals), **tv** (TV-Geräte), **projection** (Projektoren), **handheld** (mobile Geräte mit geringer Auflösung), **braille** (Braille-Zeilen-basierte Ausgabegeräte), **embossed** (Braille-Drucker), **aural** (Sprachsynthesizer) und **speech** (Screenreader). Während einige dieser Typen zunächst durchaus sinnvoll erscheinen (etwa **handheld** für mobile Geräte), hat sich gezeigt, dass eine Unterscheidung durch die technologische Entwicklung (Smartphones mit sehr hoher Auflösung) schwierig wurde. Stattdessen wurden die Medientypen reduziert und feinere Unterscheidungen können mit Hilfe der diversen Medienmerkmale (siehe Abschnitt 4.4.4) getroffen werden. Es wird daher davon ausgegangen, dass Medientypen in Zukunft ganz verschwinden und Unterscheidungen rein anhand der Medienmerkmale getroffen werden.

4.4.3 Aufbau von Media Queries

Mit der CSS3 Media Query Spezifikation [96] und nachfolgenden Media Queries Level 4 und 5 Standards [97, 98] wurde auf dem Modell der medienspezifischen Stylesheets aufgebaut und dieses um Medienmerkmale erweitert. Formal besteht eine Media Query aus einem optionalen Media Query Modifikator (**only** oder **not**), einem optionalen Medientyp (siehe Abschnitt 4.4.2) und 0 oder mehr Medienmerkmalen (siehe Abschnitt

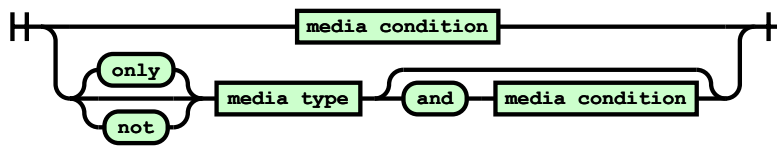


Abbildung 4.19: Der schematische Aufbau einer Media Query. Quelle: [98].

4.4.4). Die Kombination von mehreren Medienmerkmalen mit Hilfe boolescher Algebra wird dann allgemeiner als Medienbedingung (engl. „media condition“) bezeichnet.

Eine Media Query ist dabei ein logischer Ausdruck, der entweder **true** oder **false** ergibt. Ersteres ist dabei der Fall, wenn der optional spezifizierte Medientyp mit dem gerade verwendeten Gerät übereinstimmt und die Medienbedingung (mit den darin enthaltenen Medienmerkmalen) erfüllt ist. Abbildung 4.19 stellt dies schematisch dar.

Im folgenden Beispiel wird etwa eine Media Query für die Ausgabe am Bildschirm und Geräte mit einer maximalen Viewportbreite von 480 Pixeln erstellt. In den geschweiften Klammern stehen dann die CSS-Regeln für diesen Fall:

```
1 @media screen and (max-width: 480px) {
2   /* CSS-Regeln */
3 }
```

Wird vor die Media Query das Schlüsselwort **not** gestellt, dann wird sie entsprechend negiert, würde also für alle Anwendungsfälle *außer* dem spezifizierten gelten.

Durch die Angabe des Schlüsselworts **only** vor der Media Query kann die gesamte Media Query vor alten Browsern, die dieses Feature nicht unterstützen, versteckt werden. Ein alter Browser würde nämlich zwar den ersten Teil, d. h. die Angabe des Medientyps interpretieren, jedoch nicht die nachfolgenden Medienmerkmale. Dies könnte zur Folge haben, dass Styles angewandt werden, die eigentlich nicht gewünscht sind. Durch Voranstellen von **only** ignorieren alte Browser eine Media Query komplett.

4.4.4 Medienmerkmale

Medienmerkmale sind feiner aufgelöste Testkriterien als Medientypen. Sie erlauben es eine einzelne, spezifische Eigenschaft eines Geräts oder auch eines Browsers abzufragen. Syntaktisch sind sie zunächst wie CSS-Eigenschaften aufgebaut, sie bestehen aus dem Merkmal, einem Doppelpunkt und einem Wert auf den getestet wird, z. B. **max-width: 480px**, um auf eine maximale Breite von 480 Pixeln abzufragen. Allerdings werden Medienmerkmale immer in Klammern gesetzt und können in der neuesten Spezifikation auch Vergleichsoperatoren anstelle des Doppelpunkts haben oder einfach nur auf die Eigenschaft prüfen. So ist **(width <= 480px)** möglich oder auch **(color)**. Letzteres überprüft die Farbtiefe eines Ausgabegeräts und ist dann **true**, wenn der gelieferte Wert nicht 0 ist.

Die folgenden Medienmerkmale sind unter anderem verfügbar, viele besitzen auch noch die jeweiligen **min-** und **max-** Angaben. Diese sind meist zu bevorzugen, da etwas die Größe des Viewports eines Desktop-Browser selten der genauer Pixelwert ist und somit eine Abfrage auf Minimal- oder Maximalwerte wesentlich zuverlässiger ist. Eine

vollständige Auflistung findet sich in [96–98].

- **width**: Breite des Viewports (der innere Bereich des Browserfensters oder Ausgabegeräts, ohne Menüs, Scrollbalken etc.).
 - **min-width**: Die minimale Breite des Viewports (alles größer gleich des angegebenen Werts ergibt **true**).
 - **max-width**: Die maximale Breite des Viewports (alles bis zum angegebenen Wert ergibt **true**).
- **height**: Höhe des Viewports.
 - **min-height**: Minimale Höhe des Viewports.
 - **max-height**: Maximale Höhe des Viewports.
- **orientation**: Ausrichtung mit den Werten **portrait** und **landscape**. Interessant für mobile Endgeräte.
- **aspect-ratio**: Verhältnis zwischen **width** und **height**. Werte werden mit Schrägstrich notiert (z. B. 16/9).
 - **min-aspect-ratio**: Das minimale Seitenverhältnis.
 - **max-aspect-ratio**: Das maximale Seitenverhältnis.
- **color**: Die Farbtiefe des aktuellen Geräts in Bits pro Farbkomponente.
 - **min-color**: Die minimale Farbtiefe.
 - **max-color**: Die maximale Farbtiefe.
- **resolution**: Die Auflösung des Geräts in DPI (z. B. 72dpi), DPPX (Dots per Pixel) oder DPCM (Dots per CM).
 - **min-resolution**: Die minimale Auflösung.
 - **max-resolution**: Die maximale Auflösung.
- **pointer**: Erkennt das Vorhandensein eines bestimmten Eingabegeräts. Es existieren die Werte **none** (nur Tastatur), **coarse** (limitierte Genauigkeit wie Touchinput) und **fine** (präzise Genauigkeit wie etwa Mäuse, Touchpads oder Stifte).
- **prefers-reduced-motion**: Erkennt, ob die Systemeinstellungen von Nutzer*innen auf reduzierte Bewegungen gesetzt sind. Damit können etwa Animationen auf der Webseite deaktiviert werden. Werte sind **no-preference** oder **reduce**.
- **prefers-contrast**: Erkennt, ob Nutzer*innen erhöhten oder verringerten Kontrast bevorzugen. Werte sind **no-preference**, **high**, **low** oder **forced**.

4.4.5 Verknüpfung von Medienmerkmalen

In einer Media Query können mehrere Medienmerkmale miteinander verknüpft werden (was dann als Medienbedingung bezeichnet wird). Dies kann mit boolescher Algebra geschehen:

- Logisches Und: Mit dem **and** Schlüsselwort.
- Logisches Oder: Mit einem Komma (,) oder seit Media Queries Level 4 mit dem **or** Schlüsselwort.
- Logische Verneinung: Mit dem **not** Schlüsselwort.

Darüber hinaus kann mit Klammerung die Auswertung beeinflusst werden.

Die folgende Media Query gilt nur für Geräte, deren Breite zwischen 768 und 960 Pixeln liegt:

```
1 @media screen and (min-width: 768px) and (max-width: 960px) {  
2   /* CSS-Regeln */  
3 }
```

4.4.6 Viewport angeben

Damit eine Media Query auf einem mobilen Gerät sinnvoll verwendet werden kann, ist die Angabe des Viewports notwendig. Browser auf mobilen Geräten geben sich in der Regel nämlich wie Desktopbrowser aus, d. h. sie weisen wesentlich mehr Höhe und Breite aus, als ihnen eigentlich zur Verfügung steht, damit Webseiten, die nicht für mobile Ansicht optimiert wurden komplett (aber in der Regel sehr klein) dargestellt werden. Wäre das nicht der Fall, wären nicht optimierte Webseiten nur zum Teil sichtbar und es müsste auch horizontal gescrollt werden. Smartphones haben zwar Bildschirme mit vielen Pixeln, durch die hohe Pixeldichte (Device Pixel Ratio, DPR) müssen die horizontale und vertikale Auflösung durch die DPR dividiert werden, um die tatsächlichen „CSS-Pixel“ zu erhalten. So hat ein Smartphone mit 1.355 Pixel breitem Display und einer DPR von 3 eine Breite von 448 Pixeln.

Dieses Verhalten führt also dazu, dass Media Queries nur bedingt greifen, da der Browser am Mobiltelefon zunächst wesentlich mehr Breite als etwa die in der Media Query angegebenen 480 ausweist. Dies bedeutet, dass am Mobiltelefon trotzdem die Desktop-Variante angezeigt wird. Um dies zu verhindern, muss eine weitere Meta-Angabe in den `<head>` des Dokuments gesetzt werden:

```
1 <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Dadurch wird auf mobilen Geräten die Breite auf die tatsächliche Gerätebreite gelegt und der Zoom auf den Faktor 1 gestellt. Somit identifiziert sich obiges Smartphone mit 448 Pixeln und die Media Query für die Mobilansicht greift.

Mit der Angabe `user-scalable=no` könnte das Zoomen ganz verhindert werden (in der Regel nicht empfehlenswert), mit `minimum-scale` und `maximum-scale` kann die Zoomstufe begrenzt werden.

4.4.7 Media Queries einsetzen

Bei der Arbeit mit Media Queries geht es darum zu entscheiden, welches Layout auf Geräten akzeptabel funktioniert bzw. gut aussieht. Es müssen so genannte *Breakpoints* gesetzt werden, bei denen eine Layoutänderung stattfindet, d. h. wo etwa die Navigation nicht mehr links und der Inhalt rechts angeordnet werden, sondern beides untereinander. Ein Breakpoint wird durch eine `@media`-Regel definiert, mit der neue Style-Regeln ab oder bis zu einer gewissen Breite in Kraft treten.

Diese Breakpoints können einerseits an den Auflösungen bekannter Geräte² ausgerichtet werden. Allerdings kommen laufend neue Geräte mit immer neuen Auflösungen auf den Markt, was dieses Vorhaben mitunter schwierig macht.

²Eine Übersicht bietet etwa [22].

Ein aktuellerer Ansatz ist es, die Breakpoints am Inhalt und dem Design der eigenen Seite auszuwählen. Also selbst, nach dem eigenen ästhetischen Empfinden zu bestimmen, ab welcher Größe eine Änderung stattfinden soll. Sie können dazu nach zwei Ansätzen vorgehen: *Desktop first* und *Mobile first*.

Desktop first

Beim Desktop first Ansatz, wird zunächst das komplette Layout, optimiert für eine Desktop Auflösung definiert. Also etwa wie im zweispaltigen, seitenbreiten Layout mit zwei Spalten nebeneinander. Nun wird die Seite in immer schmäleren Breiten (etwa durch einfaches Zusammenschieben des Browsers oder mit Hilfe der des Device Modus³ der Chrome DevTools betrachtet (andere Browser bieten ebenfalls derartige Funktionalität an). Wenn das Layout nicht mehr gut aussieht (zu schmale Spalten etc.) oder überhaupt unschön umbricht, wird ein Breakpoint definiert, d. h. eine Media Query definiert, die ab dieser Breite etwas am Layout ändert. Dies kann natürlich an mehreren Stellen geschehen und sollte mit mehreren Geräten getestet werden.

Beim Desktop first Ansatz kommen in der Regel Medieneigenschaften wie `max-width` zum Zug, um alles unter einer bestimmten Auflösung abzuändern.

Im folgenden Beispiel kommt das dreispaltige Layout aus Abschnitt 4.3.2 zum Einsatz. Das HTML-Markup bleibt dabei völlig unverändert.

In der Desktop first Variante ist auch das CSS fast identisch, lediglich die Spaltenbreiten werden in Prozent anstatt Pixel angegeben. Dies ermöglicht eine Stufenlose Anpassung der Spalten, wenn etwas das Browserfenster verkleinert wird. Zunächst wird auch der Container fix mit 960 Pixeln breite angegeben.

Der CSS-Code für die wichtigsten Elemente (andere finden sich im Code zur Vorlesung):

```
1 div#container {
2   width: 960px;
3   margin: 0 auto;
4   background-color: rgb(255 132 0);
5 }
6 header {
7   background-color: rgb(250 94 94);
8 }
9 nav {
10  width: 25%;
11  float: left;
12 }
13 main {
14  width: 75%;
15  float: right;
16  background-color: rgb(228 42 135);
17 }
18 section#content {
19  width: 80%;
20  float: left;
21  background-color: rgb(255 182 0);
22  color: rgb(67 67 67);
23 }
```

³<https://developer.chrome.com/docs/devtools/device-mode/>

```
24 aside {  
25   width: 20%;  
26   float: right;  
27 }
```

Nun werden mit Hilfe von Media Queries mehrere Breakpoints gesetzt. Der erste passiert, wenn die Breite von 960 Pixeln unterschritten wird. Dabei wird die Breite des Containers auf 100 % umgestellt, sodass er sich fortan dem Viewport anpasst.

```
1 @media screen and (max-width: 960px) {  
2   div#container {  
3     width: 100%;  
4   }  
5 }
```

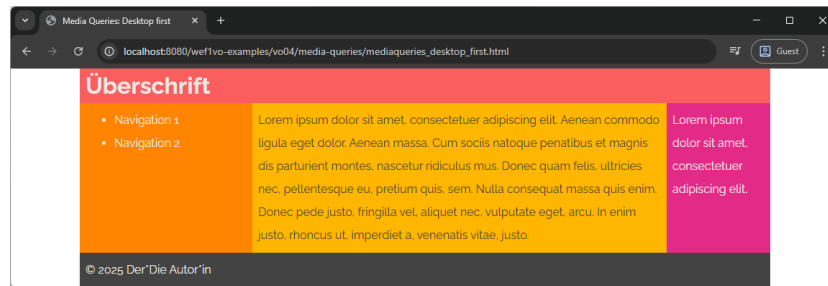
Der nächste Breakpoint wird bei 768 Pixeln gesetzt. Hier soll die Navigation über den zwei gefloateten Inhaltsbereichen platziert werden. Dazu wird das Floating aufgehoben und die Navigationspunkte nebeneinander angeordnet. Damit der Inhalt nun die volle Breite annimmt, wird das `<main>`-Element auf 100 % Breite gesetzt (das Floating nach rechts wird beibehalten, da das Element sonst leer wäre und kollabieren würde, dadurch hätte die Sidebar keine entsprechende Hintergrundfarbe mehr). Dies entspricht in der Regel der Darstellung auf einem Tablet.

```
1 @media screen and (max-width: 768px) {  
2   nav {  
3     float: none;  
4     width: 100%;  
5     text-align: center;  
6   }  
7   nav ul {  
8     padding: 0;  
9   }  
10  nav ul li {  
11    list-style-type: none;  
12    display: inline;  
13  }  
14  nav ul li:not(:last-child) {  
15    margin-right: 0.5em;  
16  }  
17  main {  
18    width: 100%;  
19  }  
20 }
```

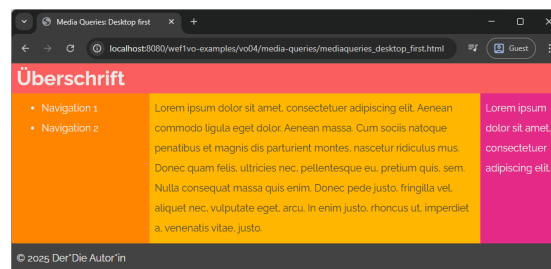
Schließlich wird noch eine Darstellung für Smartphones angestrebt. Da hier weniger horizontaler Platz ist, werden die beiden Inhaltsbereiche und der Hauptbereich nicht mehr gefloatet und auf volle Breite skaliert. Alles ist nun untereinander angeordnet.

```
1 @media screen and (max-width: 480px) {  
2   section#content, aside, main {  
3     float: none;  
4     width: 100%;  
5   }  
6 }
```

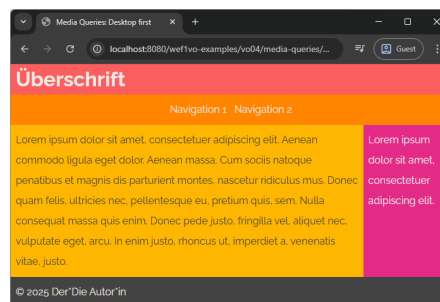
Abbildung 4.20 zeigt alle vier Zustände der Seite.



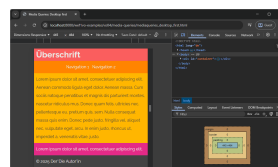
(a)



(b)



(c)



(d)

Abbildung 4.20: Das Layout nach den jeweiligen Breakpoints: bei 1156px Breite (a), bei 810px Breite (b), bei 630px Breite (c) und bei 456px Breite (d). Bei der letzten Einstellung muss der Developer Mode der Chrome DevTools verwendet werden, da aktuelle Chrome-Versionen keine Fenstergrößen unter 500 Pixel zulassen.

Mobile first

Der Mobile first Ansatz funktioniert umgekehrt. Zunächst wird die Seite für eine mobile Auflösung, also für geringe Breite, mit kleineren Grafiken etc. konzipiert. Dann wird in Richtung einer Desktop-Auflösung gearbeitet und es werden überall dort Breakpoints gesetzt, wo das bessere Ausnützen des zusätzlichen horizontalen Platzes, Sinn macht.

Beim Mobile first Ansatz werden daher Medieneigenschaften wie `min-width` verwen-

det, um alles *über* einer bestimmten Auflösung abzuändern.

Das obige Beispiel wird Mobile first wie folgt umgesetzt. Zunächst werden Formatierungen wie Hintergrundfarben und das horizontale Ausrichten der Navigation gesetzt, da alles untereinander angeordnet wird und dies keine besondere Formatierung benötigt:

```
1 div#container {
2   background-color: rgb(255 132 0);
3 }
4 nav {
5   text-align: center;
6 }
7 nav ul {
8   padding: 0;
9 }
10 nav ul li {
11   list-style-type: none;
12   display: inline;
13 }
14
15 nav ul li:not(:last-child) {
16   margin-right: 0.5em;
17 }
18
19 main {
20   background-color: rgb(228 42 135);
21 }
22
23 section#content {
24   background-color: rgb(255 182 0);
25   color: rgb(67 67 67);
26 }
```

Dann wird ein Breakpoint bei 480 Pixeln definiert. Ab dieser Größe sollen die beiden Spalten für den Inhalt wieder nebeneinander stehen. Dazu werden sie zusammen mit dem `<main>`-Element gefloatet. Ebenso werden die Breitenangaben angepasst.

```
1 @media screen and (min-width: 480px) {
2   main {
3     width: 100%;
4     float: right;
5   }
6   section#content {
7     float: left;
8     width: 80%;
9   }
10  aside {
11    float: left;
12    width: 20%;
13  }
14  footer {
15    clear: both;
16  }
17 }
```

In der nächsthöheren Stufe ab 768 Pixeln soll dann auch die Navigation wieder links neben dem Inhalt erscheinen. Also floatet auch sie und erhält angepasste Breitenangaben. Die Navigation erscheint wieder als Liste, weshalb die entsprechenden Angaben

gesetzt werden.

```
1 @media screen and (min-width: 768px) {  
2   nav {  
3     float: left;  
4     width: 25%;  
5     text-align: left;  
6   }  
7   main {  
8     width: 75%;  
9   }  
10  nav ul {  
11    padding-left: 40px;  
12  }  
13  nav ul li {  
14    list-style-type: disc;  
15    display: list-item;  
16  }  
17  nav ul li:not(:last-child) {  
18    margin-right: 0;  
19  }  
20 }
```

Schließlich wird noch ab 960 Pixeln der Container fest auf diese Breite gesetzt.

```
1 @media screen and (min-width: 960px) {  
2   div#container {  
3     width: 960px;  
4     margin: 0 auto;  
5   }  
6 }
```

Damit wurde ein Layout mit demselben Verhalten wie das Desktop first Beispiel geschaffen, Abbildung 4.20 gilt also auch hier. Der Ansatz ist jedoch genau umgekehrt.

Welches das bessere Rezept ist, hängt sicherlich vom Anwendungsfall ab und kann nicht pauschal beantwortet werden. Im Sinne eines steigenden Marktanteils mobiler Endgeräte ist die Mobile first Lösung vielleicht etwas vorzuziehen, zumal sie Entwickler*innen auch dazu zwingt, von einem reduzierten Design und Umfang auszugehen und diese übersichtlich aufzubauen.

4.5 Weiterführende Literatur

Informationen zum CSS Box-Modell und zur Positionierung von Elementen werden in [18, 37, 40, 58] ausführlich erläutert.

Selbstverständlich bietet auch die Dokumentation zur CSS Recommendation [62] eine Übersicht, wenngleich diese mitunter sehr technisch und nicht immer ganz einfach zu lesen ausfällt.

Vorlesung 5

CSS Layout-Frameworks

5.1 CSS Flexible Box Layout

In Vorlesung 4 wurden mehrere Layout-Modelle erwähnt:

- Block-Layout (`display: block`), mit dem Dokumente standardmäßig dargestellt werden,
- Inline-Layout (`display: inline`), mit dem Elemente auf Textebene angeordnet werden,
- Position-Layout (`position`) für explizite Positionierungen und
- Tabellen-Layout (`display: table`), welches die Tabellendarstellung regelt (aber nicht im Detail vorgestellt wurde).

Das *CSS Flexible Box Layout* [71], kurz Flexbox stellt nun ein weiteres Modell – nämlich Flex-Layout – dar. Es behebt einige grundsätzliche Probleme von den zuvor erwähnten Modellen und auch dem eigentlich zweckentfremdeten Floating. Zunächst lassen sich Boxen einfach nebeneinander positionieren, ohne auf `float` zurückgreifen zu müssen. Diese Boxen nützen selbst den vorhandenen Platz aufgrund zugewiesener Verhältnisse aus und passen sich auch gleichmäßig in der Höhe an, wenn ein Element etwa mehr Text enthält als ein anderes.

5.1.1 Einen flexiblen Container definieren

Die prinzipielle Arbeitsweise mit flexiblen Boxen sieht so aus: ein Container mit mehreren Kindelementen enthält die Eigenschaft `display: flex`, wodurch das Flex-Layout aktiviert wird. Dieser Container verbreitert seine Inhalte, um freien Platz zu nützen oder schrumpft sie, um einen Text-Überfluss zu verhindern. Der verwendete Layout-Algorithmus ist dabei unabhängig von Richtungen (im Gegensatz zu Block-Elementen die vertikal-lastig und Inline-Elementen, die horizontal-lastig sind).

Die Eigenschaft `flex-direction` bestimmt, ob Boxen nebeneinander oder untereinander platziert werden. Dies kann durch die Werte `row` oder `column` (bzw. `row-reverse` oder `column-reverse`) festgelegt werden. Mittels `flex-wrap` und den Werten `wrap` (bzw. `wrap-reverse`) oder `nowrap` kann dann noch bestimmt werden, ob ein flexibler Container ein- oder mehrzeilig werden darf. `flex-flow` ist ein Shorthand-Property und nimmt zwei Angaben für die Richtung und das Umbruchsverhalten.

Im folgenden Beispiel wird ein dreispaltiges Layout mit flexiblen Boxen erstellt. Als Grundlage dient das folgende HTML-Grundgerüst. Innerhalb eines `<main>`-Elements befinden sich ein `<section>`-Element für den Inhalt, ein `<nav>`-Element für die Navigation und ein `<aside>`-Element mit Zusatzinformationen in einer Art Sidebar. Die Anordnung der inneren Elemente (`<nav>` zwischen `<article>` und `<aside>`) wurde bewusst so gewählt, um eine mögliche (und mit Flexbox einfache) Neuordnung in CSS demonstrieren zu können.

```
1 <body>
2   <div id="container">
3     <header>
4       <h1>Die Überschrift</h1>
5     </header>
6     <main>
7       <section id="content">Lorem ipsum...</section>
8       <nav>
9         <ul>
10          <li>Navigation 1</li>
11          <li>Navigation 2</li>
12        </ul>
13      </nav>
14      <aside>Lorem ipsum...</aside>
15    </main>
16    <footer>&copy; 2025 Der*Die Autor*in</footer>
17  </div>
18 </body>
```

Das `<main>`-Element dient als Flexbox-Container und erhält die beiden Eigenschaften `display: flex` und `flex-direction: row`. Dadurch werden die drei Gliederungselemente innerhalb nebeneinander angezeigt. Die Angabe von `min-height` garantiert eine Mindesthöhe der Elemente. Ansonsten wären diese so hoch wie das höchste der drei Elemente. Der Container, der alles umgibt, wird mit 960 Pixeln Breite definiert.

```
1 * {
2   margin: 0;
3   box-sizing: border-box;
4 }
5 body {
6   font-family: Raleway, Arial, sans-serif;
7   color: rgb(236 233 230);
8   line-height: 2rem;
9   hyphens: auto; /* Silbentrennung */
10 }
11 div#container {
12   width: 960px;
13   margin: 0 auto;
14   background-color: rgb(255 132 0);
15 }
16 main {
17   display: flex;
18   flex-direction: row;
19   min-height: 25rem;
20 }
```

5.1.2 Flexibilität und Reihenfolge der Boxen festlegen

Mit dem Schlüsselwort **flex** wird nun festgelegt, wie sich die im flexiblen Container enthaltenen Elemente verhalten. Erwartet werden drei Angaben: die erste definiert die Wachstumsrate im Bezug zu den anderen, die zweite die Schrumpfrate, der dritte Wert bestimmt die Basisgröße. **flex: 1 1 auto** bedeutet, dass ein Element gleichmäßig wächst und schrumpft, **flex: 5 1 auto** lässt ein Element im Vergleich zu den anderen 5 Mal so stark anwachsen. Da in beiden Fällen der dritte Wert mit **auto** festgelegt ist, nehmen die Spalten von Beginn an die gleiche Breite ein. Alternativ können etwa auch Prozentangaben angegeben werden: gibt es zwei Container mit den Angaben **flex: 3 1 30%** und **flex: 1 1 30%**, so stehen beiden Containern jeweils 30 % der Gesamtbreite zu. De facto wird der erste Container von den nicht aufgeteilten 40% jedoch drei Mal so viel beanspruchen wie der zweite, weshalb der erste Container 60 % und der zweite Container 40 % der Gesamtbreite einnehmen werden. Eine **flex**-Angabe von 0 bedeutet, dass das Element gar nicht flexibel ist, also seine ursprüngliche Größe immer beibehalten wird.

flex ist das Shorthand-Property für **flex-grow**, **flex-shrink** und **flex-basis**, könnte also auch mit drei separaten CSS-Eigenschaften geschrieben werden.

Über die Eigenschaft **order** kann die Reihenfolge der Darstellung geändert werden, es kommen Zahlenwerte von 1 bis n zum Einsatz.

Im Beispiel teilen sich die drei Elemente ausgehend von den 960 Pixeln Breite des Containers auf 600px (entspricht 62,5 %), 240px (25 %) und 120px (12,5 %) auf, somit sind 100 % erreicht und es ist bei dieser Breite kein Platz zum Wachsen oder Schrumpfen. Der Abschnitt darf aber beim Wachsen doppelt so viel Platz in Anspruch nehmen wie Navigation und Seitenleiste. Beim Schrumpfen verliert er aber auch anteilmäßig doppelt so viel Platz wie die anderen beiden Elemente. Durch die Angabe von **order** wird die Navigation ganz links, der Artikel in der Mitte und die Sidebar ganz rechts dargestellt.

```

1 main > section {
2   flex: 2 2 600px; /* 62,5 % von 960px */
3   order: 2;
4   color: rgb(67 67 67);
5   background-color: rgb(255 182 0);
6 }
7 main > nav {
8   flex: 1 1 240px; /* 25 % von 960px */
9   order: 1;
10  background-color: rgb(255 132 0);
11 }
12 main > aside {
13  flex: 1 1 120px; /* 12,5 % von 960px */
14  order: 3;
15  background-color: rgb(228 42 135);
16 }
```

Wachsen von Elementen

Der Container ist grundsätzlich hier mit einer Breite von 960 Pixeln festgelegt. Auch die Basisbreite der Elemente ist genau auf diese Dimension ausgerichtet. Die folgende Media Query legt jedoch fest, dass ab 1200 Pixeln Fensterbreite auch der Container auf

dieselbe Breite gesetzt wird:

```
1 @media all and (min-width: 1200px) {
2   div#container {
3     width: 1200px;
4   }
5 }
```

Somit stehen für die drei mit Flexbox angeordneten Elemente nun nicht mehr 960 Pixel, sondern 1200 Pixel, also um 240 Pixel mehr Bereite zur Verfügung. Diese werden nun anteilmäßig aufgeteilt. Die Verhältnisse sind mit 2 zu 1 zu 1 in der ersten `flex`-Eigenschaft definiert, d. h. es gibt 4 Anteile, die zu vergeben sind. Mit folgender Formel lässt sich nun der neu gewonnene Platz berechnen:

$$width_{new} = \left(\frac{flex-grow}{flex-grow_{total}} \cdot free-space \right) \cdot width$$

Für das Abschnitt-Element ist der *flex-grow* Wert 2, für die anderen beiden jeweils 1, *flex-grow_{total}* ist also 4 und *free-space* ist 240. *width* ist die jeweils ursprüngliche Breite, also 600, 240 und 120 Pixel. Somit ergeben sich folgende neue Breiten:

- Abschnitt: $width_{new} = \left(\frac{2}{4} \cdot 240 \right) \cdot 600 = 720$
- Navigation: $width_{new} = \left(\frac{1}{4} \cdot 240 \right) \cdot 240 = 300$
- Seitenleiste: $width_{new} = \left(\frac{1}{4} \cdot 240 \right) \cdot 120 = 180$

Anders ausgedrückt, die 240 Pixel Platz verteilen sich im Verhältnis 2:1:1, also 50 %, 25 % und 25 % auf die drei Elemente, was einen Platzgewinn von 120, 60 und 60 Pixeln bedeutet.

Schrumpfen von Elementen

Das Schrumpfen von Elementen ist etwas komplizierter zu berechnen, da nicht nur der neu gewonnene Platz aufgeteilt werden muss, sondern das Schrumpfen im Zusammenhang mit der bisherigen Breite (`flex-shrink`) steht. Um den Container unter 960 Pixel schrumpfen zu lassen, ist folgende Media Query verantwortlich. Sie setzt den Container auf die volle Browserfensterbreite:

```
1 @media all and (max-width: 960px) {
2   div#container {
3     width: 100%;
4   }
5 }
```

Wird dieses nun auf 720 Pixel verkleinert, entsteht die entsprechende Containerbreite, somit fehlen nun 240 Pixel verglichen zur ursprünglichen Größe. Um nun die Schrumpfung der drei Elemente zu berechnen, muss zuerst die Schrumpfrate ermittelt werden:

$$shrink-ratio = \frac{width \cdot flex-shrink}{shrink-scaled-width_{total}}$$

shrink-scaled-width_{total} ist die Summe aller Breiten multipliziert mit der jeweiligen `flex-shrink`-Angabe. Hier also $600 \cdot 2 + 240 \cdot 1 + 120 \cdot 1 = 1560$. Somit ergeben sich die folgenden Schrumpfraten:

- Abschnitt: $shrink-ratio = \frac{600 \cdot 2}{1560} = 0,7693$
- Navigation: $shrink-ratio = \frac{240 \cdot 1}{1560} = 0,1538$
- Seitenleiste: $shrink-ratio = \frac{120 \cdot 1}{1560} = 0,0769$

Somit ist berechnet, wie stark ein Element schrumpft. Nun kann seine neue Breite ausgerechnet werden:

$$width_{new} = width - (shrink-space \cdot shrink-ratio)$$

Mit den Schrumpfraten von oben, den ursprünglichen Breiten (*width*) von 600, 240 und 120 Pixeln sowie dem fehlenden Platz (*shrink-space*) von 240 Pixeln ergeben sich die folgenden neuen Breiten:

- Abschnitt: $width_{new} = 600 - (240 \cdot 0,7693) = 415,38$
- Navigation: $width_{new} = 240 - (240 \cdot 0,1538) = 203,08$
- Seitenleiste: $width_{new} = 120 - (240 \cdot 0,0769) = 101,54$

Beim Schrumpfen werden also die fehlenden 240 Pixel nicht im Verhältnis 2:1:1 aufgeteilt, vielmehr spielt die ursprüngliche Breite bzw. das ursprüngliche Verhältnis hier eine Rolle bei der Berechnung. Ebenfalls können die Werte abweichen, wenn sich in den Boxen Inhalt (Text) befindet, der das Schrumpfen etwa bei nicht möglichen Zeilenumbrüchen behindert.

5.1.3 Ausrichtung der Boxen bestimmen

Ist ein Container größer (d. h. breiter oder höher) als die Boxen darin (da diese etwa über fixe Angaben in ihrer Größe beschränkt sind) können die Boxen einfach angeordnet werden. Hier zeigt sich der große Vorteil der flexiblen Boxen gegenüber den bisherigen Modellen, da mit zwei Anweisungen die verschiedensten horizontalen und vertikalen Platzierungen erreicht werden können. Die Angaben für die Ausrichtung werden direkt beim Flex-Container gesetzt.

Mit **justify-content** kann die Ausrichtung der Boxen an der Hauptachse bestimmt werden. Diese ist durch **flex-flow** vorgegeben. Ist der Wert **row**, so ist die Hauptachse in der Horizontalen, ist der Wert **column** ist sie vertikal angeordnet. Es gibt daher auch nur die Angaben **flex-start** und **flex-end** (und nicht etwa **left**, **right** oder **top**, **bottom**). Ersteres richtet die Elemente am Beginn, zweiteres am Ende der Hauptachse aus. Ist die Hauptachse also horizontal, bedeutet **flex-start** linksbündig, ist sie vertikal bedeutet es oben ausgerichtet, **flex-end** bedeutet damit rechtsbündig bzw. unten ausgerichtet. Weiters existieren noch die Werte **center** (zentriert), **space-between** (äußerste Elemente am Rand, Platz dazwischen gleichmäßig verteilt), **space-around** (jedes Element hat links und rechts gleich viel Abstand) und **space-evenly** (der Abstand zwischen allen Elementen und dem Rand ist gleich).

align-items entspricht der Ausrichtung von Elementen auf der Querachse (die rechtwinklig zur Hauptachse steht). Hier gibt es die Werte **flex-start** (oben/links), **flex-end** (unten/rechts), **center** (zentriert), **stretch** (gestreckt über die gesamte Höhe) und **baseline** (Elemente sind an einer Grundlinie ausgerichtet).

Existieren auf der Querachse mehrere Reihen an Boxen, können diese mit der Eigenschaft **align-content** ausgerichtet werden. Die Werte gleichen denen für **align-items**,

bei `flex-start` etwas werden alle Reihen ganz oben angeordnet, bei `center` werden sie zentriert usw.

Mit der Eigenschaft `gap` kann ein gleichmäßiger Abstand zwischen den Boxen in beiden Richtungen erzielt werden, mit `row-gap` und `column-gap` je auf einer Achse.

Sollte die Ausrichtung eines einzelnen Elements anders als bei den übrigen im Container gewünscht sein, so kann mittels `align-self` diese Angabe direkt beim Element gemacht werden und die Werte des Containers überschreiben.

Im Beispiel ist nun auch das `<header>`-Element ein Flex-Container und richtet die darin enthaltene Überschrift mit einer Anweisung vertikal zentriert aus. Im Footer wird der Text rechts unten durch eine zweimalige Angabe von `flex-end` bei `align-items` und `justify-content` platziert.

```
1 header {
2   display: flex;
3   align-items: center;
4   background-color: rgb(250 94 94);
5 }
6 footer {
7   display: flex;
8   align-items: flex-end;
9   justify-content: flex-end;
10  background-color: rgb(67 67 67);
11 }
12 header, footer {
13   min-height: 6rem;
14 }
```

Selbstverständlich kann auch hier responsive gearbeitet werden. Die folgenden Media Queries erwirken einen fluiden (d. h. 100 Prozent breiten) Container unter 960 Pixeln und eine Untereinanderreihung der Spalten bei Displays, die schmaler als 640 Pixel sind. Dazu wird die Richtung der flexiblen Boxen von Reihe auf Spalte gewechselt. Zudem wird die ursprüngliche Ordnung der Elemente wiederhergestellt und `flex-basis` wird auf `auto` gestellt, denn ansonsten würden die Pixelangaben jetzt für die Höhe gelten, was nicht gewünscht wäre:

```
1 @media all and (max-width: 960px) {
2   div#container {
3     width: 100%;
4   }
5 }
6 @media all and (max-width: 640px) {
7   main {
8     flex-direction: column;
9   }
10  main > section, main > nav, main > aside {
11    order: 0;
12    flex-basis: auto;
13  }
14 }
```

Abbildung 5.1 zeigt dieses Layout mit 960er, 1200er, 720er und 600er Containerbreite.

Eine sehr gute Einführung zum Thema Flexible Box mit allen Begriffen und bildlichen Erklärungen bietet [12].



Abbildung 5.1: Bei 960 Pixeln ist die Spaltenbreite bei 240 für die Navigation, 600 für den Abschnitt und 120 für die Seitenleiste (a). Bei 1200 Pixeln wachsen die Spalten wie beschrieben auf 300, 720 und 180 Pixel (b). Bei 720 Pixeln schrumpfen sie auf 203,08, 415,38 und 101,54 Pixel (c) und unter 640 Pixeln werden sie untereinander angeordnet und die Reihenfolge wird geändert (d).

5.2 CSS Grid Layout

Elemente an einem Raster auszurichten ist ein üblicher Prozess in der Gestaltung, sowohl für den Druck- als auch den Webbereich. In der Vergangenheit existierten jedoch für den Webbereich nur unzulängliche Lösungen, dies zu erreichen. Layouts mittels `float`-Anweisungen funktionieren prinzipiell, haben jedoch das Problem, dass im normalen Darstellungsmodell Boxen nur so hoch sind, wie ihr Inhalt, was für mehrspaltige Layouts problematisch ist und Workarounds bedarf (die z. B. in verschiedensten CSS-Frameworks fast bis zur Perfektion getrieben wurden). Das Flexbox-Layout löst dieses Problem, allerdings ist es per se nur ein eindimensionales Layout, d. h. Elemente werden nur in einer Zeile oder Spalte angeordnet und brechen einfach um. Die Unzulänglichkeiten hierbei zeigen sich etwa, wenn sieben flexible Boxen so formatiert werden, dass sich drei in einer Zeile ausgehen. Es entsteht ein dreispaltiges Layout, wobei die ersten zwei Zeilen jeweils drei Boxen enthalten, die dritte Zeile jedoch nur eine, welche die gesamte Breite beansprucht. Auch dies ist wiederum nur mit Workaround zu lösen.

Das *CSS Grid Layout* [74] löst all diese Probleme, indem es ermöglicht, ein zweidimensionales Raster in CSS zu erstellen. Die grundsätzliche Vorgangsweise ist dabei ähnlich wie bei Flexbox: ein Element wird zum Container gemacht (hier der Grid-Container), dieser definiert die grundlegenden Eigenschaften (hier z. B. Reihen und Spalten) die darin enthaltenen Elemente werden darin angeordnet.

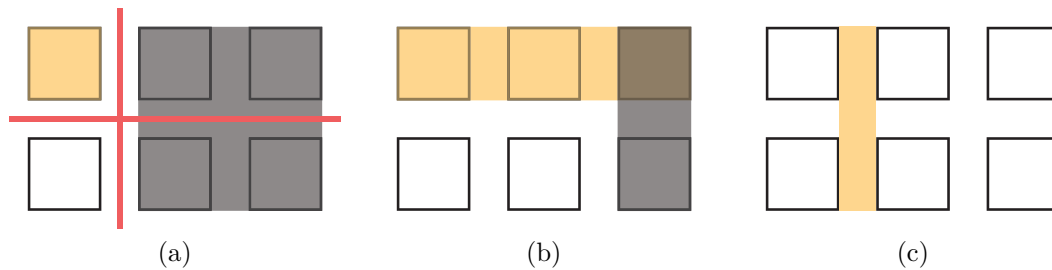


Abbildung 5.2: Terminologie im Grid. (a): Grid Cell (gelb), Grid Lines (rot), Grid Area (grau). (b): Grid Tracks: Grid Line (gelb) und Grid row (grau). (c): Gutter.

5.2.1 Grid Terminologie

Vor der Arbeit mit dem Grid Layout sollen einige Begriffe zu definiert werden, um das Verständnis zu erleichtern:

- **Grid Lines** (Rasterlinien): Die vertikalen und horizontalen Linien, die den Raster unterteilen und die Reihen und Spalten voneinander trennen (Abbildung 5.2 (a)).
- **Grid Cell** (Rasterzelle): Eine einzelne Einheit im CSS Grid (Abbildung 5.2 (a)).
- **Grid Area** (Rasterbereich): Ein rechteckiger Bereich, bestehend aus mehreren Rasterzellen, der von vier Rasterlinien begrenzt wird (Abbildung 5.2 (a)).
- **Grid Track** (Rasterspur): Der Bereich zwischen zwei Rasterlinien, entweder horizontal oder vertikal (Abbildung 5.2 (b)).
- **Grid Row** (Rasterzeile): Eine horizontale Rasterspur (Abbildung 5.2 (b)).
- **Grid Column** (Rasterspalte): Eine vertikale Rasterspur (Abbildung 5.2 (b)).
- **Gutter** (Steg): Der Zwischenraum zwischen Reihen und Spalten in einem Grid (Abbildung 5.2 (c)).
- **Grid Container:** Das Element, welches das gesamte Grid beinhaltet.
- **Grid Item:** Jedes direkte Kindelement eines Grid Containers.

5.2.2 Einen Grid-Container definieren

Zunächst wird ein Element bestimmt, das als Grid-Container agiert. Es beinhaltet das gesamte Grid und wird mit der Eigenschaft `display: grid` als solches festgelegt. Alternativ kann mit `display: inline-grid` auch ein Grid auf Textebene definiert werden.

Aufteilung des Grids

Für dieses Element wird nun die grundsätzliche Aufteilung des Rasters definiert. Mit den Eigenschaften `grid-template-columns` und `grid-template-rows` werden die Spalten und Zeilen definiert. Beide Eigenschaften akzeptieren durch Leerzeichen getrennte Angaben. `grid-template-columns: 150px 150px 150px` etwa erzeugt drei Spalten mit jeweils 150 Pixeln Breite.

Mit dem Schlüsselwort `auto` kann der verbleibende Platz zwischen fixen Angaben automatisch bzw. flexibel vergeben werden, so erzeugt z. B. `grid-template-rows: 200px`

`auto 200px` die erste und dritte Reihe mit 200 Pixeln Höhe während die zweite den restlichen, im Container verbleibenden Platz einnimmt.

Soll der freie, verbleibende Platz in einem Verhältnis aufgeteilt werden, kann die extra für das Grid eingeführte Einheit `fr` (für Fraction = Anteil) verwendet werden. `grid-template-columns: 1fr 1fr 1fr` erzeugt drei Spalten, die sich jeweils zu gleichen Anteilen den Platz im Container aufteilen.

Um unnötiges Wiederholen derartiger Angaben zu vermeiden, kann die `repeat()`-Syntax angewandt werden. Das Beispiel mit den drei anteilsmäßig gleich verteilten Spalten lässt sich auch folgendermaßen schreiben: `grid-template-columns: repeat(3, 1fr)`.

Auto-Placement von Elementen im Grid und Subgrid

Sobald ein Grid definiert (aufgeteilt) ist, werden die direkten Kindelemente des Grid-Containers darin platziert. Dieses Verhalten nennt sich *Auto-Placement* und ordnet pro Grid Cell ein Element an. Werden nur Spalten mit `grid-template-columns` definiert, werden dabei so viele Zeilen erzeugt, wie nötig.

Im folgenden Beispiel wird ein `<div>`-Element zum Grid-Container mit zwei Spalten. Die darin verschachtelten `<div>`-Elemente werden durch Auto-Placement in die Grid-Zellen gelegt.

```
1 <p>Der folgende Container nimmt standardmäßig die volle verfügbare Breite ein:</p>
2
3 <div class="container block-grid">
4   <div class="item">A</div>
5   <div class="item">B</div>
6 </div>
7
8 <p>Dadurch wird der Textfluss vor und nach dem Grid unterbrochen.</p>
```

Das Grid wird nur durch diesen Code erzeugt, in der Klasse `.item` werden nur optische Formatierungen gemacht, keine explizite Platzierung (daher Auto-Placement). Das Ergebnis ist unter Punkt 1 in Abbildung 5.3 zu sehen.

```
1 .block-grid {
2   display: grid;
3   grid-template-columns: 1fr 1fr; /* Zwei gleich große Spalten */
4   gap: 0.625rem;
5 }
6 .item {
7   background-color: rgb(255 204 0);
8   padding: 0.625rem;
9   text-align: center;
10  font-weight: bold;
11 }
```

Ebenso kann ein Grid in einem Inline-Element erstellt werden. Dafür wird die Eigenschaft `display: inline-grid` gesetzt. Der Rest funktioniert identisch. Das folgende Grid wird in einem `<span>`-Element angelegt, funktioniert ansonsten aber gleich wie ein Block-Grid. Abbildung 5.3 zeigt das Inline-Grid unter Punkt 2.

```
1 <p>
2   Hier ist ein Fließtext, und mittendrin befindet sich
3   <span class="container inline-grid">
```

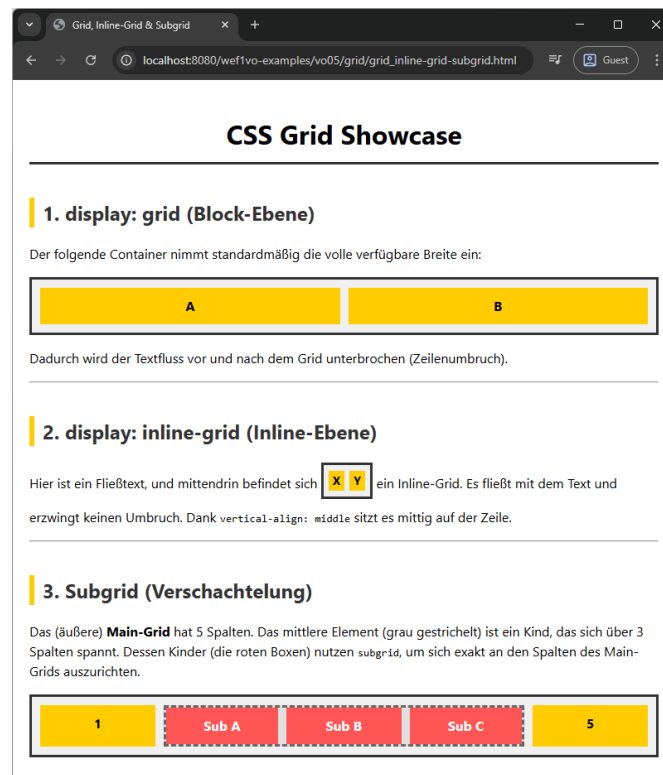


Abbildung 5.3: Teil 1 zeigt ein Grid mit `display: grid` als Block-Element. Teil 2 zeigt ein Inline-Grid mit `display: inline-grid` und in Teil 3 wird ein Subgrid mit `display: grid` und `grid-template-columns: subgrid` dargestellt.

```

4     <span class="item">X</span>
5     <span class="item">Y</span>
6   </span>
7   ein Inline-Grid. Es fließt mit dem Text und erzwingt keinen Umbruch.
8 </p>

```

Beim Inline-Grid macht die Spaltenbreite mit dem Wert `auto` Sinn, damit sie sich dem Inhalt anpassen. Die Grid-Items werden auch hier automatisch platziert.

```

1 .inline-grid {
2   display: inline-grid;
3   grid-template-columns: auto auto; /* Spalten passen sich Inhalt an */
4   gap: 0.4rem;
5   vertical-align: middle; /* Zentriert das Grid vertikal zur Textzeile */
6   padding: 0.4rem;
7 }
8 .inline-grid .item {
9   padding: 0.1rem 0.3rem;
10  font-size: 0.9em;
11 }

```

Schließlich ist es mit dem CSS Grid Layout Module Level 2 [75] auch möglich, Subgrids zu definieren. War es zunächst nur vorgesehen, die direkten Kindelemente des Grid-Containers im Grid anzuordnen, kann das Grid nun auch an weiter „unten“ liegen-

de Elemente vererbt werden. So kann mit `display: grid` ein weiteres Grid innerhalb eines bestehenden erzeugt werden und diese durch `grid-template-columns: subgrid` angewiesen werden, die Spaltenaufteilung des äußeren Grids zu übernehmen.

Das folgende Beispiel legt ein äußeres Grid mit 5 Spalten an. Das 1. und 5. Element werden direkt durch dieses Grid angeordnet. Das zweite Element erstreckt sich über drei Spalten, ist selbst wieder ein Grid übernimmt die Einteilung des äußeren Grids. Das Ergebnis ist in Teil 3 von Abbildung 5.3 zu sehen.

```
1 <div class="container main-grid">
2   <div class="item">1</div>
3   <div class="subgrid-container">
4     <div class="sub-item">Sub A</div>
5     <div class="sub-item">Sub B</div>
6     <div class="sub-item">Sub C</div>
7   </div>
8   <div class="item">5</div>
9 </div>
```

Die Klasse `.sub-item` dient hier auch nur zur Formatierung. Die Elemente werden im Grid autoplatziert.

```
1 .main-grid {
2   display: grid;
3   grid-template-columns: repeat(5, 1fr);
4   gap: 0.625rem;
5 }
6 .subgrid-container {
7   grid-column: span 3;
8   display: grid;
9   grid-template-columns: subgrid;
10  background-color: rgb(224 224 224);
11  border: 0.2rem dashed rgb(102 102 102);
12  padding: 0;
13 }
14 .sub-item {
15   background-color: rgb(255 85 85);
16   color: white;
17   padding: 0.625rem;
18   text-align: center;
19   font-weight: bold;
20 }
```

Benennung von Rasterlinien und Bereichen

Für einfache Anordnungen im Grid reicht Auto-Placement. Oft sollen jedoch Elemente mehrere Grid-Cells, d. h. größere Bereiche einnehmen. Um nun festzulegen, wie viele Zellen ein Element später einnimmt (siehe Abschnitt 5.2.3), wird angegeben, zwischen welchen Rasterlinien es liegen soll. Diese werden von links nach rechts bzw. von oben nach unten einfach durchnummeriert. Um hier nicht auf die Zahlen angewiesen zu sein, können bei der Definition des Rasters auch Namen vergeben werden. Diese werden in eckige Klammern vor und nach die jeweiligen Spalten- und Zeilenangaben gestellt, mit Leerzeichen können für eine Linie auch mehrere Namen vergeben werden. `grid-template-columns: [col1-start] 1fr [col1-end col2-start] 1fr`

[col2-end] definiert etwa für zwei Spalten die drei Rasterlinien als col1-start (1. Linie), col1-end und col2-start (2. Linie) und col2-end (3. Linie).

Alternativ können Elemente über benannte Bereiche anstatt zwischen Grid Lines platziert werden. Um solche Grid Areas zu definieren, sind spezielle Angaben über **grid-template-areas** möglich. Diese Eigenschaft akzeptiert mehrere, durch Leerzeichen getrennte Zeichenketten. Eine Zeichenkette (Text zwischen zwei Anführungszeichen) steht für den Inhalt einer Zeile, innerhalb der Zeichenketten werden die Namen der Rasterbereiche angegeben, wie sich diese über die Spalten hinweg aufteilen. **grid-template-areas: "header header" "nav main" "footer footer"** etwa setzt den Bereich **header** in der gesamten ersten Zeile (zwei Spalten) fest, die zweite Zeile wird in **nav** und **main** geteilt, die dritte Zeile ist als Bereich **footer** (wieder über zwei Spalten) gekennzeichnet.

Um den Steg (Gutter), also den Abstand zwischen den Rasterzellen zu definieren, kommt (wie bei Flexbox) die Eigenschaft **gap** zum Einsatz. **gap: 10px** erzeugt einen Abstand von 10 Pixeln zwischen Zeilen und Spalten, **gap: 10px 15px** erzeugt 10 Pixel Abstand zwischen den Zeilen und 15 Pixel Abstand zwischen den Spalten. Dies kann auch jeweils einzeln über die Angaben **row-gap** und **column-gap** definiert werden.

Ausrichtung von Inhalten

Mit den Angaben **justify-items** und **align-items** können über die Werte **start**, **end**, **center** und **stretch** die Zelleninhalte sowohl horizontal als auch vertikal ausgerichtet werden. Mit **justify-self** und **align-self** kann dies für einzelne Zellen überschrieben werden. **justify-content** und **align-content** besitzen dieselben Werte, richten aber nicht den Inhalt der Zellen sondern das Grid innerhalb des Containers aus, falls dieser größer als der Inhalt sein sollte.

Als Beispiel soll ein einfaches, zweispaltiges Layout dienen. Als Container wird ein `<div>`-Element verwendet, darin befinden sich die üblichen Elemente für Header, Navigation, Hauptbereich und Footer.

```

1 <body>
2   <div id="container">
3     <header>
4       <h1>Überschrift</h1>
5     </header>
6     <nav>
7       <ul>
8         <li>Navigation 1</li>
9         <li>Navigation 2</li>
10      </ul>
11    </nav>
12    <main>Lorem ipsum...</main>
13    <footer>&copy; 2025 Der*Die Autor*in</footer>
14  </div>
15 </body>

```

Zunächst werden die Eigenschaften für den Container gesetzt. Definiert werden zwei Spalten und drei Zeilen, wobei die erste Spalte ein Viertel des Platzes (**1fr**) und die zweite Spalte dreiviertel (**3fr**) erhält. Die erste und dritte Zeile (Header und Footer) sind mit einer fixen Höhenangabe gesetzt, der verbleibende Platz wird mittels **1fr**-Angabe flexibel gehandhabt.

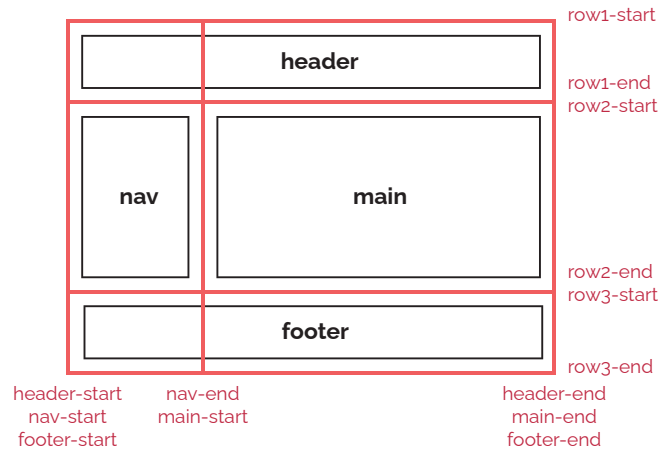


Abbildung 5.4: Das Layout mit seinen Rasterlinien (rot) und den benannten Grid-Bereichen.

In den Angaben für die Spalten und Reihen sind auch die Rasterlinien benannt, zusätzlich werden Template Areas („header“, „nav“, „main“ und „footer“) definiert. Ebenso wird ein Steg definiert.

```

1 div#container {
2   display: grid;
3   max-width: 75rem;
4   margin: 0 auto;
5   grid-template-columns:
6     [header-start nav-start footer-start] 1fr
7     [nav-end main-start] 3fr
8     [header-end main-end footer-end];
9   grid-template-rows:
10    [row1-start] 6rem
11    [row1-end row2-start] 1fr
12    [row2-end row3-start] 6rem
13    [row3-end];
14   grid-template-areas: "header header"
15                        "nav main"
16                        "footer footer";
17   gap: 0.5rem;
18 }

```

Abbildung 5.4 stellt dieses Layout mit seinen Bezeichnungen schematisch dar.

5.2.3 Elemente im Raster anordnen

Nachdem im Container nun das Layout des Grids (das Template-Grid) mit seinen Abmaßen und auch Bezeichnungen (Grid Lines oder alternativ Grid Areas) festgelegt wurden, müssen nun konkrete HTML-Elemente den einzelnen Bereichen zugewiesen werden.

Dies kann auf zwei Arten geschehen: entweder werden mittels **grid-column** und **grid-row** die Start- und End-Rasterlinien für Spalten und Zeilen (entweder über Nummern oder die vergebenen Namen) definiert oder mittels **grid-area** der vordefinierte Bereich im Raster angegeben.

Im Beispiel werden beide Arten verwendet, um dies zu demonstrieren. Im Produktionseinsatz sollte zwischen einer der beiden Methoden entschieden werden, denn beide bewirken dasselbe und sind redundant.

```
1 header {
2   grid-column: header-start / header-end;
3   grid-row: row1-start / row1-end;
4   grid-area: header;
5   background-color: rgb(250 94 94);
6 }
7 nav {
8   grid-column: nav-start / nav-end;
9   grid-row: row2-start / row2-end;
10  grid-area: nav;
11  background-color: rgb(255 132 0);
12 }
13 main {
14  grid-column: main-start / main-end;
15  grid-row: row2-start / row2-end;
16  grid-area: main;
17  color: rgb(67 67 67);
18  background-color: rgb(255 182 0);
19 }
20 footer {
21  grid-column: footer-start / footer-end;
22  grid-row: row3-start / row3-end;
23  grid-area: footer;
24  background-color: rgb(67 67 67);
25 }
```

Auch beim Grid Layout können Media Queries eingesetzt werden, um auf unterschiedliche Ausgabegeräte zu reagieren. Im Beispiel werden unter 640 Pixeln Breite etwa Navigation und Hauptbereich untereinander angeordnet. Dazu werden im Container vier Reihen definiert und die Angabe der Start- und End-Rasterlinien für Navigation, Hauptbereich und Footer entsprechend angepasst.

```
1 @media all and (max-width: 640px) {
2   div#container {
3     grid-template-rows:
4       [row1-start] 6rem
5       [row1-end row2-start] auto
6       [row2-end row3-start] auto
7       [row3-end row4-start] 6rem
8       [row4-end];
9   }
10  nav {
11    grid-column: nav-start / header-end;
12  }
13  main {
14    grid-column: header-start / main-end;
15    grid-row: row3-start / row3-end;
16  }
17  footer {
18    grid-row: row4-start / row4-end;
19  }
20 }
```

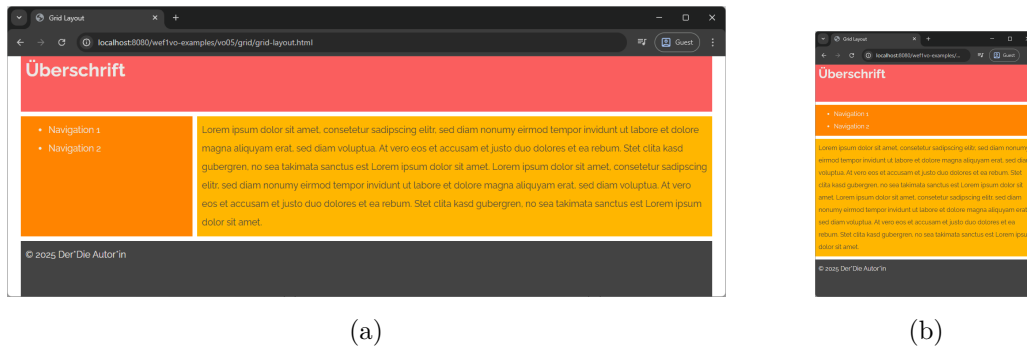


Abbildung 5.5: Ist das Browserfenster breiter als 640 Pixel, kommt die normale Formattierung zum Einsatz und die drei Spalten werden nebeneinander angeordnet (a). Wird die Breite auf unter 640 Pixel reduziert, werden die Spalten untereinander angeordnet (b).

Würde auf die Neudefinition von `grid-template-rows` verzichtet, so würde dennoch eine vierte Reihe generiert werden. Diese sogenannten *impliziten Grid Tracks* können über die Angaben `grid-auto-columns` und `grid-auto-rows` gesteuert werden. `grid-auto-rows: 100px` würde etwa jede implizit generierte Grid-Reihe mit einer Höhe von 100 Pixeln erzeugen.

Abbildung 5.5 zeigt das erstellte Layout in voller und reduzierter Breite.

Eine sehr gute Einführung zum Thema Grid Layout mit allen Begriffen und bildlichen Erklärungen bietet [26]. Ein interaktives Tutorial mit Erklärung praktischer, für das Grid ausgelegter Tools, bietet [38].

5.3 CSS-Frameworks: Bootstrap

Bevor 2014 mit dem Flexible Box Layout (Abschnitt 5.1) und 2017 dem Grid Layout (Abschnitt 5.2) zwei offizielle, von der W3C CSS Working Group entwickelte Standards zur Verfügung standen, um Layouts zu erstellen, existierten schon länger diverse Layout-Frameworks, auch einfach *CSS-Frameworks* genannt. Diese Frameworks arbeiten mit den in CSS verfügbaren Mitteln und bieten vordefinierte Stylesheets, mit deren Hilfe ohne großen Aufwand ein standardkonformes Layout erzeugt werden kann.

Die Anordnung der einzelnen Bereiche (z. B. Header, Footer, Navigation und Inhalt) erfolgt bei fast allen dieser Frameworks mit Hilfe eines Grids. Dieser Raster besteht aus einer Reihe von Spalten, die über die Gesamtbreite eines Containers aufgeteilt sind. Meist kommen dabei 12 Spalten zum Einsatz, aber auch 16 oder 24 sind möglich. Abbildung 5.6 zeigt schematisch einen solchen 12-spaltigen Raster am Beispiel von *Bootstrap*, welches im folgenden Abschnitt näher erklärt wird.

5.3.1 Bootstrap allgemein

Das Framework *Bootstrap*¹, aktuell in Version 5.3.8 verfügbar, wurde 2011 von Twitter (mittlerweile dem Bootstrap Team) entwickelt und gehört zu den populärsten CSS-

¹<https://getbootstrap.com/>

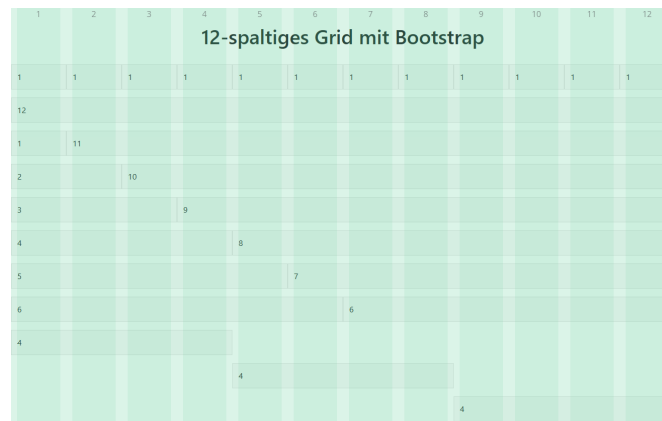


Abbildung 5.6: Boxen, angeordnet in einem 12-spaltigen Grid-System. Das farbige Overlay wird mit Hilfe einer Browser-Extension erzeugt und dient nur zur Visualisierung der Spalten.

Frameworks. Es arbeitet mit einem 12-spaltigen Rastersystem, ist responsive und verfolgt dabei einen mobile-first Ansatz. Seit Version 4 des Frameworks werden die Elemente am Raster mit Hilfe des Flexible Box Layout Moduls (Flexbox) angeordnet, in Version 3 kamen dafür noch Floats zum Einsatz. Seit Version 5.1.0 kann experimentell auch ein Modus aktiviert werden, der das CSS Grid verwendet.

Neben dem Grid-System bietet das Framework jedoch auch konsistente Lösungen für alle anderen Bereiche einer Webseite. So können Bilder oder Tabellen responsive gestaltet werden, Formularfelder ausgerichtet und gestyled werden oder User Interface Elemente wie Karusselle, Akkordeons oder Tab-Leisten erstellt werden. Für viele dieser Funktionalitäten werden dabei jedoch JavaScript-Plugins benötigt, die in Bootstrap enthalten sind. Eine Übersicht, welche Funktionalitäten Bootstrap in Detail bietet, findet sich in der offiziellen Dokumentation [47].

5.3.2 Bootstrap einbinden

Um mit Bootstrap zu arbeiten, müssen zunächst die CSS-Dateien, die das Framework zur Verfügung stellt, heruntergeladen² werden. Dies sind die so genannten kompilierten CSS- und JS- (JavaScript) Dateien, die so direkt verwendet werden können. In der Zip-Datei befinden sich die Ordner `css` und `js`, die ins eigene Projekt entpackt werden. Werden keine JavaScript-Funktionalitäten benötigt (weil etwa nur das Grid-System verwendet wird), kann auf den Ordner `js` auch verzichtet werden.

Nun wird die Bootstrap-Hauptdatei als Stylesheet in der eigenen HTML-Datei im `<head>` vor eventuellen eigenen CSS-Dateien eingebunden:

```
1 <link rel="stylesheet" href="css/bootstrap.css">
```

Um im Produktivbetrieb weniger Overhead zu haben, existiert auch eine minifizierte Version, die statt ca. 273 KB nur 226 KB groß ist. Sie wird so eingebunden:

```
1 <link rel="stylesheet" href="css/bootstrap.min.css">
```

²Für Version 5.3.x unter <https://getbootstrap.com/docs/5.3/getting-started/download/>

Möchte man nur das Bootstrap-Grid-System verwenden, aber keine von Bootstrap mitgebrachten anderen Formatierungen einsetzen, kann `bootstrap-grid.css` (bzw. `bootstrap-grid.min.css`) verwendet werden.

```
1 <link rel="stylesheet" href="css/bootstrap-grid.css">
```

Wer die Bootstrap-Dateien nicht selbst im Projekt integrieren möchte, kann auf ein so genanntes Content Delivery Network (kurz CDN) zurückgreifen. Hier findet sich die aktuellste Version des Frameworks online. Der Vorteil hierbei liegt darin, dass dieses CDN von vielen Seiten verwendet wird. Wurde im eigenen Browser zuvor bereits eine Bootstrap-Seite besucht, hat der Browser Bootstrap bereits im Cache und die Dateien müssen nicht erneut geladen werden. Der Nachteil ist, dass nicht offline entwickelt werden kann, da die Dateien auf einem Server liegen und eine Internetverbindung bestehen muss. Um Bootstrap 5.3.8 einzubinden, wird folgendes Markup benötigt (die aktuellste Version findet sich in der Dokumentation [47]).

```
1 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
  rel="stylesheet" integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIA
  vNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
```

Die Angaben für `integrity` und `crossorigin` ermöglichen das so genannte Sub-resource Integrity Feature und stellen sicher, dass der Code im CDN nicht böswillig verändert wurde.

Schließlich können auch die Quelldateien von Bootstrap heruntergeladen und selbst kompiliert werden. Diese liegen in der Stylesheet-Sprache Sass vor und benötigen eine Reihe von Tools wie einen Compiler und einen Autoprefixer. Dies bietet die größte Flexibilität, setzt jedoch erweitertes Wissen voraus. Vorlesung 11 setzt sich mit diesen Workflows auseinander und gibt einen ersten Einblick.

5.3.3 Das Bootstrap Grid verwenden

Um das 12-spaltige Grid-System von Bootstrap zu verwenden, wird zunächst ein Container-Element benötigt, das dieses Grid beinhaltet. In diesem Grid werden dann Zeilen definiert, in denen dann die gewünschte Anzahl Spalten definiert werden. All dies geschieht, indem von Bootstrap vorgegebene CSS-Klassen zu HTML-Elementen hinzugefügt werden.

Bootstrap Container

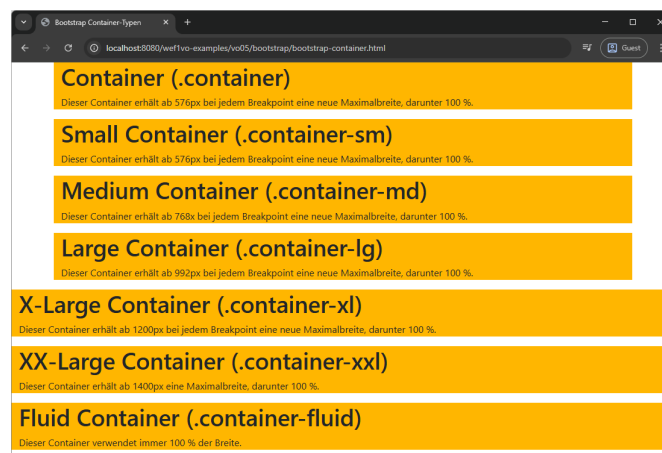
Bootstrap bietet drei unterschiedliche Container-Klassen an.

- `.container`: Setzt eine Maximalbreite (`max-width`) bei jedem responsive Breakpoint.
- `.container-fluid`: Definiert `width: 100%` bei allen Breakpoints.
- `.container-{breakpoint}`: Setzt `width: 100%` bis zum angegebenen Breakpoint und eine Maximalbreite ab diesem und bei allen weiteren.

Bootstrap definiert dabei 6 *Breakpoints*, diese sind X-Small (XS), Small (SM), Medium (MD), Large (LG), X-Large (XL) und XX-Large (XXL). Tabelle 5.1 gibt eine Übersicht über die Breakpoints und die verwendeten Breiten.

Tabelle 5.1: Bootstrap Breakpoints und die dadurch entstehenden Containerbreiten.

Klasse Breakpoint	X-Small < 576px	Small ≥ 576px	Medium ≥ 768px	Large ≥ 992px	X-Large ≥ 1200px	XX-Large ≥ 1400px
<code>.container</code>	100%	540px	720px	960px	1140px	1320px
<code>.container-sm</code>	100%	540px	720px	960px	1140px	1320px
<code>.container-md</code>	100%	100%	720px	960px	1140px	1320px
<code>.container-lg</code>	100%	100%	100%	960px	1140px	1320px
<code>.container-xl</code>	100%	100%	100%	100%	1140px	1320px
<code>.container-xxl</code>	100%	100%	100%	100%	100%	1320px
<code>.container-fluid</code>	100%	100%	100%	100%	100%	100%

**Abbildung 5.7:** Die Bootstrap Container-Typen bei einer Breite von 1.100 Pixel. Die ersten vier haben eine Maximalbreite von 960 Pixel, die letzten drei nehmen 100 % der Breite ein.

Wird etwa die Klasse `.container` verwendet, nimmt der Container 100 % der Viewport breite bis 575 Pixel ein. Zwischen 576 und 767 Pixel ist der Container dann 540 Pixel breit, zwischen 768 und 991 Pixel ist er 720 Pixel breit. Zwischen 992 und 1.199 Pixel ist die Breite 960 Pixel, zwischen 1.200 und 1.399 Pixel ist er 1.140 Pixel breit und ab 1.400 Pixel ist bleibt der Container bei einer Breite von 1.320 Pixel.

Abbildung 5.7 zeigt alle Containertypen bei einer Viewport-Breite von 1.100 Pixel. Die Klassen `.container`, `.container-sm`, `.container-md` und `.container-lg` sind 960 Pixel breit, die Klassen `.container-xl`, `.container-xxl` und `.container-fluid` nehmen 100 % der Breite ein.

Bootstrap Spalten am Grid erstellen

In diesen Containern können nun Spalten im Zwölferraster eingefügt werden. Spalten müssen immer in einer Zeile mit der Klasse `.row` eingefügt werden.

Darin werden Klassen, die mit `.col-` beginnen, eingefügt. `.col-1` ist eine Spalte, die sich über eine Spalte des Rasters erstreckt, `.col-6` über 6 Spalten und `.col-12` belegt alle 12 Spalten, nimmt also die gesamte Breite ein. In einer Reihe sollten Spaltenzahlen

Tabelle 5.2: Bootstrap Breakpoints für Spaltenangaben.

Feature Breakpoint	X-Small < 576px	Small ≥ 576px	Medium ≥ 768px	Large ≥ 992px	X-Large ≥ 1200px	XX-Large ≥ 1400px
Containerbreite Klassenpräfix	Auto .col-	540px .col-sm-	720px .col-md-	960px .col-lg-	1140px .col-xl-	1320px .col-xxl-

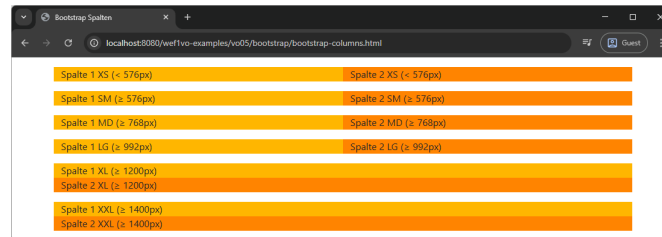


Abbildung 5.8: Die Bootstrap Spalten-Typen bei einer Breite von 1.100 Pixel. Die ersten vier sind nebeneinander angeordnet, weil die notwendige Minimalbreite erreicht ist, die letzten beiden sind untereinander mit voller Breite angeordnet, weil die Breakpoints noch nicht erreicht sind.

möglichst auf 12 addieren. Der folgende Code zeigt eine Zeile mit zwei gleich verteilten Spalten:

```

1 <div class="container">
2   <div class="row">
3     <div class="col-6">Spalte 1</div>
4     <div class="col-6">Spalte 2</div>
5   </div>
6 </div>

```

Weiters gibt es auch hier Spaltenklassen, die auf die schon beim Container gezeigten Breakpoints reagieren. Ab diesem Breakpoint nehmen die Spalten die angegebene Breite ein, darunter 100 %. Tabelle 5.2 zeigt die Breakpoints der Spalten.

Werden Spalten mit `.col-` angelegt, so bleiben diese auch beim kleinsten Container immer nebeneinander. Spalten mit `.col-sm-` stehen ab 576 Pixel nebeneinander, Spalten mit `.col-md-` stehen ab 768 Pixel nebeneinander, Spalten mit `.col-lg-` ab 992 Pixel, Spalten mit `.col-xl-` ab 1.200 Pixel und Spalten mit `.col-xxl-` ab 1.400 Pixel.

Abbildung 5.8 zeigt alle Spaltentypen in einer 2-spaltigen Anordnung bei einer Viewportbreite von 1.100 Pixel. Die Klassen `.col-6`, `.col-sm-6`, `.col-md-6` und `.col-lg-6` werden nebeneinander angeordnet, `.col-xl-6` und `.col-xxl-6` untereinander, weil die Breakpoints von 1.200 und 1.400 Pixel bei 1.100 Pixel noch nicht erreicht sind.

Diese verschiedenen Spaltenklassen können und sollen auch miteinander kombiniert werden. So können responsive Layouts einfach erreicht werden. Der folgende Code zeigt ein Layout mit drei Spalten ab dem Large-Breakpoint von 992 Pixel. Darunter – zwischen 991 und 768 Pixel – greifen Medium-Spalten. D.h. hier entsteht ein Layout in dem die ersten beiden Spalten nebeneinander und die dritte darunter liegt. Unter 767 Pixel liegen alle drei Spalten untereinander.

```

1 <div class="container">

```

```

2 <div class="row">
3   <div class="col-md-4 col-lg-3">Spalte 1</div>
4   <div class="col-md-8 col-lg-6">Spalte 2</div>
5   <div class="col-md-12 col-lg-3">Spalte 3</div>
6 </div>
7 </div>

```

In diesem Fall addieren die Angaben für Medium und Large immer auf 12. Es ist jedoch auch möglich, in einer Zeile wesentlich mehr Spalten unterzubringen, um noch flexibler agieren zu können.

```

1 <div class="container">
2   <div class="row">
3     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 1</div>
4     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 2</div>
5     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 3</div>
6     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 4</div>
7     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 5</div>
8     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 6</div>
9     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 7</div>
10    <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 8</div>
11    <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 9</div>
12    <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 10</div>
13    <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 11</div>
14    <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 12</div>
15  </div>
16 </div>

```

In diesem Beispiel werden ab 1.400 Pixel 4 Spalten erzeugt, zwischen 1.200 und 1.399 Pixel 3 Spalten, zwischen 768 und 1.199 Pixel 2 Spalten und alles darunter wird in einer Spalte angeordnet.

Zweispaltiges Layout mit Bootstrap

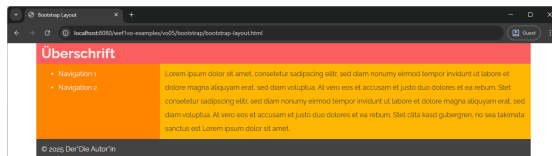
Das zweispaltige Layout, das bereits in Abschnitt 5.2 erstellt wurde, soll nun auch mit Bootstrap realisiert werden. Das HTML-Markup sieht wie folgt aus:

```

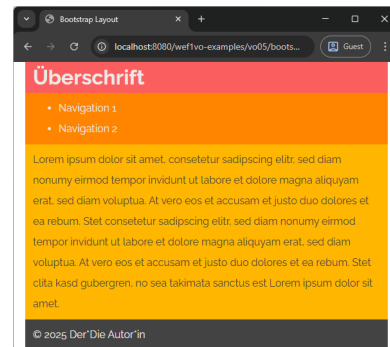
1 <body>
2   <div class="container">
3     <div class="row">
4       <header class="col-md-12">
5         <h1>Überschrift</h1>
6       </header>
7       <nav class="col-md-3">
8         <ul>
9           <li>Navigation 1</li>
10          <li>Navigation 2</li>
11        </ul>
12      </nav>
13      <main class="col-md-9">Lorem ipsum...</main>
14      <footer class="col-md-12">&copy; 2025 Der*Die Autor*in</footer>
15    </div>
16  </div>
17 </body>

```

Verwendet werden die klassischen Strukturelemente, zusätzlich kommt noch ein `<div>` als Zeile ins Spiel. Die Spaltenklassen werden bei den Elementen hinzugefügt.



(a)



(b)

Abbildung 5.9: Ist das Browserfenster breiter als 768 Pixel, kommt die normale Formattierung zum Einsatz und die drei Spalten werden nebeneinander angeordnet (a). Wird die Breite auf unter 768 Pixel reduziert, werden die Spalten untereinander angeordnet (b).

In diesem Fall wird auf die Medium-Klassen zurückgegriffen, somit werden Navigation und Hauptbereich ab 768 Pixel nebeneinander und darunter untereinander angeordnet. Abbildung 5.9 stellt beide Fälle dar.

5.3.4 Zusätzliche Bootstrap Funktionalitäten

Neben den soeben erwähnten, grundsätzlichen Grid-Funktionalitäten bietet Bootstrap noch einige weitere Dinge, die eingesetzt werden können. Von *Gutter-Klassen*, die im Grid verwendet werden können, um Abstände zwischen den Spalten (Gutter) zu regeln (`.gx-` und `.gy-`) bis hin zu *Utility-Klassen* um Margin und Padding anzupassen (`.m-` und `.p-`), Hintergründe zu setzen oder schnell einen Rahmen um ein Element zu erstellen.

Die Bootstrap-Dokumentation [47] listet all diese und auch noch darüber hinausgehende Funktionalitäten auf.

5.3.5 Weitere CSS-Frameworks

Neben Bootstrap existieren noch eine Vielzahl anderer Frameworks. Aktuelle Vertreter sind responsive, auch Konzepte wie *mobile-first* werden von diesen Frameworks abgedeckt. Der Vollständigkeit halber ist auch das erste bekannte Grid-Framework aufgelistet.

960.gs und Unsemantic

Das *960 Grid System*³ (kurz 960.gs) ist quasi der Urvater aller Grid-basierten Layout-Frameworks. Entstanden 2009 bietet es die Möglichkeit, ein Layout in 12, 16 oder auch 24 Spalten zu gestalten. Das Framework ist statisch, d. h. nicht responsive und liegt auch in einfachem CSS vor. JavaScript wird nicht benötigt. Dies macht es zum idea-

³<https://960.gs/>

len Einsteiger*innen-Framework, um sich mit der Arbeitsweise eines Grid-Frameworks vertraut zu machen.

Der Nachfolger des 960.gs ist *Unsemantic*⁴, ein ebenfalls sehr einfaches, aber bereits responsive ausgelegtes Layout-Framework. Es versteht sich als sogenanntes „Fluid Grid System“ und definiert keine fixe Anzahl an Spalten. Vielmehr können Spalten in Prozentwerten angegeben werden, dabei enthält das Framework Klassen in 5-Prozent-Abständen, also für 5, 10, 15 etc. Prozent. Darüber hinaus gibt es noch Klassen für 33 und 66 Prozent, um die Seite dritteln zu können. Die Gesamtsumme der Spalten muss immer 100 Prozent ergeben. Auch hier liegt der Fokus auf Einfachheit. Unsemantic stellt die grundlegenden Layoutfunktionalitäten zur Verfügung, der Rest kann selbst angepasst werden.

Beide Frameworks verwenden Floats um Elemente nebeneinander in Spalten anzuordnen. Sie sind aufgrund ihres Alters jedoch kaum mehr im Einsatz.

Foundation

Das CSS-Framework *Foundation*⁵ existiert wie Bootstrap seit dem Jahr 2011 und basiert ebenfalls auf einem 12er-Grid. Das Framework ist ebenso responsive, als Stylesheet-Sprache kommt Sass zum Einsatz. Flexbox ist für die Grid-Anordnung in Version 6 verantwortlich. Ebenso wird auf JavaScript für zusätzliche Funktionalitäten zurückgegriffen.

Tailwind CSS

*Tailwind CSS*⁶ geht einen etwas anderen Weg als Bootstrap und Foundation. Es gehört zur Kategorie der sogenannten Utility-Frameworks. Es stellt eine Reihe von Utility-Klassen zur Verfügung, mit dem Layouts völlig modular zusammengebaut werden können. Oft verwendete Utilities können zu Komponenten zusammengefasst werden. Tailwind funktioniert mit jeder Stylesheet-Sprache, das Konzept ist jedoch, wenig CSS selbst zu schreiben, sondern die vorgegebenen Klassen in HTML anzuwenden. Zusätzliche Konfiguration erfolgt in JavaScript. Dieses ist auch essentiell zur Installation.

5.4 CSS Custom Properties

Bei der Arbeit mit Layouts, aber prinzipiell, wann immer verschiedene Werte in Stylesheets öfters verwendet werden, stellt sich die Frage der Codewiederholung. Etwa wenn bestimmte Abstände oder Farbwerte an mehreren Stellen benötigt werden, ergibt sich schnell eine Änderungsfeindlichkeit, wenn diese Werte angepasst werden müssen. Dies muss dann unweigerlich an mehreren Stellen geschehen. *CSS Custom Properties*, auch gerne CSS Variablen genannt, schaffen hier Abhilfe.

⁴<https://unsemantic.com/>

⁵<https://get.foundation/>

⁶<https://tailwindcss.com/>

5.4.1 Anlegen von Custom Properties

Ein Custom Property muss immer innerhalb eines Selektors angelegt werden und beginnt mit zwei Bindestrichen (--).

```
1 body {  
2   --bg-color: rgb(255 182 0);  
3 }
```

Hier wird der Farbwert `rgb(255 182 0)` im Custom Property `--bg-color` definiert. Es gilt im gesamten `<body>` Element und wird auch in alle darin enthaltenen Elemente weitervererbt. Sollen also Custom Properties überall im CSS gültig sein, empfiehlt es sich, diese möglichst weit oben in der HTML-Hierarchie zu definieren. Hierfür existiert auch die Pseudoklasse `:root`, die mit dem HTML-Element gleichzusetzen ist und oft verwendet wird, um CSS Custom Properties „ganz oben“ zu definieren.

```
1 :root {  
2   --bg-color: rgb(255 182 0);  
3 }
```

Custom Properties müssen keineswegs immer einen so großen Gültigkeitsbereich aufweisen. Wird ein Wert etwa nur in Absätzen benötigt, kann er auch nur für diese definiert werden:

```
1 p {  
2   --text-color: rgb(67 67 67);  
3 }
```

5.4.2 Custom Properties anwenden

Um nun den im Property hinterlegten Wert anzuwenden, wird der Name des Properties in der Funktion `var()` angegeben. Der folgende Aufruf setzt die im `<body>` definierte Hintergrundfarbe in allen `<div>`- und `<p>`-Elementen angewandt:

```
1 div, p {  
2   background-color: var(--bg-color);  
3 }
```

Sollte das Property aus irgendeinem Grund nicht zur Verfügung stehen, kann mit Beistrich getrennt auch ein Fallback-Wert angegeben werden:

```
1 div, p {  
2   background-color: var(--bg-color, red);  
3 }
```

Dieser ist in diesem Fall der Farbwert `red`, könnte aber auch ein weiteres Custom Property sein.

5.4.3 Die Kaskade und Vererbung ausnützen

Bei Custom Properties kommt – wie von CSS generell bekannt – die Kaskade zur Anwendung. Spezifischere Selektoren kommen also eher zum Zug, ebenso später definierte. Ebenso werden Custom Properties in der Hierarchie vererbt.

Es ist also möglich, den Wert eines Properties in einem weiteren Selektor zu ändern und damit das Ergebnis zu beeinflussen. Hier werden im `<body>`-Element zunächst einige

Properties gesetzt, etwa eine Hintergrundfarbe und eine Schriftfarbe. Die Schriftfarbe wird sogleich im gesamten `<body>` angewandt:

```
1 body {
2   --bg-color: rgb(255 182 0);
3   --text-color: rgb(236 233 230);
4   --margin: 10px;
5   --size: 200px;
6   font-family: Raleway, Arial, sans-serif;
7   color: var(--text-color);
8   display: flex;
9 }
```

Alle `<div>`- und `<p>`-Elemente bekommen dann einen die restlichen Werte gesetzt:

```
1 div, p {
2   width: var(--size);
3   height: var(--size);
4   margin: var(--margin);
5   background-color: var(--bg-color, red);
6   display: flex;
7   justify-content: center;
8   align-items: center;
9 }
```

Um nun aber allen `<div>`-Elementen eine andere Hintergrundfarbe zu setzen als in dieser Formatierung vorgeschlagen, kann einfach das Custom Property `--bg-color` für alle `<div>`-Elemente überschrieben werden. Somit erhalten sie diese Farbe, ohne die gesamte `background-color`-Eigenschaft neu setzen zu müssen.

```
1 div {
2   --bg-color: rgb(250 94 94);
3 }
```

Möchte man nun die Schriftfarbe für alle `<p>`-Elemente setzen, kann auch für diese spezifisch eine neue Farbe gesetzt werden. Da aber die Schriftfarbe mit (`color`) für das `<body>`-Element gesetzt wurde, muss hier die Eigenschaft erneut angegeben werden, denn sonst wäre zwar das Custom Property neu definiert, aber es würde nicht angewendet werden (während `--bg-color` ja für alle `<div>`- und `<p>`-Elemente angewandt wurde, daher funktioniert hier einfach die Neudefinition des Properties).

```
1 p {
2   --text-color: rgb(67 67 67);
3   color: var(--text-color);
4 }
```

Abbildung 5.10 zeigt die Formatierungen angewandt auf zwei `<div>`- und ein `<p>`-Element.

CSS Custom Properties können noch viel mehr. Auch hierfür gibt es ein ausgezeichnetes Tutorial unter [11].

5.5 CSS Resets & Normalisierung

Trotz ständiger Weiterentwicklung der einzelnen Browser und auch gemeinsamer Standardisierungsbestrebungen, gibt es nach wie vor Unterschiede in der Darstellung. Dies

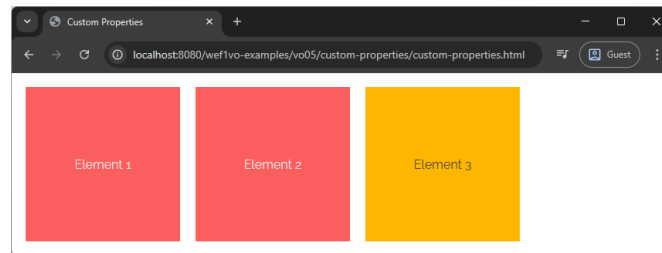


Abbildung 5.10: Die ersten beiden Boxen sind `<div>`-Elemente, die dritte ein `<p>`-Element. Höhe, Breite und Margins werden zentral gesetzt, ebenso die Hintergrundfarbe. Diese wird aber für `<div>`-Elemente nochmals überschrieben und ist daher anders. Das `<p>`-Element erhält eine andere Schriftfarbe.

liegt vor allem in verschiedenen Default-Stylesheets, die in den einzelnen Browsern hinterlegt sind. Um nun nicht auf jeden Browser Rücksicht nehmen zu müssen, also z. B. abzufragen, um welchen Browser es sich handelt und dabei bestimmte Dinge speziell anzupassen, hat es sich etabliert, die entsprechenden Styles komplett zurückzusetzen, d. h. das Browser-Stylesheet völlig außer Kraft zu setzen. Dies bietet die Möglichkeit, mit den eigenen Styles quasi von Null zu beginnen. Ein solches Rücksetzen nennt sich *CSS Reset*.

5.5.1 Alles zurücksetzen: CSS Reset

In vielen der Beispiele in dieser und der vorangehenden Vorlesung haben Sie vermutlich die folgende CSS-Angabe bemerkt:

```
1 * {
2   margin: 0;
3 }
```

Sie ist die einfachste Form eines CSS Resets und entfernt für jedes HTML-Element etwaige voreingestellte Außenabstände. Damit ist es jedoch kaum getan. Zusätzlich sollten Innenabstände, Rahmen, etwaige Outlines entfernt werden, die Zeilenhöhe auf einfach und die Schriftgröße auf 100 % gesetzt werden und noch einige andere Dinge.

Um dies nicht selbst erledigen zu müssen, haben sich im Web verschiedene, sogenannte Reset Stylesheets etabliert. Das historisch bekannteste ist *Reset CSS* von Eric Meyer [36]. Es nimmt die folgenden Einstellungen vor, diese sollen laut dem Autor jedoch primär als Startpunkt gesehen und nach Belieben angepasst werden:

```
1 /* http://meyerweb.com/eric/tools/css/reset/
2    v2.0 | 20110126
3    License: none (public domain)
4 */
5
6 html, body, div, span, applet, object, iframe,
7 h1, h2, h3, h4, h5, h6, p, blockquote, pre,
8 a, abbr, acronym, address, big, cite, code,
9 del, dfn, em, img, ins, kbd, q, s, samp,
10 small, strike, strong, sub, sup, tt, var,
11 b, u, i, center,
12 dl, dt, dd, ol, ul, li,
```

```

13 fieldset, form, label, legend,
14 table, caption, tbody, tfoot, thead, tr, th, td,
15 article, aside, canvas, details, embed,
16 figure, figcaption, footer, header, hgroup,
17 menu, nav, output, ruby, section, summary,
18 time, mark, audio, video {
19     margin: 0;
20     padding: 0;
21     border: 0;
22     font-size: 100%;
23     font: inherit;
24     vertical-align: baseline;
25 }
26 /* HTML5 display-role reset for older browsers */
27 article, aside, details, figcaption, figure,
28 footer, header, hgroup, menu, nav, section {
29     display: block;
30 }
31 body {
32     line-height: 1;
33 }
34 ol, ul {
35     list-style: none;
36 }
37 blockquote, q {
38     quotes: none;
39 }
40 blockquote:before, blockquote:after,
41 q:before, q:after {
42     content: '';
43     content: none;
44 }
45 table {
46     border-collapse: collapse;
47     border-spacing: 0;
48 }

```

Ein modernerer Ansatz für einen CSS-Reset wird von Bell in [5] beschrieben.

5.5.2 Sinnvoll zurücksetzen: CSS Normalisierung

CSS Resets stellen einen sehr radikalen Eingriff in das Renderverhalten der Webbrowser dar. Denn es werden nicht nur ungeliebte Einstellungen entfernt, sondern durchaus auch sinnvolle, wie etwa die Einrückung bei Aufzählungen. Diese müssen dann in der Regel im eigenen Stylesheet wieder definiert werden, was durchaus nicht unumstritten ist.

Aus diesem Grunde wird in vielen Webprojekten und Frameworks (wie etwas auch in Bootstrap) kein Reset Stylesheet verwendet, sondern auf *CSS Normalisierung* gesetzt. Bei diesem Ansatz werden nur Dinge korrigiert, die tatsächlich in Browsern unterschiedlich sind und zu Darstellungsproblemen führen können.

Das bekannteste Normalisierungs-Stylesheet ist *Normalize.css* von Nicolas Gallagher [19]. Es kann, nachdem es heruntergeladen wurde, auf zwei Weisen verwendet werden:

- Als Ausgangspunkt für das eigene Stylesheet. Die Inhalte von `normalize.css` werden als Basis angesehen, eigene Styles werden hinzugefügt bzw. die Ausgangsbasis angepasst.

- Als zusätzliches Stylesheet: `normalize.css` wird vor dem eigenen Stylesheet unverändert eingebunden, die eigenen Styles überschreiben – wo nötig – das Normalisierungs-Stylesheet.

Da die CSS-Datei sehr umfangreich ist, wird auf eine Angabe im Skriptum verzichtet. Unter [19] kann der Quellcode angesehen bzw. heruntergeladen werden. Da das Projekt seit 2018 jedoch nicht mehr weiterentwickelt wird, haben sich daraus zwei Abspaltungen (engl. „forks“) gebildet, die auf `Normalize.css` basieren, jedoch noch aktiv entwickelt werden:

- *@csstools/normalize.css* [13]: Vom Co-Autor vom ursprünglichen `Normalize.css`, Jonathan Neal, entwickelt und nach Auffassungsunterschieden mit Nicolas Gallagher selbstständig fortgeführt.
- *modern-normalize* [44]: Eine auf `Normalize.css` basierende Variante, die jedoch nur mehr moderne Browser berücksichtigt.

`Normalize.css` findet sich wie bereits erwähnt auch in aktuellen Bootstrap-Versionen. Ein Fork wird dort als *Bootstrap Reboot*⁷ bezeichnet.

Neben `Normalize.css` und seinen Forks hat sich *CSS Remedy*⁸ als moderner Ansatz etabliert. Es verfolgt das Ziel, die sinnvollen Standardformatierungen der Browser beizubehalten, aber problematische Unterschiede zu beheben.

5.6 CSS Methodiken

Resets und Normalisierung bilden eine die *technische Grundlage*, um mit konsistenten Ausgangswerten in allen Browsern zu arbeiten. Sobald ein Projekt jedoch größer wird oder Layouts bzw. Komponenten wiederverwendbar sein sollen, stellt sich zusätzlich die Frage, *wie* Styles sinnvoll strukturiert und organisiert werden können. Genau hier setzen verschiedene *CSS-Methodiken und Architekturansätze* an.

Bei kleinen Webprojekten ist die Organisation der Stylesheets bzw. der darin enthaltenen Regeln normalerweise kein großes Problem. Styles werden für den jeweiligen Anlassfall geschrieben, möglicherweise kommt ein CSS-Präprozessor zur Unterstützung hinzu (siehe Vorlesung 11). Wächst jedoch ein Projekt oder sollen Layouts bzw. Teile davon wiederverwendbar sein, so ist eine gewisse Struktur und Methodik unumgänglich. Im Laufe der Zeit haben sich einige Ansätze herauskristallisiert, wie CSS organisiert und auch modularisiert werden kann, um die Entwicklung effektiver zu gestalten.

Dieser Abschnitt soll als Anregung dienen, sich zu gegebener Zeit mit diesen Techniken auseinanderzusetzen, falls der Bedarf besteht bzw. der Eindruck entsteht, dass die eigene CSS-Codebasis einer Strukturierung bedarf.

5.6.1 Historische Architekturansätze: OOCSS, SMACSS und ITCSS

Bevor moderne Sprachfeatures wie `@layer` oder Custom Properties verfügbar waren, wurden verschiedene Architekturansätze entwickelt, um große Stylesheets besser handhabbar zu machen.

⁷<https://getbootstrap.com/docs/5.3/content/reboot/>

⁸<https://github.com/jensimmons/cssremedy>

Object Oriented CSS (OOCSS) versucht, wiederverwendbare „Objekte“ zu definieren, indem Struktur (z. B. Breiten, Abstände) und Aussehen (Farben, Hintergründe) getrennt beschrieben werden. Ziel ist es, Duplikate zu vermeiden und Komponenten möglichst unabhängig vom Kontext zu halten.

SMACSS (Scalable and Modular Architecture for CSS) ordnet Regeln in Kategorien wie *Base*, *Layout*, *Module*, *State* und *Theme* ein. Ideen daraus finden sich heute noch in vielen Codebasen wieder (z. B. die Trennung von Grundformatierungen und Komponenten).

ITCSS (Inverted Triangle CSS) organisiert Styles in einer Schichtenpyramide vom Allgemeinen zum Speziellen (z. B. *Settings* → *Tools* → *Generic* → *Elements* → *Objects* → *Components* → *Utilities*). Dieses Schichtenmodell lässt sich heute sehr gut mit `@layer` (siehe unten) direkt in CSS nachbilden.

Diese Ansätze sind aus historischer und konzeptioneller Sicht interessant, werden aber in modernen Projekten häufig durch eine Kombination aus klaren Namenskonventionen (z. B. BEM), Utility-Klassen und nativen Sprachmitteln wie `@layer` ersetzt. Eine gute Übersicht über diese klassischen Ansätze bietet <https://cathydutton.co.uk/posts/css-layers/>.

5.6.2 Block Element Modifier (BEM)

BEM ist ein Ansatz, wiederverwendbare Komponenten mit Hilfe von CSS-Klassen zu kreieren. Stile, die nach dem BEM-Prinzip erzeugt worden sind, können völlig frei in jedem Szenario wiederverwendet werden und erzeugen daher große Flexibilität.

Zu jedem der drei namensgebenden Teile gibt es eine Terminologie gepaart mit einer strikten Namenskonvention:

- **Block:** eigenständige, logische Einheit auf der Seite (z. B. Header, Formular, Menü).
- **Element:** Bestandteil eines Blocks (z. B. Button innerhalb eines Formulars).
- **Modifier:** Variation oder Zustand eines Blocks oder Elements (z. B. „dunkel“, „kompakt“).

Block

Blöcke sind eigenständig und dürfen keine Abhängigkeiten zu anderen Blöcken oder Elementen haben. Sie werden mit einer CSS-Klasse definiert (hier etwa die Suche auf einer Seite):

```
1 .site-search {
2   ...
3 }
```

Angewandt würde die Klasse beim `<form>`-Element:

```
1 <form action="..." class="site-search">
2   ...
3 </form>
```

Element

Als Element wird ein Teil eines Blocks bezeichnet, also etwa der Titel im Header, der Login-Button, ein Menü-Eintrag oder das Suchfeld.

Elemente existieren immer in Zusammenhang mit ihrem Block, haben alleine also keine Bedeutung. Wiederum wird eine CSS-Klasse definiert, als Namenskonvention wird der Name des Blocks, gefolgt von zwei Unterstrichen und dem Elementnamen gewählt:

```
1 .site-search__field {
2   ...
3 }
```

Diese Klasse würde beim Suchfeld angewandt werden (ebenso würde für den Button eine eigene Klasse entsprechend gewählt werden):

```
1 <form action="..." class="site-search">
2   <input type="search" class="site-search__field">
3   <button type="submit" class="site-search__button">Search</button>
4 </form>
```

Modifier

Modifier beschreiben Variationen eines Blocks, also ein zentrierter Header, ein verkürztes Login-Formular (ohne Labels), ein vertikales Menü oder eine anders gestylte Suche.

Modifier beziehen sich ebenfalls konkret auf einen Block. Die CSS-Klasse besteht aus dem Blocknamen, gefolgt von zwei Bindestrichen und dem Modifikator:

```
1 .site-search--full {
2   ...
3 }
```

Angewandt wird die Klasse beim Block selbst:

```
1 <form action="..." class="site-search site-search--full">
2   <input type="search" class="site-search__field">
3   <button type="submit" class="site-search__button">Search</button>
4 </form>
```

Auch Elemente können Modifikatoren haben. Diese werden dann etwa wie folgt definiert:

```
1 .site-search__button--clicked {
2   ...
3 }
```

Mehr Details zu BEM finden sich auf der Projektseite unter <http://getbem.com/>.

5.6.3 Style Tools for UI Components (SUIT CSS)

SUIT CSS ist eine Modifikation der BEM-Prinzipien, um kleine Probleme mit diesen zu lösen, so ist einerseits die Benennung der einzelnen Teile, aber vor allem die Namenskonvention anders und soll besser lesbar sein. Blocks heißen nun Components, Elements werden als Descendants bezeichnet. Neben den bekannten Modifiern gibt es auch Zustände für Komponenten.

Zusätzlich führt SUIT CSS noch Utilities ein, die innerhalb von Components verwendet werden können und oft gebrauchte Formatierungen beinhalten.

Components

Components entsprechen genau den Blöcken in BEM. Sie sollen alleinstehend funktionieren und mit jedem Element anwendbar sein. Daher werden Klassenselektoren verwendet.

Komponenten werden jedoch in Pascal Case (groß begonnen, Wortgrenzen mit Großbuchstaben) geschrieben, um sie leichter von altem, „legacy“ Code (der nicht BEM oder SUIT CSS verwendet) unterscheiden zu können. Die Suche würde in SUIT CSS wie folgt definiert:

```
1 .SiteSearch {  
2   ...  
3 }
```

Angewandt würde die Klasse beim `<form>`-Element:

```
1 <form action="..." class="SiteSearch">  
2   ...  
3 </form>
```

Descendants

Descendants entsprechen den Elementen in BEM, gehören also logisch zu einer Component. Sie werden mit einem Bindestrich vom Component-Namen getrennt und in Camel Case (klein begonnen, Wortgrenzen mit Großbuchstaben) geschrieben:

```
1 .SiteSearch-field {  
2   ...  
3 }
```

Diese Klasse würde beim Suchfeld angewandt werden, ebenso ein Descendant für den Button:

```
1 <form action="..." class="SiteSearch">  
2   <input type="search" class="SiteSearch-field">  
3   <button type="submit" class="SiteSearch-button">Search</button>  
4 </form>
```

Modifier

Modifier werden auch hier mit doppeltem Bindestrich geschrieben, auch bei ihnen kommt Camel Case zum Einsatz:

```
1 .SiteSearch--full {  
2   ...  
3 }
```

Angewandt wird die Klasse beim Block selbst:

```
1 <form action="..." class="SiteSearch SiteSearch--full">  
2   <input type="search" class="SiteSearch-field">  
3   <button type="submit" class="SiteSearch-button">Search</button>  
4 </form>
```

States

Mit States wird eine Zustandsänderung einer Komponenten bezeichnet. Die Notation beginnt mit `is-` und wird in Camel Case fortgesetzt. Diese Klassen existieren immer nur als Zusätze zu bestehenden Komponenten:

```
1 .SiteSearch {  
2   ...  
3 }  
4 .SiteSearch.is-searching {  
5   ...  
6 }
```

Angewandt würde dies wie folgt:

```
1 <form action="..." class="SiteSearch is-searching">  
2   ...  
3 </form>
```

Utilities

Utilities schließlich sind Klassen, die in Components für oft benötigte Dinge zum Einsatz kommen. Sie definieren etwa Abstände, Media Query Breakpoints oder Floats. Sie Beginnen mit dem `u-`-Präfix und werden in Camel Case geschrieben.

Das folgende Utility etwa zentriert Text innerhalb von Components:

```
1 .u-textCenter {  
2   text-align: center;  
3 }
```

Es wird z.B. in der Suche angewandt, um darin alles zentriert darzustellen:

```
1 <form action="..." class="SiteSearch u-textCenter">  
2   ...  
3 </form>
```

Mehr Details zu SUIT CSS finden sich auf der Projektseite unter <https://suitcss.github.io/>.

5.6.4 Cascade Layers (@layer)

Ergänzend zu den vorgestellten Methodiken sein nochmals *CSS Cascade Layers* explizit erwähnt, die bereits im Rahmen der Vorstellung der Kaskade in Abschnitt 3.6 vorkamen. Eingeführt 2021 in [69], sind diese seit 2022 in allen modernen Browsern unterstützt. Mit ihnen lässt sich die Reihenfolge verschiedener Stilgruppen festlegen.

Dadurch können Entwickler*innen Regeln logisch strukturieren, bevor Dinge wie Specificity und Reihenfolge schlagend werden. Damit ist **@layer** das native, moderne Gegenstück zu früheren Architekturansätzen wie SMACSS oder ITCSS und ergänzt Konzepte wie CSS Resets und Normalisierung aus Abschnitt 5.5, die heute oft in eine eigene Layer (**@layer reset**) gelegt werden.

Mit dem Schlüsselwort (auch „at-rule“ genannt) **@layer** wird zunächst eine Reihenfolge an Layern definiert. Dazu werden Layer-Namen mit Komma getrennt angegeben. Namen, die weiter hinten stehen, haben mehr Gewicht.

Mit


```
1 @layer reset, base, components, utilities;
```

werden die vier Layer `reset`, `base`, `components` und `utilities` definiert.

Um nun Regeln einem Layer zuzuwiesen, können diese mit der `@layer`-Angabe und dem Namen gekapselt werden:

```
1 @layer reset {
2   *, *::before, *::after {
3     box-sizing: border-box;
4     margin: 0;
5     padding: 0;
6   }
7 }
8 @layer base {
9   body {
10    font-family: system-ui, sans-serif;
11    line-height: 1.5;
12  }
13 }
14 @layer components {
15   .button {
16     background: steelblue;
17     color: white;
18   }
19 }
20 @layer utilities {
21   .text-center {
22     text-align: center;
23   }
24 }
```

Alternativ kann auch bei einem Import ein Layer angegeben werden und so die Regeln eines gesamten Stylesheets einem Layer zugewiesen werden:

```
1 @import url("modern-normalize.css") layer(reset);
2 @import url("base.css") layer(base);
3 @import url("components.css") layer(components);
4 @import url("utilities.css") layer(utilities);
```

Styles in früher definierten Layers (z. B. `reset`) werden immer vor späteren Layers (z. B. `utilities`) berücksichtigt – unabhängig von der Reihenfolge im Stylesheet oder der Selektor-Specificity. Dies macht komplexe Stylesheets deutlich berechenbarer und erlaubt eine klare Trennung von Basisformatierungen (auch Resets oder Normalisierung) und projekt- oder komponentenspezifischen Stilen.

5.6.5 Weitere Methodiken und moderne Konzepte

Neben den erwähnten Dingen sind auch die folgenden Dinge interessant bzw. nützlich:

- **Atomic CSS:** Statt größere Komponentenklassen zu definieren, werden sehr kleine, hochgradig spezialisierte Klassen verwendet, die jeweils genau eine Eigenschaft setzen (z. B. `.mt-4` nur für den oberen Außenabstand oder `.text-center` nur für die Textausrichtung). Layouts entstehen durch das Kombinieren vieler solcher „Atome“ direkt im HTML. Eine ausführliche Diskussion des Begriffs findet sich unter <https://css-tricks.com/lets-define-exactly-atomic-css/>.

- **Utility-First CSS:** Frameworks wie Tailwind CSS setzen fast ausschließlich auf kleine, wiederverwendbare Klassen, die direkt im HTML kombiniert werden: <https://heydonworks.com/article/what-is-utility-first-css/>.
- **CSS Nesting:** Regeln können direkt innerhalb eines Selektors verschachtelt werden (z. B. `&:hover`), was die Lesbarkeit erhöht und Abhängigkeiten deutlicher macht. <https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Nesting/Using> gibt eine gute Übersicht.

5.7 Weiterführende Literatur

Zum Thema Layouts finden sich grundsätzliche Ansätze wiederum in den Standardwerken [40, 58].

Generell kann für alle erwähnten Frameworks (Bootstrap, Foundation, Tailwind CSS) primär die eigene Dokumentation empfohlen werden. Selbstverständlich finden sich zahllose Tutorials online.

Zu den Themen Flexible Box Layout, Grid Layout und Custom Properties sind die erwähnten Tutorials [11, 12, 26, 38] derzeit sicherlich die beste (freie) Referenz.

Die Themen CSS Reset, Normalisierung sowie CSS Methodiken werden in den erwähnten Online-Ressourcen am besten abgearbeitet.

Vorlesung 6

Web Accessibility

6.1 Allgemeines

Web Accessibility (engl. wörtlich „Zugänglichkeit“), auch oft barrierefreies Web (oder fälschlich barrierefreies Internet) oder kurz A11Y (für Accessibility – zwischen A und Y sind 11 Buchstaben) genannt, bezeichnet die Praxis, Websites so zu gestalten, dass sie für alle Personen – unabhängig von Behinderungen oder sonstigen Einschränkungen – verwendbar sind.

Tim Berners-Lee, Vorsitzender des W3C und Erfinder des World Wide Web, drückt dies wie folgt aus [52]:

The power of the Web is in its universality. Access by everyone regardless of disability is an essential aspect.

Genauer ausgedrückt bedeutet dies, dass Personen mit Behinderungen das Web wahrnehmen, verstehen, navigieren, mit ihm interagieren und zum Web etwas beitragen können sollen. Dabei werden alle Behinderungen gleichermaßen berücksichtigt:

- Sehbehinderungen (z. B. Blindheit, Farbenblindheit, geringe Sehkraft),
- Hörbehinderungen (z. B. Gehörlosigkeit, Schwerhörigkeit),
- physische Behinderungen (z. B. Amputationen, Arthritis, Muskeldystrophie),
- kognitive und neurologische Behinderungen (z. B. Aufmerksamkeitsdefizit- oder Hyperaktivitätsstörung, Legasthenie, Epilepsie, Gedächtnisstörung),
- Sprachbehinderungen (z. B. Stummheit, Stottern, Sprechapraxie).

Darüber hinaus nimmt Web Accessibility aber auch auf ältere Nutzer*innen Rücksicht, die das Web oft aufgrund reduzierter motorischer Fähigkeiten nur mehr eingeschränkt nutzen können. Ebenso werden Personen mit temporären Einschränkungen, wie z. B. einem gebrochenen Arm unterstützt oder auch Personen, die von technologischen Limitierungen wie etwa geringer Bandbreite oder veralteten Geräten betroffen sind. Dies fällt explizit nicht unter den Begriff „Behinderung“, weshalb oft auch der generellere Ausdruck „Einschränkung“ zum Einsatz kommt.

Das W3C stellt auf seiner Seite Szenarios von Web User*innen mit Einschränkungen¹ vor, die als Beispiele dafür dienen sollen, wie die Arbeit mit dem Web durch unter-

¹<https://www.w3.org/WAI/people-use-web/user-stories/>

schiedlichste Behinderungen beeinflusst bzw. durch richtige Accessibility entsprechend erleichtert werden kann.

Das Gremium des W3C zur Förderung und Etablierung von Web Accessibility ist die *Web Accessibility Initiative* – kurz WAI – deren Aufgaben und Ziele im folgenden Abschnitt erläutert werden.

6.2 Die Web Accessibility Initiative

Die *Web Accessibility Initiative* [107] ist eine Arbeitsgruppe innerhalb des W3C, die sich ausschließlich mit dem barrierefreien Zugang zum Web beschäftigt.

6.2.1 Ziele

Die Ziele der WAI lassen sich wie folgt zusammenfassen:

- Entwicklung von Richtlinien, die als internationaler Standard für Web Accessibility angesehen werden,
- Veröffentlichung von Unterlagen, die zum Verständnis und zur Implementierung von Web Accessibility beitragen,
- Entwicklung von Accessibility Ressourcen durch internationale Zusammenarbeit.

Die dabei entstehenden Materialien durchlaufen dabei den üblichen W3C Veröffentlichungsmechanismus, wie er schon von Technologien wie etwa bei CSS bekannt ist. Die entstehenden Richtlinien beschäftigen sich jedoch nicht nur mit der Frage, wie Websites barrierefrei(er) gemacht werden können. Das Ziel lautet, den gesamten Prozess besser zu gestalten. Die entwickelten Richtlinien können dabei grob in drei verschiedene Kategorien eingeteilt werden:

- Richtlinien für Inhalte im Web (d. h. Websites),
- Richtlinien für Autor*innenwerkzeuge (d. h. Software, IDEs, CMS),
- Richtlinien für Clients (d. h. Webbrowser).

Innerhalb der WAI existieren einzelne Arbeitsgruppen, die sich mit der Erstellung der verschiedenen Richtlinien beschäftigen. Communitygruppen bieten darüber hinaus jedem*jeder die Möglichkeit, sich an Themen der Barrierefreiheit rund um die WAI zu beteiligen.

6.2.2 Arbeits- und Communitygruppen

Innerhalb der WAI existieren vier *Arbeitsgruppen*, die an verschiedenen Themen und Spezifikationen rum um Accessibility arbeiten. Im Folgenden wird ein kurzer Überblick über ihre Aufgaben gegeben. Über Communitygruppen können sich Interessierte an der WAI beteiligen, ohne direkt in einer Arbeitsgruppe mitzuarbeiten. Diese Gruppen sind jedoch nicht für die Erstellung von Standards zuständig, sondern dienen dem Austausch von Informationen und Ideen.

Accessibility Guidelines Working Group (AG WG)

Die wohl wichtigste – und meist mit dem Thema Accessibility direkt in Verbindung gebrachte – Arbeitsgruppe ist die *Accessibility Guidelines Working Group*². Sie veröffentlicht die Web Content Accessibility Guidelines (siehe Abschnitt 6.3), die die Referenz für Accessibility im Web darstellen.

Accessible Platform Architectures (APA) Working Group

Die *Accessible Platform Architectures Working Group*³, kurz APA, beschäftigt sich mit der Kontrolle von anderen W3C Spezifikationen hinsichtlich Accessibility auf einem technischen Level. Andere Arbeitsgruppen sollen dabei bei der Umsetzung von Accessibility Richtlinien unterstützt und mit technischem Know-How versorgt werden. Über einzelne, so genannte Task Forces werden spezielle Accessibility Brennpunkte gezielt bearbeitet. Zu den von dieser Arbeitsgruppe entwickelten Dokumenten gehören unter anderem die Synchronization Accessibility User Requirements [103] oder die XR Accessibility User Requirements [112]. Diese Dokumente beschreiben die Anforderungen, die Personen mit Einschränkungen in Bezug auf Synchronisierung in Audio und Video bzw. in Virtual und Augmented Reality haben.

Accessible Rich Internet Applications (ARIA) Working Group

Die *Accessible Rich Internet Applications (ARIA) Working Group*⁴, kurz ARIA WG, entwickelt Technologien, die bestehende Webinhalte für Personen mit Behinderungen im Bezug auf Accessibility verbessern sollen. Der Kern dabei sind die Richtlinien zu Accessible Rich Internet Applications (WAI-ARIA), welche in Abschnitt 6.4 vorgestellt werden. Einzelne Task Forces widmen sich innerhalb der Arbeitsgruppe verschiedenen Spezialthemen.

WAI Interest Group (WAI IG)

Die *WAI Interest Group*⁵, kurz WAI IG, produziert selbst keine Materialien zum Thema Accessibility sondern übernimmt die Revision und Diskussion von Richtlinien aus anderen Arbeitsgruppen. Darüber hinaus dient die Gruppe für den allgemeinen Informationsaustausch zum Thema Accessibility. Der Beitritt zu Mailinglisten oder das Folgen des Mastodon Feeds⁶ bietet einen guten Einblick über aktuelle Entwicklungen rund um das Thema Accessibility im W3C.

Community Gruppen

Neben den offiziellen Arbeitsgruppen gibt es auch *Community Gruppen*⁷, die sich mit speziellen Themen rund um Accessibility beschäftigen. Diese Gruppen sind offen für alle

²<https://www.w3.org/WAI/about/groups/agwg/>

³<https://www.w3.org/WAI/about/groups/apawg/>

⁴<https://www.w3.org/WAI/about/groups/ariawg/>

⁵<https://www.w3.org/WAI/about/groups/waiig/>

⁶<https://w3c.social/@wai>

⁷<https://www.w3.org/community/>

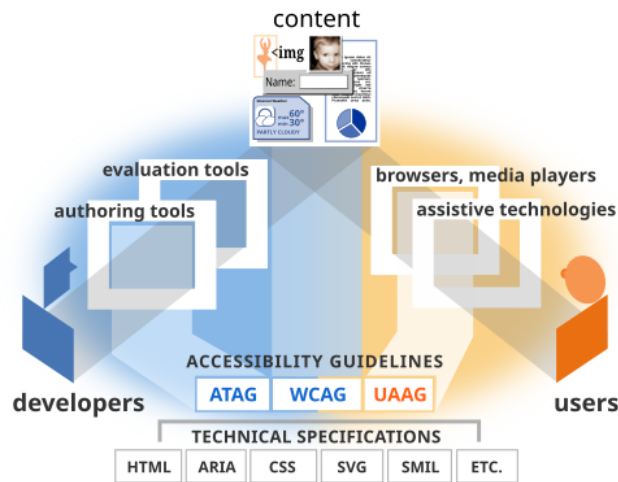


Abbildung 6.1: Essentielle Komponenten der Web-Accessibility und die Guidelines, die diese beschreiben. Quelle: [51].

Interessierten und bieten eine Plattform, um sich über spezielle Themen auszutauschen. Diese Gruppen sind für Webentwickler*innen und Interessierte gedacht, die sich für Accessibility interessieren und Ideen vorschlagen und diskutieren möchten. Die Gruppen sind dabei selbstorganisiert und können eigene Arbeitsgruppen bilden, um spezielle Themen zu bearbeiten.

6.2.3 Ergebnisse und Guidelines

Relevanter als die einzelnen Arbeitsgruppen sind die Ergebnisse, die von diesen hervor- gebracht werden. Analog zu den vom W3C spezifizierten Technologien, die als Recom- mendations den Stellenwert eines Standards erreichen, gibt es diesen Prozess auch für die einzelnen Richtlinien. Das W3C hat sich also zum Ziel gesetzt, konkrete Vorgaben im Bereich Accessibility herauszugeben, die es ermöglichen, die Anforderungen dieses Bereichs zu vereinheitlichen.

Während bei Accessibility jedoch zunächst meist nur wichtig erscheint, wie reguläre Websites für beeinträchtigte Personen leichter zugänglich gemacht werden können, ist der Bereich in Wirklichkeit jedoch viel umfangreicher. So ist es auch wichtig, Tools zur Entwicklung von Websites barrierefrei zu gestalten. Genauso sollen Webbrowser entspre- chend einfach zu bedienen sein und mit dem Aufkommen von dynamischen Webinhalten und User Interfaces ergeben sich erneut unterschiedliche Anforderungen.

Im folgenden wird zunächst ein Überblick über die wichtigsten Guidelines gegeben. Deren Zusammenhänge werden gut in Abbildung 6.1 dargestellt.

Web Content Accessibility Guidelines (WCAG)

Die *Web Content Accessibility Guidelines*⁸, kurz WCAG, sind Richtlinien zur barrierefreien Gestaltung von Webseiten. Sie sind mit Abstand die wichtigsten Regeln der WAI und werden daher in Abschnitt 6.3 gesondert im Detail erklärt.

Authoring Tools Accessibility Guidelines (ATAG)

Die *Authoring Tool Accessibility Guidelines*⁹ [61] sind Accessibility-Richtlinien, die sich mit der Gestaltung von so genannten Authoring Tools, also Software zum Erstellen von Webinhalten beschäftigen. Das W3C definiert den Begriff der Authoring Tools sehr weitläufig:

- Software, die zum Erzeugen von Webinhalten verwendet wird, z. B. WYSIWYG-Editoren wie Adobe Dreamweaver, oder auch JetBrains PhpStorm.
- Werkzeuge, die es ermöglichen, Dokumente in Web-Formaten abzuspeichern (z. B. Microsoft Word, das seine Dokumente auch als HTML-Datei speichern kann).
- Tools zum Erzeugen von Multimedia-Inhalten (Video, Audio etc.), die im Web zum Einsatz kommen.
- Content- und Site-Management-Systeme, die Webinhalte dynamisch aus Vorlagen (Templates) erzeugen können.
- Websites, die es Benutzer*innen ermöglichen, Inhalte hinzuzufügen (z. B. Blogs, Wikis, soziale Netze etc.).

Die ATAG-Richtlinien sind dabei in zwei *Teile* unterteilt. Teil A betrifft die Accessibility der Benutzer*innenoberfläche des Programms, Teil B betrifft die Generierung von barrierefreien Inhalten. Jeder Teil besteht aus *Prinzipien*, die die grobe Struktur vorgeben. Jedes der Prinzipien enthält wiederum die eigentlichen *Richtlinien*, also die Ziele, die von Authoring Tools erfüllt werden sollen, um als barrierefrei zu gelten. Für jede Guideline sind wiederum *Erfolgskriterien* definiert, die klar testbar sind und somit die schlussendliche Konformität zu ATAG 2.0 definieren. Hierfür gibt es drei Erfüllungsgrade: A, AA und AAA.

User Agent Accessibility Guidelines (UAAG)

Die *User Agent Accessibility Guidelines*¹⁰ [105], kurz UAAG, sind Richtlinien, die sich mit der barrierefreien Umsetzung von so genannten User Agents, also Software zur Darstellung von Webinhalten beschäftigen. Diese Richtlinien sind in erster Linie für Entwickler*innen von Webbrowsern, Media Playern oder unterstützenden Technologien wie Screen Readern wichtig.

Die Richtlinien sind dabei in fünf *Prinzipien* unterteilt, welche die grobe Struktur vorgeben. Jedes der Prinzipien enthält wiederum die eigentlichen *Richtlinien*, also die Ziele, die von User Agents erfüllt werden sollen, um als barrierefrei zu gelten. Für jede Guideline sind wiederum *Erfolgskriterien* definiert, die klar testbar sind und somit die

⁸<https://www.w3.org/WAI/standards-guidelines/wcag/>

⁹<https://www.w3.org/WAI/standards-guidelines/atag/>

¹⁰<https://www.w3.org/WAI/standards-guidelines/uaag/>

schlussendliche Konformität zu UAAG 2.0 definieren. Hierfür gibt es drei Erfüllungsgrade: A, AA und AAA.

Accessible Rich Internet Applications (WAI-ARIA)

Moderne Websites greifen vermehrt auf dynamische Elemente zurück, die es ermöglichen, neue Inhalte nachzuladen, ohne die Seite neu zu laden, Drag & Drop im Browser zu verwenden, oder grafische Effekte wie Ein- und Ausklappen von Elementen zu realisieren. Man spricht hierbei auch von so genannten *Rich Internet Applications*, kurz RIA. Für sie hat das W3C eine eigene Spezifikation veröffentlicht: *WAI-ARIA*¹¹. Diese wird in Abschnitt 6.4 ausführlich beschrieben.

6.3 Web Content Accessibility Guidelines (WCAG)

Die *Web Content Accessibility Guidelines* [108], kurz WCAG und zu deutsch Richtlinien für barrierefreie Webinhalte, sind ein Standard des W3C zur barrierefreien Gestaltung von Web Inhalten.

6.3.1 Allgemeines

Die WCAG sind mit Abstand die bekanntesten Richtlinien im Bereich der Accessibility. Oftmals werden sie auch stellvertretend für den Bereich Accessibility im Web angesehen, was allerdings nicht stimmt. Wie in den vorigen Abschnitten erwähnt, existieren auch Guidelines für andere Bereiche wie User Agents oder Authoring Tools, die ebenso wichtig, aber für „die breite Masse“ im Regelfall weniger relevant sind.

Die Notwendigkeit, Richtlinien zur barrierefreien Gestaltung von Webinhalten zu erstellen, kam mit der zunehmenden Verbreitung des World Wide Web in den 1990er Jahren auf. Während die Anfänge des Web nur textbasiert waren, kamen zunehmend Multimediainhalte auf, die für beeinträchtigte Personen nicht mehr zugänglich waren. Zusätzlich wurde diese Problematik einerseits durch die fehlende Standardkonformität der Browser und andererseits durch mangelhaftes Problembewusstsein bei Webautor*innen verschärft. Letzteres äußert sich in der Formulierung von Inhalten, die primär aufgrund visueller Gegebenheiten erfolgen (z. B. ein Link mit dem Text „hier klicken“ – dieser Link bietet keinerlei Informationen im Text, was sich dahinter verbirgt).

HTML bietet zwar in seiner aktuellen Version, d. h. dem Living Standard, sowie in seinen Vorgängerversionen HTML 4.01 bzw. auch XHTML 1.0 genügend Möglichkeiten, Barrierefreiheit zu realisieren, jedoch ist dies ohne konkrete Vorgaben und Anleitungen ein schwieriges Unterfangen. Webautor*innen müssen mit der Thematik vertraut gemacht werden, nur dann ist eine weit verbreitete Umsetzung möglich. Zu diesem Zweck wurden die WCAG entworfen, die aktuell in Version 2.2 vom Dezember 2024 als Recommendation vorliegen und seit September 2025 auch ein offizieller ISO/IEC Standard sind [29]. Die Arbeit an den WCAG 3.0 wurde Ende 2021 gestartet und befindet sich derzeit noch in einem sehr frühen Stadium [106].

¹¹<https://www.w3.org/WAI/standards-guidelines/aria/>

6.3.2 WCAG 2.2

Die *Web Content Accessibility Guidelines (WCAG) 2.2* [108] sind eine Weiterentwicklung der Version 2.1, wobei 9 neue Erfolgskriterien hinzukamen und ein obsoletes entfernt wurde. Die Änderungen ergeben sich hierbei in der Regel durch die Weiterentwicklung der Webtechnologien, die neue Erfordernisse mit sich bringen. Die WCAG 2.2 Kriterien sind sowohl maschinell als auch durch menschliche Evaluierung testbar – ersteres ist nicht immer mit jedem Tool sofort möglich, da viele oft länger benötigen, bis der aktuelle Standard inkludiert ist.

Wie auch schon zuvor bei den ATAG 2.0 und UAAG 2.0 zu sehen war, existieren auch für WCAG 2.2 vier große *Prinzipien*, welche wiederum mit *Richtlinien* unterlegt sind. Jede dieser Guidelines hat testbare Erfolgskriterien in drei verschiedenen Ebenen (A, AA, AAA), die bestimmen, wie sehr eine Website zu WCAG 2.2 konform ist:

- **Stufe A:** Level A Konformität stellt das Minimum an Konformität dar. Alle Erfolgskriterien auf Ebene A sind erfüllt.
- **Stufe AA:** Level AA Konformität bedeutet, dass alle Level A und Level AA Erfolgskriterien erfüllt sind.
- **Stufe AAA:** Level AAA Konformität wird durch die Erfüllung aller A, AA und AAA Erfolgskriterien erreicht.

WCAG 2.2 Guidelines

Die vier zugrunde liegenden Prinzipien sind im Englischen unter dem Akronym **POUR** zusammengefasst: **P**erceivable (wahrnehmbar), **O**perable (bedienbar), **U**nderstandable (verständlich), **R**obust. Im Folgenden sind die vier Prinzipien mit den dazugehörigen Guidelines und einigen wichtigen Erfolgskriterien aufgeführt.

1. **Prinzip 1: Perceivable/Wahrnehmbar** – Informationen und Bestandteile der Benutzer*innenschnittstelle müssen den Benutzer*innen so präsentiert werden, dass diese sie wahrnehmen können.
 - **Richtlinie 1.1: Textalternativen:** Stelle Textalternativen für alle Nicht-Text-Inhalte zur Verfügung, so dass diese in andere, von Benutzer*innen benötigte Formen geändert werden können, wie zum Beispiel Großschrift, Braille, Symbole oder einfachere Sprache.
 - Alle Nicht-Text-Inhalte, die den Benutzer*innen präsentiert werden, haben eine Textalternative, die einem gleichen Zweck dient (STUFE A).
 - **Richtlinie 1.2: Zeitbasierte Medien:** Stelle Alternativen für zeitbasierte Medien zur Verfügung.
 - Es werden Alternativen für reine Audio- und Videoinhalte bereit gestellt (STUFE A).
 - Untertitel werden für alle aufgezeichneten Audioinhalte in synchronisierten Medien bereitgestellt (STUFE A).
 - ...
 - **Richtlinie 1.3: Anpassbar:** Erstelle Inhalte, die auf verschiedene Arten

dargestellt werden können (z. B. einfacheres Layout), ohne dass Informationen oder Struktur verloren gehen.

- Informationen, Struktur und Beziehungen, die über die Darstellung vermittelt werden, können durch Software bestimmt werden oder stehen in Textform zur Verfügung (STUFE A).
- Wenn die Reihenfolge, in der Inhalte präsentiert werden, sich auf deren Bedeutung auswirkt, kann die korrekte Leseabfolge durch Software bestimmt werden (STUFE A).
- Anweisungen, die für das Verständnis und die Bedienung von Inhalt bereitgestellt werden, stützen sich nicht nur auf sensorische Eigenschaften von Komponenten wie Form, Größe, visuelle Position, Ausrichtung oder Ton (STUFE A).
- Der Inhalt der Seite wird nicht auf eine einzelne Orientierung der Darstellung (Hoch- oder Querformat) reduziert, außer dies ist absolut notwendig (STUFE AA).
- ...
- **Richtlinie 1.4: Unterscheidbar:** Mache es Benutzer*innen leichter, Inhalt zu sehen und zu hören einschließlich der Trennung von Vorder- und Hintergrund.
 - Farbe wird nicht als einziges visuelles Mittel benutzt, um Informationen zu vermitteln, eine Handlung zu kennzeichnen, eine Reaktion zu veranlassen oder ein visuelles Element zu unterscheiden (STUFE A).
 - Die visuelle Darstellung von Text und Bildern von Text hat ein Kontrastverhältnis von mindestens 4,5:1 (STUFE AA).
 - ...

2. Prinzip 2: Operable/Bedienbar – Bestandteile der Navigation und Benutzer*innenschnittstelle müssen bedienbar sein.

- **Richtlinie 2.1: Per Tastatur zugänglich:** Sorge dafür, dass alle Funktionalitäten per Tastatur zugänglich sind.
 - Alle Funktionalitäten des Inhalts sind durch eine Tastaturschnittstelle bedienbar, ohne dass eine bestimmte Zeiteinteilung für einzelne Tastenanschläge erforderlich ist (STUFE A).
 - Sind Keyboard-Shortcuts vorhanden, die nur aus Buchstaben, Zahlen oder Punktionszeichen bestehen, müssen diese umbelegt, deaktiviert oder nur aktiv sein, wenn diese Komponente den Fokus besitzt (STUFE A).
 - ...
- **Richtlinie 2.2: Ausreichend Zeit:** Gib den Benutzer*innen ausreichend Zeit, Inhalte zu lesen und zu benutzen.
 - Benutzer*innen können die zeitliche Begrenzung abschalten, anpassen oder ausweiten, bevor sie darauf treffen (STUFE A).
 - Benutzer*innen können bewegende, blinkende oder scrollende Informationen anhalten oder pausieren (STUFE A).

- Benutzer*innen werden über die maximal mögliche Dauer von Inaktivität gewarnt, falls es dadurch zu Datenverlust kommen kann (STUFE AAA).
 - ...
 - **Richtlinie 2.3: Anfälle und körperliche Reaktionen:** Gestalte Inhalte nicht auf Arten, die bekanntermaßen Anfälle oder körperliche Reaktionen auslösen können.
 - Webseiten enthalten nichts, was öfter als dreimal in einem beliebigen, eine Sekunde dauernden Zeitraum blitzt, oder der Blitz ist unterhalb der allgemeinen Grenzwerte zu Blitzen und roten Blitzen (STUFE A).
 - Animationen, die durch Interaktion mit der Seite hervorgerufen werden, können deaktiviert werden, außer diese Interaktion ist für die Funktionalität essentiell (STUFE AAA).
 - ...
 - **Richtlinie 2.4: Navigierbar:** Stelle Mittel zur Verfügung, um Benutzer*innen dabei zu unterstützen zu navigieren, Inhalte zu finden und zu bestimmen, wo sie sich befinden.
 - Es gibt einen Mechanismus, um Inhaltsblöcke zu umgehen, die auf verschiedenen Webseiten wiederholt werden (STUFE A).
 - Webseiten haben einen Titel, der Thema oder Zweck beschreibt (STUFE A).
 - Überschriften und Labels beschreiben ein Thema oder einen bestimmten Zweck (STUFE AA).
 - ...
 - **Richtlinie 2.5: Eingabemodalitäten:** Mache es für Benutzer*innen einfacher, Funktionalitäten mit anderen Eingabemodalitäten außer dem Keyboard zu verwenden.
 - Alle Funktionalitäten, die Multi-Point oder pfadbasierte Gesten verwenden, sind auch durch einen Single-Pointer (Tap, Click, Double-Tap oder Long-Press) erreichbar (STUFE A).
 - Alle Komponenten der Benutzer*innenschnittstelle, die mit einem Label beschriftet sind, enthält der (programmatische) Name den Text, der visuell dargestellt wird. (STUFE A).
 - ...
3. **Prinzip 3: Understandable/Verständlich** – Informationen und Bedienung der Benutzer*innenschnittstelle müssen verständlich sein.
- **Richtlinie 3.1: Lesbar:** Mache den Textinhalt lesbar und verständlich.
 - Die voreingestellte menschliche Sprache jeder Webseite kann durch Software bestimmt werden (STUFE A).
 - Es gibt einen Mechanismus, um spezielle Definitionen von Wörtern oder Wendungen zu erkennen, die auf ungewöhnliche oder eingeschränkte Weise benutzt werden, Idiome und Jargon eingeschlossen (STUFE AAA).

- Es gibt einen Mechanismus, um die ausgeschriebene Form oder Bedeutung von Abkürzungen zu erkennen (STUFE AAA).
 - ...
 - **Richtlinie 3.2: Vorhersehbar:** Sorge dafür, dass Webseiten vorhersehbar aussehen und funktionieren.
 - Wenn irgendein Bestandteil den Fokus erhält, dann löst dies nicht eine Änderung des Kontextes aus (STUFE A).
 - Navigationsmechanismen, die auf mehreren Webseiten innerhalb eines Satzes von Webseiten wiederholt werden, treten jedes Mal, wenn sie wiederholt werden, in der gleichen relativen Reihenfolge auf (STUFE AA).
 - Wenn eine Webseite einen Hilfemechanismus enthält und dieser auf mehreren Webseiten innerhalb einer Reihe von Webseiten wiederholt wird, erscheint er in der gleichen Reihenfolge in Bezug auf andere Seiteninhalte [...] (STUFE A).
 - ...
 - **Richtlinie 3.3: Hilfestellung bei der Eingabe:** Hilf den Benutzer*innen dabei, Fehler zu vermeiden und zu korrigieren.
 - Wenn ein Eingabefehler automatisch erkannt wird, dann wird das fehlerhafte Element identifiziert und der Fehler wird den Benutzer*innen in Textform beschrieben (STUFE A).
 - Wenn der Inhalt eine Eingabe durch die Benutzer*innen verlangt werden Beschriftungen (Labels) oder Anweisungen bereitgestellt (STUFE A).
 - Es gibt eine kontextsensitive Hilfe (STUFE AAA).
 - Informationen, die Nutzer*innen zuvor eingegeben haben oder die ihnen zur Verfügung gestellt wurden und die im selben Vorgang erneut eingegeben werden müssen, werden entweder automatisch ausgefüllt, oder für die Benutzer*innen zur Auswahl verfügbar [...] (STUFE A).
 - ...
4. **Prinzip 4: Robust** – Inhalte müssen so robust sein, dass sie von einer Vielzahl von User Agents, einschließlich assistiver Technologien, interpretiert werden können.
- **Richtlinie 4.1: Kompatibel:** Maximiere die Kompatibilität mit aktuellen und zukünftigen User Agents, einschließlich assistiver Technologien.
 - Für alle Komponenten der Benutzeroberfläche (einschließlich, aber nicht beschränkt auf: Formularelemente, Links und von Skripten generierte Komponenten) können Name und Rolle programmatisch festgelegt werden; Zustände, Eigenschaften und Werte, die von Benutzer*innen eingestellt werden können, können programmatisch eingestellt werden; und die Benachrichtigung über Änderungen an diesen Elementen ist für User Agents, einschließlich assistiver Technologien, verfügbar (STUFE A).
 - In Inhalten, die mit Auszeichnungssprachen implementiert wurden, können Statusmeldungen programmatisch durch Rollen oder Eigenschaften

bestimmt werden, so dass sie dem Nutzer von assistiven Technologien präsentiert werden können, ohne dass sie den Fokus erhalten (STUFE AA).

– ...

Dokumente zur Umsetzung

Zusätzlich zu den WCAG 2.2 Richtlinien existieren Dokumente die bei der Umsetzung unterstützen:

- *How to Meet WCAG (Quick Reference)*¹²,
- *Understanding WCAG 2.2*¹³,
- *Techniques for WCAG 2.2*¹⁴.

6.3.3 Evaluierung mit den WCAG 2.2

Die eigene Webseite auf Accessibility-Probleme zu überprüfen ist (leider) kein komplett automatisierbarer Prozess, sondern erfordert primär das Verständnis und das Hintergrundwissen von Entwickler*innen. Das W3C stellt jedoch Materialien und Tools zur Verfügung, die den Prozess vereinfachen und vor allem anleiten können.

Grobe Evaluierung mit Easy Checks

Die Seite *Easy Checks*¹⁵ stellt die häufigsten und am schnellsten behebbaren Probleme mit Accessibility übersichtlich dar. Die Rationale hinter dieser Liste ist, dass sie schnell abzuarbeiten ist somit grundsätzliche Probleme einfach eliminieren kann. Trotzdem kann die eigene Seite nach dem Abarbeiten der Easy Checks-Liste noch Accessibility-Probleme aufweisen, weshalb eine genauere Evaluierung sinnvoll ist.

Easy Checks besteht aus dreizehn Punkten, unterteilt in drei Kategorien, die auf der zugehörigen Seite mit Beispielen und technischen Tipps erläutert werden, sowie Bookmarklets zum Überprüfen beinhalten. Die Liste ist in Tabelle 6.1 dargestellt.

Detaillierte Evaluierung mit WCAG-EM

Die *Website Accessibility Conformance Evaluation Methodology* (WCAG-EM) [109] ist ein vom W3C strukturierter Prozess, um die Accessibility von Webseiten zu evaluieren. Er besteht aus fünf Schritten, die in Abbildung 6.2 visualisiert sind und im Folgenden kurz erklärt werden.

1. **Den Bereich der Evaluierung festlegen:** Welche *Seite* wird nach welchem *WCAG Erfüllungsgrad* in welchem *Endgerät* und mit welchen optionalen *Zusatzkriterien* evaluiert?

¹²<https://www.w3.org/WAI/WCAG22/quickref/>

¹³<https://www.w3.org/WAI/WCAG22/Understanding/>

¹⁴<https://www.w3.org/WAI/WCAG22/Techniques/>

¹⁵<https://www.w3.org/WAI/test-evaluate/easy-checks/>

Tabelle 6.1: Die Easy Checks-Liste des W3C mit ihren 13 Punkten.

Allgemeine Checks	
Alternativtext für Bilder	Jedes Bild benötigt eine aussagekräftige Textalternative.
Seitentitel	Die Seite soll einen sinnvollen und beschreibenden Titel aufweisen.
Überschriften	Überschriften sollen korrekt verwendet werden und eine sinnvolle Outline generieren.
Kontraste	Der Kontrast zwischen Vorder- und Hintergrund soll ausreichend sein. Die Darstellung muss auch in Graustufen noch funktionieren.
Skip Link	Die Seite hat einen Skip Link als erstes interaktives Element, um direkt zum Hauptinhalt zu springen.
Sichtbarer Keyboard-Fokus	Jedes interaktive Element auf der Seite soll ein offensichtliches, visuelles Styling aufweisen, wenn es den Fokus erhält.
Sprache der Seite	Die Sprache der Seite soll korrekt (mit dem <code>lang</code> -Attribut beim <code><html></code> -Element) deklariert sein.
Zoom	Die Seite soll auch bei 200 % Zoomfaktor noch korrekt dargestellt werden.
Audiovisuelle Checks	
Untertitel	Videos weisen Untertitel auf und diese sind von guter Qualität.
Transkripte	Medien (Videos und Audio) weisen ein Transkript auf, das die Inhalte verschriftlich darstellt.
Audiobeschreibung	Videos weisen eine Audiobeschreibung auf, die die visuellen Inhalte beschreibt.
Formular-Checks	
Labels	Formularelemente sind mit Labels versehen.
Pflichtfelder	Pflichtfelder sind klar gekennzeichnet, etwa mit einem Sternchen.

- (a) **Seite:** Welche Webseite wird überhaupt evaluiert und welche Art von Unterseiten (d. h. Webpages) davon werden inkludiert und welche nicht? Mobile Seiten, verschiedene Sprachversionen etc.
- (b) **Erfüllungsgrad:** Wird nach Level A, AA oder AAA evaluiert?
- (c) **Endgerät:** In welchem Browser wird getestet? Kommt ein Screenreader zum Einsatz, wenn ja, welcher?

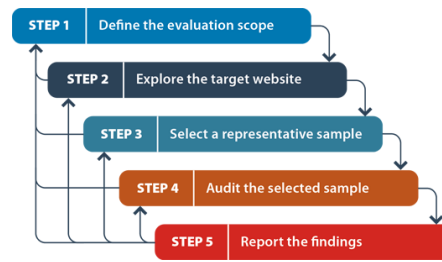


Abbildung 6.2: Der standardisierte Accessibility-Evaluationsprozess, definiert durch das W3C. Quelle: [109].

- (d) **Zusatzkriterien:** Werden nur Fehler aufgezeigt oder Lösungen angeboten? Gibt es spezielle Anwendungsfälle, die berücksichtigt werden müssen etc.
2. **Die Webseite untersuchen:** Die Struktur der Seite soll untersucht und verstanden werden. Welche *Unterseiten* gibt es? Welche *Funktionalitäten* bietet die Seite? Welche *Typen* von Seiten gibt es und welche *Technologien* werden verwendet?
 - (a) **Wichtige Unterseiten:** Welche wichtigen Unterseiten hat die aktuelle Webseite? Diese sind in der Regel über die Navigation erreichbar.
 - (b) **Funktionalitäten:** Was kann auf diesen Seiten getan werden? Zum Beispiel ein Produkt in einem Shop kaufen, Formulare ausfüllen und absenden oder sich registrieren.
 - (c) **Typen:** Welche Arten von Seiten gibt es? Zum Beispiel Seiten mit unterschiedlicher Struktur, Seiten mit verschiedenen Technologien, Seiten mit unterschiedlichen Interface Elementen etc.
 - (d) **Technologien:** Welche Technologien wurden verwendet? Etwa HTML, CSS, JavaScript, PDF, SVG etc.
3. **Ein repräsentatives Sample erstellen:** Nicht alle Seiten können oft evaluiert werden. Daher muss ein *Sample erstellt* werden, das Pages umfasst die aus den wichtigen Unterseiten mit den entsprechenden Funktionalitäten und Typen sowie Technologien besteht. Zusätzlich wird ein *zufälliges Sample* erstellt. Gibt es *komplette Prozesse*, müssen diese extra zusammengestellt werden.
 - (a) **Zusammengestelltes Sample:** Dieses muss alle in Schritt 2 definierten Teile (wichtige Seiten, Funktionalitäten, Typen und Technologien) enthalten, um repräsentativ zu sein.
 - (b) **Zufälliges Sample:** Um zu verifizieren, dass das zusammengestellte Sample repräsentativ ist, werden zusätzlich noch Seiten zufällig ausgewählt. Die Ergebnisse beider Samples sollten korrelieren.
 - (c) **Komplette Prozesse:** Gibt es komplette Prozesse, wie etwa einen Bestellvorgang in einem Onlineshop muss das Sample alle Teilseiten davon beinhalten.
4. **Das Sample prüfen:** Alle Seiten in den Samples werden nun anhand der WCAG-Kriterien *geprüft*. Beim zufälligen Sample dürfen keine neuen Typen, Technologien

oder Funktionalitäten erkannt werden, da sonst das zusammengestellte Sample nicht vollständig wäre. Komplette Prozesse müssen vollständig evaluiert werden.

- (a) **Prüfen:** Mit Hilfe der WCAG-Richtlinien bzw. der Quick Reference *How to Meet WCAG 2.2*¹⁶ werden die einzelnen Erfolgskriterien, die für den Erfüllungsgrad (A, AA oder AAA) relevant sind, überprüft.
- 5. **Die Resultate dokumentieren:** Die einzelnen Schritte müssen dokumentiert und das Ergebnis aufbereitet werden. Ein solcher standardisierter Bericht kann mit dem *WCAG-EM Report Tool*¹⁷ erstellt werden und enthält alle wichtigen Daten aus den einzelnen Schritten.

6.4 Accessible Rich Internet Applications (WAI-ARIA)

Die *Accessible Rich Internet Applications* [59] sind Richtlinien, die ein Framework zur Verbesserung der Accessibility von Webinhalten bereitstellen.

6.4.1 Allgemeines

Webseiten werden immer moderner und benutzer*innenfreundlicher. Statische Seiten werden mehr und mehr durch dynamische Ansätze ersetzt, neue und interaktive Elemente – oft Widgets genannt – ermöglichen Drag & Drop, zeigen Tabs auf einer Webseite an oder aktualisieren Teile der Webseite im Hintergrund, ohne diese neu laden zu müssen.

Für derartige Effekte ist fast ausnahmslos immer eine Skriptsprache wie JavaScript nötig, mit der eine fertig geladene HTML-Seite im Nachhinein noch dynamisch verändert werden kann. So können etwa neue Elemente erzeugt oder andere verändert und entfernt werden. Mit Hilfe verschiedener, vordefinierter JavaScript Komponenten (aus Frameworks wie etwa Bootstrap¹⁸) können etwa auch spezielle Elemente in eine Webpage integriert werden, die mit „HTML-Bordmitteln“ nicht möglich wären.

Diese Flexibilität geht jedoch sehr stark auf Kosten der Accessibility, da es für unterstützende Technologien normal nicht möglich ist, diese dynamischen Änderungen mitzuverfolgen und entsprechend darauf zu reagieren. Somit ist es etwa für einen Screenreader nicht möglich, dynamisch generierte Inhalte vorzulesen, da sie dieser schlichtweg nicht erkennt. Selbst wenn die Inhalte erkannt werden, so fehlen oft jedoch semantische Informationen, die es dem Screenreader ermöglichen zu erkennen, ob es sich bei dem generierten Inhalt etwa um ein Menü, einen Slider oder ein völlig anderes Element handelt. Nur wenn dies bekannt ist, kann der Screenreader das Element entsprechend behandeln. Die *WAI-ARIA*¹⁹ schaffen hierfür Abhilfe.

6.4.2 WAI-ARIA 1.2

Die Spezifikation zu *Accessible Rich Internet Applications 1.2* [59] ist seit Juni 2023 eine W3C Recommendation (WAI-ARIA 1.3 [60] ist noch im Status eines Working Drafts

¹⁶<https://www.w3.org/WAI/WCAG22/quickref/>

¹⁷<https://www.w3.org/WAI/eval/report-tool/>

¹⁸<https://getbootstrap.com/>

¹⁹<https://www.w3.org/WAI/standards-guidelines/aria/>

und wurde erst begonnen). Sie beschreibt ein Accessibility Framework, das Webentwickler*innen eine Reihe von *Rollen*, *Zuständen* und *Eigenschaften* bietet, um dynamischen Inhalten Bedeutungen zu geben. Diese Eigenschaften werden als HTML-Attribute in bestehendes Markup integriert und erlauben so den unterstützenden Technologien, darauf zu reagieren.

Rollen

Rollen werden mit dem Attribut **role** hinzugefügt und werden in fünf Kategorien eingeteilt:

Widget-Rollen: Diese Rollen kennzeichnen Elemente als standalone Widgets oder als Teile von größeren, zusammengesetzten Widgets. Folgende Rollen sind verfügbar:

- **button** (Schaltfläche),
- **checkbox** (Checkbox),
- **gridcell** (Rasterzelle),
- **link** (Link),
- **menuitem** (Option in einem Menü),
- **menuitemcheckbox** (Checkbox in einem Menü),
- **menuitemradio** (ein Radiobutton in einem Menü),
- **option** (Option in einer Auswahlliste),
- **progressbar** (Fortschrittsanzeige),
- **radio** (Radiobutton),
- **scrollbar** (Scrollleiste),
- **searchbox** (Suchfeld),
- **separator** (Trennlinie),
- **slider** (Slider),
- **spinbutton** (Schaltfläche zum Wählen diskreter Nummernwerte),
- **switch** (binäre Ein/Aus-Checkbox),
- **tab** (Tab/Reiter),
- **tabpanel** (Behälter für Tabs),
- **textbox** (Textbox) und
- **treeitem** (Option in einem Baum).

Diese Elemente sind in oft in anderen, so genannten Container-Elementen enthalten. Für diese gibt es die folgenden Rollen:

- **combobox** (Kombinationselement aus Textbox und z. B. Listbox),
- **grid** (Raster),
- **listbox** (Auswahl mehrere Einträge aus einer Liste),
- **menu** (Auswahlmenü),
- **menubar** (Menüleiste, enthält meist das Menü),
- **radiogroup** (Gruppe von Radiobuttons),
- **tablist** (Container für mehrere Tab-Panels),

- **tree** (Baumstruktur, die aus- und einklappbar ist) und
- **treegrid** (Raster, der aus- und einklappbar ist).

Dokumenten-Rollen: Diese Rollen beschreiben Strukturen, die Inhalte auf einer Seite organisieren. Dokumentstrukturen sind normalerweise nicht interaktiv:

- **application** (generelle Applikation, die nicht durch einen Widget abgedeckt ist),
- **article** (eigenständiger Artikel),
- **blockquote** (Zitat aus einer anderen Quelle),
- **caption** (Beschriftung für eine Abbildung, Tabelle, Grid oder Treegrid),
- **cell** (Tabellenzelle),
- **columnheader** (Header für eine Tabellenspalte),
- **definition** (Definition eines Begriffs),
- **deletion** (als entfernt markierte, oder zur Entfernung vorgeschlagene Inhalte),
- **directory** (Verzeichnis z. B. von Personen),
- **document** (Dokument, das (vor-)gelesen werden soll),
- **emphasis** (betonter Inhalt),
- **feed** (Liste an Beiträgen),
- **figure** (Abbildung),
- **generic** (namenloses Containerelement ohne semantische Bedeutung),
- **group** (Set an UI-Elementen, die nicht vom Screenreader vorgelesen werden sollen),
- **heading** (Überschrift),
- **img** (Bild),
- **insertion** (als hinzugefügt markierte Inhalte),
- **list** (Liste),
- **listitem** (Element einer Liste),
- **math** (mathematischer Ausdruck),
- **meter** (Maß innerhalb eines skalen Bereichs),
- **none** (Element wird von unterstützenden Technologien ignoriert),
- **note** (Anmerkung),
- **paragraph** (Absatz mit Inhalten),
- **presentation** (Synonym für **none**),
- **row** (Tabellenreihe),
- **rowgroup** (mehrere Reihen einer Tabelle),
- **rowheader** (Header für eine Tabellenreihe),
- **separator** (Trennlinie),
- **strong** (wichtige, ernste oder dringende Inhalte),
- **subscript** (tiefgestellte Zeichen),
- **superscript** (hochgestellte Zeichen),
- **table** (Tabelle),
- **term** (Ausdruck mit Definition),

- **time** (Datum und/oder Uhrzeit),
- **toolbar** (Toolbar) und
- **tooltip** (Tooltip-Popup).

Landmark-Rollen: Diese Rollen beschreiben wichtige Bereiche auf der Seite, zu denen User*innen leicht Zugriff bekommen sollten. Folgende gibt es dabei:

- **banner** (Seitenweiter Bereich etwa mit einem Logo),
- **complementary** (Zusatzinformationen),
- **contentinfo** (Informationen zum Dokument wie Copyright),
- **form** (Formular),
- **main** (Hauptbereich),
- **navigation** (Navigation),
- **region** (genereller Abschnitt) und
- **search** (Suche).

Live Region-Rollen: Diese Rollen kennzeichnen Bereiche auf der Seite, die sich dynamisch ändern können:

- **alert** (wichtige Informationen und Meldungen),
- **log** (Live-Log bei dem neue Informationen erscheinen und alte verschwinden),
- **marquee** (schnell ändernde, unwichtige Informationen),
- **status** (weniger wichtige Statusinformationen) und
- **timer** (Timer oder Countdown).

Window-Rollen: Diese Rollen definieren Fenster innerhalb des Browsers:

- **alertdialog** (Alert-Dialogfenster) und
- **dialog** (Dialogfenster).

Anwendung von Rollen

Rollen kommen dort zum Einsatz, wo semantische HTML-Elemente nicht verfügbar sind oder nicht entsprechend eingesetzt werden können. So gibt es keine HTML-Elemente um Tabs in einer Seite anzuzeigen. Das Markup hierfür könnte mit Hilfe einer geordneten Liste erstellt werden. Diese hat allerdings keinerlei semantische Informationen über Tabs, daher werden die entsprechenden WAI-ARIA Rollen bei den Elementen hinzugefügt. **tablist** beim `<ol>`-Element kennzeichnet die gesamte Tab-Liste, jedes `<li>`-Element wird mittel **tab** zum Reiter und die `<div>`-Elemente mit **tabpanel** werden als Inhaltsbereiche des Tabs ausgewiesen. Abbildung 6.3 zeigt eine mögliche Darstellung der Tabs.

```

1 <ol role="tablist">
2   <li id="ch1Tab" role="tab">
3     <a href="#ch1Panel">Chapter 1</a>
4   </li>
5   <li id="ch2Tab" role="tab">
6     <a href="#ch2Panel">Chapter 2</a>

```

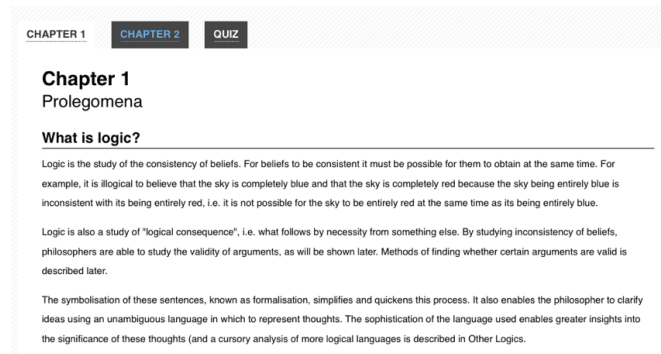


Abbildung 6.3: Visuell sind die Tabs sofort erkennbar. Im Markup müssen WAI-ARIA-Rollen nachhelfen. Quelle: [34].

```

7   </li>
8   <li id="quizTab" role="tab">
9     <a href="#quizPanel">Quiz</a>
10  </li>
11 </ol>
12
13 <div>
14   <div id="ch1Panel" role="tabpanel" aria-labelledby="ch1Tab">Chapter 1 content goes
15     here</div>
16   <div id="ch2Panel" role="tabpanel" aria-labelledby="ch2Tab">Chapter 2 content goes
17     here</div>
18   <div id="quizPanel" role="tabpanel" aria-labelledby="quizTab">Quiz content goes
19     here</div>
20 </div>

```

Zustände und Eigenschaften

Zustände und Eigenschaften sind Attribute (beginnend mit `aria-`), die zu HTML-Elementen hinzugefügt werden und ergänzen Rollen durch Zusatzinformationen. Sie sind im Konzept sehr ähnlich, Zustände sind jedoch Attribute deren Wert sich ändern kann (z. B. `aria-checked="true"` zu `false`, um den Zustand einer Checkbox anzugeben), während Eigenschaften in der Regel unverändert bleiben (z. B. `aria-labelledby`, um zu beschreiben, durch welches Element ein anderes beschriftet wird – siehe dazu Code zu den Tabs im Abschnitt vorher). Generell hat diese Unterscheidung jedoch keine Konsequenzen, weshalb hier auch einfach von ARIA-Attributen gesprochen wird.

Attribute werden in vier Kategorien eingeteilt:

Widget-Attribute: Diese Attribute finden sich in der Regel bei Widgets, d. h. Elementen mit den entsprechenden Rollen. Die Attributwerte werden hier nicht aufgelistet und können aus der WAI-ARIA Spezifikation [59] entnommen werden:

- `aria-autocomplete` (Element kann Autocompletion vornehmen),
- `aria-checked` (Element ist angehakt oder nicht),
- `aria-disabled` (Element ist deaktiviert oder nicht),

- **aria-errormessage** (Element stellt eine Fehlermeldung dar),
- **aria-expanded** (Widget ist zu- oder aufgeklappt),
- **aria-haspopup** (Element zeigt ein Popup an),
- **aria-hidden** (Element ist vor der unterstützten Technologie versteckt),
- **aria-invalid** (eingegebener Wert ist ungültig),
- **aria-label** (Element ist ein Label für ein anderes),
- **aria-level** (definierte die Hierarchietiefe eines Elements),
- **aria-modal** (Element ist ausschließlich modal, d. h. immer im Vordergrund),
- **aria-multiline** (mehrzeilige Eingaben sind möglich oder nicht),
- **aria-multiselectable** (mehrere Elemente können gleichzeitig gewählt werden),
- **aria-orientation** (Ausrichtung eines Elements),
- **aria-placeholder** (Platzhaltertext),
- **aria-pressed** (Button ist gedrückt),
- **aria-readonly** (Element ist nicht editierbar),
- **aria-required** (Element muss ausgefüllt werden),
- **aria-selected** (Element ist ausgewählt),
- **aria-sort** (Sortierung eines Widgets),
- **aria-valuemax** (der maximal erlaubte Eingabewert in einem Feld),
- **aria-valuemin** (der minimal erlaubte Eingabewert),
- **aria-valuenow** (der aktuell eingegebene Wert) und
- **aria-valuetext** (textuelle Beschreibung des eingegebenen Werts).

Live-Region-Attribute: Diese Attribute werden im Umgang mit Live-Regions, d. h. Bereiche deren Inhalte sich aufgrund von externen Ereignissen ändern können, verwendet:

- **aria-atomic** (assistive Technologien lesen alles oder nur die Änderungen neu vor),
- **aria-busy** (Element wird gerade verändert),
- **aria-live** (Element wird mit Inhalten aktualisiert werden) und
- **aria-relevant** (assistive Technologien lesen nur relevante Änderungen vor).

Drag-und-Drop-Attribute: Diese Attribute sind speziell für Drag & Drop Widgets konzipiert:

- **aria-dropeffect** (was passiert beim Loslassen eines Objekts) und
- **aria-grabbed** (Objekt wurde aufgenommen).

Beziehungsattribute: Diese Attribute drücken Beziehungen zwischen Elementen aus:

- **aria-activedescendant** (identifiziert das aktive Unterelement mit Fokus),
- **aria-colcount** (Anzahl der Spalten in einer Tabelle, Grid oder Treegrid),
- **aria-colindex** (Index der aktuellen Spalte),
- **aria-colspan** (Zelle erstreckt sich über mehrere Spalten),
- **aria-controls** (identifiziert das Element, das das aktuelle Element kontrolliert z. B. der Tab das Tabpanel),

- **aria-describedby** (identifiziert das Element, welches das aktuelle beschreibt),
- **aria-details** (identifiziert das Element, das Details über das aktuelle enthält),
- **aria-errormessage** (identifiziert das Objekt, das eine Fehlermeldung für das aktuelle beinhaltet),
- **aria-flowto** (identifiziert das nächste Element, wenn dieses visuell neu angeordnet wurde),
- **aria-labelledby** (identifiziert das Element, das als Label für das aktuelle agiert),
- **aria-owns** (identifiziert das Element, das das aktuelle beinhaltet),
- **aria-posinset** (Index des Elements in einem Set von Listelementen oder Treeitems),
- **aria-rowcount** (Anzahl von Reihen in einer Tabelle, einem Grid oder Treegrid),
- **aria-rowindex** (Index der aktuellen Zeile),
- **aria-rowspan** (Zelle erstreckt sich über mehrere Zeilen) und
- **aria-setsize** (Anzahl der Elemente in einem Set von Listelementen oder Treeitems).

Anwendung von Zuständen und Eigenschaften

Zustände und Eigenschaften – oder kurz Attribute – werden dort verwendet, wo Zuordnungen gemacht werden müssen (um Zusammenhänge darzustellen) oder Dinge sich ändern. Folgendes Beispiel beschreibt einen Tooltip für ein Formularfeld. Der Tooltip wird in einem `<div>`-Element hinterlegt, über JavaScript kann dann gesteuert werden, ob er angezeigt wird (wenn die Maus über dem Formularfeld ist), oder nicht. Dementsprechend muss mit WAI-ARIA-Attributen gekennzeichnet werden, ob der Tooltip sichtbar ist oder nicht (**aria-hidden**). Die Werte der Attribute werden dabei mittels JavaScript mitverändert. Zusätzlich wird noch angegeben, dass der Tooltip das Formularfeld beschreibt (**aria-describedby**) und dass das `<label>`-Element das Formularfeld beschriftet (**aria-labelledby**).

```

1 <div class="text">
2   <label id="tp1-label" for="first">First Name:</label>
3   <input type="text" id="first" name="first" size="20" aria-labelledby="tp1-label"
      aria-describedby="tp1" aria-required="false">
4   <div id="tp1" class="tooltip" role="tooltip" aria-hidden="true">Your first name is
      optional</div>
5 </div>

```

Ist das ARIA-Attribut **true**, wird der Tooltip einfach mittels CSS ausgeblendet:

```

1 div.tooltip[aria-hidden="true"] {
2   display: none;
3 }

```

6.4.3 WAI-ARIA einsetzen

Viele der Rollen und Attribute in WAI-ARIA decken sich mit Elementen aus dem HTML Living Standard. Diese Elemente sollten immer zuerst verwendet werden, dafür muss *kein* WAI-ARIA-Attribut hinzugefügt werden. Folgendes Markup ist daher redundant und *nicht* notwendig:

```
1 <nav role="navigation">...</nav>
```

Ebenso sollen durch die Attribute bestehende Semantiken nicht umdefiniert werden. Anstatt einer `<h2>`-Überschrift die Rolle eines Tabs zu geben, ist es besser, ein allgemeines, bedeutungsloses `<div>`-Element zu nehmen:

```
1 <div role="tab">Header Tab</div>
```

Keine ARIA sind besser als schlechte ARIA

ARIA Rollen, Zustände und Eigenschaften entsprechen in etwa CSS-Anweisungen für unterstützende Technologien. Sie bestimmen, wie diese Technologien Webinhalte für sie wiedergeben. Falsch angewandte ARIA können daher auch verheerende Effekte auf Personen mit Beeinträchtigungen haben. Die WAI hat daher zwei Prinzipien definiert, die vor der Verwendung von ARIA bedacht werden sollten:

1. **Eine Rolle ist ein Versprechen:** Ein Element wie `<div role="button">Place Order</div>` beinhaltet, dass das `<div>`-Element sich wie ein HTML-Button verhält, d. h. auch etwa JavaScript-Code für korrekte Bedienung mit dem Keyboard enthält. Fehlt dies, verlassen sich User*innen darauf, dass dieses Element sich wie ein Button verhält und werden dabei enttäuscht.
2. **ARIA können sowohl verdecken als auch verbessern – dies ist gleichzeitig mächtig und gefährlich:** ARIA kann die ursprüngliche Semantik von Elementen überschreiben oder auch neue Bedeutungen schaffen. Dies ist zwar sehr mächtig, kann bei falscher Anwendung aber auch der Barrierefreiheit schaden.

Der *ARIA Authoring Practice Guide (APG)*²⁰ bietet praktische Beispiele zur Implementierung von gängigen Webkomponenten. Hier finden sich auch die zwei soeben beschriebenen Prinzipien²¹. Ebenso finden sich Informationen zur Umsetzung von ARIA im Dokument *Using ARIA*²².

6.5 Unterstützende Technologien

Damit beeinträchtigte Personen auf Webinhalte zugreifen können, benötigen sie mitunter Technologien, die etwa einen Bildschirm ersetzen bzw. bei der Betrachtung unterstützend eingreifen. Gebräuchliche Werkzeuge sind Screenreader, die den Bildschirminhalt vorlesen oder über Braillezeilen in Brailleschrift ausgeben.

Um Screenreader selbst einfach zu testen ist vor allem *NVDA*²³ (Non Visual Desktop Access) empfehlenswert, da es sich um einen der wenigen Open Source Screenreader handelt. Andere, bekannte Produkte wie etwa der Marktführer *JAWS*²⁴ sind kostenpflichtig.

Weiters haben die bekannten Betriebssysteme meist Bildschirmlesetechnologien bereits integriert. So existiert für macOS etwa *VoiceOver*²⁵, Windows 11 haben den *Micro-*

²⁰<https://www.w3.org/WAI/ARIA/apg/>

²¹<https://www.w3.org/WAI/ARIA/apg/practices/read-me-first/>

²²<https://www.w3.org/TR/using-aria/>

²³<https://www.nvaccess.org/>

²⁴<https://www.freedomscientific.com/Products/software/JAWS/>

²⁵<https://support.apple.com/de-at/guide/voiceover-guide/welcome/web>

soft Narrator (auf Deutsch einfach „Sprachausgabe“ genannt)²⁶ integriert. Diese Tools lassen sich ohne weitere Installation nützen und erlauben ein Testen und eigenständiges Erfahren von Screenreader-Technologien.

6.5.1 Testen mit unterstützenden Technologien

Um einen Eindruck zu bekommen, wie Personen, die auf Screenreader angewiesen sind, die eigene Webseite wahrnehmen, empfiehlt es sich, eines der soeben genannten Werkzeuge zu installieren und die eigene Seite damit zu betrachten. So lassen sich sehr schnell Probleme mit unzureichenden Alternativtexten für Bilder, problematische Lesereihenfolge, Probleme mit Formularen oder weitere Accessibility-Aspekte herausfinden.

Einige Überlegungen und einen guten Startpunkt mit weiteren Links zu diversen Technologien findet sich unter https://webaim.org/articles/screenreader_testing/.

6.6 Weiterführende Literatur

Die Literatur zu Web Accessibility ist spärlich. Es existieren meist Bücher die zehn Jahre oder älter sind. Mit [20] gibt es jedoch ein sehr neues Werk, das Barrierefreiheit im Web auf einfache Art erklärt und dabei auch die WCAG 2.2 bereits berücksichtigt.

Aktuell ist ebenfalls [16], in dem die Autorin zunächst verschiedene Probleme der Barrierefreiheit erläutert und dann mit Hilfe von Webtechnologien wie ARIA oder CSS Lösungen für diese Probleme anbietet. Auch in [32] finden sich die Themen WCAG und WAI-ARIA zusammen mit der rechtlichen Dimension.

Wer ein praxisorientiertes Buch mit konkreten Vorschlägen zur Gestaltung von Komponenten sucht, kann auf [33] zurückgreifen. Das Buch behandelt weniger die Grundlagen sondern liefert konkrete Codebeispiele. Es wurde von einem Österreicher geschrieben.

Darüber hinaus sind die Standard-Dokumente zu WCAG [108] und WAI-ARIA [59] die beste Ressource für Informationen.

²⁶<https://support.microsoft.com/de-de/windows/vollst%C3%A4ndige-anleitung-f%C3%BCr-die-sprachausgabe-e4397a0d-ef4f-b386-d8ae-c172f109bdb1>

Vorlesung 7

Datenformate

In der modernen Informatik, insbesondere im Bereich der Webentwicklung und verteilten Systeme, ist die persistente Speicherung und der Austausch von Datenstrukturen eine fundamentale Herausforderung. Applikationen halten Daten zur Laufzeit typischerweise in komplexen, sprachspezifischen Strukturen im Arbeitsspeicher, beispielsweise als Arrays oder Objekte in Java. Um diese Daten über ein Netzwerk zu übertragen (z. B. von einem Server zu einem Client) oder dauerhaft auf einem Datenträger zu speichern, wird ein Datenformat benötigt. Dieses kann binär oder textbasiert sein. Während binäre Formate zwar für die maschinelle Verarbeitung aufgrund ihrer Effizienz sehr gut geeignet sind, können sie nicht von Menschen einfach so gelesen werden. Textbasierte Formate haben dieses Problem nicht und haben sich daher vor allem im Web etabliert. Sie mit jedem Texteditor geöffnet, ihr Inhalt betrachtet und auch bearbeitet werden.

Dieses Kapitel beschäftigt sich mit den drei wichtigsten, textbasierten Datenformaten im Web: XML, JSON und YAML und stellt ihre Syntax und Einsatzgebiete dar.

7.1 XML

XML, kurz für *Extensible Markup Language*, ist eine vom W3C standardisierte Markup-Sprache [91]. Die Syntax ähnelt der bereits aus HTML bekannten – `<tag>Inhalt</tag>` – es gibt jedoch einige fundamentale Unterschiede.

7.1.1 Einführung

Bei XML handelt es sich um eine so genannte *Meta-Markup-Sprache*. Dies bedeutet, dass die Sprache dazu verwendet wird, eine Dokumentenbeschreibungssprache zu definieren. Da also mit XML eine eigene Sprache definiert wird, gibt es keine fixe Anzahl an Tags (wie bei HTML), sondern es können mit der Sprache beliebige Elemente festgelegt werden. Genau so, wie es der jeweilige Einsatzzweck erfordert. Dies wird auch durch den Ausdruck „extensible“ (deutsch: erweiterbar) im Namen der Sprache unterstrichen.

Eine solche selbst definierte Menge an eigenen Tags und Attributen wird auch eine *XML-Anwendung* (engl. „XML application“) genannt.

XML ist (wie auch HTML) eine Untermenge der *Standard Generalized Markup Lan-*

*guage*¹ (SGML). Da sich SGML jedoch als viel zu groß und komplex erwies, wurde beschlossen, mit XML eine einfachere, jedoch nicht minder flexible Meta-Markupsprache zu entwickeln.

Die Grammatik von XML ist sehr streng. Alle XML-Anwendungen müssen dieser Grammatik folgen (siehe Abschnitt 7.1.3).

Da XML, wie bereits einführend angemerkt, ein *Textformat* ist, ist es leicht lesbar und verarbeitbar. Dies eröffnet nicht nur die Möglichkeit, es plattformübergreifend verwenden zu können, auch eine gewisse Langlebigkeit ist dadurch garantiert. Binäre Datenformate haben oft das Problem, dass sie zwar auf einer Plattform verarbeitet werden, auf anderen Systemen jedoch nicht gelesen werden können. Und selbst wenn eine Datei prinzipiell gelesen werden kann, ist noch nicht garantiert, dass ihr Inhalt interpretiert werden kann. So gibt es von den Mondlandungen in den späten 1960er und frühen 1970er Jahren zwar noch Bänder mit den aufgezeichneten (binären) Daten, die Information, wie diese Datenströme zu lesen sind, existieren jedoch nicht mehr. Somit sind die Informationen effektiv verloren [24]. Textdaten sind nicht mit diesen Problemen konfrontiert. Zwar können hier unterschiedliche Kodierungen zum Einsatz kommen, doch diese lassen sich im schlimmsten Fall per Trial-and-Error herausfinden.

Der (meist) einfache und hierarchische Aufbau ermöglicht darüber hinaus eine gute Strukturierung von Dokumenten. Dies macht das Format universell einsetzbar.

7.1.2 Ein XML-Beispieldokument

Im Folgenden wird der grundsätzliche Aufbau von XML-Dokumenten beschrieben. Dies soll anhand eines Beispieldokuments geschehen, das eine mögliche XML-Anwendung darstellt. Nachfolgende Abschnitte greifen auf dieses Beispiel zu, um Aspekte von XML – wie etwa die Grammatik – zu erläutern.

Ein Rezept könnte in XML wie folgt dargestellt werden:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <rezept quelle="https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386">
3   <gericht>Der klassische Tiroler Knödel</gericht>
4   <zutaten>
5     <zutat>
6       <ingredient>Semmelwürfel</ingredient>
7       <menge>350</menge>
8       <einheit>g</einheit>
9     </zutat>
10    <zutat>
11      <ingredient>Milch</ingredient>
12      <menge>200</menge>
13      <einheit>ml</einheit>
14    </zutat>
15    <zutat>
16      <ingredient>Eier</ingredient>
17      <menge>4</menge>
18      <einheit>Stück</einheit>
19    </zutat>
20    <zutat>
21      <ingredient>Speck</ingredient>
22      <menge>100</menge>

```

¹<https://de.wikipedia.org/wiki/SGML>

```

23     <einheit>g</einheit>
24   </zutat>
25   <zutat>
26     <ingredienz>Bergsteigerwurst</ingredienz>
27     <menge>100</menge>
28     <einheit>g</einheit>
29   </zutat>
30   <zutat>
31     <ingredienz>Butter</ingredienz>
32     <menge>20</menge>
33     <einheit>g</einheit>
34   </zutat>
35   <zutat>
36     <ingredienz>Zwiebel</ingredienz>
37     <menge>1</menge>
38     <einheit>Stück</einheit>
39   </zutat>
40   <zutat>
41     <ingredienz>Petersilie</ingredienz>
42     <menge>10</menge>
43     <einheit>g</einheit>
44   </zutat>
45   <zutat>
46     <ingredienz>Salz</ingredienz>
47     <menge>1</menge>
48     <einheit>Prise</einheit>
49   </zutat>
50   <zutat>
51     <ingredienz>Muskatnuss</ingredienz>
52     <menge>1</menge>
53     <einheit>Prise</einheit>
54   </zutat>
55   <zutat>
56     <ingredienz>Mehl</ingredienz>
57     <menge>80</menge>
58     <einheit>g</einheit>
59   </zutat>
60   <zutat>
61     <ingredienz>Schnittlauch</ingredienz>
62     <menge>10</menge>
63     <einheit>g</einheit>
64   </zutat>
65 </zutaten>
66 <zubereitung>
67   <schritt>Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen
        lassen.</schritt>
68   <schritt>Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in
        zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten
        und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen
        Zutaten eine feste, aber nicht zu feuchte Masse kneten.</schritt>
69   <schritt>Mit befeuchteten Händen daraus Knödel formen.</schritt>
70   <schritt>Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat
        der Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die
        Knödelmasse kneten.</schritt>
71   <schritt>Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.
        </schritt>
72   <schritt>Tiroler Knödel mit Schnittlauch bestreut servieren.</schritt>

```

```
73   </zubereitung>
74 </rezept>
```

Das Dokument – ein Rezept für überaus wohlschmeckende Tiroler Knödel – besitzt ein Wurzelement namens `<rezept>`, das alle anderen Elemente beinhaltet. Dieses Element hat zusätzlich ein Attribut `quelle`, das nähere Informationen über die Herkunft des Rezepts spezifiziert. Darunter folgen die Elemente `<gericht>`, `<zutaten>` und `<zubereitung>`, die alle auf der gleichen Ebene liegen.

`<zutaten>` besteht aus einer Reihe von `<zutat>`-Elementen, welche alle jeweils ein `<ingredienz>`-, ein `<menge>`- und ein `<einheit>`-Element enthalten und damit die Zutat genauer spezifizieren.

Das Element `<zubereitung>` enthält einzelne `<schritt>`-Elemente, welche die einzelnen Zubereitungsschritte textuell beinhalten.

7.1.3 XML-Grammatik

Wie in Abschnitt 7.1.2 ersichtlich, besteht ein XML-Dokument prinzipiell aus Text, der mit Tags wie etwa `<ingredienz>` ausgezeichnet wurde. Jeder Tag kann auch noch Attribute enthalten. Dies unterscheidet sich in der grundsätzlichen Verwendung nicht von HTML, der erste offensichtliche Unterschied ist jedoch der Tag-Name `<ingredienz>`, der in HTML so nicht existiert, da HTML nur eine endliche Menge an Tags definiert. Während also XML wesentlich freizügiger bei der Wahl der Tag-Namen ist, ist es wesentlich strikter, wenn es darum geht, wie diese geschrieben und platziert werden. Alle XML-Dokumente müssen nämlich *well-formed* sein, d. h. strikten grammatikalischen Regeln entsprechen.

Ein Wurzelement

Jedes XML-Dokument hat genau ein *Wurzelement* (engl. „root element“), also ein Element, das kein Elternelement besitzt. Im Beispiel in Abschnitt 7.1.2 ist `<rezept>` das Wurzelement.

Paarweise Tags

Jedes Element muss in XML ein öffnendes und schließendes Tag haben, also etwa `<ingredienz>Mehl</ingredienz>`. Standalone-Elemente wie in HTML (z. B. `<br>`) gibt es nicht. Hat ein Element keinen Inhalt, so wird es in sich geschlossen. In XML-Schreibweise (d. h. XHTML, siehe Abschnitt 7.1.6) lautet das Element für Zeilenumbrüche `<br />`. Die Schreibweise `<br></br>` ist äquivalent dazu, aber eher ungebräuchlich.

Keine überkreuzenden Tags

Wie in HTML gilt auch bei XML, dass Tags in der umgekehrten Reihenfolge geschlossen werden müssen, in der sie geöffnet wurden. Überkreuzungen sind also nicht erlaubt und führen zu einem Fehler. Somit gilt:

- **Korrekt:** `<a><b>...</b></a>`.
- **Inkorrekt:** `<a><b>...</a></b>`.

Attribute: Anführungszeichen und Einzigartigkeit

Jedes Element kann zwischen 0 und n *Attribute* besitzen. Ein Attribut ist wie in HTML eine Zusatzangabe in der Form `name="wert"`. Im Gegensatz zu HTML müssen in XML alle Attributwerte in Anführungszeichen stehen (auch dann, wenn der Wert des Attributs keine Leerzeichen enthält). Möglich sind doppelte Anführungszeichen, also z. B. `quelle="https://..."`, oder auch einfache Anführungszeichen (z. B. `quelle='https://...'`).

Darüber hinaus darf jedes Element nur ein Attribut mit demselben Namen haben. Würde das Rezept aus Abschnitt 7.1.2 auf mehreren Seiten im Web existieren und sollen mehrere solcher Quellen über das Attribut `quelle` angegeben werden, so wäre dies nicht möglich. Es darf nur ein Attribut `quelle` beim Element `<rezept>` geben. Die Lösung in diesem Fall wäre, die Quelle nicht als Attribut, sondern als Kindelement von Rezept anzugeben, denn davon darf es durchaus mehrere geben (wie es z. B. auch bei `<zutat>` ersichtlich ist). Die Diskussion, ob Daten als Attribut oder Element abgebildet werden sollen, ist eine sehr oft geführte. Als Faustregel kann folgendes gelten:

Attribute werden für Metainformationen, die ein Element ergänzen, verwendet. Elemente enthalten immer die eigentliche Information.

Generell sollte aber je nach XML-Anwendung selbst entschieden werden.

Namen von Elementen und Attributen

Bei der Verwendung von *Namen in XML* ist immer die Groß- und Kleinschreibung zu beachten. Die Tags `<rezept>` und `<Rezept>` sind zwei unterschiedliche Elemente.

Darüber hinaus können Namen alle alphanumerischen Zeichen enthalten, auch nicht-englische wie etwa deutsche Umlaute, kyrillische Zeichen oder griechische Symbole. Weiters sind der Underscore (`_`), der Bindestrich (`-`) und der Punkt (`.`) erlaubt. Wichtig ist, dass der Name eines Elements mit einem Buchstaben oder Underscore beginnen muss. Andere Interpunktionszeichen oder Leerzeichen sind nicht erlaubt.

Die folgenden Elemente sind also alle well-formed (aus [24]):

- `<Drivers_License_Number>98NY32</Drivers_License_Number>`,
- `<month-day-year>7/23/2001</month-day-year>`,
- `<first_name>Alan</first_name>`,
- `<_4-lane>I-610</_4-lane>`,
- `<téléphone>011 33 91 55 27 55 27</téléphone>`.

Inkorrekt sind hingegen die folgenden Elemente:

- `<Driver's_License_Number>98NY32</Driver's_License_Number>` (Apostroph),
- `<month/day/year>7/23/2001</month/day/year>` (Schrägstrich),
- `<first name>Alan</first name>` (Leerzeichen),
- `<4-lane>I-610</4-lane>` (beginnt mit Zahl).

Entitäten, PCDATA und CDATA

Der Inhalt eines Elements kann beliebig gestaltet werden, es gibt jedoch fünf Zeichen, die durch so genannte *Entitäten* (bekannt bereits aus HTML) ersetzt werden müssen:

- `<`: `<`; (wird sonst für den Beginn eines Tags angenommen),
- `>`: `>`; (wird sonst für das Ende eines Tags angenommen),
- `&`: `&`; (wird sonst als Beginnzeichen einer Entität interpretiert),
- `"`: `"`; (wird beim Attributwert als doppeltes Anführungszeichen erkannt),
- `'`: `'`; (wird beim Attributwert als einfaches Anführungszeichen erkannt).

Wie aus der Aufzählung ersichtlich ist, müssen diese fünf Elemente durch Entitäten ersetzt (engl. „escaped“) werden, da sie beim maschinellen Verarbeiten der XML-Datei (Parseen, mehr dazu im zweiten Semester in der Web Technologies Vorlesung) fälschlicherweise für etwas gehalten werden (z. B. Beginn eines Tags), das sie gar nicht darstellen. Es kann etwa ein mathematischer Ausdruck einfach nur `5 > 3` enthalten – wird das „Größer“-Zeichen nicht durch eine Entität ersetzt, so „stolpert“ der XML-Parser. Dieser sieht nämlich standardmäßig das gesamte Dokument als PCDATA (Parsed Character Data) an, also Inhalt, der vom Parser abgearbeitet wird. Dabei werden Verschachtelungen und andere XML-Eigenschaften erkannt und richtig interpretiert.

Enthält aber ein XML-Element nun viele Zeichen, die escaped werden müssen, so ist dies mitunter sehr aufwändig. Abhilfe schaffen kann ein so genannter CDATA-Abschnitt. CDATA steht für Character Data, auf Deutsch etwa „Zeicheninhalt“. Dieser wird vom Parser einfach als Inhalt wahrgenommen, er versucht darin nicht, etwas zu interpretieren. In einem CDATA-Block können also Zeichen wie `<`, `>` oder `&` gefahrlos verwendet werden. Ein CDATA-Block beginnt immer mit einer spitzen Klammer, danach folgt ein Ausrufezeichen, gefolgt von einer eckigen Klammer, dem Wort „CDATA“ und einer erneuten eckigen Klammer. Geschlossen wird der CDATA-Abschnitt mit zwei eckigen und einer spitzen Klammer.

Im folgenden XML-Ausschnitt wird etwa in einem XML-Dokument ein HTML-Element erläutert. Überall dort, wo zu ersetzende Zeichen vorkommen, wird ein CDATA-Block verwendet und somit das Escapen hinfällig:

```

1 <element>
2   <name><![CDATA[<h1>]]></name>
3   <description>Heading Level 1</description>
4   <example><![CDATA[<h1>Hello World</h1>]]></example>
5 </element>
```

Kommentare

Kommentare in XML werden in derselben Syntax wie in HTML verfasst. Das öffnende Kommentar wird mit spitzer Klammer, Ausrufezeichen und zwei Bindestrichen, das schließende Kommentar mit zwei Bindestrichen und einer spitzen Klammer erstellt. Kommentare dienen für Anmerkungen im Quellcode und sollten von Parsern oder anderen Programmen nicht ausgewertet werden. Sie dürfen überall im Inhaltsbereich des Dokuments vorkommen, nur nicht innerhalb von Tags. Das folgende Beispiel zeigt einen XML-Kommentar:

```

1 <!-- Das Root-Element -->
```

Processing Instructions

Während Kommentare als Zusatzinformationen für Autor*innen oder Leser*innen einer XML-Datei dienen, ermöglichen es die so genannten *Processing Instructions* Anweisungen an die Software zu geben, welche die XML-Datei maschinell verarbeitet (z. B. der Parser). Processing Instructions beginnen mit spitzer Klammer und einem Fragezeichen und werden durch ein Fragezeichen und eine spitze Klammer abgeschlossen.

Um etwa für XML-Dokumente anzugeben, dass sie von Suchmaschinen (Robots) indiziert werden sollen, aber keinen enthaltenen Links folgen sollen, ist die folgende Processing Instruction möglich:

```
1 <?robots index="yes" follow="no"?>
```

Die Einbindung von PHP-Code (sowohl in XML als auch in HTML) erfolgt ebenfalls über eine Processing Instruction – mehr dazu dann im zweiten Semester.

```
1 <?php
2 phpinfo();
3 ?>
```

XML-Deklaration

Von der Syntax her identisch zu den Processing Instructions ist die *XML-Deklaration* (auch XML-Präfix). Wenn sie verwendet wird, muss sie immer am Anfang eines XML-Dokuments stehen und gibt dem Parser Auskunft, welche XML-Version und Zeichenkodierung verwendet werden. Ist sie nicht vorhanden, muss der Parser dies selbst herausfinden, was nicht immer akkurat gelingt. Das XML-Beispiel in Abschnitt 7.1.2 beinhaltet ebenfalls eine Deklaration in Zeile 1. Dies ist die universellste Art der Deklaration: Version 1.0 [91] ist der meistgenutzte² Standard, die Zeichenkodierung UTF-8 erlaubt die höchste Flexibilität. Sie sieht daher wie folgt aus:

```
1 <?xml version="1.0" encoding="UTF-8"?>
```

7.1.4 Well-formed und Valid

Ein XML-Dokument gilt als *well-formed* (deutsch: „wohlgeformt“ (sic!)), wenn es die soeben erwähnten Grammatikregeln erfüllt. Am wichtigsten sind die folgenden, noch einmal kurz zusammengefassten. Darüber hinaus gibt es natürlich noch eine große Anzahl weiterer Faktoren, die eingehalten werden müssen bzw. die falsch gemacht werden können, um ein Dokument nicht mehr well-formed erscheinen zu lassen:

1. Jedes Start-Element muss ein entsprechendes End-Element besitzen.
2. Elemente können verschachtelt, dürfen aber nie überkreuzt werden.
3. Es darf nur genau ein Wurzelement geben.
4. Attributwerte müssen unter Anführungszeichen gesetzt werden.
5. Ein Element darf nicht zwei Attribute mit demselben Namen haben.

²Obwohl Version 1.1 [92] des Standards existiert, ist dieser nur in den seltensten Fällen nötig und die Verwendung von Version 1.0 empfohlen. Unter <https://www.ibiblio.org/xml/books/effectivexml/chapters/03.html>, einem Ausschnitt aus [23], wird erläutert warum.

6. Kommentare und Processing Instructions dürfen nicht innerhalb von Tags auftreten.
7. Die Zeichen `<` und `&` dürfen nicht unescaped im Inhalt von Elementen vorkommen.

Ein Dokument gilt in weiterer Folge als *valid* (deutsch: valide, gültig), wenn es einerseits well-formed ist und andererseits einem definierten Schema entspricht. Ein Schema ist entweder eine Document Type Declaration (kurz: DTD oder DOCTYPE) oder ein XML Schema. Beides wird nun näher in Abschnitt 7.1.5 erläutert.

7.1.5 Validierung durch Schemas

Während XML durch die freie Wahl der Elementnamen für Entwickler*innen sehr flexibel ist, bedeutet diese Flexibilität für Applikationen, die mit XML umgehen müssen, mitunter eine Herausforderung. Die Anwendung muss beim maschinellen verarbeiten genau wissen, wie eine XML-Applikation aufgebaut ist, ansonsten kann sie mit den Daten nicht arbeiten. Auch zu überprüfen, ob die zugrunde liegende XML-Datei korrekt ist, kann essentiell sein, um eine funktionierende Verarbeitung zu ermöglichen.

Dieses Dilemma lösen *Schemas*, die beschreiben, wie ein XML-Dokument aufgebaut sein muss. Dabei gibt es zwei Möglichkeiten: Document Type Definitionen (DTDs) und XML Schemas.

Document Type Definition

Die *Document Type Definition*, kurz *DTD* oder auch DOCTYPE genannt, ist in einer eigenen, formalen Sprache verfasst und beschreibt den Aufbau einer XML-Applikation. Es werden Aussagen getroffen wie „das Wurzelement ist `<rezept>`“ oder „jedes `<zutat>`-Element beinhaltet die Elemente `<ingredienz>`, `<menge>` und `<einheit>`“. Ein validierender Parser kann dann den Aufbau der XML-Datei mit der DTD vergleichen und auf Korrektheit überprüfen.

Aufbau einer DTD: Eine Document Type Definition ist ein einfaches Textdokument mit der Erweiterung `.dtd`. Sie enthält verschiedene Arten von Deklarationen. Eine Deklaration beginnt immer mit einer spitzen Klammer und einem Ausrufezeichen, danach folgt der Typ der Deklaration mit den nötigen Zusatzangaben, danach endet die Deklaration mit einer spitzen Klammer:

```
1 <!TYP Zusatzangaben>
```

Folgende Typen können spezifiziert werden:

- **ELEMENT:** Deklariert ein Element, d. h. ein Tag.
- **ATTLIST:** Deklariert Attribute zu einem Element.
- **ENTITY:** Deklariert eine Konstante.
- **NOTATION:** Deklariert nicht-XML-Dateien, um diese in XML verwenden zu können. Dies wird jedoch selten verwendet und daher in diesem Abschnitt auch nicht näher erläutert.

Element-Deklaration: *Element-Deklarationen* werden verwendet, um Elemente einzuführen. Die generelle Syntax lautet wie folgt:


```
1 <!ELEMENT title (content)>
```

Der Typ ist also **ELEMENT**, **title** wird durch den Namen des zu deklarierenden Elements ersetzt und die Angabe von **content** in Klammern spezifiziert, welcher Inhalt in diesem Element enthalten ist.

Leere Elemente: Leere Elemente werden definiert, indem anstelle von **content** der Wert **EMPTY** angegeben wird. Soll also beispielsweise ein XML-Element `<foo />` deklariert werden, so muss in der DTD folgendes angegeben sein:

```
1 <!ELEMENT foo (EMPTY)>
```

Elemente mit Textinhalt: Elemente, die nur Text beinhalten sollen, können mit dem Wert **#PCDATA** (Parsed Character Data, siehe Abschnitt 7.1.3) für **content** deklariert werden. Um ein Element `<anmerkung>Das hier ist wichtig!</anmerkung>` (Textinhalt kann beliebig sein) zu ermöglichen, ist folgender Eintrag in der DTD notwendig:

```
1 <!ELEMENT anmerkung (#PCDATA)>
```

Elemente mit Kindelementen: Sollen Elemente andere Elemente enthalten, werden diese in der Klammer von **content** angegeben. Dabei gibt es verschiedene Möglichkeiten:

Die Angabe von **ANY** beim Element `<random>` erlaubt jedes beliebige Kindelement:

```
1 <!ELEMENT random (ANY)>
```

Die Angabe eines Elements erlaubt eben nur dieses. Darf ein Element `<familie>` nur das Element `<kind>` enthalten, so wird dies folgendermaßen deklariert:

```
1 <!ELEMENT familie (kind)>
```

Mehrere Kindelemente können durch Komma separiert angegeben werden. Soll das Element `<auto>` die Kindelemente `<hersteller>`, `<typ>` und `<baujahr>` haben, so wird dies wie folgt deklariert:

```
1 <!ELEMENT auto (hersteller, typ, baujahr)>
```

Gibt es eine Auswahl an möglichen Kindelementen, d. h. ein Element `<sport>` kann entweder das Kindelement `<fußball>`, oder das Kindelement `<tennis>`, oder das Kindelement `<laufen>` beinhalten, so kann dies durch den „Oder“-Operator `|` angegeben werden:

```
1 <!ELEMENT sport (fußball|tennis|laufen)>
```

Schließlich kann durch Zusatzangaben die Häufigkeit des Auftretens spezifiziert werden:

- **?**: Optionales Auftreten, das Kindelement kann 0 oder 1 Mal vorkommen.
- **+**: Das Kindelement kann 1 bis *n* mal vorkommen (d. h. mindestens einmal).
- *****: Das Kindelement kann 0 bis *n* mal vorkommen (d. h. beliebig oft oder auch gar nicht).

Über Klammerung ist zudem noch eine Gruppierung möglich. Soll ein Element `<nachricht>` deklariert werden, das ein Element `<absender>`, mindestens ein Element `<empfänger>`, ein Element `<inhalt>` und optional entweder das Element `<bcrypt>` oder

`<argon2i>` (zur Angabe eines möglichen Hash-Wertes) beinhaltet, würde dies wie folgt deklariert:

```
1 <!ELEMENT nachricht (absender, empfänger+, inhalt, (bcrypt|argon2i)?)>
```

Attribut-Deklaration: Mit einer *Attribut-Deklaration* wird angegeben, welche Attribute ein Element besitzt. Die generelle Syntax lautet wie folgt:

```
1 <!ATTLIST tag attr type default>
```

Der Typ lautet **ATTLIST**, **tag** bezeichnet das Element, für das die Attribute angegeben werden. Mit **attr** wird der Name des Attributs spezifiziert, **type** gibt die Art des Attributs an. **default** spezifiziert, ob ein Attribut verlangt wird oder ausgelassen werden kann bzw. ob es einen Standardwert besitzt.

Folgende Werte sind für den Typ des Attributs möglich:

- **CDATA:** Das Attribut kann jeden beliebigen Textinhalt aufweisen.
- **ID:** Das Attribut muss einen eindeutigen Inhalt aufweisen (wie es in HTML für das `id`-Attribut üblich ist).
- **IDREF:** Das Attribut muss auf ein anderes vom Typ ID verweisen (wie das `for`-Attribut bei `<label>`, das auf `id` von `<input>` verweist).
- **IDREFS:** Das Attribut enthält eine durch Leerzeichen getrennte Liste von Werten, die auf Attribute vom Typ ID verweisen.
- **ENTITY:** Das Attribut verweist auf eine Konstante.
- **ENTITIES:** Das Attribut enthält die Namen mehrerer Konstanten.
- **NOTATION:** Das Attribut enthält den Namen einer Notation.
- **NMTOKEN:** Das Attribut enthält einen Wert, der der XML-Namensgebung entspricht (keine Leerzeichen, nur alphanumerische Zeichen etc.– wie in Abschnitt 7.1.3 erwähnt).
- **NMTOKENS:** Das Attribut enthält mehrere Werte, die der XML-Namensgebung entsprechen.
- **Aufzählung:** Nur die in der Liste angegebenen Werte sind erlaubt.

Folgende Werte sind für **default** möglich:

- **#IMPLIED:** Das Attribut ist optional und es ist kein Standardwert vorgegeben.
- **#REQUIRED:** Das Attribut muss immer angegeben werden, es ist aber kein Standardwert vorgegeben.
- **#FIXED "value":** Das Attribut muss angegeben werden und muss den Wert **value** haben.
- **"value":** Das Attribut ist optional. Wird es nicht angegeben, wird der mit **value** angegebenen Standardwert angenommen.

Attribute mit Textinhalt: Soll das Element `<song>` ein Attribut `title` besitzen, welches jeden textuellen Wert annehmen kann, aber angegeben werden muss, lautet die Deklaration wie folgt:

```
1 <!ATTLIST song title CDATA #REQUIRED>
```

Attribute mit Aufzählungen: Um eine vordefinierte Liste an Werten anzugeben, werden diese in Klammer mit dem „Oder“-Operator (senkrechter Strich, Pipe) aneinander gereiht: (wert1|wert2|wert3|...)

Soll für ein Element <vehikel> ein Attribut **typ** existieren, das nur die Werte **pkw**, **lkw**, **flugzeug** oder **boot** annehmen soll und bei dem der Standardwert **pkw** lautet, falls es nicht angegeben wird, so lautet die Deklaration:

```
1 <!ATTLIST vehikel typ (pkw|lkw|flugzeug|boot) "pkw">
```

Entität-Deklaration: *Entitäten* sind eine Art Konstante, bei der ein vordefinierter Text hinterlegt werden kann. Dies ist dann nützlich, wenn derselbe Text oft im Dokument vorkommt. Durch das Definieren einer Entität wird er einmal zentral definiert und kann überall dann referenziert werden. Es gibt eine Reihe von Entitäten, die wichtigsten sollen hier erläutert werden:

Generelle Entitäten: Allgemeine Entitäten liegen immer in der Form **&name;** vor. Bekannt sollten etwa **&** für **&** oder **<** für **<** sein. Um eine eigene Entität zu definieren, kommt folgende Syntax zur Anwendung:

```
1 <!ENTITY name "text">
```

name bezeichnet den Namen der Entität, das initiale **&** und abschließende **;** dürfen nicht enthalten sein. **text** ist der Inhalt, der für die Entität eingesetzt werden soll.

Um eine Entität **&fhoee;** mit dem Wert **Copyright 2025 FH OÖ** zu erzeugen, ist folgende Deklaration notwendig:

```
1 <!ENTITY fhoee "Copyright 2025 FH OÖ">
```

Parameter-Entitäten: Parameter-Entitäten können verwendet werden, um eine vordefinierte Parameterliste für eine Element-Deklaration zu erstellen. Dies ist dann interessant, wenn genau dieselbe Liste oft verwendet wird. Die allgemeine Form lautet:

```
1 <!ENTITY % name "text">
```

Angenommen, verschiedene Elemente können entweder Text oder auch das Element <description>, beides aber optional enthalten, so muss die Definition dessen (**#PCDATA|description**)* lauten. Die Beschreibung dieses Inhalts kann nun in eine Parameter-Entität „verpackt“ werden:

```
1 <!ENTITY % varcontent "(#PCDATA|description)*">
```

Nun können verschiedene Elemente definiert werden, die genau diese Inhaltsbeschreibung zugewiesen bekommen:

```
1 <!ELEMENT letter %varcontent>
2 <!ELEMENT info %varcontent>
```

Beispiel-DTD: Im Folgenden soll nun die DTD für das Tiroler Knödel-Rezept aus Abschnitt 7.1.2 dargestellt werden.

```

1 <!ELEMENT rezept (gericht, zutaten, zubereitung)>
2 <!ATTLIST rezept quelle CDATA #REQUIRED>
3
4 <!ELEMENT gericht (#PCDATA)>
5
6 <!ELEMENT zutaten (zutat)+>
7
8 <!ELEMENT zubereitung (schritt)+>
9
10 <!ELEMENT zutat (ingredienz, menge, einheit)>
11
12 <!ELEMENT schritt (#PCDATA)>
13
14 <!ELEMENT ingredienz (#PCDATA)>
15 <!ELEMENT menge (#PCDATA)>
16 <!ELEMENT einheit (#PCDATA)>

```

Das Element `<rezept>` enthält genau jeweils ein Element der Typen `<gericht>`, `<zutaten>` und `<zubereitung>` und besitzt ein verpflichtendes Attribut `quelle`, dessen Inhalt Text ist.

Das Element `<gericht>` enthält beliebigen Text. `<zutaten>` enthält mindestens ein `<zutat>`-Element. Das Element `<zubereitung>` enthält mindestens ein `<schritt>`-Element. `<zutat>` enthält je ein Element der drei Typen `<ingredienz>`, `<menge>` und `<einheit>`. `<schritt>` enthält beliebigen Text. Die Elemente `<ingredienz>`, `<menge>` und `<einheit>` enthalten jeweils beliebigen Text.

DTDs einbinden: Die Angabe einer DTD erfolgt in der XML-Datei direkt nach der XML-Deklaration, also in der zweiten Zeile. Die Syntax ist analog zu der aus HTML und lautet wie folgt:

```
1 <!DOCTYPE name SYSTEM url>
```

`!DOCTYPE` zeigt die Angabe einer DTD an, `name` ist der eindeutige Name. Dieser muss immer mit dem Namen des Wurzelements übereinstimmen, im Falle des Tiroler Knödel-Rezepts ist dies `rezept`. `SYSTEM` gibt an, dass die DTD an einem externen Ort liegt, dass es sich jedoch nicht um eine öffentliche DTD handelt. Mit `url` wird die Adresse zur DTD im Web angegeben, es kann jedoch auch eine relative URL sein. Die Angabe der DTD für das Tiroler Knödel-Rezept kann wie folgt aussehen:

```
1 <!DOCTYPE rezept SYSTEM "https://www.hochleitner.cc/dtds/rezept.dtd">
```

Öffentliche DTDs: Standardisierte DTDs verwenden nicht den Identifikator `SYSTEM`, sondern `PUBLIC`. Dies bedeutet, dass es sich um bekannte DTDs handelt, die beim Validieren etwa nicht online „nachgeschlagen“ werden müssen, da sie dem Validator in der Regel bekannt sind. Ein Beispiel hierfür ist der DOCTYPE für XHTML 1.0:

```

1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

```

XML Schema Definition (XSD)

Lange Zeit waren die DTDs der Standard zur Validierung von XML-Daten, sie wurden jedoch bald aufgrund fehlender Datentypen und eigener Syntax weitgehend durch die

XML Schema Definition (XSD) [111] ersetzt.

Aufbau eines Schemas: Ein XML Schema ist wie selbst ein XML-Dokument, das Mechanismen zur Typisierung (String, Integer, Date), Kardinalität (wie oft darf ein Element vorkommen) und Inhaltsmodellierung bietet. Es wird in einer Datei mit der Erweiterung `.xsd` gespeichert. Das Schema beginnt wie ein normales XML-Dokument mit einer XML-Deklaration, danach folgt das Wurzelement:

```
1 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
2   ...
3 </xs:schema>
```

Tags und auch Datentypen in XSD beginnen immer mit dem Präfix `xs:` (für XML Schema). Die Attributangabe spezifiziert die Schema-Definition, in diesem Fall XML Schema Version 1.0.

Innerhalb dieses Wurzelements folgen weitere Angaben:

- `<xs:element>`: Deklariert ein Element, d. h. ein Tag.
- `<xs:attribute>`: Deklariert Attribute zu einem Element.
- `<xs:complexType>`: Deklariert ein Element mit Unterelementen/Attributen.
- `<xs:simpleType>`: Beschreibt einen einfachen Datentyp ohne Kindelemente, kann jedoch Einschränkungen enthalten.
- `<xs:sequence>`, `<xs:choice>`, `<xs:all>`: Beschreiben Kindelemente und deren Reihenfolge bzw. Alternativen.

Element-Deklaration: Mit einer *Element-Deklaration* wird festgelegt, welche Elemente im XML-Dokument vorkommen dürfen und welcher Typ diesen Elementen zugeordnet ist. Die allgemeine Syntax lautet:

```
1 <xs:element name="gericht" type="xs:string"/>
```

Dies deklariert ein Element `<title>`, das beliebigen Textinhalt (Datentyp String) besitzt.

Einfache Elemente: Elemente, die nur Inhalt enthalten sollen, erhalten einen einfachen Datentyp wie `xs:string`, `xs:integer`, `xs:decimal`, `xs:boolean` oder ähnliche Typen des XML-Schemas:

```
1 <xs:element name="menge" type="xs:decimal"/>
```

Komplexe Elemente: Enthält ein Element weitere Kindelemente oder Attribute, so wird es als *komplexer Typ* definiert. Dies geschieht mit `xs:complexType` und einem Inhaltsmodell für die Reihenfolge mit `xs:sequence`:

```
1 <xs:element name="zutat">
2   <xs:complexType>
3     <xs:sequence>
4       <xs:element name="ingredienz" type="xs:string"/>
5       <xs:element name="menge" type="xs:decimal"/>
6       <xs:element name="einheit" type="xs:string"/>
7     </xs:sequence>
```

```

8   </xs:complexType>
9 </xs:element>

```

Wie bei DTDs steuern zusätzliche Angaben die Kardinalität der Elemente:

- `minOccurs="0"`: Optionales Auftreten.
- `maxOccurs="unbounded"`: Beliebig oft wiederholbar.
- `minOccurs="1"`: Mindestens einmal.

Attribut-Deklaration: Attribute werden mittels `xs:attribute` deklariert. Hier kommen ebenfalls einfache Datentypen zum Einsatz:

```

1 <xs:attribute name="id" type="xs:ID" use="required" />

```

Das Attribut `use` steuert die Verfügbarkeit:

- `use="optional"`: Das Attribut kann angegeben werden.
- `use="required"`: Das Attribut muss angegeben werden.
- `use="prohibited"`: Das Attribut ist verboten.

Einfache Typen: Mit `xs:simpleType` können eigene Datentypen definiert werden, etwa um Aufzählungen oder Einschränkungen festzulegen.

Aufzählungen: Aufzählungen legen fix definierte Werte für Inhalte fest:

```

1 <xs:simpleType name="EinheitTyp">
2   <xs:restriction base="xs:string">
3     <xs:enumeration value="g"/>
4     <xs:enumeration value="ml"/>
5     <xs:enumeration value="Stück"/>
6   </xs:restriction>
7 </xs:simpleType>

```

Dies erzeugt einen Datentyp namens „EinheitTyp“, der auf Text (String) basiert und die Werte „g“, „ml“ und „Stück“ haben darf.

Einschränkungen: Einschränkungen definieren Grenzen für Werte:

```

1 <xs:simpleType name="MengeTyp">
2   <xs:restriction base="xs:decimal">
3     <xs:minExclusive value="0"/>
4   </xs:restriction>
5 </xs:simpleType>

```

Dies definiert einen Datentyp namens „MengeTyp“, der auf einer Zahl (Decimal) basiert, aber nur positive Dezimalzahlen größer als 0 erlaubt.

Beispiel-XSD: Im Folgenden nun das XML Schema für das Rezept aus Abschnitt 7.1.2 dargestellt:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
3   <xs:element name="rezept">
4     <xs:complexType>
5       <xs:sequence>

```

```

6      <xs:element ref="gericht"/>
7      <xs:element ref="zutaten"/>
8      <xs:element ref="zubereitung"/>
9      </xs:sequence>
10     <xs:attribute name="quelle" type="xs:anyURI" use="required"/>
11   </xs:complexType>
12 </xs:element>
13
14 <xs:element name="gericht" type="xs:string"/>
15
16 <xs:element name="zutaten">
17   <xs:complexType>
18     <xs:sequence>
19       <xs:element maxOccurs="unbounded" ref="zutat"/>
20     </xs:sequence>
21   </xs:complexType>
22 </xs:element>
23
24 <xs:element name="zubereitung">
25   <xs:complexType>
26     <xs:sequence>
27       <xs:element name="schritt" type="xs:string" maxOccurs="unbounded"/>
28     </xs:sequence>
29   </xs:complexType>
30 </xs:element>
31
32 <xs:element name="zutat">
33   <xs:complexType>
34     <xs:sequence>
35       <xs:element name="ingredienz" type="xs:string"/>
36       <xs:element name="menge" type="xs:decimal"/>
37       <xs:element name="einheit" type="xs:string"/>
38     </xs:sequence>
39   </xs:complexType>
40 </xs:element>
41 </xs:schema>

```

Dieses Schema legt fest, dass ein `<rezept>`-Element ein verpflichtendes Attribut `quelle` besitzt und die Elemente `<gericht>`, `<zutaten>` und `<zubereitung>` genau in dieser Reihenfolge enthalten muss. Die Zutatenliste enthält mindestens ein `<zutat>`-Element, welches wiederum die Elemente `<ingredienz>`, `<menge>` und `<einheit>` in genau dieser Reihenfolge enthält. `<zubereitung>` umfasst beliebig viele `<schritt>`-Elemente. Datentypen schränken die Inhalte der Elemente zusätzlich ein.

XML Schema einbinden: Damit ein XML-Dokument gegen ein XML-Schema validiert werden kann, muss im Dokument selbst ein Verweis auf die entsprechende `.xsd`-Datei angegeben werden. Im Gegensatz zu DTDs erfolgt diese Einbindung nicht über einen `<!DOCTYPE>`-Eintrag, sondern über ein spezielles Attribut im Wurzelement des XML-Dokuments.

Zur Einbindung wird der sogenannte *XML Schema Instance*-Namespace (`xsi`) eingebunden und anschließend mit dem Attribut `xsi:noNamespaceSchemaLocation` auf die Datei verwiesen, welche das Schema enthält. Die Syntax lautet wie folgt:

```

1 <rootElement xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="schema.xsd">
2 ...
3 </rootElement>

```

`xmlns:xsi` deklariert das XML Schema Instance-Namespace, welches die Attribute zur Einbindung von Schemas bereitstellt. Mit `xsi:noNamespaceSchemaLocation` wird die URL oder der relative Pfad zur Schema-Datei angegeben. Dieses Attribut wird verwendet, wenn das XML-Dokument *keine Namespaces* verwendet (was bei einfachen XML-Strukturen typischerweise der Fall ist).

Für das Tiroler Knödel-Rezept sieht die Einbindung wie folgt aus:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <rezept xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:noNamespaceSchemaLocation="https://www.hochleitner.cc/schemas/rezept.xsd"
4     "
5     quelle="https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386">
6 </rezept>

```

7.1.6 XML-Applikationen

XML-Applikationen sind verschiedene Markupsprachen, die mittels XML definiert wurden. Bekannte Vertreter sind u. a.:

- **SVG**: Scalable Vector Graphics³ – das bekannteste und am weitesten verbreitete Format zur Darstellung von Vektorgrafiken im Web.
- **MathML**: Mathematical Markup Language⁴ – Ein Dokumentenformat zur Darstellung mathematischer Formeln im Web.
- **XHTML**: Extensible Hypertext Markup Language⁵ – die Neuformulierung von HTML 4.01 in XML. In den 2000er Jahren der Standard, mit dem Aufkommen von HTML5 obsolet geworden.
- **SMIL**: Synchronized Multimedia Integration Language⁶ – eine Auszeichnungssprache für zeitsynchronisierte, multimediale Inhalte.
- **RDF**: Resource Description Framework⁷ – eine Sprache zur Beschreibung von Informationen über Objekte (Metadaten) im Web. Wichtig für das Semantic Web⁸.
- **RSS**: Really Simple Syndication⁹ – ein Format zur Veröffentlichung von Änderungen auf Websites (bekannt als RSS-Feeds).

7.2 JSON

XML ist zweifelsohne ein vielseitig verwendbares und auch gut menschenlesbares Datenformat. Daraus ergibt sich aber auch ein entscheidender Nachteil: XML ist sehr

³https://de.wikipedia.org/wiki/Scalable_Vector_Graphics

⁴https://de.wikipedia.org/wiki/Mathematical_Markup_Language

⁵https://de.wikipedia.org/wiki/Extensible_Hypertext_Markup_Language

⁶https://de.wikipedia.org/wiki/Synchronized_Multimedia_Integration_Language

⁷https://de.wikipedia.org/wiki/Resource_Description_Framework

⁸https://de.wikipedia.org/wiki/Semantic_Web

⁹[https://de.wikipedia.org/wiki/RSS_\(Web-Feed\)](https://de.wikipedia.org/wiki/RSS_(Web-Feed))

ausschweifend – im Englischen „verbose“ (wörtlich: langatmig, wortreich). Es werden also viele Zeichen (d.h. Bytes) für die Beschreibung der Daten (die Tags) verwendet, was vor allem beim maschinellen Lesen (unnötigen) Overhead bedeutet. Aus diesem Grund hat sich zu Beginn der 2000er Jahr ein weiteres Datenformat etabliert: JSON.

7.2.1 Allgemein

JSON steht für *JavaScript Object Notation*¹⁰ [15]. Dabei handelt es sich um ein kompaktes Datenformat, dessen Syntax ein Subset der Objekt-Literal-Notation von JavaScript ist (siehe dazu Vorlesung 9). D.h., in JSON notierte Daten ähneln JavaScript-Objekten (daher auch der Name), sind aber kein JavaScript-Code. JSON ist sprachunabhängig und wird in allen populären Programmier- und Scriptsprachen unterstützt. Wird JSON in Dateien abgespeichert, so handelt es sich um Textdateien mit der Endung `.json`. Sehr oft werden JSON-Daten aber einfach als Zeichenkette (String) zwischen zwei Endstellen übertragen, etwa zwischen Client und Server, wenn so genannte REST-Schnittstellen zum Einsatz kommen (mehr dazu im zweiten Semester in der Vorlesung zu Web Technologies).

7.2.2 Ein JSON-Beispieldokument

Als Beispiel soll wiederum das Knödelrezept aus Abschnitt 7.1.2 dienen. In JSON sieht dieses etwa wie folgt aus:

```
1 {
2   "gericht": "Der klassische Tiroler Knödel",
3   "quelle": "https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386",
4   "zutaten": [
5     {
6       "ingredienz": "Semmelwürfel",
7       "menge": 350,
8       "einheit": "g"
9     },
10    {
11      "ingredienz": "Milch",
12      "menge": 200,
13      "einheit": "ml"
14    },
15    {
16      "ingredienz": "Eier",
17      "menge": 4,
18      "einheit": "Stück"
19    },
20    {
21      "ingredienz": "Speck",
22      "menge": 100,
23      "einheit": "g"
24    },
25    {
26      "ingredienz": "Bergsteigerwurst",
27      "menge": 100,
28      "einheit": "g"
29    }
30  ]
31 }
```

¹⁰<https://www.json.org/>

```

29   },
30   {
31     "ingredienz": "Butter",
32     "menge": 20,
33     "einheit": "g"
34   },
35   {
36     "ingredienz": "Zwiebel",
37     "menge": 1,
38     "einheit": "Stück"
39   },
40   {
41     "ingredienz": "Petersilie",
42     "menge": 10,
43     "einheit": "g"
44   },
45   {
46     "ingredienz": "Salz",
47     "menge": 1,
48     "einheit": "Prise"
49   },
50   {
51     "ingredienz": "Muskatnuss",
52     "menge": 1,
53     "einheit": "Prise"
54   },
55   {
56     "ingredienz": "Mehl",
57     "menge": 80,
58     "einheit": "g"
59   }
60   {
61     "ingredienz": "Schnittlauch",
62     "menge": 10,
63     "einheit": "g"
64   }
65 ],
66 "zubereitung": [
67   "Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen lassen.",
68   "Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in
69   zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten
70   und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen
71   Zutaten eine feste, aber nicht zu feuchte Masse kneten.",
72   "Mit befeuchteten Händen daraus Knödel formen.",
73   "Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat der
74   Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die Knö
75   delmasse kneten.",
76   "Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.",
77   "Tiroler Knödel mit Schnittlauch bestreut servieren."
78 ]
79 }

```

Der gesamte JSON-Datensatz ist ein Objekt – eine Sammlung von Name/Wert-Paaren, gekennzeichnet durch geschwungene Klammern ({ }). Die Namen, die immer Zeichenketten (Strings) sind, werden dabei auch oft als Schlüssel bezeichnet. Auf der

obersten Ebene gibt es vier Schlüssel: **gericht**, **quelle**, **zutaten** und **zubereitung**.

Der Wert zu jedem Schlüssel steht nach einem Doppelpunkt und kann verschiedene Datentypen annehmen:

- **gericht** und **quelle**: Die Werte sind hier jeweils eine Zeichenkette (String).
- **zutaten** und **zubereitung**: Der Wert ist jeweils eine geordnete Liste von Werten (ein Array), gekennzeichnet durch eckige Klammern ([]). Bei den Zutaten sind die einzelnen Einträge der Liste wiederum Objekte (mit eigenen Name/Wert-Paaren), bei der Zubereitung sind die Werte einfache Zeichenketten.

Hier zeigt sich der grundsätzliche Unterschied zu XML: Es kommen keine Tags zum Einsatz, vielmehr werden Datenstrukturen (die aus der Programmierung in vielen Sprachen bekannt sind) eingesetzt. Die Benennung erfolgt über den Namen in den Objekten. Attribute als Zusätze zu einzelnen Daten gibt es nicht, diese müssen als eigenes Name/Wert-Paar abgebildet werden.

7.2.3 JSON-Syntax

JSON basiert, wie im Beispiel ersichtlich, auf zwei Datenstrukturen:

- **Objekt**: Eine Sammlung von Name/Wert-Paaren.
- **Array**: Eine geordnete Liste von Werten.

Darüber hinaus gibt es noch einfache Wert-Typen – Literale genannt:

- **String**: Text, also eine Folgen von Unicode-Zeichen.
- **Number**: Eine Dezimalzahl.
- **true** und **false**: Boolesche Werte, die *wahr* oder *falsch* repräsentieren.
- **null**: Gibt an, dass explizit *kein* Wert vergeben wurde.

Objekte

Objekte sind ungeordnete Sets von Name-Wert-Paaren. Ein JSON-Objekt beginnt mit einer linken, geschwungenen Klammer ({) und endet mit einer rechten, geschwungenen Klammer (}). Der Name ist immer ein String, steht unter doppelten Anführungszeichen (") und ist vom Wert durch einen Doppelpunkt (:) getrennt. Mehrere Name-Wert-Paare sind durch Beistriche (,) voneinander getrennt. Leerzeichen und Zeilenumbrüche zwischen den einzelnen Trennzeichen sind möglich und können der Lesbarkeit helfen.

So kann ein Objekt in einer Zeile geschrieben werden:

```
1 { name1: wert1, name2: wert2 }
```

Meist jedoch, werden die Klammern und Name/Wert-Paare in eigene Zeilen gesetzt:

```
1 {
2   name1: wert1,
3   name2: wert2
4 }
```

Namen/Schlüssel können nur Zeichenketten (Strings) sein, Werte können Objekte, Arrays oder Literale (String, Zahl, **true**, **false**, **null**) sein.

Im Tiroler Knödel-Rezept ist jede Zutat ein Objekt mit drei Schlüssel. Die Werte sind zwei mal ein String und einmal eine Zahl:

```
1 {  
2   "ingredienz": "Semmelwürfel",  
3   "menge": 350,  
4   "einheit": "g"  
5 }
```

Arrays

Arrays sind geordnete Listen von Werten. Sie beginnen mit einer linken, eckigen Klammer ([]) und enden mit einer rechten, eckigen Klammer (]). Die einzelnen Werte eines Arrays sind mit Beistrichen (,) voneinander getrennt. Wieder können Leerzeichen und Zeilenumbrüche beliebig verwendet werden.

Somit kann ein Array einzilig geschrieben werden:

```
1 [ wert1, wert2, wert3 ]
```

Meist wird hier jedoch eine Aufteilung auf einzelne Zeile aufgrund der Lesbarkeit bevorzugt:

```
1 [  
2   wert1,  
3   wert2,  
4   wert3  
5 ]
```

Werte eines Arrays können wiederum Objekte, Arrays oder Literale sein.

Im Tiroler Knödel Rezept ist die Zubereitung ein Array, dessen Werte Zeichenketten sind:

```
1 "zubereitung": [  
2   "Semmelwürfel mit Milch ...",  
3   "Kleinstwürfelig geschnittenen Speck ...",  
4   "Mit befeuchteten Händen ...",  
5   "Einen davon als Probeknödel ...",  
6   "Erst wenn die Masse kompakt genug ist, ...",  
7   "Tiroler Knödel mit Schnittlauch bestreut servieren."  
8 ]
```

Literale

Als *Literale* werden einfache Werte bezeichnet. Dazu gehören Zeichenketten (String), Zahlen (Number), boolesche Werte (**true/false**) und **null**.

Strings: Strings werden unter doppelten Anführungszeichen geschrieben. Zeilenumbrüche müssen über Escapezeichen wie \n realisiert werden.

```
1 {  
2   "gericht": "Der klassische Tiroler Knödel"  
3 }
```

Zahlen: Zahlen (Numbers) werden ohne Anführungszeichen geschrieben. Unterstützt werden nur Dezimalzahlen (keine Oktal-, Hexadezimal- oder Binärzahlen).

```
1 {  
2   "menge": 350  
3 }
```

true, **false** und **null**: **true** und **false** sind so genannte boolesche Werte oder auch Wahrheitswerte. Sie repräsentieren die zwei Zustände wahr und falsch und werden direkt, ohne Anführungszeichen geschrieben. **null** ist ein spezieller Wert, der einfach „nichts“ oder „leer“ repräsentiert. Diese drei Werte kommen im Tiroler Knödel-Rezept nicht vor, könnten aber wie folgt verwendet werden:

```
1 {  
2   "aus_österreich": true,  
3   "vegetarisch": false,  
4   "bewertung": null  
5 }
```

In diesem Fall wäre gekennzeichnet, dass das Rezept aus Österreich stammt, nicht vegetarisch ist und noch keine Bewertungen vorliegen.

7.2.4 Validierung durch JSON-Schema

Auch JSON-Dokumente können anhand eines Schemas validiert werden. Der etablierte Standard hierfür ist *JSON Schema*¹¹. Dabei handelt es sich um ein selbst in JSON verfasstes Format, mit dem definiert werden kann, welche Inhalte ein JSON-Dokument haben muss und welche Datentypen zum Einsatz kommen.

Aufbau eines Schemas

Ein JSON-Schema wird in einer Textdatei mit der Endung `.schema.json` gespeichert und enthält zunächst ein Objekt mit folgenden Einträgen (die hier gezeigten Werte orientieren sich immer am Tiroler Knödel-Rezept).

```
1 {  
2   "$schema": "https://json-schema.org/draft/2020-12/schema",  
3   "$id": "https://www.hochleitner.cc/schemas/rezept.schema.json",  
4   "title": "Rezept",  
5   "description": "Ein Kochrezept mit Zutaten und Zubereitungsschritten.",  
6   "type": "object",  
7   "properties": { ... },  
8   "additionalProperties": false,  
9   "required": [ ... ]  
10 }
```

Die Schlüssel bedeuten dabei folgendes:

- **\$schema**: Ist ein Schema-Schlüssel und spezifiziert die verwendete Schema-Version.
- **\$id**: Ebenfalls ein Schema-Schlüssel. Er gibt eine eindeutige Bezeichnung für das Schema an. Hierfür wird meist ein eindeutiger URL gewählt.
- **title**: Bezeichnet, was das Schema definiert.
- **description**: Eine inhaltliche Beschreibung des Schemas.

¹¹<https://json-schema.org/>

- **type**: Spezifiziert den Datentyp, in dem die gesamte JSON-Datei vorliegen muss – hier ein Objekt.
- **properties**: Definiert die Liste an Namen/Schlüsseln, die in diesem Objekt beinhaltet sein können.
- **additionalProperties**: Legt fest, ob neben den durch **properties** festgelegten Schlüsseln noch weitere erlaubt sind.
- **required**: Entscheidet, welche Schlüssel zwingend benötigt werden.

Eigenschaften deklarieren

Eigenschaften (engl. „properties“) sind die Namen oder Schlüssel einer JSON-Datei. Um sie zu definieren, kommt die **properties**-Angabe zum Einsatz:

```
1 "properties": {  
2   "gericht": {  
3     "description": "Das Gericht oder der Name des Rezepts.",  
4     "type": "string",  
5     "minLength": 1  
6   },  
7   "quelle": {  
8     "description": "Die Quelle oder URL des Rezepts.",  
9     "type": "string",  
10    "format": "uri"  
11  },  
12 }
```

Dies definiert die beiden Eigenschaften **gericht** und **quelle**. Beide sind ein String, ersterer hat die Einschränkung, dass er mindestens 1 Zeichen lang sein muss, zweiterer muss ein gültiger URI sein. Auf dieselbe Art und Weise können auch andere Typen wie Arrays oder Objekte festgelegt werden. Da letztere wiederum aus Eigenschaften bestehen, müssen diese dann wieder verschachtelt definiert werden:

```
1 "zutaten": {  
2   "description": "Die Liste der Zutaten für das Rezept.",  
3   "type": "array",  
4   "minItems": 1,  
5   "items": {  
6     "type": "object",  
7     "properties": {  
8       "ingredienz": {  
9         "description": "Eine einzelne Zutat des Rezepts.",  
10        "type": "string",  
11        "minLength": 1  
12      },  
13      "menge": {  
14        "description": "Die Menge der Zutat.",  
15        "type": "number",  
16        "exclusiveMinimum": 0  
17      },  
18      "einheit": {  
19        "description": "Die Einheit der Menge.",  
20        "type": "string",  
21        "enum": [  
22          "g",
```

```

23         "ml",
24         "Stück",
25         "Prise"
26     ]
27 }
28 }
29 }
30 }

```

zutaten ist ein Array, das mindestens einen Eintrag haben muss. Die Einträge selbst sind wieder Objekte, die aus den Eigenschaften **ingredienz**, **menge** und **einheit** bestehen. Einheit ist dabei als **enum** definiert, was nur die vier hier angegebenen Werte zulässt.

Zusätzliche und verpflichtende Werte

Schließlich kann mit den Angaben **additionalProperties** und **required** noch spezifiziert werden, ob neben den im Schema angegebenen Schlüsseln noch weitere erlaubt sind und welche von den angegeben vorkommen *müssen*, d. h. verpflichtend sind. Diese Angaben werden nach der **properties**-Definition gemacht und können so aussehen:

```

1 "properties": {
2   ...
3   "additionalProperties": false,
4   "required": [
5     "gericht",
6     "quelle",
7     "zutaten",
8     "zubereitung"
9   ]
10 }

```

7.2.5 Beispiel-Schema

Im Folgenden nun das JSON-Schema für das Rezept aus Abschnitt 7.2.2 dargestellt:

```

1 {
2   "$schema": "https://json-schema.org/draft/2020-12/schema",
3   "$id": "https://www./schemas/rezept.schema.json",
4   "title": "Rezept",
5   "description": "Ein Kochrezept mit Zutaten und Zubereitungsschritten.",
6   "type": "object",
7   "properties": {
8     "gericht": {
9       "description": "Das Gericht oder der Name des Rezepts.",
10      "type": "string",
11      "minLength": 1
12    },
13    "quelle": {
14      "description": "Die Quelle oder URL des Rezepts.",
15      "type": "string",
16      "format": "uri"
17    },
18    "zutaten": {
19      "description": "Die Liste der Zutaten für das Rezept.",

```

```
20     "type": "array",
21     "minItems": 1,
22     "items": {
23       "type": "object",
24       "properties": {
25         "ingredienz": {
26           "description": "Eine einzelne Zutat des Rezepts.",
27           "type": "string",
28           "minLength": 1
29         },
30         "menge": {
31           "description": "Die Menge der Zutat.",
32           "type": "number",
33           "exclusiveMinimum": 0
34         },
35         "einheit": {
36           "description": "Die Einheit der Menge.",
37           "type": "string",
38           "enum": [
39             "g",
40             "ml",
41             "Stück",
42             "Prise"
43           ]
44         }
45       },
46       "additionalProperties": false,
47       "required": [
48         "ingredienz",
49         "menge",
50         "einheit"
51       ]
52     },
53   },
54   "zubereitung": {
55     "description": "Die Zubereitungsschritte des Rezepts.",
56     "type": "array",
57     "minItems": 1,
58     "items": {
59       "type": "string",
60       "minLength": 1
61     }
62   },
63 },
64 "additionalProperties": false,
65 "required": [
66   "gericht",
67   "quelle",
68   "zutaten",
69   "zubereitung"
70 ]
71 }
```


7.2.6 JSON-Schema verwenden

Im Gegensatz zu XML-Dokumenten, gibt es in JSON keine Möglichkeit, das Schema in der JSON-Datei anzugeben. Ein Tool, wie etwa eine IDE, muss also wissen, dass es ein Schema gibt, wo sich dieses befindet und es laden und damit arbeiten.

In PhpStorm kann dies unter **File** » **Settings** » **Schemas and DTDs** » **JSON Schema Mappings** geschehen. Mit  kann eine Schemadatei ausgewählt werden. Zu dieser können dann zu überprüfende JSON-Dateien angegeben werden. PhpStorm kann dann beim Ausfüllen die Angaben im Schema kontrollieren und eventuelle Abweichungen anzeigen.

Für bekannte und öffentliche JSON-Formate, die etwa mit Bibliotheken oder Frameworks verwendet werden, existiert der *JSON Schema Store*¹². Er enthält die Schemas für eine Vielzahl an solchen Formaten. IDEs wie PhpStorm verwenden ihn automatisch und können daher bekannte Formate direkt validieren.

7.3 YAML

YAML – kurz für *YAML Ain't Markup Language* – ist ein menschenlesbares Datenformat und eine vereinfachte Auszeichnungssprache (engl. „lightweight markup language“) [42].

7.3.1 Einführung

YAML hat denselben Einsatzzweck wie XML und JSON: Daten sollen menschenlesbar gespeichert werden. Es wurde als Reaktion auf die Popularität von JSON erdacht, da dieses zwar kompakt und sowohl für Menschen als auch für Maschinen lesbar ist, bei sehr großen Dateien aber aufgrund der vielen geschwungenen Klammern unübersichtlich wird. Der von YAML verfolgte Ansatz ist dabei noch minimalistischer und stellt mehr die Daten in den Vordergrund als das Markup. YAML ist dabei in seiner aktuellsten Version 1.2 ist eine Übermenge von JSON, d. h. jedes JSON-Dokument ist auch ein gültiges YAML 1.2-Dokument (aber nicht umgekehrt).

YAML-Dateien sind ebenfalls Textdokumente, als Endungen kommen `.yaml` oder `.yml` zum Einsatz, wobei ersteres bevorzugt wird.

YAML kommt in vielen Webanwendungen, Frameworks oder Softwarekomponenten zum Einsatz, um etwa Konfigurationen zu hinterlegen oder auch Daten zu übertragen.

7.3.2 Ein YAML-Beispieldokument

Das Knödelrezept aus den Abschnitten 7.1.2 und 7.2.2 soll nun in YAML verfasst werden. Dies kann so aussehen:

```
1 # Tiroler Knödel Rezept
2 %YAML 1.2
3 ---
4 gericht: Der klassische Tiroler Knödel
5 quelle: https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386
6 zutaten:
7   - ingredient: Knödelbrot
```

¹²<https://www.schemastore.org/>

```
8   menge: 350
9   einheit: g
10  - ingredient: Milch
11   menge: 200
12   einheit: ml
13  - ingredient: Eier
14   menge: 4
15   einheit: Stück
16  - ingredient: Speck
17   menge: 100
18   einheit: g
19  - ingredient: Bergsteigerwurst
20   menge: 100
21   einheit: g
22  - ingredient: Butter
23   menge: 20
24   einheit: g
25  - ingredient: Zwiebel
26   menge: 1
27   einheit: Stück
28  - ingredient: Petersilie
29   menge: 10
30   einheit: g
31  - ingredient: Salz
32   menge: 1
33   einheit: Prise
34  - ingredient: Muskatnuss
35   menge: 1
36   einheit: Prise
37  - ingredient: Mehl
38   menge: 80
39   einheit: g
40  - ingredient: Schnittlauch
41   menge: 10
42   einheit: g
43 zubereitung:
44  - >
45    Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen lassen.
46  - >
47    Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in
48    zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten
49    und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen
50    Zutaten eine feste, aber nicht zu feuchte Masse kneten.
51  - >
52    Mit befeuchteten Händen daraus Knödel formen.
53  - >
54    Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat der
55    Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die
56    Knödelmasse kneten.
57  - >
58    Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.
59  - >
60    Tiroler Knödel mit Schnittlauch bestreut servieren.
```

Wie ersichtlich ist, verfolgt YAML nicht den klassischen Markup-Ansatz mit Tags. Vielmehr kommt eine vereinfachte Syntax nach dem Schema **Schlüssel: Wert** zum Einsatz. Auffallend ist auch, dass es – umgesetzt auf das XML-Dokument keinen Eintrag

für „Rezept“ gibt und auch kein „Zutat“-Schlüssel vorhanden ist, sondern die Auflistung (die über den Bindestrich – erreicht wird) direkt unter „Zutaten“ steht. Der Grund hierfür ist, dass wie schon erwähnt die Daten im Vordergrund stehen. Eine Verschachtelung unter einem „Rezept“-Schlüssel brächte keinerlei Vorteile, vielmehr sollte das ganze Dokument einfach als Rezept angesehen werden. YAML ist somit viel näher an klassischen Programmier- oder Skriptsprachen, da eigentlich Datenstrukturen (Arrays bzw. assoziative Listen) abgebildet werden.

Ebenso fällt auf, dass es keine Attribute wie bei XML gibt, daher ist die Quelle einfach ein weiterer Eintrag.

7.3.3 YAML-Syntax

YAML verfolgt denselben Ansatz wie JSON, nämlich, dass sich die abzubildenden Daten mit Hilfe von drei Datenstrukturen – Mappings, Sequenzen und Skalaren – darstellen lassen. Wichtig dabei ist auch die Einrückung, die die Struktur vorgibt.

Mappings

Mappings entsprechen den Objekten aus JSON, sind also Name-Wert-Paare. Der Schlüssel wird durch einen Doppelpunkt (:), gefolgt von einem Leerzeichen vom Wert getrennt. Jedes Mapping steht dabei in seiner eigenen Zeile, dies wird in YAML das *Block-Format* genannt.

```
1 name1: wert1
2 name2: wert2
```

Alternativ steht das so genannte *Flow-Format* zur Verfügung, dabei werden die Mappings in geschwungene Klammern durch Beistriche getrennt angegeben:

```
1 { name1: wert1, name2: wert2 }
```

Auch eine mehrzeilige Aufteilung wie in JSON üblich ist möglich:

```
1 {
2   name1: wert1,
3   name2: wert2
4 }
```

Da das Flow-Format aber den Sinn hat, die Daten kompakter als das Block-Format zu repräsentieren, wird in der Regel die einzeilige Variante gewählt (und in weiterer Folge hier auch immer verwendet).

Im Tiroler Knödel-Rezept ist jede Zutat ein Mapping mit drei Schlüsseln. Die Werte sind zwei mal ein String und einmal eine Zahl. Im Block Format wird es wie folgt geschrieben:

```
1 ingredienz: Semmelwürfel
2 menge: 350
3 einheit: g
```

Sequenzen

Sequenzen (engl. „sequences“) sind einfache Arrays. Sie werden durch einen Bindestrich (–) gekennzeichnet, jedes einzelne Element steht bei der Block-Syntax in seiner eigenen Zeile:

```
1 - wert1
2 - wert2
3 - wert3
```

Im Flow-Format werden die Elemente in eckigen Klammern mit Komma getrennt angegeben:

```
1 [ wert1, wert2, wert3 ]
```

Im Tiroler Knödel Rezept ist die Zubereitung ein Array, dessen Werte Zeichenketten sind:

```
1 - Semmelwürfel mit Milch ...
2 - Kleinwürfelig geschnittenen Speck ...
3 - Mit befeuchteten Händen ...
4 - Einen davon als Probeknödel ...
5 - Erst wenn die Masse kompakt genug ist ...
6 - Tiroler Knödel mit Schnittlauch bestreut servieren.
```

Skalare

Skalare sind entweder Text (d. h. Zeichenketten bzw. Strings), Zahlen (Number), boolesche Werte (`true/false`) oder `null`.

Strings: Strings müssen (wie zuvor in den Beispielen schon ersichtlich) *nicht* unter Anführungszeichen gestellt werden, können aber, wenn etwa bereits reservierte Zeichen darin vorkommen. Dabei sind sowohl einfache als auch doppelte Anführungszeichen möglich. Die Werte des Mapping-Beispiels waren etwa Skalare, die ohne Anführungszeichen geschrieben wurden.

Soll längerer Text erfasst werden, können zwei Stile verwendet werden: literal oder folded. Ersterer wird mit einem Pipe-Zeichen (`|`) begonnen und Zeilenumbrüche werden so beibehalten, wie sie im Text aufscheinen. Der zweite Stil beginnt mit einer spitzen Klammer (`>`), Zeilenumbrüche werden einfach auf Leerzeichen umgewandelt. Bei beiden Stilen wird das jeweilige Zeichen in die erste Zeile gesetzt und der Text dann um zwei Leerzeichen eingerückt in die darauffolgenden:

```
1 - |
2   Dieser Text bricht
3   genauso um, wie er
4   hier geschrieben ist.
5 - >
6   Dieser Text ist im
7   Prinzip ein langer
8   String und hier nur
9   zwecks Lesbarkeit auf
10  mehrere Zeilen aufgeteilt.
```

Da im Tiroler Knödel-Rezept die Zubereitungsschritte lange sind, kommt hier der folded Stil zum Einsatz:

```
1 - >
2   Semmelwürfel mit Milch ...
3 - >
4   Kleinwürfelig geschnittenen Speck ...
5 - >
```

```

6   Mit befeuchteten Händen ...
7 - >
8   Einen davon als Probeknödel ...
9 - >
10  Erst wenn die Masse kompakt genug ist...
11 - >
12  Tiroler Knödel mit Schnittlauch bestreut servieren.

```

Zahlen: Zahlen werden ebenfalls ohne Anführungszeichen geschrieben. YAML erlaubt hier (im Vergleich zu JSON) nicht nur Dezimalzahlen, sondern auch Oktalzahlen (mit führender Null, z.B. 0755) und Hexadezimalzahlen (mit führendem 0x, z.B. 0xFF). Ebenso können Zahlen mit dem Underscore (_) als Tausendertrennzeichen verwendet werden (z.B. 1_000_000), um sie optisch besser darzustellen.

```
1 menge: 350
```

true, false und null: Die Wahrheitswerte und null werden analog zu JSON verwendet.

```

1 aus_österreich: true
2 vegetarisch: false
3 bewertung: null

```

Struktur und Direktiven

YAML-Dokumente werden wie in JSON in eine eigene Datei geschrieben. Dabei ist es jedoch möglich, dass eine YAML-Datei mehrere Dokumente enthält. Diese werden mit drei Bindestrichen (---) von einander getrennt. Dieses Zeichen ist also der Beginn eines neuen Dokuments mit Inhalt. Ist nur ein Dokument in der Datei, können die drei Bindestriche entfallen.

Alternativ können vor dem eigentlichen Dokument auch so genannte Direktiven stehen. Diese sind kein Inhalt, sondern ähnlich wie XML-Processing Instructions Anweisungen an etwaige Parser. Direktiven in YAML werden mit dem Prozentzeichen (%) geschrieben. Die Direktiven werden dann mit --- vom eigentlichen Inhalt getrennt.

Um das Ende eines Dokuments zu markieren, werden drei Punkte (...) verwendet. Diese sind jedoch nicht zwingend notwendig und können auch entfallen.

Das folgende Beispiel besteht aus zwei Abschnitten, zu Beginn steht eine Direktive zur YAML-Versionsnummer:

```

1 %YAML 1.2
2 ---
3 rezept: Tiroler Knödel
4 schwierigkeit: einfach
5 vegetarisch: false
6 ...
7 ---
8 rezept: Käsespätzle
9 schwierigkeit: einfach
10 vegetarisch: true
11 ...

```

Kommentare

YAML unterstützt im Gegensatz zu JSON Kommentare. Diese werden mit der Raute (#) verfasst und sind einzeilig.

```
1 # Tiroler Knödel Rezept
2 %YAML 1.2
3 ---
4 gericht: Der klassische Tiroler Knödel
```

Einrückungen

Einrückungen spielen bei den Block-Stilen in YAML eine große Rolle. Dabei dürfen ausschließlich Leerzeichen und keine Tabulatoren verwendet werden. Die Anzahl ist dabei freigestellt, in der Regel kommen zwei oder vier Leerzeichen für eine Einrückungsstufe zum Einsatz. Eine Einrückung zeigt dabei eine Hierarchieebene in der Datenstruktur an, diese wird wieder beendet, sobald ein Element folgt, das weniger eingerückt ist. Im folgenden wird ein Mapping mit einer Sequenz als Wert, die wiederum einzelne Mappings enthält gezeigt. Jede Stufe ist mit 2 Leerzeichen eingerückt.

```
1 zutaten:
2   - ingredienz: Semmelwürfel
3     menge: 350
4     einheit: g
5   - ingredienz: Milch
6     menge: 200
7     einheit: ml
```

Weitere Merkmale

YAML beherrscht noch weitere Fähigkeiten wie etwa die explizite Angabe von Datentypen oder auch das Setzen von Aliases, einem Referenz-Mechanismus, falls sich Werte wiederholen. Für diese Dinge sei auf die YAML-Dokumentation unter [42] verwiesen.

7.3.4 YAML validieren

Auch YAML bietet die Möglichkeit einer Validierung. Dabei kommt auch JSON Schema zum Einsatz, da YAML dieselben Datenstrukturen und -typen verwendet. Wiederum gilt auch hier, dass das Schema nicht im Dokument angegeben wird, sondern über Tools eingebunden wird. Der Prozess ist dabei identisch wie für JSON in Abschnitt 7.2.6.

Das Schema für das Tiroler Knödel-Rezept in YAML ist also genau dasselbe wie das in Abschnitt 7.2.5 gezeigte.

7.4 Weiterführende Literatur

Das Angebot an Fachliteratur zum Thema XML ist überdurchschnittlich groß. Demnach ist es auch nicht einfach, *das* Standardwerk schlechthin zu empfehlen. Ein Buch, das schon länger am Markt ist, ist [24]. Es bietet einen soliden Überblick über die Grundlagen und verschiedenen Anwendungsgebiete von XML und ist mittlerweile schon in

seiner dritten Auflage vorhanden. Diese ist jedoch schon aus dem Jahr 2004, wobei es bei XML selbst in den letzten Jahren keine Neuerungen mehr gab.

Ebenfalls vom selben Autor ist [23], welches jedoch mehr mit praxisbezogenen Tipps aufwartet.

Wer ein deutsches Buch zum Thema XML sucht, dem sei [50] ans Herz gelegt. Das Buch hat 2018 seine neunte Auflage bekommen, ist also im Vergleich zu anderen Werken aktuell und enthält alle wichtigen Themen auch für Einsteiger gut aufbereitet.

Literatur zu JSON und YAML hingegen ist spärlich. Um die Formate zu erlernen, bieten sich die offiziellen Dokumentationen [15, 42] an, des weiteren finden sich online diverse Tutorials, die beide Formate erklären.

Vorlesung 8

JavaScript-Grundlagen

8.1 Allgemeines zu JavaScript

JavaScript ist eine Skriptsprache, die hauptsächlich zur Erzeugung dynamischer Inhalte im Web eingesetzt wird. Die folgenden Abschnitte geben einen allgemeinen Überblick über die Sprache.

8.1.1 Was ist JavaScript?

JavaScript ist eine interpretierte Skriptsprache, d. h. in JavaScript verfasste Scripts werden von einem Interpreter¹ abgearbeitet und entsprechend ausgeführt² (das Gegenteil sind kompilierte Sprachen, bei denen ein Compiler eine ausführbare Datei erzeugt). Die Sprache wird gerne als „Programmiersprache fürs Web“ bezeichnet, da sie hauptsächlich für Webseiten zum Einsatz kommt. Die Anweisungen werden dabei in HTML eingebettet und beeinflussen die jeweilige HTML-Seite.

Die Syntax von JavaScript entspricht in großen Teilen der von C/C++ oder Java, der größte Unterschied (neben der Tatsache, dass JavaScript interpretiert und nicht kompiliert wird) ist das schwache Typkonzept von JavaScript. Variablen haben keinen fixen Datentyp (wie z. B. in Java oder C: dort kann `int nummer` nur Integer-Werte beinhalten), sondern können verschiedene Typen aufnehmen.

JavaScript kann – wie von sehr vielen Sprachen bekannt – imperativ (dabei sowohl objektorientiert als auch prozedural) programmiert werden, besitzt aber auch funktionale³ Elemente. Die Sprache weist einen Garbage Collector auf, der den Speicher aufräumt. Dies ist analog zu Java und ein Unterschied zu C, wo das Speichermanagement selbst erfolgen muss.

8.1.2 Terminologie

JavaScript selbst kann in drei grobe Teile unterschieden werden: Core JavaScript, Client-Side JavaScript und Server-Side JavaScript.

¹<https://de.wikipedia.org/wiki/Interpreter>

²Moderne Web verwenden jedoch Just-In-Time Compilation (JIT), um wichtige oder sich wiederholende Codeteile effizienter auszuführen.

³https://de.wikipedia.org/wiki/Funktionale_Programmierung

Core JavaScript

Mit *Core JavaScript* werden alle sprachlichen Funktionalitäten der Sprache bezeichnet. Also die Art, wie Variablen definiert werden, welche Datentypen es gibt, wie Objekte verwendet werden usw. Dieser Teil von JavaScript ist offiziell von der European Computer Manufacturers Association (ECMA) standardisiert und heißt *ECMAScript*, die Nummer des Standards lautet ECMA-262 [14].

Mit Core JavaScript, d. h. ECMAScript, sind beliebige Algorithmen formulierbar. Im Gegensatz zu den meisten Programmiersprachen hat ECMAScript aber kein Input- und Output-Konzept. Dieses muss von der Umgebung, in der ECMAScript eingesetzt wird (im Regelfall ein Webbrowser), zur Verfügung gestellt werden. Dies geschieht über Client-Side JavaScript.

Client-Side JavaScript

Client-Side JavaScript ermöglicht es Entwickler*innen, auf die verschiedensten Dinge rund um eine Website zuzugreifen:

- **Inhalte der Website:** Über das so genannte Document Object Model (DOM, mehr dazu in den Vorlesungen 9 und 10) kann auf alle Elemente einer Website zugegriffen werden. Dabei können Inhalte einfach ausgelesen oder auch verändert werden.
- **Eigenschaften des Browser-Fensters:** Es können eine Vielzahl an Eigenschaften wie die Größe oder Position oder auch das Vorhandensein einzelner Bestandteile (wie z. B. einer Scrollleiste) überprüft und teilweise auch verändert werden.
- **Events:** Wann immer Benutzer*innen mit einer Website interagieren, werden dabei im Webbrowser Ereignisse (so genannte Events) ausgelöst. Zum Beispiel beim Klick auf einen Button, beim Drücken einer Taste etc. Auf diese Ereignisse kann zeitnah reagiert und somit Interaktivität erreicht werden.

Server-Side JavaScript

Schließlich existiert JavaScript auch auf der Serverseite. *Server-Side JavaScript* ist Code, der in eigenen JavaScript-Engines auf Webservern läuft und dort wie andere serverseitige Programmier- und Skriptsprachen zum Einsatz kommt. D. h. es werden damit Webseiten komplett generiert und nicht nur bestehende Teile verändert bzw. dynamisch gestaltet.

Die erste serverseitige JavaScript-Implementierung wurde bereits 1995 von Netscape realisiert, der große Durchbruch kam jedoch erst ab dem Jahr 2009 mit dem Aufkommen bzw. der Verbreitung von Node.js⁴. Node.js wird im dritten Semester in Web Architectures (WAR3VO) thematisiert.

8.1.3 Versionen, Geschichte und Browserunterstützung

JavaScript wurde im Jahr 1995 von Brendan Eich, einem Entwickler bei Netscape, entworfen und zum ersten Mal mit Netscape 2.0 im Jahr 1996 ausgeliefert. Es hieß damals noch *LiveScript* wurde aber bald im Rahmen eines – eigentlich nicht sehr klugen – Marketingschachzugs auf JavaScript umbenannt. Es sollte damals die Popularität der

⁴<https://nodejs.org/>

noch jungen Programmiersprache Java dazu verwendet werden, die eigene Skriptsprache bekannter zu machen. Dies führte aber im Endeffekt dazu, dass JavaScript bis heute oft mit Java verwechselt oder als „vereinfachte“ Version von Java dargestellt wird. Dies ist jedoch nicht korrekt, denn JavaScript und Java haben (bis auf einige syntaktische Ähnlichkeiten) nichts miteinander gemeinsam. Daher ist Folgendes wichtig:

JavaScript ist nicht Java: Es ist kein Dialekt von Java, keine einfache Version von Java, nicht Java für Webbrowser – die Sprachen sind nicht miteinander verwandt und haben nichts miteinander zu tun.

Microsoft, damals mitten im bereits in Vorlesung 1 erwähnten Browserkrieg, entwickelte darauf hin seine eigene Skriptsprache namens *JScript*, die in Internet Explorer 3 zum Einsatz kam. Darauf hin reichte Netscape seine Version zur Standardisierung bei der ECMA ein. Der erste ECMAScript-Standard wurde im Juni 1997 veröffentlicht. Im August 1998 folgte die zweite Edition. 1999 wurde schließlich ECMAScript Edition 3 fertig gestellt, die zehn Jahre lang bis zum Jahr 2009 als stabiler Standard galt. Version 4 wurde nie veröffentlicht, sodass im Dezember 2009 Edition 5 herausgebracht wurde. 2011 wurde darauf aufbauend Version 5.1 in Einklang mit dem entsprechenden ISO-Standard definiert. 2015 wurde mit Version 6 (eigentlich ECMAScript 2015) eine große Anzahl an Weiterentwicklungen eingeführt und ein jährlicher Veröffentlichungszyklus beschlossen. So erschienen seitdem ECMAScript 2016–2024 und die aktuelle Version, ECMAScript Edition 2025 [14]. Diese stammt vom Juni 2025. ECMAScript wird laufend weiterentwickelt. Die aktuelle Entwicklerversion (derzeit ECMAScript 2026) kann unter <https://github.com/tc39/ecma262> eingesehen werden, unter <https://tc39.es/ecma262/> findet sich der dazugehörige Dokumentenentwurf.

ECMAScript 5 ist laut [114] derzeit in allen aktuellen Browsern flächendeckend implementiert. Version 2015 (6.0) wird so gut wie vollständig unterstützt: führend ist Safari 26, der 100 % aller Funktionen implementiert, danach folgen Chrome 143, Edge 143, Opera 112 und Firefox 145 mit 98 %. Funktionalitäten der 2016er bis 2026er Entwürfe sind ebenso gut verfügbar: Alle Browser implementieren 99 bis 97 % davon.

8.2 Lexikalische Struktur

Die lexikalische Struktur einer Programmiersprache beschreibt die Grundregeln, die für die Sprache gelten.

8.2.1 Zeichensatz

JavaScript-Programme werden in *Unicode* geschrieben, seit ECMAScript Edition 3 kann Unicode auch an allen Stellen des Programms verwendet werden. Zuvor war es nur in Kommentaren und Strings möglich.

8.2.2 Groß- und Kleinschreibung

Groß- und Kleinschreibung spielt in JavaScript eine große Rolle d.h. die Sprache ist „case-sensitive“. Schlüsselwörter wie `while` oder `if` müssen immer klein geschrieben

werden (nicht `WHILE` oder `If`), bei Variablennamen sind `online`, `Online`, `OnLine` und `ONLINE` vier unterschiedliche Variablen.

Vorsicht ist bei HTML-Attributen gegeben: auf diese kann in JavaScript über ihre Namen zugegriffen werden. Auch wenn in HTML der Attributname `TITLE` erlaubt und mit `title` gleichzusetzen ist, muss das Attribut in JavaScript *immer* als `title` verwendet werden.

8.2.3 Strichpunkte

JavaScript-Anweisungen werden immer mit einem *Strichpunkt* (`;`) beendet. Die Sprachdefinition lässt es zu, dass Code, der in einer eigenen Zeile steht, auch ohne Strichpunkt geschrieben werden kann, davon sei aber stark abgeraten, da es eine große Fehlerquelle darstellt.

Das folgende Beispiel (hier jeweils mit einem Strichpunkt), könnte auch ohne geschrieben werden:

```
1 a = 3;  
2 b = 4;
```

Dasselbe Beispiel ohne Zeilenumbruch verlangt zwingend ein Semikolon:

```
1 a = 3; b = 4;
```

8.2.4 Kommentare

JavaScript kennt ein- und mehrzeilige *Kommentare*. Sie sind mit den Kommentaren in Java identisch. Ein einzeliges Kommentar wird als hinter zwei Schrägstriche, ein mehrzeiliges zwischen Schrägstrich und Stern sowie Stern und Schrägstrich gesetzt.

```
1 // Ein einzeliges Kommentar  
2 /* Ein  
3    mehrzeiliges  
4    Kommentar */
```

8.3 Datentypen

Als *Datentypen* werden in einer Programmiersprache alle Werte bezeichnet, die manipuliert werden können. Unterschieden wird zwischen primitiven Datentypen wie Zahlen, Literalen oder booleschen Werten sowie komplexen Datentypen wie Arrays und Objekten. Hier sollen zunächst die primitiven Datentypen und Arrays erläutert werden, Objekte folgen ausführlich und gesondert in Vorlesung 9.

8.3.1 Zahlen

JavaScript repräsentiert quasi alle *Zahlen* intern als Gleitkommazahlen nach IEEE 754 Standard [27] (dies entspricht `double`-Zahlen in Java). Dennoch können in JavaScript genauso Ganzzahlen (in den verschiedensten Zahlensystemen) angegeben werden. Der Datentyp lautet schlicht `Number`.

Ganzzahlen

Ganzzahlen können als Dezimal-, Hexadezimal-, Oktal- oder Binärzahlen angegeben werden.

- Ganzzahlen zur Basis 10 werden wie gewohnt angegeben:
 - 0,
 - -3,
 - 1234.
- Hexadezimalzahlen (Basis 16) werden durch ein vorangestelltes „0x“ angegeben:
 - 0xff,
 - 0xa5,
 - 0x145.
- Oktalzahlen (Basis 8) werden durch ein vorangestelltes „0o“ angegeben:
 - 0o377,
 - 0o27,
 - 0o11.
- Binärzahlen (Basis 2) werden durch ein vorangestelltes „0b“ deklariert:
 - 0b11111111,
 - 0b0101,
 - 0b010011010010.

Gleitkommazahlen

Gleitkommazahlen werden mit Dezimalpunkt angegeben. Danach folgen die Nachkommastellen oder optional auch eine Exponentialsyntax. Letztere wird mit einem „e“, gefolgt vom Exponenten notiert und bedeutet $\cdot 10^x$ (sprich: „Mal zehn hoch x“).

Einige Beispiele:

- 3.1415,
- -2345.67,
- 6.02e23 (entspricht $6,02 \cdot 10^{23}$),
- 1.4738e-32 (entspricht $1,4738 \cdot 10^{-32}$).

NaN

Berechnungen oder Operationen, die keinen gültigen Zahlenwert liefern, werden durch den Wert NaN (not a number – keine Zahl) repräsentiert. Dies geschieht etwa, wenn versucht wird, eine Zeichenkette mit Buchstaben in eine Zahl umzuwandeln.

BigInt

Der Standard-Datentyp für Zahlen (**Number**) stößt bei sehr großen Ganzzahlen an seine Grenzen. Da er intern auf dem IEEE 754 Standard für Gleitkommazahlen basiert, ist

der Bereich für *sichere* Ganzzahlen auf $\pm(2^{53} - 1)$ beschränkt. Zahlen, die darüber hinausgehen, verlieren ihre Präzision, was zu Berechnungsfehlern führen kann.

Um beliebig große Ganzzahlen exakt darstellen zu können, wurde der Datentyp **BigInt** eingeführt. Im Gegensatz zu **Number** wird **BigInt** nicht in einer festen Bit-Breite (64-Bit) abgelegt. Stattdessen reserviert der Datentyp dynamisch Speicher, je nachdem wie groß die gespeicherte Zahl ist. Ein **BigInt** wird erstellt, indem dem Ganzzahl-Literal ein **n** angehängt wird, z. B. `9007199254740991n`

Da sich die interne Repräsentation von **BigInt** und **Number** grundlegend unterscheidet, erlaubt JavaScript (aus Sicherheitsgründen, um Präzisionsverlust zu vermeiden) keine direkte Vermischung der beiden Typen in mathematischen Operationen. Sie müssen vor der Verrechnung explizit in den jeweils anderen Typ konvertiert werden. Zudem ist zu beachten, dass es sich um reine Ganzzahlen handelt. Eine Division schneidet Nachkommastellen ersatzlos ab.

8.3.2 Zeichenketten

Als *Zeichenkette* oder auch *String* wird eine Sequenz an Unicode-Zeichen bezeichnet, die zwischen einfachen (') oder doppelten (") Anführungszeichen gesetzt wird. Dabei ist zu beachten, dass doppelte Anführungszeichen in einem String, der mit einfachen Anführungszeichen begrenzt wird, vorkommen dürfen und umgekehrt. Zeichenketten müssen im Code immer in einer Zeile angegeben werden, es darf kein Zeilenumbruch erfolgen. Soll ein Zeilenumbruch im Text verwendet werden, so kann dieser durch `\n` erzeugt bzw. repräsentiert werden.

Einige Beispiele:

- `""` (leerer String),
- `'testing'`,
- `"3.14"` (die Zahl 3,14 als String),
- `'<p title="test">'` (doppelte Anführungszeichen sind innerhalb der einfachen möglich),
- `"Hier wird\numgebrochen"` (das Wort „umgebrochen“ steht dabei in der nächsten Zeile).

Mit dem Backslash (\) können weitere, so genannte *Escape-Zeichen* erzeugt werden:

- `\t`: Tabulator,
- `\"`: Doppelte Anführungszeichen (falls diese im Text eines in doppelten Anführungszeichen gesetzten Strings benötigt werden),
- `\'`: Einfache Anführungszeichen (falls diese im Text eines in einfachen Anführungszeichen gesetzten Strings benötigt werden),
- `\\`: Backslash.

Template Literals und Stringkonkatenation

Seit ECMAScript 2015 existiert eine erweiterte Syntax für Zeichenketten, die sogenannten *Template Literals*. Sie werden nicht von einfachen oder doppelten Anführungszeichen umschlossen, sondern vom *Backtick* (```, auch Gravis genannt).

Der größte Vorteil dieser Schreibweise ist die sogenannte *String Interpolation*. Anstatt Variablen oder Ausdrücke mühsam mit dem `+`-Operator aneinanderreihen zu müssen (Stringkonkatenation, wie etwa aus Java bekannt), können diese direkt innerhalb des Strings mittels der Syntax `${...}` eingebettet werden.

Im folgenden Beispiel werden drei Variablen angelegt (Details dazu in Abschnitt 8.4) und dann konventionell mit dem `+`-Operator aneinandergehängt. Darunter werden Template Literale verwendet. Hier können Platzhalter definiert werden, in denen die Variableninhalte interpoliert (eingesetzt) werden.

```
1 let name = "Welt";
2 let a = 5, b = 10;
3
4 // Herkömmliche Konkatenation
5 let msgAlt = "Hallo " + name + ", Ergebnis: " + (a + b);
6
7 // Modern mit Template Literals
8 let msgNeu = `Hallo ${name}, Ergebnis: ${a + b}`;
```

Ein weiteres Merkmal von Template Literals ist die Unterstützung von mehrzeiligen Strings. Ein Zeilenumbruch im Code wird dabei direkt als Zeilenumbruch im resultierenden String interpretiert, ohne dass das Steuerzeichen `\n` explizit verwendet werden muss.

Im folgenden Beispiel wird ein kleines HTML-Snippet in einer Variable `html` gespeichert. Die Zeilenumbrüche bleiben so erhalten.

```
1 let html = `

2   <h1>Titel</h1>
3   <p>Ein Absatz über mehrere Zeilen.</p>
4 </div>`;


```

8.3.3 Boolesche Werte

JavaScript kennt ebenfalls das Konzept der *booleschen Werte*, also Werte, mit denen „wahr“ oder „falsch“ dargestellt werden kann. Wie auch in anderen Sprachen üblich, werden diese zwei Zustände durch `true` und `false` repräsentiert.

8.3.4 Triviale Datentypen

JavaScript definiert darüber hinaus zwei so genannte *triviale Datentypen*, nämlich `null` und `undefined`.

`null` wird verwendet, um auszudrücken, dass eine Variable oder ein Objekt explizit keinen Wert enthält, d. h. wird eine Variable angelegt und ihr als Wert `null` zugewiesen, so wird ausgedrückt, dass noch kein konkreter Wert festgelegt wurde (aber, dass dies mit Absicht geschehen ist). `null` darf niemals mit der Zahl 0 verwechselt werden, da diese einen konkreten Wert darstellt, `null` selbst aber nicht.

`undefined` kommt dann zum Einsatz, wenn eine Variable zwar deklariert (d. h. im Code angelegt), dieser aber nie ein Wert zugewiesen wurde (auch nicht `null`). Wird der Inhalt dieser Variable dann ausgegeben, wird `undefined` angezeigt.

8.3.5 Arrays

Ein *Array* ist ein zusammengesetzter, komplexer Datentyp. Es besteht nicht nur aus einem einzelnen Wert, wie die bisher aufgelisteten Datentypen, sondern enthält mehrere davon, also entweder mehrere Zahlen, Zeichenketten oder boolesche Werte. Arrays sind Listen, d. h. mehrere Datentypen werden hintereinander angegeben und in dieser Reihenfolge gespeichert. Um Werte in einem Array abzulegen, werden diese in eckige Klammern (`[]`) gestellt und durch Beistriche voneinander getrennt.

Einige Beispiele:

- `["Yara", "Tariq", "Alex"]`,
- `[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]`,
- `["WEF1VO", 2025, ["YouTube", "Microsoft Teams"]]`.

Im ersten Beispiel wird ein Array, bestehend aus drei Zeichenketten definiert, im zweiten werden zehn Ganzzahlen hinterlegt. Im dritten Beispiel zeigt sich die Besonderheit von Arrays in JavaScript. Es können durchaus auch verschiedene Datentypen in einem Array gespeichert werden. Hier ist der erste Wert eine Zeichenkette, der zweite eine Ganzzahl und der dritte ein weiteres Array, das wiederum zwei Strings enthält.

Arrays sind indexbasiert, d. h. jedem Element wird ein numerischer Index beginnend bei 0 bis $n - 1$ (wobei n die Anzahl der Elemente im Array ist) zugewiesen. Der Zugriff erfolgt über diesen Index, der hinter einer Variable (siehe dazu Abschnitt 8.4) in eckigen Klammern angegeben wird.

Dazu die wieder einige Beispiele. Es wird davon ausgegangen, dass das gemischte Array von oben (drittes Beispiel) in einer Variable `data` gespeichert ist.

- `data[0]` ergibt „WEF1VO“,
- `data[1]` ergibt 2025,
- `data[2]` ergibt `["YouTube", "Microsoft Teams"]`,
- `data[2][0]` ergibt „YouTube“ und
- `data[2][1]` ergibt „Microsoft Teams“.

Die Länge eines Arrays ist in seiner Eigenschaft `length` hinterlegt. Diese wird durch einen Punkt getrennt an den Variablennamen des Arrays gehängt. `data.length` enthält etwa die Länge des Arrays `data`. Im obigen Fall wäre dies der Wert 3, die Eigenschaft `length` zählt also nicht rekursiv (denn der dritte Eintrag im Array ist wieder ein Array, dieses wird jedoch nur als 1 gezählt).

8.4 Variablen

Variablen sind Namen, die einen Wert enthalten können. Variablen sind nötig, um in einem Programm Daten effektiv manipulieren zu können. Wie bereits in Abschnitt 8.1 erwähnt, besitzt JavaScript ein schwaches Typkonzept, d. h. es muss beim Anlegen einer Variable nicht definiert werden, welchen Inhalt sie aufnehmen soll (z. B. Ganzzahlen oder Strings). Dieses Konzept beinhaltet auch die automatische Konvertierung zwischen Datentypen. Soll etwa eine Zahl an einen String angehängt werden, wird diese zunächst automatisch in einen String umgewandelt und dann angehängt.

8.4.1 Variablennamen

Ähnlich wie XML (siehe dazu Vorlesung 7) hat auch JavaScript ein striktes *Benennungsschema*, eine vollständige und gut erläuterte Übersicht bietet dazu [8]:

- Variablennamen müssen mit einem Unicode-Buchstaben⁵, einem Underscore (_) oder einem Dollarzeichen (\$) beginnen.
- Alle folgenden Zeichen dürfen wiederum Unicode-Buchstaben und Zahlen, Underscores (_) oder Dollarzeichen (\$) sein.

Gültige Namen für Variablen sind daher z. B. folgende:

- `i`,
- `meine_variable`,
- `v13`,
- `_dummy`,
- `$str`.

Durch den Unicode-Zeichensatz sind noch weitere, in der Praxis wohl kaum verwendete Kombinationen möglich. Unter <https://mothereff.in/js-variables> existiert ein Tool, um zu überprüfen, ob ein Variablenname in JavaScript gültig ist, oder nicht.

Eine weit verbreitete Konvention bei Variablennamen ist Camel Case, d. h. Variablennamen werden mit einem Kleinbuchstaben begonnen und bei jeder Worttrennung wird ein Großbuchstabe verwendet, z. B. `noOfPersons`.

8.4.2 Deklaration

Variablen können in JavaScript explizit oder implizit *deklariert* (d. h. angelegt) werden.

Eine *explizite Deklaration* erfolgt über das Schlüsselwort `let` in Kombination mit dem gewünschten Namen:

```
1 let i;  
2 let summe;  
3 let nachricht;
```

Es können auch mehrere Variablen in einer Anweisung deklariert werden:

```
1 let i, summe, nachricht;
```

Darüber hinaus können Deklaration und Zuweisung eines Werts (Initialisierung) in einem Schritt erfolgen:

```
1 let i = 0;  
2 let summe = 3 + 5;  
3 let nachricht = "Hallo Welt!";
```

Eine *implizit deklarierte Variable* wird ohne das Schlüsselwort `let` erstellt. In diesem Fall wird einer Variable, die zuvor *nicht* mit `let` deklariert wurde, ein Wert zugewiesen. Was in anderen Programmiersprachen einen Fehler erzeugt, funktioniert in JavaScript (sofern nicht der strikter Modus – siehe Seite 8-11 – verwendet wurde). Es hat allerdings zur Folge, dass die Variable einen globalen Scope bekommt (mehr dazu in Abschnitt 8.7). Der folgende Code ist somit auch gültig:

⁵Dies sind etwa A-Z und a-z, aber auch alle unter <https://codepoints.net/search?IDS=1> aufgeführten Zeichen.


```
1 i = 3;
```

Vorsicht: Das Lesen einer noch nicht deklarierten Variable führt jedoch immer zu einem Fehler. Soll also der Wert der Variable `x` ausgelesen werden, obwohl `x` nirgends im Code bis jetzt vorgekommen ist, liefert dies einen Fehler.

Deklaration mit `var`

Das Schlüsselwort `let` wurde erst mit ECMAScript 2015 eingeführt, davor wurde `var` zur Deklaration verwendet. `let` unterscheidet sich von `var` durch den entstehenden Scope – mehr dazu in Abschnitt 8.7. Deklarationen mit `var` sehen wie folgt aus:

```
1 var i;  
2 var summe, nachricht;  
3 var nachricht = "Hallo Welt!";
```

Da ECMAScript 2015 in aktuellen Browsern vollständig unterstützt ist (siehe dazu [114]), verwendet dieses Skriptum in allen Beispielen `let` (bzw. `const` – siehe Abschnitt 8.4.4). In diversen Internet-Ressourcen oder altem Code wird jedoch noch öfters `var` zu lesen sein, da es seit Beginn die einzig gültige Syntax war.

8.4.3 Loses Typkonzept

Aufgrund des *losen Typkonzepts* ist es möglich, einer Variable verschiedene Datentypen zuzuweisen. Folgender Code funktioniert somit ohne Probleme:

```
1 let x = 42;  
2 ...  
3 x = "fourtytwo";
```

Während dies möglich und erlaubt ist, ist dennoch Vorsicht bei der Verwendung solcher Konstrukte geboten. Nicht selten führt dies zu unerwünschtem Verhalten im eigenen Script und sollte daher wirklich nur dann zum Einsatz kommen, wenn es unbedingt nötig oder beabsichtigt ist. Kommentare im Code, die erklären, warum der Datentyp geändert wird, sind dann sicherlich auch sinnvoll.

8.4.4 Konstanten

Der Wert von Variablen kann zu beliebigen Zeiten geändert werden, sogar der dahinterliegende Datentyp ist nicht festgelegt. Dennoch gibt es Fälle, in denen ein Überschreiben des Werts nicht sinnvoll oder gewünscht ist, für diesen Fall gibt es *Konstanten*. Sie können nur einmal mit einem Wert initialisiert werden und behalten diesen immer bei. Eine Konstante wird mit dem Schlüsselwort `const` deklariert.

Namen von Konstanten werden üblicherweise komplett in Großbuchstaben geschrieben, um sie deutlich von Variablen abzugrenzen. Um Wortgrenzen in Konstanten anzudeuten, wird der Unterstrich (`_`) verwendet. Im folgenden werden drei Konstanten definiert:

```
1 const LIMIT = 3600;  
2 const DEFAULT_NAME = "John Doe";  
3 const ALTER = 7 + 12;
```

Das oben gesagte gilt für primitive Datentypen wie Zahlen oder Strings. Diese sind per Definition unveränderbar, engl. *immutable*. Der Zahlenwert hinter `LIMIT` kann sich nur ändern, wenn eine neue Zahl zugewiesen wird, dabei wird aber eine neue Zahl erzeugt (nicht die bestehende verändert).

Komplexe Datentypen wie etwa Arrays (oder auch Objekte bzw. Funktionen) können ihren Wert aber ändern, ohne dass eine neue Zuweisung an die Variable erfolgt, daher gilt es zu beachten, dass `const` nur eine neue Zuweisung verhindert, die Daten dahinter sich jedoch durchaus ändern können. Der folgende Code etwa erzeugt keinen Fehler.

```
1 const myArray = [1, 2, 3];  
2 console.log("myArray vorher:", myArray); // 1,2,3  
3 myArray.push(4);  
4 console.log("myArray nachher:", myArray); // 1,2,3,4
```

An die `const`-Variable `myArray` wird ein Array mit den Inhalten 1, 2, 3 zugewiesen. Mit der Funktion `push()` (mehr über Funktionen in Abschnitt 8.7) kann in das Array am Ende ein neuer Wert (hier: 4) hinzugefügt werden. Die Zuweisung an `myArray` wird also nicht verändert (die `const`-Bedingung also nicht verletzt), der Wert des Arrays ist jetzt jedoch anders als zum Zeitpunkt der Zuweisung. In solchen Fällen ist es auch oft üblich, die normale camelCase-Schreibweise zu verwenden, um genau diese Tatsache zu verdeutlichen.

Es ist guter Stil, wann immer möglich, `const` (anstatt `let`) zur Variablendeklaration zu verwenden, um zu verdeutlichen, dass keine neue Zuweisung im Code vorgenommen werden wird. Nur wenn tatsächlich neue Zuweisungen nötig sind (etwa bei Schleifen), ist `let` vorzuziehen.

Ausgabe in Beispielen

In einigen Beispielen werden `console.log("...")`-Aufrufe verwendet. Diese ermöglichen es, eine Nachricht in die Konsole der Development-Tools der Webbrowser zu schreiben. Um sie zu sehen, müssen die Development-Tools geöffnet und dort auf den „Console“-Reiter gewechselt werden. Wird die Seite danach aufgerufen, erscheint die Nachricht. Das schreiben von Inhalten ins Dokument (d. h. die Webseite) wird in Vorlesung 10 behandelt.

8.4.5 Strikter Modus

Seit ECMAScript 5 besitzt die Sprache einen sogenannten *strikten Modus*. Dieser muss von Entwickler*innen bewusst in den eigenen Skripten aktiviert werden und erzwingt eine strengere Auslegung der JavaScript-Regeln. Was sich zunächst eher negativ anhört, hilft jedoch, sauberen Code zu erstellen und Fehler zu vermeiden, da der strikte Modus bestimmte Konstrukte, die im Standardmodus erlaubt sind, aber unter Umständen problematisch sein können, verbietet und einen Fehler anzeigt. So führen etwa implizite Variablendeklarationen zu einem Fehler, auch dürfen im strikten Modus keine Oktalzahlen mehr als 01234 definiert werden, sondern es muss die Syntax 0o1234 verwendet werden (da sie sonst zu leicht mit Dezimalzahlen verwechselt werden können). Einen guten Überblick über die vollen Auswirkungen des strikten Modus bietet MDN⁶.

⁶https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Strict_mode

Um ein Script in den strikten Modus zu versetzen, muss ganz zu Beginn die folgende Angabe gesetzt werden:

```
1 "use strict";
```

In diesem Skriptum werden alle Beispiele im strikten Modus getestet, wenngleich auch die Angabe aus Platzgründen in der Regel weggelassen wird.

8.5 Operatoren

Operatoren dienen in einer Programmiersprache zur Zuweisung oder zum Vergleichen von Werten, hängen Strings aneinander oder führen bitweise Operationen durch. Die Operatoren in JavaScript gleichen sehr stark denen von C/C++ oder Java.

8.5.1 Zuweisungsoperatoren

Wie bereits in Abschnitt 8.4 verwendet, dient `=` zur *Zuweisung* von Werten an Variablen. Zum Beispiel:

```
1 i = 0;
```

Allgemeiner gesprochen erwartet der Operator auf seiner linken Seite eine Variable und auf seiner rechten einen Wert, der dieser zugewiesen wird. Der Operator ist rechtsassoziativ, d. h. mehrere Operatoren werden immer von rechts nach links ausgewertet. Ein Beispiel:

```
1 i = j = k = 0;
```

Hier wird zunächst `k` der Wert 0 zugewiesen. Danach wird `j` der Wert von `k` (auch 0) zugewiesen. Schließlich erhält `i` den Wert von `j` (wieder 0).

Zuweisung mit Operation

Zusätzlich zur normalen Zuweisung existieren auch Operatoren, die eine Zuweisung mit einer integrierten Operation durchführen. Diese sind:

- `+=`: Addition und Zuweisung,
- `-=`: Subtraktion und Zuweisung,
- `*=`: Multiplikation und Zuweisung,
- `/=`: Division und Zuweisung,
- `%=`: Modulo und Zuweisung.

Diese Operatoren können immer dann zum Einsatz kommen, wenn zum aktuellen Wert einer Variable ein neuer Wert addiert/subtrahiert/etc. wird und das Ergebnis wiederum der Variable zugewiesen werden soll.

Anstatt also

```
1 summe = summe + zusatz;
```

zu schreiben, können diese Addition und anschließende Zuweisung auch mit dem entsprechenden Operator gelöst werden:

```
1 summe += zusatz;
```

8.5.2 Vergleichsoperatoren

Ein *Vergleichsoperator* vergleicht zwei Werte und gibt immer einen booleschen Ausdruck (**true** oder **false**) zurück. Je nachdem, ob der Vergleich zutrifft, oder nicht.

Um auf Wertegleichheit zu überprüfen, dient der Operator **==**. Um zu überprüfen, ob zwei Werte ungleich sind, kommt **!=** zum Einsatz. Zwei Beispiele:

```
1 3 == 4 // false
2 "hallo" != "welt" // true
```

Um nicht nur auf (un-)gleiche Werte, sondern auch zusätzlich auf den (un-)gleichen, dahinter liegenden Typ zu überprüfen, kommen die Operatoren **===** bzw. **!==** zum Einsatz. Dies ist aufgrund des losen Typkonzepts oft sinnvoll, denn JavaScript führt – etwa bei Vergleichsoperationen – implizite Typumwandlungen durch. Werden etwa eine Zahl und ein String verglichen, so wird versucht, den String in eine Zahl umzuwandeln. Sind in der Zeichenkette dann auch nur Zahlen enthalten, gelingt dies und eine normale Vergleichsoperation ergibt **true**:

```
1 1 == "1" // ergibt true, da der String implizit umgewandelt wird
```

Mit dem Wert- und Typ-Vergleichsoperator wird auf die ursprünglichen Datentypen Rücksicht genommen, sodass der Vergleich **false** ergibt:

```
1 1 === "1" // ergibt false, da 1 eine Zahl, "1" hingegen ein String ist
```

Um auf Verhältnisse zwischen zwei Werten zu überprüfen, dienen die folgenden Operatoren:

- **<**: kleiner als,
- **>**: größer als,
- **<=**: kleiner oder gleich als,
- **>=**: größer oder gleich als.

Dazu jeweils ein Beispiel:

```
1 3 < 4 // true
2 "hallo" > "welt" // false
3 5 <= 5 // true
4 "hallo" >= "welt" // false
```

Bei Strings wird lexikografisch verglichen. In beiden Fällen kommt *h* vor *w* (d.h. $h < w$), weswegen der Vergleich immer **false** ergibt.

8.5.3 Arithmetische Operatoren

Arithmetische Operatoren repräsentieren die Grundrechnungsarten und erfüllen auch einige andere Aufgaben. Folgende stehen dabei zur Verfügung:

- **+**: Addition, Zusammenhängen (Konkatenation) von Strings,
- **-**: Subtraktion,
- *****: Multiplikation,
- **/**: Division,
- **%**: Modulo (Rest der Division),
- ******: Potenzierung (x^y),

- `++`: Inkrement,
- `--`: Dekrement.

Modulo gibt (im Unterschied zu einer Division) nicht das Ergebnis der Division, sondern den Rest einer (ganzzahligen) Division aus. Während $5 / 2$ (also $\frac{5}{2}$) 2,5 ergibt, so ist das Ergebnis von $5 \% 2$ der Wert 1, denn „5 dividiert durch 2 ist 2 mit 1 Rest“.

Die Inkrement- und Dekrement-Operatoren funktionieren ähnlich wie die Zuweisungsoperatoren mit Operation. Zu einem Wert wird 1 addiert oder subtrahiert und das Ergebnis gleich zugewiesen. Somit entsprechen

```
1 i++;
2 j--;
```

den folgenden Zuweisungen:

```
1 i = i + 1;
2 j = j - 1;
```

Ebenso sind die Operatoren (wie in C/C++), auch als Prefix-Inkrement und Prefix-Dekrement vorhanden:

```
1 ++i;
2 --i;
```

Der Unterschied ist dabei nur dann sichtbar, wenn die Operatoren in Kombination mit anderen Operationen verwendet werden:

```
1 let i = 10, j = 20;
2 let m = i-- + j; // 30
```

Hier erhält m den Wert 30, i ist danach 9 und j bleibt 20. Beim Postfix-Operator wird also zuerst i verwendet (mit dem Wert 10) und dann dekrementiert.

```
1 let i = 10, j = 20;
2 let m = --i + j; // 29
```

Im Vergleich dazu ist das Ergebnis von m hier 29, i ist wiederum 9 und j bleibt 20. Beim Prefix Operator wird also zunächst i dekrementiert und dann erst in der Operation verwendet.

8.5.4 Logische Operatoren

Logische Operatoren werden für boolesche Algebra verwendet d.h. es werden mehrere boolesche Ausdrücke miteinander verkettet und dadurch das gesamte Ergebnis (wiederum **true** oder **false**) ausgewertet. Sie kommen in der Regel zusammen mit Vergleichen und den Schlüsselwörtern **if**, **while** oder **for** vor. Folgende Operatoren gehören zu dieser Gruppe:

- `&&`: logisches UND,
- `||`: logisches ODER,
- `!`: logisches NICHT.

Das logische UND (`&&`) verknüpft zwei boolesche Ausdrücke A und B miteinander. Das Ergebnis ist dann **true**, wenn sowohl A als auch B **true** ergeben.

Mit dem logischen ODER (`||`) werden ebenfalls zwei boolesche Ausdrücke A und B miteinander verknüpft. Hier ist das Ergebnis dann **true**, wenn entweder A oder B oder beide **true** ergeben.

Das logische NICHT (!) ist ein unärer Operator (erwartet also nicht links und rechts einen Ausdruck), der vor einen Ausdruck *A* gesetzt wird und dessen booleschen Wert umkehrt. Ist *A* zunächst **true** ergibt **!A false** und umgekehrt.

Zum Beispiel:

```
1 true && false // false
2 (3 > 4) || (5 > 3) // true
3 !(3 == 3) // false
```

8.5.5 Bitweise Operatoren

JavaScript unterstützt auch die *Bit-Operatoren* & (bitweises UND), | (bitweises ODER), ^ (bitweises Exklusiv-ODER), ~ (bitweises NICHT) sowie << (bitweise Verschiebung nach links) und >> (bitweise Verschiebung nach rechts). Mehr zu Bit-Operatoren folgt in den fortführenden Programmierlehrveranstaltungen bzw. in der Lehrveranstaltung Data Analysis.

8.5.6 Sonstige Operatoren

JavaScript stellt noch eine Reihe weiterer Operatoren zur Verfügung, von denen vor allem der konditionale Operator **?:** erwähnenswert ist. Er ist die Kurzschreibweise einer **if-else**-Bedingung und wird folgendermaßen verwendet: Vor dem **?** steht ein boolescher Ausdruck, der ausgewertet wird. Ergibt er **true**, so wird der Teil zwischen **?** und **:** verwendet, bei **false** kommt der Teil nach dem **:** zum Zug. Ein konkretes Beispiel:

```
1 x > 0 ? x * y : -x * y;
```

Vor dem **?** wird zunächst der Ausdruck **x > 0** ausgewertet. Ergibt er **true** (weil **x** z.B. 5 ist), so wird **x * y** ausgeführt. Ergibt der Ausdruck **false** (weil der Wert von **x** z.B. -3 ist), so wird **-x * y** ausgeführt.

Dies könnte mit **if-else** folgendermaßen formuliert werden:

```
1 if (x > 0) {
2   x * y;
3 } else {
4   -x * y;
5 }
```

Mehr zu Kontrollstrukturen wie **if-else** im folgenden Abschnitt.

8.6 Kontrollstrukturen

Mit *Kontrollstrukturen* können Programmabläufe gesteuert werden. Es werden Verzweigungen und Schleifen unterschieden.

8.6.1 Verzweigungen

Wie in C/C++, Java und anderen Sprachen gibt es auch in JavaScript die **if-else**-Bedingung und das **switch**-Statement, um *Verzweigungen* zu realisieren.

if-Bedingungen unterliegen der folgenden Syntax:

```
1 if (Ausdruck) {  
2   Anweisung  
3 }
```

Ergibt der boolesche Ausdruck in der Klammer **true**, wird die Anweisung in den geschweiften Klammern ausgeführt. Optional kann noch ein **else**-Zweig hinzugefügt werden, der dann ausgeführt wird, wenn der boolesche Ausdruck nicht zutrifft. Folgendes Beispiel enthält **if** mit zugehörigem **else**-Zweig:

```
1 if (name !== "") {  
2   username = name;  
3 } else {  
4   username = "John Doe";  
5 }
```

Sollen mehrere Bedingungen hintereinander getestet werden, kann im **else**-Zweig wiederum eine **if**-Bedingung angereicht werden:

```
1 if (n === 1) {  
2   // Code 1...  
3 } else if (n === 2) {  
4   // Code 2...  
5 } else if (n === 3) {  
6   // Code 3...  
7 } else {  
8   // Restlicher Code, wenn nichts zutrifft.  
9 }
```

Diese Art von **if**-Kaskade ist jedoch weder einfach zu warten noch leicht zu lesen, daher kann stattdessen eine **switch**-Anweisung zum Einsatz kommen. Die generelle Syntax lautet:

```
1 switch(Ausdruck) {  
2   case Fall1:  
3     Anweisung  
4     break;  
5   case Fall2:  
6     Anweisung;  
7     break;  
8   default:  
9     Anweisung;  
10    break;  
11 }
```

Der Ausdruck in der Klammer nach dem **switch**-Schlüsselwort wird ausgewertet, danach wird überprüft ob einer der Werte in den einzelnen **case**-Ausdrücken damit übereinstimmt. Wenn ja, wird der jeweilige Code ausgeführt. Ist dies nicht der Fall, kommt der Code beim **default**-Schlüsselwort zum Zug. Hinter jedem **case**-Ausdruck muss eine **break**-Anweisung stehen, damit wirklich nur diese eine Anweisung ausgeführt wird. Fehlt das **break**-Schlüsselwort, wird der nachfolgende **case**-Abschnitt auch ausgeführt.

Das folgende **switch**-Statement gleicht der **if**-Kaskade von oben:

```
1 switch(n) {  
2   case 1:  
3     // Code 1...  
4     break;
```

```
5   case 2:
6     // Code 2...
7     break;
8   case 3:
9     // Code 3...
10    break;
11  default:
12    // Restlicher Code, wenn nichts zutrifft.
13    break;
14 }
```

8.6.2 Schleifen

JavaScript bietet wie viele andere Programmiersprachen auch die drei bekannten (klassischen) *Schleifen* **while**, **do/while** und **for**.

Eine **while**-Schleife hat die folgende Syntax:

```
1 while (Ausdruck) {
2   Anweisung
3 }
```

Solange der Ausdruck in der Klammer der **while**-Schleife **true** ergibt, wird die Anweisung im Schleifenkörper ausgeführt. Die folgende Schleife wird genau 10 Mal durchlaufen, danach ist die Bedingung nicht mehr gültig und die Schleife wird nicht mehr betreten:

```
1 let count = 0;
2 while (count < 10) {
3   console.log(count);
4   count++;
5 }
```

Während bei einer **while**-Schleife der Code erst ausgeführt wird, wenn die Bedingung als gültig anerkannt wird, wird bei einer **do/while** Schleife der Code zunächst ein Mal ausgeführt und dann überprüft, ob er wiederholt werden soll. Die Syntax sieht wie folgt aus:

```
1 do {
2   Anweisung
3 } while (Ausdruck);
```

Bei der **while**-Schleife ist ersichtlich, dass die Zählervariable, die in der Bedingung ausgewertet wird, selbst hochgezählt werden muss (durch die **count++**-Anweisung). Wird etwa auf diese Zeile vergessen, so wird eine Endlosschleife erzeugt, da die Bedingung immer **true** ergibt. Eine **for**-Schleife ist in solchen Fällen oft praktischer, da die Zählervariable bereits im Schleifenkopf enthalten ist. Eine solche Schleife besteht aus Initialisierung, Abtestung und Erhöhung der Zählervariable:

```
1 for (Initialisierung; Test; Erhöhung) {
2   Anweisung
3 }
```

Um die **while**-Schleife, die genau zehn Mal durchlaufen wurde, als **for**-Schleife zu konstruieren, ist folgender Code nötig:


```
1 for (let count = 0; count < 10; count++) {  
2   console.log(count);  
3 }
```

Alle Werte eines Arrays ausgeben: `for...of`-Schleife

Zusätzlich zu den drei klassischen Schleifentypen existiert die `for...of`-Schleife. Sie iteriert über alle Einträge eines Arrays, ohne dabei einen Zähler vorrücken zu müssen. Die prinzipielle Syntax lautet wie folgt:

```
1 for (Element in Array) {  
2   Anweisung  
3 }
```

Zunächst wird eine Variable definiert, in der bei jedem Schleifendurchlauf der aktuelle Werte des Arrays enthalten sein soll. Mit diesem Wert kann dann im Rumpf der Schleife gearbeitet werden. Folgendes Beispiel durchläuft ein einfaches Array und gibt jeden Eintrag in der Konsole aus.

```
1 let namen = ["Yara", "Tariq", "Alex"];  
2  
3 for (const element of namen) {  
4   console.log(element);  
5 }
```

8.7 Funktionen

Eine *Funktion* ist ein Block mit JavaScript-Code, der einmal definiert und beliebig oft aufgerufen werden kann. Funktionen können so genannte Parameter besitzen. Dies sind Platzhalter, die beim Funktionsaufruf mit Werten (den sogenannten Argumenten) befüllt werden und die das Berechnungsergebnis der Funktion beeinflussen. Dieses Ergebnis kann als Rückgabewert zurückgegeben werden. Dieser Rückgabewert wird dann dem Wert der Variable zugewiesen, der zuvor die Funktion zugewiesen wurde.

8.7.1 Definieren von Funktionen

Eine Funktion wird mit dem Schlüsselwort `function` *definiert*. Danach folgen der Name und ggf. durch Kommas getrennte Parameter in Klammern. Der Code, der beim Funktionsaufruf ausgeführt werden soll, steht in geschweiften Klammern. Die prinzipielle Syntax sieht wie folgt aus:

```
1 function name(param1, param2, ...) {  
2   Anweisung  
3 }
```

Die Konventionen für Funktionsnamen sind dieselben, wie für JavaScript-Variablen (siehe Abschnitt 8.4).

Die folgende Funktion gibt „Hallo Welt“ in der Konsole der Browser-Development-Tools aus.

```
1 function sayHello() {  
2   console.log("Hallo Welt!");  
3 }
```

Parameter angeben

Funktionen können Parameter enthalten, diese werden mit Komma getrennt in Klammern angegeben. Die Parameter können dann im Körper der Funktion verwendet werden. Das Beispiel zeigt die vorige Funktion mit einem Parameter, der die Nachricht flexibel übergeben lässt:

```
1 function saySomething(message) {  
2   console.log(message);  
3 }
```

Parameter, deren Werte primitive Datentypen sind (Zahlen, Zeichenketten, boolesche und triviale Datentypen) werden immer nach dem *call-by-value* Prinzip übergeben. Dies bedeutet, dass der Wert in der Funktion nur eine Kopie des angegebenen Werts ist. Wird die Variable `message` in der Funktion verändert, wirkt sich das auf die Variable außerhalb, die an die Funktion übergeben wird, nicht aus. Ein Beispiel:

```
1 // Eine Nachricht wird in text hinterlegt  
2 const text = "Der Text bleibt."  
3  
4 // Die Funktion hängt " So schauts aus." an die Nachricht dran.  
5 function saySomethingWithChange(message) {  
6   message += " So schauts aus.";  
7   console.log(message);  
8 }  
9  
10 // Die Funktion wird aufgerufen.  
11 // Ausgegeben wird "Der Text bleibt. So schauts aus."  
12 saySomethingWithChange(text);  
13 // In text steht aber immer noch "Der Text bleibt."  
14 console.log(text);
```

Objekte und auch Arrays werden jedoch *call-by-reference* übergeben. Werden derartige Parameter innerhalb der Funktion verändert, wirkt sich das auf das Objekt außerhalb auch aus. Details dazu in Vorlesung 9.

Rückgabewerte definieren

Ob die Funktion einen Rückgabewert hat, ist aus der Funktionsdefinition nicht erkennbar. Dazu muss lediglich in der letzten Zeile das Schlüsselwort **return** gefolgt vom zurückzugebenden Wert angegeben werden. Die folgende Funktion quadriert einen Wert und gibt das Ergebnis zurück:

```
1 function quadrat(x) {  
2   return x * x;  
3 }
```

8.7.2 Aufrufen von Funktionen

Wie schon zuvor bei der Parameterdefinition angedeutet, wird eine Funktion einfach durch Angabe ihres Namens mit eventuellen Argumenten in Klammern *aufgerufen*. Hat eine Funktion keine Parameter, werden die Klammern ohne Inhalt angegeben. Hat die Funktion einen Rückgabewert (**return** in der letzten Zeile), kann das Ergebnis auch gleich einer Variable zugewiesen werden. Dazu einige Beispiele:

```
1 quadrat(5); // Ruft quadrat() mit dem Wert 5 auf
2 saySomethingWithChange(message); // Ruft saySomethingWithChange() mit message als
   Argument auf
3 irgendwas(); // Ruft irgendwas() ohne Argument auf
4 const total = addValues(5, 2); // Aufruf von addValues() mit Zuweisung
```

Parameter vs. Argument

Unter *Parameter* versteht man die Angaben, die bei der Definition der Funktion in Klammern angegeben werden, also etwa *x*, wenn die Funktion als **function quadrat(x)** definiert wird.

Als *Argument* wird der Wert bezeichnet, der beim Aufruf für diesen Parameter eingesetzt wird. Bei **quadrat(5)** ist etwa 5 das Argument des Aufrufs.

Funktionen mit weniger Argumenten als Parametern aufrufen

JavaScript-Funktionen können auch mit *weniger* Argumenten aufgerufen werden, als in der Funktionsdefinition Parameter angegeben wurden. Jeder Parameter, der nicht durch ein Argument belegt wird, erhält in der Funktion dann den Wert **undefined**.

Angenommen, es gibt die folgende Funktion, die Vorname und Nachname einer Person auf die Konsole ausgibt:

```
1 function logData(first, last) {
2   console.log("Anzahl der Argumente:", arguments.length);
3   console.log("Vorname:", first);
4   console.log("Nachname:", last);
5 }
```

Mit dem Aufruf **logData("John", "Doe")** ist die Ausgabe wie folgt:

- Anzahl der Argumente: 2
- Vorname: John
- Nachname: Doe

Wird die Funktion nur mit einem Argument aufgerufen – **logData("John")** – ist die Ausgabe wie folgt:

- Anzahl der Argumente: 1
- Vorname: John
- Nachname: undefined

Funktionen mit mehr Argumenten als Parametern aufrufen

Auch der umgekehrte Fall ist möglich. Werden beim Aufruf einer Funktion mehr Argumente angegeben, als Parameter in der Signatur vorhanden sind, so werden diese von links nach rechts den Parametern zugeordnet. Alle „überschüssigen“ Argumente sind nicht über Parameter zugreifbar.

Sie sind dennoch nicht verloren, sondern können über das Array **arguments** erreicht werden. **arguments[0]** enthält das erste Argument, **arguments[1]** das zweite usw. Hat eine Funktion nun zwei Parameter, so kann überprüft werden, ob **arguments** länger als 2 ist und diese überschüssigen Werte verwendet werden.

Die Funktion **logData()** könnte wie folgt mit überschüssigen Argumenten umgehen:

```
1 function logData(first, last) {
2   console.log("Anzahl der Argumente:", arguments.length);
3   console.log("Vorname:", first);
4   console.log("Nachname:", last);
5   if (arguments.length > 2) {
6     for (let i = 2; i < arguments.length; i++) {
7       console.log("Zusätzlich:", arguments[i]);
8     }
9   }
10 }
```

Sind mehr als zwei Argumente vorhanden, wird eine Schleife über die Einträge im `arguments`-Array (beginnend beim Eintrag 2) gestartet und jeder Eintrag wird separat auf die Konsole ausgegeben.

Der Aufruf `logData("John", "Doe", 23, "Doeville")` erzeugt diese Ausgabe:

- Anzahl der Argumente: 4
- Vorname: John
- Nachname: Doe
- Zusätzlich: 23
- Zusätzlich: Doeville

Funktionsaufrufe mit einer variablen Anzahl an Argumenten

Eine flexiblere, seit ECMAScript 2015 verfügbare Variante, mit einer variablen Anzahl von Argumenten umzugehen, sind so genannte *Rest-Parameter*. Sie werden mit drei Punkten vor dem Parameternamen angegeben, z.B. `...numbers`. Eine Funktion `addValues()`, die eine beliebige Anzahl von Zahlen addieren soll, könnte mit Hilfe eines Rest-Parameters wie folgt geschrieben werden:

```
1 function addValues(...numbers) {
2   let sum = 0;
3   for (let number of numbers) {
4     sum += number;
5   }
6   return sum;
7 }
8
9 console.log("3 Argumente:", addValues(1, 2, 3)); // 6
10 console.log("5 Argumente:", addValues(1, 2, 3, 7, 11)); // 24
11 console.log("10 Argumente:", addValues(1, 2, 3, 7, 11, 81, 19, 2, 33, 10)); // 169
```

Der Aufruf kann nun mit einer beliebigen Anzahl an Argumenten, getrennt durch Beistriche erfolgen.

Der Unterschied zum `arguments`-Array ist, dass in diesem *alle* Argumente enthalten sind (auch solche, für die ein Parameter existiert), während in Rest-Parametern nur jene, ohne expliziten Namen enthalten sind (im Beispiel sind dies alle, da der Rest-Parameter der einzige ist).

Standardwerte für Parameter definieren und aufrufen

Wie ersichtlich ist, müssen nicht alle Parameter beim Aufruf einer Funktion angegeben werden. In der Regel ist jedoch `undefined` selten ein gewünschter Wert für nicht belegte

Parameter. Um dieses Problem zu lösen, können seit ECMAScript 2015 für Funktionsparameter auch Standardwerte angegeben werden.

Denken Sie zurück an die Funktion `quadrat(x)`. Diese nimmt ein Argument, multipliziert es mit sich selbst und gibt es zurück:

```
1 function quadrat(x) {  
2   return x * x;  
3 }
```

Würde diese Funktion ohne Parameter aufgerufen, also einfach `quadrat()`, so wäre das Ergebnis `NaN` (not a number). Denn `x` hätte den Wert `undefined` und dies mit sich selbst multipliziert, ergibt keine gültige Zahl.

Um diesen Fehlerfall einfach abzufangen, kann `x` nun mit einem Standardwert, z. B. 0 vorbelegt werden. Dazu muss dieser Wert nur durch ein `=`-Zeichen getrennt neben dem Parameter notiert werden:

```
1 function quadrat(x = 0) {  
2   return x * x;  
3 }
```

Wird nun `quadrat()` aufgerufen, so ist das Ergebnis 0.

8.7.3 Funktionen und der Gültigkeitsbereich von Variablen

Der *Gültigkeitsbereich* (engl. „scope“) einer Variable bestimmt, an welchen Stellen im Programm die Variable gültig, d. h. zugreifbar ist. Unterschieden wird zwischen globalem und lokalem Scope bzw. einer globalen oder lokalen Variable.

Eine globale Variable gilt in der gesamten JavaScript-Datei. Sie wird außerhalb jeder Funktion mit dem `let`- oder `const`- (bzw. `var`-) Schlüsselwort deklariert und gilt somit auch überall im Code – auch innerhalb von Funktionen (außer, sie wird von einer gleichnamigen lokalen Variable überdeckt).

Eine lokale Variable wird innerhalb einer Funktion mit dem `let`- oder `const`- (oder `var`-) Schlüsselwort deklariert und gilt innerhalb der gesamten Funktion bzw. des umgebenden Blocks. Hier dürfen `let`, `const` oder `var` bei der Deklaration auf keinen Fall weggelassen werden, da sonst eine globale Variable erzeugt wird bzw. im strikten Modus ein Fehler erzeugt wird. Lokale Variablen überdecken globale bei Namensgleichheit.

```
1 let scope = "global";  
2  
3 function f() {  
4   let scope = "local";  
5   console.log(scope);  
6 }  
7  
8 f(); // local  
9 console.log(scope); // global
```

Zunächst wird die Variable `scope` angelegt. Da sie außerhalb jeder Funktion deklariert wird, ist sie global. Danach wird die Funktion `f()` angelegt. In ihr wird eine zweite Variable `scope` definiert. Diese hat einen lokalen Gültigkeitsbereich. Da sie gleich wie die globale Variable heißt, überlagert sie in diesem Fall die globale, daher wird beim Aufruf der Funktion `f()` die Zeile `console.log(scope);` auch immer „local“ als Wert

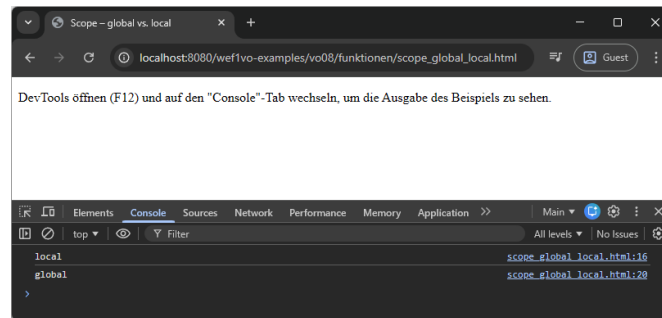


Abbildung 8.1: Die globale Variable wird von der lokalen überdeckt. Die Console zeigt wie erwartet „local“ und „global“ an.

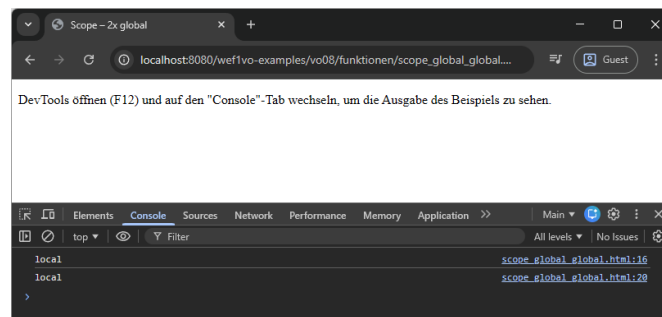


Abbildung 8.2: Die globale Variable wird in der Funktion verändert. Die Console zeigt daher „local“ und „local“ an.

ausgeben. Der zweite Aufruf von `console.log(scope)`; geschieht nun abseits der Funktion. Die dort definierte lokale Variable hat mit dem Ende der Funktion zu existieren aufgehört, daher bleibt nur noch die globale Variable übrig, weshalb der Aufruf auch „global“ zurück gibt. Abbildung 8.1 zeigt das Ergebnis in der Konsole.

Das folgende – leicht abgewandelte Beispiel – demonstriert, was geschieht, wenn die Variable `scope` in der Funktion ohne das `let`-Schlüsselwort verwendet wird. In diesem Fall greift `scope` in der Funktion auf die globale Variable `scope` zu (es wird also nichts überdeckt) und somit wird der Wert der globalen Variable von „global“ auf „local“ geändert. Somit geben sowohl der Funktionsaufruf als auch der Aufruf von `console.log(scope)`; beide Male „local“ aus. Abbildung 8.2 zeigt dies dargestellt in der Konsole.

```
1 let scope = "global";
2
3 function f() {
4   scope = "local";
5   console.log(scope);
6 }
7
8 f(); // local
9 console.log(scope); // local
```

8.7.4 Function Scope vs. Block Scope – var vs. let

Bemerkenswert ist schließlich noch, dass JavaScript vor der Veröffentlichung von ECMAScript 2015 keinen so genannten *Block Scope* besaß. Dieser wirkt sich in anderen Programmiersprachen (wie z. B. C oder Java) so aus, dass Variablen, die in einem Block (ein Block ist alles, das in zwei geschwungenen Klammern steht, wie eine Funktion oder eine `if`-Bedingung) definiert wurden, auch nur dort gelten und außerhalb nicht. In JavaScript traf dies jedoch nicht zu, sodass eine Variable auch außerhalb eines solchen Blocks galt.

Anstatt eines Block Scopes verwendete JavaScript traditionell einen so genannten *Function Scope*. Dies bedeutet, dass Variablen, die innerhalb einer Funktion definiert wurden, in der gesamten Funktion gültig (d. h. deklariert) sind, *egal* in welcher Zeile sie zum ersten Mal angeführt wurde oder anders ausgedrückt, dass Variablen, die in Blöcken innerhalb einer Funktion deklariert wurden, nicht in diesem, sondern im übergeordneten, d. h. in der Funktion gültig waren. Dies wird auch als „Variable hoisting“ (zu Deutsch etwa „Hochziehen von Variablen“) bezeichnet und vom folgenden Beispiel illustriert:

```
1 var scope = "global";
2
3 function f() {
4   var enterBlock = true;
5   console.log(scope); // undefined
6   if (enterBlock) {
7     var scope = "local";
8   }
9   console.log(scope); // local
10 }
11
12 f();
```

Die Variable `enterBlock` wird nur zum Betreten der `if`-Bedingung benötigt. Danach wird zum ersten Mal `console.log(scope)` aufgerufen und `undefined` zurück gegeben. Dies deutet darauf hin, dass nicht die globale Variable ausgegeben wird, sondern die lokale. Diese ist lediglich noch nicht initialisiert worden, aber überdeckt bereits die globale, ist also in der gesamten Funktion gültig. Die Variable selbst wird danach innerhalb der `if`-Bedingung deklariert. Beim zweiten Aufruf von `console.log(scope)` (nachdem `scope` den Wert „local“ zugewiesen bekommen hat), wird auch „local“ ausgegeben. Dies ist nur möglich, da bei der Verwendung von `var` zur Variablendeklaration kein Block Scope existiert und somit `scope` auch nach Ende der `if`-Bedingung noch existiert.

Mit der Einführung des Schlüsselworts `let` unter ECMAScript 2015 ist es nun auch möglich, das eher gewohnte Block Scope-Verhalten zu erzeugen. Jede, mit `let` deklarierte Variable, ist nur in dem Block gültig, in dem sie angelegt wurde. Ein leicht modifiziertes Beispiel demonstriert dies:

```
1 let scope = "global";
2
3 function f() {
4   let enterBlock = true;
5   console.log(scope); // global
6   if (enterBlock) {
7     let scope = "local";
8     console.log(scope); // local
```

```
9   }  
10  console.log(scope); // global  
11 }  
12  
13 f();
```

Zunächst wird eine globale Variable `scope` angelegt. Hier macht es keinen Unterschied, ob `let` oder `var` verwendet wird. Ebenso bei der Definition der Variable `enterBlock`. Da diese auf Funktionsebene angelegt wird, gilt sie – egal ob mit `let` oder `var` – in der gesamten Funktion (da dies der umgebende Block ist). Innerhalb der `if`-Bedingung bewirkt `let`, dass die lokale Variable `scope` nur in derselben gültig ist. Somit überlagert sie nur hier die globale Variable, beim ersten Aufruf von `console.log(scope)` jedoch nicht (weswegen dort auch „global“ und nicht „undefined“ ausgegeben wird).

Das doch ungewohnte Verhalten eines Function Scopes ist der Grund, weshalb eine Variablendeklaration mittels `let` auf jeden Fall bevorzugt werden sollte.

8.7.5 Funktionen: Ein Beispiel

Das folgende Beispiel für eine Funktion prüft ab, ob es sich bei einer Zahl um eine Primzahl handelt. Zunächst wird die Funktion definiert und danach mit einer Zahl aufgerufen. Das Ergebnis wird in der Konsole angezeigt.

```
1 function isPrime(number) {  
2   let isPrimeNumber = true;  
3   let i = 2;  
4   while (i * i <= number && isPrimeNumber) {  
5     if (number % i === 0) {  
6       isPrimeNumber = false;  
7     }  
8     i++;  
9   }  
10  return isPrimeNumber;  
11 }  
12  
13 const x = 5;  
14 console.log("%d ist %s Primzahl", x, isPrime(x) ? "eine" : "keine");
```

Der Funktion wird eine Zahl `number` übergeben. Die Funktion merkt sich mit der booleschen Variable `isPrimeNumber` ob die Zahl eine Primzahl ist. Zunächst wird immer davon ausgegangen. Danach wird die zu überprüfende Zahl beginnend bei 2 durch jede Zahl dividiert, die kleiner als $\sqrt{\text{number}}$ ist. Genauer gesagt wird der Modulo-Operator verwendet, denn wenn eine Division keinen Rest erzeugt (Modulo ergibt 0), dann wurde eine Zahl gefunden, durch die `number` teilbar ist. Somit kann es sich um keine Primzahl mehr handeln. Schließlich gibt die Funktion `true` oder `false` zurück, je nachdem, ob es sich um eine Primzahl handelt oder nicht.

Danach wird die Funktion aufgerufen – in diesem Fall mit der Variable `x`, die den Wert 5 hat. In der letzten Zeile wird der Output in der Konsole generiert, dabei wird mit Hilfe des konditionalen Operators entweder „eine“ oder „keine“ eingesetzt, je nachdem, ob `isPrime(x)` `true` oder `false` zurück liefert.

8.7.6 Anonyme Funktionen und Arrow Functions

Im Verlauf dieses Abschnitts wurden Funktionen immer mit einem Namen definiert (`function name (...) { }`). Da Funktionen in JavaScript jedoch wie Werte behandelt werden, können sie auch ohne Namen erstellt und direkt in Variablen gespeichert werden. Dies nennt man *anonyme Funktionen*.

Seit ECMAScript 2015 gibt es dafür eine besonders kompakte Schreibweise: die *Arrow Functions* (Pfeilfunktionen). Sie sind das JavaScript-Äquivalent zu den Lambda-Ausdrücken in Java, verwenden jedoch einen „fetten Pfeil“ (`=>`) statt des Bindestrichs (`->`).

Klassische anonyme Funktionen werden mit dem Schlüsselwort `function()` erstellt, es folgt jedoch kein Funktionsname. Dieser ist nicht nötig, wenn die Funktion einer Variable zugewiesen wird, denn über deren Namen kann sie aufgerufen werden:

```
1 const calcAsAnonFunction = function(x) {  
2   return x * x;  
3 };  
4  
5 console.log("Anonyme Funktion:", calcAsAnonFunction(4));
```

Arrow Functions verzichten auf das Schlüsselwort `function()`. Es wird nur eine runde Klammer gesetzt, die Parameter enthalten kann (nicht muss). Danach folgt ein Pfeil `=>` und dann die geschwungenen Klammern mit dem Code.

```
1 const calcAsArrowFunction = (x) => {  
2   return x * x;  
3 };  
4  
5 console.log("Arrow Funktion:", calcAsArrowFunction(4));
```

Hat eine Arrow Function genau einen Parameter, können die runden Klammern um diesen weggelassen werden. Besteht der Funktionskörper zudem aus nur einer einzigen Anweisung, können auch die geschwungenen Klammern und das `return` entfallen. Dies macht die Syntax extrem kompakt:

```
1 const calcAsShortArrowFunction = x => x * x;  
2  
3 console.log("Kurze Arrow-Funktion:", calcAsShortArrowFunction(4));
```

Wann wird welche Syntax verwendet?

Arrow Functions haben sich in der modernen JavaScript-Entwicklung für kurze, prägnante Operationen durchgesetzt.

Vorteile von Arrow Functions:

- **Kürze:** Der Code wird deutlich lesbarer, besonders bei Einzeilern.
- **Kontext (`this`):** Arrow Functions haben keinen eigenen `this`-Kontext, sondern erben ihn aus der Umgebung. Dies ist ein entscheidender Vorteil bei der Programmierung von Klassen oder Event-Handleern (mehr dazu in den Vorlesungen 9 und 10).

Nachteile/Einschränkungen:

- Sie besitzen kein eigenes **arguments**-Objekt (wie in Abschnitt 8.7.2 erwähnt).
- Sie können nicht als Konstruktor für Objekte verwendet werden.

In der Praxis werden Arrow Functions vor allem dann verwendet, wenn eine Funktion als Argument an eine andere Funktion übergeben wird (sogenannte *Callbacks*), etwa bei Event-Handleern („Klick auf Button“) oder bei der Verarbeitung von Arrays. Mehr zu Events in Vorlesung 10.

8.7.7 Vordefinierte Funktionen

JavaScript (bzw. genauer ECMAScript) bietet eine Reihe von vordefinierten Funktionen. Die folgende Aufzählung ist nur ein Ausschnitt, die vollständige Liste kann unter [14, Abschnitt 19.2] oder beim MDN⁷ nachgeschlagen werden.

- **isFinite(number)**: Überprüft, ob es sich bei **number** um eine Zahl handelt oder nicht. Gibt **true** oder **false** zurück.
- **isNaN(value)**: Überprüft, ob **value** keine Zahl (not a number, NaN) ist. Gibt **true** oder **false** zurück.
- **parseFloat(string)**: Wandelt einen String, der eine Gleitkommazahl enthält (z. B. aus einem Eingabeformular) in eine Zahl um und gibt diese zurück.
- **parseInt(string, radix)**: Wandelt einen String, der eine Ganzzahl enthält (z. B. aus einem Formular) in eine Zahl um und gibt diese zurück. Mit dem zweiten Parameter wird der Radix (die Basis des Zahlensystems angegeben; Standard ist 10).
- **encodeURIComponent(uri)**: Kodiert einen vollständigen Uniform Resource Identifier (URI), indem Sonderzeichen durch UTF-8-Escape-Sequenzen ersetzt werden (z. B. wird ein Leerzeichen zu %20). Steuerzeichen wie /, : oder ? bleiben dabei erhalten.

8.8 Weiterführende Literatur

Das derzeit wohl umfangreichste und aktuellste Werk ist [1], das bereits ECMAScript 2015 und neuer abdeckt und auch sonst alle Details zur Sprache auf über 1.200 Seiten enthält.

Ebenso empfehlenswert ist [48], das etwas aktueller, mit ca. 500 Seiten jedoch etwas knapper ausgeführt ist.

Das langjährige Standardwerk zu JavaScript von David Flanagan [17] hat 2020 eine neue Auflage bekommen und kann somit auch noch als (englischsprachige) Referenz angesehen werden. Die vorangegangenen Auflagen sind aufgrund ihres Alters nicht mehr empfehlenswert.

Schließlich findet sich in [58] auch eine Einführung in die Thematik, die zwar oberflächlich bleibt, aber zum Erlernen der Sprache vielen auch ausreichen dürfte.

⁷https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects#function_properties

Vorlesung 9

JavaScript im Browser

9.1 Objekte

JavaScript ist eine objektorientierte Sprache. Da Objekte eine Voraussetzung für die Arbeit mit JavaScript im Browser sind, soll hier zunächst eine kurze Einführung in die Thematik gegeben werden, um ein grundsätzliches Verständnis dafür aufzubauen. Es werden bewusst nicht alle Details und Hintergründe erläutert, da dies in der Lehrveranstaltung *Algorithmisches Denken* geschieht. Ziel in *Web Fundamentals* ist es, dass sie mit Objekten grundsätzlich arbeiten können und ihren Sinn bzw. grundsätzliche Funktionsweise verstehen.

9.1.1 Was ist ein Objekt?

Objekte sind eine weitere Art von Datentyp, zusätzlich zu den in Abschnitt 8.3 erwähnten. Im Gegensatz zu diesen (Arrays ausgenommen) enthalten sie nicht einfach nur einen einzigen Wert, sondern kapseln eine Vielzahl von Daten, die auch thematisch zusammengehören. Sie zählen daher zu den komplexen Datentypen.

Während primitive Datentypen lediglich einen Wert enthalten, bilden Objekte komplexere Inhalte ab die sich an Dingen aus dem realen Leben orientieren. Ein Beispiel:

In JavaScript sollen die Daten und das Verhalten von Studierenden erfasst bzw. abgebildet werden. Erfasst werden sollen Name, Alter, Studiengang und die beherrschten Programmiersprachen. Zudem sollen die beherrschten Programmiersprachen ausgegeben und auch neue erlernt, d. h. hinzugefügt werden können. Mit einfachen, primitiven Datentypen und Funktionen könnte dies so aussehen:

```
1 // Daten des*der Studenten*Studentin
2 let studentName = "John Doe";
3 let studentAge = 22;
4 let studentProgram = "MTD";
5 let studentLanguages = ["Java", "C#"];
6
7 // Ausgabe bzw. Änderung der Daten
8 function showLanguages() {
9   console.log(`${studentName} (${studentAge}, ${studentProgram}) beherrscht diese
      Sprachen: ${studentLanguages}`);
10 }
11
```

```

12 function learnNewLanguage(language) {
13     studentLanguages[studentLanguages.length] = language;
14 }
15
16 // Aufrufe
17 showLanguages(); // Java,C#
18 learnNewLanguage("JavaScript");
19 showLanguages(); // Java,C#,JavaScript

```

Während dies zwar funktioniert, haben die einzelnen Variablen und Funktionen logisch gesehen nichts miteinander zu tun. Vielmehr existieren Sie als globale Variablen und Funktionen im gesamten Script, dass `showLanguages()` etwa mit den Werten von `studentName`, `studentAge`, `studentProgram` oder `studentLanguages` arbeitet ist zwar möglich und auch sinnvoll, müsste aber nicht so sein. Es könnte sich dabei auch um vier Variablen und zwei Funktionen handeln, die völlig unterschiedliche Zwecke haben, da es nichts gibt, dass sie logisch miteinander verbindet. Objekte handhaben dies anders.

Objekte: Datenkapseln mit Eigenschaften und Methoden

Ein Objekt stellt eine solche logische Klammer dar. Es kapselt Daten, die inhaltlich zusammengehören. Dabei bestehen Objekte aus zwei Dingen: *Eigenschaften* (engl. „properties“), auch Attribute genannt, repräsentieren die Daten bzw. den Zustand des Objekts. *Methoden* definieren das Verhalten des Objekts. Sie sind nichts anderes als Funktionen, die allerdings zum Objekt gehören und dessen Daten, d.h. die Eigenschaften ausgeben oder verändern.

Das Studierendenbeispiel als Objekt konstruiert sieht nun wie folgt aus:

```

1 // Daten des*der Studenten*Studentin & Ausgabe bzw. Änderung der Daten
2 const student = {
3     name: "John Doe",
4     age: 22,
5     program: "MTD",
6     languages: ["Java", "C#"],
7     showLanguages() {
8         console.log(`${this.name} (${this.age}, ${this.program}) beherrscht diese
          Sprachen: ${this.languages}`);
9     },
10    learnNewLanguage(language) {
11        this.languages[this.languages.length] = language;
12    }
13 };
14
15 // Aufrufe
16 student.showLanguages(); // Java,C#
17 student.learnNewLanguage("JavaScript");
18 student.showLanguages(); // Java,C#,JavaScript

```

Ersichtlich ist, dass nun alles – sowohl die Daten des*der Studenten*Studentin, als auch die Funktionen zum Ausgeben und Erlernen von Sprachen – in einer einzigen Variable, nämlich `student` gespeichert sind. Diese Variable enthält das Objekt, dessen Bestandteile in den geschweiften Klammern notiert sind.

Die Bezeichner `name`, `age`, `program`, `languages`, `showLanguages` und schließlich `learnNewLanguage` fungieren dabei als die Schlüssel (Keys), über die auf die Inhalte

zugegriffen wird.

Dabei kommen zwei Schreibweisen zum Einsatz:

- **Eigenschaften (Daten):** Handelt es sich beim Wert um einen Datentyp (String, Zahl, Array etc.), so wird das Schema **Schlüssel: Wert** verwendet. `name`, `age`, `program` und `languages` sind solche Eigenschaften.
- **Methoden (Verhalten):** Handelt es sich um eine Funktion, so wird die moderne Kurzschreibweise (Method Shorthand) verwendet. Sie ähnelt der Methodendefinition in Java: Der Schlüssel wird wie ein Funktionsname notiert, gefolgt von Klammern und dem Funktionsrumpf. Das Schlüsselwort `function` und der Doppelpunkt entfallen hier. `showLanguages` und `learnNewLanguage` sind die Methoden dieses Objekts.

Diese Variable, d. h. dieses Objekt, repräsentiert eine*n Studentin*Studenten als logische Einheit. Dies zeigt sich auch im Zugriff: Um die Methode `showLanguages()` auszuführen, muss sie über die angelegte Objekt-Variable aufgerufen werden – also mittels `student.showLanguages()`. Eine globale Funktion `showLanguages()`, die für sich allein steht, existiert in diesem Kontext nicht.

9.1.2 Zugriff auf Eigenschaften und Methoden

Der *Zugriff* auf einzelne Eigenschaften oder Methoden eines Objekts erfolgt immer über eine simple Punktnotation:

```
1 <Name des Objekts>.<Name der Eigenschaft>
2 <Name des Objekts>.<Name der Methode()>
```

Dies war schon in Abschnitt 8.7 zu sehen, als der Aufruf `console.log()` (Output in die Konsole der Developer-Tools schreiben) verwendet wurde. `console` ist ein Objekt, `log()` ist eine Methode dieses Objekts.

Im Studierenden-Beispiel ist `student` das Objekt, der Aufruf zum Ausgeben der beherrschten Sprachen sowie zum Hinzufügen einer neuen Sprache wird wie folgt formuliert:

```
1 student.showLanguages();
2 student.learnNewLanguage("JavaScript");
```

Die erste Methode ist rein zur Ausgabe und manipuliert das Objekt nicht. Die zweite hingegen verändert die Eigenschaft `language`, indem sie einen neuen Eintrag zum Array hinzufügt.

Die Punktnotation gilt auch für Eigenschaften. Will man etwa den Namen des*der Studenten*Studentin abfragen und in der Konsole ausgeben, sowie dieser Person nach dem Geburtstag ein neues Alter setzen, wird dies wie folgt geschrieben:

```
1 console.log(student.name);
2 student.age = 23;
```

9.1.3 Das eigene Objekt mit `this` ansprechen

Bei den Objektmethoden `showLanguages()` und `learnNewLanguage()` fällt auf, dass der Zugriff auf die Eigenschaften für Name, Alter, Studiengang und beherrschten Sprachen nicht einfach über ihren direkten Eigenschaftsnamen geschieht. Vielmehr erfolgt der Zugriff über `this.name`, `this.age`, `this.program` oder `this.languages`.

Das Schlüsselwort **this** repräsentiert – wenn es wie hier innerhalb eines Objekts verwendet wird – immer das Objekt selbst. Auf den konkreten Fall umgelegt, steckt in **this** also das gesamte Studierenden-Objekt, also äquivalent zu **student**. Letzteres ist allerdings nur der Variablenname des Objekts, der ja beliebig gewählt ist und sich jederzeit ändern könnte. Somit wäre es nicht zielführend, **student.name** etc. zu schreiben, denn würde die Variable anders gewählt, würde dies nicht mehr funktionieren.

Mit der Angabe von **this.name** wird also explizit angegeben, dass die Eigenschaft **name** des aktuellen Objekts ausgegeben (oder verändert) werden soll. Würde in der Methode nicht **this.name** sondern nur **name** verwendet, so würde versucht werden, auf eine globale oder auch lokale Variable **name** zuzugreifen.

this und der Kontext

this verweist immer auf das Objekt, in dessen Kontext es gerade verwendet wird. Wird es in einer Objektmethode verwendet, so enthält es das Objekt, zu dem die Methode gehört.

Außerhalb von Objekten existiert **this** auch. Außerhalb jeglicher Funktion, also im globalen Kontext, zeigt **this** immer auf das globale **window**-Objekt. Innerhalb von Funktionen zeigt es im normalen Modus ebenfalls auf das globale **window**-Objekt, im strikten Modus enthält **this** allerdings **undefined**, wenn nicht explizit ein Wert gesetzt wird oder die Funktion etwa über das **window**-Objekt aufgerufen wird.

9.1.4 Definieren von Objekten

Die in Abschnitt 9.1.1 demonstrierte Art, ein Objekt zu definieren, nennt sich Objekt-Literal-Syntax. Sie stellt jedoch nur eine von mehreren Arten dar. Es existieren ebenfalls Konstruktorfunktionen und seit ECMAScript 2015 auch Klassen.

Objekt-Literal-Schreibweise

Die *Objekt-Literal-Schreibweise* ist die einfachste Art und Weise, ein Objekt zu definieren. Nach der Angabe eines Variablennamens werden die Daten des Objekts in geschwungene Klammern geschrieben. Darin werden, durch Beistriche getrennt, Eigenschaften (als **Schlüssel: Wert**) und Methoden (in der Kurzschreibweise) notiert:

```
1 const myObject = {  
2   key1: value1,  
3   key2: value2,  
4   method1() { ... }  
5 };
```

Da es sich um die Definition einer Variable handelt (auch wenn diese über mehrere Zeilen geht), sollte auf den abschließenden Strichpunkt nach der geschlossenen geschwungenen Klammer nicht vergessen werden.

Die Objekt-Literal-Schreibweise ist die schnellste und einfachste Art, in JavaScript Objekte zu erzeugen. Sie eignet sich vor allem für Objekte gut, die primär aus Eigenschaften bestehen, wenngleich die Definition von Methoden auch problemlos möglich ist.

Der Nachteil dieser Syntax ist jedoch, dass hier eine spezifische Instanz eines Objekts direkt erzeugt wird. Wird ein zweites Objekt benötigt, das identisch aufgebaut ist, aber andere Daten enthalten soll, so muss es erneut komplett definiert, d. h. notiert werden.

Dies ist vor allem bei Methoden unerwünscht, weil hier Codeverdoppelung entsteht, da die Methoden ja nur die Namen der Eigenschaften verwenden, also komplett gleich formuliert werden.

Konstruktorfunktionen

Eine *Konstruktorfunktion* behebt das Problem der einzigartigen Objekte, das die Objekt-Literal-Schreibweise mit sich bringt. Sie ist eine Funktion, die zum Erzeugen neuer Objekte verwendet werden kann. Da Konstruktorfunktionen wie reguläre Funktionen aussehen, wird in der Regel Pascal Case, also ein großer Anfangsbuchstabe mit Großbuchstaben an Wortgrenzen verwendet, um sie zu unterscheiden.

Eine Konstruktorfunktion enthält üblicherweise Parameter, die zum Initialisieren der Objekteigenschaften verwendet werden. Die Syntax sieht wie folgt aus:

```
1 function MyConstructorFunction(param1, param2) {  
2   this.key1 = param1;  
3   this.key2 = param2;  
4   this.key3 = function() { ... }  
5 }
```

Die Objekteigenschaften erhalten wieder Namen, die über das `this`-Schlüsselwort notiert werden. Als Werte werden meist die Argumente der Parameter zugewiesen. Zusätzlich können auch Methoden definiert werden, die in diesem Fall Funktionen sind, die weiteren Eigenschaften zugewiesen werden (eine Kurzschreibweise wie beider Objekt-Literal-Syntax existiert hier nicht).

Das Studierendenbeispiel realisiert als Konstruktorfunktion definiert sich folgendermaßen:

```
1 function Student(name, age, program, languages) {  
2   this.name = name;  
3   this.age = age;  
4   this.program = program;  
5   this.languages = languages;  
6   this.showLanguages = function () {  
7     console.log(`${this.name} (${this.age}, ${this.program}) beherrscht diese  
       Sprachen: ${this.languages}`);  
8   };  
9   this.learnNewLanguage = function (language) {  
10    this.languages[this.languages.length] = language;  
11  };  
12 }
```

Die Funktion heißt `Student` und hat vier Parameter, die 1:1 den Eigenschaften `name`, `age`, `program` und `languages` zugewiesen werden. Weiters werden die zwei bekannten Methoden `showLanguages()` und `learnNewLanguage()` definiert, die mit den Eigenschaften arbeiten.

Dies ist jedoch nur die *Definition* der Konstruktorfunktion. Durch sie alleine wird noch kein Objekt erzeugt. Dieser Vorgang geschieht mit dem Schlüsselwort `new` und dem Namen der Konstruktorfunktion:

```
1 const myObject1 = new MyConstructorFunction(arg1, arg2);  
2 const myObject2 = new MyConstructorFunction(arg3, arg4);
```

Das Schlüsselwort **new** macht die an sich „normale“ Funktion zur Konstruktorfunktion. Ersichtlich ist, dass keinerlei **return**-Anweisung darin vorkommt, stattdessen erzeugt die Funktion in Kombination mit **new** implizit ein neues Objekt und gibt es zurück, sodass es einer Variable zugewiesen werden kann. Ebenso ist es nun möglich, mehrere Objekte desselben Typs zu erzeugen, in dem die Konstrukturfunktion mehrmals mit unterschiedlichen Argumenten aufgerufen wird.

Umgelegt auf das Studierendenbeispiel können nun beliebig viele (in diesem Fall zwei) Studierende erzeugt werden. Ihre Struktur, d. h. ihre Eigenschaften und Methoden, ist identisch. Die Werte der Eigenschaften unterscheiden sich jedoch.

```
1 const student1 = new Student("John Doe", 22, "MTD", ["Java", "C#"]);
2 const student2 = new Student("Jane Doe", 25, "SE", ["Pascal", "C", "Java"]);
3
4 student1.showLanguages(); // Java,C#
5 student1.learnNewLanguage("JavaScript");
6 student1.showLanguages(); // Java,C#,JavaScript
7 student2.showLanguages(); // Pascal,C,Java
```

Hinweis zur Performance

Werden Objekte auf diese Weise erstellt, wird für jedes neue Objekt (jede*n Studierende*n) auch jede Methode im Arbeitsspeicher neu angelegt. Bei drei Studierenden existiert die Funktion **showLanguages** also dreimal im Speicher.

In älteren JavaScript-Versionen wurde dies durch komplizierte Zuweisungen an den sogenannten **prototype** gelöst. In modernem JavaScript gibt es dafür eine wesentlich elegantere Lösung, die dieses Problem im Hintergrund automatisch behebt: **Klassen**.

Klassen

Seit ECMAScript 2015 gibt es auch die Möglichkeit, Objekte über die Angabe einer *Klasse* zu definieren. Klassen sind in anderen Programmiersprachen quasi die Baupläne für Objekte. Sie bestehen in der Regel aus einem Konstruktor, der implizit beim Anlegen eines Objekts mit **new** aufgerufen wird und in dem die Zuweisungen der Eigenschaften geschehen und der Definition der Methoden.

Die Klassensyntax in JavaScript imitiert dieses Verhalten. Technisch gesehen basiert sie im Hintergrund weiterhin auf dem Prinzip der Konstrukturfunktionen und Prototypen, weshalb sie oft als syntaktische Alternative („Syntactic Sugar“) bezeichnet wird. Allerdings bietet die **class**-Syntax einen entscheidenden Vorteil: Methoden, die innerhalb der Klasse definiert werden, werden automatisch speicherschonend angelegt (sie landen im sogenannten Prototyp) und nicht für jedes Objekt einzeln kopiert.

Die Syntax lautet wie folgt: mit dem Schlüsselwort **class** wird der Name der „Klasse“ spezifiziert, alle weiteren Angaben folgen in geschweiften Klammern. Der Konstruktor muss immer **constructor()** heißen und wird wie eine Funktion, allerdings ohne das Schlüsselwort **function** angegeben. Alle Methoden werden im Anschluss spezifiziert, das Schlüsselwort **function** entfällt ebenfalls wieder.

```
1 class MyClass {
2   constructor(param1, param2) {
3     this.key1 = param1;
4     this.key2 = param2;
```



```

5   }
6   method1() { ... }
7 }

```

Umgelegt auf das Studierendenbeispiel ergibt sich die folgende Definition:

```

1 class Student {
2   constructor(name, age, program, languages) {
3     this.name = name;
4     this.age = age;
5     this.program = program;
6     this.languages = languages;
7   }
8
9   showLanguages() {
10    console.log(`${this.name} (${this.age}, ${this.program}) beherrscht diese
    Sprachen: ${this.languages}`);
11  }
12
13  learnNewLanguage(language) {
14    this.languages[this.languages.length] = language;
15  }
16 }

```

Das Erzeugen eines neuen Studierenden-Objekts erfolgt genauso wie bei den Konstruktorfunktionen, der Zugriff auf Methoden ist ebenfalls gleich und wird nicht mehr angeführt:

```

1 const student1 = new Student("John Doe", 22, "MTD", ["Java", "C#"]);
2 const student2 = new Student("Jane Doe", 25, "SE", ["Pascal", "C", "Java"]);

```

9.1.5 Vordefinierte Objekte

JavaScript (ECMAScript) bietet eine Reihe von vordefinierten Objekten, die nützliche Funktionalitäten zur Verfügung stellen. Eine vollständige Aufzählung bieten [14, Kapitel 20–28] oder MDN¹, hier werden zunächst einige wichtige überblicksartig beschrieben.

Object

Von diesem Objekt stammen alle anderen Objekte in JavaScript ab, somit ist jedes Objekt – egal ob selbst erstellt oder vordefiniert – immer auch von Typ `Object`.

Ein neues Objekt kann auch wie folgt erzeugt werden, dies ist aber eher ungebräuchlich und in der Regel wird die Objekt-Literal-Syntax stattdessen verwendet.

```

1 const data = new Object();

```

Wesentlich häufiger kommen jedoch die statischen Hilfsmethoden zum Einsatz, um Inhalte eines Objekts als Array auszulesen (etwa um danach mit Array-Methoden weiterzuarbeiten):

- `Object.keys(obj)`: liefert ein Array aller Eigenschaftsnamen (Schlüssel) zurück.
- `Object.values(obj)`: liefert ein Array aller Werte zurück.

¹https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

- `Object.entries(obj)`: liefert ein Array aus Arrays zurück, wobei jedes innere Array ein Paar aus [Schlüssel, Wert] enthält.

```
1 const person = { name: "John", age: 22 };
2 console.log(Object.keys(person)); // ["name", "age"]
3 console.log(Object.values(person)); // ["John", 22]
4 console.log(Object.entries(person)); // [["name", "John"], ["age", 22]]
```

Number

Number ist die Objektversion für alle primitiven Zahlendatentypen. Diese existiert, um mit Zahlen bestimmte Operationen (z. B. Rundung oder Umwandlung) durchführen zu können bzw. um Informationen über bestimmte Zahlenwerte im System zu bekommen.

Entscheidend für Objektversionen von primitiven Datentypen ist, dass sie nicht explizit mit `new` angelegt werden müssen. Es ist also unüblich, eine Zahl in ein **Number**-Objekt zu verpacken, da dies eher unübersichtlich und auch langsamer ist.

Dennoch ist Folgendes möglich: zunächst wird ein **Number**-Objekt angelegt und dann mit der Methode `toFixed()`, die jedes **Number**-Objekt besitzt, auf zwei Nachkommastellen gerundet und dies ausgegeben.

```
1 const number1 = new Number(12.3456);
2 const number1rounded = number1.toFixed(2);
3 console.log(number1rounded); // 12.35
```

Diese Methoden können jedoch auch direkt mit Zahlen, d. h. mit den primitiven Datentypen verwendet werden. JavaScript konvertiert dieses im Hintergrund automatisch in das entsprechende **Number**-Objekt, der Code bleibt jedoch übersichtlich. Das obige Beispiel sollte also immer wie folgt geschrieben werden:

```
1 const number2 = 12.3456;
2 const number2rounded = number2.toFixed(2);
3 console.log(number2rounded); // 12.35
```

Im **Number**-Objekt befinden sich auch diverse statische Eigenschaften, die abgefragt werden können. Dazu werden der Klassenname und die Eigenschaft in der Punktnotation verwendet. Hier werden die größte und kleinste darstellbare Zahl im System abgefragt und ausgegeben:

```
1 console.log(Number.MAX_VALUE); // größte darstellbare Zahl
2 console.log(Number.MIN_VALUE); // kleinste darstellbare Zahl
```

String

Auch Zeichenketten sind Objekte, nämlich vom Typ **String**. Sie könnten ebenso über `const text = new String("Hallo Welt");` erzeugt werden, was aber wiederum unüblich ist. Vielmehr bietet das **String**-Objekt wieder viele nützliche Methoden, darunter die folgenden:

- `charAt(n)`: gibt das Zeichen an der Position *n* zurück (alternativ ist der Zugriff auch wie bei Arrays über `string[n]` möglich),
- `indexOf(char)`: gibt die Position des ersten Vorkommens von `char` zurück (oder `-1`, falls nicht gefunden),

- `includes(char)`: gibt `true` zurück, wenn der String `char` enthält (moderne Alternative zu `indexOf`, wenn die Position egal ist),
- `substring(start, end)`: gibt den Teilstring zwischen den Indizes `start` und `end` (exklusive) zurück,
- `slice(start, end)`: ähnlich wie `substring`, akzeptiert jedoch auch negative Werte, um vom Ende des Strings zu zählen,
- `split(char)`: teilt den String bei allen Vorkommnissen von `char` in ein Array auf.

Die Eigenschaft `length` gibt die Anzahl der Zeichen im String zurück.

```

1  const text = "Das ist ein langer Text";
2
3  // Zeichen an Position ausgeben
4  console.log(text.charAt(4)); // i
5  // Prüfen, ob "langer" enthalten ist (gibt true/false)
6  console.log(text.includes("langer")); // true
7  // Substring: Start (inkl.) bis Ende (exkl.)
8  console.log(text.substring(4, 7)); // ist
9  // Slice: Die letzten 4 Zeichen ausschneiden (Negativer Index)
10 console.log(text.slice(-4)); // Text
11 // String bei den Leerzeichen teilen
12 console.log(text.split(" ")); // ["Das", "ist", "ein", "langer", "Text"]
13 // Länge des Strings
14 console.log(text.length); // 23

```

Date

Das Objekt `Date` gewährt Zugriff auf das aktuelle Datum und die Uhrzeit. Wird ein neues `Date`-Objekt mit `new` erzeugt, so wird das zu diesem Zeitpunkt aktuelle Datum (am ausführenden Rechner) darin gekapselt. Intern wird es durch die Anzahl der Millisekunden repräsentiert, die seit dem 1. Jänner 1970 um 00:00:00 Uhr vergangen sind. Das Objekt bietet aber viele Methoden, um einzelne Aspekte dieses Datums abzufragen oder auszugeben:

- `getFullYear()`: gibt das Jahr im Datum zurück,
- `getDay()`: gibt den Wochentag als Zahl (0–6) zurück,
- `getHours()`: gibt die aktuelle Stunde aus, oder
- `getMinutes()`: gibt die aktuelle Minute zurück.
- `toLocaleDateString()`: gibt einen formatierten, sprachsensitiven Datums-String aus.
- `Date.now()`: Gibt den aktuellen Unix-Timestamp (Millisekunden seit 01.01.1970) zurück (statische Methode).

```

1  const d = new Date();
2
3  // Komplettes Datum ausgeben (entspricht d.toString())
4  console.log(d); // z.B. Fri Jan 09 2026 13:29:12 GMT+0100 (Central European Standard
   Time)
5  // Jahr ausgeben
6  console.log(d.getFullYear()); // z.B. 2026
7  // Wochentag ausgeben

```

```

8 console.log(d.getDay()); // z.B. 5 für Freitag
9 // Stunde ausgeben
10 console.log(d.getHours()); //z.B. 13
11 // Minute ausgeben
12 console.log(d.getMinutes()); // z.B. 29
13 // Formatiert für Österreich ausgeben
14 const options = { weekday: "long", year: "numeric", month: "long", day: "numeric" };
15 console.log(d.toLocaleDateString("de-AT", options)); // z.B. Freitag, 9. Januar 2026
16 // Unix-Timestamp
17 console.log(Date.now()); // z.B. 1767961752219

```

Math

Das Objekt `Math` ist ein mathematisches Hilfsobjekt. Es stellt verschiedene Konstanten wie π (`Math.PI`) oder e (`Math.E`) zur Verfügung oder bietet Methoden wie `Math.sin()` oder `Math.cos()`. `Math` ist ein statisches Objekt, es muss daher nicht mit `new` erzeugt werden.

Das Objekt `Math` dient als Container für mathematische Konstanten und Funktionen. Es ist ein statisches Objekt und muss daher – im Gegensatz zu `Date` – nicht mit `new` instanziiert werden. Die wichtigsten Eigenschaften und Methoden für den Alltag sind:

- `Math.PI`: die Kreiszahl π ,
- `Math.round(x)`: rundet eine Zahl kaufmännisch auf die nächste Ganzzahl,
- `Math.ceil(x)` / `Math.floor(x)`: rundet eine Zahl stets auf bzw. ab,
- `Math.random()`: liefert eine Zufallszahl im Intervall $[0, 1[$,
- `Math.max(...args)` / `Math.min(...args)`: ermittelt den größten bzw. kleinsten Wert aus einer Liste von Argumenten.

```

1 // Konstanten
2 console.log(Math.PI); // 3.141592653589793
3 // Runden
4 console.log(Math.round(4.6)); // 5
5 console.log(Math.floor(4.9)); // 4
6 // Zufallszahl zwischen 1 und 10 (Ganzzahl)
7 const randomInt = Math.floor(Math.random() * 10) + 1;
8 console.log(randomInt); // z.B. 2
9 // Maximum ermitteln (Spread-Operator '...' entpackt das Array)
10 const values = [10, 42, 5, 100];
11 console.log(Math.max(...values)); // 100

```

Array

Auch Arrays sind in JavaScript Objekte (vom Typ `Array`). Im Gegensatz zu Java-Arrays sind sie jedoch dynamisch (ihre Größe kann sich jederzeit ändern) und können theoretisch Datentypen mischen.

Die Erzeugung erfolgt fast ausschließlich über die Literal-Schreibweise mit eckigen Klammern. Der Konstruktor `new Array()` existiert zwar, wird aber aufgrund von eigenwilligem Verhalten (bei nur einem Zahl-Argument wird ein leeres Array mit dieser Länge erzeugt) kaum verwendet.

```

1 // Standard: Array-Literal

```

```

2 const numbers = [1, 2, 3];
3 // Ungewöhnlich / Veraltet
4 const empty = new Array(3); // Erzeugt [undefined, undefined, undefined]

```

Das Array-Objekt bietet eine Vielzahl an Methoden zur Manipulation und Iteration. Hier sind die wichtigsten, unterteilt in klassische Manipulation und moderne, funktionale Methoden. Die vollständige Liste findet sich unter [14, Abschnitt 23.1.3] oder bei MDN².

Manipulation (verändert das Original-Array):

- `push(element)`: fügt ein Element am Ende hinzu,
- `pop()`: entfernt das letzte Element und gibt es zurück,
- `splice(start, count)`: entfernt `count` Elemente ab Index `start` (und kann optional neue einfügen),
- `sort()`: sortiert das Array (standardmäßig alphabetisch, auch bei Zahlen).

Iteration und Abfrage (gibt ein neues Array zurück):

- `includes(value)`: prüft, ob `value` im Array enthalten ist (liefert `true/false`),
- `forEach(callback)`: führt eine Funktion für jedes Element aus (Ersatz für `for`-Schleifen),
- `filter(callback)`: erstellt ein *neues* Array mit allen Elementen, für die die Callback-Funktion `true` liefert,
- `map(callback)`: erstellt ein *neues* Array, indem die Callback-Funktion auf jedes Element angewendet wird (Transformation).

```

1 const grades = [1, 2, 3, 4, 5];
2
3 // 1. Element hinzufügen
4 grades.push(1);
5 console.log(grades); // [1, 2, 3, 4, 5, 1]
6 // 2. Filtern (nur positive Noten <= 4)
7 // Die Arrow-Function übergibt jedes Element im Array und gibt nur die zurück, die <= 4 sind
8 const passed = grades.filter(n => n <= 4);
9 console.log(passed); // [1, 2, 3, 4, 1]
10 // 3. Transformieren (Noten verdoppeln – Mapping)
11 const doubled = passed.map(n => n * 2);
12 console.log(doubled); // [2, 4, 6, 8, 2]
13 // 4. Prüfen ob enthalten
14 console.log(grades.includes(5)); // true

```

9.1.6 Alle Eigenschaften eines Objekts ausgeben: `for...in` Schleife

Während die in Abschnitt 8.6.2 behandelte `for...of`-Schleife ideal ist, um über die Werte von Arrays zu iterieren, funktioniert sie nicht direkt auf regulären Objekten (da diese standardmäßig nicht iterierbar sind).

Um ein Objekt zu durchlaufen, wird stattdessen die `for...in`-Schleife verwendet. Sie iteriert nicht über die Werte, sondern über die **Schlüssel** (Eigenschaften) des Objekts.

²https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array#instance_methods

Ebenfalls zu den Kontrollstrukturen aus Abschnitt 8.6 gehört die `for...in`-Schleife. Sie iteriert über ein Objekt (d.h. auch über ein Array) und gibt dabei jeden Eintrag zurück. Die prinzipielle Syntax lautet wie folgt:

```
1 for (Variable in Objekt) {  
2   Anweisung  
3 }
```

Zunächst wird eine Variable definiert, in der bei jedem Schleifendurchlauf der Name der aktuellen Eigenschaft gespeichert wird. Um an den dazugehörigen Wert zu gelangen, muss auf das Objekt mittels der Klammern-Notation (`Objekt[Variable]`) zugegriffen werden. Folgendes Beispiel durchläuft ein vereinfachtes Studierenden-Objekt (ohne Methoden) und gibt jede Eigenschaft in der Konsole aus.

```
1 const student = {  
2   name: "John Doe",  
3   age: 22,  
4   program: "MTD",  
5   languages: ["Java", "C#"]  
6 };  
7  
8 for (const property in student) {  
9   console.log("Eigenschaft:", property);  
10  console.log("Wert:", student[property]);  
11 }
```

Unterschied zu `for...of`

Es ist wichtig, die beiden Schleifentypen nicht zu verwechseln:

- `for...of` iteriert über Werte (ideal für Arrays).
- `for...in` iteriert über Schlüssel (ideal für Objekte).

Wird `for...in` auf ein Array angewandt, werden nicht die Werte, sondern der Index (0, 1 etc.) zurückgegeben. Zudem ist die Reihenfolge bei `for...in` nicht garantiert, weshalb sie für Arrays vermieden werden sollte.

9.1.7 JavaScript-Objekte und JSON

JSON basiert auf JavaScript und ist ein Subset der dortigen Objekt-Literal-Notation, die in Abschnitt 9.1.4 vorgestellt wurde. Somit ist jedes gültige JSON-Dokument auch gültiges JavaScript. Um externe JSON-Daten (die meist als String vorliegen) in ein nutzbares JavaScript-Objekt umzuwandeln, steht das globale Objekt `JSON`³ zur Verfügung. Dessen Methode `parse()`⁴ analysiert den String und erstellt das entsprechende Objekt.

Um auf die Werte eines JSON-Datensatzes in JavaScript zuzugreifen, kommt die übliche Syntax für Objekte und Arrays zum Einsatz. Eigenschaften von Objekten werden über die Punktnotation angesprochen, auf Einträge in Arrays wird mit eckigen Klammern und der Indexposition zugegriffen. Da JSON ein Subset der Objekt-Literal-Notation ist, kann das Tiroler Knödel-Rezept aus Vorlesung 7, Abschnitt 7.2.2 zu Test-

³https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON

⁴https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/parse

zwecken (etwas reduziert) als Variable – hier `rezept` – angelegt werden. Die einzelnen Werte werden dann wie bei Objekten und Arrays abgefragt und ausgegeben:

```
1 const rezept = {
2   "gericht": "Der klassische Tiroler Knödel",
3   "zutaten": [
4     {
5       "ingredienz": "Semmelwürfel",
6       "menge": 350,
7       "einheit": "g"
8     },
9     {
10      "ingredienz": "Milch",
11      "menge": 200,
12      "einheit": "ml"
13    }
14  ],
15  "zubereitung": [
16    "Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen lassen.",
17    "Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen Zutaten eine feste, aber nicht zu feuchte Masse kneten."
18  ]
19 };
20
21 // Zugriff auf einfache Eigenschaft
22 console.log(rezept.gericht); // Der klassische Tiroler Knödel
23 // Zugriff auf Array an Index 0, davon Eigenschaft 'ingredienz'
24 console.log(rezept.zutaten[0].ingredienz); // Semmelwürfel
25 // Zugriff auf Array an Index 1, davon Eigenschaft 'menge'
26 console.log(rezept.zutaten[1].menge); // 200
27 // Zugriff auf zweiten Zubereitungsschritt
28 console.log(rezept.zubereitung[1]); // Kleinwürfelig geschnittenen Speck ...
```

9.2 Client-Side JavaScript

Alle bisher besprochenen Dinge betrafen ausschließlich die Sprache JavaScript (d.h. ECMAScript). Bis auf die Teile, wo Output in der Konsole zum besseren Verständnis erzeugt wurde, war jeglicher Code völlig vom Ausgabemedium unabhängig. Sobald JavaScript aber für den Einsatz im Browser konzipiert werden soll, reicht diese Ausgabeunabhängigkeit nicht mehr aus. Es muss Mechanismen geben, wie mit JavaScript generierter Code dargestellt werden kann, oder wie Daten zur Weiterverarbeitung in JavaScript erfasst werden können. Dieser Teil wird *Client-Side JavaScript* genannt – JavaScript für den Einsatz in Webbrowsern.

9.2.1 Allgemeines

JavaScript im Webbrowser sollte dazu verwendet werden, Websites hinsichtlich ihrer Funktionalität zu bereichern. Jedoch sollte eine Seite auch ohne JavaScript immer funktionieren, wenngleich die Funktionalität eingeschränkt sein mag. JavaScript kann von

Benutzer*innen jederzeit deaktiviert werden, auch für Suchmaschinen und ähnliche automatisierte Systeme kann JavaScript-Code nicht vollständig lesbar sein. Daher sollten Webseiten Prinzipien wie *Graceful Degradation* oder *Progressive Enhancement*⁵ berücksichtigen und auch dann noch funktionieren, wenn das Script nicht verfügbar ist. Ist dies nicht möglich, so sollte zumindest das Element `<noscript>` verwendet werden, in dem Benutzer*innen darauf hingewiesen werden können, dass die Seite sehr stark auf JavaScript basiert und dieses zur Zeit nicht verfügbar ist. Der Inhalt dieses Elements wird nur dann angezeigt, wenn JavaScript nicht aktiviert ist.

Um nun JavaScript im Webbrowser verwenden zu können, stellt dieser eine Umgebung zur Verfügung, mit der spezifisch auf das Browserfenster und die darin angezeigte Website zugegriffen werden kann. Diese Zugriffsmöglichkeiten sind:

- das `window`-Objekt für den Zugriff auf das Browserfenster (siehe Abschnitt 9.2.3),
- das `document`-Objekt (mit dem Document Object Model) für den Zugriff auf die aktuelle Website (siehe Abschnitt 9.2.4 und Vorlesung 10).
- ein event-basiertes Programmiermodell, welches es erlaubt, auf Eingaben der Benutzer*innen zu reagieren (siehe Vorlesung 10).

Bevor auf diese Dinge genauer eingegangen wird, ist zunächst die prinzipielle Einbindung von JavaScript in HTML wichtig.

9.2.2 Einbindung in HTML

Um JavaScript im Zusammenhang mit HTML verwenden zu können, ist eine *Einbindung* ins HTML-Markup nötig. Wie bereits bei CSS gibt es auch hier mehrere Möglichkeiten:

- direkt im HTML-Markup (Inline),
- Einbindung als externe JavaScript-Datei,
- Verwendung mit Event-Handlern.

Inline-Einbindung

Um Scripts in HTML einbinden zu können, steht das Element `<script>` (geschlossen durch `</script>`) zur Verfügung. Optional ist die Angabe des Attributs `type`, welches im Falle von JavaScript den Wert `text/javascript` haben muss. Mit HTML5 wurde dieser Typ jedoch als Standard für das `<script>`-Element festgelegt, sodass die Angabe nicht benötigt wird.

Da Browser HTML-Dokumente zeilenweise von oben nach unten abarbeiten, ist die Position des Skripts entscheidend: Greift das Skript auf HTML-Elemente zu (wie im nachfolgenden Beispiel auf das `<div>`-Element), müssen diese Elemente zum Zeitpunkt der Ausführung bereits existieren.

Bei direkter Einbindung platziert man das `<script>`-Element daher klassischerweise am Ende des `<body>`-Bereichs, direkt vor dem schließenden `</body>`-Tag. So ist sichergestellt, dass der gesamte HTML-Inhalt geladen ist, bevor das Skript läuft.

Es können beliebig viele `<script>`-Elemente zum Einsatz kommen, sie alle zählen als ein großer JavaScript-Kontext, d. h. eine globale Variable gilt in allen Bereichen derselben HTML-Datei.

⁵https://www.w3.org/wiki/Graceful_degradation_versus_progressive_enhancement

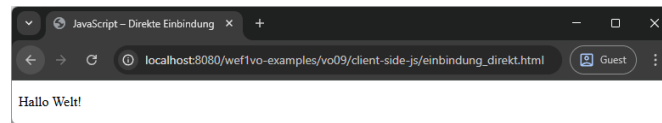


Abbildung 9.1: Der Text wird mittels JavaScript dynamisch in das `<div>`-Element eingefügt.

Das folgende Beispiel erzeugt seinen Output nur über JavaScript. Im `<body>` liegt ein leeres `<div>`-Element mit der ID `output`, der `<script>`-Tag ist danach platziert. Über das `document`-Objekt (siehe Abschnitt 9.2.4) wird mit der Methode `getElementById()` das `<div>` abgefragt. Über die Eigenschaft `innerHTML` wird ein HTML-Fragment – in diesem Fall ein Absatz mit Text – in das Element gesetzt. Abbildung 9.1 stellt das Ergebnis dar.

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript &ndash; Inline-Einbindung</title>
6   </head>
7   <body>
8     <div id="output"></div>
9     <script>
10      document.getElementById("output").innerHTML = "<p>Hallo Welt!</p>";
11    </script>
12  </body>
13 </html>

```

Bei einem Blick in den Quellcode, sind lediglich das leere `<div>` und die JavaScript-Anweisung ersichtlich, nicht etwa der von JavaScript erzeugte HTML-Code. Die Dev-Tools im Browser können allerdings auch generierten Code anzeigen, also wie die HTML-Struktur aussieht, nachdem JavaScript abgearbeitet wurde.

Externe Einbindung

Die wesentlich elegantere Methode ist die Einbindung über eine *externe* JavaScript-Datei. JavaScript-Dateien sind normale Textdateien und haben die Endung `.js`. Sie werden beim `<script>`-Element mit dem `src`-Attribut eingebunden. Wichtig ist, dass das `<script>`-Element trotzdem mit `</script>` abgeschlossen wird, auch wenn es durch die externe Einbindung keinen Inhalt mehr hat.

Bei extern eingebundenen JavaScript-Dateien gibt es mehr Möglichkeiten für die Platzung des `<script>`-Elements.

1. **Im `<head>` mit dem Attribut `defer` (empfohlen):** Das Skript wird parallel zum HTML heruntergeladen, aber erst ausgeführt, wenn das HTML vollständig geladen ist. Somit funktionieren auch Skripte, die auf die HTML-Struktur zugreifen, denn obwohl das Skript zuerst im Quellcode an die Reihe kommt, wartet der Browser aufgrund von `defer` mit dessen Abarbeitung bis die Seitenstruktur fertig geladen ist. Dies ist aktuell die beste Lösung.
2. **Im `<head>` mit `async`:** Das Skript wird ebenfalls parallel zur HTML-Verarbeitung

geladen, jedoch *sofort* nach dem fertigen Download ausgeführt (wobei der HTML-Parser kurzzeitig pausiert). Dies eignet sich daher nur für unabhängige Skripte, die nicht auf Elemente der Seite, die erst später geladen werden, zugreifen und ist im Kontext dieser Lehrveranstaltung daher nur kaum zu gebrauchen.

3. **Am Ende des <body>:** Früher war dies der Standard, um das Blockieren des Seitenaufbaus zu verhindern. Das Skript wird erst geladen, wenn der Browser am Ende der Seite ankommt. Dadurch ist zwar garantiert, dass die HTML-Struktur geladen und bekannt ist, allerdings kann das Laden der JavaScript-Datei erst danach erfolgt, während mit **defer** diese schon parallel geladen wird, weshalb Option 1 zu bevorzugen ist.
4. **Ohne Attribute im <head>:** Dies sollte vermieden werden, da das Skript das Laden der restlichen Seite blockiert, bis es vollständig heruntergeladen und ausgeführt wurde. Außerdem sind die Elemente der Seite im <body> noch nicht verfügbar, da das Skript vor deren Parsen ausgeführt wird.

Das folgende Markup zeigt das obige Beispiel mit externer Einbindung im <head> mit **defer**-Attribut.

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript &ndash; Externe Einbindung</title>
6     <script src="extern.js" defer</script>
7   </head>
8   <body>
9     <div id="output"></div>
10  </body>
11 </html>

```

Der JavaScript-Code wurde in eine eigene Datei (**extern.js**) ausgelagert. Das Ergebnis gleicht dem in Abbildung 9.1.

```

1 document.getElementById("output").innerHTML = "<p>Hallo Welt!</p>";

```

Einbindung über Event-Handler

JavaScript stellt so genannte *Event-Handler* zur Verfügung, mit denen auf Eingaben von Benutzer*innen reagiert werden kann. Code wird hier nur ausgeführt, wenn ein bestimmtes Ereignis (z. B. ein Klick) eintritt. Diese Event Handler haben Namen wie **onclick**, **onmousedown** etc. und werden im einfachsten Fall als HTML-Attribute angeführt und können in ihrem Wert JavaScript-Code aufrufen.

Folgender Code erzeugt beim Klick auf einen generischen Button (**type="button"**) eine Dialogbox mit Text, wenn darauf geklickt wird.

```

1 <button type="button" onclick="alert('Hallo Welt!')">Klick mich!</button>

```

Die Definition solcher Event-Handler ist zwar für einfache Beispiele nützlich, in komplexeren Anwendungen werden diese jedoch meist direkt in JavaScript registriert. Mehr Details dazu folgen in Vorlesung 10.

9.2.3 window-Objekt

Das **window**-Objekt wird vom Browser automatisch zur Verfügung gestellt und stellt die Grundlage dar, um JavaScript-Code im Browser ausführen zu können. Es wird auch als *globale Objekt* (engl. „global object“) bezeichnet und definiert den gesamten Ausführungskontext. Es bietet vorrangig Zugriff auf das Browserfenster und seine Eigenschaften und erlaubt es, diese sowohl auszulesen, als auch neu zu setzen. Im Folgenden werden die Eigenschaften des **window**-Objekts – auch bezeichnet als das Browser Object Model (BOM) – überblicksartig beschrieben. Eine vollständige Auflistung bietet wie immer MDN⁶.

Eigenschaften

Das **window**-Objekt selbst bietet unter anderem folgende Eigenschaften. Mit ihnen können Details zum Browserfenster abgefragt und teilweise auch gesetzt werden. Unter den Eigenschaften sind auch wiederum einige Objekte, die Detailinformationen über bestimmte Dinge (wie etwa die aktuelle Seite) beinhalten. Der Zugriff erfolgt über Punktnotation (z. B. **window.document**, der Objektname **window** kann aber auch weggelassen werden, wenn das aktuelle Fenster bearbeitet wird (also nur noch **document**)).

- **document**: das aktuell angezeigte HTML-Dokument (Objekt),
- **history**: die aktuelle Browser-History (Objekt). Die letzte aufgerufene Seite kann etwa über den Aufruf von **history.back()** erreicht werden,
- **location**: Zugriff auf den vollständigen URL der Seite (Objekt),
- **screen**: Informationen über den Bildschirm, wie etwa Höhe, Breite oder Farbtiefe (Objekt),
- **innerWidth**: die Breite des Anzeigebereichs im Browserfenster,
- **innerHeight**: die Höhe des Anzeigebereichs im Browserfenster.

Methoden

Das **window**-Objekt bietet auch Methoden an, die vornehmlich zur Erzeugung und Veränderung von Browserfenstern eingesetzt werden können:

- **open(url, name, features)**: Öffnet ein neues Browserfenster und lädt die Seite an der Adresse **url**, gibt dem Fenster den Namen **name** (um es später damit in JavaScript wieder ansprechen zu können) und stellt die Eigenschaften **features** im neuen Fenster bereit (z. B. **width**, **height**). Moderne Browser verhindern dies aber, wenn es nicht explizit durch direkten Klick ausgelöst wird.)
- **close()**: schließt ein Fenster (funktioniert in der Regel aber nur, wenn das Fenster auch per Script geöffnet wurde),
- **alert(text)**: zeigt eine Dialogbox mit dem Inhalt **text** an,
- **scrollBy(x, y)**: scrollt im Fenster um eine durch **x** und **y** spezifizierte Anzahl von Pixeln,
- **focus()**: schaltet das aktuelle Fenster aktiv (holt es in den Vordergrund).

⁶<https://developer.mozilla.org/en-US/docs/Web/API/Window>



Abbildung 9.2: Zunächst wird ein neues Fenster auf Klick geöffnet. Solange es offen ist, zeigt das Dialogfenster diesen Umstand korrekt an.

Beispiel: Fenster öffnen

Das folgende Beispiel öffnet ein Fenster mit JavaScript und stellt eine Funktion zur Verfügung, die überprüft, ob das Fenster geöffnet oder geschlossen wurde. Ebenso kann das Fenster wieder geschlossen werden. Der JavaScript-Code wird über eine externe Datei eingebunden. Es werden Buttons verwendet, deren `onclick`-Event die Funktionen auslösen. Abbildung 9.2 zeigt das Ergebnis im Browser.

```

1 <html lang="de">
2   <head>
3     <meta charset="utf-8">
4     <title>window-Objekt &ndash; Beispiel</title>
5     <script src="window-demo.js" defer></script>
6   </head>
7   <body>
8     <button type="button" onclick="openWindow()">Öffne Fenster</button>
9     <button type="button" onclick="closeWindow()">Schließe Fenster</button>
10    <button type="button" onclick="checkOpen()">Überprüfe Fenster</button>
11  </body>
12 </html>

```

Der JavaScript-Code sieht wie folgt aus:

```

1 let myWindow = null;
2
3 function openWindow() { // Öffnet ein kleines Popup-Fenster
4   myWindow = window.open("window.html", "NewWindow", "width=300, height=200");
5   if (myWindow) {
6     myWindow.focus();
7   }
8 }
9
10 function closeWindow() { // Schließt das Popup-Fenster
11   if (myWindow && !myWindow.closed) {
12     myWindow.close();
13   }
14 }
15
16 function checkOpen() { // Gibt den Status des Popup-Fensters aus
17   alert(myWindow && !myWindow.closed ? "Fenster ist offen!" : "Fenster ist
18     geschlossen!");
19 }

```

9.2.4 document-Objekt

Das `document`-Objekt ist ein Unterobjekt des `window`-Objekts und bietet umfassenden Zugriff auf das zurzeit im Browser geöffnete Dokument. Auch es besitzt eine Reihe von Eigenschaften und Methoden, mit denen das aktuelle Dokument verändert werden kann. Es folgen die wichtigsten dieser Punkte im Überblick, eine vollständige Auflistung bietet MDN⁷.

Eigenschaften

Folgende beispielhafte Eigenschaften können etwa vom aktuellen Dokument ausgelesen bzw. teilweise auch gesetzt werden:

- `title`: der Titel des Dokuments, wie mittels `<title>` spezifiziert,
- `URL`: der URL des aktuellen Dokuments,
- `referrer`: die zuvor besuchte Seite (von der auf die aktuelle verlinkt wurde),
- `lastModified`: das Datum der letzten Änderung am Dokument.

Methoden

Während früher Methoden wie `document.write()` genutzt wurden, um Text direkt in den Datenstrom der Seite zu schreiben, gilt dies heute als veraltet und sollte nicht mehr verwendet werden. Das `document`-Objekt dient heute primär als Einstiegspunkt in das DOM, um Elemente zu finden. Hier zwei Beispiele, mehr dann in Vorlesung 10.

- `getElementById(id)`: Findet ein HTML-Element anhand seiner ID.
- `querySelector(selector)`: Findet das erste Element, das auf einen CSS-Selektor passt.

Beispiel: Dokument-Eigenschaften nutzen

Das folgende Beispiel ruft auf einen Klick eine JavaScript-Funktion auf, die Informationen über das aktuelle Dokument ausgibt und den Titel der Seite dynamisch ändert. Abbildung 9.3 zeigt die Seite vor und nach dem Klick.

```
1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <meta charset="utf-8">
5     <title>Dokument-Eigenschaften auslesen</title>
6     <script src="document-demo.js" defer></script>
7   </head>
8   <body>
9     <h1>Dokument-Manipulation</h1>
10    <p>DevTools öffnen (F12) und auf den "Console"-Tab wechseln, um die Ausgabe des
      Beispiels zu sehen.</p>
11    <button type="button" onclick="showInfo()">Infos anzeigen & Titel ändern</button>
12  </body>
13 </html>
```

⁷<https://developer.mozilla.org/en-US/docs/Web/API/Document>

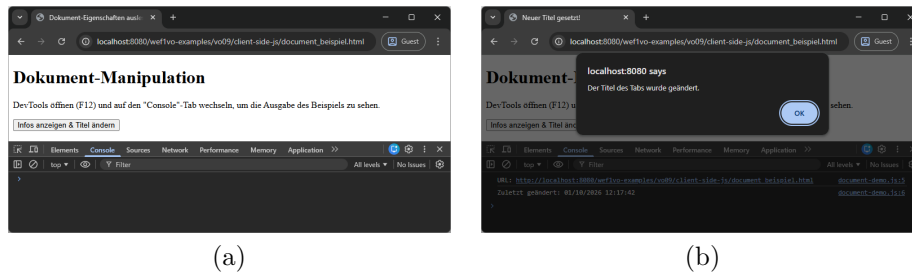


Abbildung 9.3: Das „alte“ Dokument (a) ist der Ursprungszustand. Der Titel lautet „Dokument-Eigenschaften auslesen“, die Console ist leer. Wird auf den Link geklickt, wird die Funktion `showInfo()` aufgerufen, die Informationen in die Konsole schreibt und den Titel der Seite auf „Neuer Titel gesetzt!“ ändert (b).

In der JavaScript-Datei wird `showInfo()` definiert:

```
1 function showInfo() {
2   // Zugriff auf Eigenschaften
3   console.log("URL: " + document.URL);
4   console.log("Zuletzt geändert: " + document.lastModified);
5
6   // Ändern einer Eigenschaft
7   document.title = "Neuer Titel gesetzt!";
8   alert("Der Titel des Tabs wurde geändert.");
9 }
```

Document Object Model

Die wichtigste Aufgabe des `document`-Objekts ist jedoch wie schon kurz erwähnt das Zurverfügungstellen des Document Object Models (kurz DOM). Dieses erlaubt wahlfreien Zugriff auf alle Elemente des aktuellen HTML-Dokuments. Dieses Thema wird das Hauptaugenmerk von Vorlesung 10 und wurde daher an dieser Stelle nur kurz vorab erwähnt.

9.3 Weiterführende Literatur

Die Literaturempfehlungen gleichen prinzipiell denen von Vorlesung 8. Empfohlen für die Arbeit mit Objekten und die Einbindung im Browser werden insbesondere [1, 48, 58] empfohlen. Sie gehen im Detail auf diese Konzepte ein und warten mit vielen Beispielen auf.

Vorlesung 10

JavaScript DOM – Document Object Model

10.1 Allgemeines

Client-Side JavaScript ermöglicht es, aus statischen HTML-Webseiten dynamische und interaktive Dokumente zu kreieren. Dabei auf den Inhalt von Webseiten (also auf die einzelnen Elemente) zugreifen zu können, ist die wichtigste Eigenschaft in diesem Zusammenhang. Der Zugriff auf diese Inhalte erfolgt über das bereits in Vorlesung 9 vorgestellte `document`-Objekt und ist genau spezifiziert. Die Regeln und Methoden für den Zugriff sind im so genannten *Document Object Model*, kurz DOM, festgelegt.

Das *DOM* bezeichnet also eine Schnittstelle (engl. „API – application programming interface“) für den geregelten Zugriff auf hierarchische Markup-Dokumente (wie HTML oder XML). Die WHATWG (und zuvor auch das W3C) spezifiziert eine standardisierte Version des DOM, die in allen modernen Browsern gut unterstützt ist. Die Geschichte des DOM zeigt jedoch, dass dies nicht immer der Fall war.

10.1.1 Versionen und Geschichte

Man spricht grob von vier unterschiedlichen DOM-Ausprägungen:

- Legacy DOM (DOM Level 0),
- Intermediate DOMs (IE4 DOM und Netscape 4 DOM),
- W3C DOM (DOM Level 1, 2, 3 & 4),
- WHATWG DOM Living Standard.

Netscape führte mit Version 2 ein DOM ein, das den Zugriff auf einige wenige Elemente auf der Seite ermöglichte. Darunter waren Links, Bilder und Formularelemente. Dieses DOM ist heute als *Legacy DOM* oder *DOM Level 0* bekannt, kommt allerdings in modernen Anwendungen nicht mehr zum Einsatz.

Mit Internet Explorer 4 brachte Microsoft eine eigene DOM-Variante heraus, die erstmals Zugriff auf alle Elemente der Seite ermöglichte. Sie wird heute in der Regel als IE4 DOM bezeichnet. Netscape – damals mit Microsoft mitten im Browserkrieg – entwickelte eine eigenständige, konkurrierende Variante zu der von Microsoft. Während das IE4 DOM zumindest nach wie vor der letzten Internet Explorer Versionen funktioniert,

war Netscapes Gegenstück eine Sackgasse und kam nur in Netscape 4 zum Einsatz. Da beide DOM-Varianten heutzutage aber de facto veraltet sind und nicht mehr zum Einsatz kommen, wird gesammelt von sogenannten *Intermediate DOMs* gesprochen. Auf sie wird auch nicht näher eingegangen.

Schließlich gibt es noch das bereits angesprochene, vom W3C standardisierte DOM, welches als *W3C DOM* bezeichnet wird. Da es in vier große (aufeinander aufbauende) Teile untergliedert ist, finden sich auch oft die Bezeichnungen *DOM Level 1*, *DOM Level 2*, *DOM Level 3* oder *DOM 4*. Level 1 enthält dabei die Kernaspekte des DOM-Scriptings (DOM Core), welche für diese Vorlesung relevant sind (siehe Abschnitt 10.2). Level 2, 3 und 4 erweitern das DOM um zusätzliche Funktionalitäten.

Seit 2019 wurde die kontinuierliche Weiterentwicklung des DOM – ähnlich wie bei HTML – an die WHATWG im Rahmen des *DOM Living Standards* übergeben.

10.2 W3C DOM und DOM Living Standard

Das *W3C DOM* ist eine offizielle, ursprünglich vom W3C verabschiedete Schnittstelle, die es Scriptsprachen erlaubt, dynamisch den Inhalt von Dokumenten zu verändern. Sie war in drei bestehende Ebenen gegliedert. Jede Ebene baute auf der vorhergehenden auf und erweitert den Funktionsumfang. Mittlerweile wird das DOM als WHATWG Living Standard (ähnlich wie HTML) kontinuierlich weiterentwickelt, die W3C Recommendations sind seit Ende 2020 als überholt (engl. „superseded“) eingestuft.

10.2.1 Ebenen des W3C DOM

DOM Level 1 [85] war seit dem Jahr 1998 eine Recommendation und gliederte sich in zwei große Teile:

- **DOM Core Level 1:** Stellte die generellen Funktionalitäten zur Verfügung, die benötigt werden, um Elemente in der Dokumentenstruktur zu finden oder zu erzeugen. Diese sind sprachunabhängig, d.h. funktionieren sowohl für HTML als auch für alle XML-basierten Sprachen.
- **DOM HTML Level 1:** Stellte HTML-spezifische Funktionalitäten für den Umgang mit HTML-Elementen bereit.

DOM Level 2 baute auf Level 1 auf und erweiterte sowohl die Core Funktionalitäten [86] als auch die HTML-Schnittstellen [88]. Zusätzlich wurde noch ein eigenes Event-System ergänzt [87], mit dem auf Aktionen im Browser reagiert werden kann, sowie ein eigenes Stylesheet Objektmodell [89], mit dem auf Stylesheets zugegriffen werden kann. Die meisten Spezifikationen waren seit 2000 eine Recommendation, die HTML-Erweiterung war es seit 2003.

DOM Level 3 brachte weitere Ergänzungen für den Core [90], die im Jahr 2004 zur Recommendation erhoben wurden.

10.2.2 DOM und HTML Living Standards

2015 wurde vom W3C *DOM Level 4* mit weiteren Ergänzungen als Recommendation veröffentlicht. In den darauffolgenden Jahren wurde jedoch beschlossen, die kontinuierliche Arbeit am DOM der WHATWG zu übertragen, weshalb die letztgültige Variante des

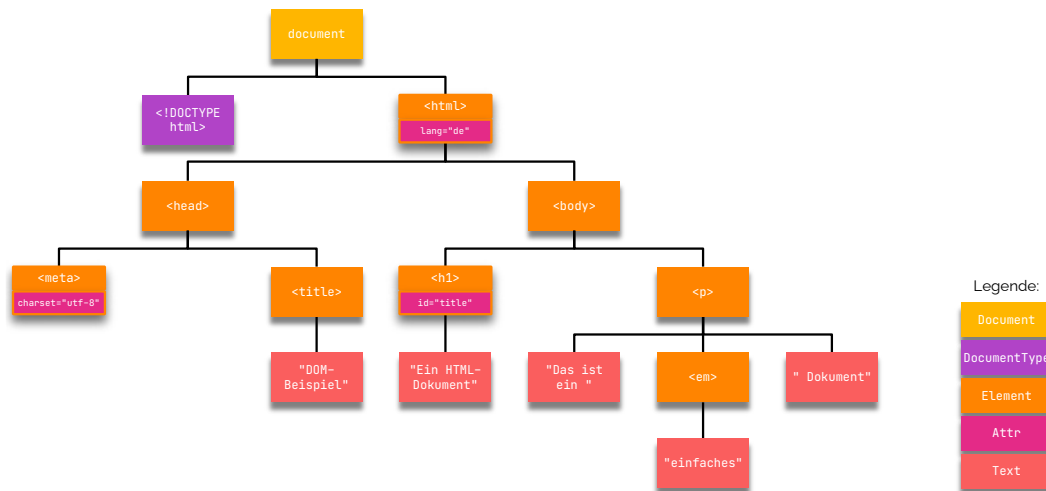


Abbildung 10.1: Am Beginn der Baumstruktur steht das `document`-Objekt. Darunter entfaltet sich der Baum mit derselben hierarchischen Struktur, die durch den HTML-Code vorgegeben ist.

DOM nun im so genannten *DOM Living Standard* [53] abgebildet wird. Dies entspricht dem DOM Core und Event-Systemen aus den W3C Standards.

Die DOM HTML-Spezifika finden sich nun im *HTML Living Standard* [54] bei der Definition der jeweiligen Elemente und Attribute.

10.3 Repräsentation der Dokumentstruktur

HTML-Dokumente weisen eine hierarchische Struktur von ineinander verschachtelten Tags auf, wobei das `<html>`-Element immer das so genannte Wurzelement (auch Root-Element genannt) darstellt. Das Document Object Model repräsentiert diese Hierarchie in einer Baumstruktur. Die einzelnen Elemente in diesem Baum (HTML-Tags, Text und Attribute) werden *Knoten* genannt. Im DOM werden diese als Objekte vom Typ `Node` repräsentiert.

Dazu zunächst ein Beispiel. Ausgehend von folgendem HTML-Markup soll der entstandene DOM-Baum betrachtet werden.

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <meta charset="utf-8">
5     <title>DOM-Beispiel</title>
6   </head>
7   <body>
8     <h1 id="title">Ein HTML-Dokument</h1>
9     <p>Das ist ein <em>einfaches</em> Dokument</p>
10  </body>
11 </html>
  
```

Die entstandene Baumstruktur ist in Abbildung 10.1 ersichtlich.

Aus dem Beispiel wird klar, dass es nicht nur den generischen `Node`, sondern mehrere

Typen von Knoten gibt. Diese sind wiederum eigene Objekte, die aber von `Node` erben, d. h. jedes spezifischere Objekt ist auch immer ein `Node`.

- **Document**: Dieser Typ repräsentiert das gesamte Dokument und stellt die Wurzel des Baumes dar.
- **DocumentType**: Repräsentiert den `<!DOCTYPE>` eines Dokuments.
- **Element**: Element-Knoten repräsentieren in diesem Fall alle HTML-Elemente (`<h1>`, `<p>`, `<em>` etc.). In einem XML-Dokument werden XML-Elemente repräsentiert.
- **Attr**: Elemente von diesem Typ beinhalten Attribute, die – wie von HTML bekannt – einen Namen und einen Wert haben.
- **Text**: Umfasst Knoten, die jegliche Form von Text beinhalten. Eine Ausnahme bilden Kommentare. Diese werden durch den eigenen Knotentyp `Comment` repräsentiert.

Bei der Vorstellung der DOM-Gliederung in den Abschnitten 10.2.1 und 10.2.2 wurde in *DOM Core* und *DOM HTML* unterschieden. Die soeben vorgestellten Typen sind dem DOM Core, d. h. dem DOM Living Standard [53] zuzuordnen, da sie völlig allgemein bezeichnet sind. Elemente, Attribute, Text – all dies gibt es in jeder hierarchischen Markupsprache. Um nun aber diese Elemente aber spezifisch als HTML-Elemente zu klassifizieren, wird eine genauere Kategorisierung benötigt. Diese liefert das DOM HTML, also der HTML Living Standard [54]. In ihm sind Knotentypen speziell für HTML definiert bzw. allgemeine Typen erweitert. So gibt es:

- **Document** (eine Erweiterung des `Document`-Typs um HTML-spezifische Inhalte).
- **HTMLElement** (die spezifischere Form von `Element`).

Für `Text` und `Attr` gibt es keine eigenen Knotentypen, da diese in HTML nicht anders funktionieren als in anderen Markupsprachen. Dafür hat aber der Knotentyp für `HTMLElement` eine Reihe noch spezifischerer Untertypen und zwar einen für jedes HTML-Element, das eigene Attribute besitzt (also mehr als nur die globalen Attribute):

- **HTMLHtmlElement** (repräsentiert `<html>`),
- **HTMLHeadElement** (repräsentiert `<head>`),
- **HTMLBodyElement** (repräsentiert `<body>`),
- **HTMLTitleElement** (repräsentiert `<title>`),
- **HTMLParagraphElement** (repräsentiert `<p>`),
- ...

Abbildung 10.2 stellt die Zusammenhänge der einzelnen Knotentypen im DOM grafisch dar.

Neben `Document`, das das gesamte Dokument repräsentiert, existiert auch noch `DocumentFragment`. Dabei handelt es sich um eine leichtgewichtige `Document`-Version, die keinen Elternknoten besitzt. Sie wird oft zur Performance-Optimierung verwendet, um Teilbäume im Speicher vorzubereiten, bevor sie in das sichtbare Dokument eingehängt werden. Mehr zum Erzeugen von Inhalten im nun folgenden Abschnitt 10.4.

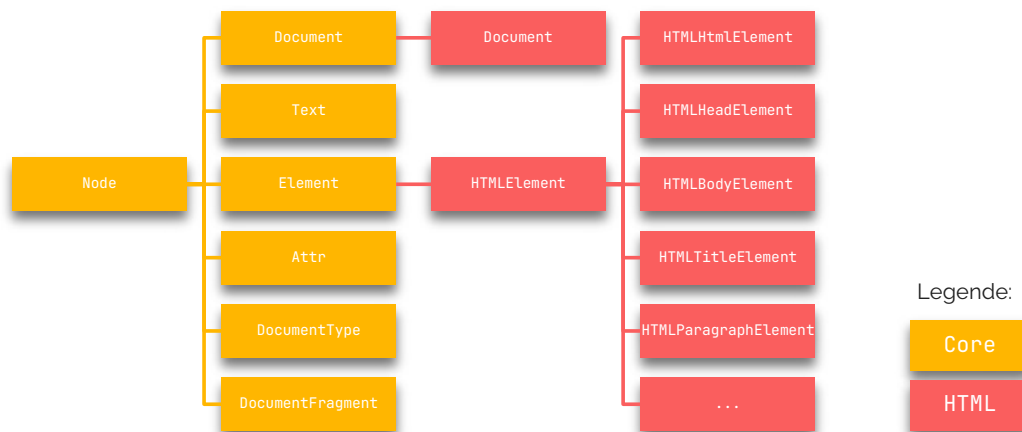


Abbildung 10.2: Das Zusammenspiel zwischen Core- und HTML-DOM. Ein `<p>`-Element ist vom Typ `HTMLParagraphElement`, dieses Element ist wiederum eine spezifischere Version von `HTMLElement`, welches eine spezifischere Ausprägung von `Element` ist. Ganz am Anfang der Hierarchie steht `Node`. Jeder Knoten in einem DOM-Baum ist also prinzipiell vom Typ `Node`, wird aber dann schlussendlich von einem Subtypen genau spezifiziert.

10.4 Manipulation der Dokumentstruktur

Nachdem nun erörtert wurde, dass ein HTML-Dokument im DOM durch einen Baum, bestehend aus Knoten, repräsentiert wird, sollen nun die Möglichkeiten aufgezeigt werden, mit denen dieser Baum manipuliert werden kann. Manipulation bezeichnet das Hinzufügen, Verändern oder Löschen von Knoten, also das Verändern des HTML-Dokuments, das ja durch diese Knoten repräsentiert wird.

Die Funktionalität zur Manipulation der Struktur steckt in den Knoten selbst. Diese sind in JavaScript einfache Objekte, d. h. sie haben Eigenschaften (von denen Informationen über den Knoten abgefragt oder auch gesetzt werden können) und Methoden (mit denen grundlegende Änderungen wie z. B. das Erzeugen eines neuen Elements durchgeführt werden können). Allgemeine Eigenschaften und Methoden, die für alle Sprachen (HTML, XML etc.) gelten, finden sich bei Objekten im Core-Teil der Hierarchie (`Node`, `Document`, `Element`, `Attr`). Informationen, die nur für HTML relevant sind, können von den Objekten aus dem HTML-DOM (`HTMLElement`, `HTMLFormElement` etc.) abgefragt werden. Nachfolgend werden diese Eigenschaften und Methoden nach Objekten gruppiert vorgestellt.

Node

Die Eigenschaften beim Typ `Node` liefern Informationen über den Knoten sowie den Kontext, in dem er sich befindet. Die dazugehörigen Methoden ermöglichen das Einfügen und Entfernen (aber nicht Erzeugen und Löschen) von neuen Knoten in den Knotenbaum. Es folgen die am häufigsten benötigten, MDN¹ bietet in seiner Auflistung noch

¹<https://developer.mozilla.org/en-US/docs/Web/API/Node>

einige weitere, weniger oft gebrauchte Eigenschaften und Methoden.

Eigenschaften: Folgende wichtige Eigenschaften sind über **Node** verfügbar. Zur Referenz sei folgendes HTML-Fragment gegeben:

```
1 <div><h1 id="title">Überschrift</h1><p>Hallo</p></div>
```

- **nodeName**: Name des Knotens bei **Element**- und **Attr**-Knoten. z. B. **div** oder **id**, **Text**-Knoten haben immer den Namen **#text**.
- **nodeValue**: Wert/Inhalt des Knotens bei **Text**- und **Attr**-Knoten. z. B. **Hallo** oder **title**. **null** für alle anderen Knoten.
- **textContent**: Der reine Textinhalt eines Knotens und dessen Kindknoten. z. B. **Hallo** für **<p>** oder **Überschrift Hallo** für **<div>**.
- **nodeType**: Der Typ des jeweiligen Knotens in Form einer Zahl: 1 für **Element**, 3 für **Text**, 8 für **Comment** oder 9 für **Document**.
- **firstChild**: Der erste Kindknoten. z. B. **<h1>** für das Element **<div>**.
- **lastChild**: Der letzte Kindknoten. z. B. **<p>** für das Element **<div>**.
- **nextSibling**: Das nächste Element auf der gleichen Ebene z. B. **<p>** für das Element **<h1>**.
- **previousSibling**: Das vorige Element auf der gleichen Ebene z. B. **<h1>** für das Element **<p>**.
- **parentNode**: Der Elternknoten. z. B. **<div>** für das Element **<h1>**.
- **parentElement**: Der Elternknoten, wenn dieser ein Element ist. **<div>** für **<h1>** und **<p>**.
- **childNodes**: Ein Array, das alle Kindknoten beinhaltet. z. B. **<h1>** und **<p>** für das Element **<div>**.

Methoden: Folgende Methoden sind **Node** zugeordnet:

- **insertBefore(newChild, refChild)**: Fügt einem Element den neuen Kindknoten **newChild** vor dem bestehenden Knoten **refChild** ein.
- **replaceChild(newChild, oldChild)**: Ersetzt aus Sicht eines Knotens den Kindknoten **oldChild** durch **newChild**.
- **appendChild(newChild)**: Hängt einem bestehenden Knoten den neuen Knoten **newChild** als letzten Kindknoten dazu.
- **removeChild(oldChild)**: Löscht einem Element seinen Kindknoten **oldChild**.
- **hasChildNodes()**: Gibt zurück, ob ein Element Kindknoten besitzt.
- **contains(node)**: Gibt zurück, ob **node** ein Unterknoten ist.
- **cloneNode(deep)**: Erstellt eine identische Kopie eines Knotens. Wird für **deep** der Wert **true** übergeben, werden auch alle Unterknoten mit kopiert, sonst nur der aktuelle.

Diese Eigenschaften und Methoden werden immer auf einem konkreten Knoten aufgerufen:

```
1 const theDiv = document.querySelector("div");  
2
```

```

3 console.log("Name des Knotens:", theDiv.nodeName); // Ausgabe: "DIV"
4 console.log("Typ des Knotens:", theDiv.nodeType); // Ausgabe: 1
5
6 const h1 = theDiv.firstChild;
7 console.log("Erstes Kind (h1):", h1); // Gibt das <h1>-Element in der Konsole aus
8
9 const p = theDiv.lastChild;
10 console.log("Letztes Kind (p):", p); // Gibt das <p>-Element aus
11
12 // Ersetzen
13 theDiv.replaceChild(p, h1);
14 console.log("Nach replaceChild (h1 ist weg):", theDiv.innerHTML);
15
16 // Löschen
17 theDiv.removeChild(p);
18 console.log("Nach removeChild (leer):", theDiv.innerHTML);
19
20 console.log("Hat noch Kinder?", theDiv.hasChildNodes()); // Ausgabe: false

```

Damit `firstChild` hier auch tatsächlich das `<h1>`-Element zurückgibt, dürfen, wie im Snippet ersichtlich, keine Zeilenumbrüche im HTML-Markup vorkommen. Diese erzeugen ansonsten einen Textknoten in dem nur der Zeilenumbruch und allfällige Leerzeichen oder Tabulatoren für die Einrückung enthalten sind. Dieser Knoten wäre aber dann in diesem Fall erste Kindknoten des `<div>`-Elements.

Document

Das Objekt `Document` erweitert `Node` und stellt bereits einige konkretere Methoden zur Verfügung. Während mittels `Node` nur Elemente in den DOM-Baum eingefügt oder gelöscht werden können, bietet `Document` nur die Möglichkeiten, neue Knoten überhaupt zu erzeugen und auch bestimmte Knoten zu finden. MDN² stellt auch hier eine vollständige Auflistung zur Verfügung.

Eigenschaften: `Document` bietet Zugriff auf einige allgemeine Eigenschaften:

- `URL` bzw. `documentURI`: Der aktuelle URL des Dokuments.
- `characterSet`: Die verwendete Zeichenkodierung, meist UTF-8.
- `contentType`: Der Content-Type der Datei, z. B. `text/html` für HTML-Dateien oder `application/xml` für XML-Dateien.
- `compatMode`: Der vom Browser verwendete Modus. Ist ein DOCTYPE vorhanden, wird `CSS1Compat` zurückgegeben, sonst `BackCompat`.

Methoden: Folgende wichtige Methoden stellt `Document` zur Verfügung:

- `getElementById(id)`: Wählt das Element aus, dessen ID-Attribut den Wert `id` besitzt.
- `getElementsByTagName(tagName)`: Gibt alle Elemente, mit dem übergebenen Tagnamen `tagName` zurück (z. B. alle `<p>`-Elemente auf der Seite). Diese Methode liefert immer eine Sammlung an Elementen (`HTMLCollection`), auch wenn es nur ein Element mit diesem Tagnamen gibt.

²<https://developer.mozilla.org/en-US/docs/Web/API/Document>

- `getElementsByClassName(classNames)`: Gibt alle Elemente mit den im Argument `classNames` angegebenen Klassen zurück. Mehrere Klassen werden durch Leerzeichen getrennt spezifiziert. Gibt auch eine (`HTMLCollection`) zurück.
- `querySelector(selector)`: Gibt das erste Element retour, das dem übergebenen CSS-Selektor entspricht.
- `querySelectorAll(selector)`: Gibt eine Liste an Elementen (`NodeList`) retour, die einem CSS-Selektor entsprechen.
- `createElement(tagName)`: Erzeugt ein neues Element (einen Element-Knoten) mit dem Namen `tagName` (z. B. ein `<div>`-Element). Bevor es im Dokument erscheint, muss es mit den Methoden von `Node` (z. B. über `appendChild()`) noch in den DOM-Baum gehängt werden.
- `createTextNode(data)`: Erzeugt einen neuen Text-Knoten mit dem Inhalt `data`.
- `createAttribute(name)`: Erzeugt ein neues Attribut `name`, aber ohne Wert, das auch nicht an ein Element gehängt ist. Die Methoden dazu werden bei `Element` vorgestellt.

Diese Methoden werden immer mit dem `document`-Objekt aufgerufen. Als Beispiel sei wieder ein HTML-Fragment gegeben:

```
1 <div id="content">
2   <p class="red underline">Red and Underlined</p>
3 </div>
```

Auf diesem werden verschiedene Operationen ausgeführt:

```
1 const content = document.getElementById("content");
2 console.log("Gefundenes Element (ID):", content);
3
4 const bodyList = document.getElementsByTagName("body");
5 const body = bodyList[0];
6 console.log("Gefundenes Body-Element:", body);
7
8 const redUnder = document.getElementsByClassName("red underline");
9 console.log("Anzahl Elemente mit Klassen:", redUnder.length);
10
11 const pInDiv = document.querySelector("div p");
12 console.log("QuerySelector Ergebnis:", pInDiv);
13
14 // Erstellen
15 const newDiv = document.createElement("div");
16 console.log("Neu erstelltes DIV:", newDiv); // Zeigt <div></div> (noch leer)
17
18 const text = document.createTextNode("Hallo Welt!");
19 console.log("Text-Knoten:", text);
20
21 const idAttr = document.createAttribute("id");
22 console.log("Attribut-Knoten:", idAttr);
```

Ein wichtiger Hinweis zur Unterscheidung: Methoden wie `getElementsByTagName()` liefern eine sogenannte *live* `HTMLCollection`, die sich automatisch aktualisiert, sobald sich der DOM-Baum ändert (z. B. wenn neue Elemente hinzugefügt werden). Im Gegensatz dazu gibt `querySelectorAll()` eine *statische* `NodeList` zurück, die wie ein Schnappschuss den Zustand exakt zum Zeitpunkt der Abfrage behält und spätere Änderungen an der Struktur nicht widerspiegelt.

Document (HTML-Spezifika)

Da die grundsätzlichen Methoden von `Document` sehr allgemein gehalten sind (also genauso für XML-Dokumente funktionieren), wird `Document` im Browser noch um solche erweitert, die spezifisch für HTML-Dokumente wichtig sind. Ebenso gibt es einige interessante Eigenschaften. Als Beispiel dient das folgende Fragment:

```
1 <form action="mailto:me@example.com">
2   <label for="wef">Web Fundamentals</label>
3   <input type="radio" name="favorite" value="WEF1" id="wef"><br>
4   <label for="mco">Modern Code</label>
5   <input type="radio" name="favorite" value="MCO1" id="mco"><br>
6   <label for="gec">Game Engine Concepts</label>
7   <input type="radio" name="favorite" value="GEC1" id="gec"><br>
8   <button type="submit">Senden</button>
9 </form>
```

Eigenschaften: Die hier zur Verfügung gestellten Eigenschaften wurden quasi bereits in Vorlesung 9 rund um das `document`-Objekt vorgestellt. Neben dem direkten Zugriff auf Struktur-Elemente (`head`, `body`) gibt es auch Zugriff auf Listen von spezifischen HTML-Elementen sowie Browser-Informationen.

- **title:** Der Titel des aktuellen Dokuments, wie er im `<title>`-Element spezifiziert wurde.
- **head:** Das `<head>`-Element der Seite.
- **body:** Das `<body>`-Element der Seite.
- **forms:** Eine automatisch aktualisierte Liste (`HTMLCollection`) aller `<form>`-Elemente im Dokument.
- **images:** Eine Liste aller `<img>`-Elemente im Dokument.
- **cookie:** Ermöglicht den Zugriff auf die Cookies, die zu diesem Dokument gehören.
- **location:** Ein Objekt, das Informationen zur aktuellen URL enthält und auch zum Navigieren verwendet werden kann.

Methoden: Bei den Methoden ist vor allem die Selektion nach dem `name`-Attribut wichtig, welches häufig in Formularen vorkommt. Die Methoden zum direkten Schreiben in den Dokumentenstrom gelten heute als veraltet („deprecated“) und sollten in modernen Anwendungen nicht mehr verwendet werden.

- **getElementByName(name):** Wählt die Element mit den entsprechenden `name`-Attributen aus. Liefert immer ein Array.
- **open(), write(), close():** Diese Methodengruppe erlaubt es, einen Dokumentenstrom zu öffnen, Text direkt hineinzuschreiben und ihn wieder zu schließen. *Achtung:* Da diese Methoden das Dokument blockieren oder komplett überschreiben können, werden sie heute fast ausschließlich zur Fehlersuche oder in sehr spezifischen Altlasten verwendet. Die Manipulation des DOM erfolgt stattdessen über das Erzeugen und Einhängen von Knoten.

Auch diese Eigenschaften und Methoden werden mit dem `document`-Objekt aufgerufen:

```
1 console.log("Seitentitel:", document.title);
2 console.log("Body-Referenz:", document.body);
3
4 const favorites = document.getElementsByName("favorite");
5 console.log("Gefundene Radio-Buttons:", favorites.length);
6
7 // HTML-Spezifische Collections (automatisch indiziert)
8 console.log("Anzahl Formulare:", document.forms.length);
9 console.log("Anzahl Bilder:", document.images.length);
10
11 // Zugriff auf das erste Formular direkt über den Index
12 console.log("Erstes Formular:", document.forms[0]);
13
14 // Cookies lesen und schreiben
15 // (Funktioniert meist nur, wenn die Seite über einen Server geladen wird)
16 document.cookie = "lecture=WEF1; SameSite=Lax";
17 console.log("Alle Cookies:", document.cookie);
18
19 // URL-Informationen
20 console.log("Aktuelle URL:", document.location.href);
21
22 // Schreiben ins Dokument (veraltet, nicht mehr verwenden, daher auskommentiert)
23 // document.open();
24 // document.write("<p>Hallo Welt!</p>");
25 // document.close();
```

Element

Element erweitert wie **Document** das Objekt **Node** und führt zusätzliche Eigenschaften und Methoden im Umgang mit Elementen ein. MDN³ enthält wiederum eine vollständige Auflistung aller Eigenschaften und Methoden.

Eigenschaften: Diese Eigenschaften sind bei jedem **Element**-Knoten abfragbar:

- **tagName**: Der Name des Elements (z. B. **p**).
- **className**: Der Wert des **class**-Attributs eines Elements.
- **classList**: Gibt eine **DOMTokenList** zurück, die eine komfortable Schnittstelle zur Manipulation der CSS-Klassen bietet. Im Gegensatz zur manuellen Bearbeitung des Strings in **className** bringt sie Methoden wie **add()**, **remove()**, **toggle()** (Umschalten) und **contains()** (Prüfen) mit.
- **id**: Der Wert des **id**-Attributs
- **innerHTML**: Liefert den Inhalt eines HTML-Elements, inklusive aller Nachfahr-Elemente. Bei **textContent** (Eigenschaft von **Node**) werden die Elemente entfernt und nur der Text zurückgeliefert.
- **firstElementChild**: Liefert den ersten Kindknoten, der ein Element ist.
- **lastElementChild**: Liefert den letzten Kindknoten, der ein Element ist.
- **nextElementSibling**: Selektiert den nächsten Geschwisterknoten, der ein Element ist.
- **previousElementSibling**: Selektiert das vorangehende Geschwisterelement.

³<https://developer.mozilla.org/en-US/docs/Web/API/Element>

Methoden: Die Methoden, die `Element` zur Verfügung stellt, dienen meist zum Umgang mit Attributen, jedoch können auch Unterelemente damit gesucht werden.

- `getAttribute(name)`: Gibt den Wert des Attributs `name` zurück.
- `setAttribute(name, value)`: Setzt (erzeugt) ein Attribut mit dem Namen `name` und Wert `value`.
- `removeAttribute(name)`: Löscht das Attribut `name`.
- `hasAttribute(name)`: Überprüft, ob ein Element das Attribut `name` besitzt.
- `setAttributeNode(name)`: Hängt an ein Element einen Attributknoten `name`, der zuvor z. B. mit `createAttribute(name)` erzeugt wurde.
- `getAttributeNode(name)`: Fragt den Attributknoten `name` vom Element ab.
- `remove()`: Entfernt das Element aus dem Baum, in dem es hängt.

Darüber hinaus sind hier nochmals alle Selektionsmethoden aus der `Document`-Schnittstelle, also etwa `querySelector()` oder `getElementsByClassName()` vorhanden. Sie funktionieren gleich, allerdings selektieren Sie nur Elemente, die Kinder desjenigen sind, auf dem die Methode ausgeführt wird. Sie suchen also nicht im gesamten Dokument.

Die Eigenschaften und Methoden werden direkt auf Elemente (Knoten vom Typ `Element`) angewandt, dieses wird hier zunächst erzeugt:

```
1 const heading = document.createElement("h1");
2 heading.textContent = "Überschrift";
3
4 heading.setAttribute("id", "important");
5 console.log("ID gesetzt:", heading.id); // Ausgabe: "important"
6
7 heading.id = "normal";
8 console.log("ID geändert:", heading.getAttribute("id")); // Ausgabe: "normal"
9
10 heading.className = "blue";
11 console.log("Klasse gesetzt:", heading.className);
12
13 heading.classList.add("underline");
14 console.log("Klassenliste:", heading.classList); // DOMTokenList ["blue", "underline"]
15
16 document.body.appendChild(heading);
17 console.log("Element in den Body eingehängt:", heading);
```

HTMLElement

`HTMLElement` stellt spezifische Eigenschaften vor, die nur für HTML gelten. Sie decken sich mit den Universalattributen, die es bei jedem HTML-Element gibt. Dieser Zugriff auf `Attribute` ist in der Regel einfacher als mit `getAttribute()` und `setAttribute()` und wird daher bevorzugt.

Eigenschaften: Die folgenden Universalattribute sind vorhanden:

- `title`: Das `title`-Attribut.
- `lang`: Das `lang`-Attribut.
- `dir`: Das `dir`-Attribut (Textleserichtung).

- **style**: Das **style**-Attribut.
- **accessKey**: Der zugeordnete Access-Key.
- **tabIndex**: Der zugeordnete Tab-Index.

Diese Eigenschaften werden immer auf Elemente angewandt, wiederum wird zu-nächst eines erzeugt.

```
1 const divElem = document.createElement("div");
2 divElem.textContent = "Hallo Welt!"; // Textinhalt setzen
3 divElem.lang = "de"; // Sprache setzen
4 divElem.style.marginTop = "2em"; // Stil-Eigenschaft setzen
5
6 console.log("Konfiguriertes Element:", divElem);
7 // In der Konsole kann man das Element aufklappen und die Styles prüfen
8
9 document.body.appendChild(divElem);
```

Subtypen von HTMLElement

Die restlichen HTML-Attribute sind – je nachdem welchem Element sie angehören – bei den jeweiligen Objekten angeführt. Einige Beispiele:

- **HTMLLinkElement**: Eigenschaften: **href**, **target**, **rel** etc.
- **HTMLFormElement**: Eigenschaften: **action**, **method**, **name** etc.
- **HTMLInputElement**: Eigenschaften: **name**, **value**, **size** etc. Methoden: **blur()**, **focus()** etc.
- **HTMLImageElement**: Eigenschaften: **src**, **alt**, **width**, **height** etc.

Diese Eigenschaften werden immer auf Elemente angewandt. Hier wird ein **<input>**-Element erstellt und einige Eigenschaften verändert:

```
1 // 1. Container-Formular erstellen
2 const form = document.createElement("form");
3
4 // 2. Das Input-Element erstellen
5 const input = document.createElement("input");
6
7 // Typ ändern
8 input.type = "password";
9
10 // Autocomplete setzen (z.B. für Passwort-Manager)
11 input.autocomplete = "new-password";
12
13 input.placeholder = "Geheimcode eingeben";
14 input.value = "123456";
15
16 // Deaktivieren
17 input.disabled = true;
18
19 console.log("Aktueller Wert:", input.value);
20 console.log("Ist deaktiviert:", input.disabled);
21
22 // 3. Zusammenbauen
23 form.appendChild(input);
24 document.body.appendChild(form);
```

Attr

Die Attributknoten vom Typ `Attr` sind sehr einfach aufgebaut und enthalten lediglich zwei Eigenschaften, die ein Attribut beschreiben:

Eigenschaften: Ein Attribut kann zwei Eigenschaften haben:

- **name**: Der Name des Attributs.
- **value**: Der Wert des Attributs.

Diese Eigenschaften werden immer auf Attributknoten angewandt:

```
1 const divElem = document.createElement("div");
2 divElem.textContent = "An ordinary div";
3
4 const titleAttr = document.createAttribute("title");
5 titleAttr.value = "Some Container";
6 console.log("Attribut-Wert:", titleAttr.value);
7
8 divElem.setAttributeNode(titleAttr);
9 console.log("HTML mit Attribut:", divElem.outerHTML);
10 // Ausgabe z.B.: <div title="Some Container">An ordinary div</div>
11
12 document.body.appendChild(divElem);
```

Die explizite Erstellung von `Attr`-Knoten über `createAttribute()` und das Einhängen via `setAttributeNode()` verdeutlicht zwar die interne Knoten-Struktur des DOM, ist im Alltag jedoch unnötig umständlich. In der modernen Webentwicklung wird fast ausschließlich die wesentlich einfachere Methode `setAttribute("name", "value")` oder der direkte Zugriff über Eigenschaften (z. B. `element.id = "..."`) verwendet.

10.5 Events

Bisher wurde JavaScript-Code in den Beispielen vorwiegend linear betrachtet: Der Browser liest das Dokument, stößt auf ein `<script>`-Element und führt den Inhalt sofort aus. Dies geschieht synchron während des Ladevorgangs (Parsing).

Interaktive Anwendungen funktionieren jedoch anders. Aktionen sollen oft nicht sofort beim Laden, sondern erst zu einem späteren, unbestimmten Zeitpunkt stattfinden – etwa wenn Benutzer*innen einen Button anklicken, eine Eingabe tätigen oder die Fenstergröße ändern. Zudem erfordert die DOM-Manipulation zwingend, dass die entsprechenden Elemente bereits vollständig geladen und im Baum verfügbar sind, bevor das Skript darauf zugreift.

Für Entwickler*innen ist es daher essentiell zu wissen, *wann* solche Situationen eintreten. Zu diesem Zweck generiert der Browser immer dann, wenn eine relevante Aktion stattfindet, sogenannte *Events* (deutsch: „Ereignisse“). Dies ermöglicht es, den Code von der linearen Ausführung zu entkoppeln und gezielt auf Interaktionen zu reagieren.

10.5.1 Arten von Events

Im Browser werden die unterschiedlichsten Arten von Ereignissen generiert. Viele davon entstehen aufgrund von Eingaben der Benutzer*innen (z. B. Mausklicks oder Tastatureingaben), andere Events werden implizit ausgelöst, etwa wenn die Seite vollständig

geladen ist oder ein Formular abgeschickt wurde. Ähnlich wie beim DOM gibt es auch hier verschiedene Schnittstellen, die oberste und generellste ist dabei **Event** aus dem DOM Living Standard [53]. Je nach Typ gibt es dann spezifischere Subtypen, die etwa aus der W3C UI Events Spezifikation [104] stammen. Um die Maus als Eingabegerät zu generalisieren, wurden Pointer Events [99] geschaffen, die neben der Maus auch Stifte und Touch repräsentieren und daher zu bevorzugen sind. Wenn ein Event verarbeitet wird (siehe dazu Abschnitt 10.5.2) steht das Event-Objekt selbst zur Verfügung. Es enthält die genauen Informationen über das jeweilige Ereignis (z. B. wo genau geklickt wurde) und hat einen spezifischen Typ – hat also je nach Ereignisart andere Eigenschaften und Methoden.

Tabelle 10.1 gibt eine Übersicht über die in der DOM-Spezifikation definierten Events und ihre Typen. Es sei angemerkt, dass viele andere Spezifikationen, die im Umfeld von HTML und JavaScript angesiedelt sind, weitere Events definieren. Auf diese wird hier jedoch nicht eingegangen.

10.5.2 Verarbeiten von Events

Die generierten Ereignisse alleine geben jedoch nur Aufschluss darüber, *dass* etwas Bestimmtes aufgetreten ist. Ohne einen Mechanismus, darauf zu reagieren, ist diese Information aber nur bedingt nützlich. JavaScript stellt jedoch Mechanismen zur Verfügung, um genau dies zu tun. Der generelle Ablauf, auf einen Event zu reagieren sieht dabei wie folgt aus:

1. **Element auswählen, bei dem das Ereignis stattfindet:** Im HTML oder mittels einer DOM-Methode wird das Element gesucht bzw. ausgewählt, auf dem eine Benutzer*inneninteraktion stattfindet bzw. wo vom Browser implizit Ereignisse generiert werden, z. B. ein Button, der geklickt werden soll.
2. **Event spezifizieren, der abgefangen werden soll:** Unter Angabe eines der Events, die in Tabelle 10.1 aufgelistet wurden, wird angegeben, worauf reagiert werden soll, z. B. auf einen Mausklick, der auf den zuvor spezifizierten Button ausgeführt wird.
3. **Verarbeitung in einer Funktion angeben:** Bei der Verarbeitung eines Ereignisses wird immer eine Funktion aufgerufen, in dieser wird nun spezifiziert, was genau passieren soll, z. B. eine Alert-Box mit einer Nachricht ausgeben, wenn auf den Button mit der Maus geklickt wurde.

Diese drei Schritte sind immer dieselben, wenn auf Events reagiert werden soll. Es gibt jedoch unterschiedliche Ansätze, wie Events abgefangen werden können:

- als HTML-Event-Handler,
- als DOM-Event-Handler, oder
- als DOM-Event-Listener.

Verarbeitung mittels HTML-Event-Handler

HTML-Event-Handler sind nichts anderes als HTML-Attribute, den den Namen des Events mit vorangestelltem **on** tragen. Für einen **click**-Event heißt das Attribut also **onclick**, für den **blur**-Event **onblur** usw.

Tabelle 10.1: Im DOM spezifizierte Events und ihre Typen.

Event	Beschreibung	Objektyp
abort	Das Laden einer Ressource (z. B. Bild) wurde abgebrochen.	UIEvent
blur	Ein Element hat den Fokus verloren.	FocusEvent
change	Ein Formular-Element hat einen geänderten Wert erhalten.	Event
click	Maus- bzw. Pointerклик.	PointerEvent
dblclick	Maus-Doppelклик.	MouseEvent
DOMContentLoaded	Das aktuelle Dokument (d. h. sein DOM-Baum) wurde fertig geladen, jedoch nicht externe Ressourcen.	Event
error	Fehler beim Laden einer Ressource.	UIEvent
focus	Ein Element hat den Fokus erhalten.	FocusEvent
focusin	Ein Element hat den Fokus erhalten.	FocusEvent
focusout	Ein Element hat den Fokus verloren.	FocusEvent
keydown	Taste wird gedrückt gehalten.	KeyboardEvent
keyup	Taste wurde losgelassen.	KeyboardEvent
load	Dokument ist fertig geladen.	UIEvent
pointerdown	Ein Pointer (Maus, Pen, Touch etc.) wurde gedrückt.	PointerEvent
pointerenter	Der Pointer wird über ein Element bewegt.	PointerEvent
pointerleave	Der Pointer wird von einem Element wegbewegt.	PointerEvent
pointermove	Der Pointer wurde bewegt.	PointerEvent
pointerout	Der Pointer wird von einem Element oder einem seiner Kinder wegbewegt.	PointerEvent
pointerover	Der Pointer wird über ein Element oder eines seiner Kinder bewegt.	PointerEvent
pointerup	Der Pointer wurde losgelassen.	PointerEvent
reset	Ein Formular wurde zurückgesetzt.	Event
resize	Die Browserfenstergröße wurde geändert.	UIEvent
scroll	Es wurde nach unten oder oben gescrollt.	UIEvent
select	Text wurde selektiert.	UIEvent
submit	Ein Formularfeld wurde abgeschickt.	Event
unload	Das aktuelle Dokument wurde geladen, d. h. eine neue Seite aufgerufen.	Event

Als Wert des Attributs folgt der JavaScript-Code, der aufgerufen werden soll, in Form einer Funktion. Angenommen, im Code ist eine Funktion `alertMe()` definiert, so kann diese bei einem Klick auf einen Button folgendermaßen aufgerufen werden:

```
1 <button id="button1" type="button" onclick="alertMe()">Button 1</button>
```

Diese Art und Weise, Event-Handler zuzuweisen, erscheint zwar effizient, vermischt aber Struktur und Logik einer Website, d. h. HTML und JavaScript. Daher gilt sie mittlerweile als veraltet und sollte nicht mehr angewandt werden.

Verarbeitung mittels DOM-Event-Handler

Um diese beiden Dinge sauber zu trennen, kann der Event-Handler auch rein im JavaScript-Code zugewiesen werden. Dazu wird zunächst das HTML-Element mit der entsprechenden DOM-Methode gesucht und diesem dann der Event-Handler als Eigenschaft zugewiesen. Auch hier lauten die Eigenschaften wie der Event-Name mit vorangestelltem `on`, also `onclick`, um den `click`-Event abzufangen.

```
1 <button id="button2" type="button">Button 2</button>
```

Die Zuweisung über JavaScript geschieht wie folgt:

```
1 const btn2 = document.getElementById("button2");  
2 btn2.onclick = alertMe;
```

Zunächst wird der Button mit der Methode `getElementById()` gesucht und in einer Variable gespeichert. Dann kann mit der einfachen Punkt-Syntax eine neue Eigenschaft, in diesem Fall `onclick` gesetzt werden. Der Wert ist die Funktion, die aufgerufen werden soll.

Vorsicht: Die Zuweisung muss *ohne* abschließende Klammern erfolgen. Dies garantiert, dass die gesamte Funktion dem `onclick`-Handler zugewiesen und diese erst beim eigentlichen Klick aufgerufen wird. Würde stattdessen `btn2.onclick = alertMe();` geschrieben, so würde `alertMe()` sofort ausgeführt werden und der Rückgabewert der Funktion (in diesem Fall nur `undefined`) würde der `onclick`-Eigenschaft zugewiesen.

Wichtig ist ebenfalls, dass zum Zeitpunkt der Zuweisung das HTML-Element bereits existieren muss. Dies bedeutet, die Zuweisung muss in einem `<script>`-Block ganz am Ende des `<body>`-Elements oder im `<head>` mit dem `defer`-Attribut geschehen.

Diese Art der Event-Handler-Zuweisung hat jedoch einen Nachteil: Es lässt sich nur eine Funktion pro Event zuweisen. Um diese Einschränkung zu umgehen, kann eine *anonyme Funktion* verwendet werden. Eine solche Funktion hat keinen Namen und existiert nur zum Zeitpunkt ihres Aufrufs. Mit ihr können aber mehrere weitere Funktionsaufrufe gekapselt werden, sodass etwa `alertMe()` zwei Mal hintereinander aufgerufen werden könnte:

```
1 btn2.onclick = function () {  
2   alertMe();  
3   alertMe();  
4 };
```

Verarbeitung mittels DOM-Event-Listener

Die flexibelste Art, Events zu verarbeiten sind die im DOM Living Standard [53] definierten Methoden zum Hinzufügen und Entfernen von Event-Listnern:

- `addEventListener(event, function)`,
- `removeEventListener(event, function)`.

Um damit beim Klick auf einen Button die Funktion `alertMe()` aufzurufen, werden zunächst wieder ein Button spezifiziert

```
1 <button id="button3" type="button">Button 3</button>
```

und dann in der Methode der Eventname (ohne „on“) und der Name der aufzurufenden Funktion (wieder ohne Klammern) angegeben:

```
1 const btn3 = document.getElementById("button3");
2 btn3.addEventListener("click", alertMe);
```

Event-Listener haben den Vorteil, dass beliebig viele verarbeitete Funktionen für einen Event zugewiesen werden können, dazu wird `addEventListener()` einfach mehrmals mit demselben Event aufgerufen. Auch das Entfernen von Event-Listnern ist ein großer Vorteil. In diesem Skriptum kommen daher nur Event-Listener zum Einsatz.

10.5.3 Standardverhalten von Events unterbinden

Bei vielen Interaktionen im Browser werden automatisch Events ausgelöst. Wird etwa ein Link angeklickt, erfolgt ein `click`-Event, der den Browser dazu veranlasst, dem im `href`-Attribut spezifizierten URL zu folgen. Wird nun der Link jedoch verwendet, um mittels JavaScript selbst etwas auszuführen, so ist oft dieses Standardverhalten unerwünscht und muss unterdrückt werden.

Folgendes Beispiel illustriert diese Situation, es sei diese Funktion definiert:

```
1 function showURL() {
2   const link = document.querySelector("a");
3   window.alert(link.href);
4 }
```

Sie nimmt das erste `<a>`-Element der Seite und gibt den Wert dessen `href`-Attributs in einem Dialogfenster aus. Der Inhalt des `<body>`-Elements sieht wie folgt aus:

```
1 <a href="https://www.fh-ooe.at/">Ein Link</a>
2 <script>
3   const link = document.querySelector("a");
4   link.addEventListener("click", showURL);
5 </script>
```

Zunächst wird ein Link erstellt, der auf eine Seite verweist, danach wird ein Event-Listener auf diesen Link zugewiesen, der die Funktion `showURL()` beim Klick auf den Link aufruft.

Wird nun diese HTML-Datei im Browser geöffnet und auf den Link geklickt, so erscheint zunächst eine Dialogbox, die den Inhalt des `href`-Attributs (also den URL) ausgibt. Wird diese Box geschlossen, wird die Standardaktion des Browsers ausgeführt und zum angegebenen URL gewechselt. Da es nur um die Ausgabe des Attributwerts geht, ist diese nicht erwünscht, es muss also die Standardaktion, d. h. das Folgen des Links, unterbunden werden.

Um dies zu erreichen, wird der Funktion das Event-Objekt (das von Browser standardmäßig erzeugt wird) als Parameter eingetragen. Dieses Objekt besitzt die Methode `preventDefault()`, die innerhalb der Funktion am Event-Objekt aufgerufen werden muss und das Standardverhalten unterbindet. `showURL()` muss also wie folgt aussehen:

```
1 function showURL(event) {
2   const link = document.querySelector("a");
3   window.alert(link.href);
4   event.preventDefault();
5 }
```

Nun zeigt der Browser das Dialogfenster an, wechselt aber danach nicht auf den URL sondern bleibt auf der aktuellen Seite.

10.5.4 Zugriff auf das Listener-Element

Bei genauerer Betrachtung des letzten Beispiels fällt auf, dass das gewünschte Element (der Link) eigentlich zweimal gesucht wird: einmal beim Zuweisen des Event-Listeners und ein zweites Mal innerhalb der Funktion `showURL()`. Auch wenn dies per se nicht falsch ist, so ist es nicht sonderlich elegant und performant (zwei Abfragen im DOM-Baum) und macht die Funktion `showURL()` auch nicht wiederverwendbar. Sie ist fest an einen spezifischen Selektor gebunden. Werden weitere Links eingefügt, müsste die Funktion für jeden Link neu angepasst oder kopiert werden.

Um die Funktion universell zu gestalten, wird die Möglichkeit benötigt zu sagen „Gib den Inhalt des `href`-Attributs von *demjenigen* Element aus, dessen Event-Listener gerade die Funktion aufgerufen hat“. Die Lösung hierfür ist der Zugriff über das `event`-Objekt, genauer gesagt über die Eigenschaft `event.currentTarget`. Diese Eigenschaft verweist immer zuverlässig auf das Element, an dem der Event-Listener registriert wurde – unabhängig davon, ob klassische Funktionen oder Arrow-Functions verwendet werden.

Zunächst wird die Funktion entsprechend geändert. Sie selektiert nun kein Element mehr selbst, sondern greift direkt auf das auslösende Element zu:

```
1 function showURL(event) {  
2   // Zugriff auf das Element, das den Listener ausgelöst hat  
3   window.alert(event.currentTarget.href);  
4   event.preventDefault();  
5 }
```

Im HTML-Markup werden nun zwei Hyperlinks platziert. Um den Event-Listener an alle Links zu binden, wird eine `for...of`-Schleife verwendet. Damit können jederzeit neue Links hinzugefügt werden und von der Funktion berücksichtigt werden.

```
1 <a href="https://www.fh-ooe.at/">Ein Link</a>  
2 <a href="https://orf.at/">Noch ein Link</a>  
3 <script>  
4   const links = document.querySelectorAll("a");  
5  
6   // Jeden Link durchlaufen und Listener anhängen  
7   for (const link of links) {  
8     link.addEventListener("click", showURL);  
9   }  
10 </script>
```

Die Funktion `showURL()` ist nun universell einsetzbar: sie zeigt automatisch den URL jenes Links an, der gerade angeklickt wurde, ohne dass die Funktion selbst wissen muss, um welchen Link es sich handelt.

10.5.5 Event-Listener Beispiele

Zur Demonstration von einigen Event-Listnern folgen nun drei Beispiele, von denen vor allem das erste eine wichtige Funktionalität zeigt.

Auf vollständiges Laden der Seite warten

Ein häufiges Problem bei der DOM-Manipulation ist das Timing. Da der Browser HTML-Dateien linear von oben nach unten abarbeitet, existieren Elemente im `<body>`

noch nicht, wenn ein `<script>`-Block im `<head>` bereits ausgeführt wird. Ein Zugriff via `document.querySelector()` würde ins Leere laufen.

Früher (und in vielen älteren Tutorials) wurde dieses Problem gelöst, indem entweder das `<script>` ganz ans Ende des `<body>` platziert wurde oder in JavaScript explizit auf Events gewartet wurden. Heute steht, wie in Vorlesung 9 gezeigt, mit dem `defer`-Attribut eine elegantere Lösung zur Verfügung, die bevorzugt werden sollte. Dennoch sollen die Events zum Laden der Seite hier nicht unerwähnt bleiben.

DOMContentLoaded: Technisch gesehen wartet das `defer`-Attribut auf den Zeitpunkt, an dem der Browser das Event `DOMContentLoaded` feuern würde. Dieses Event markiert den Moment, in dem das reine HTML vollständig verarbeitet und in den DOM-Baum umgewandelt wurde.

Möchte man JavaScript-Code direkt innerhalb der HTML-Datei, also im `<script>`-Element, ohne externe Datei im `<head>` schreiben, funktioniert `defer` nicht. In diesem Fall muss manuell auf das Event `DOMContentLoaded` gewartet werden, um zu wissen, wann der DOM-Baum erstellt wurde:

```
1 document.addEventListener("DOMContentLoaded", function () {  
2   // Dieser Code läuft erst, wenn der DOM bereit ist  
3   console.log("HTML ist fertig geparsed!");  
4 });
```

In der modernen Praxis wird jedoch fast immer mit externen Dateien und `defer` gearbeitet, weshalb der explizite Einsatz von `DOMContentLoaded` selten geworden ist.

load: Wichtig ist die Abgrenzung zum `load`-Event beim `window`-Objekt. Während `defer` und `DOMContentLoaded` feuern, sobald die *Struktur* (HTML) steht, wartet `load`, bis wirklich alles geladen ist – also auch große Bilder, Stylesheets und externe Skripte. Dies wird nur benötigt, wenn das Skript von visuellen Eigenschaften abhängt, die erst nach dem Laden von Assets bekannt sind (z. B. „Wie hoch ist dieses Bild in Pixeln wirklich?“).

Beispiel: Das folgende Beispiel demonstriert zeigt Events. Die JavaScript-Datei wird im `<head>` der HTML-Datei eingebunden. Auf das Attribut `defer` wird bewusst verzichtet. In der Datei befindet sich ein `<div>`-Element, in das im Anschluss Status-Meldungen geschrieben werden sowie ein Bild und ein Button.

```
1 <!DOCTYPE html>  
2 <html lang="de">  
3   <head>  
4     <meta charset="utf-8">  
5     <title>Unterschied: DOMContentLoaded vs. load</title>  
6     <script src="domcontentloaded-load.js"></script>  
7   </head>  
8   <body>  
9     <div id="status">Warte auf Events...</div>  
10      
11    <button id="actionButton">Klick mich</button>  
12  </body>  
13 </html>
```

In der JavaScript-Datei werden zwei Funktionen definiert. `initInteractions()`, die Event-Listener hinzufügt, um beim Button und beim Bild auf Klicks zu reagieren. Ebenso wird die Meldung in der Statusbox aktualisiert:

```
1 function initInteractions() {
2   const statusBox = document.querySelector("#status");
3   const btn = document.querySelector("#actionButton");
4   const img = document.querySelector("#demoImage");
5
6   statusBox.textContent = "DOMContentLoaded gefeuert: Struktur ist bereit.";
7   statusBox.style.backgroundColor = "#e0f7fa"; // Hellblau
8
9   // Listener anhängen möglich, da das HTML-Element existiert
10  btn.addEventListener("click", function () {
11    window.alert("Button funktioniert, obwohl das Bild evtl. noch lädt!");
12  });
13
14  img.addEventListener("click", event => {
15    console.log("Bild wurde angeklickt:", event.currentTarget.src);
16  });
17 }
```

Die zweite Funktion heißt `finalizePage()` und zeigt in der Statusbox die Größe des Bildes an. Dies funktioniert erst, wenn das Bild auch tatsächlich geladen ist:

```
1 function finalizePage() {
2   const statusBox = document.querySelector("#status");
3   const img = document.querySelector("#demoImage");
4
5   // Erst jetzt kennen wir die echte Größe des Bildes
6   const width = img.naturalWidth;
7   const height = img.naturalHeight;
8
9   statusBox.textContent += ` | Load gefeuert: Bild geladen (${width}x${height}px).`;
10  statusBox.style.backgroundColor = "#c8e6c9"; // Hellgrün
11  statusBox.style.border = "2px solid green";
12 }
```

Damit nun beide Funktionen zur richtigen Zeit ausgeführt werden, werden die entsprechenden Event-Listener registriert:

- `DOMContentLoaded` beim `document`-Objekt und
- `load` beim `window`-Objekt.

```
1 document.addEventListener("DOMContentLoaded", initInteractions);
2 window.addEventListener("load", finalizePage);
```

Anonyme Funktionen: Oft soll bei einem Event-Listener (insbesondere beim Warten auf das Laden der Seite) mehr als nur eine Funktion angegeben werden, die aufgerufen werden soll, wenn der Event stattfindet. Dafür könnte eine eigene Funktion erstellt werden, in der wiederum mehrere andere Funktionen aufgerufen werden. Alternativ kann aber auch eine anonyme Funktion erstellt und dem Listener zugewiesen werden. Anonyme Funktionen existieren nur an Ort und Stelle ihrer Definition und haben auch keinen Namen (daher anonym). Sie sind einfach wie folgt aufgebaut:

```

1 function () {
2   // Inhalt der anonymen Funktion
3 }

```

Soll nun vor der `initInteractions()`- und der `finalizePage()`-Funktion jeweils noch eine Dialogbox geöffnet werden (um z. B. zu erfahren, dass der jeweilige Event vom Browser ausgelöst wurde), so kann dies in einer anonymen Funktion (klassisch oder als Arrow-Funktion) verpackt werden:

```

1 document.addEventListener("DOMContentLoaded", function () {
2   window.alert("DOMContentLoaded-Event ausgelöst!");
3   initInteractions();
4 });
5
6 window.addEventListener("load", () => {
7   window.alert("load-Event ausgelöst!");
8   finalizePage();
9 });

```

Absenden eines Formulars überprüfen

Ein Event-Listener für das `submit`-Event kann beim `<form>`-Element verwendet werden. Er wird beim Absenden des Formulars aufgerufen. `preventDefault()` ist auch hier interessant, denn es verhindert das Absenden des Formulars. Somit eignet sich der `submit`-Event bestens für eine Formularfeldüberprüfung mit JavaScript. Es ruft eine Funktion auf, welche die Eingaben aller Formularfelder überprüft. Sind alle Kriterien erfüllt (z. B. alle Pflichtfelder ausgefüllt und E-Mail-Adressen korrekt eingegeben), so wird das Formular wie erwartet verschickt. Sind noch Fehler in den Eingaben vorhanden, wird mittels `preventDefault()` das Absenden verhindert und es können etwa neben die jeweiligen Formularfelder Hinweise platziert werden, wo sich Fehler befinden.

Das Formularfeld kann wie folgt aussehen:

```

1 <form action="mailto:john.doe@johndoe.at" method="post"
2   enctype="text/plain" id="dataform">
3   <label for="name">Name:</label>
4   <input type="text" id="name">
5   <button type="submit">Absenden</button>
6 </form>

```

In JavaScript wird dann dem Formular der Listener zugewiesen:

```

1 // Merkt sich, ob Fehler aufgetreten sind
2 let hasErrors = false;
3
4 function isValid(event) {
5   // Validierung der Formularfelder und setzen von hasErrors auf true, wenn ein Fehler
6     passiert ist
7   document.querySelector("input").value === "" ? hasErrors = true : hasErrors =
8     false;
9
10  if (hasErrors) {
11    window.alert("Absenden verhindert");
12    event.preventDefault();
13  }
14 }

```

```
13
14 // Zuweisen der Funktion auf submit-Listener des Formulars
15 document.getElementById("dataform").addEventListener("submit", isValid);
```

Auf Änderungen überprüfen

Während `click` für Buttons ideal ist, benötigen Formularelemente wie Dropdown-Menüs (`<select>`), Checkboxes oder Textfelder einen anderen Auslöser. Hier ist das `change`-Event das Mittel der Wahl. Es wird ausgelöst, sobald der Wert eines Elements geändert und die Änderung „abgeschlossen“ wurde (z. B. durch Drücken von Enter oder Verlassen des Feldes).

Besonders bei Dropdowns bietet das DOM einen sehr komfortablen Zugriff, denn es existiert eine Eigenschaft `value` beim `<select>`-Element, die immer den Wert der ausgewählten Option enthält. Es muss also nicht „händisch“ abgefragt werden, welches `<option>`-Element gewählt wurde.

Folgendes Formular mit einem Dropdown sei gegeben:

```
1 <form action="" method="post">
2   <label for="choice">Auswahl:</label>
3   <select id="choice">
4     <option value="A">Wert A</option>
5     <option value="B">Wert B</option>
6     <option value="C">Wert C</option>
7   </select>
8 </form>
```

Im `<head>` der HTML-Datei ist wieder eine JavaScript-Datei mit `defer` eingebunden. Deren Inhalt ist wie folgt:

```
1 function valChange(event) {
2   window.alert(`${event.currentTarget.value} wurde ausgewählt`);
3 }
4
5 document.getElementById("choice").addEventListener("change", valChange);
```

Nachdem `defer` sichergestellt hat, dass das HTML vollständig geladen ist, wird das `<select>`-Element über seine `id` gesucht. Seinem `change`-Listener wird die Funktion `valChange()` zugewiesen. Wann immer in der Selectbox etwas neues ausgewählt wird, wird diese Funktion aufgerufen.

`valChange()` gibt in einem Dialogfenster aus, welcher Wert ausgewählt wurde. Da der Event-Listener am `<select>`-Element angebracht ist, kann – wie in Abschnitt 10.5.4 gezeigt – über `event.currentTarget` dieses abgefragt werden. In der Eigenschaft `value` steht jener Wert aus der Option, die gewählt wurde. Hier ist dies entweder „A“, „B“ oder „C“.

Abbildung 10.3 zeigt das Ergebnis, nachdem Wert C ausgewählt wurde.

10.6 Weiterführende Literatur

Zur Arbeit mit dem DOM sind die deutschen JavaScript-Fachbücher [1, 48, 58] zu empfehlen, die dank ihrer kontinuierlichen Neuauflagen auch moderne DOM-Neuerungen

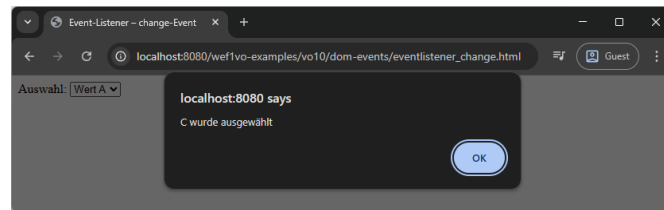


Abbildung 10.3: Sobald eine Option ausgewählt wird, kommt ein Dialogfenster und gibt dessen Wert aus. Das Fenster erscheint dabei noch bevor der Wert in der Selectbox umgestellt wird.

miteinbeziehen. Alle DOM-Spezialwerke sind mittlerweile zu alt, um empfohlen werden zu können.

Als Online-Referenz sei an dieser Stelle auch nochmals [39] erwähnt, welches zum Nachschlagen diverser JavaScript-Objekte hilfreich sein kann.

Vorlesung 11

JavaScript Workflow und Tools

11.1 Allgemeines zu Workflows

JavaScript hat sich in den letzten Jahren zur dominanten Sprache für Web-Frontend-Entwickler*innen entwickelt. Dabei haben sich auch die Arbeitsweisen stark diversifiziert. Von der Modifizierung einzelner Elemente mittels nativem DOM, über die Arbeit mit klassischen Hilfsbibliotheken wie jQuery¹ (welches heute meist nur mehr in Legacy-Projekten verwendet wird) bis hin zum Einsatz moderner Frontend-Frameworks wie React, Vue.js, Angular oder Svelte – die Arbeitsweisen und Ansätze sind vielfältig. Durch die aufkommende Popularität von Node.js haben sich ebenfalls neue Möglichkeiten ergeben, die in den folgenden Abschnitten auch angeschnitten werden.

Ebenso haben sich im Bereich der Stylesheets neue Möglichkeiten aufgetan. Zwar bietet natives CSS mittlerweile mächtige Features, die früher Präprozessoren vorbehalten waren (z. B. CSS-Variablen oder Verschachtelung), doch in professionellen Projekten stoßen reine CSS-Dateien oft an organisatorische Grenzen. Hier helfen Workflows, um Stylesheets modular zu verwalten, automatisch für unterschiedliche Browser zu optimieren oder mathematische Operationen schon vor der Auslieferung zu berechnen.

Schließlich sind auch Performance und Codequalität Faktoren, die in modernen Webprojekten entscheidend sind. Schnelle Ladezeiten sind essentiell für eine gute User Experience (UX) und das Ranking in Suchmaschinen. Zu diesem Zweck müssen Style- und Script-Dateien technisch optimiert und verkleinert (minifiziert) werden. Parallel dazu gilt es, durch Coding-Standards (Linting) Flüchtigkeitsfehler zu vermeiden und sauberen, wartbaren Code zu erzeugen – eine Anforderung, die nicht nur in großen Teams wichtig ist.

Das Ziel solcher Workflows ist die Automatisierung: Sie sollen Entwickler*innen von repetitiven Aufgaben entlasten, damit der Fokus auf der eigentlichen Umsetzung liegen kann. Statt manuell Dateien zu verkleinern oder Syntax zu prüfen, übernehmen Tools diese Arbeit im Hintergrund. Dieses Kapitel zeigt auf, wie ein solcher Workflow beispielhaft aufgebaut werden kann.

¹<https://jquery.com/>

11.2 Node.js zur Workflow-Unterstützung

Wenn von Workflows gesprochen wird, geht es in der Regel immer auch um Automatisierung. Dinge sollen möglichst ohne wiederholendes Zutun des*der Entwicklers*Entwicklerin geschehen, damit diese sich auf den Kern ihrer Arbeit – das Entwickeln einer Webseite konzentrieren können. Um Automatisierung zu ermöglichen, benötigt es Tools, die bestimmte Aufgaben ausführen, etwa Dateien verarbeiten oder auf Änderungen in diesen zu überprüfen. Diese Aufgaben übernehmen ausführbare Dateien, engl. „binaries“, die in verschiedenen Sprachen geschrieben werden können. Populär sind etwa Tools in Ruby oder Python, Linux/Unix-Bash bzw. (Power-) Shell-Skripte, selbst Windows-Executables sind möglich. Sind Tools in derartigen Sprachen verfasst, ist es notwendig, die entsprechenden Laufzeitumgebungen auf dem eigenen Rechner installiert zu haben, etwa einen Ruby-Interpreter, um Ruby-Programme (sogenannte „Gems“) ausführen zu können. Dies ist jedoch nicht immer einfach, da die verschiedenen Plattformen unterschiedlich auf diversen Betriebssystemen unterstützt sind. Windows-Executables lassen sich nur mit viel Aufwand auf Unix-System ausführen, auch umgekehrt muss die entsprechende Umgebung installiert werden, damit Bash-Skripte unter Windows ausführbar sind. Darüber hinaus ist es dann vielfach unumgänglich, die jeweiligen Sprachen zu beherrschen.

Zusätzlich zu den soeben genannten Umgebungen hat sich seit einigen Jahren noch eine weitere etabliert, die jedoch nicht immer sofort als solche erkannt wird: *Node.js*.

11.2.1 Node.js im Überblick

Node.js ist eine Plattform zur Entwicklung von serverseitigem JavaScript. Sie baut auf Googles V8 JavaScript-Laufzeitumgebung (engl. „runtime“) auf und ermöglicht es, JavaScript-Applikationen am Server auszuführen. Somit lassen sich – einfach ausgedrückt – komplette Webseiten mit JavaScript erstellen, so wie es mit anderen serverseitigen Sprachen wie etwa PHP möglich ist. Node.js baut dabei auf einem eventbasierten Workflow auf, d. h. ein Webserver, auf dem Node.js läuft, generiert einen Event, wenn eine neue Verbindung eingeht und ruft eine Funktion auf (sprich einen Event-Handler), die sich um diese Verbindung kümmert.

Der folgende Code startet einen Webserver mit Node.js und zeigt bei jedem Request „Hello World!“ an:

```
1 import {createServer} from "node:http";
2
3 const hostname = "127.0.0.1";
4 const port = 3000;
5
6 const server = createServer((request, response) => {
7   response.statusCode = 200;
8   response.setHeader("Content-Type", "text/plain");
9   response.end("Hello World");
10 });
11
12 server.listen(port, hostname, () => {
13   console.log(`Server running at http://${hostname}:${port}/`);
14 });
```

Der Code liegt in einer Datei namens `nodestserver.mjs`. Die Erweiterung kennzeichnet ein so genanntes ECMAScript-Modul, welches in modernen Node.js-Anwendungen zum Einsatz kommt.

ECMAScript Module

ECMAScript Module (ESM, Dateierweiterung `.mjs`) sind ein standardisiertes Modulsystem für JavaScript, das es ermöglicht, Code in separate Dateien zu organisieren und wiederzuverwenden. Sie nutzen die `import` und `export` Schlüsselwörter, um Funktionen, Objekte oder Werte zwischen Modulen auszutauschen. ESM bieten einen sauberen, zukunftssicheren Ansatz für die Strukturierung von JavaScript-Code, der sowohl in Browsern als auch in Node.js-Umgebungen funktioniert.

Zunächst wird aus dem Node-Module (auch Package genannt) `node:http`, welches die Funktionalität enthält, um einen Webserver zu starten die Methode `createServer()` importiert. Die Angabe in geschweiften Klammern nennt man eine destrukturierende Zweisung² und importiert nur diese eine Methode und nicht das gesamte Modul.

Nach der Angabe des Hostnamens und eines Ports, auf dem der Server laufen soll wird mit `createServer()` ein neues Webserver-Objekt erstellt. Als Parameter wird eine sogenannte Callback-Funktion übergeben (hier in Form einer Arrow-Funktion mit `() => { ... }`, es könnte aber auch als anonyme Funktion der Form `function () { ... }` geschrieben werden), die immer dann aufgerufen wird, wenn eine Verbindung zum Webserver hergestellt wird, d. h. wenn jemand die Seite im Browser öffnet. In diesem Fall wird der Statuscode „200 – OK“ gesetzt und der Inhalt als reines Textdokument definiert. Danach wird „Hello World!“ ausgegeben und die Ausgabe beendet. Über den Aufruf von `listen()` hört der Webserver auf dem angegebenen Port. Im Browserfenster kann nun über `http://127.0.0.1:3000/` oder alternativ `http://localhost:3000/` diese Seite geöffnet werden. In der Konsole wird der URL zum Server mit Port angegeben.

Damit dies alles jedoch überhaupt funktioniert, muss zunächst Node.js installiert und dieses Script damit ausgeführt werden.

11.2.2 Node.js installieren

Damit mit Node.js gearbeitet werden kann, muss es am eigenen Rechner installiert werden (damit später Applikationen auf einem Webserver ausgeführt werden können, muss es selbstredend auf dem Webserver ebenso installiert sein). Zur Installation wird von der offiziellen Website³ unter [Download](#) das Installationsprogramm („Prebuilt Installer“) für die aktuellste Version – derzeit 25.4.0 – für das jeweilige Betriebssystem heruntergeladen und im Anschluss installiert.

Bei der Installation unter Windows wird abgefragt, ob Tools für native Module installiert werden sollen. Diese sind nötig, um für manche Pakete, entsprechende ausführbare Dateien zu kompilieren. Wählen Sie diese Option daher aus (siehe Abbildung 11.1).

Alternativ können auch Package-Manager unter den jeweiligen Betriebssystemen verwendet werden, um den Installer nicht manuell herunterladen zu müssen:

²<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring>

³<https://nodejs.org/>

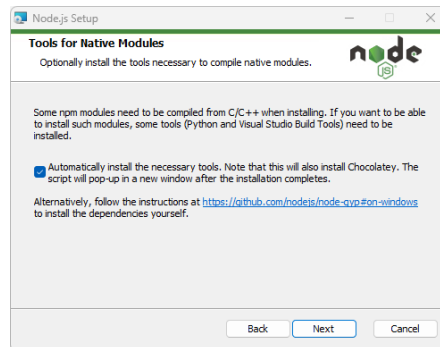


Abbildung 11.1: Bei der Installation sollen die Werkzeuge für native Module mitinstalliert werden, um später auch zuverlässig jedes Node-Package hinzufügen zu können.

- Windows mit *winget*: `winget install OpenJS.NodeJS`
- macOS mit *Homebrew*: `brew install node`

Um zu überprüfen, ob die Installation erfolgreich war, kann unter Windows die Eingabeaufforderung oder PowerShell bzw. unter macOS der Terminal geöffnet werden. Dort steht nun der Befehl `node` zur Verfügung, über den die Node.js Laufzeitumgebung gestartet wird. Durch `node -v` kann zunächst die installierte Version angezeigt werden:

```
> node -v
```

Die Ausgabe sieht wie folgt aus:

```
v25.4.0
```

Um nun das Webserver-Beispiel aus Abschnitt 11.2.1 auszuführen, wird der Name der JavaScript-Datei, die den Code enthält (hier `nodestserver.mjs`), als Argument an den `node`-Befehl übergeben:

```
> node nodestserver.mjs
```

11.2.3 Node.js Packages

Neben der Möglichkeit, skalierbare Netzwerkanwendungen zu erstellen, bietet Node.js noch ein zweites, großes Potential: Packages, auch Module genannt. Im Webserver-Beispiel in Abschnitt 11.2.1 wurde in Zeile 1 ein solches Package, nämlich `http` (im Namensraum `node`) eingebunden. Es enthält alle Funktionalitäten, um den Webserver auszuführen. Ebenso wie dieses Modul gibt es eine Vielzahl von der Community erstellten Packages, die unterschiedlichste Funktionalitäten vordefiniert zur Verfügung stellen. Einige davon sind für Workflow-Tätigkeiten konzipiert und werden in den folgenden Abschnitten im Detail erläutert.

Packages verwalten mit npm

Um Packages zu installieren, verwalten, aktualisieren oder auch zu entfernen, dient *npm*, der Node.js Package Manager. Gestartet als Zusatzprojekt ist dieser im Installationspaket von Node.js enthalten und kann direkt in der Kommandozeile bzw. im Terminal

verwendet werden. Um die aktuell installierte Version abzufragen (die übrigens nicht mit der Node.js Version übereinstimmt, da es sich um ein separates Projekt handelt), wird `npm -v` eingetippt:

```
> npm -v
```

Die Ausgabe der aktuellen Version lautet dann:

11.8.0

npm kann Packages global und lokal installieren. Globale Packages gelten für die gesamte Node.js-Installation und sind unabhängig von Projekten immer aufrufbar, d. h. sie agieren wie Commandline-Tools. Lokale Packages sind nur in dem Projekt (sprich Verzeichnis) verfügbar, in dem sie installiert werden.

Packages global installieren

Um ein Package oder Modul global zu installieren, kommt die folgende Syntax zur Anwendung:

```
> npm install -g <package>
```

Die Option `-g` stellt die globale Installation sicher, `<package>` wird mit dem jeweiligen Paketnamen ersetzt. Dieser Aufruf installiert das Package **sass** (eine JavaScript-Implementierung für den CSS-Präprozessor Sass, siehe Abschnitt 11.3) global:

```
> npm install -g sass
```

Packages lokal installieren

Um ein Package nur für das aktuelle Projekt zu installieren, wird folgender Aufruf eingesetzt:

```
> npm install <package>
```

`package` symbolisiert erneut das zu installierende Paket, eine Option ist keine nötig. Der folgende Aufruf installiert **sass** für das aktuelle Projekt, also *nicht* global:

```
> npm install sass
```

Die lokale Installation erzeugt einen Ordner **node_modules** im aktuellen Verzeichnis, in den das Modul und alle seine Abhängigkeiten (engl. „dependencies“) installiert werden.

Lokale Packages verwalten

Um die genauen Versionsnummern der installierten Packages zu verwalten, kann eine Datei namens **package.json** erstellt werden. Sie enthält eine Auflistung aller Module, die vom aktuellen Projekt benötigt werden. Dabei wird unterschieden, ob ein Package für die Ausführung, d. h. für den Produktivbetrieb oder nur für die Entwicklung vorausgesetzt wird.

Um eine derartige Datei zu erzeugen, muss folgender Aufruf in der Kommandozeile getätigt werden:

```
> npm init
```

Es folgt eine kurze Abfrage verschiedener Details zum Projekt wie dessen Name, Version und Beschreibung. Die meisten Optionen können so bestätigt werden, wie sie vorgeschlagen werden. Wer die Abfrage ganz umgehen möchte, gibt gleich `npm init -yes` ein. Um benötigte Packages anzugeben, werden diese lokal mit `npm install` installiert, dies wird in der Datei `packages.json` unter `dependencies` vermerkt. Um etwa das `sass`-Package zu installieren, wird folgendes aufgerufen:

```
> npm install sass
```

Eine `package.json`-Datei für ein Projekt das `sass` benötigt, könnte so aussehen:

```
1 {
2   "name": "wef1vo-sass-example",
3   "version": "1.0.0",
4   "description": "Example for using Sass in your project",
5   "keywords": [
6     "sass",
7     "example",
8   ],
9   "license": "ISC",
10  "author": "Wolfgang Hochleitner",
11  "type": "module",
12  "main": "index.js",
13  "scripts": {
14    "test": "echo \"Error: no test specified\" && exit 1"
15  },
16  "dependencies": {
17    "sass": "^1.97.3"
18  }
19 }
20 }
```

Es ist ersichtlich, dass nicht nur der Package-Name sondern auch die Versionsnummer spezifiziert wird. Dies ermöglicht es, die Abhängigkeiten des eigenen Projekts besser reproduzierbar und somit mit anderen teilbar zu machen.

11.3 Stylesheet Präprozessoren am Beispiel von Sass

Mit dem Wissen über Node.js und seine Module kann nun ein erstes Beispiel für einen typischen Frontend-Workflow demonstriert werden. Im Zentrum dessen steht Sass – ein CSS-Präprozessor.

11.3.1 Sass Einführung

*Sass*⁴ – kurz für Syntactically Awesome Stylesheets – ist ein CSS-Präprozessor, der die Arbeit mit Stylesheets erleichtern und Schwachstellen von CSS ausmerzen soll. Die grundsätzliche Funktionsweise besteht darin, dass Sass eine eigene Syntax, die SCSS-Syntax⁵ (Sassy CSS), bereitstellt, in der Stylesheets geschrieben werden. Die SCSS-

⁴<https://sass-lang.com/>

⁵Sass kennt auch die von YAML abgeleitete Sass-Syntax, diese ist jedoch weniger weit verbreitet und wird hier nicht näher besprochen.

Syntax mit Endung `.scss` ist dabei ein Superset der CSS-Syntax, d. h. jede CSS-Datei ist automatisch eine SCSS-Datei, letztere besitzen aber noch zusätzliche Befehle, die es in CSS nicht gibt. Über ein Commandline-Tool – den Präprozessor – werden SCSS-Dateien dann in herkömmliche CSS-Dateien umgewandelt und sind somit für den Browser verwendbar.

Wurde Sass für Node.js (**sass**) wie in Abschnitt 11.2.3 global installiert, kann der Präprozessor wie folgt aufgerufen werden:

```
> sass input.scss output.css
```

Die Input-Datei `input.scss` wird vom Präprozessor verarbeitet, aus ihr wird die Datei `output.css` generiert.

Im Folgenden werden die wichtigsten Grundelemente von Sass gezeigt, anhand derer einerseits die Syntax erklärt und auch die Grundidee vermittelt werden sollen. Das Grundprinzip von Sass lautet dabei *DRY* – Don't repeat Yourself. Codeteile, die in CSS aufgrund der Syntax öfters wiederholt werden müssen (da es nicht anders möglich ist), sollen in Sass nur einmal definiert werden.

11.3.2 Variablen

Ein Grundproblem in CSS ist die ungewollte Wiederholung von Informationen. Ein typisches Beispiel ist die Definition von Farben. Werden in einem Projekt etwa Grundfarben für Schrift und Hintergrund festgelegt, so kommen diese in der Regel an vielen Stellen im Stylesheet bzw. den Stylesheets vor. Soll die Farbe im Verlauf des Projekts geändert werden, müssen alle Vorkommen gesucht und entsprechend geändert werden. Sass eliminiert dieses Problem durch die Einführung von *Variablen*. Diese werden mit einem `$`-Zeichen definiert und können beliebige Werte wie Farben oder Schriftarten aufnehmen.⁶

Im folgenden Beispiel werden drei Variablen für Schriftfarbe (`$main-color`) und Schrift (`$main-font`), sowie für die Hintergrundfarbe (`$bg-color`) definiert und diese bei der Formatierung des `<body>`- und der `<a>`-Elemente angewandt. Die Quelldatei liegt unter `_scss/main.scss`, die Zieldatei unter `css/main.css`.

```
1 $main-color: #bada55;
2 $main-font: system-ui, sans-serif;
3 $bg-color: rgb(41 49 51);
4
5 body {
6   font-family: $main-font;
7   color: $main-color;
8   background-color: $bg-color;
9 }
10
11 a:link, a:visited, a:hover, a:active {
12   color: $main-color;
13 }
```

Nun muss der CSS-Präprozessor aufgerufen werden:

⁶Anmerkung: CSS beherrscht, wie in Abschnitt 5.4 gezeigt, Custom Properties, die Präprozessor-Variablen ähnlich sind. Es bestehen jedoch erhebliche Unterschiede, die hier erläutert werden: <https://www.smashingmagazine.com/2018/05/css-custom-properties-strategy-guide/>.

```
sass _scss/main.scss css/main.css
```

Dies erzeugt die folgende CSS-Datei:

```
1 body {
2   font-family: system-ui, sans-serif;
3   color: #bada55;
4   background-color: rgb(41, 49, 51);
5 }
6
7 a:link, a:visited, a:hover, a:active {
8   color: #bada55;
9 }
```

Die Werte für Schriftfarbe und -art wurden an den entsprechenden Stellen eingesetzt und mussten nicht mehrfach geschrieben werden. Ebenso ist ersichtlich, dass der Sass-Präprozessor noch die ältere `rgb()`-Schreibweise mit Beistrichen erzeugt.

11.3.3 Verschachtelung

Sass erlaubt es, CSS-Selektoren zu verschachteln, sodass spezifischere Selektoren dabei entstehen, ohne dass wiederum Information verdoppelt wird. Im folgenden Beispiel werden Elemente einer Navigation – die ungeordnete Liste, die Listenelemente, sowie die enthaltenen Links – im `<nav>`-Element *verschachtelt*:

```
1 nav {
2   ul {
3     list-style: none;
4     padding: 0;
5   }
6
7   li {
8     display: inline-block;
9   }
10
11   a {
12     text-decoration: none;
13   }
14 }
```

Nachdem der Präprozessor wie zuvor gezeigt aufgerufen wurde, ergibt sich folgender CSS-Code:

```
1 nav ul {
2   list-style: none;
3   padding: 0;
4 }
5
6 nav li {
7   display: inline-block;
8 }
9
10 nav a {
11   text-decoration: none;
12 }
```

Durch die Verschachtelung ist die hierarchische Zusammengehörigkeit der Elemente besser gegeben bzw. überhaupt ablesbar. In CSS liegen alle Selektoren auf einer Ebene.

Mittlerweile (seit Dezember 2023) unterstützen alle modernen Browser native CSS Verschachtelung (engl. „nesting“)⁷ [78]. Die Syntax dabei der von Sass sehr ähnlich. Sass zu verwenden bedeutet jedoch, dass das erzeugte CSS jedoch ohne Nesting entsteht, was maximale Kompatibilität – auch mit älteren Browsern – ermöglicht.

11.3.4 Partials und Module

Sass erlaubt die einfache Modularisierung von SCSS-Dateien. Um eine Datei mit nur kleinen SCSS-Teilen zu erstellen, wird dem Dateinamen ein Underscore (`_`) vorangestellt. Dadurch weiß der Präprozessor, dass es sich um ein sogenanntes *Partial* handelt und der Präprozessor versucht nicht, eine eigene CSS-Datei daraus zu generieren. Um das Partial zu verwenden, wird es mittels `@use`-Regel eingebunden. Damit behandelt Sass das Partial als Modul, alle Variablen oder Mixins (siehe Abschnitt 11.3.5) werden dann mit dem Namen des Partials vorangestellt angesprochen. Diese Art der Modularisierung ist sehr ähnlich zu ECMAScript-Modulen.

Im folgenden Beispiel, werden die drei Variablendeklarationen von Abschnitt 11.3.2 in eine eigene Datei namens `_variables.scss` ausgelagert:

```
1 $main-color: #bada55;  
2 $main-font: system-ui, sans-serif;  
3 $bg-color: rgb(41 49 51);
```

Diese Datei wird nun in `main.scss` ganz zu Beginn als Modul geladen, danach folgen die restlichen Regeln, in denen die Variablen mit dem Modulnamen `variables` vorangestellt angesprochen werden:

```
1 @use "variables";  
2  
3 body {  
4   font-family: variables.$main-font;  
5   color: variables.$main-color;  
6   background-color: variables.$bg-color;  
7 }  
8  
9 a:link, a:visited, a:hover, a:active {  
10  color: variables.$main-color;  
11 }
```

Es entsteht dieselbe CSS-Datei wie in Abschnitt 11.3.2. Bei der `@use`-Anweisung müssen sowohl der Underscore als auch die Dateiendung *nicht* angegeben werden.

Partials haben keine direkte Entsprechung in nativem CSS. die `@import`-Anweisung kann zwar auch mehrere CSS-Dateien in einer einbinden, dies erzeugt jedoch zusätzliche HTTP-Requests.

11.3.5 Mixins

Mixins sind Gruppen von SCSS-Deklarationen, die ähnlich wie eine Funktion einmal definiert und beliebig oft angewandt werden. Das Schlüsselwort lautet `@mixin`, danach folgt der Name des Mixins und in Klammern können – wie bei einer Funktion – Parameter angegeben werden, die beim Einbinden des Mixins angegeben werden.

⁷<https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Nesting>

Das folgende Mixin setzt den linken und rechten Innenabstand (Padding) auf den Wert des ersten Arguments, und den Wert des oberen und unteren Außenabstands (Margin) auf den Wert des zweiten Arguments und wendet es für alle Absätze an. Dazu wird die `@include`-Direktive benötigt:

```
1 @mixin padding-margin($pad, $mar) {  
2   padding-left: $pad;  
3   padding-right: $pad;  
4   margin-top: $mar;  
5   margin-bottom: $mar;  
6 }  
7  
8 p {  
9   @include padding-margin(10px, 20px);  
10 }
```

Nach Ausführung des CSS-Präprozessors entsteht folgender CSS-Code:

```
1 p {  
2   padding-left: 10px;  
3   padding-right: 10px;  
4   margin-top: 20px;  
5   margin-bottom: 20px;  
6 }
```

Dieses Mixin kann nun beliebig oft bei anderen Elementen mit unterschiedlichen Werten verwendet werden und erspart viel Schreibarbeit und/oder Code-Wiederholung.

Mixins existieren in nativem CSS bisher nur bedingt. Mit dem *CSS Functions and Mixins Module* [73] gibt es jedoch einen Entwurf, mit dem die Direktive `@function` in CSS eingeführt wird, um genau diese Funktionalität zu bieten. Derzeit funktioniert diese experimentell nur in Blink-basierten Browsern (Chrome, Edge, Opera).

11.3.6 Vererbung

Vererbung ist eine der nützlichsten Eigenschaften von Sass. Sie erlaubt es, Deklarationen, die in mehreren Selektoren vorkommen sollen, nur einmal anzugeben. Im Gegensatz zu Mixins sind keine Argumente möglich, dafür erzeugt Vererbung in der Regel weniger aufgeblähten CSS-Code.

Im nachfolgenden Beispiel werden drei Klassen für Nachrichten erzeugt. Eine generische `.message`-Klasse, die Rahmen und Schriftfarbe festlegt und eine `.status`- sowie `.error`-Klasse, die auf der vorigen aufbauen, aber den Rahmen grün bzw. rot definieren:

```
1 .message {  
2   border: 3px solid #ccc;  
3   padding: 10px;  
4   color: #333;  
5 }  
6  
7 .status {  
8   @extend .message;  
9   border-color: green;  
10 }  
11  
12 .error {  
13   @extend .message;
```

```
14 border-color: red;
15 }
```

Nach der Ausführung des Präprozessors entsteht folgender CSS-Code:

```
1 .message, .error, .status {
2   border: 3px solid #ccc;
3   padding: 10px;
4   color: #333;
5 }
6
7 .status {
8   border-color: green;
9 }
10
11 .error {
12   border-color: red;
13 }
```

Vererbung auf Selektorebene ist mit nativem CSS derzeit nicht realisierbar.

11.3.7 Weitere Funktionen

Sass wartet noch mit einer Reihe an weiteren Funktionen auf. Hierfür sei auf die Sass-Dokumentation [46] verwiesen.

11.4 Tasks automatisieren mit Gulp

Das Beispiel Sass hat gezeigt, dass mit Hilfe von Tools wie einem CSS-Präprozessor ein effizienterer Arbeitsstil erreicht werden kann. Das große Problem am gezeigten Workflow ist allerdings, dass der Präprozessor jedes Mal händisch in der Kommandozeile gestartet werden musste, sobald etwas an einer der SCSS-Dateien verändert wird:

```
sass _scss\main.scss css\main.css
```

Weiters skaliert diese Art Tools aufzurufen nur sehr schlecht. Sollen die Stylesheets im Anschluss auf Korrektheit überprüft und minifiziert werden, müssen noch zwei weitere Befehle auf der Commandline aufgerufen werden, sodass der Aufwand, die Tools zu starten, den Nutzen irgendwann übersteigt. Aus diesem Grund gibt es sogenannte Task-Runner, die derartige Aufgaben effektiv übernehmen. Ein solcher ist Gulp.

11.4.1 Gulp im Überblick

*Gulp*⁸ ist ein Task-Runner, öfters auch Build Tool genannt, das wiederholende und zeitintensive Aufgaben automatisiert. Diese Aufgaben werden in Tasks zusammengefasst, die in der Regel wie folgt ablaufen:

1. Quelldateien (z. B. `.js`, `.scss` etc.) aus dem Dateisystem laden.
2. Tools in der Pipeline hintereinander auf diese Dateien ausführen (z. B. Sass-Präprozessor, Minifier, Linter etc.).

⁸<https://gulpjs.com/>

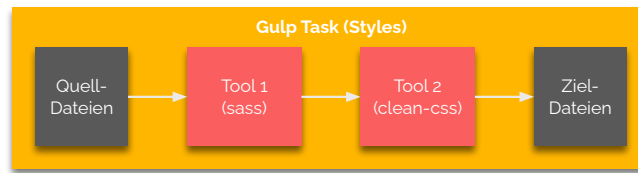


Abbildung 11.2: Ein exemplarischer Task in Gulp. Zuerst werden die Quelldateien geladen, darauf werden zwei Tools ausgeführt und das Ergebnis wird wieder gespeichert.

3. Veränderte Dateien wieder zurück ins Dateisystem (in der Regel mit neuem Namen) schreiben.

Abbildung 11.2 stellt diesen Ablauf für einen Styles-Task, bei dem zuerst Sass und dann ein Minifizierer (clean-css) ausgeführt werden, schematisch dar.

In den folgenden Abschnitten wird zunächst Gulp konfiguriert und dann für verschiedene Tasks eingesetzt.

11.4.2 Gulp vorbereiten

Gulp ist wie viele andere Tools als Node.js Modul verfügbar. Da es sich aber um ein explizites Commandline-Tool handelt, besteht es eigentlich aus zwei Teilen:

1. **Gulp-CLI:** Das Gulp Commandline-Interface (CLI) wird global installiert. Es ändert sich selten und hat nur die Aufgabe, das lokal installierte Gulp zu starten.
2. **Gulp:** Die lokale Gulp-Installation übernimmt die eigentliche Arbeit. Sie muss lokal in jedem Projekt separat installiert werden.

Gulp CLI installieren

Im ersten Schritt muss das Gulp CLI installiert werden. Dazu eine Commandline bzw. einen Terminal öffnen und folgendes eingeben:

```
> npm install -g gulp-cli
```

package.json initialisieren

Als nächstes muss das Verzeichnis des eigenen Webprojekts mit einer `package.json`-Datei initialisiert werden. Dazu in der Commandline ins entsprechende Verzeichnis wechseln und folgendes eingeben:

```
> npm init --yes
```

Dies erzeugt eine entsprechende Datei mit den Standardwerten.

Gulp lokal installieren

Nun muss die lokale Gulp-Installation vollzogen werden. Dazu wird Gulp mit dem Parameter `--save-dev` oder kürzer `-D` zu den Development-Dependencies hinzugefügt. Es wird somit als Tool zur Entwicklung, nicht zur Produktion gekennzeichnet.

```
> npm install -D gulp
```

Gulpfile erzeugen

Die Tasks für Gulp werden als ECMAScript-Modul in einer Datei namens `gulpfile.mjs` angelegt. Diese wird im Projektverzeichnis leer (z. B. mit PhpStorm) erstellt. Nun können in dieser Datei Tasks angelegt werden. Dieser Prozess wird im folgenden Abschnitt erklärt.

11.4.3 Tasks definieren

Gulp Tasks werden in `gulpfile.mjs` definiert. Diese Datei wird wie ein ECMAScript-Modul behandelt. Ein Modul ist, wie schon in Abschnitt 11.2.1 gezeigt, eine Datei mit JavaScript-Code, deren Funktionen so aufbereitet werden können, dass sie woanders wiederverwendbar sind. Dazu existiert das Schlüsselwort **export**. Alle Werte und Funktionen, denen dieses Schlüsselwort vorangestellt wird, werden exportiert und können „von Außen“ verwendet werden, wenn das Modul eingebunden wird.

So verhält es sich auch mit Gulp-Tasks. Diese werden als einfache Funktionen definiert. Werden sie anschließend dem `exports`-Objekt zugewiesen, sind sie öffentlich und können von Gulp aus als Task aufgerufen werden. Geschieht dies nicht, sind sie private Tasks, die nur als Hilfs- oder Teiltasks innerhalb eines öffentlichen Tasks verwendet werden können.

Der folgende Code definiert einen öffentlichen Gulp-Task, der „Hallo Welt!“ auf der Console ausgibt und das prinzipielle definieren eines Tasks veranschaulicht:

```
1 export function test(cb) {  
2   console.log("Hallo Welt!");  
3   cb();  
4 }
```

Zunächst wird eine Funktion `test()` mit vorangestelltem **export** definiert, die über `console.log()` die Hallo-Welt-Botschaft ausgibt. Da Gulp-Tasks asynchron sind (mehr dazu im zweiten Semester), muss angedeutet werden, wann die Funktion mit ihren Aufgaben fertig ist. Dies geschieht entweder, wenn etwas mit **return** zurückgegeben wird, oder muss sonst – wie in diesem Beispiel – mit einem sogenannten „Error-first Callback“ geschehen. Dies ist eine Funktion, die aufgerufen wird, um den Abschluss der Aufgaben einer Funktion anzuzeigen. Sie wird hier `cb()` (kurz für Callback) genannt, kann aber prinzipiell jeden Namen tragen. Wird sie ohne Parameter aufgerufen, zeigt Gulp an, dass alles korrekt funktioniert hat. Mehr dazu in der Gulp Dokumentation⁹.

Um den Task auszuführen, wird er mit dem Schlüsselwort **gulp** in der Konsole eingegeben:

```
> gulp test
```

Das Ergebnis sieht wie folgt aus:

```
[12:00:00] Using gulpfile ...\\wef1vo-examples\\vo11\\gulp\\gulpfile.mjs  
[12:00:00] Starting 'test'...  
Hallo Welt!  
[12:00:00] Finished 'test' after 1.68 ms
```

⁹<https://gulpjs.com/docs/en/getting-started/async-completion/>

Komplexere Tasks verwenden für gewöhnlich ein Node.js-Modul, das eine bestimmte Aufgabe übernimmt, z. B. Stylesheet-Preprocessing. Daher sind in der Regel immer mehrere Schritte notwendig.

1. Das benötigte Node.js-Modul (Package) wird als Development-Dependency installiert: `npm install -D <package>`.
2. Das Package (Modul) in `gulpfile.mjs` mit `import` einbinden: `import name from "package";`.
3. Gulp selbst ebenfalls mit `import` destrukturierend einbinden, um auf dessen Methoden `src()` (Lesen von Quelldateien) und `dest()` (Schreiben in Zieldateien) zugreifen zu können.
4. Mittels `src()` die Quelldateien auswählen, die verarbeitet werden sollen.
5. Das Modul mit der `.pipe()`-Methode einbinden, sodass es auf die Quelldateien wirkt.
6. Das Ergebnis mit der `.pipe()`-Methode in ein durch `dest()` spezifiziertes Ziel schreiben.

Dieser Prozess wird nun anhand von mehreren Beispielen demonstriert.

11.4.4 JavaScript-Minifizierung mit gulp-terser

Eine einfache Art und Weise, die Ladegeschwindigkeit von Seiten zu erhöhen, ist die *Minifizierung* (engl. „minification“) von JavaScript-Dateien. Dabei werden alle unnötigen Leerzeichen entfernt und Variablennamen so kurz wie möglich gehalten. Ein Tool, das dies beherrscht, ist `terser`¹⁰, welches für Gulp als `gulp-terser`¹¹ verfügbar ist.

Es muss zunächst im Projektverzeichnis lokal als Development-Dependency installiert werden:

```
> npm install -D gulp-terser
```

Danach wird `gulpfile.mjs` editiert: zunächst wird den Konstanten `src` und `dest` das Gulp-Modul zugewiesen. Dies ist die bereits angesprochene destrukturierende Zuweisung und wählt von einem Modul (hier das Gulp-Modul) zwei Methoden (hier `src()` und `dest()`) aus. Dies entspricht dem kompletten Import des Gulp-Moduls und den anschließenden Zuweisungen `const src = gulp.src` und `const dest = gulp.dest`, ist jedoch effizienter, da eben nicht das komplette Modul geladen werden muss. Anschließend wird das `gulp-terser`-Modul eingebunden und dafür eine Task-Funktion namens `scripts` erstellt, die mit `export` exportiert wird:

```
1 import { src, dest } from "gulp";
2 import terser from "gulp-terser";
3
4 export function scripts() {
5   return src("_js/**/*.js")
6     .pipe(terser())
7     .pipe(dest("js"));
8 }
```

¹⁰<https://www.npmjs.com/package/terser>

¹¹<https://www.npmjs.com/package/gulp-terser>

In der Methode `scripts()` wird zunächst mittels `src()` eine Auswahl an Quelldateien getroffen. Das Argument `_js/**/*.js` ist das sogenannte *Globstar*-Muster. Es besagt, dass im Verzeichnis `_js` (der erste Teil des Musters) und in allen Unterverzeichnissen (der `**`-Teil) alle Dateien mit der Endung `.js` (der `.js`-Teil) miteinbezogen werden sollen.

Über die `pipe()`-Methoden werden nun auf diese ausgewählten Dateien Funktionen aufgerufen. Zunächst ist dies `terser()`, die Hauptmethode des `gulp-terser`-Packages. Sie bewirkt das oben beschriebene Minifizierungs-Verhalten.

Nachdem die Datei durch diese Methode verändert wurde, muss der veränderte Inhalt wieder zurück ins Dateisystem geschrieben werden. Dies geschieht durch den Aufruf `dest()`, der ebenfalls mit `pipe()` angehängt wird. Hier wird nur der Verzeichnisname (in diesem Fall `js`) angegeben, der Dateiname ändert sich nicht. Wäre dies gewünscht, könnte dazu das Modul `gulp-rename` zuvor in die Pipeline gehängt werden.

Im Verzeichnis `_js` befindet sich als Ausgangsbasis die Datei `scripts.js` mit folgendem Inhalt:

```
1 "use strict";
2
3 const contentContainer = document.getElementById("content");
4
5 const tools = ["Node.js", "Gulp", "Sass", "clean-css"];
6
7 for (const toolName of tools) {
8   const card = document.createElement("p");
9   card.classList.add("tool-card");
10  card.textContent = `Tool: ${toolName}`;
11  contentContainer.appendChild(card);
12 }
```

Die Datei wird mit `defer` eingebunden und erzeugt nach dem Laden des Dokuments für jeden Eintrag im Array `tools` einen Absatz, der mit der Klasse `tool-card` formatiert wird und hängt diesen ins `<main>`-Element, welches durch die ID `content` gefunden wird. Nun wird der `scripts`-Task ausgeführt:

```
> gulp scripts
```

Das Ergebnis ist die Datei `js▶scripts.js` mit folgendem, einzeiligem Inhalt:

```
1 "use strict";const contentContainer=document.getElementById("content"),tools=["Node.js","Gulp","Sass","clean-css"];for(const t of tools){const n=document.createElement("p");n.classList.add("tool-card"),n.textContent=`Tool: ${t}`,contentContainer.appendChild(n)}
```

In diesem einfachen Beispiel ist der Gewinn durch die Minifizierung nicht sehr groß (261 Bytes vs. 345 Bytes), bei umfangreicheren Dateien zahlt sich dies in der Regel immer aus. Globale Variablen auf oberster Ebene, wie hier `tools` werden standarmäßig nicht minifiziert, da sie möglicherweise in einer anderen JavaScript-Datei oder im HTML angesprochen werden. Ist man sich sicher, dass dies nicht der Fall ist, kann über die Verwendung eines Options-Objekt als Parameter für `terser()` dies geändert werden: `terser({ toplevel: true })`.

11.4.5 Präprozessor für Styles mit Minifizierung und Sourcemaps

Auch für CSS-Dateien gibt es Gulp-Workflows. Im folgenden Beispiel wird zunächst der CSS-Präprozessor Sass gestartet, um SCSS-Dateien in CSS-Dateien zu konvertieren. Dann wird `clean-css` aufgerufen, um das Ergebnis zu minifizieren. Schließlich werden noch sogenannte Sourcemaps generiert, sodass in den Browser-Development-Tools nicht die minifizierten CSS-Dateien, sondern die unveränderten SCSS-Dateien betrachtet werden können.

Zunächst werden die vier Packages `sass`¹², `gulp-sass`¹³, `gulp-clean-css`¹⁴ sowie `gulp-sourcemaps`¹⁵ als Development-Dependencies installiert. Dies kann in einem Aufruf geschehen:

```
> npm install -D sass gulp-sass gulp-clean-css gulp-sourcemaps
```

In `gulpfile.mjs` werden alle vier Module eingebunden und eine neue Task-Funktion namens `styles()` erstellt und mit `export` gekennzeichnet. Das Modul `sass` wird dabei extra benötigt, da `gulp-sass` nicht auf einen Sass-Compiler festgelegt ist und dieser zunächst spezifiziert werden muss. Dieser Aufruf von `gulpSass` mit dem Compiler `dartSass` wird in der Variable `sass` hinterlegt und kann somit in `scripts()` einfacher aufgerufen werden.

```
1 import * as dartSass from "sass";
2 import gulpSass from "gulp-sass";
3 import cleanCSS from "gulp-clean-css";
4 import sourcemaps from "gulp-sourcemaps";
5
6 const sass = gulpSass(dartSass);
7
8 export function styles() {
9   return src("_scss/main.scss")
10     .pipe(sourcemaps.init())
11     .pipe(sass())
12     .pipe(cleanCSS())
13     .pipe(sourcemaps.write())
14     .pipe(dest("css"));
15 }
```

Als Quelldatei wird hier kein Globstar-Muster gesetzt, sondern direkt die Datei `_scss/main.scss` verwendet. Sie bindet mittels `@use`-Anweisungen weitere SCSS-Dateien ein, daher ist sie die einzige, die verarbeitet werden muss.

Im ersten `pipe()`-Aufruf werden die Sourcemaps mittels `sourcemaps.init()` initialisiert. Dies muss geschehen, bevor andere Operationen in der Pipeline ausgeführt werden, weiters müssen alle nun folgenden Plugins Sourcemaps unterstützen.

Der nächste Aufruf ist `sass()`, welches eine Datei `main.css` aus der SCSS-Datei erzeugt.

Dann geschieht durch den Aufruf von `cleanCSS()` die Minifizierung und schließlich werden die Sourcemaps mittels `sourcemaps.write()` in die CSS-Datei inline geschrieben. Das Ergebnis landet im Ordner `css`.

¹²<https://www.npmjs.com/package/sass>

¹³<https://www.npmjs.com/package/gulp-sass>

¹⁴<https://www.npmjs.com/package/gulp-clean-css>

¹⁵<https://www.npmjs.com/package/gulp-sourcemaps>

Die Ausgangsbasis der SCSS-Dateien sah wie folgt aus:

```

1  /* _variables.scss */
2  $red: rgb(255 0 0);
3  $gray-light: #f3f3f3;
4  $gray-dark: #888888;
5
6  /* _color.scss */
7  @use "variables";
8
9  h1 {
10     color: variables.$red;
11 }
12
13 /* _type.scss */
14 body {
15     font-family: system-ui, sans-serif;
16 }
17
18 /* _components.scss */
19 @use "variables";
20
21 .tool-card {
22     background-color: variables.$gray-light;
23     border: 1px solid variables.$gray-dark;
24     border-radius: 8px;
25     padding: 1rem;
26     margin-bottom: 0.5rem;
27     font-weight: bold;
28 }
29
30 /* main.scss */
31 @use "type";
32 @use "color";
33 @use "components";

```

Nun wird der Task in der Commandline aufgerufen:

```
> gulp styles
```

Das Ergebnis des Gulp Tasks in `css/main.css` sieht wie folgt aus:

```
1 body{font-family:system-ui,sans-serif}h1{color:red}.tool-card{background-color:#
  f3f3f3;border:1px solid #ddd;border-radius:8px;padding:1rem;margin-bottom:.5rem;
  font-weight:700}
2 /*# sourceMappingURL=data:application/json;charset=utf8;base64,eyJ2ZXJzaW9uIjozLCJz
  b3VyY2VzIjpbIl90eXB1bnNjc3MiLCJfY29sb3Iuc2NzcyIsIl92YXJpYWJsZXMuMuc2NzcyIsIl9jb21
  wb251bnRzLnNjc3MiXSwwbmFtZXMxO1tdLCJtYXBwaW5ncyI6IkFBQWUesSOFDRSxZQUFBLFBnBQWUesQ0
  FBQSxXQONGLEdBQOUstUNISSxJQOVOLFdBQOUsaUJERlcsUUNHWCxPQUFBLElBQWUesTUFBBQWUesLQUNBL
  GNBQWUesSUFDDQWUesRQUFBLEtBQOEsY0FBQWUesNQUNBLF1BQWUesLCJmaWxlIjoibWpfbWpbi5jc3MiLCJz
  b3VyY2VzQ29udGVudCI6WyJib2R5IHtccclxuICBmb250LWZhbWlseTogc3lzdGVtLXVpLCBzYW5zLXN1cm1
  m01xyXG59IiwicHVzZSBcInZhcm1hYmxlc1wi01xyXG5ccclxuaDEge1xyXG4gIGNvbG9y0iB2YXJpYW
  JsZXMuH1J1ZDtcclxufSIsIiRyZWQ6IHJnYyguNTUgMCAwK1tccclxuJGdyYXktbGlnaHQ6ICNmM2YzZ
  jM7XHJcbiRncmF5LWRhcms6ICNkZGRkZGQ7IiwicHVzZSBcInZhcm1hYmxlc1wi01xyXG5ccclxunRv
  b2wtY2FyZCB7XHJcbiAgYmFja2dyb3VuZC1jb2xvcjogdmFyaWVibGVzLiRncmF5LWxpZ2h001xyXG4
  gIGJvcmlRlcjogMkY6IH1HnVbG1kIHZhcmlhYmxlc1kZ3JheS1kYXJr01xyXG4gIGJvcmlRlc1yYWRpdX
  M6IDhweDtcclxuICBwYWRkaW5nO1AcXcmV01xyXG4gIG1hcmdpb1ib3R0b206IDAuNXJlbTtccclxuI
  CBmb250LXdlawdodDogYm9sZDtcclxufSJdfQ== */
```

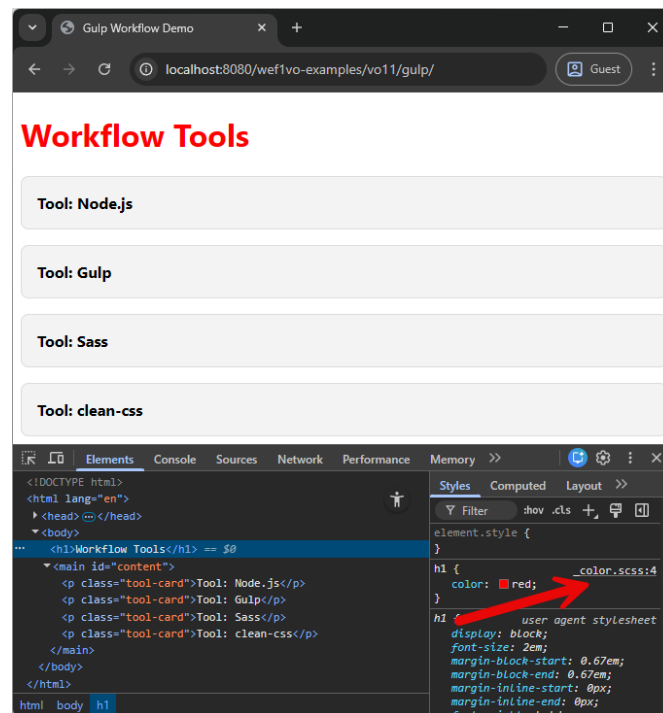


Abbildung 11.3: Die DevTools des Browsers zeigen an, dass die Definition der Regel für die `<h1>`-Überschrift aus der Datei `_color.scss` stammt, obwohl der Browser nur die minimierte CSS-Datei `main.css` zur Verfügung hat. Diese Zuordnung geschieht über die bereitgestellten Sourcemaps.

Der CSS-Code ist auf eine Zeile komprimiert, darunter befindet sich die Sourcemap, die Base64-kodiert (und daher nicht wirklich lesbar) ist. Dennoch zeigt sich in den DevTools des Browsers (Abbildung 11.3) ihre Auswirkung.

Beim Minifizieren wird sichtbar, dass `clean-css` hier wo immer möglich Zeichen spart:

- `rgb(255 0 0)` wird zu `red`,
- `#dddddd` wird zu `#ddd`,
- `0.5rem` wird zu `.5rem` und
- `bold` wird zu `700`.

11.4.6 Codequalität durch Linting

Automatisierte Tasks mit Gulp müssen nicht immer Änderungen an Dateien nach sich ziehen. Vielmehr können auch Tools aufgerufen werden, die nur lesend wirken. Ein solches Beispiel ist *Linting*, also die Ausführung eines Codeanalyse-Tools, das Fehler oder verletzte Coding-Standards anzeigt. Linter existieren für viele Sprachen, im Folgenden soll ESLint, ein Linter für JavaScript zum Einsatz kommen.

Zunächst muss das entsprechende Node.js Modul, nämlich `gulp-eslint-new`¹⁶ installiert werden:

¹⁶<https://www.npmjs.com/package/gulp-eslint-new>

```
> npm install -D gulp-eslint-new
```

Danach wird in `gulpfile.mjs` das Modul eingebunden, eine eigene Task-Funktion namens `lint()` erstellt und exportiert:

```
1 import esLintNew from "gulp-eslint-new";
2
3 export function lint() {
4   return src("_js/**/*.js")
5     .pipe(esLintNew())
6     .pipe(esLintNew.format())
7     .pipe(esLintNew.failAfterError());
8 }
```

Zunächst werden alle Dateien mit der Endung `.js` im Ordner `_js` und allen Unterverzeichnissen inkludiert. Dann wird die Funktion `esLintNew()` in der Pipeline aufgerufen. Dies speichert das Ergebnis des Lintings in der Variable `esLintNew` und macht es so für die Ausgabe oder auch allfällige andere Module verfügbar. `esLintNew.format()` gibt das Ergebnis auf der Console aus und `esLintNew.failAfterError()` weist den Prozess an, mit einem Fehler zu beenden, falls der Code nicht den angegebenen Standards entsprochen hat. Dies kann z. B. verwendet werden, um nachfolgende Tasks gar nicht erst auszuführen.

Mit diesen Einstellungen alleine kann ESLint jedoch nicht laufen. Es wird noch eine Konfiguration benötigt, die angibt, worauf überprüft werden soll. Diese kann u. a. in einer Datei namens `eslint.config.mjs` abgelegt werden. Ist sie vorhanden, liest sie ESLint automatisch aus und führt die Überprüfung danach durch. Diese Datei ist ebenfalls ein ECMAScript-Modul und kann etwa so aussehen:

```
1 export default [
2   {
3     rules: {
4       "semi": "error", // Error on missing semicolon
5       "indent": "warn", // Warning on inconsistent indentation
6       "prefer-const": "error" // Error when let is used instead of const
7     }
8   }
9 ];
```

Fehlende Strichpunkte werden als Fehler betrachtet, ist die Einrückung nicht korrekt, wird eine Warnung angezeigt. Ebenso wird ein Fehler gemeldet, wenn `let` anstatt `const` bei der Variablendeklaration verwendet wird.

Angenommen, die Datei `scripts.js` wäre wie folgt aufgebaut, d. h. in Zeile 8 würde `let` statt `const` verwendet, in Zeile 9 fehlte das Semikolon und Zeile 11 wäre falsch eingerückt ...

```
1 "use strict";
2
3 const contentContainer = document.getElementById("content");
4
5 const tools = ["Node.js", "Gulp", "Sass", "clean-css"];
6
7 for (const toolName of tools) {
8   let card = document.createElement("p");
9   card.classList.add("tool-card")
10  card.textContent = `Tool: ${toolName}`;
```



```
11   contentContainer.appendChild(card);
12 }
```

...so würde der Aufruf von

```
> gulp lint
```

folgenden Output erzeugen:

```
[12:00:00] Using gulpfile ...\\wefivo-examples\\vo11\\gulp\\gulpfile.mjs
[12:00:00] Starting 'lint'...
[12:00:00] ...\\wefivo-examples\\vo11\\gulp\\_js\\scripts.js
   8:9   error    'card' is never reassigned. Use 'const' instead    prefer-const
   9:36  error    Missing semicolon                                  semi
  11:1   warning   Expected indentation of 4 spaces but found 6      indent

X 3 problems (2 errors, 1 warning)
  2 errors and 1 warning potentially fixable -
  see https://www.npmjs.com/package/gulp-eslint-new/v/2.5.0#autofix

[12:00:00] 'lint' errored after 329 ms
[12:00:00] ESLintError in plugin "gulp-eslint-new"
Message:
  Failed with 2 errors
```

11.4.7 Testserver mit Live-Reload

Neben diversen Tools, die den Umgang mit Files vereinfachen, ist auch der Test im Browser unumgänglich. Auch hierbei gibt es Tools, die diese Angelegenheit vereinfachen. Dazu zählt das Package *Browsersync*, das automatisch einen lokalen Testserver erstellt und einen Browser zum Testen öffnet. Sind mehrere Browserfenster mit der Seite im Testserver geöffnet, können diese auch gleichzeitig aktualisiert werden.

Als ersten Schritt wird das Node.js Modul **browser-sync**¹⁷ wie bereits öfters demonstriert, als Development-Dependency installiert:

```
> npm install -D browser-sync
```

Anschließend wird in `gulpfile.mjs` eine Browsersync-Instanz mit `create()` erzeugt und eine Task-Funktion `server()` angelegt, in der Browsersync mit `init()` aufgerufen wird:

```
1 import browserSync from "browser-sync";
2
3 const bs = browserSync.create();
4
5 export function server(done) {
6   bs.init({
7     server: {
8       baseDir: "."
9     }
10  });
11  done();
12 }
```

¹⁷<https://www.npmjs.com/package/browser-sync>

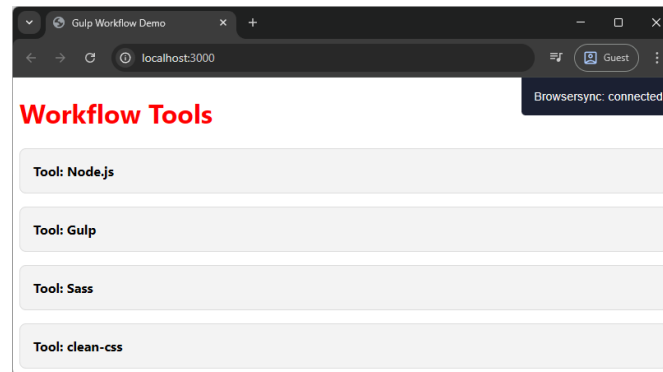


Abbildung 11.4: Nachdem der Gulp-Task ausgeführt wurde, öffnet sich der Browser und zeigt die Seite über einen lokalen Server auf Port 3000 (<http://localhost:3000/>) an.

Aufgerufen wird die Funktion `init()`, diese bekommt als Parameter ein JSON-Konfigurationsobjekt mit. In dessen Angabe `baseDir` steht der Pfad zu den HTML-Dateien. Befinden sich diese direkt auf der untersten Ebene, reicht `.` als Angabe. Liegen die Dateien im Unterverzeichnis `site`, müsste eben `site` angegeben werden. Die Funktion `done()` ist ein Error-first Callback (wie `cb()` beim Task „test“) und muss zum Schluss aufgerufen werden, um das Ende des Tasks anzudeuten, da hier anders als etwa bei „scripts“ oder „styles“ nichts zurückgegeben wird.

```
> gulp server
```

Über diesen Task wird nun ein lokaler Server auf Port 3000 gestartet und der Standardbrowser öffnet sich und zeigt die Seite an. Abbildung 11.4 stellt dies dar.

11.4.8 Automatische Erkennung von Änderungen

Das Starten eines Webservers ist eine angenehme Hilfestellung, sein volles Potential entfaltet dieser jedoch erst, wenn Änderungen an den Quelldateien erkannt und die Seite daraufhin neu geladen wird. Dies lässt sich mit einer Kombination aus Gulp *Watch-Tasks* und dem Browsersync-Reload lösen. Hierfür werden keine weiteren Module benötigt, es wird einfach ein neuer Task `watcher` in `gulpfile.mjs` angelegt und exportiert, für den Reload wird ein interner (nicht exportierter) namens `reload` erstellt. Damit die Gulp-Funktion `watch()` verfügbar ist, wird diese in der ersten Zeile ins die destrukturierende Zuweisung integriert. Ebenfalls benötigt wird `series()`, eine Funktion zum seriellen Ausführen von Tasks. Auch sie wird hinzugefügt.

```
1 import {src, dest, watch, series } from "gulp";
2
3 function reload(done) {
4   bs.reload();
5   done();
6 }
7
8 export function watcher() {
9   watch("_scss/**/*.scss", series(styles, reload));
10  watch("_js/**/*.js", series(scripts, reload));
11  watch("/*.html", reload);
```

```
12 }
```

Über den Aufruf von `watch()` können Änderungen im Dateisystem beobachtet und bei einer solchen Änderung Dinge aufgerufen werden. Der erste Parameter ist das zu beobachtende Verzeichnis, der zweite definiert die Funktion, der bei einer Änderung aufgerufen werden soll.

Der erste Aufruf in Zeile 9 beobachtet alle `.scss`-Dateien im Ordner `_scss` und allen Unterverzeichnissen. Da Änderungen an einzelnen SCSS-Dateien vorgenommen werden, genügt es hier nicht mehr, nur `main.scss` zu beobachten, sondern alle Dateien. Wenn an einer dieser Dateien eine Änderung festgestellt wurde, wird eine Serie von zwei Tasks ausgeführt: zunächst wird „styles“ aufgerufen, das den Sass-Präprozessor und den Minifier ausführt und die aktualisierte CSS-Datei generiert, im Anschluss wird mittels `reload` der Browserinhalt neu geladen. In diesem internen Hilfstask wird `bs.reload()` aufgerufen, `done()` gibt wieder an, dass die Ausführung behindert ist und Gulp mit anderen Tätigkeiten fortfahren kann.

Beim zweiten Aufruf in Zeile 10 geschieht dasselbe mit den Scripts im Verzeichnis `_js`, die bei einer Änderung erneut minimifiziert werden müssen. Dies übernimmt der Task `scripts`, im Anschluss wird der Browser neu geladen.

Der dritte Aufruf in Zeile 11 beobachtet alle HTML-Dateien im Wurzelverzeichnis. Ändert sich etwas an ihnen wird ebenfalls der `reload`-Task ausgeführt.

Somit kann nun an den CSS-, JavaScript- und HTML-Dateien gearbeitet werden und Änderungen werden automatisch im Browser reflektiert.

11.4.9 Abläufe im Default Task kombinieren

Schließlich können all diese Abläufe noch in einem Task zusammengefasst werden. Wird dieser `default` genannt, so kann er ohne Angabe eines Tasknamens einfach nur durch Eingabe von `gulp` gestartet werden. In diesem Fall werden zunächst die Tasks `styles` und `scripts` ausgeführt, um die CSS- und JavaScript-Dateien zu erzeugen, danach wird der Testserver gestartet und schließlich der Watch-Task, um Änderungen nachzuverfolgen.

Neben der zuvor bereits demonstrierten Funktion `series()` bietet Gulp auch noch `parallel()` an. Diese ermöglicht es, Tasks parallel ablaufen zu lassen. Für den Default-Task wird eine Kombination aus beidem angestrebt, da die Tasks „styles“ und „scripts“ zwar problemlos parallel laufen können, „watch“ jedoch erst ausgeführt werden sollte, wenn „server“ ausgeführt wurde und der Server erst starten sollte, wenn die CSS- und JS-Dateien erzeugt wurden.

Damit auch `parallel()` verfügbar ist, wird es beim der Destrukturierung angeführt. Eine eigene Task-Funktion muss für den Default-Task nicht erstellt werden. Beim Export werden einfach die anderen Funktionen kombiniert:

```
1 import {src, dest, watch, series, parallel} from "gulp";  
2  
3 export default series(parallel(styles, scripts), server, watcher);
```

Dieser Task wird, wie bereits erwähnt, einfach wie folgt ausgeführt:

```
> gulp
```

Somit ist ein toolgestütztes Frontend-Entwickeln möglich, das mühsame Tasks wie das Neuladen der Seite abnimmt und gleichzeitig bereits Rücksicht auf CSS-Präprozessoren und Minifizierung nimmt.

11.4.10 Dependencies ersetzen

Das Node.js-Ökosystem baut auf einer Vielzahl von Paketen auf, die zum Großteil von der Community entwickelt und gewartet werden. Im Laufe der Zeit werden in verschiedenen Node.js-Modulen Sicherheitslücken entdeckt, diese werden in der Regel auch in neuen Versionen behoben. Wird eine Bibliothek aber länger nicht aktualisiert, beruht sie weiterhin auf den alten Abhängigkeiten (Dependencies), die Sicherheitslücken aufweisen. Diese Probleme werden beim Installieren auch oft angezeigt. Weiters kann **npm audit** eingegeben werden, um einen Bericht über problematische Pakete zu erhalten.

Um nun npm zu zwingen, eine neuere Version einer Dependency zu verwenden, obwohl die beim Paket angegebenen noch eine ältere ausweisen, kann in der Datei **package.json** ein Abschnitt **overrides** spezifiziert werden. In diesem werden Paketnamen mit Versionen angegeben, die dann statt den definierten Abhängigkeiten installiert werden.

Um bei einem fiktiven Paket **some-package** Version 4.5.1 oder neuer zu erzwingen, wird folgender Abschnitt in **package.json** hinzugefügt:

```
1 "overrides": {  
2   "some-package": "4.5.1"  
3 }
```

Bei der Ersetzung von Abhängigkeiten sollte jedoch immer mit Vorsicht vorgegangen werden. Major-Versionen enthalten oft nicht rückwärtskompatible Änderungen (engl. „breaking changes“), die dann dazu führen, dass der Ablauf nicht mehr funktioniert.

Zudem werden die hier gezeigten Pakete ohnehin nur als Development-Dependencies inkludiert, d. h. sie werden nur lokal während der Entwicklung verwendet, weshalb kaum ein Risiko besteht.

11.5 Weiterführende Literatur

Bücher zum Thema Node.js sind zwar zahlreich, allerdings in der Regel schon wieder veraltet, sobald sie auf den Markt kommen, da sich das Framework mit rasenden Schritten weiterentwickelt. Wer eine Einführung sucht, der greift am Besten zu einem JavaScript-Buch wie [1], das auch eine Node.js-Einführung bietet. Ansonsten ist auch das Web voll von Tutorials und Beispielen, die beim Start helfen können.

Zum Thema CSS-Präprozessoren verhält es sich ähnlich. Bücher wie [58] bietet eine kurze Einführung in die Thematik, besser aufgehoben ist man jedoch meist mit Tutorials aus dem Web oder der offiziellen Dokumentation [46].

Zum Thema Gulp sowie Workflow-Management im Generellen gibt es jedoch eine klare Buchempfehlung. Mit [4] gibt es ein Werk, das von einem Hagenberg-Absolventen stammt und neben dem Task-Runner Gulp auch noch weitere Tools zum Workflow-Management vorstellt.

Vorlesung 12

HTML JavaScript APIs

12.1 Allgemein

Die Vorlesungen 1 und 2 gaben eine Einführung in HTML, wobei immer von seiner letztgültigen Version – dem Living Standard – ausgegangen wurde. Themen waren hierbei vor allem semantische und strukturgebende Elemente, wie etwa `<header>`, `<footer>`, `<nav>` oder `<main>`, genauso aber auch neue Formularfeld-Typen wie etwa `<input type="email">` oder `<input type="number">`.

Dies ist jedoch bei weitem nicht alles, was HTML zum Zeitpunkt seiner Einführung an Neuem mitbrachte. Dinge wie eine eigene Zeichenfläche, die Möglichkeit, Video ohne Plugin abzuspielen oder auch die Fähigkeit, die Position von Benutzer*innen abzufragen, sind mächtige Werkzeuge. Während HTML zwar dafür eigene Elemente zur Verfügung stellt, ist der Zugriff auf diese Möglichkeiten jedoch nur mit JavaScript möglich, weshalb dieses Themengebiet erst jetzt angeschnitten wird.

Genau genommen, gehören einige dieser Funktionalitäten gar nicht zur HTML-Spezifikation. Technologien wie Canvas, Video, Web Storage sowie Drag & Drop sind zwar Bestandteile des HTML Living Standards [54], Geolocation [94] und auch andere (siehe Abschnitt 12.8) sind eigene Spezifikationen, die jedoch meist unter dem Oberbegriff HTML mitverwendet werden. Man spricht in diesem Zusammenhang von HTML JavaScript APIs oder auch einfach Web APIs (API = Application Programming Interface, deutsch „Programmierschnittstelle“), da ein programmatischer Zugriff auf die Daten in den HTML-Elementen ermöglicht wird.

12.1.1 HTML-APIs und Feature Detection

Moderne Browser unterstützen die meisten der hier gezeigten APIs. Dennoch ist es gute Praxis, zu überprüfen, ob ein Feature verfügbar ist, bevor es genutzt wird. Dies nennt man *Feature Detection*.

Entwickler*innen können dazu bei *Can I use...*¹ nachschlagen und feststellen, ab welcher Browserversion bestimmte Funktionalitäten verfügbar sind.

Um programmatisch (d. h. im JavaScript-Code) die aktuellen verfügbaren Fähig-

¹<https://caniuse.com/>

keiten eines Browsers festzustellen, wurden früher eigene Bibliotheken wie *Modernizr*² eingesetzt. Heute geschieht dies meist direkt mit JavaScript. Um etwa zu prüfen, ob Geolocation verfügbar ist, reicht eine einfache If-Abfrage mit dem `in`-Operator, der `true` zurückgibt, wenn eine Eigenschaft (hier `geolocation`) in einem Objekt (hier `navigator`) vorhanden ist:

```
1 if ("geolocation" in navigator) {
2   // Geolocation ist verfügbar
3 } else {
4   // Fallback oder Fehlermeldung
5 }
```

Wurde festgestellt, dass eine Funktionalität fehlt, kann in seltenen Fällen ein sogenannter *Polyfill* (ein Nachbau der Funktion in JavaScript) nachgeladen werden.

Feature Detection für CSS

Oft ist es notwendig, nicht nur JavaScript-APIs, sondern auch die Unterstützung für bestimmte CSS-Eigenschaften zu prüfen (z. B. für komplexe Layouts wie `subgrid` oder grafische Effekte). Während dies im Stylesheet direkt mit der `@supports`-Regel³ geschehen kann, ist es manchmal notwendig, diese Information im JavaScript-Code zu haben.

Die moderne Methode hierfür ist die Funktion `CSS.supports()`⁴. Sie gibt `true` zurück, wenn der Browser die Kombination aus Eigenschaft und Wert bzw. die Bedingung versteht:

```
1 // Prüfen, ob eine Eigenschaft mit einem Wert existiert
2 if (CSS.supports("display", "grid")) {
3   // Grid-spezifische Logik aktivieren
4 }
5
6 // Eine Bedingung, wie bei @supports prüfen
7 if (CSS.supports("selector(&)")) {
8   console.log("CSS Nesting wird unterstützt");
9 }
```

12.2 Canvas

Die Definition des `<canvas>`-Elements ist in [54, Abschnitt 4.12.5] wie folgt festgelegt:

The canvas element provides scripts with a resolution-dependent bitmap canvas, which can be used for rendering graphs, game graphics, art, or other visual images on the fly.

Es handelt sich also um eine „auflösungsabhängige Bitmap-Zeichenfläche, die zur on-the-fly Darstellung von Graphen, Spielegrafiken, Kunst oder anderen Bildern“ verwendet werden kann.

²<https://modernizr.com/>

³<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@supports>

⁴https://developer.mozilla.org/en-US/docs/Web/API/CSS/supports_static

Ein `<canvas>`-Element wird zunächst in HTML mit einem `width` und `height` Attribut – also konkreten Dimensionen in Pixeln – angelegt. Zur leichteren Auffindung in JavaScript wird eine ID vergeben:

```
1 <canvas width="640" height="480" id="draw">Ihr Browser unterstützt Canvas leider  
nicht!</canvas>
```

Dies erzeugt eine Zeichenfläche mit 640×480 Pixeln, jedoch ohne jeglichen Inhalt. Der Text zwischen `<canvas>` und `</canvas>` wird dabei nur angezeigt, falls der Browser das Element nicht unterstützt. Dies könnte aber auch über JavaScript zuvor überprüft und anders abgefangen werden, sollte JavaScript aber deaktiviert sein, ist ein derartiger „Fallback“-Text durchaus sinnvoll.

Um nun im *Canvas* „zeichnen“ zu können, muss der sogenannte *Zeichenkontext* (engl. „drawing context“) abgefragt werden. Er repräsentiert quasi den editierbaren Inhalt. Derzeit existiert ein im HTML Living Standard definierter 2D-Kontext [54, Abschnitt 4.12.5.1], die Technologie WebGL⁵ bietet jedoch auch einen 3D-Kontext [30, 31], der sich an der OpenGL ES-Spezifikation orientiert.

Alle Zugriffe auf das in HTML angelegte `<canvas>`-Element erfolgen nun in JavaScript. Im folgenden Code wird das Element gesucht und sein Kontext abgefragt:

```
1 const canvas = document.getElementById("draw");  
2 const ctx = canvas.getContext("2d");
```

Sobald der Kontext vom `<canvas>`-Element abgefragt wurde, kann auf verschiedene Arten Inhalt hinzugefügt werden.

12.2.1 Rechtecke

Als einfachste zu zeichnende Form bieten sich Rechtecke an. Mehr Grundformen (wie etwa Kreise) sind derzeit nicht vorhanden.

Mit Hilfe des abgefragten Context-Objekts können drei Methoden zum Zeichnen von Rechtecken verwendet werden:

- `fillRect(x, y, width, height)`: Zeichnet ein gefülltes Rechteck an der Position (x, y) mit der Breite *width* und Höhe *height* füllt es mit dem aktuellen Füll-Stil.
- `strokeRect(x, y, width, height)`: Zeichnet ein Rechteck ohne eine Füllung aber mit Rahmenlinie an der Position (x, y) mit der Breite *width* und Höhe *height*. Die Art der Linie wird vom aktuell eingestellten Linien-Stil vorgegeben.
- `clearRect(x, y, width, height)`: Löscht die Pixel, die vom Rechteck an der Position (x, y) mit der Breite *width* und Höhe *height* verdeckt würden.

Füll- und Linien-Stil

Wie ein gefülltes oder umrandetes Rechteck genau dargestellt wird, hängt vom Füll- bzw. Linien-Stil ab. Diese werden über die beiden Eigenschaften `fillStyle` und `strokeStyle` beim Kontext gesetzt. Als Werte können entweder CSS-Farbangaben wie `blue`, `red` oder `#00FF00` oder ebenfalls mit JavaScript erzeugte Farbverläufe (siehe [54, Abschnitt 4.12.5.1.10]) verwendet werden.

⁵<https://www.khronos.org/webgl/>

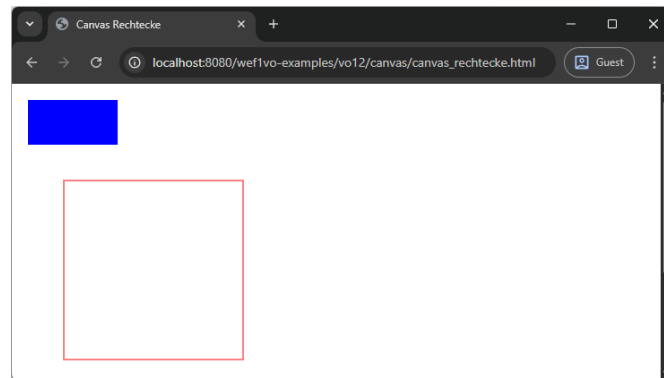


Abbildung 12.1: Durch die gesetzten Füll- und Linien-Stile sind ein blaues, gefülltes Rechteck mit 100×50 Pixeln und ein transparentes Rechteck mit roter Linie und 200×200 Pixeln entstanden. Die Ausmaße des Canvas selbst betragen 640×480 Pixel, was die Scrollbalken beim Browserfenster erklärt.

Werden der *Füll-* oder *Linienstil* gesetzt, gelten sie so lange, bis sie neu gesetzt werden, der Canvas zurückgesetzt oder die Seite neu geladen wird. Die Standardangabe ist jeweils schwarz.

Im folgenden Ausschnitt wird der Füll-Stil auf blau und der Linien-Stil auf rot gesetzt, danach werden zwei Rechtecke gezeichnet. Das Ergebnis ist in Abbildung 12.1 ersichtlich.

```
1 ctx.fillStyle = "blue";
2 ctx.strokeStyle = "#FF0000";
3 ctx.fillRect(10, 10, 100, 50);
4 ctx.strokeRect(50, 100, 200, 200);
```

Mit der Eigenschaft `lineWidth` lässt sich darüber hinaus die Stärke einer Linie bestimmen. `lineCap` bestimmt, wie die Linie abgeschlossen wird. Mögliche Werte hierfür `butt`, `round` und `square`.

Koordinaten im Canvas

Wie im obigen Beispiel ersichtlich, wird das blaue Rechteck an den Koordinaten (10,10) positioniert. Beim Vergleich mit der Darstellung in Abbildung 12.1 wird schnell klar, dass sich der Ursprung des Koordinatensystems links oben befindet. Abbildung 12.2 stellt dies schematisch dar.

12.2.2 Pfade

Fortgeschrittenere Formen lassen sich über *Pfade* realisieren. Das Vorgehen dabei gleicht dem Zeichnen mit einer Füllfeder. Diese wird an einem Punkt aufgesetzt und eine Linie bis zu einem zweiten Punkt gezogen. Doch bevor der Pfad in Tinte erscheint, wird er zunächst mit Bleistift vorgezeichnet und erst abschließend mit der Füllfeder permanent gezeichnet.

Um quasi mit dem Bleistift die Linien vorzuzeichnen, existieren zwei Methoden beim Kontext-Objekt:

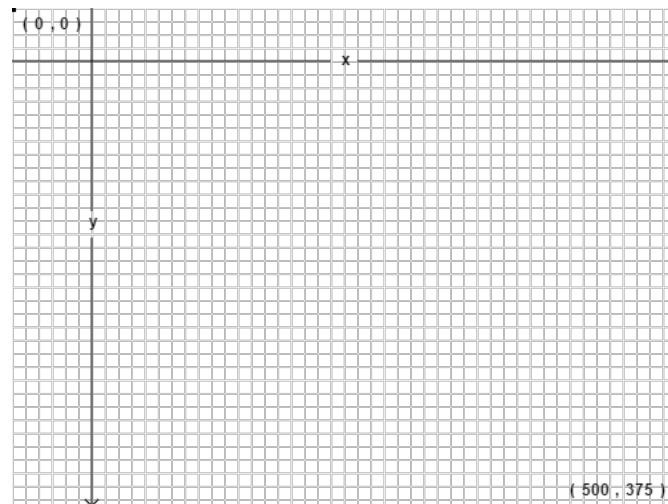


Abbildung 12.2: Der Ursprung des Canvas-Koordinatensystems liegt links oben. Die Abbildung ist aus [43, S. 60] entnommen und wurde selbst mit dem `<canvas>`-Element erzeugt.

- `moveTo(x, y)`: Setzt den Stift an die angegebene (x, y) -Position.
- `lineTo(x, y)`: Zieht eine Linie zur spezifizierten (x, y) -Endposition.

Diese Methoden lassen sich so oft wie gewünscht oder benötigt aufrufen, doch sie bewirken noch keine Darstellung der gezogenen Linien. Erst durch den Aufruf der Methode `stroke()` werden die Pfade gezeichnet, also quasi mit Tinte verewigt. Ist statt einer Linie eine Füllung des Pfades gewünscht, kann `fill()` aufgerufen werden.

Jeder Aufruf von `moveTo()` (also quasi jedes Neu-Aufsetzen des Stifts) erzeugt dabei einen eigenen Sub-Pfad. Soll dieser schnell geschlossen werden, also eine Linien vom zuletzt erstellten zum ersten Punkt gezogen werden, kann `closePath()` aufgerufen werden.

Die Methode `beginPath()` entfernt alle bisherigen erstellten und nicht mit `stroke()` oder `fill()` gezeichneten Sub-Pfade und erlaubt es so, weitere Pfade zu zeichnen, die nicht mit alten zusammenhängen.

Schließlich können auch andere Formen außer einfache Linien erstellt werden:

- `arc(x, y, radius, startAngle, endAngle)`: Erstellt einen ganzen Kreis (bzw. Kreisteil), mit Mittelpunkt (x, y) , dem Radius *radius* und gegebenen Anfangs- und Endwinkeln.
- `rect(x, y, width, height)`: Erstellt einen geschlossenen Pfad, der ein Rechteck repräsentiert.
- `quadraticCurveTo(cpx, cpy, x, y)`: Erstellt vom aktuellen Punkt aus eine quadratische Bézierkurve zum Endpunkt (x, y) mit dem Kontrollpunkt (cpx, cpy) .
- `bezierCurveTo(cp1x, cp1y, cp2x, cp2y, x, y)`: Erstellt ausgehend vom aktuellen Punkt eine kubische Bézierkurve zum Endpunkt (x, y) mit den Kontrollpunkten $(cp1x, cp1y)$ und $(cp2x, cp2y)$.
- `arcTo(x1, y1, x2, y2, radius)`: Erstellt vom aktuellen Punkt aus einen Kreis

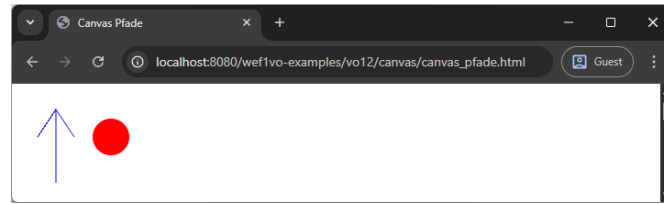


Abbildung 12.3: Zunächst wird die linke Seite der Pfeilspitze gezeichnet, dann die rechte und schließlich die Linie. Dann wird ein neuer Pfad begonnen und ein Kreis an der angegebenen Position erzeugt.

zum Endpunkt (x, y) mit den Kontrollpunkten (x_1, y_1) und (x_2, y_2) und dem Radius *radius*.

Der folgende Code für eine Verwendung im Canvas realisiert einen blauen Pfeil sowie einen rot gefüllten Kreis. Das Ergebnis ist in Abbildung 12.3 zu sehen.

```
1 ctx.strokeStyle = "blue";
2 ctx.moveTo(20, 50);
3 ctx.lineTo(40, 20);
4 ctx.lineTo(60, 50);
5 ctx.moveTo(40, 20);
6 ctx.lineTo(40, 100);
7 ctx.stroke();
8 ctx.beginPath(); // Beginne neuen unabhängigen Pfad
9 ctx.fillStyle = "red";
10 ctx.arc(100, 50, 20, 0, 2 * Math.PI)
11 ctx.fill();
```

12.2.3 Text

Text im Canvas kann mit Hilfe einiger weniger Angaben relativ schnell gesetzt werden. Dazu ist es zuerst nötig, die gewünschte Schrift anzugeben. Dies erfolgt über die Eigenschaft `font`. Diese akzeptiert eine CSS Schriften-Regel, genau wie die `font`-Eigenschaft in CSS (siehe dazu Abschnitt 3.8). Schriftgröße und -art müssen am Ende, andere Eigenschaften davor angegeben werden. Wird eine Schriftart nicht explizit angegeben, wird die für das `<canvas>`-Element geltende vererbt.

Um den Text dann darzustellen, können folgende zwei Methoden verwendet werden:

- `fillText(text, x, y)`: Schreibt den Text `text` an die Position (x, y) und füllt die Zeichen aus.
- `strokeText(text, x, y)`: Schreibt den Text `text` an die Position (x, y) , stellt jedoch nur die Outline dar.

Über die Eigenschaften `textAlign` (mit den Werten `start`, `end`, `left`, `right` und `center`) und `textBaseline` können die horizontale und vertikale Ausrichtung bestimmt werden.

Das folgende Beispiel setzt zwei Zeilen Text mit unterschiedlicher Ausrichtung. Abbildung 12.4 zeigt das Ergebnis.

```
1 ctx.font = "bold 24pt system-ui, sans-serif";
2 ctx.fillText("Hallo WEF1!", 200, 50);
```



Abbildung 12.4: Die x -Position der beiden Textzeilen ist gleich, die erste ist jedoch links-, die zweite rechtsbündig ausgerichtet. Einmal wird der Text gefüllt, beim anderen Mal wird nur die Outline dargestellt.

```
3 ctx.textAlign = "right";
4 ctx.strokeText("Hallo WEF1!", 200, 100);
```

12.2.4 Bilder

Um *Bilder* innerhalb eines Canvas darzustellen wird – wenig überraschend – ein Bild benötigt. Dieses kann entweder im HTML-Code bereits in Form eines `<img>`-Elements existieren oder auch rein in JavaScript angelegt werden.

Um das Bild im Canvas darzustellen, gibt es drei Methoden:

- `drawImage(image, dx, dy)`: Zeichnet das Bild `image` an die Zielkoordinaten (dx, dy) .
- `drawImage(image, dx, dy, dw, dh)`: Zeichnet das Bild `image` an die Zielkoordinaten (dx, dy) und skaliert es auf eine Breite von dw und eine Höhe von dh Pixeln.
- `drawImage(image, sx, sy, sw, sh, dx, dy, dw, dh)`: Entnimmt aus dem zu zeichnenden Bild `image` einen Ausschnitt mit der Breite sw und Höhe sh , ausgehend von (sx, sy) und zeichnet diesen an die Zielkoordinaten (dx, dy) und skaliert ihn auf eine Breite von dw und eine Höhe von dh Pixeln. Dies ist schematisch in Abbildung 12.5 dargestellt.

Existiert im HTML-Markup (d. h. im DOM-Baum) bereits ein `<img>`-Element und soll dessen Quelle als Bild verwendet werden, so muss es nur noch mit JavaScript abgefragt und einer `drawImage()`-Methode übergeben werden. Das Bild ist im HTML-Markup folgendermaßen definiert:

```
1 
```

Nun wird mittels JavaScript auf dieses Bild zugegriffen und es wird erneut im Canvas gezeichnet:

```
1 const img = document.getElementById("image");
2 ctx.drawImage(img, 20, 20);
```

Soll ein komplett neues Bild im Canvas geladen werden, so muss dafür in JavaScript zunächst ein `Image`-Objekt angelegt werden. Dieses repräsentiert ein Bild genauso, wie es durch das `<img>`-Element in HTML repräsentiert wird. Die Bilddatei wird dem Objekt im `src`-Attribut spezifiziert. Wichtig ist nun aber, dass die Darstellung im Canvas erst erfolgen darf, wenn das Bild vollständig geladen ist, daher muss der Aufruf von

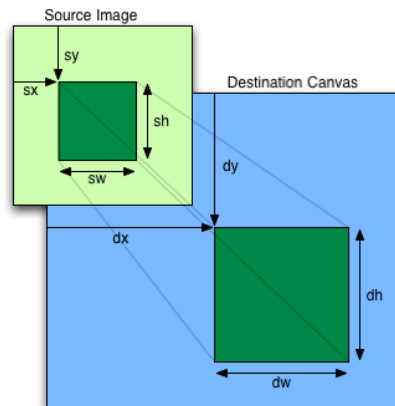


Abbildung 12.5: Aus dem Quellbild (hellgrün) wird ausgehend von der Position (sx, sy) ein Rechteck mit Breite sw und Höhe sh entnommen und in den Canvas (blau) an der Stelle (dx, dy) eingefügt. Der Ausschnitt wird dabei auf die Breite dw und die Höhe dh skaliert. Bildquelle: [54, Abschnitt 4.12.5.1.15].

`drawImage()` innerhalb eines `load`-Event-Handlers beim `Image`-Objekt erfolgen. Der Code hierfür lautet wie folgt:

```
1 const jsImg = new Image();
2 jsImg.src = "../assets/html5logo.png";
3 jsImg.addEventListener("load", function () {
4   ctx.drawImage(jsImg, 0, 24, 92, 104, 150, 60, 46, 52);
5 });
```

Abbildung 12.6 zeigt das Ergebnis beider Verfahren, wobei bei der zweiten Möglichkeit nur ein Ausschnitt aus der Grafik entnommen und skaliert dargestellt wird.

Achtung: Da hier mit einer externen Ressource (dem Bild aus dem DOM) gearbeitet wird, sollte dieser Code auf jeden Fall in einen Event-Listener zum Warten auf das vollständige Laden der Seite verpackt werden. Dabei muss auf den `load`- und nicht auf `DOMContentLoaded`-Event gewartet werden, damit das Bild auch verfügbar ist, wenn in JavaScript darauf zugegriffen wird.

12.2.5 Beispiele

Es existieren bereits zahllose Beispiele zum Thema Canvas. Hier eine kleine Auswahl:

- <https://zenphoton.com/>: Zen Photon Garden von Micah Elizabeth Scott.
- <https://git.nihilogic.dk/wolf3d/>: JavaScript Wolfenstein 3D von Jacob Seidelin.
- <http://media.chikuyonok.ru/ambilight/>: Ambilight for Video Tag von Sergey Chikuyonok.
- <https://viliusle.github.io/miniPaint/>: miniPaint von ViliusL.

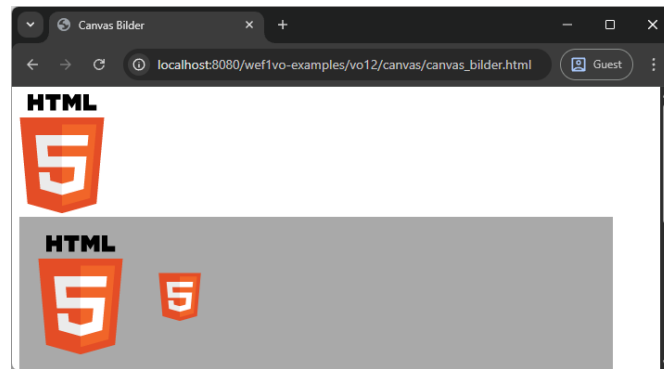


Abbildung 12.6: Die Grafik im weißen Bereich wurde in HTML mittels `<img>`-Element erzeugt. Der Canvas ist zur besseren Übersicht grau hinterlegt. Die linke Grafik ist die Kopie der in HTML erstellten, die rechte wurde komplett in JavaScript angelegt und dargestellt. Dabei kommt die Version von `drawImage()` zum Einsatz, die es erlaubt, einen Ausschnitt zu entnehmen und das Ergebnis zu skalieren.

12.3 Video

Video im Browser war lange Zeit nur durch Verwendung von Plugins wie Adobe Flash oder Apple QuickTime möglich. Trotz der großen Verbreitung dieser Plugins gab es dennoch Limitierungen, wie etwa schlechter Support auf mobilen Plattformen (bzw. völlig fehlender Support von Flash auf iOS). Es wurde daher in HTML das Element `<video>` (geschlossen durch `</video>`) definiert, um einen Standard zu schaffen, der in allen Browsern ohne zusätzliche Technologien funktioniert.

12.3.1 Container und Codecs

Beim Thema Videodateien fallen schnell Formate wie MP4 ein. Bei diesen handelt es sich aber genau genommen nur um sogenannte *Containerformate*. Container definieren nur, *wie* Inhalte gespeichert werden, nicht *welche* Art von Inhalten. Letzteres wird wiederum über sogenannte *Codecs* realisiert. Diese übernehmen die Dekodierung der Video- und Audiodaten und ermöglichen eine Ausgabe als Video am Bildschirm.

Während es im Desktopbereich eine Vielzahl von Containerformaten gibt, sind für HTML-Video nur zwei von Bedeutung:

- **MP4:** Kurz für MPEG-4. Bekannte Dateierendungen sind `.mp4` oder `.m4v`.
- **WebM:** Ein Open-Source Format von Google, die Dateierendung lautet `.webm`.

Folgende Video-Codecs sind werden von aktuellen Browsern unterstützt:

- **H.264:** Ein weit verbreiteter Video-Kompressionsstandard, der hohe Qualität bei relativ niedrigen Bitraten bietet und in vielen Anwendungen wie Streaming verwendet wird.
- **H.265:** Der Nachfolger von H.264, der etwa 25–50 % bessere Datenkompression bei gleicher Videoqualität ermöglicht und für hochauflösende Inhalte wie 4K und 8K optimiert ist.
- **VP8:** Ein offener und lizenzfreier Video-Codec, der von Google entwickelt wurde

Tabelle 12.1: Video-Containerformate mit passenden Video- und Audio-Codecs und Browserunterstützung. C144: Chrome 144, FF147: Firefox 147, E144: Edge 144, O125: Opera 125, S26.2: Safari 26.2.

<i>Format</i>	<i>C144</i>	<i>FF147</i>	<i>E144</i>	<i>O125</i>	<i>S26.2</i>
MP4: H.264 + AAC/MP3	•	○/•	•	•	•
MP4: H.265 + AAC	○	○	○	○	•
MP4: AV1 + AAC/Opus	•	○/•	•	•	•
WebM: VP8 + Vorbis	•	•	•	•	•
WebM: VP9 + Opus	•	•	•	•	•
WebM: AV1 + Opus	•	•	•	•	•

und als Alternative zu H.264 dient, aber inzwischen weitgehend durch neuere Formate ersetzt wurde.

- **VP9:** Der Nachfolger von VP8, ebenfalls von Google entwickelt, der eine verbesserte Kompressionseffizienz bietet und häufig für Streaming-Dienste wie YouTube verwendet wird.
- **AV1:** Ein offener, lizenzfreier Video-Codec, der darauf abzielt, bessere Kompression als H.265 und VP9 bei gleichzeitiger Vermeidung von Lizenzgebühren zu bieten.

Folgende Audio-Codecs können in Browsern derzeit eingesetzt werden:

- **MP3:** Ein weit verbreiteter verlustbehafteter Audiocodec, der gute Klangqualität bei relativ niedrigen Bitraten bietet und aufgrund seiner universellen Kompatibilität nach wie vor sehr populär ist.
- **AAC:** Ein fortschrittlicher verlustbehafteter Audiocodec, der als Nachfolger von MP3 entwickelt wurde und bei gleicher Bitrate eine bessere Klangqualität als MP3 erreicht.
- **Opus:** Ein vielseitiger Open-Source-Audiocodec mit sehr niedriger Latenz, der für interaktive Echtzeitanwendungen wie VoIP und Videokonferenzen optimiert ist und eine breite Palette von Bitraten und Audioqualitäten unterstützt.
- **Vorbis:** Ein freier, verlustbehafteter Audiocodec, der eine effiziente Kompression bei guter Klangqualität bietet, wobei er besonders für mittlere bis hohe Bitraten optimiert ist.

Aus dieser Auflistung an Containern, Video- und Audio-Codecs ist nun eine Reihe von Kombinationen möglich, die jeweils unterschiedlichen Browsersupport haben. Tabelle 12.1 stellt dies dar. Wie ersichtlich ist, ist die Unterstützung gut, lediglich Firefox kann MP4 mit AAC Audio Codec nur wiedergeben, wenn der Codec im Betriebssystem installiert ist und H.265 benötigt in allen Browsern (Ausnahme Safari) eine entsprechende Hardwareunterstützung. Die „sicheren“ Varianten sind derzeit H.264 Video mit MP3-Audio oder WebM, sicherheitshalber sollten jedoch mehrere Videoformate angeboten werden, um in allen Browsern ein Video angezeigt zu bekommen.

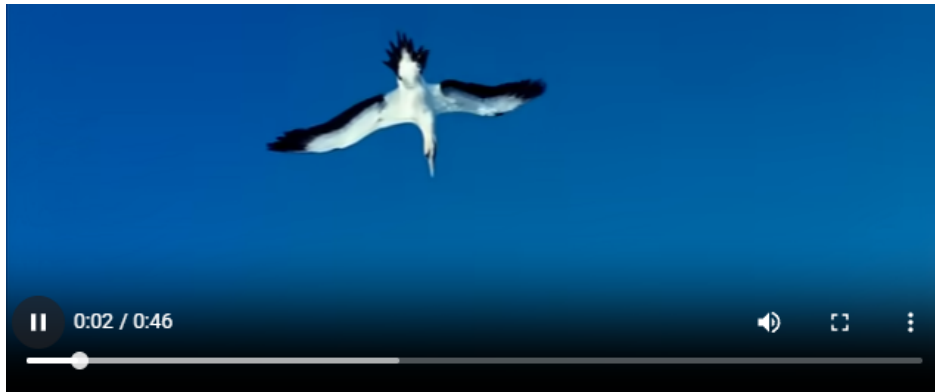


Abbildung 12.7: Chrome stellt das Video mit seinen eigenen Kontrollelementen dar.

12.3.2 Video Markup

Die einfachste Art und Weise, einen Videoclip mit dem `<video>`-Element wiederzugeben, ist die Angabe der Datei im `src`-Attribut, zusammen mit den Dimensionen des Videos (durch `width` und `height`). Durch die Angabe des Attributs `controls` lassen sich simple Kontrollelemente anzeigen. Text innerhalb des `<video>`-Elements wird nur dann angezeigt, wenn der Browser dieses Element nicht unterstützt.

```
1 <video src="../../../assets/oceans-clip_vp9_opus.webm" width="640" height="264" controls>
  Ihr Browser unterstützt Video leider nicht!</video>
```

Um nun aber sicher zu gehen, dass jeder Browser das Video wiedergeben kann, sollten beide verwendbaren Containerformate eingebunden werden. Da es aber nur ein `src`-Attribut beim `<video>`-Element gibt, muss hierbei anders vorgegangen werden. Das `<video>`-Element kann eine beliebige Anzahl von `<source>`-Elementen enthalten. Jedes dieser Elemente verweist in seinem eigenen `src`-Attribut auf die jeweilige Videodatei. Die Reihenfolge, in der die einzelnen Formate angegeben werden, ist freigestellt. Der Browser geht diese von oben nach unten durch und spielt die erste Datei, die er interpretieren kann, ab. Darüber hinaus müssen durch die Angabe des Attributs `type` der Container angegeben werden. Der folgende Code stellt eine Variante mit sechs Kombinationen aus der Tabelle dar, Abbildung 12.7 zeigt die Darstellung in Chrome.

```
1 <video width="640" height="264" controls>
2   <source src="../../../assets/oceans-clip_av1_opus.webm" type="video/webm">
3   <source src="../../../assets/oceans-clip_av1_opus.mp4" type="video/mp4">
4   <source src="../../../assets/oceans-clip_vp9_opus.webm" type="video/webm">
5   <source src="../../../assets/oceans-clip_h265_aac.mp4" type="video/mp4">
6   <source src="../../../assets/oceans-clip_h264_mp3.mp4" type="video/mp4">
7   <source src="../../../assets/oceans-clip_vp8_vorbis.webm" type="video/webm">
8 </video>
```

12.3.3 Kontrollelemente

Anstatt die vom Browser definierten *Kontrollelemente*, die durch die Angabe des HTML-Attributs `controls` zur Verfügung gestellt werden zu verwenden, kann das Video auch

über JavaScript gesteuert werden, d. h. es können eigene Kontrollelemente erstellt werden.

Im folgenden Beispiel werden ein Button für „Play“ und „Pause“ sowie zwei Buttons zur Veränderung der Lautstärke (+/−) erstellt. Diese werden über JavaScript angesprochen und steuern das Video. Das HTML-Markup für die Buttons sieht wie folgt aus, das Video ist auf die oben gezeigte Weise in dreifacher Ausführung eingebunden, das Video erhält weiters die ID `vid`.

```
1 <button type="button" id="playpause">Play</button>
2 <button type="button" id="volup">+</button>
3 <button type="button" id="voldown">-</button>
```

Nun werden in JavaScript die Buttons und das Video abgefragt:

```
1 const vid = document.getElementById("vid");
2 const playPause = document.getElementById("playpause");
3 const volUp = document.getElementById("volup");
4 const volDown = document.getElementById("voldown");
```

Nun wird auf jeden der drei Buttons ein `click`-Listener gelegt. Der Listener für den „Play/Pause“-Button fragt ab, ob das Video pausiert ist. Ist es pausiert, wird es abgespielt und umgekehrt. Je nach Status wird auch der Text des Buttons von „Play“ auf „Pause“ und umgekehrt geändert.

Bei den Buttons für die Lautstärkenregelung wird geprüft, ob die Lautstärke zwischen kleiner 1 und größer 0 ist, denn nur diese Werte sind möglich. Danach wird die Lautstärke um 0,1 nach oben oder unten verändert. Der Code dazu lautet wie folgt, Abbildung 12.8 stellt das Video mit den (einfachen) Kontrollelementen dar.

```
1 playPause.addEventListener("click", () => {
2   vid.paused ? vid.play() : vid.pause();
3   playPause.textContent = vid.paused ? "Play" : "Pause";
4 });
5 volUp.addEventListener("click", () => {
6   if (vid.volume < 1) {
7     vid.volume += 0.1;
8   }
9 });
10 volDown.addEventListener("click", () => {
11   if (vid.volume > 0) {
12     vid.volume -= 0.1;
13   }
14 });
```

Autoplay Policies und User Interaction

Moderne Browser verhindern das automatische Abspielen von Video- und auch Audiodateien mit Ton, um Nutzer*innen nicht zu belästigen. Ein Aufruf von `vid.play()` direkt beim Laden der Seite würde daher scheitern und in der Konsole einen Fehler (*NotAllowedError*) werfen.

Das hier gezeigte Beispiel funktioniert jedoch problemlos, da der Befehl `vid.play()` innerhalb eines **Click-Event-Listeners** ausgeführt wird. Der Browser erkennt, dass der Befehl als direkte Reaktion auf eine bewusste *User Interaction* (den Klick auf den Button) erfolgt, und erlaubt daraufhin die Wiedergabe. Möchte man ein Video zwingend automatisch ohne Interaktion starten (z. B. als Hintergrundvideo), muss es stummgeschaltet sein (Attribut `muted`).

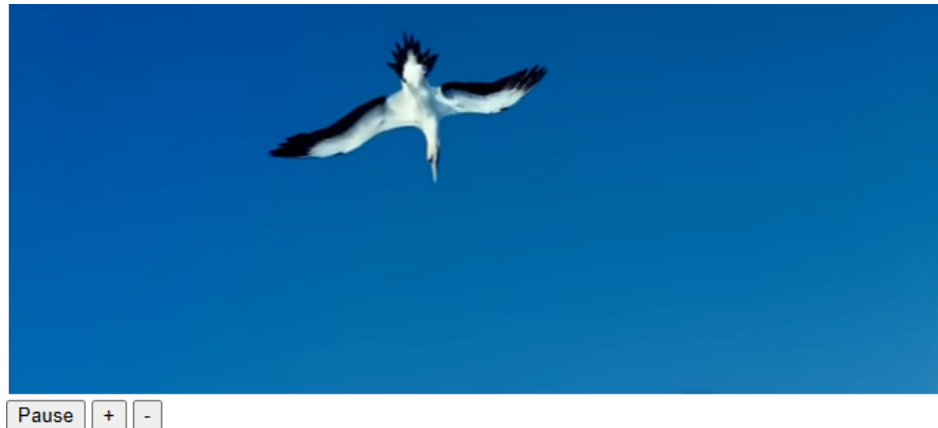


Abbildung 12.8: Das Video mit den selbst erstellten Kontrollelementen.

12.4 Audio

Das Abspielen und Steuern von reinen Audiodateien gestaltet sich sehr ähnlich dem von Video. Eingebunden werden Audiodateien über das `<audio>`-Element, geschlossen durch `</audio>`. Auch hier gibt es verschiedene Formate, die verwendet werden können.

12.4.1 Container und Codecs

Für den reinen Audiobereich sind dieselben Formate wie in Kombination mit Video verfügbar, darüber hinaus gibt es allerdings noch andere, die eingesetzt werden können. Wiederum gibt es eine Reihe von verschiedenen Container-Formaten:

- **MP3:** Ein Container für den gleichnamigen Codec, Endung lautet `.mp3`.
- **MP4:** Kurz für MPEG-4. Bekannte Dateierendungen sind `.mp4` oder `.m4a`.
- **Ogg:** Ein Open-Source-Standard, die Dateierendung lautet `.ogg` oder `.oga`.
- **WebM:** Ein Open-Source-Format von Google, die Dateierendung lautet `.webm`.
- **WAVE:** Ein populäres Format, das meist PCM-Daten enthält, Endung `.wav`.
- **FLAC:** Ein Containerformat für den gleichnamigen Audio-Codec, mit Endung `.flac`.

In diesen Containern können sich die folgenden Codecs bzw. Formate wiederfinden:

- **MP3:** MPEG-1 Audio Layer 3 ist der wohl am weitesten verbreitete verlustbehaftete Audio-Codec.
- **AAC:** Advanced Audio Coding ist der Standardcodec für Audio auf Apple-Systemen. Der Codec ist ebenfalls lizenzpflichtig.
- **Vorbis:** Ein lizenzfreier Open-Source-Codec, meist im Zusammenhang mit dem Ogg-Containerformat verwendet.
- **FLAC:** Ein freier, verlustfreier Audio-Codec.
- **PCM:** Unkomprimierte Audiodaten im Pulse-Code-Modulationsverfahren.

Tabelle 12.2 stellt dar, welche Kombinationen möglich sind und wie sie in den einzelnen Browsern unterstützt werden. AAC in Firefox ist nur möglich, wenn die Codecs im

Tabelle 12.2: Audio-Containerformate mit Codecs und Browserunterstützung. C144: Chrome 144, FF147: Firefox 147, E144: Edge 144, O125: Opera 125, S26.2: Safari 26.2.

<i>Format</i>	<i>C144</i>	<i>FF147</i>	<i>E144</i>	<i>O125</i>	<i>S26.2</i>
WAVE: PCM	•	•	•	•	•
WebM: Vorbis	•	•	•	•	•
Ogg: Vorbis	•	•	•	•	•
MP3	•	•	•	•	•
MP4: MP3	•	•	•	•	•
MP4: AAC	•	◦	•	•	•
FLAC	•	•	•	•	•
Ogg: FLAC	•	•	•	•	•



Abbildung 12.9: Die Audio-Kontrollelemente in Google Chrome.

System bereits installiert sind. Alle anderen Kombinationen funktionieren in modernen Browsern.

12.4.2 Audio Markup

Wie beim Video-Element kann im einfachsten Fall einfach eine Datei als Quelle im `src`-Attribut des `<audio>`-Elements angegeben werden. Das Attribut `controls` ist nötig, um überhaupt die Kontrollelemente zur Wiedergabe zu sehen.

```
1 <audio src="../../assets/miaow-bubble.mp3" controls>Ihr Browser unterstützt Audio
   leider nicht!</audio>
```

Wiederum sollte auf Nummer sicher gegangen werden und mit Hilfe der `<source>`-Elemente innerhalb von `<audio>` mehrere Formate angeboten werden, damit eine durchgängige Browserunterstützung gegeben ist. Abbildung 12.9 zeigt die reinen Audio-Kontrollelemente mit einer bzw. auch mehreren Quelldateien.

```
1 <audio controls>
2   <source src="../../assets/miaow-bubble.flac" type="audio/flac">
3   <source src="../../assets/miaow-bubble.mp3" type="audio/mpeg">
4   <source src="../../assets/miaow-bubble.ogg" type="audio/ogg">
5 </audio>
```

12.4.3 Kontrollelemente

Die API für Audio ist identisch zur der von Video. Auch hier können mit eigenen Elementen wie Buttons und dem entsprechenden JavaScript-Code Audiodateien abgespielt, gestoppt und etwa die Lautstärke verändert werden.

Die Methoden `play()` und `pause()` erlauben das Abspielen bzw. Pausieren, die Eigenschaft `volume` enthält die Lautstärke. Abschnitt 12.3.3 kann als Referenz für die Implementierung herangezogen werden.

12.5 Geolocation

Geolocation ist die Möglichkeit abzufragen, wo in der Welt sich die Benutzer*innen, die gerade auf der Website surfen, befinden. Die Art und Weise, wie die Position dabei bestimmt wird, hängt viel auch vom verwendeten Endgerät ab. Smartphones und Tablets bieten zumeist integrierte GPS-Module, wodurch eine Positionsbestimmung (im Freien) sehr genau möglich ist. Aber auch ohne dieses Hilfsmittel kann die Position ermittelt werden. Dabei werden etwa die IP-Adresse, MAC-Adresse oder verfügbare Drahtlosnetzwerke herangezogen.

Mit Hilfe der Geolocation API [94] kann diese Position ermittelt und für eigene Webanwendungen verwendet werden. Wichtig dabei ist, dass die Abfrage der Position niemals ohne Zustimmung der Benutzer*innen erfolgen kann und darf. Daher zeigt der Browser Anwender*innen zunächst eine Abfrage, ob diese ihre Positionsdaten mit der aktuellen Website teilen möchten. Stimmt der*die Benutzer*in nicht zu, steht diese Information nicht zur Verfügung.

12.5.1 Abfragen der aktuellen Position

Die Abfrage der Position geschieht über das `navigator`-Objekt bzw. dessen Unterobjekt `geolocation`, das die Methode `getCurrentPosition()` zur Verfügung stellt. Diese erwartet als Parameter eine sogenannte Callback-Funktion, also eine Funktion, die automatisch aufgerufen wird, sobald die Ergebnisse der Positionsabfrage vorliegen. Dies kann entweder eine selbst geschriebene, eigene Funktion sein, oder auch eine anonyme Funktion. Wichtig ist nur, dass immer der Parameter `position` vorhanden sein muss, denn er enthält die aktuelle Position.

Ein Aufruf mit einer eigenen Funktion namens `showMap()` könnte also so aussehen:

```
1 navigator.geolocation.getCurrentPosition(showMap);
2
3 function showMap(position) {
4   const lat = position.coords.latitude;
5   const long = position.coords.longitude;
6   // Zeige eine Karte
7 }
```

Das Gleiche kann ebenso mit einer anonymen Funktion so erreicht werden:

```
1 navigator.geolocation.getCurrentPosition(function(position) {
2   const lat = position.coords.latitude;
3   const long = position.coords.longitude;
4   // Zeige eine Karte
5 });
```

Wie ersichtlich ist, muss die Verarbeitung der Position innerhalb der Callback-Funktion (egal ob normal oder anonym) erfolgen, denn `getCurrentPosition()` liefert nicht wie eine normale Methode einen Wert zurück, sondern ruft die Callback-Funktion auf.

12.5.2 Anzeigen einer Karte

Um eine Karte mit der aktuellen Position anzeigen zu können, wird *Leaflet*, eine JavaScript-Bibliothek zum Anzeigen von Karten verwendet. Die Kartenteile kommen von OpenStreetMap. Alternativ kann auch die Google Maps JavaScript API⁶ verwendet werden. Diese benötigt jedoch einen API-Schlüssel, welcher nur gegen Hinterlegen einer Kreditkarte erhältlich ist und bei zu viele Aufrufen Kosten erzeugt.

Leaflet wird am einfachsten online, von einem CDN (Content Delivery Network) im `<head>` mit `defer` eingebunden. Neben der JavaScript-Datei wird auch noch eine CSS-Datei benötigt, die für die Formatierung der Kartenelemente sorgt:

```
1 <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"
  integrity="sha256-p4NxAoJBhIIN+hmNhrzRCf9tD/miZyoHS5obTRR9BMY=" crossorigin=""/>
2 <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js" integrity="sha256-20
  nQCchB9co0qIjJZRGuk2/Z9VM+kNiyxNV1lvTlZBo=" crossorigin="" defer></script>
```

Um nun die Karte anzuzeigen, wird im `<body>` ein `<div>`-Element mit der id `map` angelegt:

```
1 <div id="map"></div>
```

Der JavaScript-Code zum Anzeigen der Karte wird in eine Datei `map.js` ausgelagert, die nach Leaflet ebenfalls mit `defer` eingebunden wird. Die Funktion `initialize()` erzeugt die Karte im `<div>` mit der ID `map` und fügt einen Tile-Layer (die sichtbaren Kacheln der Karte) hinzu:

```
1 function initialize() {
2   const map = L.map("map").setView([47.61, 13.783], 8);
3
4   // Add a tile layer: we use OpenStreetMap here
5   L.tileLayer("https://tile.openstreetmap.org/{z}/{x}/{y}.png", {
6     maxZoom: 19,
7     attribution: '&copy; <a href="http://www.openstreetmap.org/copyright">
      OpenStreetMap</a>'
8   }).addTo(map);
9
10  // Geolocation Code
11 }
```

Mit `L.map()` wird die Karte erzeugt, die Funktion `setView()` erhält das Zentrum der Karte und eine initiale Zoomstufe. Der Tile-Layer stammt von OpenStreetMap und wird ebenfalls der Karte hinzugefügt.

Nun fehlt noch die aktuelle Position, auf der die Karte zentriert sein soll. Hier kommt Geolocation zum Einsatz. Zunächst wird überprüft, ob Geolocation verfügbar ist, dann die Position wie oben gezeigt ermittelt und schließlich auf der Karte mit der Methode `setView()` gesetzt. Weiters wird ein Text ins einen Marker geschrieben und der Zoomlevel angepasst.

Sollte der*die Nutzer*in die Anfrage zur Positionsbestimmung ablehnen oder der Browser Geolocation nicht unterstützen, wird die Funktion `handleLocationError` aufgerufen. Diese zeigt im Marker die entsprechende Fehlermeldung an.

```
1 function initialize() {
2   // Karten Initialisierung
```

⁶<https://developers.google.com/maps/documentation/javascript/>

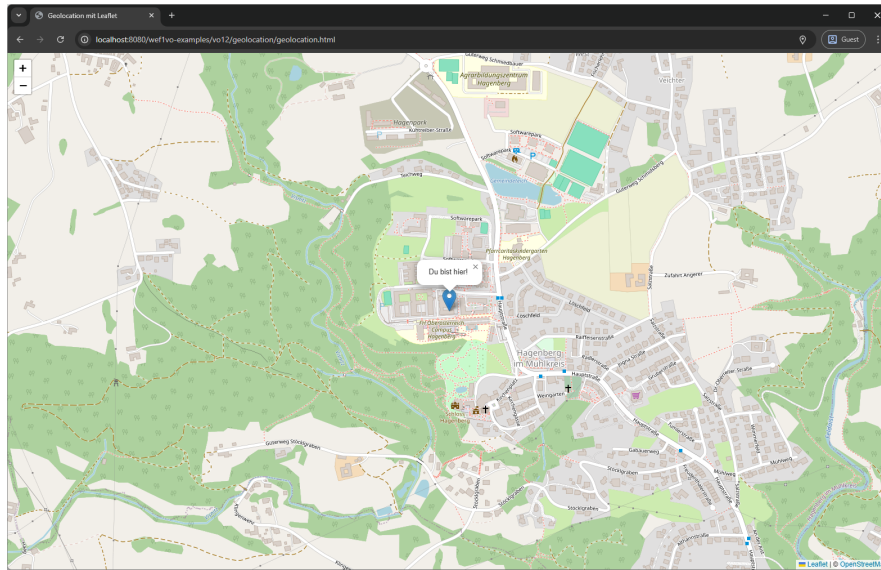


Abbildung 12.10: Der Browser zeigt (in der FH) eine Vollbildkarte mit Hagenberg im Zentrum an.

```

3
4  if (navigator.geolocation) {
5      navigator.geolocation.getCurrentPosition(position => {
6          const lat = position.coords.latitude;
7          const lng = position.coords.longitude;
8          map.setView([lat, lng], 16);
9
10         L.marker([lat, lng]).addTo(map)
11             .bindPopup("Du bist hier!")
12             .openPopup();
13     }, () => {
14         handleLocationError(true, map);
15     });
16 } else {
17     // Browser unterstützt keine Geolocation
18     handleLocationError(false, map);
19 }
20 }

```

Dieses Beispiel erzeugt eine Vollbildkarte mit der aktuellen Position und einer Nachricht im Zentrum. Das Ergebnis ist in Abbildung 12.10 ersichtlich.

12.6 Local Storage

Die Fähigkeit, Daten lokal speichern zu können, war immer ein typischer Vorteil von nativen Client-Applikationen gegenüber Web-Applikationen. Für Websites gab es nur Cookies zum Speichern von lokalen Daten, doch diese waren nie für diesen Zweck gedacht. Zunächst können sie nur 4 KB an Daten speichern, weiters werden Sie zwingend bei jedem HTTP-Request übertragen, was nicht sehr effizient ist.

Hier schafft die HTML *Web Storage API* [54, Abschnitt 12] Abhilfe. Sie stellt einen lokalen Speichermechanismus zur Verfügung, der die Speicherung beliebiger Schlüssel-Wert-Paare erlaubt. Diese Werte bleiben selbst dann gespeichert, wenn von der aktuellen Seite weg navigiert oder das Browserfenster geschlossen wird.

Der Zugriff auf diesen lokalen Speicher ist über das `localStorage`-Objekt möglich. Es ist ein Unterobjekt von `window`, also immer global verfügbar. Es besitzt folgende wichtige Methoden:

- `setItem(key, value)`: Speichert ein neues Schlüssel/Wert Paar mit dem Schlüssel `key` und dem Wert `value`.
- `getItem(key)`: Liefert den Wert des Local Storage Eintrags mit dem Schlüssel `key` zurück.
- `removeItem(key)`: Löscht den Eintrag mit dem Schlüssel `key` aus dem Local Storage.

Im folgenden Beispiel soll eine simple Post-It Applikation erzeugt werden. Als Post-It dient ein `<section>`-Element mit 200×200 Pixeln. Das Element enthält das Attribut `contenteditable` mit dem Wert `true`. Dieses HTML-Attribut ermöglicht das Editieren von Text direkt an Ort und Stelle. Der einzige Nachteil: Wird die Seite neu geladen, sind alle Änderungen wieder verschwunden. Hier schafft *Local Storage* Abhilfe. Zunächst aber der HTML-Code für das Post-It (die Formatierung erfolgt entsprechend in CSS). Das Element erhält weiters eine `id` zum leichteren Auffinden in JavaScript.

```
1 <section id="sticky" contenteditable="true"></section>
```

Im JavaScript-Code wird zunächst ein Standardtext definiert, dann das Post-It-Element gesucht und danach wird mittels der `getItem()`-Methode abgefragt, ob im Local Storage bereits ein Inhalt vorhanden ist. Dies wäre der Fall, falls mit dem Post-It bereits interagiert wurde. In diesem Fall würde der bestehende Inhalt als HTML-Inhalt eingefügt. Beim ersten Aufruf der Seite wird diese Abfrage fehlschlagen und es wird ein neuer Standardinhalt in das `<section>`-Element geschrieben. Dies wird mit der Eigenschaft `innerHTML` gelöst. Diese ersetzt den Inhalt eines HTML-Elements durch den gegebenen String. Kommen darin andere HTML-Elemente vor, so werden diese ebenfalls entsprechend verschachtelt.

Vorsicht: Wenngleich die Eigenschaft `innerHTML` praktisch ist, so kann sie ein Sicherheitsrisiko für sogenanntes Cross-Site-Scripting⁷ darstellen.

Sodann wird auf das Element ein `blur`-Event-Handler gelegt. Der `blur`-Event wird ausgelöst, wenn neben das `<section>`-Element geklickt wird. Dies passiert immer dann, wenn Benutzer*innen aufhören wollen, das Post-It zu editieren, da sie zum Schreiben einfach hinein klicken müssen. Sind sie fertig, wird überprüft, ob der Inhalt leer ist. Dies wird mit der Eigenschaft `textContent` überprüft. Diese funktioniert so ähnlich wie `innerHTML`, beschränkt sich jedoch rein auf Text. Enthaltene HTML-Elemente werden ignoriert. Dies ist für diese Abfrage vorteilhaft, da die Browser leere Elemente nicht komplett leer zurücklassen. So bleibt meist etwa ein `<br>`-Element zurück, wodurch bei der Verwendung von `innerHTML` das Element nicht mehr als leer erkannt würde.

Ist das Element leer (d. h. enthält es keinerlei Text) oder nur den Standardtext, wird der Local Storage-Eintrag gelöscht, da der*die Benutzer*in offensichtlich Inhalt

⁷<https://developer.mozilla.org/en-US/docs/Web/API/Element/innerHTML>

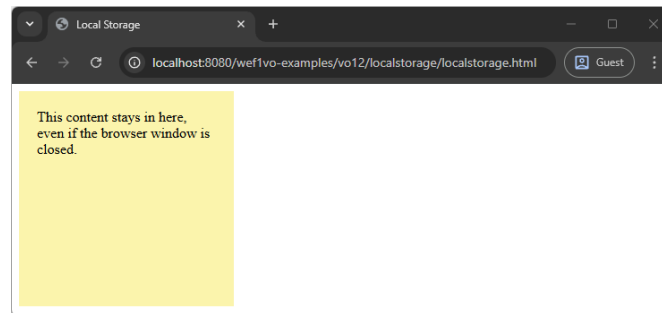


Abbildung 12.11: Der Text im Post-It kann einfach durch Anklicken geändert werden. Er bleibt lokal im Browser gespeichert und ist beim nächsten Aufruf der Seite wieder vorhanden.

entfernt hat. Danach wird der Standardtext wieder ins Element eingefügt, damit das Post-It nicht leer bleibt. Ist aber nach dem `blur`-Event noch Text vorhanden, wird er in den Local Storage geschrieben. Hier kommt wiederum der Inhalt mittels `innerHTML` zum Zug, da so auch mögliche Zeilenumbrüche mitgespeichert werden.

Wird nun die Seite neu geladen, oder nach dem Schließen des Browserfensters wieder geöffnet, steht der Inhalt nach wie vor im Post-It.

Im Folgenden der Code, das Ergebnis ist in Abbildung 12.11 zu sehen:

```
1 const defaultText = "Schreib auf mich!";
2 const note = document.getElementById("sticky");
3 const storedValue = localStorage.getItem("content");
4
5 if (storedValue) {
6   note.innerHTML = storedValue;
7 } else {
8   note.innerHTML = defaultText;
9 }
10
11 note.addEventListener("blur", (event) => {
12   const plainText = event.target.textContent;
13   if (plainText.trim() === "" || plainText === defaultText) {
14     localStorage.removeItem("content");
15     event.target.innerHTML = defaultText;
16   } else {
17     localStorage.setItem("content", event.target.innerHTML);
18   }
19 });
```

12.7 Drag & Drop

Um *Drag & Drop* zu realisieren, also ein Element mit der Maus anzuklicken und mit gedrückter Maustaste woanders hinzubewegen, musste lange auf JavaScript-Bibliotheken wie jQuery zurückgegriffen werden. Dank der HTML Drag & Drop API [54, Abschnitt 6.11] ist dies auch komplett ohne Bibliothek möglich. Dafür sind grundsätzlich vier Schritte notwendig:

1. Elemente ziehbar machen.
2. Mit dem Ziehen von Elementen starten.
3. Elemente zum Fallenlassen auswählen.
4. Operationen beim Fallenlassen ausführen.

Diese vier Schritte werden im Folgenden an einem Beispiel erklärt, in dem ein Bild zwischen drei `<div>`-Elementen verschoben werden kann.

12.7.1 Elemente ziehbar machen

Zunächst müssen Elemente bestimmt werden, mit denen Drag & Drop überhaupt möglich ist. Standardmäßig gehören dazu Bilder oder auch Links, mit Hilfe des Attributs `draggable` kann jedoch für jedes Element festgelegt werden, ob es ziehbar ist oder nicht. Wird der Wert des Attributs auf `true` gesetzt, ist dies der Fall, mit `false` kann das Ziehen unterbunden werden.

Im folgenden Beispiel existieren drei `<div>`-Elemente, die in weiterer Folge als Ziel für Drag & Drop-Operationen dienen werden. Im ersten dieser Elemente liegt ein Bild, das mit Hilfe des `draggable`-Attributs explizit als ziehbar gekennzeichnet wurde. Es besitzt weiters die ID `dragImage` mit dem es während der Operation identifiziert werden kann.

```
1 <div class="dropzone">
2   
3 </div>
4
5 <div class="dropzone"></div>
6
7 <div class="dropzone"></div>
```

12.7.2 Mit dem Ziehen von Elementen starten

Nachdem nun die ziehbaren Elemente festgelegt wurden, muss definiert werden, was beim Start eines Ziehvorgangs passiert. Dazu wird auf den `dragstart`-Event reagiert, der genau auf diesen Vorgang hin ausgelöst wird. Dazu wird (das Script liegt am Ende des `<body>`-Elements) zunächst das ziehbare Element ausgewählt und auf diesem ein `dragstart`-Listener angebracht. Innerhalb dieses Listeners werden dem Event-Objekt Daten mitgegeben, die während des Drag & Drop Vorgangs relevant sind. Diese Daten werden im Objekt `dataTransfer` gespeichert, das ein Unterobjekt des `event`-Objekts ist. Mit dessen Methode `setData()` können Daten hinzugefügt werden. Der erste Parameter ist der Typ (etwa einfacher Text), der zweite Parameter sind die Daten selbst.

Im Beispiel wird beim Start des Ziehens die ID des gezogenen Elements (`dragImage`) beim `event`-Objekt (enthält alle Daten über das Element, bei dem der Event aufgetreten ist) gespeichert. Sie kann über die Eigenschaft `target` ausgelesen werden.

```
1 const dragImage = document.getElementById("dragImage");
2
3 dragImage.addEventListener("dragstart", event => {
4   event.dataTransfer.setData("text", event.currentTarget.id);
5 });
```


12.7.3 Elemente zum Fallenlassen auswählen

Im nächsten Schritt muss festgelegt werden, auf welchen Elementen das gezogene Element überhaupt fallengelassen werden kann. Dafür steht der **dragover**-Event zur Verfügung. Um ein Fallenlassen auf ein Element zu erlauben, muss ein Listener auf diesem Event angebracht werden und dort mittels **event.preventDefault()** das Standardverhalten dieses Events unterbunden werden. Dieses verbietet nämlich das Fallenlassen, wird das Standardverhalten also deaktiviert, ist ein Fallenlassen möglich.

Im Beispiel werden zunächst die drei **<div>**-Elemente mit der Klasse **dropzone** gesucht. In einer **ForEach**-Schleife wird über die gefundenen Elemente iteriert und bei jedem ein Event-Listener mit der entsprechenden Anweisung registriert.

```
1 const dropZones = document.querySelectorAll(".dropzone");
2
3 dropZones.forEach(zone => {
4   zone.addEventListener("dragover", event => {
5     event.preventDefault();
6   });
7   ...
8 });
```

12.7.4 Operationen beim Fallenlassen ausführen

Wird schließlich über einem Element, bei dem Fallenlassen erlaubt wurde, die Maustaste losgelassen, wird der **drop**-Event ausgelöst. Im entsprechenden Event-Listener kann nun also festgelegt werden, was in diesem Moment passiert. In der Regel werden hier die Daten aus dem **dataTransfer**-Objekt entnommen und verarbeitet.

Im Beispiel passiert genau dies. Zunächst wird die ID des gezogenen Elements mit der Methode **getData()** entnommen und gespeichert. Dann wird im DOM das Element mit dieser ID (das Bild) gesucht und ebenfalls mittels DOM-Methode in das Element gehängt, auf dem der **drop**-Event aufgetreten ist. **event.preventDefault()** sollte auch hier aufgerufen werden, um zu vermeiden, dass beim gezogenen Objekt das Standardverhalten ausgelöst wird. Wäre das Element etwa ein Link, würde diesem gefolgt werden (was meist nicht gewünscht ist).

```
1 dropZones.forEach(zone => {
2   ...
3   zone.addEventListener("drop", event => {
4     event.preventDefault();
5     const data = event.dataTransfer.getData("text");
6     const draggedElement = document.getElementById(data);
7     event.currentTarget.appendChild(draggedElement);
8   });
9 });
```

Abbildung 12.12 stellt den Drag & Drop-Vorgang im Beispiel schematisch dar.

12.7.5 Weitere Events und Anmerkungen

Neben den gezeigten Events werden noch eine Reihe weiterer beim Drag & Drop-Vorgang ausgelöst. Dazu zählen etwa **dragenter** (wird beim Ziehen über ein Element



Abbildung 12.12: Das HTML5-Logo wird vom ersten Element zum zweiten gezogen und kann dort im Anschluss abgelegt werden.

ausgelöst), `dragleave` (wird beim Verlassen eines Elements ausgelöst) und `dragend` (wird beim Beenden des gesamten Vorgangs ausgelöst).

Ebenso kann im Prozess bestimmt werden, wie sich der Mauscursor verändert, was hilfreich ist um anzudeuten, wo Elemente fallengelassen werden können und wo nicht.

Weiters sei angemerkt, dass auch Sonderfälle berücksichtigt werden sollten, etwa was passiert, wenn die Maus das Browserfenster verlässt. Um große Datenmengen mit Drag & Drop zu verarbeiten sind zusätzliche APIs wie etwa die File API [93] notwendig.

12.8 Weitere Technologien

Die hier vorgestellten Technologien sind natürlich längst nicht alles, was es an Web APIs gibt. Folgende Dinge würden den Rahmen dieser Vorlesung sprengen, sie seien aber der Vollständigkeit halber erwähnt.

12.8.1 Web Sockets

HTTP ist ein zustandsloses Protokoll. Nach der Übermittlung von den Daten weiß das Protokoll nichts mehr von dieser Übermittlung. Permanent offene Verbindungen sind somit nicht möglich. *Web Sockets* [56] schaffen hierbei Abhilfe. Mit ihnen ist es möglich, Dinge quasi in Echtzeit zu erledigen, denn sie erhalten eine beständige Verbindung zum Server aufrecht. Ein Beispiel wäre ein webbasierter Chat. Beim Laden der Seite verbindet sich der Chat zum Chatserver. Solange die Verbindung offen ist, können Nachrichten verschickt und empfangen werden.

12.8.2 Indexed Database

Die vorgestellte Local Storage Implementierung ist eine schnelle Art und Weise, Daten lokal zu speichern. Komplexere Datenmodelle sind jedoch kaum damit abzubilden. Mit Hilfe der *Indexed Database API* [95] wurde eine Schnittstelle geschaffen, mit der eine indizierte Datenbank verwaltet werden kann. Derzeit implementieren alle wichtigen Browser diese Spezifikation komplett.

12.8.3 Web Workers

JavaScript ist eine sogenannte „single-threaded language“, d. h. sie kann immer nur einen Codestrang ausführen. Mehrere (pseudo) parallele Abarbeitungen sind nicht möglich. *Web Workers* [54, Abschnitt 10] heben diese Einschränkung auf. Mit ihnen ist es möglich, im Hintergrund parallel Code auszuführen, ohne dass die Website im Vordergrund dadurch beeinträchtigt wird („hängen bleibt“).

12.8.4 Service Workers

Um Webapplikationen zu realisieren, die auch offline funktionieren, wurde das Konzept der *Service Workers* [102] erdacht. Sie agieren als Netzwerk Proxy und ermöglichen volle Kontrolle, wie Kommunikation über das Netzwerk gehandhabt wird. Eine E-Mail-Applikation kann also etwa auch offline funktionieren und erst dann Daten senden, wenn wieder eine Verbindung besteht.

12.9 Weiterführende Literatur

Literatur zu diesem Thema ist derzeit noch spärlich verfügbar. Aufgrund der aktuellen Thematik ist sie auch meist bereits veraltet. Es gibt dennoch einige Werke, die empfehlenswert sind – für alle dort nicht abgedeckten Dinge muss auf die Dokumentation des jeweiligen Entwurfs vom W3C bzw. der WHATWG zurück gegriffen werden.

Eine Übersicht zu den gezeigten (und einigen weiteren) APIs bietet [58], etwas weiter ins Detail bei einigen APIs geht [1].

Die meisten Bücher zu Spezialthemen wie Canvas oder Geolocation sind schon über 10 Jahre alt und werden ohnehin durch generelle HTML- oder JavaScript-Literatur abgedeckt. Hier sei auf Codebeispiele und Lernplattformen im Web verwiesen.

Quellenverzeichnis

- [1] Philip Ackermann. *JavaScript. Das umfassende Handbuch*. 4. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. Nov. 2025 (siehe S. 8-27, 9-20, 10-22, 11-23, 12-23).
- [2] Allience for Open Media. *AV1 Image File Format (AVIF)*. AOM Final Deliverable. Version v1.1.0. 15. Apr. 2022. URL: <https://aomediacodec.github.io/av1-avif/latest-approved.html> (siehe S. 2-21).
- [3] American National Standards Institute. *Coded Character Sets – 7-Bit American Standard Code for Information Interchange (7-Bit ASCII)*. American National Standard for Information Systems. Version ANSI X3.4-1986. 26. März 1986. URL: <https://www.unicode.org/L2/L2006/06388-review-incits4.pdf> (siehe S. 2-3).
- [4] Stefan Baumgartner. *Front-End Tooling with Gulp, Bower, and Yeoman*. Shelter Island, USA: Manning, 16. Dez. 2016 (siehe S. 11-23).
- [5] Andy Bell. *A (more) Modern CSS Reset*. 18. Sep. 2023. URL: <https://piccalil.li/blog/a-more-modern-css-reset/> (besucht am 21. 11. 2025) (siehe S. 5-26).
- [6] Tim Berners-Lee, Roy Fielding und Larry Masinter. *Uniform Resource Identifier (URI): Generic Syntax*. RFC 3986. IETF, Jan. 2005, S. 1–61. URL: <https://data.ietf.org/doc/rfc3986/> (siehe S. 1-3).
- [7] Tim Berners-Lee, Larry Masinter und Mark McCahill. *Uniform Resource Locators (URL)*. RFC 1738. IETF, Dez. 1994, S. 1–25. URL: <https://data.ietf.org/doc/rfc1738/> (siehe S. 1-3).
- [8] Mathias Bynens. *Valid JavaScript variable names in ES2015*. 4. März 2015. URL: <https://mathiasbynens.be/notes/javascript-identifiers-es6> (besucht am 12. 12. 2025) (siehe S. 8-9).
- [9] Andy Clarke. *CSS Specificity Wars*. 2018. URL: https://stuffandnonsense.co.uk/archives/css_specificity_wars.html (besucht am 10. 11. 2025) (siehe S. 3-24).
- [10] Chris Coyier. *All About Floats*. 25. März 2021. URL: <https://css-tricks.com/all-about-floats/> (besucht am 16. 11. 2025) (siehe S. 4-17).
- [11] Chris Coyier. *CSS Custom Properties Guide*. 17. Okt. 2024. URL: <https://css-tricks.com/a-complete-guide-to-custom-properties/> (besucht am 21. 11. 2025) (siehe S. 5-24, 5-33).
- [12] Chris Coyier. *CSS Flexbox Layout Guide*. 12. Aug. 2024. URL: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/> (besucht am 21. 11. 2025) (siehe S. 5-6, 5-33).

- [13] csstools. *normalize.css*. Version 12.1.1. 29. Dez. 2023. URL: <https://github.com/csstools/normalize.css> (siehe S. 5-27).
- [14] European Computer Manufacturers Association. *ECMAScript[®] 2025 Language Specification*. ECMA-262. Version 16.0. Juni 2025. URL: <https://ecma-international.org/publications-and-standards/standards/ecma-262/> (siehe S. 8-2, 8-3, 8-27, 9-7, 9-11).
- [15] European Computer Manufacturers Association. *The JSON Data Interchange Syntax*. ECMA-404. Version 2.0. Dez. 2017. URL: <https://ecma-international.org/publications-and-standards/standards/ecma-404/> (siehe S. 7-17, 7-31).
- [16] Ashley Firth. *Practical Web Accessibility. A Comprehensive Guide to Digital Inclusion*. 2. Aufl. London, Vereinigtes Königreich: Apress, 18. Apr. 2024 (siehe S. 6-22).
- [17] David Flanagan. *JavaScript. The Definitive Guide*. 7. Aufl. Sebastopol, USA: O'Reilly, 9. Juni 2020 (siehe S. 8-27).
- [18] Paul Fuchs. *HTML5 und CSS3 für Einsteiger. Der leichte Weg zur eigenen Webseite*. Landshut, Deutschland: BMU Verlag, 21. Juni 2019 (siehe S. 1-16, 2-34, 3-41, 4-31).
- [19] Nicolas Gallagher. *Normalize.css. A modern, HTML5-ready alternative to CSS resets*. 5. Nov. 2018. URL: <https://necolas.github.io/normalize.css/> (besucht am 21. 11. 2025) (siehe S. 5-26, 5-27).
- [20] Dimitris Georgakas. *A11Y Unraveled. Become a Web Accessibility Ninja*. New York, USA: Apress, 24. März 2023 (siehe S. 6-22).
- [21] Google Developers. *WebP. An image format for the Web*. 7. Aug. 2025. URL: <https://developers.google.com/speed/webp/> (besucht am 03. 11. 2025) (siehe S. 2-21).
- [22] Geoff Graham. *Media Queries for Standard Devices*. 12. Aug. 2024. URL: <https://css-tricks.com/snippets/css/media-queries-for-standard-devices/> (besucht am 16. 11. 2025) (siehe S. 4-26).
- [23] Elliotte Rusty Harold. *Effective XML. 50 Specific Ways to Improve Your XML*. Amsterdam, Niederlande: Addison-Wesley Longman, 1. Sep. 2003 (siehe S. 7-7, 7-31).
- [24] Elliotte Rusty Harold und W. Scott Means. *XML in a Nutshell. A Desktop Quick Reference*. 3. Aufl. Sebastopol, USA: O'Reilly, 2. Nov. 2004 (siehe S. 7-2, 7-5, 7-30).
- [25] Michael Horvath. *HSL color solid cylinder, alpha, low gamma*. 23. März 2010. URL: https://commons.wikimedia.org/wiki/File:HSL_color_solid_cylinder_alpha_lowgamma.png (besucht am 10. 11. 2025) (siehe S. 3-28).
- [26] Chris House. *CSS Grid Layout Guide*. 14. Nov. 2025. URL: <https://css-tricks.com/snippets/css/complete-guide-grid/> (besucht am 21. 11. 2025) (siehe S. 5-15, 5-33).
- [27] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754TM-2019. 13. Juni 2019. URL: <https://ieeexplore.ieee.org/document/8766229> (siehe S. 8-4).

- [28] International Organization for Standardization. *8-bit single-byte coded graphic character sets – Part 1: Latin alphabet No.1*. ISO/IEC 8859-1:1997. 11. Nov. 1997. URL: <https://www.open-std.org/JTC1/SC2/WG3/docs/n411.pdf> (siehe S. 2-3).
- [29] International Organization for Standardization. *W3C Accessibility Guidelines (WCAG) 3.0*. W3C Working Draft. Sep. 2025. URL: <https://www.iso.org/standard/91029.html> (siehe S. 6-6).
- [30] Khronos Group. *WebGL 2 Specification*. Version 2.0.0. 11. Apr. 2017. URL: <http://registry.khronos.org/webgl/specs/2.0/> (siehe S. 12-3).
- [31] Khronos Group. *WebGL Specification*. Version 1.0.3. 27. Okt. 2014. URL: <https://registry.khronos.org/webgl/specs/1.0/> (siehe S. 12-3).
- [32] Maria Korneeva. *Barrierefreie Webentwicklung. Von den Grundlagen über die rechtlichen Aspekte bis zur praktischen Umsetzung*. O'Reilly, 30. Mai 2025 (siehe S. 6-22).
- [33] Manuel Matuzović. *Web Accessibility Cookbook. Creating Inclusive Experiences*. O'Reilly, 23. Juli 2024 (siehe S. 6-22).
- [34] MDN Contributors. *An overview of accessible web applications and widgets*. 7. Juni 2025. URL: https://developer.mozilla.org/en-US/docs/Web/Accessibility/Guides/Accessible_web_applications_and_widgets (besucht am 28.11.2025) (siehe S. 6-18).
- [35] *meta tags and attributes that Google supports*. Google Search Central. 4. Aug. 2024. URL: <https://developers.google.com/search/docs/crawling-indexing/special-tags> (besucht am 03.11.2025) (siehe S. 2-2).
- [36] Eric Meyer. *CSS Tools: Reset CSS*. Jan. 2011. URL: <https://meyerweb.com/eric/tools/css/reset/> (besucht am 21.11.2025) (siehe S. 5-25).
- [37] Eric Meyer und Estelle Weyl. *CSS. The Definitive Guide*. 5. Aufl. Sebastopol, USA: O'Reilly, 30. Mai 2023 (siehe S. 3-40, 4-31).
- [38] Mozilla. *CSS Grid PlayGround*. 27. Jan. 2018. URL: <https://mozilladevelopers.github.io/playground/css-grid/> (besucht am 21.11.2025) (siehe S. 5-15, 5-33).
- [39] Mozilla Developer Network and individual contributors. *Mozilla Developer Network*. Jan. 2026. URL: <https://developer.mozilla.org/en-US/> (besucht am 29.01.2026) (siehe S. 2-34, 3-29, 10-23).
- [40] Peter Müller. *Einstieg in HTML und CSS. Webseiten programmieren und gestalten*. 3. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Sep. 2024 (siehe S. 1-16, 2-34, 3-41, 4-31, 5-33).
- [41] Theodore H. Nelson. *Computer Lib/Dream Machines*. 2. Aufl. Tempus Books of Microsoft Press, 1. Okt. 1987 (siehe S. 1-2).
- [42] Ingy döt Net u. a. *YAML Ain't Markup Language (YAML™) version 1.2*. Revision 1.2.2. 1. Okt. 2021. URL: <https://yaml.org/spec/1.2.2/> (siehe S. 7-25, 7-30, 7-31).
- [43] Mark Pilgrim. *HTML5. Up and Running*. Sebastopol, USA: O'Reilly, 6. Aug. 2010 (siehe S. 1-12, 12-5).

- [44] Sindre Sorhus. *modern-normalize*. Version 3.0.1. 5. Sep. 2024. URL: <https://github.com/sindresorhus/modern-normalize/> (siehe S. 5-27).
- [45] StatCounter. *Desktop Browser Market Share Worldwide*. Oct 2025. Okt. 2025. URL: <https://gs.statcounter.com/browser-market-share/desktop/worldwide/#monthly-202510-202510-bar> (besucht am 21. 10. 2025) (siehe S. 1-5, 1-6, 1-7).
- [46] The Sass Team. *Sass: Documentation*. Jan. 2026. URL: <https://sass-lang.com/documentation> (siehe S. 11-11, 11-23).
- [47] The Bootstrap Team. *Bootstrap Documentation*. Version 5.3.x. 8. Nov. 2025. URL: <https://getbootstrap.com/docs/> (siehe S. 5-16, 5-17, 5-21).
- [48] Thomas Theis. *Einstieg in JavaScript*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Juli 2024 (siehe S. 8-27, 9-20, 10-22).
- [49] Alexandre Van de Sande. *Color circle (hue-sat)*, trans. 21. Mai 2023. URL: [https://commons.wikimedia.org/wiki/File:Color_circle_\(hue-sat\).png](https://commons.wikimedia.org/wiki/File:Color_circle_(hue-sat).png) (besucht am 10. 11. 2025) (siehe S. 3-28).
- [50] Helmut Vonhoegen. *XML. Einstieg, Praxis, Referenz*. 10. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. Dez. 2023 (siehe S. 7-31).
- [51] W3C Web Accesibility Initiative. *Essential Components of Web Accessibility*. Hrsg. von Shawn Lawton Henry und Michael Duffy. 14. März 2024. URL: <https://www.w3.org/WAI/fundamentals/components/> (besucht am 28. 11. 2025) (siehe S. 6-4).
- [52] W3C Web Accesibility Initiative. *Introduction to Web Accessibility*. Hrsg. von Shawn Lawton Henry. 7. März 2024. URL: <https://www.w3.org/WAI/fundamentals/accessibility-intro/> (besucht am 28. 11. 2025) (siehe S. 6-1).
- [53] Web Hypertext Application Technology Working Group. *DOM*. Living Standard. 15. Jan. 2026. URL: <https://dom.spec.whatwg.org/> (siehe S. 10-3, 10-4, 10-14, 10-16).
- [54] Web Hypertext Application Technology Working Group. *HTML*. Living Standard. 23. Jan. 2026. URL: <https://html.spec.whatwg.org/multipage/> (siehe S. 1-10, 1-16, 2-8, 2-12, 2-22, 2-24, 2-26, 2-34, 3-21, 10-3, 10-4, 12-1, 12-2, 12-3, 12-8, 12-18, 12-19, 12-23).
- [55] Web Hypertext Application Technology Working Group. *URL*. Living Standard. 13. Jan. 2026. URL: <https://url.spec.whatwg.org/> (siehe S. 1-3).
- [56] Web Hypertext Application Technology Working Group. *WebSockets*. Living Standard. 12. Aug. 2025. URL: <https://websockets.spec.whatwg.org/> (siehe S. 12-22).
- [57] Christian Wenz und Christoph Prevezanos. *Jetzt lerne ich HTML5 und CSS3*. 2. Aufl. Burgthann, Deutschland: Markt + Technik Verlag, 16. Nov. 2022 (siehe S. 1-16, 2-34, 3-41).
- [58] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 1-16, 2-34, 3-41, 4-31, 5-33, 8-27, 9-20, 10-22, 11-23, 12-23).

- [59] World Wide Web Consortium. *Accessible Rich Internet Applications (WAI-ARIA) 1.2*. W3C Recommendation. 6. Juni 2023. URL: <https://www.w3.org/TR/wai-aria-1.2/> (siehe S. 6-14, 6-18, 6-22).
- [60] World Wide Web Consortium. *Accessible Rich Internet Applications (WAI-ARIA) 1.3*. W3C First Public Working Draft. 23. Jan. 2024. URL: <https://www.w3.org/TR/wai-aria-1.3/> (siehe S. 6-14).
- [61] World Wide Web Consortium. *Authoring Tool Accessibility Guidelines (ATAG) 2.0*. W3C Recommendation. 24. Sep. 2015. URL: <https://www.w3.org/TR/ATAG20/> (siehe S. 6-5).
- [62] World Wide Web Consortium. *Cascading Style Sheets Level 2 Revision 1 (CSS 2.1) Specification*. W3C Recommendation. Edited in place 12 April 2016 to point to new work, edited in place 07 December 2021 to correct markup. 7. Juni 2011. URL: <https://www.w3.org/TR/CSS2/> (siehe S. 3-2, 3-3, 3-27, 4-1, 4-31).
- [63] World Wide Web Consortium. *Cascading Style Sheets Level 2 Revision 2 (CSS 2.2) Specification*. W3C First Public Working Draft. 12. Apr. 2016. URL: <https://www.w3.org/TR/CSS22/> (siehe S. 3-2).
- [64] World Wide Web Consortium. *Cascading Style Sheets, level 1*. W3C Recommendation. Revised 11 Apr 2008, superseded on 13 September 2018. 17. Dez. 1996. URL: <https://www.w3.org/TR/CSS1/> (siehe S. 3-2).
- [65] World Wide Web Consortium. *CSS Animations Level 1*. W3C Working Draft. 2. März 2023. URL: <https://www.w3.org/TR/css-animations-1/> (siehe S. 3-3).
- [66] World Wide Web Consortium. *CSS Backgrounds and Borders Module Level 3*. W3C Candidate Recommendation Draft. 11. März 2024. URL: <https://www.w3.org/TR/css-backgrounds-3/> (siehe S. 3-2).
- [67] World Wide Web Consortium. *CSS Box Model Module Level 3*. W3C Recommendation. 11. Apr. 2024. URL: <https://www.w3.org/TR/css-box-3/> (siehe S. 3-2, 4-1).
- [68] World Wide Web Consortium. *CSS Box Sizing Module Level 3*. W3C Working Draft. 17. Dez. 2021. URL: <https://www.w3.org/TR/css-sizing-3/> (siehe S. 4-11).
- [69] World Wide Web Consortium. *CSS Cascading and Inheritance Level 5*. W3C Candidate Recommendation Snapshot. 13. Jan. 2022. URL: <https://www.w3.org/TR/css-cascade-5/> (siehe S. 3-3, 3-20, 5-31).
- [70] World Wide Web Consortium. *CSS Color Module Level 4*. W3C Candidate Recommendation Draft. 25. Apr. 2025. URL: <https://www.w3.org/TR/css-color-4/> (siehe S. 3-2, 3-27, 3-28).
- [71] World Wide Web Consortium. *CSS Flexible Box Layout Module Level 1*. W3C Candidate Recommendation Draft. 14. Okt. 2025. URL: <https://www.w3.org/TR/css-flexbox-1/> (siehe S. 5-1).
- [72] World Wide Web Consortium. *CSS Fonts Module Level 3*. W3C Recommendation. 20. Sep. 2018. URL: <https://www.w3.org/TR/css-fonts-3/> (siehe S. 3-3, 3-32).

- [73] World Wide Web Consortium. *CSS Functions and Mixins Module*. W3C First Public Working Draft. 15. Mai 2025. URL: <https://www.w3.org/TR/css-mixins-1/> (siehe S. 11-10).
- [74] World Wide Web Consortium. *CSS Grid Layout Module Level 1*. W3C Candidate Recommendation Draft. 26. März 2025. URL: <https://www.w3.org/TR/css-grid-1/> (siehe S. 5-7).
- [75] World Wide Web Consortium. *CSS Grid Layout Module Level 2*. W3C Candidate Recommendation Draft. 26. März 2025. URL: <https://www.w3.org/TR/css-grid-2/> (siehe S. 5-10).
- [76] World Wide Web Consortium. *CSS Images Module Level 3*. W3C Candidate Recommendation Draft. 18. Dez. 2023. URL: <https://www.w3.org/TR/css-image-s-3/> (siehe S. 3-3).
- [77] World Wide Web Consortium. *CSS Multi-column Layout Module Level 1*. W3C Candidate Recommendation Snapshot. 16. Mai 2024. URL: <https://www.w3.org/TR/css-multicol-1/> (siehe S. 3-3).
- [78] World Wide Web Consortium. *CSS Nesting Module Level 1*. W3C Working Draft. 22. Jan. 2026. URL: <https://www.w3.org/TR/css-nesting-1/> (siehe S. 11-9).
- [79] World Wide Web Consortium. *CSS Positioned Layout Module Level 3*. W3C Working Draft. 7. Okt. 2025. URL: <https://www.w3.org/TR/css-position-3/> (siehe S. 4-12).
- [80] World Wide Web Consortium. *CSS Snapshot 2025*. W3C Group Note. 18. Sep. 2025. URL: <https://www.w3.org/TR/css/> (siehe S. 3-2).
- [81] World Wide Web Consortium. *CSS Syntax Module Level 3*. W3C Candidate Recommendation Draft. 24. Dez. 2021. URL: <https://www.w3.org/TR/css-syntax-3/> (siehe S. 3-2).
- [82] World Wide Web Consortium. *CSS Text Module Level 3*. W3C Candidate Recommendation Draft. 30. Sep. 2024. URL: <https://www.w3.org/TR/css-text-3/> (siehe S. 3-3).
- [83] World Wide Web Consortium. *CSS Transforms Module Level 2*. W3C Working Draft. 9. Nov. 2021. URL: <https://www.w3.org/TR/css-transforms-2/> (siehe S. 3-29).
- [84] World Wide Web Consortium. *CSS Transitions*. W3C Working Draft. 11. Okt. 2018. URL: <https://www.w3.org/TR/css-transitions-1/> (siehe S. 3-3).
- [85] World Wide Web Consortium. *Document Object Model (DOM) Level 1 Specification Version 1.0*. W3C Recommendation. 1. Okt. 1998. URL: <https://www.w3.org/TR/REC-DOM-Level-1/> (siehe S. 10-2).
- [86] World Wide Web Consortium. *Document Object Model (DOM) Level 2 Core Specification Version 1.0*. W3C Recommendation. Superseded on 3 November 2020. 13. Nov. 2000. URL: <https://www.w3.org/TR/DOM-Level-2-Core/> (siehe S. 10-2).

- [87] World Wide Web Consortium. *Document Object Model (DOM) Level 2 Events Specification Version 1.0*. W3C Recommendation. Superseded on 3 November 2020. 13. Nov. 2000. URL: <https://www.w3.org/TR/DOM-Level-2-Events/> (siehe S. 10-2).
- [88] World Wide Web Consortium. *Document Object Model (DOM) Level 2 HTML Specification Version 1.0*. W3C Recommendation. Superseded on 3 November 2020. 9. Jan. 2003. URL: <https://www.w3.org/TR/DOM-Level-2-HTML/> (siehe S. 10-2).
- [89] World Wide Web Consortium. *Document Object Model (DOM) Level 2 Style Specification Version 1.0*. W3C Recommendation. Superseded on 3 November 2020. 13. Nov. 2000. URL: <https://www.w3.org/TR/DOM-Level-2-Style/> (siehe S. 10-2).
- [90] World Wide Web Consortium. *Document Object Model (DOM) Level 3 Core Specification Version 1.0*. W3C Recommendation. 7. Apr. 2004. URL: <https://www.w3.org/TR/DOM-Level-3-Core/> (siehe S. 10-2).
- [91] World Wide Web Consortium. *Extensible Markup Language (XML) 1.0 (Fifth Edition)*. W3C Recommendation. 26. Nov. 2008. URL: <https://www.w3.org/TR/xml/> (siehe S. 7-1, 7-7).
- [92] World Wide Web Consortium. *Extensible Markup Language (XML) 1.1 (Second Edition)*. W3C Recommendation. Edited in place 29 September 2006. 16. Aug. 2006. URL: <https://www.w3.org/TR/xml11/> (siehe S. 7-7).
- [93] World Wide Web Consortium. *File API*. W3C Working Draft. 3. Dez. 2025. URL: <https://www.w3.org/TR/FileAPI/> (siehe S. 12-22).
- [94] World Wide Web Consortium. *Geolocation*. W3C Recommendation. 29. Okt. 2025. URL: <https://www.w3.org/TR/geolocation/> (siehe S. 12-1, 12-15).
- [95] World Wide Web Consortium. *Indexed Database API 3.0*. W3C Working Draft. 13. Aug. 2025. URL: <https://www.w3.org/TR/IndexedDB/> (siehe S. 12-22).
- [96] World Wide Web Consortium. *Media Queries Level 3*. W3C Recommendation. 21. Mai 2024. URL: <https://www.w3.org/TR/mediaqueries-3/> (siehe S. 3-2, 4-22, 4-23, 4-25).
- [97] World Wide Web Consortium. *Media Queries Level 4*. W3C Candidate Recommendation Draft. 25. Dez. 2021. URL: <https://www.w3.org/TR/mediaqueries-4/> (siehe S. 4-22, 4-23, 4-25).
- [98] World Wide Web Consortium. *Media Queries Level 5*. W3C Working Draft. 18. Dez. 2021. URL: <https://www.w3.org/TR/mediaqueries-5/> (siehe S. 4-22, 4-23, 4-24, 4-25).
- [99] World Wide Web Consortium. *Pointer Events Level 2*. W3C Recommendation. 4. Apr. 2019. URL: <https://www.w3.org/TR/pointerevents2/> (siehe S. 10-14).
- [100] World Wide Web Consortium. *Scalable Vector Graphics (SVG) 1.1 (Second Edition)*. W3C Recommendation. 16. Aug. 2011. URL: <https://www.w3.org/TR/SVG11/> (siehe S. 2-21).

- [101] World Wide Web Consortium. *Selectors Level 3*. W3C Recommendation. 6. Nov. 2018. URL: <https://www.w3.org/TR/selectors-3/> (siehe S. 3-2).
- [102] World Wide Web Consortium. *Service Workers Nightly*. W3C Candidate Recommendation Draft. 26. Jan. 2026. URL: <https://www.w3.org/TR/service-workers/> (siehe S. 12-23).
- [103] World Wide Web Consortium. *Synchronization Accessibility User Requirements*. W3C Group Note. 28. Juni 2023. URL: <https://www.w3.org/TR/saur/> (siehe S. 6-3).
- [104] World Wide Web Consortium. *UI Events*. W3C Working Draft. 7. Sep. 2024. URL: <https://www.w3.org/TR/uievents/> (siehe S. 10-14).
- [105] World Wide Web Consortium. *User Agent Accessibility Guidelines (UAAG) 2.0*. W3C Working Group Note. 15. Dez. 2015. URL: <https://www.w3.org/TR/UAAG20/> (siehe S. 6-5).
- [106] World Wide Web Consortium. *W3C Accessibility Guidelines (WCAG) 3.0*. W3C Working Draft. 4. Sep. 2025. URL: <https://www.w3.org/TR/wcag-3.0/> (siehe S. 6-6).
- [107] World Wide Web Consortium. *Web Accessibility Initiative*. 2025. URL: <https://www.w3.org/WAI/> (besucht am 28. 11. 2025) (siehe S. 6-2).
- [108] World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. 24. Dez. 2024. URL: <https://www.w3.org/TR/WCAG22/> (siehe S. 6-6, 6-7, 6-22).
- [109] World Wide Web Consortium. *Website Accessibility Conformance Evaluation Methodology (WCAG-EM) 1.0*. W3C Working Group Note. 10. Juli 2014. URL: <https://www.w3.org/TR/WCAG-EM/> (siehe S. 6-11, 6-13).
- [110] World Wide Web Consortium. *WOFF File Format 2.0*. W3C Recommendation. 8. Aug. 2024. URL: <https://www.w3.org/TR/WOFF2/> (siehe S. 3-33).
- [111] World Wide Web Consortium. *XML Schema Part 1: Structures Second Edition*. W3C Recommendation. 28. Okt. 2004. URL: <https://www.w3.org/TR/xmlschema-1/> (siehe S. 7-13).
- [112] World Wide Web Consortium. *XR Accessibility User Requirements*. W3C Working Group Note. 25. Aug. 2021. URL: <https://www.w3.org/TR/xaur/> (siehe S. 6-3).
- [113] François Yergeau. *UTF-8, a transformation format of ISO 10646*. RFC 3629. IETF, Nov. 2003, S. 1–14. URL: <https://datatracker.ietf.org/doc/rfc3629/> (siehe S. 2-4).
- [114] Juriy Zaytsev, Leon Arnott und Denis Pushkarev. *ECMAScript compatibility table*. 11. Nov. 2025. URL: <https://compat-table.github.io/compat-table/> (besucht am 12. 12. 2025) (siehe S. 8-3, 8-10).